

HỌC SÂU CHO VISUAL SLAM



TÓM TẮT

Phần F đặt toàn bộ năm phần trước vào một câu hỏi duy nhất: có nên thay một khối trong hệ SLAM cổ điển bằng một mạng, và nếu có thì khối nào. Tám chương đi từ khung quyết định, qua frontend học được, đóng vòng, độ sâu và dòng quang, backend khả vi, bản đồ neural, ngữ nghĩa, tới IMU và triển khai. Mỗi phương pháp được hỏi đúng bốn câu: nó chữa thất bại cụ thể nào, cơ chế nào làm được, có đáng cái giá không, và nếu không dùng thì học được gì từ thiết kế của nó. Nếu chỉ nhớ một điều: phần lớn công trình trong phần này chưa đủ chín để thay thế một hệ cổ điển đã tinh chỉnh kỹ, nhưng vài ý tưởng trong đó chữa đúng những chỗ mà hình học thuần túy không chữa nổi.

Năm phần trước dạy học sâu như một môn học. Phần này dùng nó như một hộp công cụ, cho một hệ thống cụ thể đã tồn tại và đang đau ở những chỗ cụ thể.

Cách đặt vấn đề vì thế đảo ngược so với phần lớn tài liệu về chủ đề này. Tài liệu thường bắt đầu từ phương pháp rồi đi tìm chỗ dùng. Ở đây ta bắt đầu từ bốn thất bại có thật của một hệ stereo-inertial đã tinh chỉnh kỹ — **chuyển động nhanh, ảnh mờ do chuyển động, đóng vòng không nổ, và đặt trọng số cho IMU** — rồi hỏi ngược lại xem có công trình nào thật sự chạm tới chúng.

Câu trả lời trung thực là: có, nhưng ít hơn nhiều so với ấn tượng mà số lượng paper tạo ra. Rất nhiều kết quả ấn tượng trong phần này đến từ chuỗi ngắn, cảnh trong nhà, và máy để bàn có GPU rời. Một hệ chạy trên Jetson Orin, ngoài trời, thời gian thực, với ngân sách vài chục mili-giây mỗi khung, sống trong một thế giới khác. Chương 27 dựng khung để phân biệt hai thế giới đó trước khi đọc bất cứ chương nào sau.

Một lưu ý về giọng của phần này. Nó phê phán nhiều hơn năm phần trước, và điều đó là cố ý. Với một người chưa có hệ thống, học sâu là con đường nhanh nhất để có một hệ chạy được. Với một người đã có một hệ chạy tốt, phần lớn phương pháp trong đây là một cuộc đánh đổi đắt: đổi tính chẵn đoán được, tính lặp lại và ngân sách tính toán lấy một cái thiện chỉ hiện ra trên một số cảnh. Biết trước điều đó không làm ta bỏ lỡ cơ hội; nó làm ta tiêu đúng ba tháng vào đúng một ý tưởng thay vì rải mỏng vào tám ý tưởng.

Bảng 1. Bản đồ Phần F: các chương, nội dung, và nỗi đau SLAM mà chương đó chạm tới.

Ch.	Thư mục	Nội dung	Chạm nỗi đau
27	27-dat-hoc-sau-vao-da	Năm trụ, khung bốn cổng để quyết định có nên thay một khối cổ điển bằng mạng	cả bốn (bộ lọc)
28	28-dac-trung-va-ghep-	Đặc trưng và ghép cặp học được; vì sao ORB gây và cái gì thay được	chuyển động nhanh, ảnh mờ
29	29-dinh-vi-lai-va-don	Nhận dạng lại địa điểm, định vị lại, và bốn tầng của đóng vòng	đóng vòng
30	30-do-sau-dong-quang-	Độ sâu đơn ảnh, dòng quang, và mô hình nền hình học	chuyển động nhanh, tỉ lệ
31	31-backend-kha-vi-va-	Tối ưu khả vi và hệ đầu-cuối; khả vi mua được gì, mất gì	backend
32	32-ban-do-neural	NeRF và 3D Gaussian Splatting làm bản đồ; đẹp ảnh không bằng đúng pose	biểu diễn bản đồ
33	33-ngu-nghia-va-do-th	Ngữ nghĩa, vật thể động, đồ thị cảnh 3D	cảnh động
34	34-imu-bat-dinh-va-tr	Trọng số IMU, bất định học được, ảnh mờ, và đưa lên Orin	trọng số IMU, ảnh mờ

Đọc chương 27 trước, luôn luôn: nó là bộ lọc quyết định bảy chương còn lại chương nào đáng đọc kỹ với bạn. Bảy chương sau đọc lập với nhau, chọn theo nỗi đau đang gặp chứ không đọc tuần tự.

Đặt học sâu vào đâu trong một hệ SLAM (Where Learning Fits in a SLAM System)

PHỤ THUỘC B/03-giai-phau-he-thong (đọc một hệ thống bằng luồng dữ liệu và ranh giới); E/24-thi-nghiem-va-ablation (sản nhiễu và oracle experiment — chương này dùng lại cả hai); B/11-gioi-han-tinh-toan cùng D/21-inference-va-trien-khai cho công ngân sách; B/08-dich-chuyen-phan-phoi cho công thứ tư.

TÓM TẮT

Chương này không dạy một phương pháp nào. Nó dựng bộ lọc để bạn quyết định phương pháp nào đáng đọc tiếp. Bạn sẽ có bốn công cụ: năm chỗ có thể cắm mạng vào một hệ SLAM cổ điển, một khung bốn cổng để loại sớm, một bản đồ từ bốn nỗi đau sang nhóm phương pháp, và một quy trình đọc paper trong ba mươi phút. Đọc xong bạn phát biểu được câu “chưa đáng” kèm con số thay vì kèm cảm giác, và biết đầu tư ba tháng tới vào đúng một chỗ. Nếu chỉ nhớ một điều: đo trần đóng góp của một khối trước, rồi mới đi tìm mạng để thay khối đó — và ưu tiên những chỗ mà khi hỏng thì hệ thống hỏng *ồn ào*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình học sâu** — một hàm có rất nhiều tham số, được đi tìm bằng cách cho xem nhiều ví dụ có đáp án. Trong chương này nó luôn xuất hiện như một *khối* nhận ảnh và trả về một đại lượng: điểm đặc trưng, độ sâu, mặt nạ, hoặc một pose.
- **suy luận (inference)** — một lần chạy mô hình đã huấn luyện trên một đầu vào mới. SLAM chạy suy luận trên từng khung hình, mỗi lần đúng một ảnh, nên nó bị chặn bởi *độ trễ* chứ không phải bởi *thông lượng*.
- **batch** — số mẫu được đưa qua mạng cùng lúc. GPU đạt hiệu suất cao khi batch lớn; SLAM thời gian thực buộc batch bằng 1, nên phần lớn con số FPS trong paper không áp dụng được cho bạn.
- **phân phối huấn luyện** — tập các tình huống mà mô hình đã được xem trong lúc huấn luyện. “Ra ngoài phân phối” nghĩa là gặp tình huống khác hẳn — đêm tối, mưa, hành lang không vãn, ảnh mờ — và đó là chỗ mọi bảo đảm về độ chính xác biến mất.
- **ma trận thông tin** — nghịch đảo của ma trận hiệp phương sai; trong một đồ thị nhân tử nó chính là *trọng số* của một phép đo. Đặt nó quá lớn nghĩa là bảo bộ tối ưu “hãy tin phép đo này”, và đó là cách hỏng âm thầm nhất của một hệ visual-inertial.
- **APE và RPE** — APE là sai số tuyệt đối của quỹ đạo sau khi căn chỉnh với chân trị; RPE là sai số tương đối trên các đoạn ngắn. APE nhạy với trôi tích lũy và với vòng đóng; RPE nhạy với chất lượng odometry cục bộ.
- **tỉ lệ hoàn thành chuỗi** — phần trăm khung hình mà hệ thống còn bám được. Nó phải luôn được báo cùng APE, vì một hệ mất bám giữa chừng chỉ còn được chấm trên đoạn dễ và vì thế có APE rất đẹp.
- **trần đóng góp (oracle ceiling)** — mức cải thiện tối đa mà một khối có thể mang lại, đo bằng cách thay khối đó bằng chân trị rồi chạy cả hệ. Nếu một bộ ghép cặp hoàn hảo chỉ giảm APE được một lượng c , thì mọi bộ ghép cặp học được cũng bị chặn trên bởi đúng c . Giá trị của c là thứ bạn phải tự đo trên hệ của mình; không mượn được của ai và không đọc được từ bảng của ai.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn thay DBoW bằng một mô hình nhận dạng địa điểm hiện đại. Recall@1 tăng vọt, nhưng số vòng đồng thành công không đổi một cái nào. Chuyện gì đã xảy ra, và lẽ ra bạn phải đo gì trước?
- Q2. Một paper báo APE thấp hơn ORB-SLAM3 nhiều lần. Ba câu hỏi nào phải hỏi trước khi tin, và câu nào loại được nhiều paper nhất chỉ trong ba phút?
- Q3. Vì sao một phép đo sai mà “tự tin” lại nguy hiểm hơn hẳn việc không có phép đo nào — trong khi với một bộ phân lớp ảnh thì hai chuyện đó gần như tương đương?
- Q4. Một paper báo mạng chạy rất nhanh trên một GPU để bàn. Vì sao con số đó gần như không nói gì về việc nó có chạy nổi 20 Hz trên Jetson Orin hay không?
- Q5. Bạn có ba tháng và bốn nỗi đau. Phép đo nào cho bạn quyền tiêu cả ba tháng vào đúng một nỗi đau, mà không cần đọc thêm một paper nào?

1.1 Đặt vấn đề

Bạn đang bảo trì một nhánh ORB-SLAM3 stereo-inertial [1]. Nó chạy EuRoC [2], TUM-VI [3] và OpenLORIS, triển khai trên Jetson Orin, và đã được tinh chỉnh nhiều tháng. Trên phần lớn chuỗi EuRoC nó chạy trơn chuỗi và cho một quỹ đạo mà bạn đã đo đi đo lại nhiều lần, nên bạn biết rất rõ mức sai số của nó — biết bằng số đo của chính mình chứ không bằng bảng của ai. Mỗi tuần lại có một paper nói rằng một mạng nào đó làm tốt hơn.

Câu hỏi thật không phải “học sâu có tốt hơn hình học không”. Câu hỏi đó vô nghĩa vì không có phép đo nào trả lời được nó. Câu hỏi thật gồm bốn phần, và phải hỏi theo đúng thứ tự: khối nào trong hệ của tôi đang thật sự làm hỏng việc, trên chuỗi nào, với ngân sách bao nhiêu mili-giây, và khi nó hỏng thì tôi có biết không.

Có một sự bất đối xứng khiến việc này khó hơn vẻ ngoài. Paper được viết để cho thấy nơi phương pháp *thắng*. Bạn cần biết nơi nó *thua*. Không phải vì tác giả không trung thực, mà vì cấu trúc khuyến khích của việc công bố không thưởng cho một bảng liệt kê các cảnh mà phương pháp gây. Hệ quả là bạn phải tự dựng phần thiếu đó.

Còn một bất đối xứng nữa, nặng hơn. Baseline trong phần lớn paper là ORB-SLAM3 chạy cấu hình mặc định. Baseline của bạn là ORB-SLAM3 đã được chính bạn tinh chỉnh cho cảm biến của mình. Khoảng cách giữa hai thứ đó không nhỏ. Một phần đáng kể của “cải thiện nhiều lần” trong bảng có thể chỉ là khoảng cách tinh chỉnh, chứ không phải đóng góp của phương pháp.

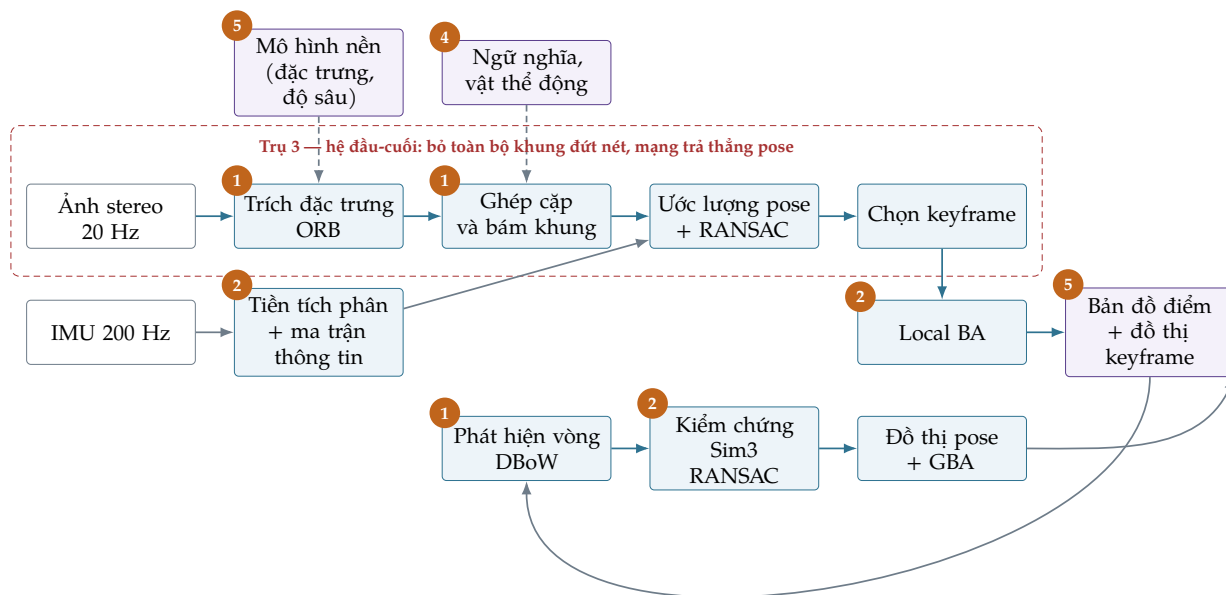
↪ **tiền đề** E/24 — đây chính là ràng buộc “baseline mạnh là nghĩa vụ” của chương 24, lần này nhìn từ phía người đọc bằng chứ không phải người làm bảng

Chương này vì thế là một bộ lọc, không phải một khảo sát. Nó có bốn phần và bốn phần đó dùng được độc lập. Phần đầu nói *có thể* cắm mạng vào đâu, tức năm trụ. Phần hai nói *có nên* cắm không, tức khung bốn cổng. Phần ba nói bốn nỗi đau cụ thể của bạn sang nhóm phương pháp nào hứa chữa và chương nào nói về nó. Phần bốn nói *cắm sâu tới mức nào*, tức ba mức can thiệp. Phần cuối là kỹ năng thao tác: đọc một paper trong ba mươi phút và biết loại nó lúc nào.

Một lời cảnh báo về giọng. Chương này sẽ kết luận “chưa đáng” nhiều lần. Đó không phải thái độ bảo thủ mà là hệ quả trực tiếp của việc bạn đã có một hệ thống chạy tốt. Với người chưa có hệ thống nào, một hệ đầu-cuối học được là con đường nhanh nhất để có thứ chạy được. Với người đã có, phần lớn phương pháp trong Phần F là một cuộc đổi chác đất: đổi tính chẩn đoán được, tính lặp lại và ngân sách tính toán, lấy một cải thiện chỉ xuất hiện trên vài cảnh.

1.2 Năm trụ: học sâu cắm được vào đâu (The Five Pillars)

Trước khi hỏi “có nên”, phải biết “ở đâu”. Một hệ SLAM cổ điển không phải một khối. Nó là một chuỗi khối nối nhau bằng những giao diện rất rõ ràng, và chính các giao diện đó mới là chỗ một mạng có thể chen vào. Hình 1.1 vẽ hệ thống đó cùng năm chỗ cắm.



Hình 1.1. Một hệ stereo-inertial cổ điển và năm chỗ có thể cắm mạng vào. Trụ 1 đổi cách *sinh ra* phép đo (đặc trưng, ghép cặp, lấy ứng viên vòng). Trụ 2 đổi cách *cân* phép đo trong bộ tối ưu. Trụ 3 xóa cả khung đứt nét đỏ và thay bằng một mạng trả pose. Trụ 4 đổi *phép đo nào được phép vào*. Trụ 5 đổi *bản đồ là cái gì*, hoặc cấp một tiên nghiệm học sẵn cho các khối khác. Đi từ 1 tới 3 là đi từ ngoài vào trong: càng vào sâu, càng ít đại lượng trung gian mà bạn còn kiểm tra được bằng một kỳ vọng vật lý.

A. Trụ 1 – frontend nhận thức

Trụ này thay cách *sinh ra phép đo*. ORB là một bộ dò góc cộng một chuỗi bit so sánh cường độ điểm ảnh. Thay nó bằng một mạng nghĩa là: ảnh vào, danh sách điểm ảnh cùng vectơ mô tả ra. Ghép cặp cũng vậy, và tầng lấy ứng viên vòng cũng vậy. SuperPoint [4] là ví dụ điển hình cho bộ dò và mô tả; SuperGlue [5] và LightGlue [6] cho bước ghép cặp; LoFTR [7] bỏ hẳn bước dò điểm.

Điều quan trọng về mặt kiến trúc là giao diện không đổi. Backend vẫn nhận đúng thứ nó vẫn nhận: các tương ứng hai chiều kèm một ước lượng độ bất định. Vì thế RANSAC, hiệu chỉnh chùm tia, đồ thị nhân tử, tất cả giữ nguyên. Đây là trụ có rủi ro thấp nhất và cũng là trụ đã chín nhất, và chương F/28 dành trọn cho nó.

B. Trụ 2 – backend tối ưu

Trụ này không đổi phép đo, nó đổi *trọng số* của phép đo. Trong một đồ thị nhân tử, mỗi cạnh có một ma trận thông tin. Với IMU, ma trận đó đến từ mô hình nhiễu của cảm biến qua bước tiên tích phân [8]. Với điểm ảnh, nó thường là một hằng số nhân với hệ số tầng kim tự tháp. Cả hai đều là giả định, và cả hai đều sai ở mức nào đó.

Học sâu vào đây theo ba đường. Thứ nhất, học độ bất định của từng phép đo thay vì dùng hằng số. Thứ hai, học mô hình nhiễu của IMU, gồm cả trôi bias và nhiễu phụ thuộc chuyển động. Thứ ba, làm cho cả bộ tối ưu khả vi để huấn luyện xuyên qua nó. Hai đường đầu nằm ở chương F/34,

đường thứ ba ở chương F/31. Đây là trụ ít được khai thác nhất và, theo đánh giá của chương này, là trụ có tỉ số lợi ích trên rủi ro tốt nhất cho một hệ đã chạy.

C. Trụ 3 — hệ đầu-cuối

Trụ này xóa cả chuỗi. Ảnh vào, pose ra, không có bước nào ở giữa mà bạn đặt tên được bằng một khái niệm hình học. DROID-SLAM [9] là mốc quan trọng nhất của dòng này; nó vẫn giữ một lối tối ưu nhưng toàn bộ phần sinh tương ứng là học được. Các hệ mới hơn đi xa hơn nữa, đưa cả bước dựng hình vào một mô hình nền duy nhất.

Cái được là tính bền: những hệ này thường không mất bám ở chỗ mà hệ đặc trưng thừa mất bám. Cái mất là toàn bộ bề mặt chẩn đoán. Không còn số inlier để nhìn, không còn hiệp phương sai để hỏi, không còn cách nào trả lời câu “vì sao khung 1842 sai”. Với một hệ sản phẩm, đó là cái giá rất khó trả. Chương F/31 nói kỹ.

D. Trụ 4 — hiểu cảnh

Trụ này không ước lượng pose và không sinh phép đo. Nó quyết định *phép đo nào được phép vào*. Ví dụ điển hình là loại bỏ điểm nằm trên vật thể chuyển động: DynaSLAM [10] dùng mặt nạ ngữ nghĩa để vớt các điểm rơi trên người và xe trước khi chúng vào bộ tối ưu. Ở mức cao hơn, đồ thị cảnh 3D như Kimera [11] dựng một tầng ngữ nghĩa nằm *cạnh* bộ ước lượng hình học chứ không thay nó.

Đặc điểm cấu trúc của trụ này rất đáng chú ý: nó chỉ *trừ đi* chứ không cộng thêm. Nếu mạng chết, hệ thống trở về đúng hành vi cổ điển. Đó là lý do nó thuộc mức can thiệp thấp nhất, dù mô hình bên trong có thể rất lớn. Chương F/33 nói kỹ.

E. Trụ 5 — mô hình nền và biểu diễn neural

Trụ này gồm hai thứ hay bị gộp làm một, và tách chúng ra là việc đầu tiên phải làm.

- **Mô hình nền hình học** — một mạng lớn huấn luyện trên rất nhiều dữ liệu, trả về độ sâu, đặc trưng, hoặc trực tiếp một đám mây điểm đã căn chỉnh. DUST3R [12] là mốc; MAST3R-SLAM [13] là một nỗ lực đưa dòng đó vào SLAM thời gian thực. Chúng cấp *tiền nghiệm* cho các khối khác.
- **Biểu diễn bản đồ neural** — NeRF [14] và 3D Gaussian Splatting [15] đổi câu trả lời cho câu hỏi “bản đồ là cái gì”. Thay vì một đám mây điểm thưa, bản đồ thành một trường liên tục hoặc một tập splat. MonoGS [16] là ví dụ đưa nó vào một vòng SLAM.

Hai thứ đó phục vụ hai mục đích khác nhau. Mô hình nền nhắm vào độ bền của phép đo. Biểu diễn neural nhắm vào chất lượng bản đồ dựng lại, mà chất lượng đó *không* đồng nghĩa với độ chính xác quỹ đạo. Trộn hai mục tiêu là nguồn hiểu nhầm lớn nhất khi đọc mẩu này, và chương F/32 mở đầu bằng đúng việc tách chúng.

Bảng 1.1. Năm trụ đọc theo cùng bốn tiêu chí. Cột cuối là cột đáng giá nhất: nó cho biết khi nào lợi ích hứa hẹn của trụ đó biến mất.

Trụ	Cơ chế	Giả định	Hỏng khi nào
1. Frontend nhận thức	Mạng sinh điểm, mô tả, tương ứng, hoặc ứng viên vòng; giao diện với backend không đổi	Cảnh triển khai nằm trong phân phối huấn luyện; có đủ ngân sách độ trễ	Cảnh lạ về kết cấu hoặc ánh sáng; mạng vẫn trả đủ điểm nhưng mô tả không còn phân biệt được
2. Backend tối ưu	Học trọng số, hiệp phương sai, hoặc mô hình nhiễu IMU; cấu trúc đồ thị nhân tử giữ nguyên	Độ bất định thật sự phụ thuộc dữ liệu quan sát được, không chỉ phụ thuộc cảm biến	Trọng số học được quá tự tin; bộ tối ưu tin nhầm một cạnh sai và không có tín hiệu nào báo
3. Hệ đầu-cuối	Mạng nhận ảnh và trả pose hoặc trả trực tiếp quỹ đạo; xoá các bước hình học tường minh	Chuyển động và cảnh khi triển khai giống lúc huấn luyện; tài nguyên GPU dồi dào	Chuỗi dài hơn nhiều so với lúc huấn luyện; bộ nhớ tăng theo số khung; không còn đại lượng trung gian để chẩn đoán
4. Hiểu cảnh	Mạng quyết định phép đo nào bị loại, hoặc dựng một tầng ngữ nghĩa cảnh bản đồ hình học	Vật thể động nhận dạng được bằng loại của nó; loại bỏ nhầm ít hơn giữ lại nhầm	Vật động không thuộc lớp đã học (rèm cửa, bóng, phản chiếu); cảnh mà phần lớn điểm nằm trên vật động
5. Mô hình nền, bản đồ neural	Tiên nghiệm học sẵn cho độ sâu và đặc trưng, hoặc đối hẩn cấu trúc bản đồ	Tiên nghiệm đủ đúng để không kéo lệch hình học; đủ bộ nhớ giữ biểu diễn	Sai tỉ lệ tích lũy; chuỗi dài ngoài trời; ảnh dựng lại đẹp trong khi quỹ đạo tệ đi

F. Vì sao thứ tự năm trụ lại chính là thứ tự lịch sử

Năm trụ không phải một danh sách phẳng. Chúng xếp theo độ sâu của can thiệp, và độ sâu đó gần như trùng với thứ tự mà cộng đồng đã đi qua. Mạch dưới đây giải thích vì sao.

I Mạch 1 — vì sao học sâu vào SLAM từ frontend trước

- Vấn đề** Hệ cổ điển đo cảnh bằng đặc trưng do người thiết kế. ORB rẻ, bất biến với xoay trong mặt phẳng ảnh, và chạy được trên CPU. Nhưng nó giả định ảnh sắc nét, có vân, và độ sáng ổn định.

Giải pháp Thay bộ dò và bộ mô tả bằng một mạng, huấn luyện trên các cặp ảnh đã biết tương ứng.

Giả định Cảnh triển khai giống cảnh huấn luyện, và có GPU rảnh để chạy suy luận mỗi khung.

Giá phải trả thêm độ trễ, thêm bộ nhớ, thêm một phụ thuộc phần cứng.
- Vấn đề** Đầu ra vẫn là điểm và mô tả, nên backend không phải sửa dòng nào. Đó chính là điều làm bước này hấp dẫn: rủi ro tích hợp thấp.

Giải pháp Giữ nguyên toàn bộ phần hình học, chỉ đổi nguồn phép đo.

Vấn đề mới Bạn mất một tín hiệu cũ rất quý. ORB không tìm được góc nào thì trả về danh sách rỗng, và hệ thống biết ngay là đang mù. Một mạng gần như luôn trả đủ số điểm bạn yêu cầu, kể cả trên tường trắng.
- Vấn đề** Nếu phép đo đã học được, vì sao còn cân chúng bằng hằng số do người đặt? Ma trận thông tin của cảnh ảnh trong phần lớn hệ thống là một hằng số nhân hệ số tăng.

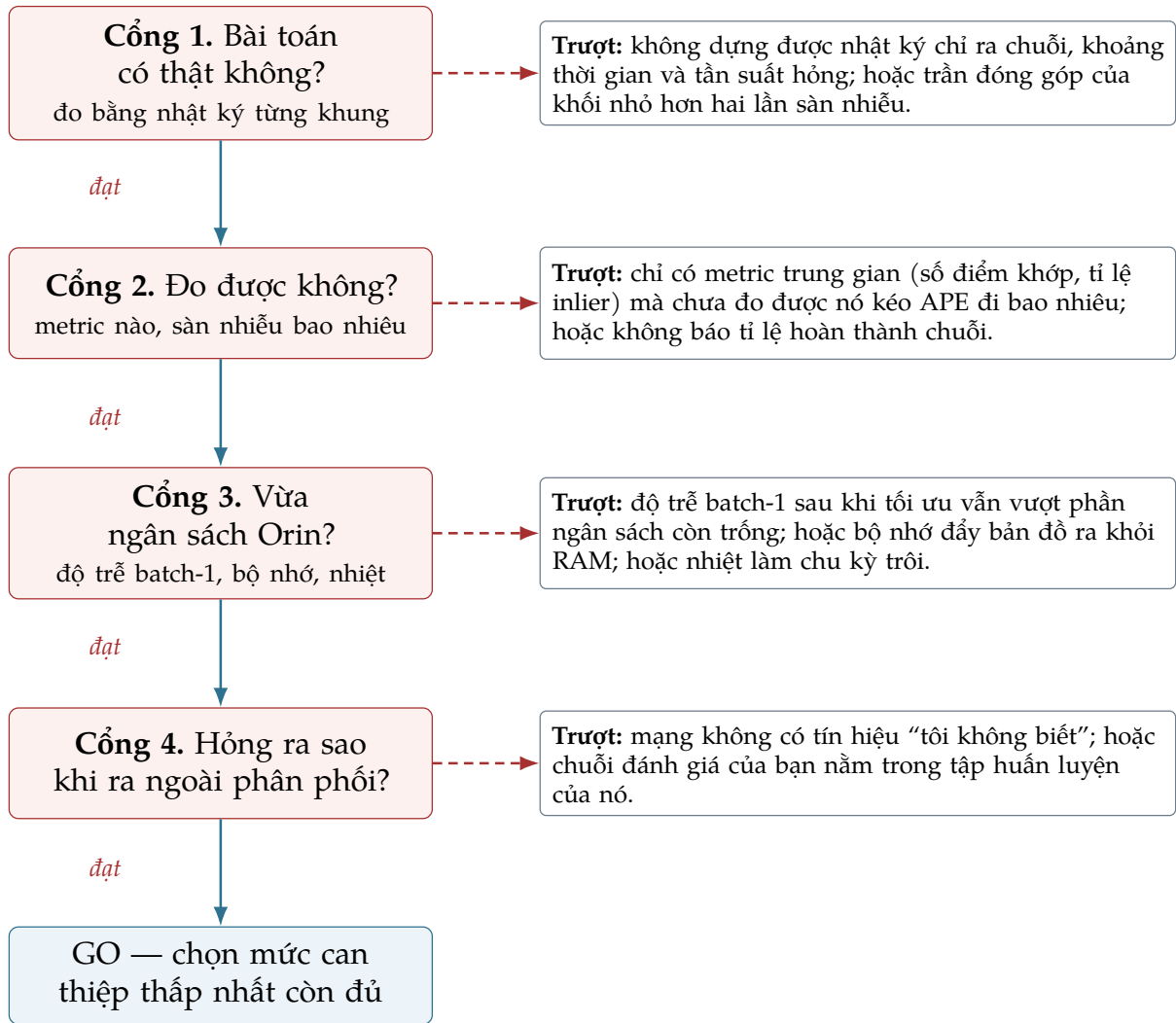
Giải pháp Học độ bất định của từng phép đo, hoặc học mô hình nhiễu IMU.

Giải định	Có chân trị đủ tốt để dạy mạng biết khi nào phép đo của chính nó sai.
Hệ quả	Đây là trụ 2. Nó khó hơn trụ 1 vì nhãn khó kiểm, nhưng nó nằm hoàn toàn bên trong bộ ước lượng cổ điển, nên không phá vỡ gì.
4 Vấn đề	Cả hai bước trên vẫn bị chặn bởi thiết kế của hệ cổ điển: đặc trưng thưa, tương ứng rời rạc, keyframe rời rạc.
Giải pháp	Làm cho cả chuỗi khả vi rồi huấn luyện đầu-cuối.
Giá phải trả	mất bề mặt chẩn đoán, mất khả năng quay lại, và mất luôn nhiều năm tính chính đã nằm trong các ngưỡng của hệ cũ.
Vấn đề mới	Khi hệ đầu-cuối sai, bạn không còn đại lượng trung gian nào để hỏi. Cách sửa duy nhất là huấn luyện lại, và đó là vòng lặp tính bằng tuần chứ không bằng phút.

Mạch này cho một quy tắc đọc rất hữu dụng. Khi gặp một phương pháp mới, hãy hỏi ngay: nó đang ở trụ nào? Câu trả lời cho biết bạn phải xoá cái gì trong hệ hiện tại để nhận nó, và đó thường là thông tin quyết định, chứ không phải con số APE trong bảng.

1.3 Khung bốn cổng (The Four-Gate Decision Framework)

Biết một phương pháp nằm ở trụ nào chưa đủ để quyết định. Cần một quy trình loại bỏ, và quy trình đó phải rẻ. Bốn cổng dưới đây được xếp theo chi phí tăng dần: cổng 1 tốn vài ngày đo trên hệ của bạn, cổng 4 tốn nhiều nhất và vì thế đứng cuối. Hình 1.2 là sơ đồ đầy đủ, kèm lối ra ở mỗi cổng.



Hình 1.2. Bốn cổng xếp theo chi phí tăng dần, mỗi cổng có tiêu chí trượt cụ thể. Phần lớn ứng viên chết ở cổng 1 và cổng 3, và cả hai cổng đó đều trả lời được bằng phép đo trên hệ của chính bạn, không cần chạy phương pháp mới lần nào.

A. Cổng 1 – bài toán có thật không, và đo bằng gì

Đây là cổng loại được nhiều ứng viên nhất, và cũng là cổng hay bị bỏ qua nhất. Lý do bỏ qua rất người: một paper mô tả rất sống động một thất bại, bạn nhận ra hệ mình cũng có thất bại đó, và thế là “vấn đề đã được xác nhận”. Nhưng nhận ra không phải là đo.

Cổng này đòi hai thứ. Thứ nhất là **nhật ký từng khung**: chuỗi nào, khoảng thời gian nào, triệu chứng gì, tần suất bao nhiêu. Nếu bạn nói “hệ hay mất bám khi quay nhanh” mà không nói được nó xảy ra ở bao nhiêu khung trên tổng số bao nhiêu, bạn chưa qua cổng. Thứ hai là **trần đóng góp**: thay khối bị nghi bằng chân trị rồi đo lại cả hệ.

Bước thứ hai là bước quyết định, và nó rõ hơn về ngoài. Muốn biết trần của một bộ ghép cặp tốt hơn, không cần bộ ghép cặp nào. Dùng pose chân trị để chiếu ngược và loại mọi tương ứng sai, rồi chạy lại. Con số thu được là mức tốt nhất mà *bất kỳ* bộ ghép cặp nào cũng không vượt qua được. Gọi nó là c : mọi paper hứa hẹn về ghép cặp đều bị chặn trên bởi đúng c , bất kể bảng của họ đẹp tới đâu. Và c chỉ có nghĩa khi nó là số bạn tự đo trên chuỗi của mình.

→ **trường hợp riêng của** E/24 — oracle experiment ở chương 24, áp cho một hệ SLAM: nó biến câu hỏi “nên đầu tư vào đâu” thành một phép đo chạy trong một ngày

Có một điều kiện trượt thứ ba, ít trực giác hơn nhưng rất hay xảy ra: thất bại giải thích được bằng một lỗi cấu hình. Trước khi kết luận rằng frontend yếu, hãy kiểm tra ngưỡng, số tầng kim tự tháp, chính sách chọn keyframe, và thời gian “vừa mới định vị lại”. Sửa những thứ đó không tốn gì cả, và chúng giải thích một phần lớn các thất bại nghe có vẻ cần học sâu.

B. Cổng 2 — đo được không, metric nào, sần nhiều bao nhiêu

Giả sử vấn đề có thật. Câu tiếp theo: bạn sẽ nhận ra cải thiện bằng cách nào? Cổng này có ba yêu cầu.

- **Metric đầu cuối, không phải metric trung gian.** Số điểm khớp, tỉ lệ inlier, độ chính xác ghép cặp đều là đại lượng trung gian. Chúng có thể tăng mà APE không nhúc nhích, vì bộ tối ưu vốn đã loại được phần lớn phép đo xấu. Nếu bạn định dùng metric trung gian thì phải đo trước mỗi liên hệ của nó với APE trên chính hệ của bạn.
- **Tỉ lệ hoàn thành chuỗi báo cùng APE.** Đây là cái bẫy đặc thù của SLAM. Một hệ mất bám giữa chừng chỉ được chấm trên đoạn nó còn sống, mà đoạn đầu của một chuỗi hầu như luôn là phần dễ. APE của nó vì thế đẹp một cách vô nghĩa, và càng mất bám sớm thì trông càng đẹp.
- **Sần nhiều đo trước.** Nhánh của bạn phi tất định: thứ tự duyệt các container đánh chỉ mục bằng con trỏ và tranh chấp giữa luồng bám và luồng dựng bản đồ đều làm kết quả đổi giữa các lần chạy. Vì thế mọi so sánh phải chạy nhiều lần, và “nhiều” là một con số tính ra được từ độ lệch chuẩn giữa các lần chạy.

Tiêu chí trượt của cổng này rất gọn: nếu mức cải thiện mà phương pháp hứa nhỏ hơn sần nhiều bạn đã đo, thì bạn không có cách nào xác nhận nó, và việc tích hợp là một canh bạc chứ không phải một quyết định kỹ thuật.

Ví dụ tính tay, chạy trong đầu được. Mọi con số dưới đây là số *giả định* để thấy hình dạng của công thức, không phải số đo của một hệ nào. Giả sử bạn đo sần nhiều và được $\sigma = 0,6$ cm APE giữa các lần chạy. Bạn muốn phát hiện một cải thiện thật $\Delta = 1,0$ cm. Công thức cỡ mẫu quen thuộc cho số lần chạy mỗi cấu hình:

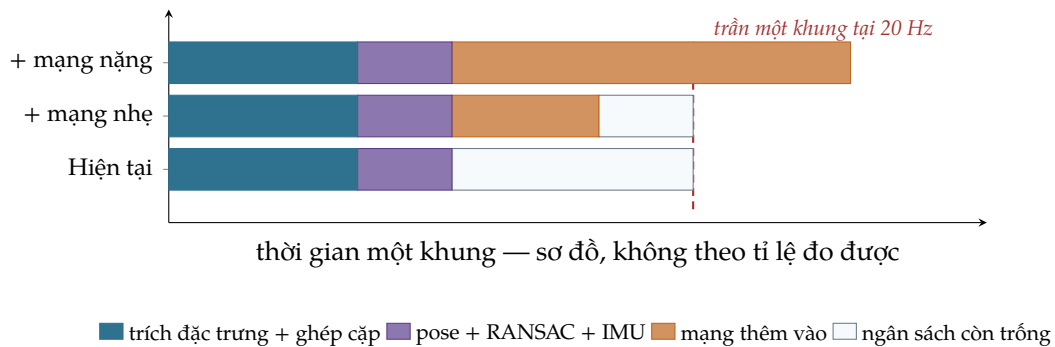
$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\Delta^2} = \frac{2(1,96 + 0,84)^2 (0,6)^2}{(1,0)^2} \approx 5,6.$$

Tức khoảng sáu lần chạy mỗi bên, mười hai lần cho một so sánh. Bây giờ thử $\Delta = 0,4$ cm: mẫu số nhỏ đi hơn sáu lần, nên n vọt lên khoảng 35 mỗi bên. Hình dạng đó nói thẳng một điều rất hữu ích, và nó đúng bất kể σ của bạn bằng bao nhiêu: số lần chạy tăng theo *bình phương* tỉ số giữa sần nhiều và mức cải thiện cần phát hiện. Nghĩa là có một ranh giới cứng — cải thiện nhỏ hơn sần nhiều chừng một nửa thì nằm ngoài tầm ngân sách thí nghiệm của bạn, và một paper hứa hẹn ở mức đó đã trượt cổng 2 trước cả khi bạn tải mã nguồn về. Ranh giới cứng đó nằm ở đâu thì bạn thay σ đo được của mình vào công thức trên là ra.

C. Cổng 3 — có vừa ngân sách Jetson Orin không

Cổng này là số học, và chính vì là số học nên nó loại được rất nhanh. Bắt đầu từ tốc độ khung hình. Camera EuRoC chạy 20 Hz, tức 50 mili-giây mỗi khung [2]. Đó là toàn bộ ngân sách của luồng bám, và ngân sách đó đã có người ở: trích đặc trưng trên hai ảnh, ghép cặp, RANSAC, tối ưu pose. Phần còn trống là phần bạn được tiêu.

Hình 1.3 vẽ *hình dạng* của phép tính đó và cố ý không mang một con số nào. Nó chỉ phát biểu ba điều: ngân sách là một cái hộp có kích thước cố định, cái hộp đó đã có người ở, và một mạng thêm vào thì hoặc lọt vào phần trống hoặc đẩy cả luồng bám vượt trần. Việc đầu tiên khi qua cổng này là điền số đo thật của chính bạn vào từng dải.



Hình 1.3. Hình dạng của phép tính ngân sách ở cổng 3. Trục không có vạch số và các dải không phải số đo của bất kỳ máy nào — chúng chỉ nói rằng cái hộp có kích thước cố định và đã có người ở. Ba điều đáng mang đi là: một mạng đủ nhẹ thì lọt vào phần còn trống; một mạng nặng thì đẩy luồng bám vượt trần và hệ bắt đầu bỏ khung; và phần còn trống là đại lượng bạn phải *đo* trên Orin, ở đúng chế độ điện và sau khi máy đã nóng, trước khi nói bất cứ điều gì về ứng viên.

Bốn cái bẫy làm phép tính này sai lệch, và cả bốn đều đẩy về hướng lạc quan.

- **Số FPS trong paper là số thông lượng.** Nó đo với batch lớn, ảnh đã nằm sẵn trên GPU, và thường không tính khâu tiền xử lý. SLAM chạy batch bằng 1 và phải trả cả chi phí chuyển dữ liệu. Chênh lệch giữa hai chế độ này không phải chuyện xê dịch vài phần trăm: nó lớn tới mức một kết luận rút ra từ FPS trong bảng hoàn toàn có thể đảo chiều khi bạn tự đo.
- **GPU của bạn có thể đã bận.** Nếu bạn đã chuyển trích đặc trưng sang GPU, thì mạng mới không chạy trên một GPU rảnh mà tranh chấp với chính phần đó.
- **Bộ nhớ dùng chung.** Trên Orin, CPU và GPU chia nhau cùng vùng nhớ. Một mô hình chiếm vài gigabyte không chỉ tốn bộ nhớ của nó, nó lấy đi chỗ của bản đồ.
- **Nhiệt.** Số đo trong ba mươi giây đầu không phải số đo lúc vận hành. Chu kỳ trôi khi máy nóng, và một hệ vừa đủ 50 ms lúc nguội sẽ bỏ khung sau mười phút.

Có một phép sàng nhanh giúp loại ứng viên *trước* khi cài đặt bất cứ thứ gì, và nó chỉ cần hai đại lượng. Tử số: số phép nhân cộng mà mạng cần cho một ảnh — đọc từ bảng của paper, hoặc đếm bằng công cụ. Mẫu số: thông lượng mà máy của bạn *thật sự* đạt được ở batch bằng 1, ở độ chính xác bạn định dùng, trên đúng dạng phép toán đó. Chia tử cho mẫu ra một khoảng thời gian, rồi nhân đôi nếu là ảnh stereo.

Toàn bộ chỗ dễ sai nằm ở mẫu số, và nó sai theo một hướng cố định là hướng lạc quan. Hiệu năng đỉnh ghi trên tờ thông số là con số đạt được ở batch lớn với ma trận lớn; ở batch bằng 1, phần lớn thời gian trôi vào việc chờ dữ liệu chứ không vào việc nhân, và với các phép toán ma trận nhỏ thì thứ chặn là băng thông bộ nhớ chứ không phải số nhân. Vì thế mẫu số phải là một con số bạn đã *đo* bằng một mạng nhỏ đã biết trên chính máy đó, không phải một con số bạn tra được.

Giá trị của phép chia này không nằm ở kết quả mà ở bậc độ lớn của nó, và nó cho ba phán quyết chứ không cho một con số. Nếu kết quả lớn hơn phần ngân sách còn trống nhiều lần thì trượt, khỏi cần cài đặt. Nếu nó cùng bậc với phần còn trống thì chưa kết luận được gì, và đáng bỏ một ngày ra đo thật. Nếu nó nhỏ hơn hẳn thì phép chia này đã hết việc, và rủi ro chuyển sang ba cái bẫy còn lại ở trên: tranh chấp GPU, bộ nhớ dùng chung, và nhiệt.

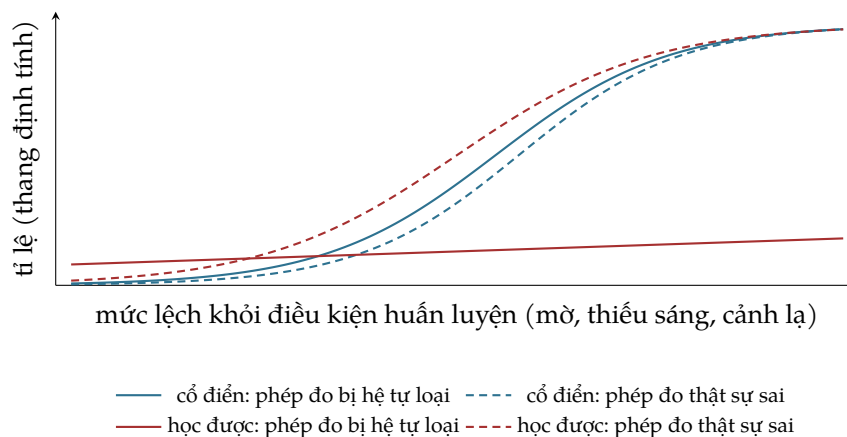
↪ **ràng buộc bởi B/11** — đây chính là chỗ “bộ nhớ và băng thông, không phải FLOPs, mới là thứ chặn” của chương 11, xuất hiện dưới dạng một ngân sách mili-giây cứng

D. Cổng 4 – hỏng ra sao khi ra ngoài phân phối

Ba cổng đầu nói về lợi ích và chi phí. Cổng này nói về rủi ro, và với một hệ SLAM nó quan trọng hơn cả ba cổng kia cộng lại.

Lý do nằm ở cấu trúc, không nằm ở độ chính xác. SLAM là một vòng phản hồi. Phép đo đi vào bản đồ, bản đồ quyết định phép đo tiếp theo được ghép với cái gì. Một phép đo sai không chỉ làm sai một khung; nó ở lại trong bản đồ. Với một vòng đóng sai, hậu quả gần như không sửa được: cả một đoạn quỹ đạo bị kéo lệch vĩnh viễn.

Bây giờ so sánh hai kiểu hỏng. Khi ORB gặp một bức tường trắng, nó không tìm được góc nào. Số điểm về gần 0, RANSAC thất bại, hệ thống tuyên bố mất bám và chuyển sang định vị lại. Thất bại đó *ồn ào*: nó tự báo cáo. Khi một mạng gặp bức tường trắng, nó vẫn trả về đúng số điểm bạn yêu cầu, kèm điểm tin cậy trông bình thường. Thất bại đó *im lặng*, và im lặng là chỗ chết người trong một vòng phản hồi.



Hình 1.4. Hai kiểu hỏng, vẽ theo dạng định tính chứ không phải theo số đo. Với khối cổ điển, hai đường gần như trùng nhau: cái gì sai thì cũng bị chính hệ thống loại, nên bạn nhìn số inlier là biết. Với khối học được, hai đường tách ra: tỉ lệ sai tăng trong khi hệ thống vẫn nhận vào bình thường. Khoảng cách giữa hai đường đồ chính là lượng phép đo sai chảy thẳng vào bản đồ mà không ai chặn.

TRỰC GIÁC

Hãy nghĩ về bộ tối ưu như một quan toà và các phép đo như nhân chứng. Ma trận thông tin là mức tin cậy toà dành cho từng nhân chứng, và RANSAC là phần đối chất. Một bộ dò đặc trưng cổ điển là nhân chứng thật thà kiểu cũ: hỏi tới chỗ không thấy thì nói thẳng “tôi không biết”. Một mạng chưa được hiệu chỉnh là nhân chứng ăn nói lưu loát, không bao giờ nói “tôi không biết”, và trả lời tự tin cả về những việc chưa từng chứng kiến. Vấn đề không phải nhân chứng ấy nói sai nhiều hơn. Vấn đề là toà mất khả năng biết lúc nào nên giảm mức tin cậy, vì tín hiệu tự tin đã không còn mang thông tin.

Từ đó ra hai tiêu chí trượt của cổng 4. Thứ nhất: nếu mạng không có một tín hiệu “tôi không biết” dùng được trong lúc chạy, thì nó chỉ nên được cấm ở nơi mà đầu ra của nó *trừ đi* chứ không cộng vào, hoặc phải có một tầng kiểm chứng hình học đứng sau. Thứ hai: nếu chuỗi bạn dùng để đánh giá nằm trong tập huấn luyện của mô hình, thì con số bạn đo được không nói gì về hành vi khi triển khai. Điều này xảy ra thường xuyên hơn người ta tưởng, vì EuRoC và TUM-VI là các tập rất phổ biến.

→ **cùng nguyên lý với B/08** — đây là dịch chuyển phân phối nhìn từ phía hệ thống: cái đặt không phải độ chính xác giảm, mà là mất tín hiệu báo rằng độ chính xác đã giảm

Bảng 1.2. Bốn cổng, phép đo quyết định của từng cổng, và tiêu chí trượt. Cột thứ ba là chỗ nên đầu tư công sức: nó nói bạn phải chạy thí nghiệm nào, chứ không phải đọc thêm paper nào.

Cổng	Câu hỏi	Phép đo quyết định	Tiêu chí trượt
1	Bài toán có thật không	Nhật ký từng khung kèm tần suất; oracle thay khối bằng chân trị để lấy trần đóng góp	Không dựng được nhật ký; trần nhỏ hơn hai lần sàn nhiễu; thất bại giải thích được bằng một tham số cấu hình
2	Đo được không	Sàn nhiễu từ nhiều lần chạy cùng cấu hình; APE kèm tỉ lệ hoàn thành; kiểm mối liên hệ giữa metric trung gian và APE	Cải thiện hứa hẹn nhỏ hơn sàn nhiễu; chỉ có metric trung gian; không báo tỉ lệ hoàn thành
3	Vừa ngân sách Orin không	Hồ sơ thời gian luồng bám trên Orin; độ trễ batch-1 của mạng sau khi tối ưu, đo lúc máy đã nóng	Vượt phần ngân sách còn trống; bộ nhớ lẫn bản đồ; chu kỳ trôi theo nhiệt
4	Hông ra sao khi ra ngoài phân phối	Chạy trên cảnh cố ý khác: thiếu sáng, mờ, hành lang không vắn; đo tỉ lệ phép đo sai lọt qua tầng kiểm chứng	Không có tín hiệu “tôi không biết”; chuỗi đánh giá nằm trong tập huấn luyện; hông im lặng ở chỗ cộng thêm phép đo

E. Chạy thử khung: một ứng viên đi qua bốn cổng

Khung chỉ có nghĩa khi chạy được trên một ví dụ. Lấy ứng viên phổ biến nhất: thay ORB bằng một bộ dò và mô tả học được, kèm một bộ ghép cặp học được. Về vị trí, nó là trụ 1 ở mức can thiệp 2. Dưới đây cố ý không có một con số nào, vì con số phải là của bạn. Thứ đáng học ở ví dụ này là *thứ tự câu hỏi* và *dạng kết luận* mà mỗi cổng cho phép phát biểu.

Cổng 1. Bạn dựng nhật ký và thấy sai số không rải đều mà dồn vào các đoạn xoay nhanh. Bạn chạy oracle ghép cặp: dùng pose chân trị loại mọi tương ứng sai rồi chạy lại. Kết luận có đúng một dạng, và dạng đó là một tỉ số: trần đóng góp lớn hơn sàn nhiễu *bao nhiêu lần*. Lớn hơn nhiều lần thì qua cổng. Xấp xỉ bằng thì dừng lại ngay ở đây, và bạn vừa tiết kiệm toàn bộ phần còn lại.

Cổng 2. Bạn báo APE kèm tỉ lệ hoàn thành, và bạn đã có sàn nhiễu từ nhiều lần chạy cùng một cấu hình. Bạn cũng ghi chú rằng không được kết luận bằng số điểm khớp: đó là metric trung gian, và cái trần đo được ở cổng 1 mới là thứ đáng so. Qua cổng.

Cổng 3. Đây là chỗ ứng viên gặp khó, và lý do là cấu trúc chứ không phải một con số. Bộ dò chạy trên hai ảnh mỗi khung, bộ ghép cặp chạy thêm một lần nữa mỗi khung, và cả ba khoản đó nằm trong luồng bám, tức là cùng chen vào một ngân sách vốn đã có người ở. Với một mô hình cỡ trung, kết luận thường là trượt — nhưng trượt *có điều kiện*, và điều kiện đó rất cụ thể.

Ở đây có một lối thoát mà phép tính thô che mất. Không phải mọi chỗ dùng ghép cặp đều chạy ở nhịp camera. Tầng ghép cặp trong đường đóng vòng chỉ chạy khi có ứng viên, tức vài lần mỗi phút, nên ngân sách của nó rộng hơn ngân sách một khung của luồng bám nhiều bậc. Vậy cùng một mô hình trượt cổng 3 ở luồng bám lại lọt thoải mái ở đường đóng vòng. Đó là kết luận đáng giá nhất của cả ví dụ này, và nó chỉ hiện ra khi bạn tính ngân sách *theo từng đường*, không tính theo cả hệ.

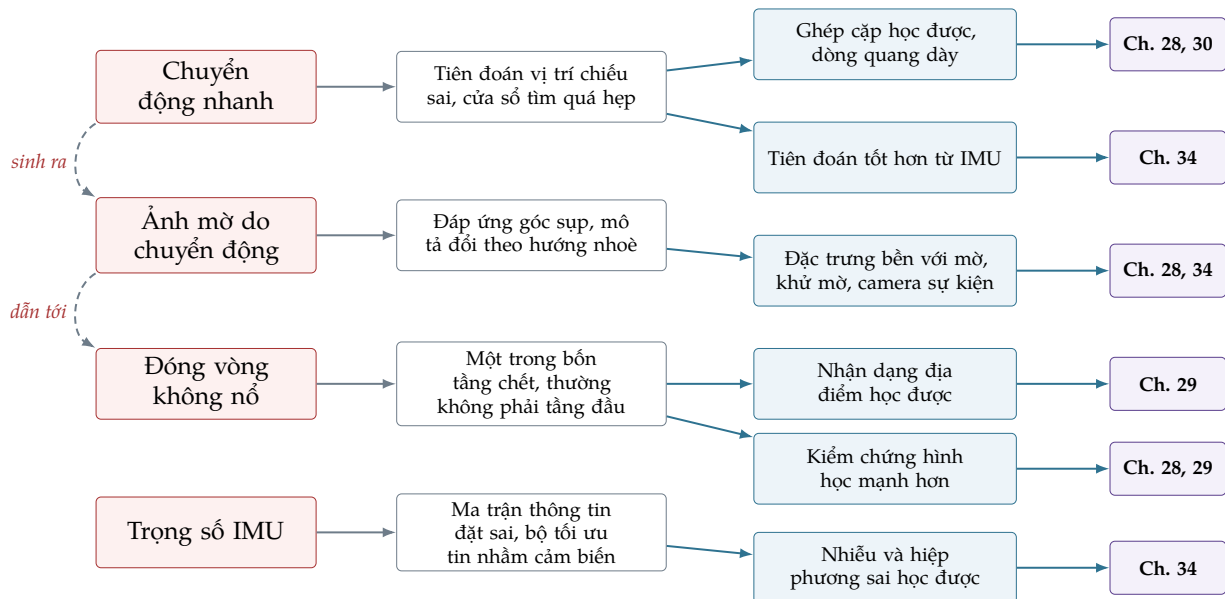
Cổng 4. Bộ dò học được không có tín hiệu “tôi không biết” dùng được. Nhưng ở đường đóng vòng, đằng sau nó vẫn còn tầng kiểm chứng Sim3 bằng RANSAC, tức vẫn còn một trọng tài hình học. Rủi ro vì thế bị chặn: một tương ứng sai muốn vào bản đồ phải qua được kiểm chứng hình học. Điều kiện kèm theo là bạn không được nới ngưỡng inlier để bù cho việc mô hình mới trả nhiều tương ứng hơn.

Phán quyết. Không phải một chữ GO hay NO-GO, mà là một câu có điều kiện: chưa đưa vào luồng bám vì cổng 3, nhưng đáng thử ở đường đóng vòng vì ở đó cổng 3 rộng và cổng 4 đã có người canh. Chi phí thử: cỡ một tuần. Điều kiện xem lại phán quyết đầu tiên: khi độ trễ của luồng bám giảm đủ để mở ra một chỗ trống thật, hoặc khi có một mô hình nhẹ hơn ở cùng chất lượng.

1.4 Bản đồ bốn nỗi đau (Mapping Four Pains to Method Families)

Đây là mục trung tâm của chương. Bốn cổng cho bạn cách loại; mục này cho bạn cách *chọn*. Nó trả lời một câu duy nhất: với mỗi nỗi đau cụ thể, nhóm phương pháp nào hứa chữa, chương nào nói về nó, và điều kiện gì phải đúng thì mới đáng thử.

Trước khi vào bảng, một lời cảnh báo về cấu trúc. Bốn nỗi đau không độc lập. Chuyển động nhanh sinh ra ảnh mờ; ảnh mờ làm mất bám; mất bám làm ta cần định vị lại và cần vòng đóng. Vì thế quy gán một con số APE cho đúng một nỗi đau là việc khó, và cách duy nhất làm được là quy gán theo từng khung. Hình 1.5 vẽ bản đồ ba tầng: nỗi đau, cơ chế hỏng, nhóm phương pháp.



Hình 1.5. Từ nỗi đau sang cơ chế hỏng sang nhóm phương pháp. Cột giữa là cột phải đo trước: nó nói *cái gì* hỏng, và chỉ khi biết điều đó thì cột phải mới có nghĩa. Hai mũi tên đứt bên trái nhắc rằng bốn nỗi đau nối nhau, nên một cải thiện ở nỗi đau đầu có thể làm ba nỗi đau còn lại nhẹ đi mà không cần đụng tới chúng.

A. Chuyển động nhanh

Cơ chế hỏng ở đây không phải “ảnh xấu”. Nó là một chuỗi rất cụ thể. Bộ bám chiếu các điểm bản đồ vào khung mới bằng một tiên đoán pose, rồi tìm mô tả khớp trong một cửa sổ quanh vị trí chiếu. Khi camera xoay nhanh, tiên đoán lệch nhiều hơn bán kính cửa sổ. Số tương ứng tụt, RANSAC còn ít điểm, và một loạt quyết định phía sau đổi theo.

Điều đó cho ra hai hướng chữa hoàn toàn khác nhau, và chúng nằm ở hai trụ khác nhau. Hướng thứ nhất là làm cho việc ghép cặp không cần tiên đoán tốt: ghép cặp học được kiểu SuperGlue hoặc LightGlue dùng ngữ cảnh toàn ảnh, và dòng quang dày kiểu RAFT [17] không cần cửa sổ nào cả. Hướng thứ hai là làm cho tiên đoán tốt hơn, bằng chính IMU mà bạn đã có.

Nhận định thẳng thắn của chương này: hướng thứ hai nên thử trước, vì nó rẻ hơn nhiều bậc. IMU 200 Hz cho một ước lượng xoay rất tốt trên khoảng thời gian 50 mili-giây. Nếu tiên đoán của bạn còn dùng mô hình vận tốc không đổi ở những đoạn xoay nhanh, thì bạn đang bỏ không một cảm biến đã có sẵn. Chỉ khi hướng đó đã cạn mới đáng trả giá độ trễ cho một bộ ghép cặp học được.

B. Ảnh mờ do chuyển động

Ảnh mờ tấn công đúng giả định nền của ORB: rằng một góc là một góc. Khi ảnh nhoè theo một hướng, đáp ứng góc sụp xuống, và cái còn lại giống một cạnh hơn một góc. Tệ hơn, mô tả đổi theo *hướng* nhoè, nên hai ảnh mờ theo hai hướng khác nhau của cùng một cảnh cho hai mô tả khác nhau.

Có bốn nhóm chữa, và chúng khác nhau cả về trụ lẫn về giá. Đặc trưng học được bền hơn với mờ ở mức nào đó, và đó là mảng của chương F/28. Khử mờ trước khi trích đặc trưng là một tầng cộng thêm, tốn đúng ngân sách mà bạn đang thiếu. Hiệu chỉnh chùm tia có ý thức về mờ mô hình hoá chính quá trình phơi sáng, đắt nhưng đúng về mặt vật lý. Camera sự kiện thì bỏ hẳn khái niệm phơi sáng. Ba nhóm sau nằm ở chương F/34.

Điều kiện đáng thử ở đây nghiêm ngặt hơn ta tưởng. Trước tiên phải đo: trên chuỗi của bạn, bao nhiêu phần trăm khung thật sự mờ tới mức làm hỏng việc, và bao nhiêu phần APE quy được về những khung đó. Nếu chỉ một phần rất nhỏ số khung bị mờ nặng và hệ thống vượt qua chúng nhờ IMU, thì cả bốn nhóm trên đều đang chữa một vết thương không chảy máu.

C. Đóng vòng — ví dụ mẫu của cổng 1

Đóng vòng là nỗi đau minh hoạ rõ nhất vì sao khung bốn cổng lại cần thiết. Nó có bốn tầng nối tiếp, và một vòng chỉ đóng khi qua được cả bốn:

1. **Lấy ứng viên** — tìm các keyframe cũ có thể là cùng một chỗ. Cổ điển dùng túi từ trên mô tả nhị phân.
2. **Ghép đặc trưng** — tìm tương ứng giữa khung hiện tại và ứng viên.
3. **Kiểm chứng hình học** — ước lượng Sim3 bằng RANSAC và đòi đủ số inlier.
4. **Tối ưu** — hiệu chỉnh đồ thị pose rồi hiệu chỉnh chùm tia toàn cục.

Bây giờ là chỗ mấu chốt. Phần lớn công trình học sâu về đóng vòng cải thiện tầng 1: NetVLAD [18] và các hậu duệ của nó, hoặc một pipeline phân cấp kiểu HF-Net [19]. Nếu tầng chết trên hệ của bạn không phải tầng 1, thì mọi cải thiện đó là công cốc, và bạn sẽ thấy đúng hiện tượng ở câu hỏi mở đầu: recall tăng mạnh, số vòng đóng không đổi.

I MẠCH 2 — chẩn đoán đóng vòng trước khi chọn phương pháp

- | | |
|--|--|
| <p>1 Vấn đề</p> <p>Giải pháp</p> <p>Giả định</p> <p>Giá phải trả</p> | <p>Trên chuỗi dài, vòng gần như không đóng. Triệu chứng dễ thấy nhất là quỹ đạo trôi mà không bao giờ được kéo về.</p> <p>Đếm số ứng viên sống sót sau <i>từng</i> tầng trong bốn tầng, trên vài chục lần đi qua cùng một chỗ.</p> <p>Bốn tầng nối tiếp và độc lập đủ để quy trách nhiệm cho một tầng.</p> <p>một buổi thêm bộ đếm và một buổi chạy lại.</p> |
| <p>2 Vấn đề</p> <p>Giải pháp</p> <p>Hệ quả</p> | <p>Bộ đếm đặt sai chỗ thì số ra vô nghĩa. Nếu bộ đếm nằm trước câu lệnh điều kiện, nó đếm cả những ứng viên chưa từng được xét.</p> <p>Đặt bộ đếm ngay sau mỗi điều kiện, và ghi cả lý do bị loại.</p> <p>Đây không phải chi tiết vụn vặt. Một lần đo nội bộ trên chính nhánh này bạn đầu quy sai thủ phạm chỉ vì bộ đếm nằm trước điều kiện, và kết luận đảo ngược sau khi sửa.</p> |
| <p>3 Vấn đề</p> | <p>Giả sử phép đếm đúng cho thấy phần lớn ứng viên chết ở tầng 3, tức Sim3 RANSAC không hội tụ.</p> |

Giải pháp	Khi đó tầng đáng đầu tư là tầng 2, vì RANSAC chết do quá ít tương ứng đúng. Ghép cặp học được ở chương F/28 chứa đúng chỗ đó.
Vấn đề mới	Nới ngưỡng inlier để RANSAC dễ hội tụ hơn là cá mỗ lớn nhất và cũng là sai lầm đắt nhất, vì nó mua thêm vòng đóng bằng cách mua thêm vòng <i>sai</i> .
4 Vấn đề	Một vòng sai đắt hơn một vòng bị bỏ lỡ rất nhiều. Bỏ lỡ chỉ để lại độ trôi; đóng sai kéo lệch cả một đoạn bản đồ và gần như không cứu được.
Giải pháp	Chọn điểm vận hành lệch hẳn về phía precision, và giữ tầng kiểm chứng hình học kể cả khi tầng lấy ứng viên đã học được.
Giả định	Chi phí của hai loại lỗi bất đối xứng, và bạn ước lượng được mức bất đối xứng đó.

CẠM BẦY

Cạm bẫy hay gặp nhất khi đọc mẩu này là tin vào một metric của tầng 1 để quyết định cho cả bốn tầng. Recall@1 cao nghĩa là “ứng viên đúng có mặt trong danh sách”. Nó không nói gì về việc ứng viên ấy có đủ tương ứng để Sim3 hội tụ hay không. Hai đại lượng đó chỉ trùng nhau khi tầng 2 và tầng 3 dư sức, mà đó chính là điều bạn đang nghi ngờ. Cách phân biệt rõ nhất: cấp thẳng ứng viên đúng từ chân trị vị trí, rồi đếm xem bao nhiêu ứng viên hoàn hảo đó vẫn không đóng được vòng. Nếu con số đó lớn, tầng 1 không phải thủ phạm.

D. Trọng số cho IMU

Nỗi đau thứ tư ít được nói tới nhất, và theo đánh giá của chương này, nó là chỗ có tỉ số lợi ích trên rủi ro tốt nhất. Lý do rất cấu trúc: mọi thứ ở đây diễn ra *bên trong* bộ ước lượng cổ điển, nên không có khối nào bị xóa đi.

Cơ chế hỏng như sau. Trong đồ thị nhân tử, cạnh IMU mang một ma trận thông tin suy ra từ mật độ phổ nhiễu của con quay và gia tốc kế, lan truyền qua bước tiền tích phân [8]. Các con số đó đến từ bảng thông số cảm biến hoặc từ một lần hiệu chỉnh tĩnh. Chúng không mô tả nhiễu thật lúc thiết bị đang rung, đang nóng, hoặc đang chịu gia tốc lớn. Đặt trọng số quá cao thì bộ tối ưu bám theo IMU và bỏ qua ảnh; quá thấp thì IMU thành vô dụng và bạn mất tỉ lệ tuyệt đối cùng khả năng vượt qua vài khung mù.

Nhóm phương pháp hứa chữa gồm: học mô hình chuyển động quán tính trực tiếp từ chuỗi IMU, khử nhiễu con quay bằng mạng, và học hiệp phương sai theo từng phép đo thay vì dùng hằng số. TLIO [20] là mốc dễ đọc nhất của nhóm đầu. Tất cả nằm ở chương F/34.

Điều kiện đáng thử: bạn phải chỉ ra được rằng trọng số hiện tại *đang* sai. Phép thử rõ nhất là một oracle trên chính siêu tham số đó. Quét hệ số nhân của ma trận thông tin IMU trên một lưới thô, lấy kết quả tốt nhất cho từng chuỗi, và so với giá trị mặc định. Khoảng cách đó là trần đóng góp của mọi phương pháp học trọng số. Nếu nó nhỏ, cả nhóm bị loại ở cổng 1 và bạn tiết kiệm được nhiều tháng.

Bảng 1.3. Bản đồ bốn nỗi đau. Cột “điều kiện để đáng thử” là cột quyết định: nó là phép đo bạn phải chạy trước, và trong phần lớn trường hợp nó sẽ tự loại bớt ba trong bốn dòng.

Nỗi đau	Cơ chế hỏng và cách đo	Nhóm hứa chữa	Chương	Điều kiện để đáng thử
Chuyển động nhanh	Tiên đoán chiều lệch quá bán kính cửa sổ tìm. Đo: vận tốc góc so với số tương ứng còn lại, theo từng khung	Ghép cặp học được, dòng quang đây; hoặc tiên đoán tốt hơn bằng IMU	28, 30, 34	Đã dùng hết thông tin xoay của IMU cho tiên đoán mà số tương ứng vẫn tụt ở đúng các đoạn xoay nhanh
Ảnh mờ	Đáp ứng góc sụp, mô tả đổi theo hướng nhỏ. Đo: chỉ số nhỏ theo khung, đổi chiều với khung bị mất bám	Đặc trưng bền với mờ; khử mờ; hiệu chỉnh chùm tia có ý thức về mờ; camera sự kiện	28, 34	Tỉ lệ khung mờ nặng đủ lớn, và quy được một phần APE về đúng các khung đó
Đóng vòng	Một trong bốn tầng chết. Đo: đếm ứng viên sống sót sau từng tầng, bộ đếm đặt sau điều kiện	Nhận dạng địa điểm học được (tầng 1); ghép cặp học được (tầng 2); kiểm chứng mạnh hơn (tầng 3)	28, 29	Phép đếm chỉ đúng tầng mà nhóm đó chữa. Cấp ứng viên từ chân trị để loại trừ tầng 1 trước
Trọng số IMU	Ma trận thông tin đến từ bảng thông số, không mô tả nhiều lúc rung và lúc nóng. Đo: quét hệ số nhân trên lưới thô	Học nhiều và bias IMU; hiệp phương sai theo từng phép đo; khử nhiễu con quay	34	Khoảng cách giữa hệ số tốt nhất theo chuỗi và giá trị mặc định lớn hơn hẳn sản nhiễu

E. Thứ tự ưu tiên đề xuất, và lập luận đằng sau nó

Nếu buộc phải xếp thứ tự khi chưa có bảng trần đóng góp của riêng bạn, chương này đề xuất thứ tự sau, kèm lý do. Đây là phán đoán chứ không phải kết quả đo, nên hãy coi nó là điểm khởi hành và thay bằng số của bạn ngay khi có.

1. **Trọng số IMU trước.** Nó nằm hoàn toàn bên trong bộ ước lượng cổ điển, nên không phải xoá khỏi nào. Phép thử oracle của nó rẻ nhất trong bốn nỗi đau: chỉ là quét một siêu tham số. Và nếu trần lớn, cách chữa đầu tiên thậm chí chưa cần mạng nào.
2. **Đóng vòng thứ hai.** Chi phí một vòng sai rất cao, nhưng phép chẩn đoán bốn tầng lại rẻ và cho kết luận sắc. Nhiều khả năng nó chỉ ra rằng tầng cần chữa không phải tầng mà phần lớn paper chữa, và riêng kết luận đó đã tiết kiệm hàng tháng.
3. **Chuyển động nhanh thứ ba.** Hướng rẻ là dùng tốt hơn IMU đã có, và hướng đó nên cạn trước khi trả giá độ trễ cho ghép cặp học được.
4. **Ảnh mờ cuối cùng.** Không phải vì nó không quan trọng, mà vì nó khó quy trách nhiệm nhất và cách chữa đắt nhất. Nó cũng chông lên chuyển động nhanh, nên một phần lợi ích của nó sẽ tự đến khi bạn xử lý mục thứ ba.

Thứ tự này có một điểm chung đáng chú ý: nó xếp theo *chi phí của phép chẩn đoán*, không xếp theo mức đau. Lý do là ở giai đoạn này bạn đang mua thông tin chứ chưa mua cải thiện, và thông tin rẻ nhất nên mua trước.

1.5 Ba mức can thiệp (Three Levels of Intervention)

Giả sử một ứng viên đã qua bốn cổng. Còn một quyết định nữa, và nó quyết định phần lớn chi phí thật: cắm sâu tới mức nào. Có ba mức, và chúng khác nhau không phải ở độ khó viết code mà ở *thứ bạn phải từ bỏ*.

A. Mức 1 — bổ trợ

Mạng chỉ cấp thêm một tín hiệu; đường cổ điển vẫn chạy và vẫn quyết định. Ba ví dụ điển hình: một mặt nạ vật thể động chỉ dùng để *loại* phép đo; một hiệp phương sai học được chỉ dùng để *cân lại* phép đo đã có; một tiên đoán học được chỉ dùng để *thu hẹp* cửa sổ tìm.

Tính chất định nghĩa của mức này là: nếu mạng chết hoặc trả rác, hệ thống suy biến về đúng baseline cổ điển. Điều đó cho bạn hai thứ vô giá. Thứ nhất là một công tắc: bật và tắt để so sánh, trên cùng một lần chạy nếu muốn. Thứ hai là một trần thiết hại: điều tệ nhất có thể xảy ra là mất một ít phép đo tốt.

B. Mức 2 — thay một khối

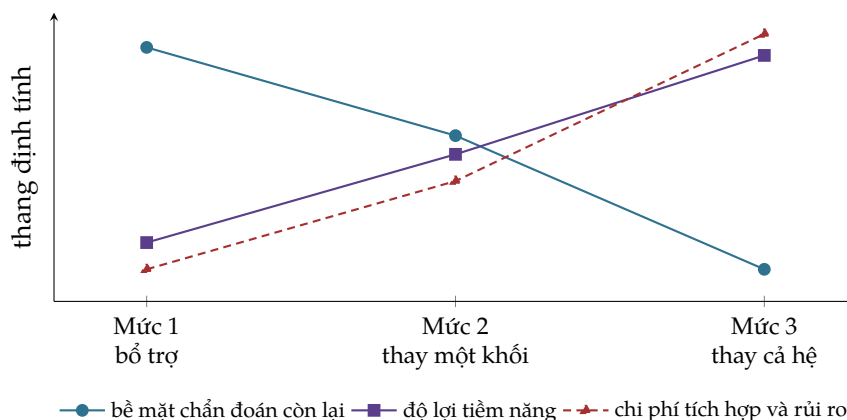
Mạng thay hẳn một khối cổ điển, nhưng giữ nguyên giao diện của khối đó. Trích đặc trưng học được thay ORB; nhận dạng địa điểm học được thay DBoW; độ sâu học được thay stereo. Backend không đổi một dòng.

Cái phải từ bỏ ở đây thường bị đánh giá thấp. Bạn không chỉ bỏ đoạn code cũ. Bạn bỏ luôn toàn bộ tinh chỉnh đã gắn với nó, bỏ tập các chế độ hỏng mà bạn đã thuộc lòng, và bỏ khả năng chạy song song hai đường để đối chứng lúc gỡ lỗi. Bề mặt chẩn đoán vẫn còn — vẫn có điểm, vẫn có tương ứng, vẫn có inlier — nhưng mỗi triệu chứng bây giờ có hai chỗ trú, và tách chúng ra cần một oracle chứ không cần một câu lệnh in.

C. Mức 3 — thay cả hệ

Mạng thay cả chuỗi ước lượng. Đây là trụ 3, và nó không phải một cuộc tích hợp mà là một cuộc chuyển hệ. Bạn được một hệ thống mới với các chế độ hỏng mới, và mất những thứ mà một hệ trưởng thành mang theo: quản lý bản đồ tăng dần, nhiều bản đồ, định vị lại, khả năng hỏi hiệp phương sai, và nhiều năm tinh chỉnh ngưỡng.

Nói thế không có nghĩa mức 3 luôn sai. Nó là lựa chọn đúng trong hai tình huống. Thứ nhất, khi bạn đang bắt đầu từ con số không trên một miền mà hệ đặc trưng thừa vốn đã không chạy nổi. Thứ hai, khi bạn cần bản đồ dày cho một mục đích khác và độ chính xác quỹ đạo chỉ là yêu cầu phụ.



Hình 1.6. Ba mức can thiệp trên một thang định tính, không phải thang đo. Điều đáng chú ý không phải ba đường mà là *khoảng cách* giữa chúng: độ lợi tăng gần tuyến tính, còn chi phí và rủi ro tăng nhanh hơn, trong khi bề mặt chẩn đoán — số đại lượng trung gian bạn còn kiểm tra được bằng một kỳ vọng vật lý — sụt gần như đứng vững ở mức 3.

D. Quy tắc chọn mức, và vì sao nó là quy tắc một chiều

Quy tắc rất gọn: **không lên mức tiếp theo cho tới khi đã đo được rằng mức hiện tại bị chặn dưới mức bạn cần**. Nó nghe hiển nhiên nhưng gần như không ai làm, vì mức cao hơn luôn hấp dẫn hơn về mặt trí tuệ.

Có một lý do sâu hơn để đi từ dưới lên: **tính đảo ngược**. Mức 1 gỡ ra bằng một cờ cầu hình. Mức 2 gỡ ra trong vài ngày nếu bạn còn giữ đường cũ. Mức 3 thì không gỡ ra được, vì sau sáu tháng thì mọi thứ quanh nó đã được xây theo nó. Chi phí thật của mức 3 không phải chi phí tích hợp mà là chi phí của một quyết định không thu hồi được.

Bảng 1.4. Ba mức can thiệp. Cột “cách quay lại” là cột hay bị bỏ qua nhất khi lên kế hoạch, và nó là cột định giá rủi ro thật của quyết định.

Mức	Cơ chế	Phải bỏ đi cái gì	Cách quay lại	Hỏng ra sao
1. Bổ trợ	Mạng cấp thêm tín hiệu; đường cổ điển vẫn quyết định	Không bỏ gì; chỉ thêm một tầng lọc hoặc một trọng số	Một cờ cầu hình	Mất một ít phép đo tốt; hệ suy biến về baseline
2. Thay một khối	Mạng thay khối cổ điển, giữ nguyên giao diện với backend	Đoạn code cũ, phần tinh chỉnh gắn với nó, và tập chế độ hỏng đã quen	Vài ngày, nếu còn giữ đường cũ chạy song song	Triệu chứng có hai chỗ trú; cần oracle để tách; hỏng im lặng nếu mạng không biết nói “không biết”
3. Thay cả hệ	Mạng thay cả chuỗi ước lượng, không còn bước hình học tường minh	Quản lý bản đồ, nhiều bản đồ, định vị lại, hiệp phương sai, nhiều năm tinh chỉnh	Gần như không; sau vài tháng mọi thứ quanh nó đã xây theo nó	Không còn đại lượng trung gian để hỏi; sửa lỗi nghĩa là huấn luyện lại

CẦN NHỚ

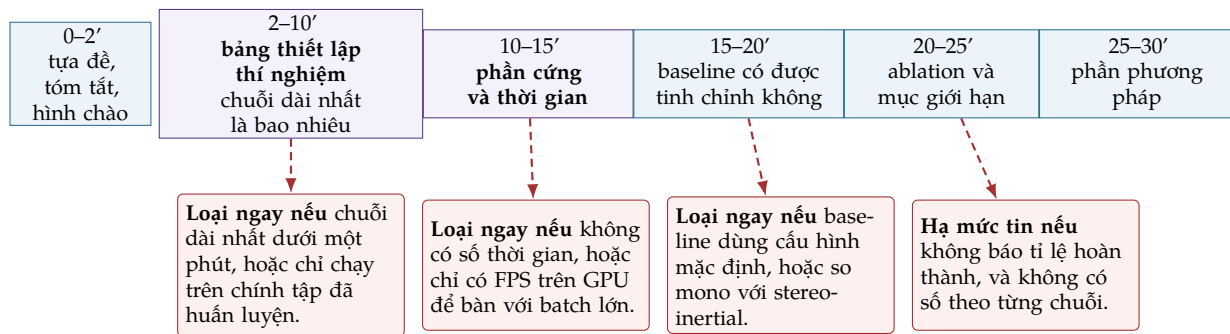
Hai câu quyết định phần lớn các lựa chọn trong Phần F. Thứ nhất: đo trần đóng góp của một khối *trước*, rồi mới đi tìm phương pháp thay khối đó — nếu một khối hoàn hảo cũng chỉ cải thiện được vài phần trăm thì cả nhánh nghiên cứu đó đã bị chặn trên. Thứ hai: trong một vòng phản hồi, một phép đo sai mà im lặng đắt hơn nhiều một phép đo thiếu mà lên tiếng, nên hãy ưu tiên các can thiệp chỉ *trừ đi* phép đo hoặc chỉ *cân lại* chúng. [T1]

1.6 Đọc một paper SLAM học sâu trong ba mươi phút (Reading a Deep SLAM Paper in 30 Minutes)

Bốn công và ba mức là khung để quyết định. Mục này là kỹ năng thao tác hằng ngày: mở một paper, và trong ba mươi phút biết nên bỏ hay nên đọc kỹ. Chương E/23-doc-paper đã dạy đọc paper nói chung theo lối ba lượt [21]. Ở đây thứ tự bị đảo lại có chủ ý, vì với mảng này thông tin quyết định nằm ở phần thí nghiệm chứ không ở phần phương pháp.

A. Thứ tự đọc, và vì sao nó ngược

Lý do đảo thứ tự rất đơn giản. Phần phương pháp cho bạn biết tác giả *định* làm gì. Phần thí nghiệm cho bạn biết điều đó đã được kiểm chứng tới đâu, và với mảng SLAM học sâu thì khoảng cách giữa hai thứ ấy thường rất lớn. Một phương pháp thanh lịch chỉ được thử trên chuỗi trong nhà dài một phút không nói gì về hệ của bạn.



Hình 1.7. Thứ tự đọc ba mươi phút, đảo ngược so với thứ tự trình bày của paper. Hai ô tím là hai ô đắt nhất về mặt thông tin và rẻ nhất về mặt thời gian, nên chúng đứng trước. Bốn ô đỏ là các cổng loại: chạm vào một trong ba ô đầu thì dừng đọc, khỏi mở phần phương pháp.

B. Sáu chặng, và câu hỏi của từng chặng

- **0-2 phút, định vị.** Phương pháp này ở trụ nào trong năm trụ, và nó đòi mức can thiệp nào? Chỉ hai câu đó thôi đã đủ để biết bạn phải bỏ cái gì nếu nhận nó.
- **2-10 phút, bảng thiết lập thí nghiệm.** Tập nào, chuỗi nào, và quan trọng nhất: *chuỗi dài nhất họ chạy dài bao nhiêu*. Rất nhiều kết quả ấn tượng của mảng bản đồ neural đến từ các cảnh tổng hợp trong nhà, mỗi cảnh cỡ hai nghìn khung. Một chuỗi EuRoC ngoài đời dài vài phút, và một lần chạy sản phẩm dài hàng chục phút. Cũng ở chặng này: dữ liệu huấn luyện của họ có chứa tập đánh giá của bạn không.
- **10-15 phút, phần cứng và thời gian.** GPU nào, độ trễ hay thông lượng, batch bao nhiêu, và phép đo có tính cả tiền xử lý lẫn chuyển dữ liệu không. Nếu không có mục nào nói về thời gian, hãy coi như phương pháp chưa được thiết kế cho thời gian thực.
- **15-20 phút, baseline.** Họ so với ORB-SLAM3 phiên bản nào, cấu hình nào, có bật đóng vòng không, đơn mắt hay stereo-inertial. So một hệ stereo-inertial học được với một baseline đơn mắt là phép so sánh phổ biến và vô nghĩa.
- **20-25 phút, ablation và mục giới hạn.** Mục giới hạn viết thật là dấu hiệu chất lượng mạnh nhất mà bạn có. Nếu paper có một đoạn nói thẳng phương pháp gãy ở đâu, hãy tăng mức tin cậy cho toàn bộ phần còn lại.
- **25-30 phút, phương pháp.** Và chỉ đọc với đúng một câu hỏi trong đầu: *ý tưởng thiết kế nào còn giá trị kể cả khi tôi không dùng hệ này*. Đó thường là thứ duy nhất bạn mang về được.

C. Ba dấu hiệu bên ngoài bài báo

Ba việc kiểm tra sau tốn dưới năm phút và thường quyết định nhiều hơn cả bài báo. Thứ nhất, có mã nguồn chạy được không, và lần cập nhật gần nhất là bao giờ. Thứ hai, mục theo dõi lỗi của kho mã: nếu nhiều người báo cùng một vấn đề tái lập mà không có trả lời, đó là dữ liệu. Thứ ba, có ai độc lập với nhóm tác giả chạy lại được chưa. Trong mảng SLAM, khoảng cách giữa “chạy được trên máy tác giả” và “chạy được trên máy bạn” đặc biệt rộng, vì phụ thuộc vào phiên bản thư viện, dữ liệu và hiệu chỉnh cảm biến.

D. Ba câu hỏi bạn phải tự trả lời sau ba mươi phút

Kết thúc một buổi đọc bằng ba câu, viết ra giấy, không giữ trong đầu. Một: phương pháp này chạm nổi đau nào trong bốn nổi đau, và nó chữa *tầng* nào của nổi đau đó. Hai: nếu nhận nó, tôi phải xoá cái gì khỏi hệ hiện tại. Ba: bằng chứng cụ thể nào sẽ làm tôi đổi kết luận, và bao giờ tôi nên xem lại. Câu thứ ba biến một kết luận “chưa đáng” thành một kết luận có hạn dùng, thay vì thành một định kiến.

E. Bốn câu hỏi mang sang bảy chương sau

Cả Phần F được viết theo đúng bốn câu hỏi dưới đây, và bạn nên đọc mọi phương pháp bằng chúng. Chúng là bản rút gọn của cả chương này thành một khuôn dùng lại được.

1. **Nó giải quyết việc gì?** Phát biểu bằng một thất bại cụ thể của hệ cổ điển, có chuỗi và có tần suất. “Học đặc trưng tốt hơn” không phải câu trả lời; “giữ được tương ứng ở các đoạn xoay nhanh, nơi hệ hiện tại tụt xuống dưới ngưỡng inlier” thì có.
2. **Cơ chế nào làm được điều đó?** Một đoạn, nói đúng thiết kế cốt lõi. Nếu bạn không tóm được cơ chế trong một đoạn thì hoặc paper viết chưa rõ, hoặc bạn chưa hiểu, và cả hai trường hợp đều nên dừng lại.
3. **Có đáng làm không?** Đây là bốn cổng: chi phí độ trễ trên Orin, lượng dữ liệu cần, cách nó hỏng khi ra ngoài phân phối, và — câu hay bị quên nhất — *cái gì trong hệ cổ điển phải bị bỏ đi để nhận nó*.
4. **Nếu đi sâu thì học được gì từ thiết kế của nó?** Bài học còn giá trị kể cả khi bạn không dùng phương pháp. Đây thường là thứ duy nhất bạn mang về được, và cũng là lý do đọc tiếp bảy chương sau dù phần lớn phán quyết là “chưa đáng”.

Câu thứ ba là câu mà chương này phục vụ. Câu thứ tư là câu giữ cho việc đọc không biến thành một chuỗi từ chối, và nó là lý do bảy chương sau vẫn đi vào cơ chế chứ không dừng ở kết luận.

1.7 Giới hạn và trường hợp hỏng

Khung trong chương này có giới hạn riêng, và một chương bộ lọc mà không tự soi thì chính nó thành cái bẫy.

Nó lệch về phía từ chối. Bốn cổng tối ưu cho việc không phí công, chứ không tối ưu cho việc không bỏ lỡ. Một khung loại hết mọi thứ cũng là một khung hỏng, chỉ là hỏng theo cách khó thấy. Thuốc giải là giữ một danh sách theo dõi ngắn: mỗi ứng viên bị loại kèm đúng một dòng ghi bằng chứng nào sẽ lật kết luận, rồi chạy lại khung mỗi năm một lần.

Trần đóng góp giả định các khối tách rời được. Thay một khối bằng chân trị làm đổi phân phối đầu vào của khối đứng ngay sau. Trong một hệ có vòng phản hồi giữa bản đồ và quyết định keyframe, “đóng góp của khối X” không còn là một đại lượng xác định. Đây đúng là giới hạn đã nêu ở chương E/24, và ở SLAM nó nặng hơn vì vòng phản hồi chặt hơn.

Phép tính ngân sách giả định pipeline nối tiếp và bị chặn bởi độ trễ. Nếu CPU đang là nút thắt còn GPU rảnh, một mạng có thể gần như miễn phí về thời gian tường. Ngược lại, nếu bạn đã đẩy trích đặc trưng lên GPU thì phép cộng đơn giản ở hình 1.3 sẽ cho kết luận sai theo hướng lạc quan.

Bốn nỗi đau không độc lập. Quy một con số APE về đúng một nỗi đau đòi hỏi quy gán theo từng khung, mà việc đó chưa có phương pháp gọn. Trong thực tế bạn sẽ phải sống với những kết luận kiểu “phần lớn sai số tập trung ở một đoạn vài chục giây, nơi có cả xoay nhanh lẫn ảnh mờ”, và chấp nhận rằng tách hai nguyên nhân đó cần một thí nghiệm riêng.

Mọi phán quyết trong Phần F đều có hạn dùng. Chúng chốt tại tháng 9/2026 và nói về *trạng thái bằng chứng*, không nói về giới hạn của cái khả dĩ. Mô hình nền hình học đang chuyển động nhanh, và một kết luận “chưa đáng” của hôm nay hoàn toàn có thể sai trong mười hai tháng.

Khung này được hiệu chỉnh cho người đã có hệ thống. Nếu bạn đang bắt đầu từ đầu, trên một cảm biến hoặc một miền mà hệ đặc trưng thừa vốn không chạy nổi, thì thứ tự ưu tiên đảo lại: một hệ đầu-cuối có thể là con đường nhanh nhất tới thứ chạy được, và tính chẩn đoán được là món xa xỉ bạn mua sau.

1.8 Thực hành

Repo và tài nguyên. Dựng lại nền so sánh trước khi thử bất cứ thứ gì: [UZ-SLAMLab/ORB_SLAM3](#) làm baseline, [MichaelGrupp/evo](#) cho APE và RPE, và một harness chạy lập có ghi seed. Để thử nhanh phía học được mà không phải viết lại pipeline: [cvv/Hierarchical-Localization](#) cho tuyến đặc trưng và nhận dạng địa điểm, [cvv/glue-factory](#) cho ghép cặp, [kornia](#) cho các phép hình học khả vi. Phía triển khai: TensorRT và các công cụ đo trên Jetson. Bản đồ hướng của mảng này thì xem kho [changh95/visual-slam-roadmap](#). Danh sách tên mô hình dẫn đầu chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; hạ tầng như COLMAP, evo, TensorRT thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Với hệ hiện tại của bạn, oracle nào nên chạy *đầu tiên*, và nó chặn trên cái gì? Nếu recall@1 của tầng lấy ứng viên tăng mạnh mà số vòng đóng không đổi, ba cách giải thích là gì và phép thử nào tách được chúng? Một mạng báo một tốc độ khung hình rất cao trên GPU để bàn thì con số đó nói được gì — và không nói được gì — về Orin ở batch bằng 1, và bạn sẽ đo phần nó không nói bằng cách nào? Một can thiệp mức 1 và một can thiệp mức 2 cùng hứa cải thiện như nhau: chọn cái nào và vì sao?

Mini project. Hai khối dự án ngay dưới đây là phần thực hành chính của chương; chạy chúng theo thứ tự, vì khối thứ hai dùng lại số đo của khối thứ nhất.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Trụ 1 — frontend nhận thức	Mạng sinh ra phép đo: điểm, mô tả, tương ứng, ứng viên đóng vòng. Giao diện với backend không đổi, nên đây là trụ rủi ro thấp nhất và chín nhất.
••• Trụ 2 — backend tối ưu	Mạng không đổi phép đo mà đổi <i>trọng số</i> của nó: hiệp phương sai học được, mô hình nhiễu IMU, hoặc bộ tối ưu khả vi. Nằm hoàn toàn bên trong bộ ước lượng cổ điển.
••• Trụ 3 — hệ đầu-cuối	Ảnh vào, pose ra, không còn bước hình học tường minh. Được tính bền, mất toàn bộ đại lượng trung gian để chẩn đoán.
•• Trụ 4 — hiểu cảnh	Mạng quyết định phép đo nào được phép vào, thường bằng cách loại điểm trên vật thể động. Nó chỉ trừ đi, nên mạng chết thì hệ về đúng hành vi cổ điển.
•• Trụ 5 — mô hình nền và bản đồ neural	Hai thứ khác nhau bị gộp tên: tiên nghiệm học sẵn cho độ sâu và đặc trưng, và việc đổi bản đồ thành trường liên tục hay tập splat. Cái đầu nhắm độ bền phép đo, cái sau nhắm chất lượng ảnh dựng lại.
••• Trần đóng góp	Mức cải thiện tối đa của một khối, đo bằng cách thay khối đó bằng chân trị rồi chạy cả hệ. Nếu trần là c thì mọi phương pháp cho khối đó đều bị chặn trên bởi đúng c , bất kể bảng của họ đẹp tới đâu — và c phải do bạn đo, không do ai báo.
••• Sàn nhiễu	Độ lệch chuẩn giữa các lần chạy cùng một cấu hình. Nó là đơn vị đo của mọi so sánh; cải thiện nhỏ hơn nó thì không xác nhận được, nên ứng viên đó trượt cổng 2.
••• Tỷ lệ hoàn thành chuỗi	Phần trăm khung hệ thống còn bám được. Phải báo cùng APE, vì một hệ mất bám giữa chừng chỉ được chấm trên phần dễ và vì thế có APE đẹp giả.
•• Batch bằng 1	Chế độ chạy bất buộc của SLAM thời gian thực: mỗi lần đúng một khung. GPU chỉ đạt hiệu suất cao ở batch lớn, nên FPS trong paper thường lệch rất xa thực tế của bạn, xa tới mức không dùng để kết luận được.
••• Hông im lặng	Khối vẫn trả về đầu ra trông bình thường, kèm điểm tin cậy bình thường, trong khi đầu ra đó sai. Nguy hiểm vì SLAM là vòng phản hồi: phép đo sai ở lại trong bản đồ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Bề mặt chẩn đoán	Số đại lượng trung gian mà bạn còn kiểm tra được bằng một kỳ vọng vật lý: số inlier, sai số chiều ngược, hiệp phương sai. Nó sứt gần như dựng đứng khi lên mức can thiệp 3.
●●● Bốn tầng của đóng vòng	Lấy ứng viên, ghép đặc trưng, kiểm chứng hình học bằng Sim3 RANSAC, rồi tối ưu. Một vòng chỉ đóng khi qua đủ bốn, nên trước khi chọn phương pháp phải biết tầng nào đang chết.
●● Ba mức can thiệp	Bổ trợ (mạng chỉ cấp thêm tín hiệu), thay một khối (giữ nguyên giao diện), thay cả hệ. Chúng khác nhau chủ yếu ở thứ phải từ bỏ và ở khả năng quay lại.
●●● Vì sao đọc bảng thí nghiệm trước phần phương pháp?	Vì phần phương pháp nói tác giả <i>định</i> làm gì, còn bảng thí nghiệm nói điều đó đã được kiểm chứng tới đâu. Với mảng này khoảng cách giữa hai thứ rất lớn, nên đọc ngược tiết kiệm phần lớn thời gian.
●●● Vì sao không suy ngân sách Orin từ FPS trong paper?	Vì FPS là số thông lượng, đo ở batch lớn với dữ liệu sẵn trên GPU. SLAM chạy batch bằng 1, trả thêm chi phí chuyển dữ liệu, chia bộ nhớ với bản đồ, và chậm dần khi máy nóng.
●●● Vì sao vòng đóng sai đắt hơn vòng bỏ lỡ?	Bỏ lỡ chỉ để lại độ trôi, còn sửa được sau. Đóng sai kéo lệch cả một đoạn bản đồ và gần như không cứu được, nên điểm vận hành phải lệch hẳn về phía precision.
●● Vì sao đi từ mức 1 lên, không nhảy thẳng mức 3?	Vì tính đảo ngược. Mức 1 gỡ bằng một cờ, mức 2 gỡ trong vài ngày, mức 3 thì sau vài tháng mọi thứ quanh nó đã xây theo nó. Chi phí thật của mức 3 là chi phí của một quyết định không thu hồi được.
●●● Recall@1 tăng mạnh mà số vòng đóng không đổi — nguyên nhân?	Tầng chết không phải tầng lấy ứng viên. Thường là Sim3 RANSAC không hội tụ vì quá ít tương ứng đúng. Phân biệt: cấp ứng viên đúng từ chân trị vị trí rồi đếm xem bao nhiêu ứng viên hoàn hảo vẫn không đóng được vòng.
●●● APE đẹp bất thường trên chuỗi khó — nghĩa là gì?	Nghi ngay mắt bắm giữa chừng: hệ chỉ được chằm trên đoạn dễ. Phân biệt bằng tỉ lệ hoàn thành chuỗi; nếu nó dưới 100% thì con số APE kia không so sánh được với ai.
●● Mạng löt cồng 3 lúc thử nhưng bỏ khung sau mười phút — nguyên nhân?	Nhiệt và chế độ điện. Chu kỳ trôi khi máy nóng, nên số đo trong ba mươi giây đầu không phải số đo lúc vận hành. Phân biệt: đo lại phân vị 95 sau mười phút chạy liên tục ở đúng chế độ điện.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
hloc [19]	Trụ 1 ở mức 2: thay cả tầng lấy ứng viên lẫn tầng ghép cặp bằng khối học được, giữ nguyên phần kiểm chứng hình học	Bảng chứng rằng mức 2 đã chín cho định vị ngoại tuyến; chưa phải cho vòng bám thời gian thực
DynaSLAM [10] và các nhánh cùng ý tưởng	Trụ 4 ở mức 1: mặt nạ ngữ nghĩa chỉ <i>loại</i> phép đo, không thêm gì vào bộ tối ưu	Ví dụ sạch nhất của can thiệp đảo ngược được bằng một cờ
Kimera [11]	Trụ 4: tầng ngữ nghĩa dựng <i>cạnh</i> bộ ước lượng hình học chứ không thay nó	Kiến trúc đáng học kể cả khi không dùng phần ngữ nghĩa
DROID-SLAM [9]	Trụ 3 ở mức 3: xoá chuỗi đặc trưng thừa, giữ lại một lõi tối ưu khả vi	Mốc tham chiếu của mức 3; ngân sách GPU vượt xa Orin theo cấu hình gốc
MASt3R-SLAM [13]	Trụ 5: mô hình nền hình học làm nguồn tương ứng và độ sâu cho một vòng SLAM	Rất mới; ở đây chỉ nhắc ý tưởng, không dẫn con số
TLIO [20]	Trụ 2: học mô hình quán tính và độ bất định của nó thay vì lấy từ bảng thông số cảm biến	Nhóm gần nhất với nỗi đau trọng số IMU

MINI PROJECT SLAM · Bảng trần đóng góp bốn khối — đo trước khi đọc thêm paper nào · 2 ngày

Mục tiêu	biến câu hỏi “nên đầu tư vào đâu” thành một bảng bốn dòng có số, để ba tháng tới đi vào đúng một chỗ.
Dữ liệu	ba chuỗi EuRoC với độ khó khác nhau, trong đó phải có một chuỗi bạn biết là hay hỏng; cấu hình mặc định của chính nhánh bạn.
Các bước	trước hết đo sàn nhiều: chạy cùng cấu hình mười lần mỗi chuỗi, ghi APE cùng tỉ lệ hoàn thành, lấy độ lệch chuẩn. Rồi chạy bốn biến thể oracle. Một: dùng pose chân trị chiều ngược để loại mọi tương ứng sai, đo trần của ghép cặp. Hai: cấp ứng viên đóng vòng từ khoảng cách vị trí chân trị, đo trần của tầng lấy ứng viên. Ba: quét hệ số nhân của ma trận thông tin IMU trên lưới thô rồi lấy giá trị tốt nhất mỗi chuỗi, đo trần của việc học trọng số. Bốn: bỏ mọi khung có chỉ số nhỏ vượt ngưỡng, đo phần APE quy được về ảnh mờ.
Xong khi	bạn có một bảng bốn dòng, mỗi dòng ghi mức cải thiện trần kèm sai số, và bạn khoanh được đúng những dòng có trần lớn hơn hai lần sàn nhiều. Kiểm tra chéo: tổng bốn trần phải lớn hơn khoảng cách giữa hệ của bạn và một hệ hoàn hảo; nếu nhỏ hơn thì có một nguồn sai số thứ năm bạn chưa gọi tên.
Nối với	E/24 cho công thức cỡ mẫu và cho khái niệm oracle; bảng này là đầu vào bắt buộc của khối dự án tiếp theo và của cả bảy chương sau.

MINI PROJECT SLAM · Ngân sách Orin, đo trước khi viết dòng code tích hợp nào · 1 ngày

Mục tiêu	biến cổng 3 từ một phỏng đoán thành một con số, và biết trước một ứng viên có lọt hay không.
Dữ liệu	chính Jetson Orin của bạn, ở đúng chế độ điện dùng khi vận hành; một chuỗi EuRoC chạy được ít nhất mười phút liên tục.
Các bước	lập hồ sơ thời gian luồng bám theo từng khối: trích đặc trưng, ghép cặp, RANSAC, tối ưu pose, tiền tích phân IMU. Ghi cả trung vị lẫn phân vị 95, vì phân vị mới là thứ làm bỏ khung. Tính phần ngân sách còn trống so với trần 50 mili-giây tại 20 Hz. Rồi lấy một mạng ứng viên, chuyển sang TensorRT ở độ chính xác nửa, và đo độ trễ ở batch bằng 1 trên đúng máy đó. Đo hai lần: lúc máy nguội và sau mười phút chạy liên tục.
Xong khi	bạn phát biểu được một câu có ba con số: “luồng bám đang tiêu X mili-giây ở phân vị 95, còn trống Y , mạng tốn Z ở batch 1 sau khi máy nóng — do đó GO hoặc NO-GO”. Kiểm tra chéo: chênh lệch giữa số lúc nguội và lúc nóng phải khác không; nếu bằng nhau thì bạn chưa đo đúng chế độ điện.
Nối với	D/21-inference-va-trien-khai cho phần chuyển đổi và lượng tử hoá; B/11-gioi-han-tinh-toan cho lý do vì sao batch bằng 1 lại xa peak tới vậy; chương F/34 cho phần đo lại cho trung thực trên phần cứng nhúng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bạn đã cải thiện tầng không hỏng. Vòng đóng có bốn tầng nối tiếp, và nhận dạng địa điểm chỉ chạm tầng đầu. Lẽ ra phải đếm số ứng viên sống sót sau *từng* tầng trước, với bộ đếm đặt sau câu lệnh điều kiện chứ không phải trước.
- A2. Ba câu: chuỗi dài nhất họ chạy dài bao nhiêu, đo trên phần cứng nào ở batch bao nhiêu, và baseline có được tinh chỉnh không. Câu thứ nhất loại được nhiều paper nhất, vì nó trả lời được trong ba phút bằng cách nhìn đúng một bảng.
- A3. Vì SLAM là vòng phản hồi còn phân lớp ảnh thì không. Một dự đoán sai của bộ phân lớp làm sai đúng một mẫu. Một phép đo sai trong SLAM đi vào bản đồ, rồi bản đồ quyết định phép đo tiếp theo; với một vòng đóng sai thì cả đoạn quỹ đạo lệch vĩnh viễn.
- A4. Vì đó là số thông lượng ở batch lớn, còn bạn chạy batch bằng 1 và bị chặn bởi độ trễ. Thêm vào đó: chi phí chuyển dữ liệu, GPU có thể đã bận với phần trích đặc trưng, bộ nhớ dùng chung với bản đồ, và chu kỳ trôi khi máy nóng. Cách duy nhất là đo trên chính Orin của bạn.

- A5. Bảng trần đóng góp. Thay lần lượt từng khối bằng chân trị rồi đo cả hệ; dòng nào có trần lớn hơn hai lần sàn nhiễu thì đáng, còn lại thì không. Bảng đó chạy trong hai ngày và nó quyết định ba tháng.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn có một bếp ăn chạy tốt nhiều năm. Bạn biết từng vị trí, biết ai chậm ở đâu, biết khi nào món ra trễ thì phải nhìn chỗ nào. Một người bán hàng mang tới cái máy thái rau rất nhanh, đẹp hơn tay người, và có clip chứng minh.

Trước khi mua, bạn hỏi bốn câu. Một: món ra trễ có thật sự vì khâu thái rau không, hay vì cái bếp lò? Cách biết duy nhất là bấm giờ từng khâu trong một tuần. Hai: nếu mua, tôi nhận ra cái thiện bằng gì, khi mà mỗi tối bếp lại đông khác nhau? Ba: cái máy có vừa mặt bàn và vừa cầu dao không, lúc mọi thứ khác cũng đang bật? Bốn: khi gặp một loại rau nó chưa từng thấy, nó dừng lại hay vẫn đẩy ra một mớ trông giống rau đã thái?

Câu thứ tư là câu đắt nhất. Một người thái rau sẽ dừng và hỏi. Cái máy thì không biết dừng, và thứ nó đẩy ra sẽ đi thẳng vào nồi. Trong một cái bếp mà món này nấu tiếp từ món kia, một sai lầm im lặng lan ra khắp bữa ăn.

Rồi còn chuyện lắp sâu tới đâu. Đặt cái máy cạnh bàn cũ thì hôm nào hỏng bạn rút phích. Bỏ hẳn bàn cũ thì phải chờ mua bàn mới. Đập cả bếp đi xây lại quanh cái máy thì không quay về được nữa. Cái giá không nằm ở tiền mua máy. Cái giá là mỗi cái máy lắp thêm là một chỗ bạn không còn nhìn vào bên trong được, khi món ra sai mà không hiểu vì sao.

Chỗ không rút gọn được: chỗ bốn nỗi đau chồng lên nhau. Nói “chứa ảnh mờ thì sai số giảm” nghe gọn nhưng sai, vì đoạn ảnh mờ cũng là đoạn xoay nhanh, và xoay nhanh mới là thứ làm mất bám. Chia phần trách nhiệm giữa hai nguyên nhân cùng xảy ra một lúc thì hiện chưa có cách làm gọn, và mọi cách nói đơn giản về nó đều đang giấu bước đó đi.

Một câu: Trước khi hỏi mạng nào tốt hơn, hãy đo xem khối nào trong hệ của bạn đang thật sự làm hỏng việc, và bạn còn bao nhiêu mili-giây để trả cho nó.

1.9 Kết nối

B/03-giai-phau-he-thong là tiền đề trực tiếp: năm trụ chỉ là câu hỏi “độ khó đã bị đẩy đi đâu” đặt cho một hệ SLAM, và ranh giới giữa các khối chính là chỗ một mạng chen vào được. E/24-thi-nghiem-va-ablation cấp hai công cụ mà cổng 1 và cổng 2 dùng lại nguyên vẹn, là oracle experiment và sàn nhiễu. B/07-chia-du-lieu-va-danh-gia là nơi đọc ý nghĩa của các con số, đặc biệt phần precision và recall cho tầng lấy ứng viên đóng vòng. B/08-dich-chuyen-phan-phoi là mặt lý thuyết của cổng 4: nó giải thích vì sao độ chính xác tụt khi ra ngoài phân phối, còn chương này quan tâm chuyện khác — vì sao bạn không biết là nó đã tụt. B/11-gioi-han-tinh-toan và D/21-inference-va-trien-khai là hai chương phải mở khi làm cổng 3.

Về phía trước, bảy chương còn lại của Phần F đều dùng khung này. F/28 và F/30 là trụ 1. F/29 là nỗi đau đóng vòng, và nó bắt đầu đúng bằng phép đếm bốn tầng của mạch 2. F/31 là trụ 2 và trụ 3. F/32 là trụ 5, đọc với cảnh giác rằng chất lượng ảnh dựng lại không phải độ chính xác quỹ đạo. F/33 là trụ 4. F/34 gom hai nỗi đau còn lại, trọng số IMU và ảnh mờ, cộng phần đưa lên phần cứng. Cuối cùng, E/26-lo-trinh-hoc cho biết nên đọc bảy chương đó theo thứ tự nào tùy mục tiêu của bạn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao trần đóng góp phải đo trước khi đọc paper, chứ không phải sau khi đã chọn được phương pháp ứng viên?

2. Một can thiệp mức 1 và một can thiệp mức 2 hứa cùng mức cải thiện: bạn chọn cái nào, và lập luận của bạn dựa vào đại lượng nào chứ không dựa vào độ chính xác?
3. Điều gì làm một phép đo sai mà im lặng đắt hơn hẳn trong SLAM so với trong một bài toán phân lớp ảnh, và cơ chế nào của hệ thống gây ra sự khác biệt đó?
4. Bốn tầng của đóng vòng có thể chết ở tầng nào, và với mỗi tầng thì nhóm phương pháp nào chữa đúng chỗ — vì sao chọn nhầm tầng lại cho ra hiện tượng “metric tăng mà kết quả không đổi”?
5. Phép tính ngân sách ở cổng 3 hỏng trong trường hợp nào, và sai lệch của nó đi theo hướng lạc quan hay bi quan?
6. Nếu bốn nỗi đau không độc lập thì việc quy một phần APE về đúng một nỗi đau còn có nghĩa không, và bạn sẽ phát biểu kết luận theo dạng nào để vẫn dùng được?
7. Khung bốn cổng lệch về phía từ chối: cơ chế nào trong quy trình làm việc của bạn ngăn nó biến thành một định kiến lâu dài?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., “Image Matching Across Wide Baselines: From Paper to Practice,” *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., “RoMa: Robust Dense Feature Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., “Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., “HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, “Closed-form solution of absolute orientation using unit quaternions,” *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., “Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, “Rethinking Visual Geo-localization for Large-Scale Applications,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, “MixVPR: Feature Mixing for Visual Place Recognition,” in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., “EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, “Optimal Transport Aggregation for Visual Place Recognition,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., “AnyLoc: Towards Universal Visual Place Recognition,” *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Đặc trưng và ghép cặp học được (Learned Features and Matching)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng — chương này là bài tập lớn đầu tiên của khung đó); C/16 cho *attention* (SuperGlue và LoFTR đều là *transformer*); E/24-thi-nghiem-va-ablation cho *oracle* experiment, công cụ quyết định của chương này.

TÓM TẮT

Chương này trả lời một câu hỏi rất hẹp: có nên thay ORB bằng một bộ đặc trưng học được không. Câu trả lời cần ba thứ. Thứ nhất là chẩn đoán: ORB gậy ở đúng bốn chỗ, và cả bốn đọc được từ cơ chế của FAST và BRIEF. Thứ hai là bản đồ phương pháp: SuperPoint, R2D2, DISK, XFeat, ALIKED cho đặc trưng; SuperGlue và LightGlue cho ghép cặp; LoFTR và RoMa bỏ hẳn bộ phát hiện. Thứ ba là cổng đánh giá, và đây là phần quan trọng nhất. Đọc xong bạn nói được bằng cơ chế vì sao ảnh mờ giết bộ *phát hiện* còn đối độ sáng giết bộ *mô tả*, viết được công thức chi phí bậc hai của SuperGlue theo số điểm, và biết chính xác những gì trong ORB-SLAM3 phải bỏ đi để nhận một bộ đặc trưng mới. Nếu chỉ nhớ một điều: tính lặp lại, độ chính xác ghép và tỉ lệ inlier đều có thể tăng mạnh mà APE không nhúc nhích, nên hãy chạy *oracle* ghép cặp *trước* khi tích hợp bất cứ mạng nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FAST** — bộ phát hiện góc của ORB. Nó xét 16 điểm ảnh trên một vòng tròn bán kính 3 quanh điểm p , và nhận p là góc nếu có ít nhất 9 điểm liên tiếp cùng sáng hơn $I_p + t$ hoặc cùng tối hơn $I_p - t$, với t là ngưỡng tương phản [22].
- **BRIEF** — bộ mô tả của ORB. Nó lấy 256 cặp điểm cố định trong một ô ảnh nhỏ quanh góc, mỗi cặp trả về một bit “điểm A sáng hơn điểm B hay không” [23]. ORB thêm bước xoay bộ cặp theo hướng của góc.
- **khoảng cách Hamming** — số bit khác nhau giữa hai chuỗi nhị phân. Với ORB thì hai bộ mô tả 256 bit gần nhau nghĩa là hai ô ảnh trông giống nhau.
- **tính lặp lại (repeatability)** — tỉ lệ điểm được bộ phát hiện tìm thấy ở cùng một vị trí trên vật thật trong hai ảnh khác nhau. Đây là thước đo của riêng bộ phát hiện, chưa dính gì tới bộ mô tả.
- **độ chính xác ghép trung bình (MMA)** — trong các cặp được ghép, tỉ lệ cặp có sai số vị trí dưới một ngưỡng điểm ảnh cho trước. Đây là thước đo chung của cả bộ mô tả lẫn bộ ghép cặp.
- **tỉ lệ inlier** — tỉ lệ cặp ghép sống sót sau khi RANSAC ước lượng mô hình hình học. Nó đo mức sạch của tập ghép cặp đưa vào tối ưu.
- **APE** — sai số vị trí tuyệt đối của quỹ đạo so với chân trị, sau khi căn chỉnh hai quỹ đạo. Đây là con số cuối cùng mà một hệ SLAM bị đánh giá, và là thứ duy nhất trong bốn thước đo trên thật sự quan trọng.
- **vận tải tối ưu (optimal transport)** — bài toán tìm cách chuyển một phân phối khối lượng sang một phân phối khác với tổng chi phí nhỏ nhất. Ghép hai tập điểm ảnh với ràng buộc mỗi điểm dùng một lần chính là một bài toán như vậy.

CÂU HỎI MỞ ĐẦU

- Q1. Ảnh mờ giết FAST qua đúng một đại lượng. Đại lượng đó là gì, những tham số nào của hệ bạn quyết định ngưỡng, và bạn đổi ngưỡng đó sang tốc độ quay bằng phép chia nào?
- Q2. BRIEF chỉ so sánh “A sáng hơn B hay không”, nên nó bất biến với mọi phép đổi độ sáng đơn điệu. Vậy vì sao đổi độ sáng vẫn giết nó trong thực tế?
- Q3. SuperGlue tốn chi phí tỉ lệ N^2 theo số điểm. Hệ SLAM của bạn đã có một dự đoán chuyển động từ IMU. Vì sao dự đoán đó biến N^2 thành gần như tuyến tính, và vì sao không paper nào tận dụng nó?
- Q4. Bạn thay ORB bằng SuperPoint và thấy số cặp ghép đúng tăng mạnh, tỉ lệ inlier cũng tăng theo. Vì sao APE vẫn có thể không đổi một milimét nào?
- Q5. Nếu bỏ bộ phát hiện đi thì bức tường trắng có điểm ghép. Vậy vì sao không hệ SLAM thời gian thực nào bỏ bộ phát hiện?

2.1 Đặt vấn đề

Frontend là khối bị đề nghị thay bằng mạng nhiều hơn mọi khối khác trong SLAM. Lý do dễ hiểu. Nó đứng ngoài cùng, nó chạm trực tiếp vào điểm ảnh, và nó là chỗ mà bạn *nhìn thấy* hệ thống chết. Khi camera quay nhanh và tracking mất, thứ hiện ra trên màn hình gỡ lỗi là số cặp ghép tụt từ vài trăm xuống một nhúm. Kết luận trực giác là frontend yếu.

Kết luận đó đúng một nửa. Frontend đúng là yếu ở bốn chỗ cụ thể, và chương này chỉ ra chính xác bốn chỗ đó bằng cơ chế chứ không bằng cảm giác. Nhưng nửa còn lại quan trọng hơn: frontend hiếm khi là ràng buộc chặt nhất của độ chính xác quỹ đạo. Số cặp ghép chỉ tụt xuống mức nguy hiểm trong những khoảng rất ngắn của một chuỗi dài nhiều phút, và tỉ lệ giữa hai khoảng thời gian đó là con số đầu tiên bạn phải tự đo. Trong phần còn lại, ORB đã cho nhiều cặp ghép hơn mức tối ưu hoá cần.

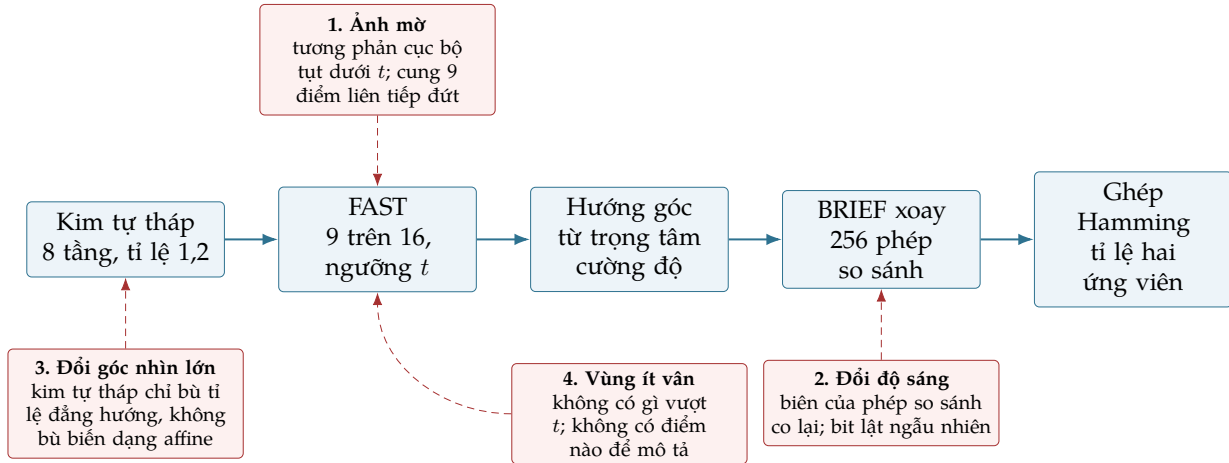
Vì thế câu hỏi đúng không phải “bộ đặc trưng nào tốt hơn”. Câu hỏi đúng là: *trong bao nhiêu phần trăm thời gian của chuỗi, frontend là thứ chặn?* Và trong khoảng thời gian đó, một frontend hoàn hảo thì cứu được bao nhiêu? Chương 24 đã cho công cụ trả lời câu này, đó là *oracle* experiment. Chương này chỉ áp nó vào một khối cụ thể.

Cách trình bày vì thế đi theo thứ tự chẩn đoán rồi mới tới thuốc. Mục 2 mổ xẻ bốn chế độ hỏng của ORB bằng chính công thức của FAST và BRIEF, kèm hai phép tính tay ra con số. Mục 3 và 4 là bản đồ phương pháp, tách làm hai vì đặc trưng và ghép cặp là hai bài toán khác nhau và tiến hoá tách nhau. Mục 5 nói về nhánh bỏ hẳn bộ phát hiện. Mục 6 là cổng đánh giá và là mục nên đọc trước nếu bạn chỉ có mười lăm phút.

→ *tiền đề* F/27 — bốn cổng của chương 27 — bài toán có thật, đo được, vừa ngân sách, hông có kiểm soát — là khung mà cả chương này đang điền vào

2.2 Bốn chế độ hỏng của ORB, đọc bằng cơ chế (Four Failure Modes)

ORB là hai thứ ghép lại: một bộ phát hiện góc FAST và một bộ mô tả nhị phân BRIEF có xoay [24]. Bốn chế độ hỏng phân bố không đều giữa hai thứ đó, và biết cái nào hỏng ở đâu quyết định bạn nên thay cái gì. Hình 2.1 đặt cả bốn lên đúng vị trí của chúng trong đường ống.



Hình 2.1. Bốn chế độ hỏng của ORB gắn vào đúng khối mà chúng phá. Hai chế độ đầu tấn công hai khối khác nhau, nên chúng đòi hai thuốc khác nhau: mờ giết bộ *phát hiện*, đổi sáng giết bộ *mô tả*. Chế độ 3 là giới hạn của bất biến kim tự tháp. Chế độ 4 không phải lỗi cài đặt mà là hệ quả tất yếu của việc bắt bộ phát hiện quyết định sớm và cục bộ.

A. Ảnh mờ do chuyển động giết bộ phát hiện, và đại lượng quyết định là gì

FAST hỏi một câu hoàn toàn cục bộ: quanh điểm này, có chín điểm ảnh liên tiếp trên vòng tròn bán kính 3 nào lệch hơn ngưỡng t không. Mờ do chuyển động là một phép lấy trung bình dọc theo hướng chuyển động. Trung bình làm giảm biên độ của mọi biến thiên có bước sóng ngắn hơn cửa sổ trung bình.

Mô hình một chiều đủ để thấy hết cơ chế. Gọi cấu trúc ảnh quanh góc có bề rộng w điểm ảnh và biên độ tương phản C mức xám. Làm mờ bằng một cửa sổ dài d điểm ảnh giữ lại xấp xỉ

$$C_{\text{hiệu dụng}} \approx C \cdot \min\left(1, \frac{w}{d}\right).$$

Góc còn được phát hiện chừng nào $C_{\text{hiệu dụng}} > t$. Cho hai vế bằng nhau thì được độ dài vết mờ tới hạn:

$$d^* \approx \frac{wC}{t}.$$

Công thức này nói ba điều, và cả ba dùng được ngay mà không cần một con số nào. Thứ nhất, dung sai mờ tỉ lệ *ngược* với ngưỡng t : hạ ngưỡng thì mua thêm dung sai theo tỉ lệ, chứ không phải bù trừ chút ít. Thứ hai, dung sai tỉ lệ *thuận* với bề rộng cấu trúc, nên cấu trúc thô sống lâu hơn cấu trúc mịn — và đó là lý do khi mờ thì tập điểm còn sót lại toàn góc thô, những góc định vị kém, làm sai số tái chiếu tăng ngay cả khi số cặp ghép chưa tụt. Thứ ba, dung sai tỉ lệ thuận với tương phản, nên cùng một chuyển động sẽ giết bộ phát hiện sớm hơn hẳn trong cảnh thiếu sáng.

Đổi d^* sang đơn vị vật lý cũng chỉ là một phép chia. Một chuyển động quay làm ảnh trượt d điểm ảnh trong thời gian phơi sáng Δt ứng với tốc độ góc

$$\omega \approx \frac{d/f}{\Delta t},$$

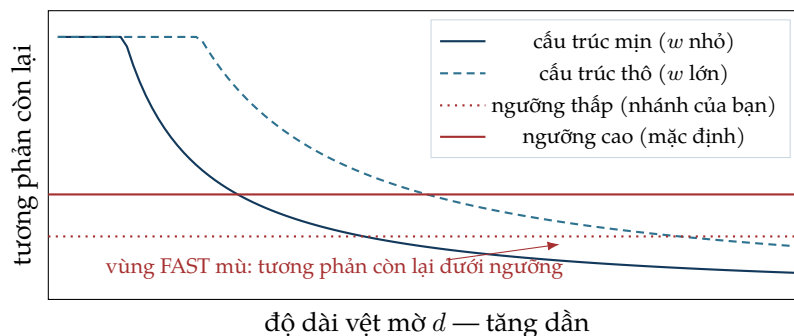
với f là tiêu cự tính bằng điểm ảnh. Ghép hai công thức lại thì ngưỡng tốc độ góc của *hệ bạn* tỉ lệ với $wC/(t f \Delta t)$.

Chương này cố ý dừng ở đó và không đưa một con số. Lý do rất đơn giản: w , C và Δt thật của chuỗi bạn chạy thì mình chưa đo, và một con số chưa đo thì không đáng nằm trong tài liệu học — dán nhãn “ước lượng” cũng không cứu được, vì con số vẫn sẽ được nhớ và được trích lại. Bốn đại lượng ở vế phải đều nằm trong tầm tay bạn: ngưỡng FAST nằm trong cấu hình, tiêu cự nằm

trong file hiệu chỉnh, thời gian phơi sáng nằm trong nhật ký camera, còn w và C thì đo được từ chính ảnh. Thay chúng vào là bạn có ngưỡng của riêng mình trong vòng mười phút, và có nó bằng số đo chứ không bằng lời của ai.

Cái đáng mang đi là một dự đoán định tính, và nó đủ sắc để kiểm chứng được: ngưỡng ấy nằm trong tầm tốc độ quay của một người cầm camera đi bộ bình thường, chứ không phải một tình huống hiểm — nói cách khác, các đợt xoay tay bình thường đã đủ đẩy FAST vào vùng mù, và các chuỗi khó của EuRoC thì vào vùng đó liên tục. Dự án thứ hai ở cuối chương là cách kiểm chứng dự đoán đó trên dữ liệu của chính bạn, và nó chạy trong nửa ngày.

Hình 2.2 vẽ quan hệ đó dưới dạng thuần hình dạng, không một vạch số nào. Điều đáng chú ý nhất trong hình không phải đường cong mà là độ dốc: quanh vùng ngưỡng, giảm t một chút mua thêm được khá nhiều dung sai mờ. Đó chính là lý do hạ iniThFAST từ 20 xuống 12 có tác dụng thật, và cũng là lý do nó có giới hạn — hạ tiếp thì nhiều bắt đầu qua cửa.



Hình 2.2. Tương phản còn lại sau khi làm mờ, theo mô hình trung bình một chiều. Trục cố ý không có vạch số: hình này nói về *hình dạng*, không về một mức mờ cụ thể nào. Ba điều đọc được từ hình: giao điểm với đường ngưỡng là chỗ FAST ngừng thấy góc; hạ ngưỡng đẩy giao điểm sang phải khá nhiều, nên nó có tác dụng thật; và cấu trúc thô sống lâu hơn cấu trúc mịn, nên khi mờ thì các góc còn sót lại đều là góc thô — chúng định vị kém hơn, và sai số tái chiếu tăng ngay cả khi số cặp ghép chưa tụt. Muốn có con số thì thay w, C, t của chính bạn vào $d^* = wC/t$.

Có một hệ quả tinh tế mà mô hình trên chưa nói hết. FAST không đòi biên độ lớn, nó đòi *chín điểm liên tiếp*. Mờ do chuyển động là bất đẳng hướng: nó xóa tương phản theo hướng chuyển động và giữ nguyên theo hướng vuông góc. Nên với một góc, cung liên tiếp bị cắt đứt ở hai đầu trước khi biên độ trung bình chạm ngưỡng. Nói cách khác, FAST chết sớm hơn mô hình một chiều dự đoán, và chết theo cách phụ thuộc vào hướng của cạnh so với hướng quay. Đó là lý do khi quay ngang thì các cạnh dọc biến mất trước, và tập điểm còn lại bị lệch về một hướng — một tập điểm lệch hướng làm ma trận thông tin của bài toán tư thế bị điều kiện xấu, chứ không chỉ làm ít điểm đi.

B. Đổi độ sáng giết bộ mô tả, qua biên chứ không qua giá trị

Đây là chỗ trực giác thông thường sai. BRIEF không lưu giá trị cường độ; nó chỉ lưu 256 kết quả so sánh dạng $I(a) > I(b)$. Một phép biến đổi độ sáng *đơn điệu tăng* và *toàn cục* — chỉnh gamma, đổi thời gian phơi sáng — không đảo được dấu của bất kỳ phép so sánh nào. Về lý thuyết, BRIEF bất biến hoàn toàn với nhóm biến đổi đó.

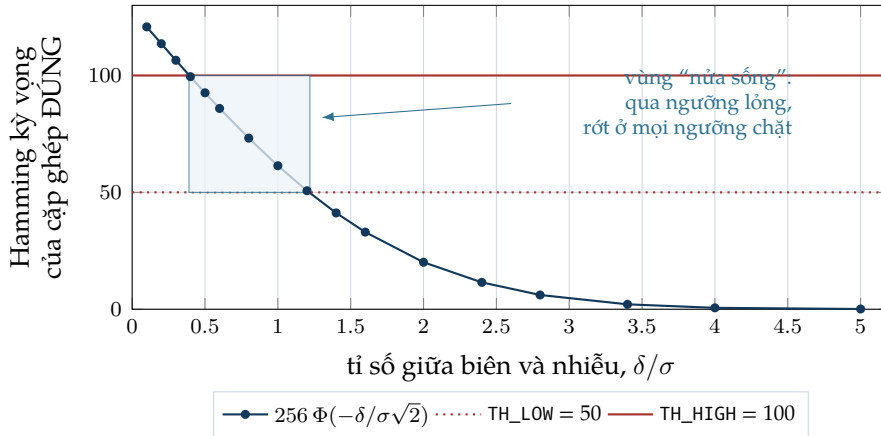
Thứ giết nó là biên. Gọi δ là chênh lệch cường độ thật của một cặp thử, và σ là độ lệch chuẩn của nhiễu trên mỗi mẫu đã làm trơn. Bit lật khi nhiễu đủ lớn để đảo dấu, với xác suất

$$p_{\text{lật}} = \Phi\left(-\frac{\delta}{\sigma\sqrt{2}}\right).$$

Khoảng cách Hamming kỳ vọng giữa hai bộ mô tả của *cùng một* điểm 3D là $256 p_{\text{lật}}$. Đây là đại lượng phải so với hai hằng số mà bạn đã có trong mã: TH_LOW = 50 và TH_HIGH = 100.

Đọc công thức đó theo tỉ số δ/σ thì mọi giá trị tuyệt đối biến mất và cái còn lại mới là thứ dùng được. Khi biên lớn hơn nhiều vài lần, xác suất lật gần như bằng không và khoảng cách Hamming của cặp đúng chỉ vài bit — ghép cặp thoải mái. Khi biên tụt về cùng bậc với nhiễu, Φ bước vào vùng dốc nhất của nó, nên Hamming kỳ vọng nhảy vọt chỉ sau một mức nén tương phản rất nhỏ.

Điều quan trọng không phải nó nhảy bao nhiêu, mà nhảy *vào đâu*. Hai hằng số trong mã chia trực Hamming thành ba vùng, và vùng giữa — trên TH_LOW, dưới TH_HIGH — ứng với một dải δ/σ rất hẹp. Trong dải đó, cặp ghép vẫn qua được mọi hàm dùng ngưỡng lỏng nhưng bị loại ở mọi hàm dùng ngưỡng chặt, tức là tam giác hoá điểm mới và ghép theo túi từ. Hình 2.3 cho thấy toàn bộ đường cong đó; vì trực hoành là một tỉ số không thứ nguyên, hình đúng với mọi mức nhiễu và mọi mức sáng, và bạn chỉ cần biết σ của cảm biến mình để đọc ra ngưỡng δ tương ứng.



Hình 2.3. Vì sao đổi độ sáng giết BRIEF dù BRIEF bất biến với đổi sáng đơn điệu. Đại lượng quyết định là tỉ số giữa biên δ và nhiễu σ , không phải giá trị tuyệt đối của độ sáng. Đường cong là hàm $256 \Phi(-\delta/\sigma\sqrt{2})$, tức là toán chứ không phải số đo; hai đường ngang là hai hằng số có thật trong mã nguồn. Điều đáng mang đi: dải δ/σ nằm giữa hai hằng số ấy rất hẹp, nên chỉ cần tương phản nén đi một chút là cặp ghép rơi từ “qua mọi cửa” xuống “chỉ qua cửa lỏng”. Hệ quả chẩn đoán: khi vào vùng thiếu sáng, thứ tụt trước là số điểm bản đồ mới được tam giác hoá, chứ không phải số cặp bám được.

Điều này giải thích một triệu chứng quen thuộc mà trực giác “đổi sáng làm hỏng mô tả” không giải thích được. Khi robot đi vào hành lang tối, tracking vẫn bám nhưng bản đồ ngừng lớn. Nguyên nhân nằm ở đúng khoảng cách giữa hai ngưỡng trong hình.

TRỰC GIÁC

Coi mỗi bit của BRIEF như một đồng xu có thiên lệch. Thiên lệch đến từ tương phản cục bộ: tương phản càng cao, đồng xu càng chắc chắn rơi về một mặt, và chuỗi 256 mặt đó là vân tay của điểm ảnh. Làm mờ ảnh hay nén tương phản không đổi *mặt* nào được ưa; nó chỉ làm mọi đồng xu công bằng dần. Một vân tay gồm toàn đồng xu công bằng thì khớp với tất cả mọi thứ, tức là không khớp với gì cả. Đó là lý do bộ mô tả không “sai” khi ảnh mờ — nó trở nên vô nghĩa, và hai chuyện này cần hai cách chữa khác nhau.

C. Đổi góc nhìn lớn vượt quá bất biến có sẵn

ORB có đúng hai loại bất biến. Bất biến xoay trong mặt phẳng, nhờ xoay bộ cặp thử theo hướng trọng tâm cường độ. Và bất biến tỉ lệ *đẳng hướng*, nhờ kim tự tháp tám tầng hệ số 1,2. Dải tỉ lệ mà kim tự tháp phủ là $1,2^7 \approx 3,6$ lần.

Đổi góc nhìn ngoài mặt phẳng không nằm trong hai loại đó. Nhìn một mặt phẳng nghiêng một góc θ thì ảnh của nó bị nén theo *một* hướng với hệ số $\cos \theta$, còn hướng vuông góc giữ nguyên. Đó là một biến dạng affine bất đẳng hướng. Kim tự tháp co đều cả hai hướng, nên nó không bù được.

Ở $\theta = 45^\circ$ hệ số nén là 0,71; ở 60° là 0,5. Văn liệu về vùng bất biến affine đã đo điều này từ lâu: điểm số ghép của các bộ mô tả không affine sụp rất nhanh khi góc nhìn ra khỏi vùng gần trực diện, trong khi các vùng bất biến affine giữ được lâu hơn hẳn [25]. Ngưỡng cụ thể phụ thuộc bộ mô tả và bộ dữ liệu, nên nó phải đọc từ chính bài đó chứ không mượn lại ở đây.

Với SLAM cầm tay đi liên tục thì chế độ hồng này hiếm, vì hai khung hình liên tiếp cách nhau vài chục mili-giây. Nó chỉ nổ ở đúng một chỗ: **đóng vòng**. Quay lại một hành lang theo hướng ngược lại là đổi góc nhìn tối đa. Đây là lý do đóng vòng và bám khung hình đòi hai loại đặc trưng khác nhau, và là lý do một hệ dùng chung một bộ mô tả cho cả hai việc luôn phải thỏa hiệp.

↪ **tiền đề F/29** — chế độ hồng thứ ba là nguyên nhân gốc của phần lớn vòng bị bỏ lỡ; chương 29 mô tả bốn tầng của đóng vòng để xem tầng nào thật sự giết vòng

D. Vùng ít vân: không phải lỗi, mà là hệ quả của thiết kế

Chế độ thứ tư khác hẳn ba chế độ trên. Ba chế độ đầu là chuyện ngưỡng và bất biến, có thể chỉnh. Chế độ thứ tư là hệ quả logic của việc có một bộ phát hiện.

Bộ phát hiện phải ra một quyết định **cứng**, **sớm** và **cục bộ**: điểm ảnh này có đáng giữ không, quyết bằng thông tin trong một cửa sổ bán kính 3. Trên một bức tường trắng, không cửa sổ nào chứa đủ thông tin để trả lời. Hạ ngưỡng xuống 4 thì cái được chọn là nhiều cảm biến, và nhiều không lặp lại giữa hai khung hình. Không có ngưỡng nào đúng, vì thông tin cần thiết không nằm trong cửa sổ đó — nó nằm ở mép tường cách xa hai trăm điểm ảnh.

Đây là điểm cần nhớ khi đọc mục 5. Bộ phát hiện không phải một thủ thuật kỹ thuật; nó là cách duy nhất để hoãn một quyết định mà thông tin cục bộ không đủ để quyết. Cái giá của việc hoãn thì được nói ở đúng mục đó.

E. Bốn chế độ cộng dồn thế nào trong một giây thật

Bốn chế độ ở trên không xảy ra riêng lẻ. Trong một đợt quay nhanh, chúng nối thành một chuỗi, và biết chuỗi đó giúp bạn đọc nhật ký gỡ lỗi nhanh hơn nhiều.

Bước một, tốc độ góc vượt ngưỡng suy ra từ d^* ở mục A, nên tương phản cục bộ tụt và FAST mất phần lớn góc nhỏ. Bước hai, các góc còn sót lại đều là cấu trúc thô, nên chúng định vị kém hơn và sai số tái chiếu của chúng lớn hơn. Bước ba, các góc còn lại phân bố lệch theo hướng vuông góc với chuyển động, nên ma trận thông tin tư thế xấu đi theo đúng một hướng — không phải xấu đều. Bước bốn, số cặp ghép rơi xuống dưới cổng “dưới 15 cặp thì coi như mất bám”, và hệ chuyển sang chế độ dùng IMU dự đoán.

Điểm đáng chú ý là bước năm. Khi hệ quay lại chế độ bám, nó phải ghép khung hình mới với các điểm bản đồ được tạo *trước* đợt mờ. Các điểm đó có bộ mô tả sắc nét, còn khung hình mới vẫn còn hơi mờ, nên khoảng cách Hamming của cặp đúng nằm trong vùng “nửa sống” của Hình 2.3. Kết quả là hệ bám lại chậm hơn mức đáng lẽ có, và trong khoảng chờ đó IMU tích lũy trôi. Phần lớn sai số quỹ đạo do một đợt quay nhanh gây ra không nằm trong lúc quay, mà nằm trong khoảng *sau* khi quay xong.

Chuỗi này có một hệ quả thực hành ngay. Nếu bạn định can thiệp một chỗ duy nhất, hãy can thiệp ở bước năm chứ không ở bước một — nơi ngưỡng ghép cặp *tạm thời* sau một đợt mờ rẻ hơn nhiều so với làm bộ phát hiện chịu mờ giỏi hơn. Đây là suy luận từ cơ chế; mình chưa đo nó, và nó xứng đáng là một thí nghiệm ablation.

Bảng 2.1. Bốn chế độ hỏng, khối bị phá, và thứ thật sự chữa được nó. Cột cuối là cột dùng được nhất: nó cho biết đối bộ đặc trưng có phải là thuốc đúng bệnh không.

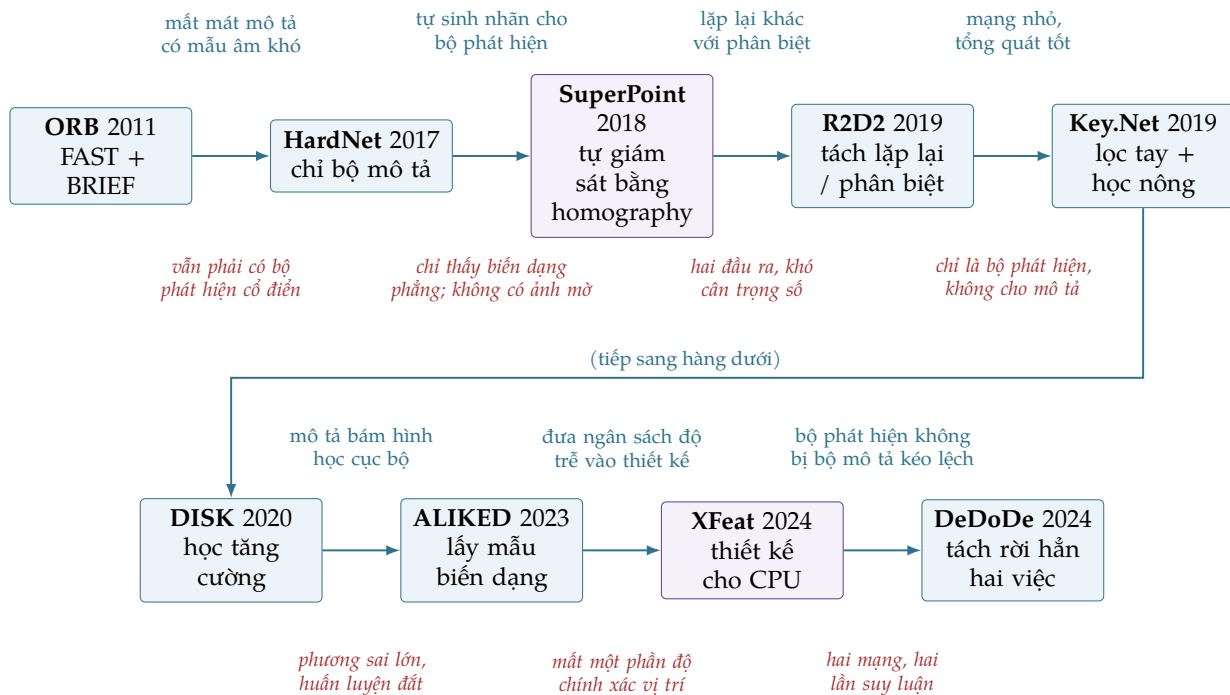
Chế độ	Khối bị phá	Cơ chế	Thuốc đúng bệnh
Ảnh mờ do chuyển động	Bộ phát hiện (FAST)	Trung bình dọc hướng chuyển động kéo tương phản xuống dưới ngưỡng, và cắt đứt cung 9 điểm	Bộ phát hiện học được, hoặc bỏ hẳn bộ phát hiện; giảm phơi sáng nếu đủ sáng; camera sự kiện
Đổi độ sáng	Bộ mô tả (BRIEF)	Biên δ của các phép so sánh co lại, bit lật ngẫu nhiên, Hamming của cặp đúng trôi qua ngưỡng	Bộ mô tả thực học bằng mất mát tương phản (HardNet, SuperPoint)
Đổi góc nhìn lớn	Bộ mô tả và kim tự tháp	Biến dạng affine bất đẳng hướng nằm ngoài bất biến tỉ lệ đẳng hướng	Bộ mô tả học được trên dữ liệu có biến dạng; hoặc bộ ghép cặp có ngữ cảnh toàn cục
Vùng ít vân	Bản thân ý tưởng có bộ phát hiện	Quyết định cứng, sớm, cục bộ trên vùng không có thông tin cục bộ	Ghép cặp dày không cần bộ phát hiện; hoặc chấp nhận và dựa vào IMU

Bảng 2.1 đã trả lời trước một phần câu hỏi của cả chương. Chỉ có hai trong bốn chế độ được chữa bởi việc “đổi ORB sang SuperPoint”. Chế độ thứ nhất được chữa một phần, và chế độ thứ tư thì không — nó cần một loại phương pháp khác hẳn.

2.3 Bộ phát hiện và mô tả học được (Learned Detectors and Descriptors)

Học một bộ mô tả là chuyện dễ hiểu: có nhãn, vì hai ô ảnh của cùng một điểm 3D là một cặp dương. Học một bộ *phát hiện* thì không. Không tồn tại chân trị cho câu “điểm ảnh này có đáng là điểm đặc trưng không”. Toàn bộ lịch sử của dòng này là lịch sử của các cách lách quanh chỗ thiếu nhãn đó, và mỗi cách lách để lại một dấu vết riêng trong cách phương pháp hỏng.

Hình 2.4 xếp cả dòng lên một trục. Mỗi mũi tên ghi nút thắt được gỡ và vấn đề mới sinh ra. Đọc hình trước rồi đọc phần chữ sẽ nhanh hơn hẳn.



Hình 2.4. Dòng đặc trưng học được, đọc theo nút thắt chữ không theo năm. Nhãn phía trên mỗi mũi tên là thứ được gỡ; nhãn phía dưới in nghiêng đỏ là vấn đề mới sinh ra. Hai ô tô tím là hai bước ngoặt về *cách nghĩ*: SuperPoint giải quyết chỗ thiếu nhãn, XFeat là công trình đầu tiên trong dòng đặt ngân sách độ trễ làm ràng buộc thiết kế thay vì làm ghi chú cuối bài.

A. HardNet: hàm mất mát phải mô phỏng đúng quy tắc quyết định lúc chạy

HardNet chỉ làm nửa dễ, tức bộ mô tả, nhưng nó là công trình có bài học thiết kế chuyển được sang việc khác nhiều nhất [26]. Cơ chế của nó gói gọn trong một câu. Lấy một *batch* gồm n cặp ô ảnh khớp nhau, tính ma trận khoảng cách $n \times n$, rồi với mỗi mô neo lấy *mẫu âm gần nhất trong batch* làm mẫu âm, và tối đa hoá biên giữa cặp dương và mẫu âm đó.

Vì sao chọn mẫu âm khó nhất chứ không lấy ngẫu nhiên? Vì lúc chạy thật, đối thủ của một bộ mô tả không phải một ô ảnh ngẫu nhiên. Đối thủ của nó là láng giềng gần nhất trong toàn bộ ảnh kia. Mẫu âm ngẫu nhiên gần như luôn ở rất xa, nên *gradient* tắt và mạng học được rất ít. Sâu hơn nữa: quy tắc quyết định lúc chạy là *phép thử tỉ lệ* giữa ứng viên tốt nhất và ứng viên nhì. Lấy mẫu âm khó nhất trong batch chính là cách xấp xỉ ứng viên nhì đó ngay trong hàm mất mát.

Bài học chuyển được: hàm mất mát nên mô phỏng quy tắc quyết định sẽ dùng lúc chạy, không phải một đại lượng dễ tối ưu hơn. Bạn dùng được bài học này ngay cả khi không dùng mạng nào. Nếu frontend của bạn quyết bằng tỉ lệ giữa hai ứng viên tốt nhất, thì mọi phép đo chất lượng đặc trưng cũng phải đo theo tỉ lệ đó, chứ không đo khoảng cách tuyệt đối.

↔ **cùng nguyên lý với B/09** — đây là dạng cụ thể nhất của nguyên tắc “hàm mất mát mã hoá điều bạn thật sự quan tâm” ở chương thiết kế hàm mất mát

B. SuperPoint: chế ra nhãn từ một phép biến đổi bạn nghịch đảo được

SuperPoint giải chỗ thiếu nhãn bằng ba bước [4]. Bước một, huấn luyện một bộ phát hiện thô trên ảnh tổng hợp gồm các hình khối đơn giản, nơi vị trí góc biết trước theo dựng hình. Bước hai gọi là làm giàu bằng homography: lấy ảnh thật, áp hàng trăm phép homography ngẫu nhiên, chạy bộ phát hiện thô trên từng ảnh đã biến đổi, rồi *ánh xạ ngược* các điểm về ảnh gốc và cộng dồn. Điểm nào sống sót qua nhiều phép biến đổi thì được ghi nhận là điểm giả định. Bước ba, huấn luyện chung một mạng có hai đầu ra — một đầu cho bản đồ điểm, một đầu cho bộ mô tả

256 chiều — với nhân của bộ mô tả lấy từ chính homography đã dùng, vì homography cho biết chính xác điểm nào ứng với điểm nào.

Ý tưởng cốt lõi rất gọn: **khi nhãn không tồn tại, hãy chế nó ra từ một phép biến đổi mà bạn nghịch đảo được chính xác**. Đây là cùng một mạch tư duy với tự giám sát bằng bất biến ở phần C, chỉ khác là phép biến đổi ở đây có ý nghĩa hình học.

Cái giá nằm ở chính chỗ đó, và với người làm SLAM thì đây là điều quan trọng nhất phải biết về SuperPoint. Homography là biến đổi của một *mặt phẳng*. Phân phối huấn luyện vì thế không chứa thị sai do độ sâu, không chứa che khuất, và — điểm chí mạng — **không chứa ảnh mờ do chuyển động** trừ khi bạn tự thêm vào. Nói cách khác, phương pháp được ca ngợi nhiều nhất trong họ này không hề được huấn luyện cho nỗi đau số một của bạn. Nếu định dùng nó cho chuyển động nhanh, việc đầu tiên phải làm là tăng cường dữ liệu bằng mờ có hướng, và đo lại. Đây là một suy luận từ thiết kế của phương pháp, không phải một kết quả đã công bố.

C. R2D2: lặp lại và phân biệt là hai chuyện khác nhau

R2D2 chỉ ra một lỗ hổng khái niệm mà cả ORB lẫn SuperPoint đều mắc [27]. Một điểm có thể rất *lặp lại* mà hoàn toàn vô dụng. Ví dụ chuẩn là góc của một viên gạch trên bức tường gạch. Bộ phát hiện tìm thấy nó ở đúng chỗ trong mọi khung hình, nên tính lặp lại hoàn hảo. Nhưng nó giống hết năm trăm góc gạch khác, nên ghép cặp luôn sai.

Giải pháp là hai đầu ra riêng. Đầu thứ nhất dự đoán tính lặp lại, huấn luyện bằng cách ép bản đồ điểm của ảnh gốc và bản đồ điểm của ảnh đã biến đổi có các đỉnh trùng nhau. Đầu thứ hai dự đoán *độ tin cậy*, tức là ước lượng xem bộ mô tả tại điểm ảnh đó có ghép đúng được không. Điểm được chọn là điểm cao ở cả hai bản đồ.

Bài học ở đây dùng được ngay mà không cần mạng. Bộ chọn điểm của ORB-SLAM3 tối đa hoá đúng một thứ là điểm số góc Harris, tức là một xấp xỉ của tính lặp lại. Nó không có khái niệm nào về tính phân biệt. Đó là nguyên nhân cơ chế của một lỗi rất đắt: vòng đóng sai trên cấu trúc lặp như hàng cột, cửa sổ, hay gạch lát. Nếu bạn muốn một cái tiền rẻ, hãy thêm một điểm số phân biệt cục bộ vào bước chọn điểm — chẳng hạn khoảng cách Hamming tới láng giềng gần nhất *trong cùng ảnh* — và loại các điểm quá giống hàng xóm của chính mình.

D. Key.Net, DISK và DeDoDe: ba cách khác để lách chỗ thiếu nhãn

Ba công trình còn lại tấn công cùng vấn đề bằng ba đường khác nhau, và mỗi đường dạy một điều.

- **Key.Net — dùng lại phần đã giải xong.** Nó đưa các bộ lọc đạo hàm bậc nhất viết tay ở nhiều tỉ lệ vào làm đầu vào, rồi chỉ học một mạng rất nông để tổ hợp chúng [28]. Kết quả là một mô hình rất nhỏ và tổng quát hoá tốt sang ảnh khác kiểu. Bài học: phần nào của bài toán đã có lời giải viết tay tốt thì đừng bắt mạng học lại; hãy để mạng học phần còn lại.
- **DISK — tối ưu đúng mục tiêu rời rạc.** Chọn điểm và chấp nhận một cặp ghép đều là quyết định rời rạc, nên không lấy *gradient* qua được. DISK coi chúng là hành động lấy mẫu từ một phân phối, thưởng +1 cho cặp ghép đúng và phạt -1 cho cặp sai, rồi dùng *gradient* chính sách [29]. Bài học: khi mục tiêu thật là rời rạc và nằm ở cuối đường ống, học tăng cường là lối thoát thay cho việc bịa ra một hàm thay thế khả vi. Cái giá là phương sai *gradient* lớn và huấn luyện đắt hơn hẳn.
- **DeDoDe — tách rời để không kéo lệch nhau.** Huấn luyện chung bộ phát hiện với bộ mô tả có một tác dụng phụ ngầm: bộ phát hiện dần chỉ chọn những điểm mà bộ mô tả *hiện tại* tình cờ mô tả tốt. Hai chế độ hồng quần vào nhau, không tách ra chẩn đoán được. DeDoDe huấn luyện bộ phát hiện riêng, bằng tín hiệu “điểm này có được theo dõi bền qua một tái dựng ba

chiều lớn không”, rồi huấn luyện bộ mô tả riêng [30]. Bài học chính là bài học chẩn đoán, và nó rất hợp với kỷ luật ablation của chương 24.

E. XFeat và ALIKED: hai công trình nói cùng ngôn ngữ với bạn

Hai công trình cuối đáng chú ý riêng, vì chúng là hai công trình duy nhất trong danh sách đặt ràng buộc của bạn vào ngay từ đầu.

XFeat được thiết kế cho CPU và thiết bị biên [31]. Ba lựa chọn thiết kế đáng học. Thứ nhất, mạng rất nông và giảm mẫu sớm, làm việc chủ yếu ở một phần tư độ phân giải. Thứ hai, số kênh ở các tầng đầu bị giữ rất nhỏ, vì các tầng đầu chạy trên độ phân giải cao nên chúng chiếm phần lớn chi phí. Thứ ba, độ chính xác vị trí bị mất do giảm mẫu được lấy lại bằng một mạng tinh chỉnh nhỏ chạy *sau* khi đã ghép cặp, tức là chỉ chạy trên vài trăm điểm thay vì trên toàn ảnh. Đây là mẫu thiết kế đáng nhớ nhất của cả chương: **làm phân thô trên toàn ảnh ở độ phân giải thấp, làm phân tinh chỉ ở nơi cần, sau khi đã biết nơi nào cần**. Cùng mẫu này xuất hiện lại ở LoFTR trong mục 5.

ALIKED tấn công thẳng chế độ hồng thứ ba [32]. Bộ mô tả cổ điển lấy mẫu trên một ô vuông cố định quanh điểm. Khi mặt phẳng nghiêng, ô vuông đó trên ảnh ứng với một hình bình hành trên vật thật, nên nội dung lấy được đã méo trước khi mô tả. ALIKED học các vị trí lấy mẫu *biến dạng được*: mạng dự đoán một tập nhỏ các độ dời, và bộ mô tả lấy mẫu tại các vị trí đã dời đó. Hình dạng vùng hỗ trợ vì thể thích nghi theo hình học cục bộ. Chi phí thấp vì số vị trí lấy mẫu ít.

F. Hai trật tự: phát hiện rồi mô tả, hay mô tả rồi phát hiện

Có một lựa chọn kiến trúc chạy ngầm dưới cả bảy phương pháp trên, và nó đáng được gọi tên riêng. Nó quyết định phương pháp hồng thể nào khi ảnh mờ.

Trật tự cổ điển là **phát hiện rồi mô tả**. FAST chọn điểm trước, BRIEF mô tả sau. SIFT cũng theo đúng trật tự này, chỉ khác ở chỗ nó dùng cực trị của hàm sai khác Gauss thay cho phép thử vòng tròn [33]. LIFT là bản học được đầu tiên của đúng trật tự đó: ba mạng nối tiếp cho phát hiện, ước lượng hướng, và mô tả [34]. Ưu điểm là chi phí thấp, vì bộ mô tả chỉ chạy trên vài trăm ô ảnh. Nhược điểm nằm ở chỗ nếu bộ phát hiện đã bỏ sót một điểm thì bộ mô tả không có cơ hội nào cứu.

Trật tự ngược là **mô tả rồi phát hiện**, do D2-Net đề xuất [35]. Mạng tính một bản đồ đặc trưng dày cho toàn ảnh trước. Sau đó điểm được chọn ngay trên chính bản đồ đặc trưng đó, tại các vị trí vừa là cực đại theo không gian vừa là cực đại theo kênh. Nghĩa là quyết định “đây là điểm đáng giữ” được đưa ra bằng biểu diễn đã qua nhiều tầng tích chập, chứ không bằng cường độ thô.

Khác biệt này quan trọng với bạn vì một lý do rất cụ thể. Ở trật tự cổ điển, ảnh mờ giết bộ phát hiện ở tầng cường độ thô, nơi không có gì cứu được. Ở trật tự ngược, quyết định dựa trên đặc trưng có trường tiếp nhận rộng vài chục điểm ảnh, nên cấu trúc lớn vẫn còn tín hiệu ngay cả khi cấu trúc nhỏ đã bị trung bình mất. Cái giá là chi phí: bản đồ đặc trưng dày phải tính cho toàn ảnh, và độ chính xác vị trí kém hơn vì cực đại được lấy trên lưới đã giảm mẫu. Đây chính là đánh đổi mà XFeat gỡ bằng bước tinh chỉnh sau ghép cặp, và là đánh đổi mà LoFTR đẩy tới cực đoạn ở mục 5.

G. Cần bao nhiêu dữ liệu, và bạn có nó không

Cổng thứ ba của chương 27 hỏi “cần bao nhiêu dữ liệu”. Với đặc trưng học được, câu trả lời dễ chịu hơn hầu hết mọi chỗ khác trong học sâu, và lý do đáng hiểu.

Nhân ở đây không đến từ con người. Nó đến từ hình học. Muốn biết điểm ảnh nào trong ảnh i ứng với điểm ảnh nào trong ảnh j , bạn chỉ cần tư thế tương đối và độ sâu. Vì thế mọi tập dữ liệu huấn luyện của dòng này đều là đầu ra của một hệ thống tái dựng: ảnh thu thập từ internet chạy qua tái dựng cấu trúc từ chuyển động cho cảnh ngoài trời, hoặc ảnh RGB-D quét trong nhà. Không có ai ngồi gán nhãn.

Ba mức đầu tư, theo thứ tự rẻ dần lên đắt dần.

- **Không dữ liệu của bạn.** Dùng trọng số công bố sẵn. Rủi ro là ra ngoài phân phối huấn luyện, và mục “Giới hạn” cuối chương nói rõ kiểu hỏng.
- **Chỉ ảnh của bạn, không cần tư thế.** Đây là chỗ tự giám sát bằng homography có giá trị thực dụng lớn nhất, và nó ít được nói tới. Bạn áp phép biến đổi lên chính ảnh của mình rồi huấn luyện, không cần chân trị nào cả. Nếu miền của bạn khác về *kết cấu* — ảnh hầm mỏ, ảnh ban đêm — thì mức này đã giải quyết phần lớn. Nếu miền của bạn khác về *chuyển động*, hãy thêm mờ có hướng vào tập biến đổi; đó là chỗ SuperPoint để trống.
- **Ảnh của bạn kèm tư thế.** Chạy COLMAP trên các chuỗi đã thu, lấy tư thế và độ sâu thưa, rồi sinh cặp ghép chân trị. Tốn vài ngày tới vài tuần công sức, chủ yếu là công sức lọc các tái dựng hỏng. Đổi lại bạn có một tập huấn luyện đúng miền và đúng chế độ chuyển động của mình.

Nói cách khác, đây là một trong rất ít can thiệp học sâu vào SLAM mà con đường dữ liệu rõ ràng từ đầu tới cuối và không phụ thuộc vào người gán nhãn. Đó là một lý lẽ ủng hộ thật, và nó nên được cân nhắc song song với các lý lẽ phản đối ở mục 6.

Bảng 2.2. Bảy phương pháp, so theo cùng bốn tiêu chí. Cột “hỏng khi nào” là cột quyết định: nó cho biết phương pháp mang theo giả định gì từ dữ liệu huấn luyện của nó. Cột chi phí là xếp hạng tương đối theo số phép nhân cộng và độ sâu mạng, không phải số đo trên Orin.

Tên	Cơ chế	Nguồn giám sát	Hỏng khi nào	Chi phí
HardNet	Chỉ bộ mô tả; triplet với mẫu âm khó nhất trong batch	Cặp ô ảnh khớp từ tái dựng ba chiều	Vẫn cần bộ phát hiện cổ điển, nên thừa hưởng nguyên chế độ hỏng khi ảnh mờ	Rất thấp
SuperPoint	Một mạng, hai đầu ra: bản đồ điểm và mô tả 256 chiều	Tự giám sát bằng homography ngẫu nhiên	Phân phối huấn luyện chỉ có biến dạng phẳng; không có ảnh mờ, không có thị sai	Trung bình
R2D2	Tách hai bản đồ: tính lặp lại và độ tin cậy	Cặp ảnh có dòng quang đã biết	Hai mục tiêu phải cân trọng số bằng tay; nặng hơn SuperPoint	Trung bình cao
Key.Net	Lọc đạo hàm viết tay nhiều tỉ lệ, mạng nông tổ hợp	Mất mát kiểu tính lặp lại trên cặp ảnh	Chỉ là bộ phát hiện; không cho bộ mô tả	Rất thấp
DISK	Chọn điểm và ghép cặp là hành động; thưởng theo cặp ghép đúng	Học tăng cường trên ảnh có tư thế đã biết	Huấn luyện đắt, phương sai lớn, nhạy với cách đặt phần thưởng	Trung bình
XFeat	Mạng rất nông ở một phần tư độ phân giải, tinh chỉnh sau khi ghép	Cặp ảnh tổng hợp và thật	Vị trí kém chính xác hơn ở tầng thô; đổi độ chính xác lấy tốc độ	Thấp
ALIKED	Vị trí lấy mẫu biến dạng được cho vùng hỗ trợ của bộ mô tả	Cặp ảnh có tư thế đã biết	Biến dạng học được vẫn giả định bề mặt trơn cục bộ	Thấp

Mạch 1 — vì sao dòng này tiến theo đúng thứ tự đó

- 1 Vấn đề** Không có chân trị cho câu “điểm nào đáng là điểm đặc trưng”. Mọi cố gắng học bộ phát hiện đều dừng ở đây suốt nhiều năm, trong khi bộ mô tả thì đã học được từ 2016.
- Giải pháp** SuperPoint chế nhạo từ homography nghịch đảo được. Điểm nào sống sót qua hàng trăm phép biến đổi thì được coi là điểm tốt.
- Giả định** Ảnh là của một mặt phẳng, hoặc ít nhất biến dạng giữa hai ảnh xấp xỉ được bằng homography.
- Giá phải trả** Phân phối huấn luyện mất sạch thị sai, che khuất và ảnh mờ. Mạng không bao giờ thấy nỗi đau thật của SLAM.
- Vấn đề mới** Điểm được chọn tối đa hoá tính lặp lại, mà tính lặp lại cao không có nghĩa là ghép đúng được.
- 2 Vấn đề** Cấu trúc lặp cho điểm cực kỳ lặp lại nhưng không phân biệt được, nên ghép cặp sai một cách có hệ thống.
- Giải pháp** R2D2 dự đoán hai bản đồ tách rời, chọn điểm cao ở cả hai.
- Giá phải trả** Hai hàm mất mát phải cân trọng số bằng tay, và trọng số đó không chuyển được sang tập dữ liệu khác.
- Vấn đề mới** Bộ phát hiện và bộ mô tả vẫn huấn luyện chung, nên bộ phát hiện bị kéo về phía sở thích của bộ mô tả.
- 3 Vấn đề** Mọi hàm mất mát ở trên đều là hàm thay thế. Thứ ta thật sự muốn — số cặp ghép đúng sau bước lọc — là một đại lượng rời rạc.
- Giải pháp** DISK tối ưu thẳng đại lượng rời rạc đó bằng *gradient* chính sách; DeDoDe tách hẳn hai việc để chúng không kéo lệch nhau.
- Giá phải trả** Huấn luyện đắt hơn nhiều và nhạy với cách đặt phần thưởng; DeDoDe phải chạy hai mạng.
- Hệ quả** Tới đây chất lượng đã bão hoà trên các bảng xếp hạng ghép ảnh, và trực cạnh tranh chuyển sang *chi phí*. Đó chính là chỗ XFeat và ALIKED bước vào, và cũng là chỗ người làm SLAM nên bắt đầu đọc.

2.4 Ghép cặp học được và chi phí bậc hai (Learned Matching)

Ghép cặp trong ORB-SLAM3 là một chuỗi các quyết định *độc lập và cục bộ*. Với mỗi bộ mô tả, tìm láng giềng gần nhất theo Hamming. Loại nếu tỉ lệ giữa ứng viên nhất và nhì lớn hơn mfNNratio . Giữ nếu hai chiều cùng chọn nhau. Cuối cùng, bỏ phiếu bằng biểu đồ chênh lệch góc với HISTO_LENGTH bằng 30, và giữ ba ô cao nhất.

Đọc lại danh sách đó theo con mắt khác thì nó là một xấp xỉ thô của hai ràng buộc toàn cục. Phép thử tỉ lệ là xấp xỉ cục bộ của ràng buộc *mỗi điểm chỉ được dùng một lần*. Biểu đồ góc là xấp xỉ rất thô của ràng buộc *mọi cặp ghép phải nhất quán với một phép biến đổi chung*. SuperGlue là câu trả lời cho câu hỏi: nếu ta thay hai xấp xỉ thô đó bằng hai thứ thật thì được gì.

A. SuperGlue: đưa ngữ cảnh và ràng buộc một-một vào trong mô hình

SuperGlue nhận vào hai tập điểm kèm bộ mô tả và trả về ma trận gán [5]. Bốn bước, mỗi bước gỡ một thứ.

- Nhúng vị trí.** Mỗi điểm thành một vector, ghép từ bộ mô tả và từ một phép nhúng của toạ độ cùng điểm số tin cậy. Từ lúc này, “ở đâu” và “trông thế nào” nằm chung một biểu diễn.

2. **Chín tầng *attention* xen kẽ.** Tầng tự chú ý cho mỗi điểm nhìn mọi điểm khác trong *cùng* ảnh; tầng chú ý chéo cho nó nhìn mọi điểm trong ảnh kia. Sau chín tầng, biểu diễn của một điểm không còn là mô tả cục bộ nữa mà đã mang thông tin của cả hai ảnh.
3. **Ma trận điểm số kèm thùng rác.** Tích vô hướng giữa hai tập vectơ cho ma trận $N \times N$, thêm một hàng và một cột “thùng rác” để chứa các điểm không có cặp.
4. **Sinkhorn.** Chuẩn hoá xen kẽ theo hàng và theo cột nhiều vòng, tức là giải xấp xỉ bài toán vận tải tối ưu có chính quy hoá entropy [36]. Kết quả là ma trận gần như một-một, và tính một-một do *mô hình* bảo đảm chứ không do một phép lọc sau đó.

Vì sao cơ chế này thắng phép thử tỉ lệ? Vì phép thử tỉ lệ chỉ nhìn được hai ứng viên của riêng một điểm. Trên một bức tường gạch, cả hai ứng viên đều giống nhau, nên phép thử loại luôn cặp ghép — kể cả khi cặp đúng nằm trong đó. Chú ý chéo thì cho điểm đó biết rằng các hàng xóm của nó đều ghép về một vùng nhất quán, và thông tin đó đủ để chọn đúng viên gạch. Nói gọn: **SuperGlue đổi một quyết định cục bộ lấy một quyết định toàn cục, và trả bằng chi phí bậc hai.**

B. Chi phí bậc hai, và vì sao nó chi phối mọi quyết định khác

Chi phí của *attention* tỉ lệ với bình phương số điểm. Kế toán thô nhưng đủ dùng: mỗi phép chú ý trên N điểm với số chiều d tốn khoảng $2N^2d$ phép nhân cộng, gồm ma trận điểm số và tổng có trọng số. SuperGlue có chín tầng, mỗi tầng áp cho cả hai ảnh, nên khoảng 18 phép chú ý:

$$\text{MAC} \approx 18 \cdot 2N^2d = 36 N^2d.$$

Công thức đó là toàn bộ thứ bạn cần, và nó nói hai điều. Thứ nhất, chi phí tăng theo N^2 : nhân đôi số điểm là gấp bốn chi phí, nên số điểm là đòn bẩy mạnh nhất mà bạn có, mạnh hơn mọi thủ thuật cài đặt. Thứ hai, hệ số đứng trước N^2 không nhỏ — chín tầng, mỗi tầng áp cho cả hai ảnh, số chiều d cỡ vài trăm — nên ngay ở số điểm mà ORB-SLAM3 đang dùng, đây đã là một khối lượng tính toán khác hẳn *bậc* so với một phép tìm láng giềng gần nhất trên chuỗi bit. Khác bậc, không phải khác vài phần trăm.

Đổi từ phép nhân cộng sang mili-giây thì chương này dừng lại, và dừng có lý do. Muốn chia thì phải có mẫu số, tức thông lượng thật của Orin ở batch bằng 1 trên đúng dạng phép toán này; mà *attention* với ma trận nhỏ thường bị chặn bởi băng thông bộ nhớ chứ không bởi số nhân, nên mẫu số ấy cách hiệu năng đỉnh rất xa và không tra được ở đâu cả. Một con số mili-giây suy ra từ một mẫu số đoán mò là con số vô giá trị — tệ hơn cả không có, vì nó sẽ được nhớ và được trích lại — nên ở đây không có con số nào. Cách đo mẫu số đó nằm ở mục 6.

Thứ chuyển được sang mọi phần cứng là *độ dốc*: mọi thứ ở đây tăng theo N^2 , nên quyết định về số điểm quan trọng hơn mọi thứ khác, và nó vẫn đúng nguyên khi bạn đổi máy.

C. LightGlue: ba cách rút ngắn cùng một chồng tầng

LightGlue giữ nguyên ý tưởng của SuperGlue và tấn công riêng phần chi phí [6]. Ba cơ chế, và cả ba đều dùng lại được ở nơi khác.

- **Thoát sớm thích nghi.** Sau mỗi tầng, một bộ phân lớp nhỏ ước lượng xem lời giải đã đủ chắc chưa. Cặp ảnh dễ — và trong SLAM thì gần như mọi cặp khung hình liên tiếp đều dễ — dừng sau vài tầng thay vì chạy hết chín tầng. Ý tưởng gốc: *độ sâu tính toán nên tỉ lệ với độ khó của mẫu, không phải là hằng số.*
- **Cắt điểm.** Điểm nào được dự đoán là không thể ghép thì bị loại giữa chừng, nên các tầng sau chạy trên N nhỏ hơn. Vì chi phí là bậc hai, bỏ một nửa số điểm làm phần còn lại rẻ đi bốn lần. Đây là chỗ tiết kiệm lớn nhất.

- **Bỏ Sinkhorn.** Thay vì hàng chục vòng chuẩn hoá, LightGlue dự đoán riêng một điểm số “có ghép được không” cho từng điểm, rồi kết hợp nó với ma trận tương đồng trong một bước. Phần gán không còn là vòng lặp nữa.

Bài báo báo cáo tăng tốc vài lần so với SuperGlue ở cùng mức chính xác, và mức tăng lớn hơn trên cặp ảnh dễ. Mình không dẫn con số mili-giây cụ thể ở đây, vì con số đó gần với phần cứng máy để bàn trong bài và không chuyển sang Orin được.

D. Ba thứ lấy được từ SuperGlue mà không cần chạy SuperGlue

Trước khi bàn chi phí, đáng dừng lại ở một câu hỏi rẻ hơn nhiều: trong ý tưởng của SuperGlue, phần nào dùng được ngay trong ORBmatcher hiện tại, không cần mạng nào?

- **Gán một-một thật thay cho phép thử tỉ lệ.** Bạn đã có ma trận chi phí Hamming giữa các ứng viên. Giải bài toán gán trên ma trận đó — bằng thuật toán Hungary, hoặc bằng vài vòng chuẩn hoá kiểu Sinkhorn — cho ra một lời giải toàn cục thay vì một chuỗi quyết định độc lập. Chi phí là bậc ba theo số ứng viên trong cửa sổ tìm kiếm, mà số đó rất nhỏ, nên trên thực tế nó gần như miễn phí. Đây là phần rẻ nhất và mình cho là đáng thử trước.
- **Bỏ phiếu nhất quán mịn hơn.** Biểu đồ chênh lệch góc với 30 ô là một phép bỏ phiếu một chiều rất thô. Bỏ phiếu trên một không gian hai chiều — chênh lệch góc cộng với độ dời trung bình — bắt được nhiều cặp sai hơn với cùng chi phí. Đây đúng là thứ mà tầng *attention* của SuperGlue học được, chỉ ở dạng viết tay.
- **Điểm số “có ghép được không” riêng cho từng điểm.** LightGlue dự đoán nó bằng mạng. Bạn xấp xỉ được nó bằng một đại lượng có sẵn: khoảng cách Hamming tới láng giềng gần nhất *trong cùng ảnh*. Điểm nào có láng giềng cùng ảnh rất gần thì nằm trên cấu trúc lặp, và nên bị hạ trọng số. Đây cũng chính là ý tưởng “tính phân biệt” của R2D2, áp bằng tay.

Ba việc này không thay thế được SuperGlue. Nhưng chúng cho bạn một *baseline* mạnh hơn trước khi so, và mục 6 sẽ giải thích vì sao có một *baseline* mạnh lại quan trọng tới vậy ở đây.

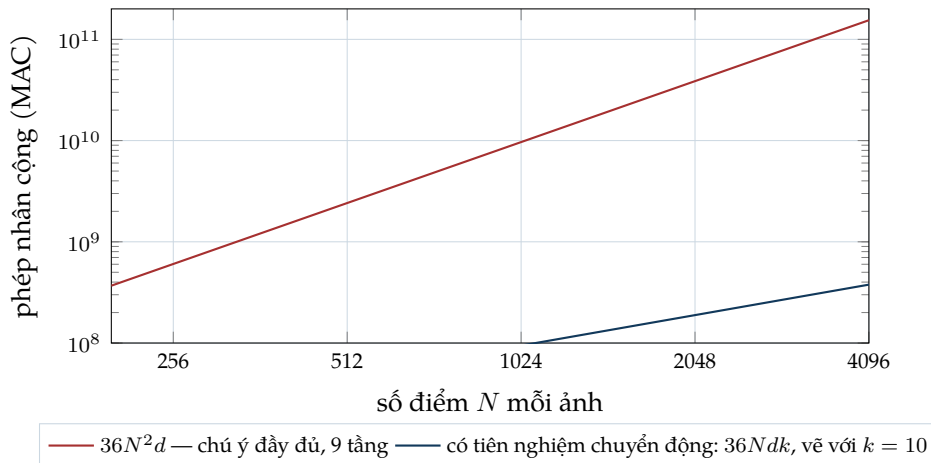
E. Sáu cách né chi phí bậc hai, xếp theo mức hữu ích cho SLAM

Đây là mục đáng giá nhất của cả phần ghép cặp, vì nó là chỗ nền SLAM của bạn có lợi thế thật so với người viết các bài báo trên.

1. **Giảm N .** Rẻ nhất và hiệu quả nhất, vì tiết kiệm là bậc hai: giảm một nửa số điểm thì chi phí còn một phần tư. Hãy đo APE theo số điểm trước khi làm bất cứ việc gì khác — rất có thể hệ của bạn đã bão hoà ở một số điểm thấp hơn hẳn cấu hình mặc định, và nếu đúng thế thì bạn vừa lấy được phần lớn khoản tiết kiệm mà không đụng tới một mô hình nào.
2. **Chỉ chạy ở nơi thật sự cần.** Giữ ORB cho bám khung hình ở 30 Hz, và chỉ gọi bộ ghép cặp học được cho ứng viên đóng vòng và định vị lại, ở cỡ 1 Hz. Ngân sách giảm khoảng ba mươi lần, và nó nhắm đúng chế độ hồng thứ ba — chế độ mà đối góc nhìn lớn thật sự xảy ra. Theo mình đây là phương án có tỉ lệ lợi ích trên rủi ro cao nhất trong cả chương.
3. **Dùng dự đoán chuyển động để giới hạn cặp ứng viên.** Hệ của bạn đã có mô hình vận tốc không đổi và có IMU. Chúng cho một cửa sổ tìm kiếm hẹp quanh vị trí chiếu. Khi đó mỗi điểm chỉ còn k ứng viên trong cửa sổ thay vì cả N điểm của ảnh kia, nên chi phí thành $N \cdot k$ chứ không phải N^2 , tức là gần như tuyến tính — và k là một đại lượng bạn đo được ngay từ nhật ký của ORBmatcher. Về mặt *attention*, đây chính là chú ý thừa theo khối.
4. **Cắt điểm và thoát sớm.** Đúng hai cơ chế của LightGlue; dùng được ngay vì đã có mã nguồn mở.

5. **Chú ý tuyến tính.** Thay hàm mũ trong *softmax* bằng một ánh xạ đặc trưng để đổi thứ tự phép nhân ma trận, cho chi phí $O(N)$ [37]. LoFTR dùng cách này. Cái giá là sức biểu diễn giảm, và với N nhỏ thì nó còn chậm hơn vì hằng số lớn.
6. **Ghép thô rồi tinh.** Ghép ở độ phân giải thấp trước rồi chỉ tinh chỉnh quanh các cặp đã thắng. Đây là mẫu thiết kế đã gặp ở XFeat và sẽ gặp lại ở LoFTR.

Điểm số ba đáng được nói thẳng, vì nó là một khoảng trống thật trong văn liệu. Gần như mọi bài báo ghép cặp đều đánh giá trên cặp ảnh hai khung nhìn rộng, lấy từ tái dựng ba chiều, *không có tiên nghiệm chuyển động nào*. Trong bối cảnh đó, giải bài toán gán toàn cục với chi phí N^2 là hợp lý. Trong bối cảnh của bạn, giữa hai khung hình cách nhau 33 mili-giây và có IMU nối chúng, phần lớn chi phí N^2 đó là tiền trả cho một bài toán bạn không có. Hình 2.5 vẽ khoảng cách giữa hai chế độ đó.



Hình 2.5. Chi phí ghép cặp theo số điểm. Hai đường là hai công thức được vẽ ra chứ không phải hai phép đo: đỏ là chú ý đầy đủ, xanh là cùng mô hình nhưng chỉ xét k ứng viên nằm trong cửa sổ do IMU dự đoán. Khoảng cách giữa hai đường đúng bằng N/k , nên nó rộng ra khi bạn xin nhiều điểm hơn và hẹp lại khi tiên nghiệm của bạn kém đi — đó là toàn bộ nội dung của hình. Trục tung đếm phép nhân cộng chứ không đếm mili-giây; muốn đổi sang thời gian thì phải có thông lượng đo được của máy bạn ở batch bằng 1, và mục 6 nói cách lấy nó.

CẠM BẮY

Bảng xếp hạng ghép cặp đo trên cặp ảnh hai khung nhìn rộng, chọn ra từ tái dựng ba chiều, thường cách nhau nhiều mét và nhiều chục độ [38]. Frontend của bạn ghép hai khung hình cách nhau 33 mili-giây, với một tiên nghiệm tư thế tốt tới vài điểm ảnh. Hai bài toán này khác nhau tới mức thứ hạng trên bảng kia gần như không dự đoán được gì cho bạn. Chuyển kết luận thẳng từ bảng xếp hạng sang hệ của mình là sai lầm đắt nhất trong cả chương — không phải vì các con số sai, mà vì chúng trả lời một câu hỏi khác. Cách kiểm tra rẻ: đo phân bố *góc giữa hai tia quan sát* của các cặp ghép trong chuỗi của bạn, rồi so với phân bố đó trong tập kiểm thử của bài báo.

2.5 Bỏ hẳn bộ phát hiện (Detector-free Matching)

Mục 2 đã kết luận rằng vùng ít vân không phải một lỗi cài đặt. Nó là hệ quả logic của việc bắt một cửa sổ bán kính 3 quyết định một câu hỏi mà thông tin trả lời nằm cách đó hai trăm điểm ảnh. Có đúng một cách thoát: đừng quyết sớm.

A. LoFTR: ghép dày ở tầng thô, rồi tinh chỉnh

LoFTR bỏ hẳn bước chọn điểm [7]. Nó chạy hai giai đoạn.

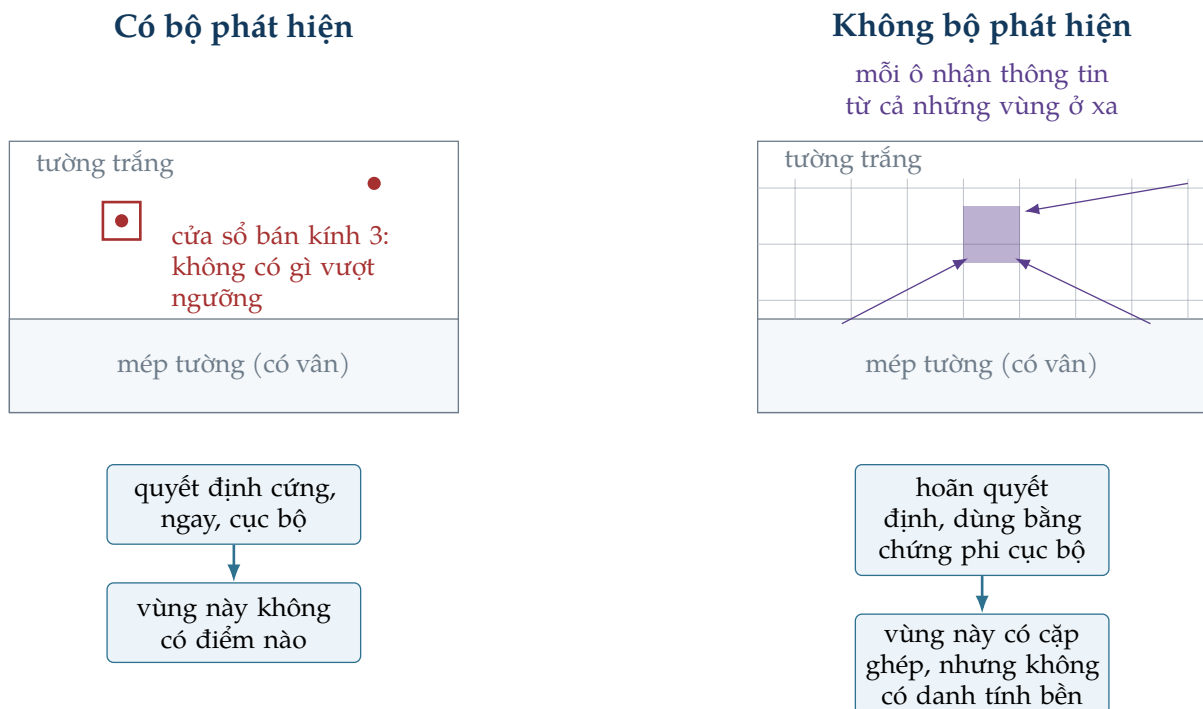
Giai đoạn thô làm việc trên bản đồ đặc trưng ở một phần tám độ phân giải. Với ảnh 640 nhân 480, đó là lưới 80 nhân 60, tức 4800 ô. Mỗi ô thành một *token*, rồi một chồng tầng tự chú ý và chú ý chéo cho mỗi ô nhìn thấy toàn bộ hai ảnh. Vì 4800 ô mà chú ý đầy đủ thì ma trận điểm số có hơn hai chục triệu ô, LoFTR bắt buộc phải dùng chú ý tuyến tính. Sau đó một tầng ghép khả vi — *softmax* hai chiều hoặc vận tải tối ưu — cho ra tập cặp ghép ở mức lưới thô.

Giai đoạn tinh chạy trên từng cặp đã thắng. Nó cắt một cửa sổ nhỏ quanh mỗi cặp trên bản đồ đặc trưng ở một phần hai độ phân giải, tương quan hai cửa sổ, rồi lấy kỳ vọng trên bản đồ tương quan để có vị trí dưới mức điểm ảnh. Đây đúng là mẫu thiết kế đã gặp ở XFeat: thô trên toàn ảnh ở độ phân giải thấp, tinh chỉ ở nơi đã biết là cần.

B. Vì sao bỏ bộ phát hiện lại cứu được vùng ít vân

Câu trả lời hay bị nói sai. Bộ phát hiện *không* làm bức tường trắng có vân. Nó đối thứ khác: *đổi loại bằng chứng* được phép dùng để quyết định một cặp ghép.

Bộ phát hiện dùng bằng chứng cục bộ và quyết ngay. Trên tường trắng thì bằng chứng cục bộ bằng không, nên quyết định nào cũng sai. Chồng tầng chú ý của LoFTR làm cho biểu diễn của một ô ở giữa bức tường mang thông tin từ mép tường, từ khung cửa, từ bố cục cả căn phòng. Ô đó vẫn “không có vân”, nhưng cặp ghép của nó được quyết bằng bằng chứng *phi cục bộ*. Hình 2.6 vẽ đúng khác biệt đó.



Hình 2.6. Vì sao bỏ bộ phát hiện cứu được vùng ít vân. Bên trái, cửa sổ bán kính 3 ở giữa bức tường không chứa thông tin nào, nên mọi ngưỡng đều sai. Bên phải, chồng tầng chú ý đưa thông tin từ mép tường và từ bố cục phòng vào chính ô đó, nên cặp ghép được quyết bằng bằng chứng phi cục bộ. Cái mất nằm ở ô cuối bên phải: cặp ghép có, nhưng điểm đó không mang một danh tính có thể tìm lại ở khung hình sau — và danh tính mới là thứ SLAM cần để dựng bản đồ.

C. Bốn cái giá, và cái thứ tư mới là cái giết

- **Chi phí sần theo diện tích ảnh, không theo số điểm.** Bạn không giảm được chi phí bằng cách xin ít điểm hơn. Sần được đặt bởi độ phân giải, và nó cao.
- **Cặp ghép neo vào lưới.** Bước tinh chỉnh lấy lại độ chính xác vị trí, nhưng không lấy lại được tính lặp lại của *vị trí trên vật thật* qua nhiều khung hình.
- **Chỉ hai ảnh một lúc.** LoFTR ghép ảnh với ảnh. SLAM cần một điểm bản đồ được quan sát bởi nhiều khung khoá, tức là một vệt theo dõi. Nối các cặp ghép hai-ảnh thành vệt thì tích lũy sai lệch và đứt ngay khi bỏ qua một khung.
- **Không có bộ mô tả để lưu.** Đây là cái giá quyết định. Bám khung hình là bài toán một-nhiều: ghép khung mới với hàng vạn điểm bản đồ. Có bộ mô tả thì đó là một phép tìm láng giềng gần nhất. Không có bộ mô tả thì bạn phải chạy mạng *một lần cho mỗi cặp ảnh ứng viên*. Với N khung khoá là N lần chạy mạng. Và túi từ nhị phân [39] — nền của cả định vị lại lẫn lấy ứng viên vòng — không còn dùng được, vì nó cần bộ mô tả.

D. RoMa và nhánh ghép dày mạnh nhất

RoMa đẩy ý tưởng đi xa hơn: thay vì một tập cặp ghép, nó hồi quy một trường biến dạng dày cho toàn ảnh, kèm một bản đồ độ chắc chắn theo từng điểm ảnh [40]. Đặc trưng thô lấy từ một mô hình nền thị giác đông lạnh, đặc trưng tinh lấy từ một mạng tích chập chuyên dụng. Điểm mạnh nhất của nó là các cặp ảnh cực khó: đổi góc nhìn rất lớn, đổi mùa, đổi ánh sáng. Đây là loại cặp ảnh mà mọi phương pháp thừa đều bỏ cuộc.

Cái giá tương ứng. Xương sống là một *transformer* lớn, nên chi phí ở bậc hoàn toàn khác so với XFeat hay LightGlue. Với ngân sách một khung hình trên Orin thì nhánh này nằm ngoài tầm, và mình không thấy lý do kỹ thuật nào để nghĩ khác. Có một bản kế nhiệm là RoMa v2, công bố năm 2025, được giới thiệu là nhanh hơn và dày hơn; mình chưa đọc kỹ nên chỉ nhắc tên và không dẫn bất kỳ con số nào. Ở nhánh nhẹ hơn, Efficient LoFTR kéo chi phí của ghép bán dày về gần mức của bộ ghép thưa bằng cách gộp phép chú ý và làm gọn giai đoạn thô [41].

E. Nếu vẫn muốn dùng ghép dày trong SLAM thì phải thêm gì

Giả sử bạn quyết định dùng LoFTR ở tầng đóng vòng. Có ba mảnh còn thiếu, và biết trước ba mảnh đó giúp bạn ước lượng đúng khối lượng công việc.

- **Một tầng lấy ứng viên.** Ghép dày chỉ trả lời câu “hai ảnh này khớp tới đâu”. Nó không trả lời câu “ảnh nào trong một vạn khung khoá là ứng viên”. Bạn vẫn cần bộ mô tả toàn ảnh cho bước đó, và đó là nội dung chương 29.
- **Một cách gán danh tính.** Sau khi có cặp ghép giữa khung mới và khung khoá, bạn phải nối chúng vào các điểm bản đồ đã tồn tại. Cách rẻ là chiếu ngược: điểm ảnh được ghép nằm gần điểm đặc trưng ORB nào của khung khoá thì nhận danh tính của điểm bản đồ đó. Cách này hoạt động, và nó cho thấy hai họ bổ sung nhau chứ không loại trừ nhau.
- **Một mô hình bất định.** Cặp ghép của LoFTR không có tầng kim tự tháp, nên không có `mvInvLevelSigma2`. Bạn phải lấy trọng số từ chỗ khác — điểm số tin cậy của tầng ghép, hoặc độ nhọn của bản đồ tương quan ở bước tinh chỉnh. Đây lại là đúng vấn đề của mục 6E, xuất hiện lần thứ hai, và sự lặp lại đó là dấu hiệu nó là vấn đề trung tâm chứ không phải chi tiết.

Bảng 2.3. Ba họ ghép cặp, so theo cùng tiêu chí, kèm cột quan trọng nhất với bạn: nên đặt nó ở đâu trong hệ SLAM. Cột chi phí là bậc tương đối theo số phép nhân cộng, không phải số đo.

Họ	Cơ chế	Mạnh khi nào	Hồng khi nào	Đặt ở đâu
Thừa cổ điển (ORB)	Láng giềng gần nhất theo Hamming, phép thử tỉ lệ, biểu đồ góc	Khung nhìn hẹp, có tiên nghiệm tốt, cần độ trễ thấp và đoán trước được	Ảnh mờ, đối sáng mạnh, đối góc nhìn lớn, vùng ít vân	Bám khung hình ở 30 Hz
Thừa học được (SuperPoint + LightGlue)	Đặc trưng học được cộng gán toàn cục bằng <i>attention</i>	Khung nhìn rộng, cấu trúc lặp, đối sáng; vẫn cho bộ mô tả lưu lại được	Ra ngoài phân phối huấn luyện; chi phí bậc hai theo số điểm	Kiểm chứng ứng viên đóng vòng, định vị lại
Dây không bộ phát hiện (LoFTR, RoMa)	Ghép mọi ô lưới, không chọn điểm; thô rồi tinh	Vùng ít vân, đối góc nhìn cực lớn, cặp ảnh khó nhất	Không có bộ mô tả để lưu; chỉ hai ảnh một lúc; chi phí theo diện tích ảnh	Ghép ngoại tuyến, tái dựng, hoặc kiểm chứng vòng khi có ngân sách

Kết luận của mục này trùng với kết luận của mục 4, và sự trùng đó đáng chú ý. Cả hai nhánh — ghép cặp học được và ghép dây không bộ phát hiện — đều quá đắt cho mỗi khung hình, và đều mạnh nhất đúng ở chỗ không có tiên nghiệm chuyển động. Chỉ có một chỗ trong hệ SLAM thoả cả hai điều kiện đó: **kiểm chứng ứng viên đóng vòng và định vị lại**. Đó là nơi nên đặt chúng.

→ **hệ quả của F/29** — chương 29 nhận đúng kết luận này làm điểm xuất phát và đo xem tầng nào của đóng vòng thật sự là nút thắt

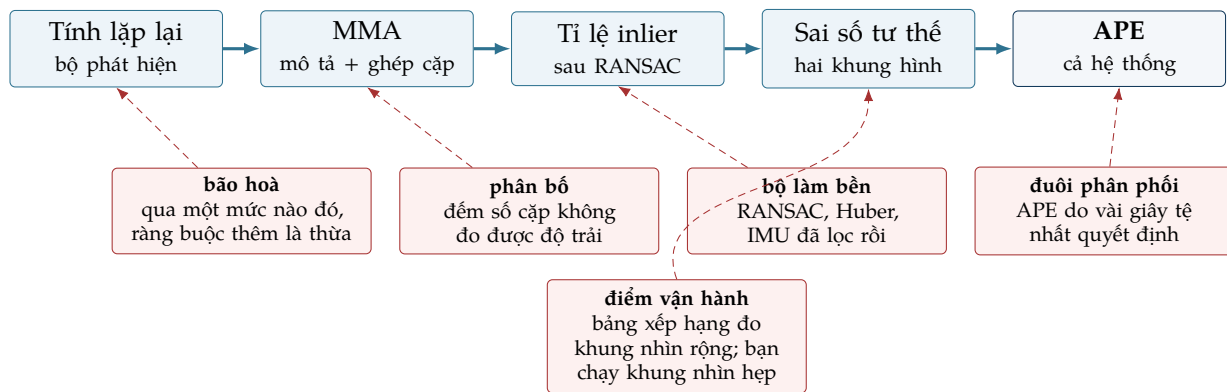
2.6 Đánh giá cho hệ thời gian thực (Evaluation for a Real-time System)

Đây là mục quan trọng nhất của chương. Ba mục trước là bản đồ; mục này là cái cân.

A. Bốn thước đo, mỗi cái cô lập một khối

- **Tính lặp lại** — tỉ lệ điểm được tìm thấy ở cùng vị trí trên vật thật trong hai ảnh. Nó cô lập riêng *bộ phát hiện*.
- **Độ chính xác ghép trung bình (MMA)** — trong các cặp được ghép, tỉ lệ cặp sai dưới ngưỡng vài điểm ảnh. Nó đo *bộ mô tả cộng bộ ghép cặp*, thường trên HPatches [42].
- **Tỉ lệ inlier sau RANSAC** — mức sạch của tập đưa vào tối ưu. Nó đo *toàn bộ frontend cộng mô hình hình học*.
- **APE** — con số duy nhất mà hệ của bạn bị đánh giá.

Điểm mấu chốt của cả chương nằm ở đây: **ba thước đo đầu có thể tăng mạnh mà APE không đổi một milimét**. Đây không phải nghịch lý, và nó có đúng năm cơ chế. Hình 2.7 vẽ chỗ tín hiệu bị hấp thụ ở từng nấc.



Hình 2.7. Thang đo từ frontend tới APE, và năm chỗ tín hiệu bị hấp thụ trên đường đi. Mỗi ô đỏ là một cơ chế độc lập; chúng cộng dồn, nên một cải thiện lớn ở nấc trái có thể tới nấc phải mà không còn gì. Hệ quả thực hành: đừng báo cáo cải tiến ở ba nấc trái rồi ngụ ý nấc phải. Hãy đo nấc phải, hoặc đo trần của nó bằng *oracle*.

B. Năm cơ chế hấp thụ, nói rõ từng cái

1. **Bảo hoà.** Tư thế có sáu bậc tự do, và mỗi cặp ghép cho hai phương trình. Vài chục cặp trải đều đã thừa sức xác định sáu ẩn đó; vượt qua mức bảo hoà — mức mà bạn đo được bằng cách quét số điểm và nhìn APE — mỗi cặp ghép thêm chỉ làm ma trận thông tin dày lên theo những hướng vốn đã chắc. Ràng buộc chặt chuyển sang chỗ khác: đường cơ sở, phân bố độ sâu, và trọng số IMU.
2. **Phân bố quan trọng hơn số lượng.** Rất nhiều cặp ghép dồn vào một góc ảnh cho ma trận thông tin điều kiện xấu hơn hẳn một số cặp ít hơn nhiều nhưng trải đều khắp khung hình. Không thước đo nào ở trên đo độ trải. Nếu muốn một chỉ số tốt hơn, hãy đo trị riêng nhỏ nhất của ma trận thông tin tư thế, không phải đếm cặp ghép.
3. **Bộ làm bền đã ăn phần lớn lợi ích.** Hệ của bạn có RANSAC, mô hình vận tốc không đổi, hàm mất mát Huber trong tối ưu, và ràng buộc IMU. Bốn thứ đó đã loại phần lớn cặp ghép xấu. Làm sạch đầu vào chỉ khiến chúng đỡ vất vả hơn. Đầu vào tốt hơn chỉ tạo khác biệt ở đúng nơi bộ làm bền đã *thất bại*, mà nơi đó chiếm rất ít thời lượng.
4. **Đuôi phân phối quyết định APE.** APE bị đặt bởi vài giây tệ nhất của chuỗi. Ba thước đo kia là trung bình trên toàn tập kiểm thử. Một cải thiện trung bình lớn hoàn toàn tương thích với việc vài giây tệ nhất không đổi gì.
5. **Sai điểm vận hành.** Bảng xếp hạng đo cặp ảnh khung nhìn rộng, còn bạn ghép hai khung hình cách nhau 33 mili-giây. Ở khung nhìn hẹp có tiên nghiệm tốt, ORB đã gần tối ưu, nên khoảng trống để cải thiện vốn đã nhỏ.

C. Thí nghiệm quyết định: *oracle* ghép cặp

Từ năm cơ chế trên suy ra một quy trình rất gọn, và nó nên chạy *trước* khi tích hợp bất cứ mạng nào.

Thay đầu ra của bộ ghép cặp bằng cặp ghép dựng từ chân trị — chiều điểm bản đồ bằng tư thế chân trị và độ sâu chân trị, rồi lấy cặp ghép trong bán kính nhỏ. Chạy lại toàn hệ. APE thu được là **trần** mà mọi cải tiến frontend bị chặn trên. Nếu trần đó chỉ tốt hơn APE hiện tại vài phần trăm, bạn vừa tiết kiệm được ba tháng. Nếu nó tốt hơn hẳn, bạn vừa biết đầu tư vào đâu và biết trước lợi ích tối đa.

Chạy thêm hai biến thể sẽ tách được câu hỏi ra rõ hơn nữa. Biến thể một: chỉ bật *oracle* trong các đoạn ảnh mờ, để đo riêng phần đóng góp của chế độ hồng thứ nhất. Biến thể hai: giữ nguyên cặp ghép của ORB nhưng thay *trọng số* của chúng bằng trọng số lý tưởng, để tách “ghép sai” khỏi

“ghép đúng nhưng cân sai”. Nếu biến thể hai cải thiện nhiều hơn biến thể một, thì bài toán của bạn nằm ở chương 34 chứ không nằm ở chương này.

→ **trường hợp riêng của E/24** — đây đúng là *oracle* experiment của chương 24, áp cho một khối cụ thể; giá trị của nó vẫn là chặn trên công sức trước khi bỏ công sức ra

D. Ngân sách trên Jetson Orin

Ở 30 khung hình mỗi giây, ngân sách là 33 mili-giây một khung, và đó là ngân sách cho *tất cả*: rút trích, bấm, quản lý bản đồ cục bộ, và phần dành sẵn cho biến động. Cách kế toán thì gọn: liệt kê từng khoản đang tiêu, đo từng khoản ở phân vị 95 chứ không ở trung vị vì phân vị mới là thứ làm bỏ khung, cộng lại, rồi lấy trần trừ đi. Chương này không đưa một bảng số mẫu, vì một bảng số chưa đo trên máy của bạn chỉ tạo cảm giác an tâm sai và rồi sẽ bị trích lại như thể nó là số đo. Nhưng ba kết luận về *cấu trúc* của bảng đó thì đúng trước cả khi bạn đo, và chúng mới là phần đáng nhớ.

- **Phần dự phòng cho biến động là khoản bị ăn mất trước tiên.** Nó không có tên trong bất kỳ hồ sơ thời gian nào, nên nó bị tiêu mà không ai ghi sổ. Khi nó hết, triệu chứng không phải chậm đều mà là bỏ khung từng đợt — và bỏ khung từng đợt trông giống hết một lỗi thuật toán, nên nó ăn mất rất nhiều thời gian gỡ lỗi.
- **Một bộ đặc trưng nhẹ chỉ vừa nếu nó thay chứ không thêm.** Nếu bạn vẫn chạy ORB song song để đối chứng, hãy tính đó là hai khoản chứ không phải một, và đừng dùng lần đo có chạy song song ấy để kết luận về ngân sách lúc vận hành.
- **Một bộ ghép cặp đầy đủ chạy mỗi khung thì gần như chắc chắn vượt trần.** Đây là hệ quả của chi phí bậc hai ở mục 4B chứ không của một con số cụ thể nào, nên nó không đôi khi bạn đổi phần cứng; cái đổi chỉ là số điểm mà tại đó nó bắt đầu vượt. Lối thoát duy nhất là hạ tần suất chạy, và đó chính là lỗi hai ở mục 6F.

Ba điều nữa mà kế toán trên chưa thể hiện. Thứ nhất, mạng phải qua TensorRT với kích thước đầu vào cố định, và bước chuyển đổi này thường lộ ra các phép toán không được hỗ trợ. Thứ hai, GPU trên Orin đã bị dùng cho những việc khác, nên độ trễ đo lúc chạy một mình không phải độ trễ lúc chạy trong hệ thật. Thứ ba, độ trễ của mạng gần như hằng số, còn độ trễ của ORB thay đổi theo nội dung ảnh — đổi sang mạng làm hệ dễ dự đoán hơn, và đó là một lợi ích thật, ít khi được kể tới.

E. Tích hợp vào ORB-SLAM3 thì phải bỏ những gì

Đây là danh sách mà mọi bài báo đều bỏ qua, và là chỗ chi phí thật nằm. Thay bộ đặc trưng không phải thay một hàm; nó gỡ ra bốn thứ đã ăn sâu vào toàn hệ thống.

- **Từ điển túi từ.** DBow2 được huấn luyện trên bộ mô tả nhị phân ORB. Bộ mô tả số thực làm nó vô dụng ngay lập tức. Mất túi từ là mất cả lấy ứng viên đóng vòng lẫn định vị lại sau khi mất bấm. Bạn phải hoặc huấn luyện lại một từ điển mới, hoặc chuyển sang bộ mô tả toàn ảnh — và đó là nội dung của chương 29.
- **Ngưỡng Hamming đã tinh chỉnh nhiều năm.** TH_HIGH bằng 100 và TH_LOW bằng 50 không còn nghĩa gì với khoảng cách L2. Mỗi chỗ gọi phải suy lại ngưỡng, và trong ORB-SLAM3 có hàng chục chỗ gọi như vậy, mỗi chỗ dùng một *mfNNratio* riêng.
- **Bất biến tỉ lệ theo kim tự tháp — và đây là chỗ nguy hiểm nhất.** Tầng kim tự tháp của mỗi điểm không chỉ dùng cho việc phát hiện. Nó làm ba việc nữa. Nó đặt bán kính tìm kiếm khi chiếu điểm bản đồ xuống ảnh. Nó đặt kiểm tra nhất quán tỉ lệ khi quyết định một điểm bản đồ có quan sát được không. Và quan trọng nhất, nó đặt *ma trận thông tin của mọi cạnh tái chiếu* trong tối ưu, qua *mvInvLevelSigma2[octave]*. Một bộ phát hiện học được không trả về

tầng kim tự tháp. Nếu bạn điền một hằng số vào đó, bạn vừa đặt trọng số bằng nhau cho mọi phương trình trong bài toán tối ưu, một cách âm thầm. Thay frontend hoá ra là thay cả backend.

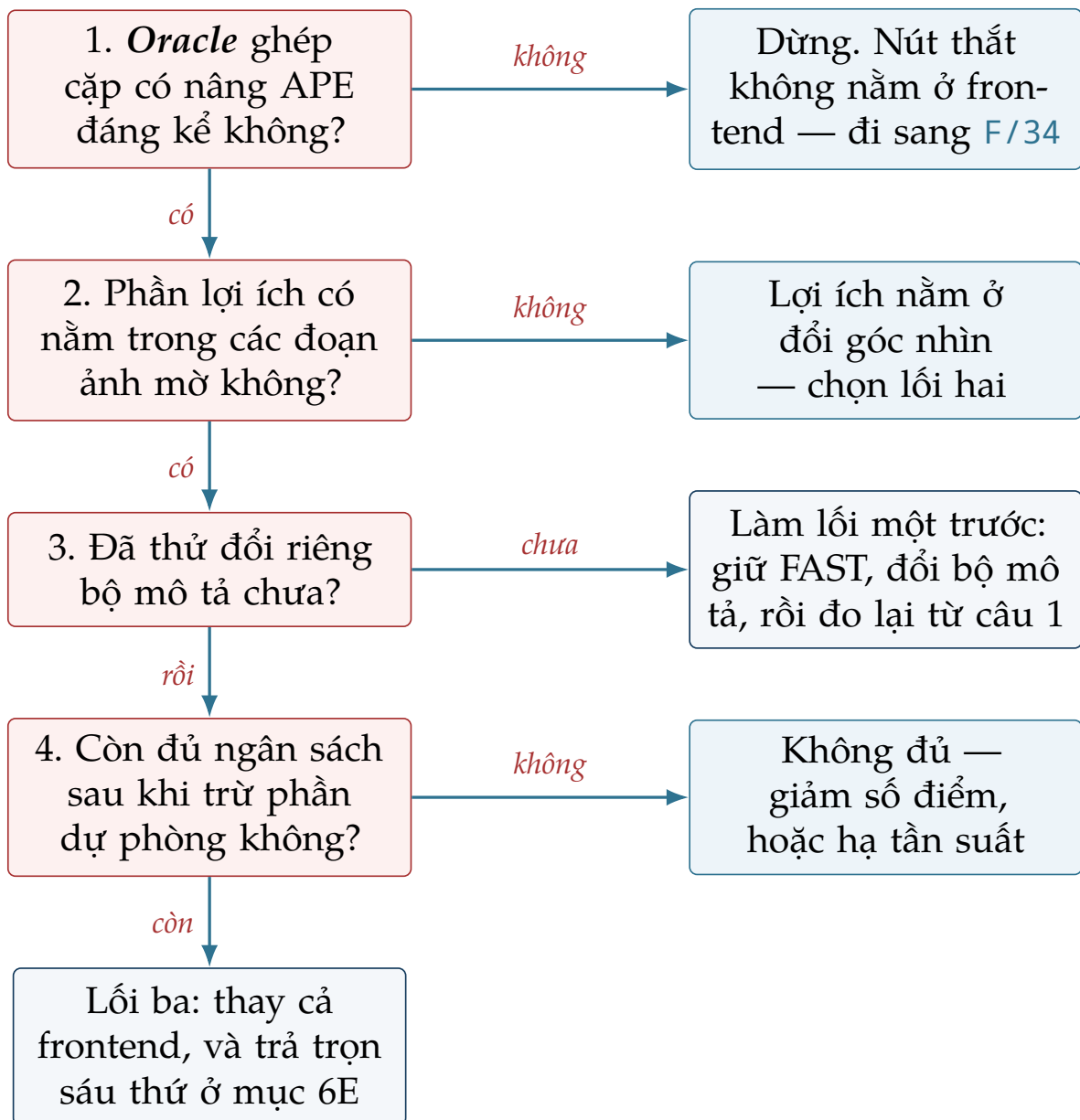
- **Bộ mô tả đại diện của điểm bản đồ.** MapPoint chọn bộ mô tả đại diện bằng cách lấy cái có khoảng cách Hamming trung vị nhỏ nhất tới các quan sát khác. Với vector số thực thì khái niệm đó phải định nghĩa lại, và trung bình không tương đương trung vị.
- **Phân bố đều theo cây tám nhánh.** DistributeOctTree ép các điểm trải đều trên ảnh. Cơ chế triệt phi cực đại của một mạng có hành vi khác hẳn. Nhớ lại cơ chế hấp thụ số hai: độ trải là thứ quyết định điều kiện của bài toán, nên đây không phải chi tiết nhỏ.
- **Các hằng số cống.** Số đặc trưng 1200, iniThFAST 12, minThFAST 4, và các cổng dạng “dưới 15 cặp thì coi như mất bám”. Tất cả đều được chỉnh cho phân phối số cặp ghép của ORB. Bộ đặc trưng mới cho phân phối khác, nên các cổng đó phải chỉnh lại từ đầu.

F. Ba lối đi giữ được phần lớn hệ thống

Danh sách trên nghe như một lời từ chối. Nó không phải. Nó chỉ nói rằng “thay ORB” là lựa chọn đắt nhất trong ba lựa chọn, và hai lựa chọn kia ít khi được nhắc tới.

- **Lối một — giữ nguyên bộ phát hiện, chỉ đổi bộ mô tả.** Vẫn chạy FAST trên kim tự tháp, vẫn có tầng, vẫn có phân bố đều theo cây tám nhánh, nhưng tính bộ mô tả bằng HardNet hoặc bằng phần mô tả của SuperPoint trên các ô ảnh đã cắt. Lối này giữ nguyên toàn bộ ba chức năng của tầng kim tự tháp, nên ma trận thông tin của backend không đổi. Nó chứa chế độ hồng số hai và một phần số ba, và không chứa số một. Chi phí thêm rất thấp, vì mạng chỉ chạy trên vài trăm ô ảnh nhỏ chứ không trên toàn ảnh. Theo mình đây là lối có tỉ lệ lợi ích trên rủi ro tốt nhất nếu bạn bắt buộc phải đổi một thứ ở mỗi khung hình.
- **Lối hai — giữ nguyên ORB, chỉ thêm một bộ ghép cặp học được ở tầng đóng vòng.** Không đụng gì tới đường bám khung hình. Bộ ghép cặp chạy ở cỡ 1 Hz, chỉ trên các cặp ứng viên đã qua bước lấy ứng viên. Nó nhắm đúng chế độ hồng số ba, và đó là chế độ mà bám khung hình không bao giờ gặp. Rủi ro thấp vì nếu nó hỏng thì bạn chỉ mất một vòng, không mất bám.
- **Lối ba — thay cả frontend.** Chỉ nên chọn khi *oracle* cho thấy trần rất cao và lối một đã được thử và không đủ. Khi đó bạn phải trả trọn danh sách sáu thứ ở mục trên, và phải tự đặt lại một mô hình bất định thay cho tầng kim tự tháp.

Hình 2.8 gói cả chương thành một cây quyết định. Nó chỉ có bốn câu hỏi, và ba trong bốn câu trả lời được bằng hai ngày làm việc chứ không bằng ba tháng.



Hình 2.8. Cây quyết định cho câu hỏi “có nên thay frontend không”. Ba trong bốn câu hỏi trả lời được bằng hai ngày đo đặc, và câu đắt nhất — câu số bốn — chỉ được hỏi sau khi ba câu kia đã cho phép. Chú ý hai nhánh sớm: nếu *oracle* cho trần thấp, hoặc nếu phần lợi ích không nằm trong các đoạn ảnh mờ, thì cả chương này không áp dụng cho hệ của bạn.

G. Dựng một cái cân của riêng bạn

Vấn đề với HPatches và các bảng xếp hạng là chúng đo sai điểm vận hành. Cách sửa không phải bỏ đo, mà là dựng một cái cân từ chính dữ liệu của bạn. Việc này tốn nửa ngày và dùng được mãi.

Nguyên liệu là một chuỗi EuRoC có chân trị tư thế và có ảnh stereo. Từ tư thế chân trị và độ sâu stereo, bạn dựng được cặp ghép đúng cho bất kỳ cặp khung hình nào: chiếu điểm từ khung i sang khung j , và chấp nhận nếu sai lệch dưới hai điểm ảnh. Đó là chân trị ghép cặp, không cần gán tay.

Điều quan trọng là *cách chọn cặp khung hình*, vì đó chính là chỗ các bảng xếp hạng sai. Hãy chia các cặp thành ba nhóm theo đúng ba chế độ vận hành của hệ bạn.

- **Nhóm tám** — cặp khung hình liên tiếp, cách nhau 33 mili-giây. Đây là tuyệt đại đa số các lần ghép cặp trong một lần chạy, nên nó phải là nhóm có trọng số lớn nhất khi bạn đọc bảng.
- **Nhóm mờ** — cặp khung hình liên tiếp nhưng lấy trong các đoạn có tốc độ góc cao nhất chuỗi, lọc từ IMU; ngưỡng thì lấy từ chính phân bố tốc độ góc của bạn, chẳng hạn phân vị trên cùng, chứ không lấy từ một con số cho sẵn.
- **Nhóm vòng** — cặp khung hình cách nhau trong thời gian nhưng gần nhau trong không gian, lấy từ chân trị. Đây là chế độ vận hành của đóng vòng.

Rồi đo cùng ba con số cho từng nhóm: số cặp ghép đúng, tỉ lệ inlier, và *độ trái* của các cặp ghép trên ảnh. Độ trái đo bằng trị riêng nhỏ nhất của ma trận hiệp phương sai toạ độ các điểm được ghép, hoặc đơn giản hơn là số ô còn có điểm khi chia ảnh thành lưới bốn nhân bốn.

Bảng ba nhóm nhân ba con số này nói được điều mà không bảng xếp hạng nào nói: phương pháp nào tốt hơn ở *nhóm nào*. Dự đoán của mình, và đây là dự đoán chứ không phải kết quả đo: các phương pháp học được sẽ hơn rất ít ở nhóm tám, hơn rõ ở nhóm mờ, và hơn nhiều ở nhóm vòng. Nếu bảng của bạn ra đúng hình dạng đó thì cây quyết định ở Hình 2.8 đưa bạn thẳng tới lối hai.

H. Điền bốn cổng của chương 27 cho ba họ

Gộp tất cả lại, Bảng 2.4 là câu trả lời của chương này cho khung bốn cổng. Đây là bảng nên chụp lại và dán lên tường.

Bảng 2.4. Bốn cổng của chương 27, điền cho ba họ phương pháp. Cột cuối là kết luận của mình tính tới tháng 9/2026, và nó có điều kiện: nó giả định bạn *đã* có một hệ stereo-inertial tinh chỉnh kỹ chạy trên Orin.

Họ	Bài toán có thật?	Đo được?	Vừa ngân sách Orin?	Kết luận
Bộ mô tả học được trên điểm ORB	Có: chế độ hồng đo đổi sáng, đo được bằng biên δ	Có: nhóm mờ và nhóm vòng của mục 6G	Nhiều khả năng vừa: mạng chỉ chạy trên vài trăm ô ảnh nhỏ	Đáng thử trước tiên
Đặc trưng học được thay cả FAST	Có, nhưng chỉ chứa hai trong bốn chế độ hồng	Có, nhưng phải đo lại toàn bộ chuỗi vì tăng kim tự tháp mất	Cần đo; XFeat và ALIKED là hai ứng viên duy nhất đáng thử	Chỉ khi <i>oracle</i> cho trần cao
Bộ ghép cặp học được mỗi khung	Bài toán gán toàn cục không phải nút thất lúc tám	Có, nhưng cải thiện để bị năm cơ chế hấp thụ	Gần như chắc chắn không, do chi phí bậc hai	Chưa đáng
Bộ ghép cặp học được ở tầng vòng	Có: đây đúng là chế độ hồng thứ ba	Có: đo bằng recall vòng, xem chương 29	Có, vì chỉ chạy ở cỡ 1 Hz trên vài cặp ứng viên	Đáng, và là phương án tốt nhất
Ghép dày không bộ phát hiện	Có: vùng ít vân là chế độ hồng thật	Khó, vì phải dựng thêm tầng danh tính mới đo được trong hệ	Không cho mỗi khung; có thể cho tầng vòng nếu tối ưu kỹ	Chưa đáng, trừ khi cảnh của bạn rất ít vân

Một lưu ý về cách đọc bảng này. Nó không nói các phương pháp ở hàng dưới kém hơn. Nó nói chúng được thiết kế cho một điểm vận hành khác, và ở điểm vận hành đó chúng thắng rõ. Nhầm hai câu này là nhầm phổ biến nhất khi đọc văn liệu ghép cặp.

CẦN NHỚ

Trước khi tích hợp bất cứ mạng nào, hãy chạy *oracle* ghép cặp: thay đầu ra của bộ ghép cặp bằng cặp ghép dựng từ chân trị, rồi đo APE. Con số đó là *trần* của mọi công sức frontend. Nếu trần chỉ hơn hiện tại vài phần trăm thì câu trả lời cho cả chương này là “chưa đáng”, và bạn biết điều đó sau hai ngày thay vì sau ba tháng. Nếu vẫn muốn đi tiếp, hãy đi theo

con đường rẻ nhất: giữ ORB cho bám khung hình ở 30 Hz, và chỉ đặt bộ ghép cặp học được ở tầng kiểm chứng đóng vòng, chạy ở cỡ 1 Hz.

2.7 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của chương này là giới hạn của bằng chứng. Gần như mọi con số so sánh đặc trưng đều đo trên cặp ảnh hai khung nhìn rộng, không có tiên nghiệm chuyển động, không có ràng buộc thời gian thực. Chưa có một nghiên cứu ablation nào đủ sạch trả lời câu “thay ORB bằng X thì APE trên EuRoC đổi bao nhiêu, với cùng ngân sách tính toán”. Vài công trình có báo cáo, nhưng chúng thường đổi nhiều thứ cùng lúc và không cố định ngân sách. Nên mọi kết luận trong chương này về APE là *suy luận từ cơ chế*, không phải kết quả đã đo, và mình muốn nói rõ điều đó.

Chế độ hỏng thứ hai là hỏng khi ra ngoài phân phối huấn luyện. Bộ đặc trưng học được mang theo giả định của dữ liệu huấn luyện nó. Nếu cảnh của bạn là hầm mỏ, ban đêm, hoặc ảnh hồng ngoại, thì mọi thứ hạng trên bảng xếp hạng ngoài trời ban ngày đều không chuyển sang được. Tệ hơn, kiểu hỏng cũng đổi: ORB hỏng bằng cách không cho điểm nào, còn mạng hỏng bằng cách cho ra các điểm tự tin nhưng sai. Kiểu hỏng thứ hai độc hơn nhiều với một đường ống có RANSAC, vì các cặp sai *nhất quán* với nhau thì RANSAC không loại được.

Chế độ hỏng thứ ba mang tính hệ thống. Chương này giả định frontend tách rời được khỏi phần còn lại, để có thể thay riêng nó. Mục 6E cho thấy giả định đó sai ở một chỗ cụ thể: tầng kim tự tháp là cầu nối giữa frontend và ma trận thông tin của backend. Khi một khối bị nối với khối khác qua một kênh không có tên trong tài liệu như vậy, ablation cho ra con số không có nghĩa như ta tưởng — đúng cảnh báo cuối chương 24.

Cuối cùng là giới hạn về tính chẩn đoán được. Khi ORB không tìm ra điểm, bạn mở ảnh lên và thấy ngay vì sao. Khi một mạng cho ra tập điểm kỳ lạ ở một cảnh cụ thể, bạn không có cách nào đi ngược từ triệu chứng về nguyên nhân trong vài giờ. Với một hệ đang chạy sản phẩm, mất khả năng chẩn đoán là một chi phí thật, và nó không xuất hiện trong bất kỳ bảng so sánh nào.

Giới hạn thứ năm là phạm vi mà chương này cố ý bỏ qua. Frontend của ORB-SLAM3 không chỉ ghép giữa hai khung hình liên tiếp. Nó còn ghép giữa hai ảnh stereo bằng ràng buộc đường epipolar, với tinh chỉnh dưới mức điểm ảnh dọc đường quét, để lấy độ sâu. Nó còn ghép giữa khung hình và bản đồ cục bộ theo phép chiếu. Ba bài toán ghép đó có ràng buộc khác nhau và điểm vận hành khác nhau. Một bộ đặc trưng học được phải làm tốt cả ba, nhưng mọi bảng xếp hạng chỉ đo bài toán thứ nhất. Đây là một khoảng trống bằng chứng nữa, và nó cùng loại với khoảng trống đã nói ở hộp cạm bẫy.

Cuối cùng là một giới hạn về chính chương này. Nó viết theo giọng hoài nghi, và giọng đó phù hợp với một người *đã có* một hệ thống chạy tốt. Với một người đang xây từ đầu, cân nhắc đảo ngược: một đường ống dựa trên SuperPoint cộng LightGlue cho kết quả chấp nhận được nhanh hơn nhiều so với việc tinh chỉnh ORB trong hai năm. Kết luận “chưa đáng” của chương này là kết luận có điều kiện, và điều kiện đó là hệ thống hiện có của bạn.

2.8 Thực hành

Repo và tài nguyên. [kornia](#) có sẵn HardNet, DISK, LoFTR và LightGlue dưới một giao diện chung, nên nó là chỗ rẻ nhất để so nhiều phương pháp mà không dựng lại từng repo. [magicleap/SuperGluePretrainedNetwork](#) và [cvgl/LightGlue](#) cho hai bộ ghép cặp;

`cvg/Hierarchical-Localization` cho một đường ống hoàn chỉnh từ đặc trưng tới định vị; `verlab/accelerated_features` cho XFeat; `zju3dv/LoFTR` cho nhánh không bộ phát hiện. Về hạ tầng dài hạn thì COLMAP và `opencv` bền hơn hẳn mọi tên mô hình ở trên. Danh sách này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; riêng bốn chế độ hồng ở mục 2 thì không cũ.

Câu hỏi kiểm tra hiểu. Viết công thức của độ dài vệt mờ tối hạn theo w, C, t , rồi nói ngưỡng tốc độ góc đổi theo hướng nào và theo tỉ lệ nào khi bạn hạ ngưỡng FAST xuống một nửa, và khi cảnh tối đi làm tương phản giảm một nửa? Nếu bạn giảm số điểm còn một phần ba thì chi phí của một bộ ghép cặp kiểu SuperGlue giảm mấy lần? Trong sáu thứ phải bỏ ở mục 6E, thứ nào chạm vào backend chứ không chỉ chạm vào frontend?

Mini project trong khối *duan bên dưới*. Hai dự án ở cuối chương nối tiếp nhau: một cái đo trần, một cái đo ngưỡng hồng. Chạy cái đo trần trước.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● FAST	Bộ phát hiện góc của ORB. Nó nhận một điểm là góc nếu có 9 điểm ảnh liên tiếp trên vòng tròn bán kính 3 cùng lệch quá ngưỡng t . Nó hồng khi ảnh mờ, vì trung bình theo hướng chuyển động kéo tương phản xuống dưới t và cắt đứt cung liên tiếp.
●●● BRIEF	Bộ mô tả nhị phân 256 bit, mỗi bit là kết quả một phép so sánh “điểm A sáng hơn điểm B”. Nó bất biến với đổi độ sáng đơn điệu toàn cục, nhưng hồng khi biên của các phép so sánh co lại và bit bắt đầu lật ngẫu nhiên.
●●● Biên δ của một phép so sánh	Chênh lệch cường độ thật của một cặp thử trong BRIEF. Xác suất lật bit là $\Phi(-\delta/\sigma\sqrt{2})$, nên chính δ chứ không phải độ sáng tuyệt đối quyết định bộ mô tả còn dùng được hay không.
●● MMA	Trong các cặp được ghép, tỉ lệ cặp có sai số vị trí dưới một ngưỡng điểm ảnh. Nó đo bộ mô tả cộng bộ ghép cặp, và nó là thước đo phổ biến nhất trên HPatches.
●● HardNet	Bộ mô tả học bằng mất mát triplet với mẫu âm khó nhất trong batch. Chọn mẫu âm khó nhất là cách xấp xỉ đúng đối thủ lúc chạy, tức ứng viên nhì trong phép thử tỉ lệ.
●●● Làm giàu bằng homography	Cách SuperPoint chế nhãn cho bộ phát hiện: áp hàng trăm homography ngẫu nhiên, phát hiện, ánh xạ ngược, rồi giữ các điểm sống sót. Giới hạn của nó là phân phối huấn luyện chỉ có biến dạng phẳng, không có ảnh mờ và không có thị sai.
●●● SuperPoint	Một mạng, hai đầu ra: bản đồ điểm và bộ mô tả 256 chiều, huấn luyện tự giám sát bằng homography. Nó là mặc định của cả dòng, nhưng chưa từng thấy ảnh mờ trong huấn luyện trừ khi bạn tự thêm vào.
●● R2D2	Tách tính lặp lại khỏi tính phân biệt thành hai bản đồ riêng. Lý do: góc của một viên gạch rất lặp lại nhưng ghép luôn sai, vì nó giống năm trăm viên khác.
● DISK	Coi chọn điểm và chấp nhận cặp ghép là hành động lấy mẫu, thường cặp đúng và phạt cặp sai, rồi học bằng <i>gradient</i> chính sách. Nó tối ưu thẳng mục tiêu rời rạc thay vì một hàm thay thế khả vi. DeDoDe đi hướng khác: tách hẳn việc huấn luyện bộ phát hiện khỏi bộ mô tả, để bộ phát hiện không bị kéo về phía sở thích của bộ mô tả.
●● XFeat	Đặc trưng thiết kế cho CPU và biên: mạng nông, chạy ở một phần tư độ phân giải, và tinh chỉnh vị trí bằng một mạng nhỏ chạy sau khi đã ghép cặp. Đây là công trình duy nhất trong dòng lấy ngân sách độ trễ làm ràng buộc thiết kế.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• ALIKED	Học các vị trí lấy mẫu biến dạng được cho vùng hỗ trợ của bộ mô tả, thay cho ô vuông cố định. Nó nhắm thẳng chế độ hồng do đối góc nhìn, vì ô vuông cố định méo khi mặt phẳng nghiêng.
••• SuperGlue	Bộ ghép cặp dùng chín tầng chú ý xen kẽ trong và giữa hai ảnh, rồi giải bài toán gán bằng Sinkhorn có thùng rác. Nó đổi quyết định cục bộ lấy quyết định toàn cục, và trả bằng chi phí bậc hai theo số điểm.
••• LightGlue	Cùng ý tưởng SuperGlue nhưng rút ngắn ba chỗ: thoát sớm khi cặp ảnh dễ, cắt bỏ điểm không ghép được giữa chừng, và bỏ vòng lặp Sinkhorn. Cắt điểm là chỗ tiết kiệm lớn nhất vì chi phí là bậc hai.
••• LoFTR	Ghép không cần bộ phát hiện: ghép dày ở một phần tám độ phân giải bằng chú ý tuyến tính, rồi tinh chỉnh dưới mức điểm ảnh trong cửa sổ nhỏ. Nó cứu vùng ít vân, nhưng chỉ ghép được hai ảnh một lúc và không để lại bộ mô tả nào để lưu.
••• Oracle ghép cặp	Thay đầu ra bộ ghép cặp bằng cặp ghép dựng từ chân trị rồi đo APE. Kết quả là trần của mọi cải tiến frontend, và nó tốn hai ngày thay vì ba tháng.
••• Vì sao chi phí SuperGlue là bậc hai?	Vì mỗi điểm phải nhìn mọi điểm khác, nên ma trận điểm số có N^2 ô. Kể toán thô là $36N^2d$ phép nhân cộng cho chín tầng, tức nhân đôi số điểm thì chi phí gấp bốn.
••• Vì sao bỏ bộ phát hiện cứu được vùng ít vân?	Vì nó không làm tường có vân, mà đổi loại bằng chứng được dùng. Chồng tầng chú ý đưa thông tin từ mép tường vào ô ở giữa tường, nên cặp ghép được quyết bằng bằng chứng phi cục bộ thay vì cục bộ.
••• Vì sao thay bộ phát hiện lại là thay cả backend?	Vì tầng kim tự tháp không chỉ dùng để phát hiện. Nó đặt bán kính tìm kiếm, kiểm tra nhất quán tỉ lệ, và đặt ma trận thông tin của mọi cạnh tái chiếu qua $mvInvLevelSigma2$. Bộ phát hiện học được không trả về nó, nên thay frontend là thay luôn trọng số của backend.
••• Tỉ lệ inlier tăng mạnh mà APE không đổi — nghĩa là gì?	Nghĩa là frontend không phải ràng buộc chặt. Năm cơ chế hấp thụ: bão hoà sau vài chục cặp trái đều, phân bố quan trọng hơn số lượng, bộ làm bền đã lọc rồi, APE do đuôi phân phối quyết định, và điểm vận hành khác với bảng xếp hạng.
••• Vào hành lang tối, tracking vẫn bám nhưng bản đồ ngừng lớn — nghĩa là gì?	Khoảng cách Hamming của cặp ghép đúng đã trôi vào khoảng giữa TH_LOW và TH_HIGH . Cặp ghép còn qua được ngưỡng lỏng của bám khung hình, nhưng rớt ở ngưỡng chặt của tam giác hoá điểm mới.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
kornia	Hiện thực chung của HardNet, DISK, LoFTR, LightGlue dưới một giao diện — chỗ so sánh rẻ nhất	Hạ tầng; bền hơn tên mô hình
hloc	SuperPoint cộng SuperGlue hoặc LightGlue trong một đường ống định vị hoàn chỉnh	Đây là nơi dòng này thật sự được dùng trong sản phẩm
COLMAP với đặc trưng học được	Ghép cặp khung nhìn rộng để tái dựng ba chiều — chỗ ghép cặp học được thắng rõ ràng nhất	Không có ràng buộc thời gian thực, nên đúng điểm vận hành
Image Matching Challenge [38]	Bảng xếp hạng quyết định thứ hạng của cả dòng này	Các giải pháp thắng thường là tổ hợp nhiều mô hình, xa mức thời gian thực

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Chính ORBmatcher của bạn	Phép thử tỉ lệ và biểu đồ nhất quán góc là hai bản thô của đúng hai ràng buộc mà SuperGlue đưa vào trong mô hình	Ví dụ nội bộ; đọc lại mã theo góc nhìn này rất đáng

MINI PROJECT SLAM · Đo trần của frontend bằng *oracle* ghép cặp · 1–2 ngày

Mục tiêu	trả lời dứt điểm câu “đổi bộ đặc trưng thì APE tốt lên nhiều nhất là bao nhiêu”, trước khi tích hợp bất cứ mạng nào.
Dữ liệu	hai chuỗi EuRoC, một dễ và một khó có chuyển động nhanh (chẳng hạn V2_03), cùng chân trị tư thế.
Các bước	dựng một chế độ <i>oracle</i> trong ORBmatcher: với mỗi điểm bản đồ, chiếu nó bằng tư thế chân trị và nhận cặp ghép nếu điểm đặc trưng gần nhất nằm trong bán kính hai điểm ảnh, bỏ qua hoàn toàn khoảng cách mô tả. Chạy ba lần mỗi cấu hình để có sần nhiễu. So ba cấu hình: ORB thường, <i>oracle</i> toàn chuỗi, và <i>oracle</i> chỉ bật trong các đoạn có tốc độ góc cao nhất chuỗi — ngưỡng thì lấy từ dự án thứ hai bên dưới, tức là từ số đo của chính bạn.
Xong khi	bạn có một bảng ba dòng kèm độ lệch chuẩn, và phát biểu được “trần của mọi cải tiến frontend trên chuỗi này là X cm”. Kiểm tra chéo: hiệu giữa dòng hai và dòng ba cho biết bao nhiêu phần của trần đó nằm riêng trong các đoạn chuyển động nhanh.
Nối với	E/24 cho cách đọc con số và cách tính sần nhiễu; F/29 sẽ dùng lại đúng chế độ <i>oracle</i> này ở tầng đóng vòng.

MINI PROJECT SLAM · Tìm ngưỡng mờ giết FAST trên chính chuỗi của bạn · nửa ngày

Mục tiêu	biến công thức $d^* = wC/t$ ở mục 2A thành một con số của bạn, đo trên dữ liệu thật, rồi biến nó thành một cờ chặn đoán chạy trực tuyến.
Dữ liệu	một chuỗi EuRoC có IMU, cộng nhật ký số điểm FAST phát hiện được mỗi khung hình.
Các bước	ghi hai chuỗi số theo thời gian: chuẩn của tốc độ góc từ IMU, và số điểm đặc trưng rút được. Vẽ số điểm theo tốc độ góc dưới dạng biểu đồ tán xạ. Lập lại với iniThFAST bằng 20 và bằng 12 để thấy đường cong dịch chuyển. Song song đó, tự tính dự đoán của mô hình: đo w và C trên vài ô ảnh điển hình của chuỗi, lấy tiêu cự từ file hiệu chỉnh và thời gian phơi sáng từ nhật ký camera, rồi ráp vào $\omega \approx (wC/t)/(f \Delta t)$.
Xong khi	bạn đọc ra được từ biểu đồ một ngưỡng tốc độ góc mà quá đó số điểm sụp, và bạn có một con số dự đoán từ mô hình để so với nó. Hai con số đều là của bạn, không mượn của ai. Kiểm tra chéo: hạ ngưỡng FAST một nửa thì ngưỡng tốc độ góc đo được phải dịch theo đúng hướng mà công thức nói, và dịch cỡ đúng tỉ lệ; nếu không thì mô hình một chiều đang thiếu một cơ chế, và việc tìm ra cơ chế đó đáng giá hơn cả con số.
Nối với	nối thẳng vào dự án trên: các đoạn vượt ngưỡng chính là các đoạn cần bật <i>oracle</i> riêng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đại lượng bị mờ tấn công là tương phản cục bộ còn lại. Mô hình trung bình một chiều cho $C_{\text{hiệu dụng}} \approx C \min(1, w/d)$, nên FAST mờ khi vệt mờ đạt $d^* \approx wC/t$. Đổi sang tốc độ góc bằng $\omega \approx (d^*/f)/\Delta t$, với f là tiêu cự tính theo điểm ảnh và Δt là thời gian phơi sáng. Ba tham số quyết định là ngưỡng FAST, tiêu cự và thời gian phơi sáng; thay số của chính bạn vào thì ra ngưỡng của bạn. Ở đây không đưa con số vì nó chưa được đo trên hệ của bạn.
- A2. Vì thứ giết BRIEF không phải giá trị mà là biên, và chính xác hơn nữa là tỉ số giữa biên và nhiễu. Xác suất lật một bit là $\Phi(-\delta/\sigma\sqrt{2})$, nên Hamming kỳ vọng của cặp đúng là $256 \Phi(-\delta/\sigma\sqrt{2})$. Khi δ/σ tụt về cùng bậc với 1, hàm Φ vào vùng dốc nhất và Hamming nhảy vọt — đủ để vượt TH_LOW trong khi vẫn dưới TH_HIGH, tức là qua cửa lỏng nhưng rớt ở mọi cửa chặt.

- A3. Vì tiên nghiệm biến bài toán gán toàn cục thành bài toán gán trong một cửa sổ hẹp: mỗi điểm chỉ còn một nhóm ứng viên, nên chi phí từ N^2 xuống $N \cdot k$ với k là số ứng viên trong cửa sổ, gần như tuyến tính. Không bài báo nào tận dụng vì bảng xếp hạng của họ đo cặp ảnh hai khung nhìn rộng, nơi không tồn tại tiên nghiệm nào để tận dụng.
- A4. Vì năm cơ chế hấp thụ nằm giữa frontend và APE: bảo hoà sau vài chục cặp trái đều, phân bố quan trọng hơn số lượng, các bộ làm bền đã lọc phần lớn rồi, APE do vài giây tệ nhất quyết định trong khi ba thước đo kia là trung bình, và điểm vận hành của bảng xếp hạng khác hẳn của bạn.
- A5. Vì bỏ bộ phát hiện thì mất bộ mô tả, mà mất bộ mô tả thì mất danh tính của điểm. SLAM cần ghép một khung mới với hàng vạn điểm bản đồ, tức một-nhiều; ghép dày chỉ làm được ảnh với ảnh. Hệ quả kèm theo: túi từ nhị phân không dùng được nữa, nên mất luôn định vị lại và lấy ứng viên đóng vòng.

NÓI LẠI THẬT ĐƠN GIẢN

Hai người muốn gặp nhau ở một góc phố nhưng chỉ nói chuyện được qua điện thoại. Cách cũ: mỗi người tự ghi lại hai trăm năm mươi sáu điều nhỏ về chỗ mình đứng, kiểu “viên gạch bên trái sẫm hơn viên bên phải”, rồi hai bên đọc danh sách và so xem có khớp không. Cách này chạy rất nhanh và rất tốt, miễn là trời sáng và hai người đứng yên. Nếu có sương mù, hoặc nếu ai đó vừa chạy vừa nhìn, thì mọi điều nhỏ đều trở nên nửa vời, và hai danh sách toàn điều nửa vời thì khớp với mọi góc phố trong thành phố.

Cách thứ hai: đừng ghi riêng, hãy nói chuyện với nhau. “Tôi thấy một mái hiên đỏ bên trái, bạn có thấy không?” Nhờ trao đổi mà từng chi tiết mơ hồ được đặt vào ngữ cảnh, nên vẫn phân biệt được viên gạch này với viên gạch kia. Cái giá là cuộc gọi tốn thời gian, và thời gian tăng theo bình phương số chi tiết mỗi bên đem ra bàn.

Cách thứ ba: bỏ hẳn việc chọn ra vài chi tiết, so nguyên cả khung cảnh. Cách này ăn được cả bức tường trắng trơn. Nhưng nó không để lại một cuốn sổ nào, nên ngày mai muốn tìm lại đúng góc phố ấy thì phải gọi điện lại từ đầu, với từng người một.

Chỗ không rút gọn được: chuyện cái thước ở đầu ống lại quyết định trọng số ở cuối ống. Trong hệ thật, việc một điểm được nhìn thấy ở mức phóng to nào cũng chính là thứ quyết định người ta tin điểm đó tới đâu khi ghép mọi thứ lại. Đổi cách chọn điểm mà quên điều này thì bạn vô tình tuyên bố mọi quan sát đáng tin như nhau, và không có cách nào nói điều đó bằng ngôn ngữ đời thường mà không mất đúng phần quan trọng.

Một câu: *Đặc trưng học được chứa được hai trong bốn chỗ ORB gây, nhưng chỗ nó chứa lại hiếm khi là chỗ quỹ đạo của bạn đang hỏng — nên hãy đo trần trước, rồi mới tiêu tiền.*

2.9 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là tiền đề trực tiếp: chương này là lần đầu tiên khung bốn cổng được điền đầy đủ cho một khối cụ thể, và kết quả điền ra là “chưa đáng cho mỗi khung hình, đáng cho tầng đóng vòng”. [F/29-dinh-vi-lai-va-dong-vong](#) nhận đúng kết luận đó làm điểm xuất phát, và trả lời câu hỏi mà chương này để lại: nếu đặt bộ ghép cặp học được ở tầng đóng vòng thì tầng nào của đóng vòng thật sự được lợi. [F/30-do-sau-dong-quang-va-mo-hinh-nen](#) là nhánh đối lập: thay vì ghép cặp thừa tốt hơn, nó bỏ ghép cặp và dùng dòng quang dày — cùng nỗi đau chuyển động nhanh, cách tiếp cận ngược lại. [F/34-imu-bat-dinh-va-trien-khai](#) nhận phần chương này cố ý để lại: nếu *oracle* cho thấy vấn đề nằm ở *trọng số* chứ không ở *cặp ghép*, thì lời giải nằm ở đó. [E/24-thi-nghiem-va-ablation](#) là công cụ của cả chương, vì *oracle* experiment là thứ duy nhất biến câu hỏi “có đáng không” thành một con số. [C/16](#) giải thích cơ chế *attention* mà SuperGlue và LoFTR đều dựa vào.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ảnh mờ và đối độ sáng cần hai loại thuộc khác nhau, dù triệu chứng bề ngoài đều là “ít cặp ghép”?
2. Mô hình một chiều dự đoán một ngưỡng vết mờ $d^* = wC/t$, nhưng thực tế FAST chết sớm hơn ngưỡng đó — cơ chế nào giải thích phần chênh, và nó ảnh hưởng thế nào tới điều kiện của bài toán tư thế?
3. SuperPoint được huấn luyện bằng homography ngẫu nhiên. Điều đó khiến nó *không* học được ba hiện tượng nào, và hiện tượng nào trong ba cái đó là nỗi đau số một của bạn?
4. Nếu bạn chỉ được giữ một trong hai — ngữ cảnh toàn cục của SuperGlue hay ràng buộc một-một của Sinkhorn — thì giữ cái nào, và trên loại cảnh nào lựa chọn đó đảo ngược?
5. Tính lặp lại, MMA và tỉ lệ inlier cùng tăng mạnh mà APE không đổi: cơ chế nào trong năm cơ chế hấp thụ bạn kiểm chứng được rẻ nhất, và bằng phép đo gì?
6. Vì sao việc bộ phát hiện học được không trả về tăng kim tự tháp lại là một thay đổi ở *backend* chứ không chỉ ở *frontend*?
7. Nếu *oracle* ghép cặp cho thấy trần chỉ tốt hơn hiện tại vài phần trăm, bạn kết luận gì, và bạn ghi lại kết luận đó kèm những điều kiện nào để nó còn dùng được sau sáu tháng?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., “Image Matching Across Wide Baselines: From Paper to Practice,” *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., “RoMa: Robust Dense Feature Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., “Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., “HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, “Closed-form solution of absolute orientation using unit quaternions,” *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., “Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, “Rethinking Visual Geo-localization for Large-Scale Applications,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, “MixVPR: Feature Mixing for Visual Place Recognition,” in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., “EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, “Optimal Transport Aggregation for Visual Place Recognition,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., “AnyLoc: Towards Universal Visual Place Recognition,” *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., “SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., “DeepFactors: Real-Time Probabilistic Dense Monocular SLAM,” *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, “NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction,” in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., “CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations,” *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, “Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, “ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., “Point-SLAM: Dense Neural Point Cloud-based SLAM,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., “GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., “BARF: Bundle-Adjusting Neural Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, “NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields,” *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., “NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM,” in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., “vMAP: Vectorised Object Mapping for Neural Field SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, “RGBD GS-ICP SLAM,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., “Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Nhận dạng lại địa điểm và đóng vòng (Place Recognition and Loop Closure)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định thay hay không thay); F/28-dac-trung-va-ghep-cap (đặc trưng và ghép cặp học được — chương này dùng lại chúng ở tầng kiểm chứng); B/07-chia-du-lieu-va-danh-gia (giao thức đánh giá, recall và precision).

TÓM TẮT

Chương này mô phỏng vòng thành bốn tầng, rồi dạy cách đo xem tầng nào đang giết vòng. Đọc xong bạn đặt được biển đếm đúng chỗ, dựng được nhãn đúng-sai từ chân trị, và phân biệt được “thiếu mẫu” với “trần inlier”. Bạn cũng biết khi nào một bộ mô tả toàn ảnh học được thật sự hơn túi từ, và khi nào nó chỉ dời nút thắt sang chỗ khác. Nếu chỉ nhớ một điều: người ta hay đổ lỗi cho tầng lấy ứng viên, nhưng trên một hệ stereo-inertial thì tầng kiểm chứng hình học — chỗ RANSAC ước lượng Sim3 — thường là bộ lọc mạnh hơn nhiều, và chương này dạy bạn cách tự đo xem điều đó có đúng với hệ của mình không thay vì tin vào một con số của người khác.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đóng vòng (loop closure)** — việc nhận ra rằng robot đang ở lại một nơi đã đi qua, rồi thêm một ràng buộc nối hai thời điểm đó trong đồ thị tư thế. Đây là cơ chế duy nhất xoá được sai số tích lũy; không có nó thì mọi hệ đo bằng chuyển động tương đối đều trôi vô hạn.
- **ứng viên (candidate)** — một khung hình khoả cũ mà tầng truy hồi cho rằng có thể là cùng một nơi với khung hình hiện tại. Ứng viên chỉ là một đề xuất, chưa phải một vòng.
- **túi từ (bag of words)** — cách mô tả một ảnh bằng biểu đồ tần suất các “từ thị giác”, tức các cụm đặc trưng cục bộ đã được gom sẵn trong một từ điển. Hai ảnh giống nhau thì biểu đồ giống nhau, nên so ảnh trở thành so hai vector thưa.
- **mô tả toàn ảnh (global descriptor)** — một vector duy nhất, thường vài trăm tới vài nghìn chiều, tóm cả bức ảnh. Truy hồi địa điểm chính là tìm láng giềng gần nhất của vector này trong cơ sở dữ liệu.
- **Sim3** — phép biến đổi gồm xoay, tịnh tiến và một hệ số tỉ lệ chung, tức bảy bậc tự do. Bản đồ đơn ảnh có tỉ lệ trôi theo thời gian, nên hai đầu của một vòng thường lệch cả tỉ lệ chứ không chỉ lệch tư thế.
- **RANSAC và inlier** — RANSAC rút ngẫu nhiên một tập nhỏ nhất các khớp, dựng một giả thuyết từ đó, rồi đếm xem bao nhiêu khớp còn lại phù hợp với giả thuyết ấy. Các khớp phù hợp gọi là inlier; giả thuyết nhiều inlier nhất được giữ lại.
- **perceptual aliasing** — hiện tượng hai nơi khác nhau trông giống hệt nhau — hành lang lặp, dây cửa giống nhau, hàng cây đều. Đây là nguồn chính của vòng sai, và nó không giảm khi ta tăng độ phân giải ảnh.
- **recall và precision** — recall là tỉ lệ vòng có thật mà hệ phát hiện được; precision là tỉ lệ vòng được chấp nhận mà thật sự đúng. Hai số này đánh đổi nhau qua một ngưỡng, nên nói một số mà giấu số kia là vô nghĩa.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn duyệt hàng chục nghìn ứng viên mà gần như không đóng được vòng nào. Bạn đặt bao nhiêu biến đếm, ở đúng những chỗ nào, để biết tầng nào đang giết vòng — và vì sao đặt sai một dòng làm bạn đổ tội nhầm cho suốt nhiều tháng?
- Q2. Nếu tầng kiểm chứng loại 99% ứng viên, đó là bằng chứng nó hỏng hay bằng chứng nó đang làm đúng việc? Bạn phân biệt hai khả năng đó bằng phép đo nào?
- Q3. Lý thuyết RANSAC nói rằng giảm tập mẫu tối thiểu từ ba điểm xuống hai điểm sẽ nâng xác suất hội tụ lên rất nhiều. Vì sao lời hứa đó vẫn có thể không đề ra thêm một vòng nào, và bạn kiểm giả định nào của nó trước khi bỏ công cài đặt?
- Q4. NetVLAD hơn túi từ trên mọi bảng xếp hạng nhận dạng địa điểm. Vì sao thay DBoW2 bằng nó vẫn có thể không đóng thêm được một vòng nào trên hệ của bạn?
- Q5. Vì sao điểm vận hành đúng của một bộ phát hiện vòng lại nằm lệch hẳn về phía precision, tới mức bạn sẵn sàng bỏ lỡ chín vòng để tránh một vòng sai?

3.1 Đặt vấn đề

Một hệ stereo-inertial đã tinh chỉnh kỹ vẫn trôi. Trôi là hệ quả tất yếu của việc cộng dồn các phép đo tương đối. Đóng vòng là cơ chế duy nhất xoá được phần trôi đã tích lũy, vì nó là ràng buộc duy nhất nối hai thời điểm cách xa nhau. Vì vậy khi vòng không nổ, sai số cuối chuỗi không có gì kéo về.

Câu hỏi mà hầu hết mọi người đặt ở điểm này là câu hỏi sai. Họ hỏi: “bộ nhận dạng địa điểm của tôi yếu, có nên thay túi từ bằng một mạng học được không?”. Câu hỏi ấy giả định trước rằng nút thắt nằm ở tầng truy hồi. Giả định đó thường sai, và nó sai theo một cách rất tốn kém: bạn bỏ ba tháng thay một tầng vốn không phải thủ phạm.

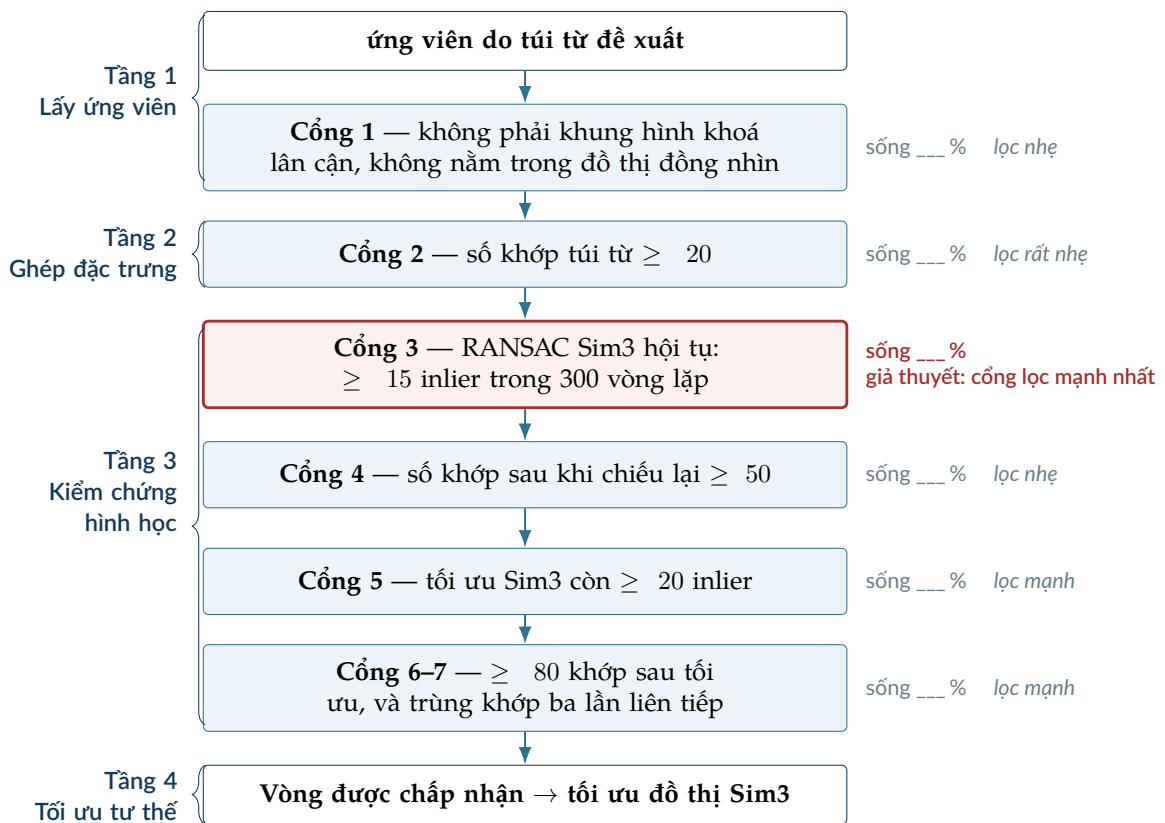
Đóng vòng không phải một khối. Nó là một dây chuyền bốn tầng, mỗi tầng là một bộ lọc riêng. Tầng lấy ứng viên đề xuất các khung hình khoá cũ có thể trùng nơi. Tầng ghép đặc trưng dựng các cặp điểm tương ứng giữa hai khung hình. Tầng kiểm chứng hình học hỏi xem tập cặp đó có giải thích được bằng một phép biến đổi cứng duy nhất không. Tầng tối ưu nhận ràng buộc đã được chấp nhận và phân bổ lại sai số cho cả bản đồ.

Mỗi tầng loại bỏ một phần ứng viên. Tổng của bốn tầng là một cái phễu rất hẹp. Nếu bạn chỉ nhìn đầu ra của phễu, bạn thấy đúng một con số: số vòng đóng được. Con số đó không cho biết tầng nào hẹp. Giá trị của chương này là biến câu “vòng không nổ” thành bốn con số riêng, mỗi con số cho một tầng, đo được trong một buổi chiều.

Sau khi có bốn con số đó, chương mới trả lời câu hỏi ban đầu. Bộ mô tả toàn ảnh học được giải quyết việc gì mà túi từ không giải quyết được, và giải quyết ở tầng nào. Định vị lại bằng mạng nơ-ron đứng ở đâu trong bốn tầng ấy. Ghép cặp học được đưa vào tầng kiểm chứng thì đổi gì. Và cuối cùng, làm sao đo cho đúng, khi mà một vòng sai đắt hơn nhiều lần một vòng bị bỏ lỡ.

Phần F bám bốn nỗi đau của một hệ đã chạy thật: chuyển động nhanh, ảnh mờ do chuyển động, đóng vòng, và đặt trọng số cho IMU. Chương này chạm thẳng vào nỗi đau thứ ba, và chỉ chạm gián tiếp vào hai nỗi đau đầu. Chuyển động nhanh và ảnh mờ xuất hiện ở đây dưới dạng hệ quả: chúng làm chất lượng cặp tương ứng tụt, và chính chất lượng cặp là thứ quyết định tầng ba. Trọng số IMU không thuộc chương này, trừ một chỗ: phương trọng lực từ IMU được dùng để rút bậc tự do trong bài toán kiểm chứng.

Hình 3.1 là bản đồ của cả chương. Nó vẽ bốn tầng cùng chuỗi công cụ thể của một hệ kiểu ORB-SLAM3, và chỗ để bạn tự điền tỉ lệ sống sót đo được trên hệ của mình. Đọc nó trước, rồi mỗi mục sau chỉ là một trạm trên đúng cái phễu đó.



Hình 3.1. Bốn tầng của đóng vòng và chuỗi cổng cụ thể thực hiện chúng; các ngưỡng ghi trong ô là hằng số trong mã nguồn của một hệ kiểu ORB-SLAM3, không phải kết quả đo. Các ô “sống ___ %” là chỗ trống bạn điền sau khi chạy phép đo ở mục C: mỗi tỉ lệ là *có điều kiện*, tức phần trăm ứng viên tới được cổng đó mà vượt qua nó. Nhân định tính bên phải là giả thuyết làm việc, không phải số đo: cổng kiểm chứng hình học thường là chỗ lọc mạnh nhất trong bốn tầng, và trên nhiều hệ thống nó cũng là nơi phần lớn ứng viên đúng bị loại. Chương này không nói tỉ lệ ấy bằng bao nhiêu trên hệ của bạn — nó dạy bạn cách tự lấy con số đó, và mục D dạy tiếp cách biết tỉ lệ ấy là loại đúng hay loại sai.

TRỰC GIÁC

Hình dung một quy trình tuyển dụng bốn vòng. Cuối quý bạn thấy phần lớn ứng viên bị loại ở vòng phỏng vấn kỹ thuật. Phản xạ đầu tiên là sửa vòng phỏng vấn: nói thang điểm, đổi người hỏi. Nhưng nếu bạn mở lại hồ sơ những người bị loại, gần như tất cả họ thật sự không đạt. Vòng phỏng vấn đang làm đúng việc của nó. Vài người giỏi bị loại oan thì bị loại vì hồ sơ gửi lên quá sơ sài, nên người phỏng vấn không có gì để hỏi. Chỗ phải sửa là khâu chuẩn bị hồ sơ, ở phía trước. Tầng kiểm chứng hình học trong đóng vòng đúng đúng vị trí vòng phỏng vấn ấy.

3.2 Bốn tầng và cách đo tầng nào đang giết vòng (Anatomy and Instrumentation)

A. Mỗi tầng làm gì, và giả định ngầm của nó

Tầng một, lấy ứng viên, so một mô tả toàn ảnh với cơ sở dữ liệu và trả về danh sách ngắn. Giả định ngầm là hai ảnh cùng một nơi thì gần nhau trong không gian mô tả. Giả định này gây khi ánh sáng đổi, khi góc nhìn lệch lớn, và khi cảnh có cấu trúc lặp.

Tầng hai, ghép đặc trưng, dựng các cặp điểm tương ứng giữa khung hình hiện tại và ứng viên. Trong ORB-SLAM3, việc này được dẫn hướng bằng chính cây từ điển: chỉ so các đặc trưng rơi

vào cùng một nút từ vựng [1]. Giả định ngầm là hai đặc trưng của cùng một điểm ba chiều sẽ rơi vào cùng một nút. Đây là một giả định mạnh, và nó là chỗ số lượng cặp bị chặn trên.

Tầng ba, kiểm chứng hình học, hỏi một câu duy nhất: có tồn tại một phép Sim3 giải thích được đủ nhiều cặp không. Đây là câu hỏi có đáp án đúng hoặc sai, nên nó là tầng duy nhất trong bốn tầng thật sự *chứng minh* được điều gì. Giả định ngầm là tỉ lệ inlier đủ cao để RANSAC bắt trúng một mẫu sạch.

Tầng bốn, tối ưu, đưa ràng buộc mới vào đồ thị và phân bổ lại sai số. Giả định ngầm ở đây là ràng buộc mới đúng. Nếu nó sai, tầng bốn không phát hiện được; nó chỉ trung thành lan cái sai ra khắp bản đồ.

B. Hai cổng rẽ nhất: ràng buộc thời gian và đồng nhìn

Trước khi bàn tới bất cứ mô-đun học được nào, cần thấy hai cổng rẽ nhất trong cả chuỗi. Cả hai không tồn một phép nhân nào, và cả hai mua precision bằng cách trả recall.

Cổng thứ nhất loại các khung hình khoá đang nằm trong vùng đồng nhìn hiện tại. Không có nó, hệ sẽ “đóng vòng” với chính chỗ nó vừa đi qua ba giây trước. Ràng buộc kiểu đó đúng nhưng vô ích, vì nó không mang thông tin mới cho đồ thị. Trong bảng sống sót, đây chính là cổng một.

Cổng thứ hai là ràng buộc trùng khớp nhiều lần liên tiếp. ORB-SLAM3 chỉ chấp nhận một vòng khi ba khung hình khoá liên tiếp cùng chỉ về một nhóm đồng nhìn. Cơ chế đằng sau rất gọn: perceptual aliasing thường không bền theo thời gian. Hai hành lang giống nhau cho điểm cao ở một khung hình, nhưng hiếm khi giữ được điểm cao qua ba khung hình liên tiếp với hình học nhất quán.

Giá phải trả thì cụ thể và đo được. Bạn mất ba khung hình khoá đầu của mỗi lần đi lại. Với một lần đi lại rất ngắn, chẳng hạn robot chỉ lướt qua chỗ cũ trong hai giây, bạn mất luôn cả vòng. Đây là ví dụ sạch nhất trong toàn chương về việc đổi recall lấy precision, và nó xảy ra trước khi bất kỳ mạng nào được gọi.

Có một biến thể đáng biết. Thay vì chấm điểm từng khung hình khoá riêng lẻ, ta gom các khung hình khoá cùng nhóm đồng nhìn lại rồi cộng điểm của cả nhóm [43]. Cách này giảm nhiễu của điểm số truy hồi, vì một nơi đúng thường làm nhiều khung hình khoá lân cận cùng ăn điểm. Đó là một dạng kiểm chứng không gian rất rẻ, làm ngay tại tầng một.

C. Đặt biến đếm ở đâu, và cái bẫy dịch nhãn

Cách đo rất rẻ. Với mỗi ứng viên, tăng một biến đếm ngay trước mỗi cổng, rồi ghi ra một dòng nhật ký cho mỗi khung hình khoá. Sau vài chục lần chạy bạn có bảng sống sót từng tầng. Toàn bộ công việc là vài chục dòng mã và không ảnh hưởng thời gian thực.

Nhưng có một cái bẫy, và nó đã làm hỏng nhiều tuần phân tích trong chính dự án này.

CẠM BÃY

Nếu bạn đặt lệnh tăng biến đếm *ngay trước* câu lệnh `if` của cổng, thì cột mang tên cổng đó thật ra đang đếm “số ca đã vượt cổng *trước* nó”. Mọi nhãn cột bị dịch đi một bậc. Hậu quả rất cụ thể và rất khó thấy: cái cổng bị đổ tội luôn là cổng đứng *sau* thủ phạm thật, và vì bảng vẫn nhất quán với chính nó nên không có triệu chứng nào để bạn nghi ngờ. Trong chính dự án này, một bảng đặt sai kiểu đó đã dẫn phân tích đi lạc suốt nhiều tuần trước khi được đếm lại từ nhật ký thô. Cách phòng: đặt lệnh tăng biến đếm *bên trong* khối `if`, hoặc viết ngay trong tên biến rằng nó đếm “đã tới cổng nào” chứ không phải “đã qua cổng nào”. Bài học tổng quát: một cột nhật ký không tự nói ra ngữ nghĩa của nó, nên ngữ nghĩa phải nằm trong tên.

D. Nhãn đúng-sai từ chân trị: phép đo quan trọng thứ hai

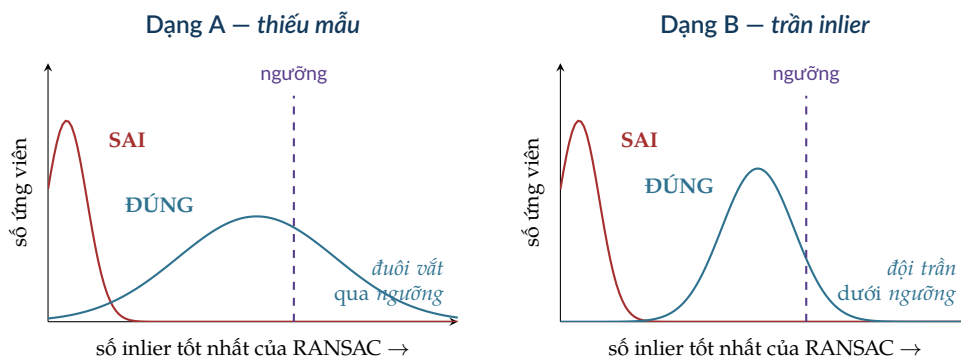
Bảng sống sót nói tăng nào loại nhiều nhất. Nó không nói tăng đó loại *đúng* hay *sai*. Đây là phân biệt quan trọng nhất của cả chương, và nó cần một phép đo thứ hai.

Phép đo ấy là gắn nhãn cho từng ứng viên bằng chân trị. Với mỗi cặp khung hình khoá được đề xuất, tra tư thế thật của cả hai, tính khoảng cách giữa chúng, rồi gọi ứng viên là “đúng” nếu khoảng cách dưới một ngưỡng và “sai” nếu vượt xa ngưỡng đó. Ngưỡng ấy là một lựa chọn thiết kế chứ không phải sự thật: trong nhà, vài mét là mức hợp lý; ngoài trời thì phải rộng hơn nhiều. Khi đã có nhãn, bảng sống sót tách làm hai bảng, và hai bảng đó kể hai câu chuyện khác nhau.

Việc phải làm sau đó là vẽ, cho riêng từng nhóm nhãn, phân bố của số *inlier* tốt nhất mà RANSAC tìm được. Đây là biểu đồ quan trọng nhất của cả chương, vì hình dạng của nó trả lời thẳng câu hỏi “cổng này loại đúng hay loại sai”. Có hai câu hỏi đọc được từ nó, và cả hai đều đọc bằng mắt.

1. **Các ứng viên sai có bao giờ tới gần ngưỡng không?** Nếu phân bố của chúng nằm sát số không và tắt hẳn từ lâu trước ngưỡng, thì mọi ca chết trong nhóm đó là *loại đúng*, và bạn không có lý do gì để dưng vào cổng. Một tỉ lệ sống sót rất thấp khi ấy là bằng chứng cổng đang làm đúng việc, không phải bằng chứng nó hỏng.
2. **Giữa hai phân bố có một khoảng trống không, và ngưỡng nằm ở đâu so với khoảng trống ấy?** Nếu có một khoảng mà chỉ ứng viên *đúng* chết trong đó và không ứng viên sai nào chạm tới, thì đúng chỗ đó — và chỉ chỗ đó — là dư địa mà việc nói ngưỡng lấy lại được. Đếm số ứng viên nằm trong khoảng trống ấy là bạn có ngay trần lợi ích của mọi ý tưởng “nói ngưỡng”, trước khi cài đặt gì.

Hình 3.2 vẽ hai hình dạng mà biểu đồ ấy có thể mang, và chúng dẫn tới hai kết luận trái ngược nhau. Điểm cốt lõi: một tỉ lệ sống sót thấp ở cổng ba *không* tự nó chứng minh điều gì. Cùng một tỉ lệ ấy có thể là dấu hiệu của một bộ kiểm chứng khoẻ mạnh hoặc của một tăng ghép cặp đôi cặp đúng, và chỉ hình dạng phân bố mới phân biệt được.



Hình 3.2. Hai hình dạng mà biểu đồ số inlier tốt nhất có thể mang, vẽ theo *cơ chế* chứ không theo số đo — các trục cố ý không có vạch số, vì con số là thứ bạn phải tự lấy trên hệ của mình. Ở cả hai hình, nhóm ứng viên SAI nằm sát số không và tắt trước ngưỡng, nghĩa là mọi ca chết trong nhóm ấy là loại đúng và cổng đang làm việc của nó. Khác biệt nằm ở nhóm ĐÚNG. Dạng A: phân bố trải rộng và vượt qua ngưỡng, tức inlier vốn có đủ mà chỉ thiếu lượt rút trúng — đây là *thiếu mẫu*, và tăng số vòng lặp hay rút mẫu có ưu tiên sẽ có lãi. Dạng B: phân bố đội trần ngay dưới ngưỡng và không có đuôi, tức số cặp đúng thật sự tồn tại đã ít hơn ngưỡng — đây là *trần inlier*, và không cách rút mẫu nào chạm tới được. Nhìn nhầm dạng B thành dạng A là cách tốn nhiều tháng nhất để không sửa được gì.

E. Phân biệt “thiếu mẫu” với “trần inlier”

Khi RANSAC không hội tụ, có đúng hai nguyên nhân, và chúng đòi hai cách chữa trái ngược nhau.

- **Thiếu mẫu (sampling failure)** — inlier có đủ, nhưng số vòng lặp không đủ để rút trúng một tập mẫu tối thiểu toàn inlier. Cách chữa: tăng số vòng lặp, giảm cỡ tập mẫu tối thiểu, hoặc rút mẫu có ưu tiên.
- **Trần inlier (inlier ceiling)** — số cặp đúng thật sự tồn tại đã ít hơn ngưỡng. Không cách rút mẫu nào cứu được. Cách chữa duy nhất là tạo ra nhiều cặp đúng hơn, tức sửa tăng hai.

Hai nguyên nhân này phân biệt được bằng một biểu đồ duy nhất, chính là Hình 3.2. Nếu số inlier tốt nhất của các ứng viên đúng phân bố cao nhưng thưa thớt qua ngưỡng, đó là thiếu mẫu. Nếu nó đội trần ngay dưới ngưỡng, đó là trần inlier.

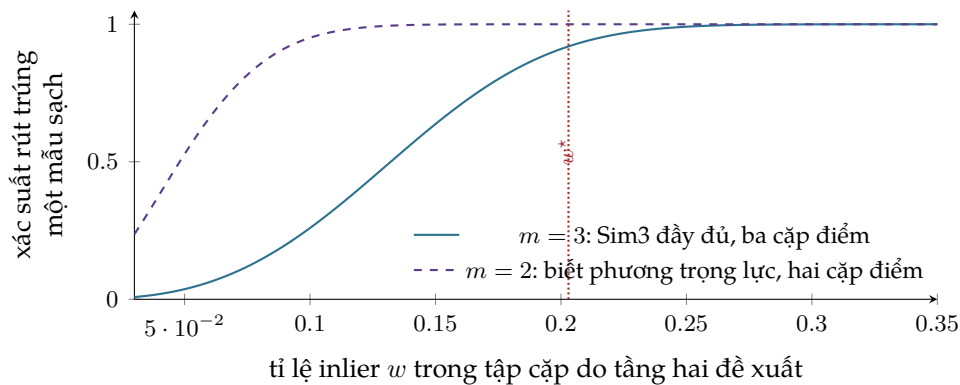
Có một con số thứ hai phải ghi kèm để đọc biểu đồ ấy cho đúng, và nó là mẫu số: *số cặp mà tăng hai đề xuất*. Số inlier tốt nhất chỉ có nghĩa khi so với mẫu số đó, vì tỉ lệ inlier w mới là đại lượng đi vào công thức bên dưới. Mười lăm inlier trên hai trăm cặp và mười lăm inlier trên bốn mươi cặp là hai tình huống hoàn toàn khác nhau. Ghi cả tử lẫn mẫu vào cùng một dòng nhật ký là việc rẻ nhất bạn làm được, và không ghi nó là lý do phổ biến nhất khiến bảng của bạn không trả lời được câu hỏi nào.

Lý thuyết RANSAC làm rõ tại sao phân biệt này quan trọng. Xác suất rút được ít nhất một tập mẫu sạch trong N vòng lặp là

$$p = 1 - (1 - w^m)^N,$$

với w là tỉ lệ inlier và m là cỡ tập mẫu tối thiểu [44]. Với Sim3 tổng quát thì $m = 3$, vì ba cặp điểm ba chiều đủ để giải bài toán định hướng tuyệt đối có tỉ lệ [45], [46]. Đây là một đẳng thức, nên thay số vào là ra ngay, và việc thay số cho thấy quan hệ dốc tới mức nào. Giả sử $N = 300$ vòng lặp: ở $w = 0,20$ ta được $p \approx 0,91$, còn ở $w = 0,10$ chỉ còn $p \approx 0,26$. Hai giá trị w này là giả định để thấy hình dạng quan hệ, không phải số đo của hệ nào; điều đáng nhớ là chia đôi tỉ lệ inlier đủ để biến một cổng gần như luôn mở thành một cổng gần như luôn đóng.

Nhưng nó chỉ là nút thắt nếu nguyên nhân là thiếu mẫu. Hình 3.3 vẽ hai đường cong cho $m = 3$ và $m = 2$. Khoảng cách giữa chúng ở vùng w thấp là rất lớn, nên lý thuyết hứa hẹn một cải thiện mạnh nếu ta giảm được cỡ tập mẫu. Mục 3.5 mổ ba giả định ẩn của lời hứa đó, và nói cách kiểm từng giả định trước khi bỏ công cài đặt.



Hình 3.3. Đồ thị của đúng công thức $1 - (1 - w^m)^N$ với $N = 300$; đây là toán, không phải số đo. Giảm cỡ tập mẫu tối thiểu từ ba xuống hai đẩy cả đường cong sang trái rất mạnh ở vùng tỉ lệ inlier thấp — đó là toàn bộ sức hấp dẫn của ý tưởng dùng trọng lực để rút bậc tự do. Đường chấm đứng là w^* , tỉ lệ inlier tối thiểu để chạm ngưỡng, tính bằng ngưỡng inlier chia cho số cặp mà tăng hai đề xuất; bạn tự tính nó cho hệ mình từ hai cột nhật ký ở Bảng 3.4. Bên trái w^* thì hai đường cong này vô nghĩa, vì ở đó số inlier *thật* đã dưới ngưỡng và không cách rút mẫu nào chạm tới được.

F. Ba con số nên có trước khi đọc bất cứ paper nào

Trước khi cân nhắc thay một tăng, hãy có ba con số cho hệ của chính bạn.

1. **Bảng sống sót từng cổng**, với biến đếm đặt đúng chỗ. Nó cho biết tầng nào hẹp.
2. **Bảng sống sót tách theo nhãn chân trị**. Nó cho biết tầng hẹp đó loại đúng hay loại sai.
3. **Số ứng viên đúng tồn tại**. Nếu một chuỗi không có lần đi lại nào thì việc nó không đóng vòng là kết quả đúng, không phải lỗi.

Con số thứ ba hay bị bỏ sót nhất, và nó rẻ đến mức không có lý do gì để bỏ. Với mỗi chuỗi, quét mọi cặp khung hình khoá bằng *chân trị* rồi đếm xem có bao nhiêu cặp thoả điều kiện “cùng nơi” mà không phải khung hình khoá lân cận về thời gian. Nếu con số đó bằng không thì chuỗi ấy đơn giản là không có lần đi lại nào, và việc hệ không đóng vòng trên nó là kết quả *đúng*, không phải lỗi. Nhiều chuỗi phổ biến trong nhà là đường đi một chiều không quay lại, nên khả năng bạn đang chữa một hệ vốn đang đúng cao hơn nhiều so với cảm giác. Không đo con số này trước là cách chắc chắn nhất để mất nhiều tuần.

↔ **cùng nguyên lý với E/24** — đây chính là thí nghiệm oracle: thay tầng lấy ứng viên bằng chân trị để đo trần đóng góp của nó, trước khi tối ưu nó

3.3 Toàn ảnh học được: từ túi từ tới SALAD và AnyLoc (Learned Global Descriptors)

A. DBoW2 làm được gì, và nó gãy ở đâu

Túi từ nhị phân của DBoW2 là một thiết kế rất kinh tế [43]. Nó dùng lại đúng các mô tả ORB đã được trích cho việc bám vết, nên chi phí trích đặc trưng bằng không. Từ điển được huấn luyện ngoại tuyến một lần, dạng cây phân cấp với khoảng một triệu từ. Truy vấn là tra chỉ mục ngược trên vectơ thừa, nên rẻ.

Về chi phí, điều đáng nói không phải một con số mà là một tính chất: chi phí biên của tăng một trong một hệ kiểu ORB-SLAM3 gần như bằng không, vì mọi thứ nó cần — các mô tả ORB — đã được trích sẵn cho việc bám vết. Trước khi tin điều đó trên hệ của mình, hãy đo hai thứ: độ trễ của tầng phát hiện ở *phân vị* 99, không phải trung vị, và độ dài hàng đợi khung hình khoá. Nếu hàng đợi luôn rỗng thì tầng một chưa phải chỗ đáng động tới, và mọi lập luận về việc thay nó phải dựa trên chất lượng chứ không dựa trên tốc độ.

DBoW2 gãy ở bốn chỗ, và cả bốn đều bắt nguồn từ cùng một điểm: nó thừa hưởng giới hạn của chính mô tả ORB.

- **Đổi ánh sáng mạnh** — ngày với đêm, trong nhà bật tắt đèn. Mô tả nhị phân dựa trên so sánh cường độ điểm ảnh, nên đảo tương phản làm chúng đổi hẳn.
- **Lệch góc nhìn lớn** — đi lại cùng hành lang theo chiều ngược, hoặc lệch ngang vài mét. Đây là chế độ hỏng thường gặp nhất trong nhà.
- **Cấu trúc lặp** — perceptual aliasing. Túi từ bỏ hoàn toàn thông tin bố cục không gian, nên hai hành lang giống nhau cho cùng biểu đồ tần suất.
- **Nền ít vân** — tường trắng, sàn phẳng. Không có đặc trưng thì không có từ.

Hai chế độ đầu là chỗ mà mô tả học được thật sự thắng. Chế độ thứ ba thì không: một mô tả toàn ảnh tốt hơn vẫn không phân biệt được hai nơi thật sự giống nhau. Đó là việc của tầng ba và của ràng buộc thời gian.

↔ **tiền đề F/28** — túi từ kế thừa mọi điểm gãy của mô tả cục bộ mà nó gom lại, nên chương 28 giải thích trước nguyên nhân gốc của ba trong bốn chế độ hỏng ở đây

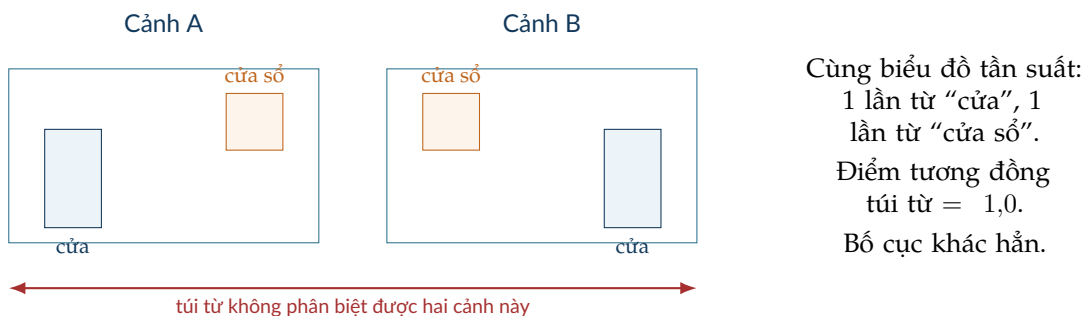
B. Vì sao túi từ mất bố cục, và điều đó có nghĩa gì

Trong bốn chế độ hỏng ở trên, chế độ cấu trúc lặp đáng nhìn kỹ, vì nó không phải lỗi của mô tả ORB mà là lỗi của phép gộp. Túi từ đếm xem mỗi từ thị giác xuất hiện bao nhiêu lần, rồi vứt bỏ hoàn toàn thông tin về vị trí của chúng. Hai bức ảnh chứa cùng các thành phần nhưng sắp xếp khác nhau cho cùng một biểu đồ tần suất.

Hình 3.4 vẽ trường hợp nhỏ nhất. Hai cảnh khác nhau, cùng số cửa và cùng số cửa sổ, chỉ đổi chỗ trái phải. Điểm tương đồng của túi từ giữa chúng bằng đúng một, tức là chúng không phân biệt được.

Điều này giải thích một quan sát thực tế hay gặp: mọi cải tiến ở tầng mô tả cục bộ đều không chứa được perceptual aliasing. Muốn chứa thì phải đưa lại thông tin không gian vào, và có ba cách. Cách rẻ nhất là ràng buộc thời gian ở mục trước. Cách thứ hai là kiểm chứng hình học ở tầng ba, vốn chính là kiểm tra bố cục một cách nghiêm ngặt. Cách thứ ba là xếp hạng lại bằng mảnh đặc trưng có vị trí, tức hướng của Patch-NetVLAD.

Đây cũng là lý do một mô tả toàn ảnh học được không tự nó xoá được perceptual aliasing. Nó gộp tốt hơn, nó bất biến với ánh sáng tốt hơn, nhưng nó vẫn là một vectơ duy nhất cho cả ảnh. Hai nơi thật sự giống nhau vẫn cho hai vectơ gần nhau, và đúng như vậy mới là hành vi đúng của một bộ mô tả.



Hình 3.4. Trường hợp nhỏ nhất cho thấy túi từ mất bố cục không gian. Hai cảnh chứa đúng cùng các thành phần, chỉ đổi vị trí trái phải, nên biểu đồ tần suất từ thị giác của chúng trùng nhau hoàn toàn. Đây là lý do mọi cải tiến ở tầng mô tả cục bộ đều không chứa được perceptual aliasing, và cũng là lý do một mô tả toàn ảnh học được cũng không chứa được.

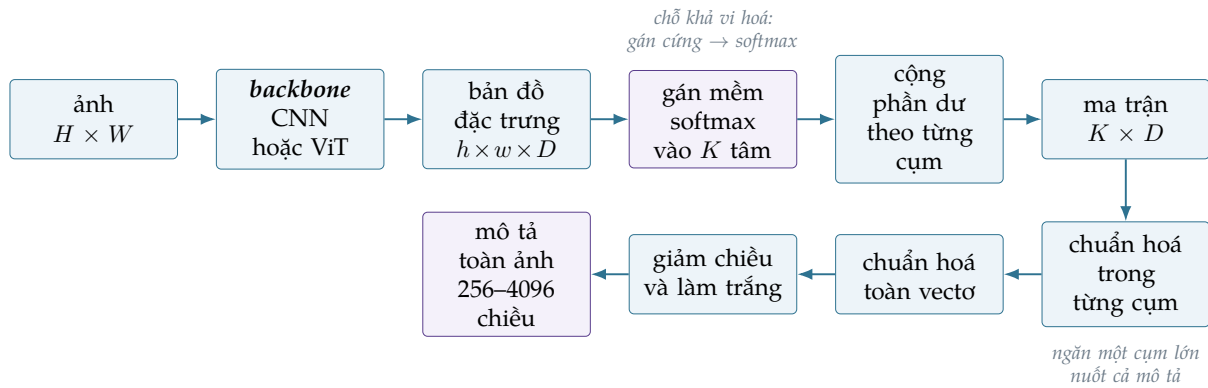
C. NetVLAD: biến phép gộp thành thứ học được

Ý tưởng của VLAD có trước học sâu. Thay vì đếm số đặc trưng rơi vào mỗi cụm, VLAD cộng *phần dư* giữa đặc trưng và tâm cụm. Kết quả là một ma trận $K \times D$, giữ được nhiều thông tin hơn hẳn một biểu đồ tần suất.

Vấn đề là phép gán đặc trưng vào cụm là gán cứng, nên không khả vi, nên không huấn luyện được đầu-cuối. NetVLAD thay gán cứng bằng gán mềm dạng softmax [47]. Chỉ một thay đổi đó làm cả khối gộp trở nên khả vi, và từ đó cả mạng trích đặc trưng lẫn các tâm cụm đều học được cùng lúc.

Điểm thứ hai quan trọng không kém là tín hiệu huấn luyện. NetVLAD học từ ảnh phổ chụp cùng một chỗ ở các thời điểm và góc nhìn khác nhau, với hàm mất mát dạng xếp hạng bộ ba. Nói cách khác, mạng được dạy trực tiếp cái bất biến mà ta cần: cùng nơi thì gần nhau, dù ánh sáng và góc nhìn đổi. Túi từ không có cách nào học điều đó, vì nó không có tham số nào để học.

Hình 3.5 vẽ đường đi của dữ liệu. Chỗ đáng chú ý là hai bước chuẩn hoá cuối: chuẩn hoá trong từng cụm trước, rồi chuẩn hoá toàn vectơ. Bước đầu ngăn một cụm chiếm ưu thế chỉ vì nó có nhiều đặc trưng, tức là ngăn một mảng tường lớn nuốt cả mô tả.



Hình 3.5. Đường đi của dữ liệu trong NetVLAD. Toàn bộ khác biệt so với VLAD cổ điển nằm ở một khối: phép gán đặc trưng vào cụm được làm mềm bằng softmax, nên khả vi, nên cả tâm cụm lẫn mạng trích đặc trưng đều học được. Hai bước chuẩn hoá cuối không phải trang trí; bỏ bước chuẩn hoá trong cụm thì một vùng ảnh đồng nhất lớn sẽ chi phối mô tả.

D. Dòng thời gian: mỗi bước gỡ nút thắt nào

Sau NetVLAD, dòng nghiên cứu này đi theo một mạch rất rõ. Điều đáng chú ý là nút thắt dịch chuyển liên tục, và không phải lúc nào nó cũng nằm ở kiến trúc.

I Mạch 1 – từ túi từ tới đặc trưng nền

- 1 Vấn đề** Túi từ mất hoàn toàn tính bất biến với ánh sáng và góc nhìn lớn, vì nó thừa hưởng mô tả thủ công.

Giải pháp Học phép gộp bằng cách làm nó khả vi, rồi huấn luyện trên ảnh cùng nơi khác thời điểm. Đó là NetVLAD.

Giá phải trả Một lượt xuôi qua mạng cho mỗi khung hình khoá, và một mô tả vài nghìn chiều thay cho vectơ thưa.

Vấn đề mới Mô tả toàn ảnh vẫn không có thông tin bố cục, nên vẫn nhảm ở cấu trúc lặp.
- 2 Vấn đề** Truy hồi toàn ảnh xếp hạng sai khi hai nơi khác nhau có cùng thống kê đặc trưng.

Giải pháp Xếp hạng lại danh sách ngắn bằng chính các mảnh đặc trưng trích từ NetVLAD, có tính tới vị trí không gian. Đó là Patch-NetVLAD [48].

Hệ quả Ranh giới giữa tầng một và tầng ba bắt đầu mờ đi: một phần kiểm chứng bị kéo lên tầng truy hồi.

Giá phải trả Chi phí xếp hạng lại tỉ lệ với số ứng viên, nên ngân sách thời gian thực bị ràng buộc lại.
- 3 Vấn đề** Chạy một mạng cho truy hồi và một mạng khác cho đặc trưng cục bộ là trả hai lần cho cùng một bức ảnh.

Giải pháp Một mạng, một *backbone* chung, ba đầu ra: mô tả toàn ảnh, điểm đặc trưng, mô tả cục bộ. Đó là HF-Net [49].

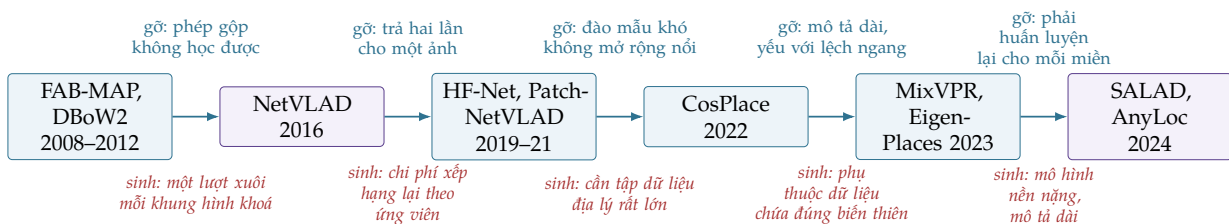
Giả định Ba nhiệm vụ đó dùng chung được biểu diễn ở tầng giữa.

Hệ quả Đây là bài học thiết kế đáng giá nhất của cả mục này với một hệ có ngân sách độ trễ chặt.
- 4 Vấn đề** Huấn luyện bằng xếp hạng bộ ba đòi đào mẫu khó, rất tốn và rất khó ổn định ở quy mô lớn.

Giải pháp Đổi bài toán thành phân lớp: chia thế giới thành các ô theo vị trí và hướng, mỗi ô là một lớp. Đó là CosPlace [50].

Hệ quả	Nút thắt hoá ra nằm ở quy trình dữ liệu và hàm mục tiêu, không nằm ở kiến trúc.
5 Vấn đề	Khối gộp NetVLAD nặng và mô tả đầu ra rất dài.
Giải pháp	Thay gộp theo cụm bằng một chồng khối trộn đặc trưng đơn giản, cho mô tả gọn hơn. Đó là MixVPR [51].
Vấn đề mới	Mô tả gọn hơn nhưng độ bền với lệch góc nhìn ngang chưa được giải quyết riêng.
6 Vấn đề	Lệch góc nhìn ngang, kiểu đi lại cùng con đường ở làn khác, vẫn phá truy hồi.
Giải pháp	Dựng các lớp huấn luyện sao cho mỗi lớp gồm nhiều hướng nhìn khác nhau của cùng một nơi. Đó là EigenPlaces [52].
Giả định	Bất biến góc nhìn học được nếu và chỉ nếu dữ liệu huấn luyện chứa đúng loại biến thiên đó.
7 Vấn đề	Gán mềm của NetVLAD vẫn ép mọi đặc trưng phải thuộc về một cụm nào đó, kể cả đặc trưng vô nghĩa.
Giải pháp	Xem việc gán đặc trưng vào cụm như một bài toán vận tải tối ưu, có thêm một cụm “thùng rác” cho đặc trưng vô ích. Đó là SALAD [53].
Giả định	Đặc trưng đầu vào đã đủ tốt, vì SALAD đặt trên nền DINOv2 [54].
8 Vấn đề	Mọi phương pháp trên đều cần huấn luyện lại khi ra khỏi phân phối: dưới nước, trên không, trong nhà ban đêm.
Giải pháp	Bỏ hẳn việc huấn luyện chuyên biệt. Lấy đặc trưng trung gian của một mô hình nền rồi gộp bằng VLAD cổ điển với từ điển dựng theo miền. Đó là AnyLoc [55].
Giá phải trả	Mô tả dài hơn và một lượt xuôi qua mô hình nền, tức đắt hơn về tính toán.
Hệ quả	Với một robot phải chạy ở miền chưa từng thấy, đây thường là lựa chọn an toàn hơn một mô hình đã tinh chỉnh chặt.

Hình 3.6 tóm mạch trên thành một trục. Điều đáng rút ra không phải thứ tự các tên, mà là việc nút thắt nhảy qua lại giữa ba chỗ: phép gộp, dữ liệu huấn luyện, và đặc trưng đầu vào.



Hình 3.6. Dòng thời gian của mô tả toàn ảnh. Hàng trên ghi nút thắt mà mỗi bước gỡ được; hàng dưới ghi vấn đề mới mà nó tạo ra. Nút thắt nhảy giữa ba chỗ: phép gộp, dữ liệu huấn luyện, và chất lượng đặc trưng đầu vào. Hai ô tô đậm là hai bước đổi hẳn cách nghĩ, không chỉ đổi con số.

E. So sánh có cột “hông khi nào”

Bảng 3.1 đặt các họ cạnh nhau. Cột cuối là cột đáng đọc nhất, vì nó cho biết bạn sẽ mất gì khi đổi.

Bảng 3.1. Các họ mô tả toàn ảnh, so theo cơ chế, giả định và chế độ hỏng. Cột chi phí cố ý ghi định tính: nó nói *loại* chi phí bạn phải trả cho mỗi khung hình khoá, không nói bao nhiêu mili-giây. Con số mili-giây chỉ có nghĩa khi đo trên chính phần cứng đích với hệ đang chạy đủ tải, và đó là việc của bạn chứ không phải của một bảng.

Họ	Cơ chế	Mạnh khi nào	Hỏng khi nào	Chi phí
DBoW2	Biểu đồ tần suất từ thị giác nhị phân, chỉ mục ngược, tf-idf	Cùng điều kiện chụp, cùng hướng đi, bản đồ vừa phải	Đổi ánh sáng mạnh, lệch góc nhìn lớn, cấu trúc lặp, nền ít vân	chi phí biên gần như bằng 0: dùng lại mô tả ORB
NetVLAD	Gộp phần dư quanh K tâm, gán mềm khả vi	Đổi thời điểm và ánh sáng, ngoài trời đô thị	Miền khác hẳn dữ liệu huấn luyện; vẫn nhằm ở cấu trúc lặp	một lượt xuôi mạng cỡ vừa mỗi khung hình khoá
Patch-NetVLAD	Xếp hạng lại danh sách ngắn bằng mảnh đặc trưng có vị trí	Cần precision cao và danh sách ngắn nhỏ	Chi phí tăng tuyến tính theo số ứng viên; không hợp ngân sách chặt	cao, theo ứng viên
HF-Net	Một <i>backbone</i> , ba đầu: toàn ảnh, điểm, mô tả cục bộ	Khi bạn cũng cần đặc trưng cục bộ học được	Ba nhiệm vụ tranh nhau sức chứa nếu <i>backbone</i> quá nhỏ	một lượt chung
CosPlace, MixVPR, EigenPlaces	Đổi hàm mục tiêu và cách dựng lớp huấn luyện	Bản đồ rất lớn; lệch góc nhìn ngang	Vẫn là mô hình huấn luyện theo miền; ra ngoài phân phối thì tụt	cùng bậc với NetVLAD
SALAD	Gán đặc trưng vào cụm bằng vận tải tối ưu, có thùng rác	Nền DINOv2 sẵn; cần mô tả gọn và mạnh	Phụ thuộc chất lượng mô hình nền bên dưới	một lượt xuôi mô hình nền: nặng hơn hẳn
AnyLoc	Đặc trưng mô hình nền, không huấn luyện lại, gộp VLAD	Miền lạ: dưới nước, trên không, trong nhà ban đêm	Khi có nhiều dữ liệu đúng miền, mô hình tinh chỉnh vẫn hơn	như SALAD, cộng mô tả dài hơn

F. Tính tay ba khoản chi phí, và khoản duy nhất đáng kể

Lập luận “túi từ nhẹ hơn” được nhắc rất nhiều nhưng hiếm khi được tính ra số. Có ba khoản chi phí, và chỉ một khoản thật sự quan trọng.

Bộ nhớ mô tả. Đây là một phép nhân chia thuận tuý, và nó cho một kết luận đi ngược trực giác. Giả sử mỗi khung hình khoá giữ khoảng một nghìn đặc trưng — cấu hình mặc định quen thuộc. Vectơ túi từ thừa của nó có tới đa một nghìn cặp gồm mã từ và trọng số; ở tám byte một cặp, đó là cỡ tám kilobyte. Một mô tả toàn ảnh 4096 chiều dạng số thực 32 bit là mười sáu kilobyte, tức lớn hơn. Nhưng nó là vectơ *dày* có độ dài cố định, nên giảm chiều xuống 256 rồi lượng tử hoá về 8 bit là hợp lệ, và khi đó nó chỉ còn vài trăm byte. Vectơ thừa của túi từ không chịu được phép nén tương tự, vì mỗi mục của nó là một cặp chỉ số–trọng số cần thiết cho việc tra chỉ mục ngược. Kết luận về *hình dạng* quan hệ: khi bản đồ lớn dần, mô tả học được là thứ nén được, còn túi từ thì không — và tỉ số nghiêng về phía học được, chứ không phải ngược lại như người ta hay nghĩ.

Chi phí truy vấn. Chỉ mục ngược chỉ duyệt các khung hình khoá có chung ít nhất một từ với truy vấn. Nó rẻ khi từ vựng phân biệt tốt và đắt dần khi bản đồ lớn. Ở phía kia, tìm láng giềng gần nhất là một phép nhân ma trận với vectơ, tức số phép tính tỉ lệ với số khung hình khoá nhân số chiều — một đại lượng nhỏ so với mọi thứ khác trong hệ, và có thư viện vectơ hoá sẵn. Với cả hai bên, nếu bạn nghi ngờ thì hãy đo; nhưng đây hiếm khi là chỗ nút thắt nằm.

Chi phí trích mô tả. Đây là khoản duy nhất đáng kể, và chênh lệch là tuyệt đối chứ không phải theo tỉ lệ. Túi từ trả bằng *không*, vì nó dùng lại đúng các mô tả ORB đã được trích cho việc bám vết. Một mô hình học được trả trọn một lượt xuôi qua mạng cho mỗi khung hình khoá, và lượt xuôi đó là một khoản mới hoàn toàn trong ngân sách. Bao nhiêu mili-giây thì chỉ có phép đo trên phần cứng đích mới nói được, và phải đo khi hệ đang chạy đủ tải chứ không đo mô-đun đứng một mình.

Vậy câu “DBoW2 nhẹ hơn” là đúng, nhưng đúng vì một lý do khác với lý do người ta hay nêu. Nó nhẹ vì chi phí biên của nó bằng không, không phải vì nó tốn ít bộ nhớ. Phân biệt này quan trọng khi bạn cân nhắc: nếu hệ của bạn đã chạy một mạng khác trên khung hình khoá, chi phí biên của việc thêm một đầu ra toàn ảnh có thể gần bằng không, và toàn bộ lập luận đảo chiều. Đó chính là ý tưởng của HF-Net.

G. Thử một bộ truy hồi mà không đụng vào mã của hệ

Có một cách rẻ để trả lời câu “mô tả này có hơn không” trước khi viết một dòng mã tích hợp. Ý tưởng là tách hoàn toàn việc đánh giá tầng một ra khỏi hệ SLAM, và làm nó ngoại tuyến trên chính dữ liệu của bạn.

Quy trình gồm bốn bước.

1. Chạy hệ như bình thường, xuất ra ảnh của mọi khung hình khoá cùng tư thế ước lượng của chúng.
2. Dựng nhãn chân trị cho từng cặp khung hình khoá: đúng nếu khoảng cách thật dưới ngưỡng và hai hướng nhìn không ngược nhau.
3. Với mỗi bộ mô tả cần so, tính vectơ cho mọi khung hình khoá rồi lấy danh sách láng giềng gần nhất.
4. Vẽ recall theo số ứng viên cho từng bộ mô tả, trên cùng một tập khung hình khoá.

Bước bốn cho một đường cong so sánh trực tiếp, trên đúng dữ liệu của bạn, mà không cần tích hợp gì. Nếu hai đường cong gần trùng nhau, câu trả lời đã có và bạn tiết kiệm được vài tuần.

Hai lưu ý làm phép so này trung thực. Thứ nhất, phải loại các cặp khung hình khoá gần nhau về thời gian, nếu không mọi bộ mô tả đều đạt recall gần một và đường cong vô nghĩa. Thứ hai, tập khung hình khoá của bạn được sinh ra bởi chính chính sách chọn khung hình khoá hiện tại, nên nó không phải mẫu ngẫu nhiên của cảnh. Kết luận rút ra chỉ có hiệu lực cho chính chính sách đó.

H. Có đáng thay DBoW2 trên hệ của bạn không

Đây là chỗ phải trả lời thẳng. Câu trả lời phụ thuộc hoàn toàn vào ba con số ở mục trước, chứ không phụ thuộc bảng xếp hạng nào.

Nếu tầng một của bạn đang để lọt phần lớn ứng viên đúng xuống tầng sau, thì thay nó không mua được gì. Đây là điều bảng sống sót trả lời trong một buổi chiều: nếu tỉ lệ sống sót ở cổng một và cổng hai đều cao, tầng một không phải nút thắt, và một bộ truy hồi tốt hơn chỉ đưa thêm ứng viên vào một cái phễu vốn đang nghẽn ở chỗ khác. Trên một hệ chạy chuỗi ngắn cùng điều kiện ánh sáng, đây là kết cục thường gặp — nhưng bạn phải tự đo chứ đừng tin câu này.

Có bốn tình huống mà mô tả học được thật sự thắng, và nên kiểm xem hệ của bạn có rơi vào tình huống nào không.

- **Đi lại sau nhiều giờ hoặc nhiều ngày** — ánh sáng đổi hẳn. Đây là điểm mạnh rõ nhất và cũng là thứ EuRoC không có.
- **Đi lại theo chiều ngược hoặc lệch ngang** — mô tả nhị phân gây sớm hơn nhiều so với mô tả học được.

- **Bản đồ rất lớn hoặc nhiều phiên** — khi số khung hình khoá lên hàng trăm nghìn, phân biệt của tf-idf bão hoà.
- **Định vị lại nguội trong bản đồ đã dựng sẵn** — không có tiên nghiệm chuyển động nào để dựa vào.

Về chi phí, có một tình tiết giảm nhẹ đáng biết: mô tả toàn ảnh chỉ cần tính trên *khung hình khoá*, không phải trên mọi khung hình, nên tần suất gọi thấp hơn tần số ảnh cả một bậc. Ngân sách vì thế thường khả thi, nhưng con số cụ thể phải đo trên phần cứng đích. Cái phải bỏ đi thì rõ ràng hơn và không cần đo: bạn mất khả năng dùng lại mô tả ORB miễn phí, mất tính suy luận được của từ điển, và thêm một phụ thuộc thời gian chạy vào một trình biên dịch suy luận.

CẦN NHỚ

Chỉ thay tầng lấy ứng viên khi bạn đã đo được rằng ứng viên đúng đang chết ở *tầng đó*. Trên một hệ stereo-inertial chạy chuỗi ngắn cùng điều kiện ánh sáng, hãy vào cuộc với giả thuyết ngược lại — rằng tầng đó không phải thủ phạm — rồi bắt bảng sống sót bác bỏ giả thuyết ấy. [T2]

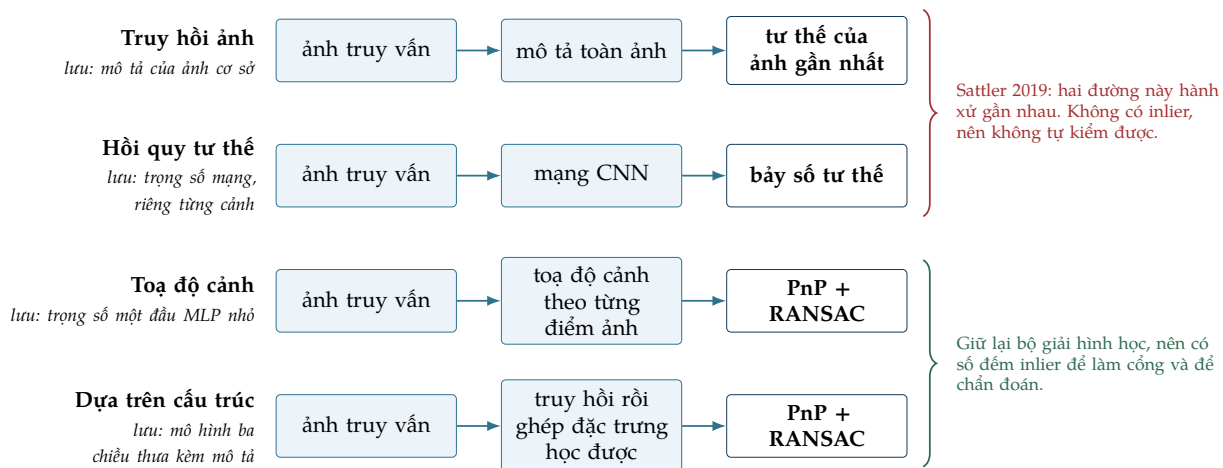
3.4 Định vị lại: hồi quy tư thế, hồi quy tọa độ cảnh và hloc (Relocalization)

A. Bốn họ, phân biệt bằng “cái gì được lưu”

Định vị lại là bài toán anh em của đóng vòng: cho một ảnh và một bản đồ, tìm tư thế camera. Khác biệt là đóng vòng có tiên nghiệm chuyển động, còn định vị lại nguội thì không. Có bốn họ giải bài toán này, và cách phân biệt gọn nhất là hỏi *cái gì được lưu trong bản đồ*.

- **Truy hồi ảnh** — lưu mô tả toàn ảnh của ảnh cơ sở. Trả về tư thế của ảnh gần nhất, hoặc nội suy vài ảnh gần nhất. Không cần hình học.
- **Hồi quy tư thế tuyệt đối** — lưu trọng số một mạng. Mạng nhận ảnh và trả thẳng ra bảy số. Đó là PoseNet [56].
- **Hồi quy tọa độ cảnh** — lưu trọng số một mạng, nhưng mạng trả về tọa độ ba chiều của *từng điểm ảnh* trong hệ tọa độ cảnh. Tư thế được giải sau đó bằng PnP và RANSAC. Đó là dòng DSAC và ACE.
- **Dựa trên cấu trúc** — lưu một mô hình ba chiều thưa kèm mô tả. Ghép hai chiều với ba chiều rồi giải PnP. Đó là hloc và cũng chính là cách ORB-SLAM3 định vị lại.

Hình 3.7 vẽ bốn đường đi này cạnh nhau. Trục ngang là “bao nhiêu hình học còn được giữ ở cuối đường ống”.



Hình 3.7. Bốn họ định vị lại, phân biệt bằng thứ được lưu trong bản đồ và bằng lượng hình học còn lại ở cuối đường ống. Hai họ dưới giữ một bộ giải hình học có kiểm chứng nên có inlier để chẩn đoán; hai họ trên không có. Đó là luận điểm của Sattler và cộng sự ở dạng sơ đồ: hồi quy tư thế hành xử gần với truy hồi ảnh hơn là với định vị dựa trên cấu trúc.

B. PoseNet và phản bác của Sattler 2019

PoseNet đưa ra một ý tưởng rất gọn: huấn luyện một mạng tích chập hồi quy thẳng từ ảnh sang tư thế [56]. Bản đồ trở thành trọng số mạng, kích thước cố định, truy vấn có thời gian cố định. Nghe rất hợp với robot.

Phản bác đến vào năm 2019 và nó rất sắc [57]. Nhóm tác giả dựng một đường cơ sở chỉ dùng truy hồi ảnh cộng nội suy tư thế trên tập huấn luyện. Đường cơ sở đơn giản đó ngang bằng hoặc tốt hơn phần lớn phương pháp hồi quy tư thế. Họ còn chỉ ra rằng sai số của hồi quy tư thế tăng vọt khi tư thế truy vấn ra ngoài phân phối tư thế huấn luyện.

Cơ chế đằng sau rất dễ hiểu khi đã nói ra. Mạng không học hình học chiều; nó học một ánh xạ từ ảnh vào đa tạp các tư thế đã thấy khi huấn luyện. Vì vậy nó nội suy tốt và ngoại suy rất tệ. Về mặt hành vi, nó là một bộ truy hồi có nội suy, chứ không phải một bộ định vị.

Với kỹ sư SLAM, hệ quả thực dụng còn quan trọng hơn kết quả số. Hồi quy tư thế không trả về inlier. Không có inlier thì không có cách nào biết một tư thế trả về là đáng tin hay rác. Trong một hệ mà một ràng buộc sai làm hỏng cả bản đồ, mất khả năng tự kiểm ấy là cái giá quá đắt.

↔ cùng nguyên lý với B/08 — đây đúng là dịch chuyển phân phối ở dạng hình học: tư thế truy vấn ngoài bao lồi của tư thế huấn luyện là một mẫu ngoài phân phối

C. Định vị lại nguội trong hệ của bạn, và ba chỗ chen được mô-đun học được

Mọi so sánh phải so với đường mà hệ của bạn đang chạy, nên cần nhắc lại đường đó trước. ORB-SLAM3 khi mất bám vết sẽ truy vấn túi từ để lấy ứng viên, ghép đặc trưng theo cây từ vựng, chạy PnP trong RANSAC để có tư thế thô, rồi tìm thêm khớp bằng cách chiếu bản đồ và tối ưu lại [1]. Đó vẫn là bốn tầng cũ, chỉ khác ở chỗ bài toán là sáu bậc chứ không phải bảy.

Có ba chỗ chen được một mô-đun học được vào đường đó, và rủi ro của chúng rất khác nhau.

- **Chỉ thay tầng một** — thêm một mô tả toàn ảnh học được, chạy song song với túi từ, rồi hợp nhất hai danh sách ứng viên. Rủi ro thấp nhất, vì tầng ba vẫn kiểm chứng mọi thứ. Nếu mô-đun mới đề xuất rác, rác bị loại.
- **Thay tầng hai cho riêng cặp ứng viên** — chạy bộ ghép cặp học được thay cho ghép theo cây từ vựng. Rủi ro trung bình, vì nó đổi phân bố inlier nên buộc phải đặt lại ngưỡng ở tầng ba.

- **Thay cả tầng ba** — dùng một mô-đun trả thẳng tư thế. Rủi ro cao nhất, vì bạn mất số đếm inlier, tức mất luôn cái cổng.

Thứ tự rủi ro này cũng là thứ tự nên thử. Nó cho phép bạn dừng ngay khi đo được rằng bước tiếp theo không mua thêm gì.

D. Từ DSAC tới ACE: giữ lại bộ giải hình học

Hồi quy toạ độ cảnh chọn một chỗ đứng khác. Mạng không dự đoán tư thế; nó dự đoán, cho mỗi điểm ảnh, toạ độ ba chiều tương ứng trong hệ toạ độ cảnh. Tư thế được giải sau bằng PnP trong vòng RANSAC. Bộ giải hình học được giữ nguyên.

DSAC giải quyết trở ngại kỹ thuật của cách làm này: RANSAC không khả vi, nên không lan được *gradient* về mạng [58]. Giải pháp là thay bước chọn giả thuyết tốt nhất bằng một phép lấy mẫu theo xác suất, rồi lấy kỳ vọng. Nhờ vậy toàn bộ đường ống huấn luyện được đầu-cuối.

DSAC++ bỏ được yêu cầu phải có mô hình ba chiều khi huấn luyện, bằng cách khởi tạo qua sai số chiếu lại [59]. DSAC* hợp nhất các biến thể ảnh màu và ảnh có độ sâu, đồng thời rút ngắn huấn luyện [60].

ACE là bước có ý nghĩa thực dụng nhất của dòng này [61]. Nó tách mạng thành hai phần: một *backbone* trích đặc trưng, huấn luyện sẵn và không phụ thuộc cảnh, cộng một đầu MLP nhỏ riêng cho từng cảnh. Chỉ đầu MLP được huấn luyện lại, trên các *batch* rất lớn gồm đặc trưng lấy ngẫu nhiên khắp tập ảnh. Kết quả là bản đồ của một cảnh thu về đúng trọng số của một đầu MLP nhỏ, và thời gian dựng nó ngắn hơn hẳn một quy trình dựng lại cấu trúc từ chuyển động.

ACE Zero đi thêm một bước: học toạ độ cảnh từ tập ảnh *không có tư thế*, tức là thay luôn vai trò của một quy trình dựng lại cấu trúc từ chuyển động [62]. Dòng này còn tiếp tục với các biến thể mang tên ACE-G và ACE-SLAM, hướng tới tổng quát hoá sang cảnh mới và tới việc nhúng thẳng vào vòng SLAM. Tôi chưa đối chiếu chi tiết hai công trình đó, nên chỉ nêu tên và ý tưởng, không trích dẫn và không mô tả kết quả số của chúng.

E. hloc: chính cái phễu của bạn, thay từng tầng bằng mô-đun học được

hloc là quy trình định vị đang được dùng như chuẩn tham chiếu [49]. Nó gồm bốn bước: truy hồi bằng mô tả toàn ảnh, trích đặc trưng cục bộ học được, ghép cặp học được, rồi PnP trong RANSAC trên một mô hình ba chiều dựng bằng COLMAP [63].

Điều cần thấy là hloc *chính là* cái phễu bốn tầng ở Hình 3.1, với ba tầng đầu được thay bằng mô-đun học được và tầng cuối giữ nguyên bộ giải hình học. Đây là kiến trúc an toàn, và nó là bài học thiết kế lớn nhất của cả chương: học phần tìm tương ứng, giữ nguyên phần ước lượng.

Lý do rất cụ thể. Bộ ước lượng hình học cho bạn ba thứ mà một mạng hồi quy không cho: một số đếm inlier để làm cổng, một hiệp phương sai để đặt trọng số trong đồ thị nhân tử, và khả năng chẩn đoán khi hỏng. Bỏ cả ba thứ đó để đổi lấy vài phần trăm recall là một cuộc đổi chác tồi với hệ đang chạy thật.

Về chi phí, hloc không được thiết kế cho thời gian thực và cũng không nên bị ép vào đó. Điều cần thấy là *cấu trúc* của chi phí chứ không phải một con số: một truy vấn đầy đủ gồm một lượt trích mô tả toàn ảnh, một lượt trích đặc trưng cục bộ, rồi *một lượt ghép cặp cho mỗi ảnh trong danh sách ngắn*. Khoản cuối là khoản nhân lên: danh sách ngắn mười ảnh nghĩa là mười lần chạy bộ ghép cặp cho một truy vấn. Vì thế hai núm vặn duy nhất có ý nghĩa là độ dài danh sách ngắn và tần suất gọi. Cách dùng thực tế trong một hệ SLAM là cắt danh sách ngắn xuống hai hoặc ba, và chỉ chạy khi đã mất tám vết hoặc khi có một ứng viên vòng, chứ không chạy mỗi khung hình.

F. Gộp bản đồ: cùng chuỗi cổng, người tiêu thụ khác

Có một chi tiết kỹ thuật quyết định việc mô-đun định vị lại nào cắm được vào hệ của bạn. ORB-SLAM3 dùng đúng một chuỗi cổng cho hai việc khác nhau: đóng vòng trong cùng một bản đồ, và gộp hai bản đồ rời [1]. Khác biệt nằm ở đầu ra chứ không ở đầu vào.

Khi gộp hai bản đồ, hai bản đồ có hai gốc tọa độ độc lập và, với hệ đơn ảnh, hai tỉ lệ độc lập. Vì vậy phép biến đổi cần tìm bắt buộc là Sim3 bảy bậc, không phải một tư thế tuyệt đối. Với hệ stereo hoặc stereo-inertial, tỉ lệ trong mỗi bản đồ đã quan sát được nên hệ số tỉ lệ gần một, nhưng nó vẫn phải được ước lượng chứ không được giả định.

Hệ quả cho các phương pháp học được rất thẳng. Một mô-đun chỉ trả về tư thế tuyệt đối trong một hệ tọa độ cố định thì không dùng được cho việc gộp bản đồ, vì nó không nói gì về quan hệ giữa hai bản đồ. Đây là một lý do kỹ thuật, không phải lý do độ chính xác, và nó thường bị bỏ qua khi người ta so bảng số của các phương pháp định vị lại.

G. Cổng đánh giá cho hệ của bạn

Bảng 3.2 tóm điều kiện dùng được. Kết luận ngắn: hồi quy tọa độ cảnh có chỗ đứng thật, nhưng không phải trong vòng SLAM trực tuyến.

Bảng 3.2. Bốn họ định vị lại, đọc từ góc nhìn một hệ SLAM đang chạy. Cột cuối nói thẳng khi nào không nên dùng.

Họ	Bản đồ là gì	Mạnh khi nào	Hỏng khi nào
Truy hồi ảnh	Mô tả toàn ảnh của ảnh cơ sở	Cần ứng viên nhanh; chấp nhận sai số tư thế lớn	Không cho tư thế chính xác, không có inlier để kiểm chứng
Hồi quy tư thế	Trọng số mạng, riêng từng cảnh	Cảnh nhỏ, tư thế truy vấn nằm trong phân phối huấn luyện	Ngoại suy ngoài quỹ đạo huấn luyện; không có cách tự kiểm
Tọa độ cảnh	Trọng số một đầu MLP nhỏ	Toà nhà đã biết, dựng bản đồ một lần, cần bản đồ rất gọn	Bản đồ đang mở rộng trực tuyến; cần Sim3 để gộp hai bản đồ
Dựa trên cấu trúc	Mô hình ba chiều thừa kèm mô tả	Cần tư thế chính xác kèm inlier; cần gộp bản đồ	Bản đồ rất lớn thì tốn bộ nhớ; ghép cặp học được thì tốn tính toán

Ba lý do khiến hồi quy tọa độ cảnh chưa vào được vòng SLAM trực tuyến. Thứ nhất, nó cần huấn luyện lại cho mỗi cảnh, trong khi bản đồ SLAM lớn dần theo thời gian. Thứ hai, nó trả về tư thế trong một hệ tọa độ cố định, còn đóng vòng cần một phép Sim3 giữa hai đoạn bản đồ. Thứ ba, nó giả định cảnh tĩnh và đã được quan sát đủ khi huấn luyện.

Ngược lại, có một tình huống triển khai mà nó rất hợp: một toà nhà cố định, quét một lần, rồi robot chạy trong đó nhiều tháng. Khi đó việc huấn luyện một đầu MLP cho mỗi khu là chi phí trả một lần, và một bản đồ gọn hơn hẳn mô hình ba chiều thừa là lợi thế thật.

3.5 Kiểm chứng: hình học so với học được (Verification)

A. Vì sao RANSAC Sim3 là chỗ hẹp nhất

Ở tầng ba, hệ phải tìm phép biến đổi bảy bậc tự do nối hai đoạn bản đồ. Bảy bậc vì tỉ lệ của bản đồ đơn ảnh trôi theo thời gian, nên hai đầu vòng nói chung khác tỉ lệ [64]. Nghiệm đóng cho ba cặp điểm đã có từ lâu [45], [46], nên tập mẫu tối thiểu là ba.

Ba yếu tố cùng đẩy tăng này vào thế khó.

- **Số cặp đầu vào ít** — tăng hai chỉ đề xuất vài chục cặp, vì việc ghép bị giới hạn trong cùng nút từ vừng. Đây là mẫu số của w , và bạn phải ghi nó lại chứ không được đoán.
- **Tỉ lệ inlier thấp** — lệch góc nhìn lớn làm phần lớn cặp sai. Ngưỡng inlier chia cho mẫu số ấy cho ra w^* , tỉ lệ inlier tối thiểu bắt buộc phải có; với vài chục cặp và ngưỡng vài chục phần trăm của chúng, w^* rơi vào vùng mà đường cong ở Hình 3.3 đang rất dốc.
- **Bậc ba trong công thức** — xác suất một mẫu sạch là w^3 , nên tụt rất nhanh khi w giảm.

Nhìn ba yếu tố này, hướng chữa hiển nhiên là giảm m . Với camera stereo, tỉ lệ đã quan sát được, nên về lý thuyết bài toán rút xuống sáu bậc. Nếu thêm phương trọng lực từ IMU, hai bậc xoay nữa bị ghim, chỉ còn góc phương vị và tịnh tiến, và tập mẫu tối thiểu rút xuống hai. Hình 3.3 cho thấy lời hứa: ở vùng w thấp, khoảng cách giữa hai đường cong rất lớn.

B. Ví dụ tính tay: vì sao ba cặp điểm là số nhỏ nhất

Lấy ba cặp điểm cụ thể. Trong bản đồ thứ nhất, ba điểm là $p_1 = (0, 0, 0)$, $p_2 = (1, 0, 0)$, $p_3 = (0, 1, 0)$. Trong bản đồ thứ hai, chúng lần lượt là $q_1 = (1, 1, 0)$, $q_2 = (1, 3, 0)$, $q_3 = (-1, 1, 0)$.

Bước một, tìm tỉ lệ. Khoảng cách $|p_2 - p_1| = 1$ còn $|q_2 - q_1| = 2$, nên $s = 2$. Kiểm chéo bằng cặp còn lại: $|p_3 - p_1| = 1$ và $|q_3 - q_1| = 2$, cũng cho $s = 2$.

Bước hai, tìm phép xoay. Sau khi chia cho tỉ lệ, vectơ $(1, 0, 0)$ đi thành $(0, 1, 0)$, và vectơ $(0, 1, 0)$ đi thành $(-1, 0, 0)$. Đó là phép xoay 90 độ quanh trục z .

Bước ba, tìm tịnh tiến. Tâm của hai bộ điểm là $\bar{p} = (\frac{1}{3}, \frac{1}{3}, 0)$ và $\bar{q} = (\frac{1}{3}, \frac{5}{3}, 0)$. Khi đó $t = \bar{q} - sR\bar{p} = (1, 1, 0)$. Kiểm lại với điểm thứ hai: $sRp_2 + t = 2(0, 1, 0) + (1, 1, 0) = (1, 3, 0) = q_2$. Đúng.

Bây giờ tới phần đáng giá. Vì sao tập mẫu tối thiểu là ba chứ không phải hai? Mỗi cặp điểm cho ba ràng buộc vô hướng. Sim3 có bảy bậc tự do. Hai cặp cho sáu ràng buộc, thiếu một. Ba cặp cho chín, dư hai. Vì vậy ba là số nhỏ nhất, và không thể nhỏ hơn nếu không có thông tin ngoài.

Câu cuối chính là chỗ ý tưởng dùng trọng lực xuất hiện. Phương trọng lực từ IMU ghim hai bậc xoay, và camera stereo ghim bậc tỉ lệ. Còn lại bốn bậc, cần sáu ràng buộc, nên hai cặp điểm là đủ. Về mặt tổ hợp, đó là một cải thiện rất lớn: xác suất rút trúng đi từ w^3 lên w^2 .

Bài học tổng quát nằm ở đây, và nó vượt ra ngoài đóng vòng. Mỗi thông tin ngoài mà bạn đưa vào một bộ ước lượng đều rút bậc tự do, và mỗi bậc rút được đều làm xác suất rút mẫu sạch tăng theo hàm mũ. Đó là lý do lập luận này nghe rất thuyết phục. Mục sau kể chuyện gì xảy ra khi đem nó đi đo.

C. Ba giả định ẩn của lời hứa đó, và cách đem nó đi đo

Lập luận ở mục trước rất chặt về mặt toán, nên nó dễ khiến người ta bỏ vài tuần cài đặt trước khi kiểm bất cứ điều gì. Đừng làm thế. Công thức $1 - (1 - w^m)^N$ đứng trên ba giả định ẩn, và mỗi giả định vỡ theo một kiểu riêng. Ba giả định ấy đều kiểm được *trước* khi viết dòng mã nào, bằng đúng cái nhậ ký ở Bảng 3.4.

I. Mạch 2 — ba giả định ẩn của việc rút tập mẫu tối thiểu

1 Vấn đề

Giả định

Hệ quả

Giả định thứ nhất: inlier có đủ, chỉ thiếu lượt rút trúng.

Toàn bộ công thức nói về xác suất *rút*. Nó im lặng hoàn toàn về việc có tồn tại đủ cặp đúng hay không.

Nếu biểu đồ ở Hình 3.2 mang Dạng B, tức đội trần dưới ngưỡng, thì giả định này sai và hạ m không mua được gì. Phép kiểm: so số inlier tốt nhất của các ứng

viên đúng với w^* nhân số cặp đề xuất. Nếu phần lớn nằm dưới, dừng lại ở đây — nút thất nằm ở tầng hai.

2 Vấn đề

Giả định

Giả định thứ hai: hai bậc xoay bị ghim bằng trọng lực là *ghim đúng*.

Phương trọng lực ước lượng được từ IMU có sai số, và sai số nghiêng còn dư tại điểm gặp lại không nhất thiết nhỏ.

Hệ quả

Một giả thuyết chỉ có góc phương vị sẽ ép phép biến đổi về thẳng đứng. Nếu phần nghiêng còn dư đáng kể, ràng buộc sinh ra sai một cách có hệ thống, và tầng bốn sẽ trung thành bơm cái sai đó ra khắp bản đồ. Phép kiểm: với mỗi vòng đóng được bằng nghiệm bảy bậc, ghi lại phần nghiêng của phép Sim3 tìm được; nếu phân bố của nó không tập trung quanh không, việc ghim hai bậc là sai mô hình.

3 Vấn đề

Giả định

Giả định thứ ba: thêm vòng thì bản đồ chỉ tốt lên.

Mỗi vòng được chấp nhận kéo theo một lần hiệu chỉnh bản đồ, và mỗi lần hiệu chỉnh là một cửa sổ mà luồng dựng bản đồ cục bộ bị chặn.

Hệ quả

Nới một cổng để nhận thêm vòng vì thế đổi cả hai đầu: recall tăng, nhưng số cửa sổ rủi ro cũng tăng. Phép kiểm: đếm số lần mất bám vết và số lần hệ sập, không chỉ đếm số vòng.

Cách đo cả ba giả định trong một thí nghiệm duy nhất thì rẻ, và nó không đòi bạn cài đặt bộ giải hai điểm. Chạy hệ như bình thường với bộ giải ba điểm, nhưng *tăng mạnh số vòng lặp RANSAC* — chẳng hạn nhân lên mười lần — rồi so số vòng đóng được. Nếu nguyên nhân thật là thiếu mẫu thì tăng N đã phải mua được một phần lợi ích, vì N và m đi vào cùng một công thức; nếu tăng N không đổi gì thì hạ m cũng sẽ không đổi gì, và bạn vừa tiết kiệm được vài tuần bằng một dòng cấu hình. Đây là thí nghiệm phải chạy *trước*, và nó đúng tinh thần của thí nghiệm oracle ở E/24: đo trần đóng góp của một hướng trước khi đầu tư vào nó.

Cần ghi kèm điều kiện hiệu lực cho toàn bộ lập luận này. Nó được viết cho một hệ stereo-inertial chạy chuỗi ngắn trong nhà, nơi tỉ lệ trôi tỉ lệ nhỏ và tỉ lệ inlier không quá thấp. Trên hệ đơn ảnh ngoài trời, tỉ lệ inlier thấp hơn nhiều, giả định thứ nhất có nhiều khả năng đúng hơn, và cân nhắc có thể đảo chiều.

D. Cổng thứ năm: cái cổng đứng ngay sau RANSAC, và vì sao nó bị bỏ quên

Cổng RANSAC chiếm hết sự chú ý vì nó là cổng lọc mạnh nhất. Nhưng cổng đứng ngay sau nó — tối ưu Sim3 rồi đếm lại inlier — thường cũng loại một phần đáng kể những gì đã vượt được RANSAC, và gần như không ai đo nó. Đây là chỗ dễ có bất ngờ nhất trong cả chuỗi, đúng vì không ai nhìn.

Có ba nguyên nhân khả dĩ, và chúng đòi ba cách xử lý khác nhau.

- **Nghiem thô quá thô** — Sim3 dựng từ ba cặp điểm nằm sát ngưỡng chỉ chính xác cục bộ. Khi dùng nó để chiếu lại và tìm thêm khớp với bán kính hẹp, phép tìm hụt.
- **Tối ưu rơi vào cực tiểu gần đó** — với ít inlier, hàm mục tiêu nông và có nhiều cực tiểu địa phương gần nhau.
- **Giả thuyết vốn dĩ sai** — một ứng viên sai tình cờ vượt được ngưỡng inlier vẫn có thể bị loại đúng ở đây. Trong trường hợp này, cổng năm đang làm hàng phòng thủ thứ hai.

Tỉ lệ giữa ba nguyên nhân này là một câu hỏi thực nghiệm, và cách trả lời nó giống hệt cách đã dùng cho cổng ba: gắn nhãn chân trị cho các ứng viên chết ở cổng năm rồi tách bảng làm hai. Nếu phần lớn là ứng viên sai, cổng năm là bạn và không nên dựng vào. Nếu phần lớn là ứng viên

đúng, thì hai nguyên nhân đầu là chỗ đáng đầu tư, và cách chữa rẻ nhất là nói bán kính chiếu lại rồi tối ưu lại một lần nữa.

Nguyên tắc chung rút ra được: mỗi khi một cổng loại nhiều, câu hỏi tiếp theo luôn giống nhau, và nó là câu hỏi về nhân chứ không phải về ngưỡng.

E. Năm hướng chữa, xếp theo chỗ chúng tác động

Bảng 3.3 liệt kê các hướng đã biết. Hai hướng đầu không dùng học sâu chút nào.

Bảng 3.3. Năm hướng chữa cổng kiểm chứng, xếp theo tầng mà chúng thật sự tác động. Cột cuối không ghi kết quả của ai cả — nó ghi *điều kiện* để hướng đó có lãi, và phép đo bạn phải chạy để biết điều kiện ấy có thoả trên hệ mình hay không.

Hướng	Tác động ở tầng	Cơ chế	Bằng chứng
Rút mẫu tốt hơn: PROSAC, LO-RANSAC, MAGSAC++	Tầng ba, phần rút mẫu	Xếp thứ tự mẫu theo điểm khớp; tối ưu cục bộ trên tập inlier; bỏ ngưỡng cứng [65]–[67]	Rẻ, không cần huấn luyện. Chỉ giúp nếu biểu đồ inlier mang Dạng A
Giảm bậc tự do bằng trọng lực và stereo	Tầng ba, phần rút mẫu	Ghim tỉ lệ bằng stereo, ghim nghiêng bằng IMU, còn hai cặp điểm	Cùng điều kiện với dòng trên, cộng hai giả định nữa. Kiểm bằng phép thử tăng N ở Mạch 2 trước khi cài
Nói cổng đếm, để tầng sau quyết	Tầng ba, phần ngưỡng	Cho giả thuyết tốt nhất đi tiếp dù dưới ngưỡng, dùng chuỗi cổng sẵn có để lọc	Trần lợi ích bằng đúng số ứng viên đúng nằm trong khoảng trống giữa hai phân bố. Đếm nó trước, rồi mới quyết
Ghép cặp học được cho riêng cặp ứng viên	Tầng hai	SuperGlue hoặc LightGlue trên cặp ảnh, tăng cả số cặp lẫn tỉ lệ inlier [68], [69]	Hướng duy nhất tấn công thẳng trần inlier. Chi phí trả một lần cho mỗi ứng viên
Ghép cặp dày không cần điểm đặc trưng	Tầng hai	LoFTR hoặc RoMa cho tương ứng dày ở đường cơ sở rộng [70], [71]	Mạnh nhất khi lệch góc nhìn lớn; cũng đắt nhất, và chi phí trả theo từng cặp ứng viên

Còn một hướng thứ sáu, rẻ và không ai nói tới, nằm ngay trong tầng hai. Việc ghép cặp hiện bị giới hạn trong cùng nút từ vừng ở một mức cây cố định. Nói mức cây đó, hoặc ghép vết cận có kiểm tra tỉ số khoảng cách chỉ cho riêng cặp ứng viên, sẽ tăng số cặp đề xuất. Chi phí trả đúng một lần cho mỗi ứng viên, không phải cho mỗi khung hình. Đây là suy luận từ cơ chế trần inlier, chưa phải một kết quả đã kiểm chứng — nhưng nó rẻ tới mức đáng thử trước mọi hướng học sâu trong bảng.

F. Kiểm chứng học được: chấm điểm một cặp thay vì giải hình học

Có một hướng khác hẳn ở tầng ba: thay vì giải một phép biến đổi, huấn luyện một mạng chấm điểm “hai ảnh này có phải cùng một nơi không”. Hướng này xuất hiện dưới nhiều tên, thường là xếp hạng lại danh sách ngắn. Patch-NetVLAD thuộc nhóm này, và mọi bộ ghép cặp học được cũng có thể dùng theo kiểu này bằng cách lấy số cặp tin cậy làm điểm.

Điểm mạnh thì rõ. Một mạng có thể học các dấu hiệu mà hình học không dùng được: ngữ cảnh toàn ảnh, kết cấu bề mặt, bố cục thô của cảnh. Nó cũng không bị chặn bởi trần inlier, vì nó không cần một tập cặp nhất quán để đưa ra điểm số.

Nhưng có một khác biệt mà không lập luận nào xoá được. Bộ giải hình học trả về một *con số kiểm chứng được*: số inlier, tức số phép đo độc lập cùng ủng hộ một giả thuyết. Một mạng trả về một điểm số đã hiệu chỉnh kém và không có ý nghĩa vật lý. Với một hệ mà một vòng sai bẻ cả bản đồ, khác biệt đó quyết định.

Vì vậy cách dùng đúng là dùng nó làm *bộ lọc trước*, không phải làm bộ thay thế. Cụ thể: cho mạng chấm điểm và sắp xếp lại danh sách ứng viên, rồi chỉ chạy hình học cho vài ứng viên đầu. Bạn tiết kiệm được phần lớn chi phí ở tầng ba mà không mất cái cổng.

Có một trường hợp mà kiểm chứng học được thật sự cần thiết: khi tỉ lệ inlier thấp tới mức hình học không bao giờ hội tụ, chẳng hạn đi lại sau sáu tháng ở cảnh ngoài trời đổi mùa. Ở đó hình học không có gì để làm việc, nên một điểm số học được vẫn hơn không có gì. Đây không phải chế độ vận hành của một hệ chạy chuỗi ngắn trong nhà.

G. Đưa ghép cặp học được vào tầng kiểm chứng thì đổi gì

SuperGlue coi ghép cặp là bài toán gán tối ưu giữa hai tập điểm, giải bằng một mạng đồ thị có *attention* chéo giữa hai ảnh [68]. LightGlue giữ cùng ý tưởng nhưng cho phép dừng sớm theo độ khó của cặp, nên chi phí thích nghi [69].

Ba thay đổi xảy ra khi đưa chúng vào tầng hai của vòng lặp.

- **Số cặp tăng và tỉ lệ inlier tăng** — đây là tác động mong muốn, và nó đánh đúng vào trần inlier.
- **Chi phí trả theo ứng viên, không theo khung hình** — khác hẳn khi dùng chúng cho bám vết. Đây là tình tiết giảm nhẹ quan trọng nhất: số lần gọi bằng số ứng viên mỗi khung hình khoá, tức nhỏ hơn tần số ảnh nhiều bậc. Con số mili-giây thì phải đo trên phần cứng đích, nhưng *cấu trúc* chi phí đã đủ để nói rằng hướng này khả thi hơn hẳn việc chạy cùng mô hình đó cho bám vết.
- **Điểm vận hành dịch chuyển** — nhiều cặp hơn cũng nghĩa là ứng viên sai có nhiều cơ hội hơn để tạo ra một Sim3 trông hợp lý. Precision có thể tụt.

Điểm thứ ba là chỗ hay bị quên. Khoảng trống giữa hai phân bố trong Hình 3.2 là một tính chất của *bộ ghép cặp hiện tại*, không phải một hằng số của bài toán. Đối bộ ghép cặp là đổi cả mẫu số lẫn phân bố, nên phải đo lại khoảng trống đó và đặt lại ngưỡng theo phân bố mới. Nếu không, bạn nhận thêm recall kèm một tỉ lệ vòng sai chưa được đo — và mục sau giải thích vì sao đó là cuộc đổi chác tồi nhất có thể.

3.6 Đo cho đúng: recall, precision và cái giá của một vòng sai (Measuring Loop Closure)

A. Định nghĩa trước, con số sau

Không có con số recall nào có nghĩa nếu chưa nói ba thứ. Thứ nhất, ngưỡng khoảng cách để gọi hai khung hình là “cùng nơi”. Ngoài trời người ta hay dùng 25 mét; trong nhà con số đó vô nghĩa, phải dùng vài mét. Thứ hai, mẫu số: đếm trên tổng số truy vấn, hay chỉ trên các truy vấn thật sự có lần đi lại. Thứ ba, đơn vị: một khung hình, hay một lần đi lại gồm nhiều khung hình.

Ba định nghĩa này có thể làm cùng một hệ hiện ra với recall rất khác nhau. Vì vậy khi đọc một bảng của người khác, hãy tìm ba định nghĩa đó trước khi đọc con số [72].

Một ví dụ tính tay cho thấy độ nhạy. Giả sử một hệ đóng đúng 40 vòng. Với ngưỡng bốn mét, có 100 lần đi lại được coi là “có thật”, nên recall là 0,40. Nới ngưỡng lên mười mét, số lần đi lại “có thật” tăng lên 150, trong khi số vòng đóng được chỉ tăng lên 45 vì các cặp mới thêm đều lệch

góc nhìn lớn. Recall khi đó là 0,30. Cùng một hệ, cùng một lần chạy, hai con số khác nhau một phần tư chỉ vì đổi định nghĩa. Đây cũng là lý do không so được recall giữa hai bài báo dùng hai ngưỡng khác nhau.

B. Đường cong theo số ứng viên, và giới hạn của nó

Chỉ số chuẩn của tầng một là recall theo số ứng viên: xác suất có ít nhất một ứng viên đúng trong N ứng viên đầu. Đường cong này tăng đơn điệu theo N , nên nó luôn đẹp lên khi ta lấy nhiều ứng viên hơn.

Đó cũng chính là giới hạn của nó. Nó là chỉ số của *một tầng*, không phải của hệ thống. Tầng N làm recall tầng một tăng, đồng thời làm khối lượng việc của tầng ba tăng tuyến tính và làm số ứng viên sai lọt vào tầng ba tăng theo. Một hệ có recall tầng một tuyệt vời vẫn có thể đóng số vòng bằng không.

Vì vậy phải báo cáo hai đường cong cạnh nhau: recall theo số ứng viên ở tầng một, và precision-recall của *toàn bộ chuỗi cổng*. Cặp đường cong này là thứ duy nhất trả lời được câu hỏi “nới ngưỡng thì được gì và mất gì”.

C. Dựng đường precision-recall cho cả chuỗi cổng

Một chuỗi sáu cổng không có một ngưỡng duy nhất để quét, nên việc dựng đường cong cần kỹ luật. Quy trình đúng gồm ba bước.

1. Chọn *một* ngưỡng làm trục quét, thường là ngưỡng inlier ở cổng ba, rồi giữ mọi ngưỡng khác cố định.
2. Với mỗi giá trị, chạy lại chuỗi cổng *trên nhật ký đã ghi*, không chạy lại hệ SLAM.
3. Với mỗi giá trị, đếm vòng được chấp nhận, tra nhãn chân trị, và ghi ra một cặp precision-recall.

Bước hai là bước hay bị làm sai nhất, và làm sai nó khiến cả đường cong vô nghĩa. Nếu bạn chạy lại toàn bộ hệ với ngưỡng mới, quỹ đạo sẽ khác, nên tập khung hình khoá khác, nên tập ứng viên khác. Hai điểm trên đường cong khi ấy không còn so được với nhau, vì chúng đến từ hai tập dữ liệu khác nhau. Đây đúng là quy tắc mỗi lần chỉ đổi một thứ, áp vào một chuỗi cổng.

Muốn quét lại trên nhật ký thì nhật ký phải đủ giàu. Bảng 3.4 liệt kê các trường tối thiểu và nói mỗi trường chẩn đoán được gì. Sáu trường này đủ để tái dựng mọi quyết định của chuỗi cổng một cách ngoại tuyến.

Bảng 3.4. Các trường tối thiểu cần ghi cho mỗi ứng viên. Cột cuối là lý do ghi: mỗi trường trả lời một câu hỏi chẩn đoán mà các trường khác không trả lời được.

Trường	Nội dung	Chẩn đoán được gì
Mã hai khung hình khoá	Định danh truy vấn và ứng viên	Tra chân trị để gắn nhãn đúng-sai
Điểm truy hồi	Điểm số của tầng một	Dựng đường recall theo số ứng viên
Số khớp túi từ	Số cặp mà tầng hai đề xuất	Mẫu số của tỉ lệ inlier; phát hiện trần inlier
Số inlier tốt nhất của RANSAC	Giả thuyết tốt nhất sau 300 vòng lặp	Phân biệt thiếu mẫu với trần inlier
Số khớp sau khi chiếu lại	Kết quả tìm thêm khớp có dẫn hướng	Cho biết nghiệm thô có đủ chính xác không
Số inlier sau tối ưu Sim3	Kết quả cổng năm	Tách ba nguyên nhân chết ở cổng năm

Chỉ số nên báo cáo là recall tại mức precision 100 %. Nó thô, nhưng nó đúng bản chất bài toán: nếu tôi không được phép sai lần nào thì tôi bắt được bao nhiêu phần số vòng có thật. Kèm theo nó phải là một cảnh báo về cỡ mẫu. Với vài chục vòng đúng trong toàn bộ thí nghiệm, đúng một vòng sai đã kéo precision từ 100 % xuống khoảng 97 %. Vì vậy luôn báo cáo số vòng tuyệt đối bên cạnh tỉ lệ phần trăm.

D. Bất đối xứng chi phí: vì sao điểm vận hành lệch hẳn về precision

Đây là điểm mấu chốt của cả mục. Một vòng bị bỏ lỡ và một vòng sai không đối xứng nhau, và khoảng cách giữa chúng rất lớn.

Một vòng bỏ lỡ để lại đúng phần trôi đã tích lũy. Phần đó có giới hạn, nó không lan ra chỗ khác, và một vòng sau vẫn có thể sửa nó.

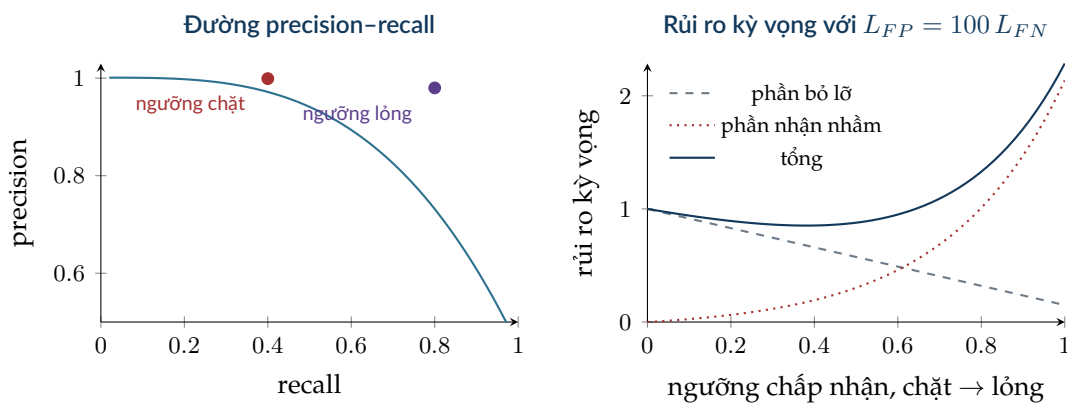
Một vòng sai thì khác hẳn. Tăng bốn nhận nó như một sự thật và phân bố lại sai số cho toàn bộ đồ thị. Bản đồ bị bẻ, và các khung hình khoá cách xa hàng trăm mét cũng bị dời. Hư hỏng không tự khỏi, và việc gỡ nó ra đòi cơ chế phục hồi bản đồ mà phần lớn hệ không có.

Viết thành rủi ro kỳ vọng thì rất gọn. Gọi p_{FP} là xác suất nhận nhầm và p_{FN} là xác suất bỏ lỡ, với L là thiệt hại tương ứng:

$$R = p_{FP} L_{FP} + p_{FN} L_{FN}.$$

Câu diễn giải: bạn không tối thiểu hoá số lần sai, bạn tối thiểu hoá *thiệt hại*. Khi L_{FP} lớn hơn L_{FN} chừng một bậc, điểm tối ưu dịch rất xa về phía precision. Đó là lý do cộng đồng nhận dạng địa điểm báo cáo “recall ở mức precision 100 %” thay vì recall trung bình.

Ví dụ tính tay được. Giả sử $L_{FP} = 100$ và $L_{FN} = 1$, tính bằng đơn vị tùy ý. Ngưỡng lỏng cho $p_{FN} = 0,2$ và $p_{FP} = 0,02$, nên $R = 2,0 + 0,2 = 2,2$. Ngưỡng chặt cho $p_{FN} = 0,6$ và $p_{FP} = 0,001$, nên $R = 0,1 + 0,6 = 0,7$. Ngưỡng chặt thắng, dù nó bỏ lỡ ba phần năm số vòng. Hình 3.8 vẽ cả hai đường.



Hình 3.8. Trái: đường precision–recall minh hoạ của một chuỗi cổng đóng vòng, kèm hai điểm vận hành. Phải: rủi ro kỳ vọng khi một vòng sai đắt gấp một trăm lần một vòng bỏ lỡ. Cực tiểu của đường tổng nằm lệch hẳn về phía ngưỡng chặt, dù ở đó recall chỉ khoảng bốn phần mười. Cả hai đồ thị là minh hoạ dạng hàm, không phải số đo.

E. Đo ở tầng hệ thống, và đọc đuôi phân phối

Recall và precision là chỉ số của bộ phát hiện. Chúng không nói bản đồ tốt lên hay xấu đi. Chỉ số hệ thống là sai số quỹ đạo, đo bằng *evo*, chạy nhiều lần.

Điểm kỹ thuật quan trọng: đọc *đuôi phân phối*, không đọc trung vị. Một vòng sai xảy ra hiếm, nên nó gần như không dời trung vị, nhưng mỗi lần xảy ra thì nó phá hỏng nguyên một lần chạy. Dấu hiệu đặc trưng vì thế rất dễ nhận: trung vị nhích lên một chút trong khi phân vị 95 nhảy lên nhiều

lần. Nếu chỉ nhìn trung vị, thay đổi trông như một khoản lỗ nhỏ đáng chấp nhận; nhìn đuôi thì nó là hổng. Đây là lý do một thay đổi ở tầng ba *bắt buộc* phải báo cáo cả hai con số.

Cách chọn số lần chạy và cách phân biệt cải tiến thật với nhiễu nằm ở B/07-chia-du-lieu-va-danh-gia và E/24-thi-nghiem-va-ablation. Ở đây chỉ cần một quy tắc: mọi so sánh về đóng vòng phải báo cáo cả trung vị lẫn phân vị 95, trên ít nhất mười lần chạy.

Còn một cạm bẫy về chân trị. Trên TUM-VI, chân trị chỉ phủ phòng có hệ bất chuyển động, nên các vòng đóng giữa hành lang không thể kiểm bằng chân trị. Ở những chuỗi như vậy, bằng chứng duy nhất là sai số quỹ đạo cuối cùng, không phải nhân đúng-sai của từng vòng.

F. Đo chi phí cho đúng: đuôi độ trễ và thời gian bị chặn

Còn một loại số phải đo, và nó không phải recall. Đóng vòng là mô-đun duy nhất trong hệ có hai chế độ chi phí rất khác nhau. Chi phí phát hiện chạy trên mọi khung hình khoá; chi phí hiệu chỉnh chỉ chạy khi có vòng.

Chi phí phát hiện phải đọc ở đuôi, không ở trung vị, và lý do là cấu trúc chứ không phải một con số cụ thể: số ứng viên mỗi khung hình khoá dao động rất mạnh, nên phân vị 99 của tầng này thường cách trung vị vài lần. Lập ngân sách theo trung vị nghĩa là trượt hạn đúng ở những chỗ hệ đang bận nhất — tức là đúng những chỗ bạn cần nó nhất.

Chi phí hiệu chỉnh thuộc loại khác hẳn, vì nó *chặn* luồng dựng bản đồ cục bộ. Nó chạy hiếm, nên nó gần như không xuất hiện trong bất kỳ thống kê trung bình nào; nhưng khi chạy thì nó dài hơn một chu kỳ khung hình rất nhiều, và trong suốt khoảng đó hệ không tạo được điểm bản đồ mới. Nếu robot đang di chuyển nhanh ngay lúc ấy, cửa sổ đó là một rủi ro mất bám vết thật. Con số phải đo là thời gian *bị chặn* của luồng dựng bản đồ, không phải thời gian chạy của hàm hiệu chỉnh.

Ba quy tắc đo rút ra được, và cả ba đều dễ bỏ sót.

1. Đo trên phần cứng đích khi hệ đang chạy đủ tải, không đo mô-đun đứng một mình.
2. Báo cáo phân vị 99 bên cạnh trung vị, cho cả phát hiện lẫn hiệu chỉnh.
3. Ghi lại thời gian mà các luồng khác bị chặn, vì đó mới là chi phí hệ thống thật của một vòng.

G. Ba dấu hiệu của một vòng sai mà bạn chưa phát hiện

Vòng sai hiếm khi tự khai. Nó không sinh ra lỗi, không làm sập hệ, và trên phần lớn chuỗi nó không dờn trung vị. Vì vậy cần biết ba dấu hiệu gián tiếp, và cả ba đều rẻ để ghi lại.

Dấu hiệu thứ nhất: bước nhảy tư thế tại thời điểm hiệu chỉnh. Sau khi đóng vòng, tư thế của khung hình khoá gần nhất bị dờn. Lượng dờn đó có ý nghĩa: nó chính là phần trôi mà vòng vừa xoá. Nếu nó lớn hơn nhiều so với trôi hợp lý của quãng đường đã đi, vòng đó đáng nghi. Ghi lại chuẩn của lượng dờn cho mỗi lần hiệu chỉnh là một dòng mã.

Dấu hiệu thứ hai: chi phí còn dư sau tối ưu đồ thị tăng vọt. Một ràng buộc sai mâu thuẫn với phần còn lại của đồ thị, nên nó để lại phần dư lớn ở chính nó hoặc ở các cạnh lân cận. Ghi tổng chi phí trước và sau mỗi lần tối ưu, rồi so với các lần trước, cho một cảnh báo sớm.

Dấu hiệu thứ ba: quỹ đạo có đoạn gấp khúc không giải thích được bằng chuyển động. Robot không dịch chuyển tức thời. Một đoạn quỹ đạo gãy góc tại đúng thời điểm hiệu chỉnh là bằng chứng hình học trực tiếp.

Ba dấu hiệu này đều là dấu hiệu *sau khi* vòng đã được nhận. Chúng không thay được công ở tầng ba, nhưng chúng biến một lỗi im lặng thành một lỗi có triệu chứng. Với một hệ chạy nhiều giờ

ngoài hiện trường, đó là khác biệt giữa việc phát hiện vấn đề trong ngày hôm đó và phát hiện nó sau ba tuần.

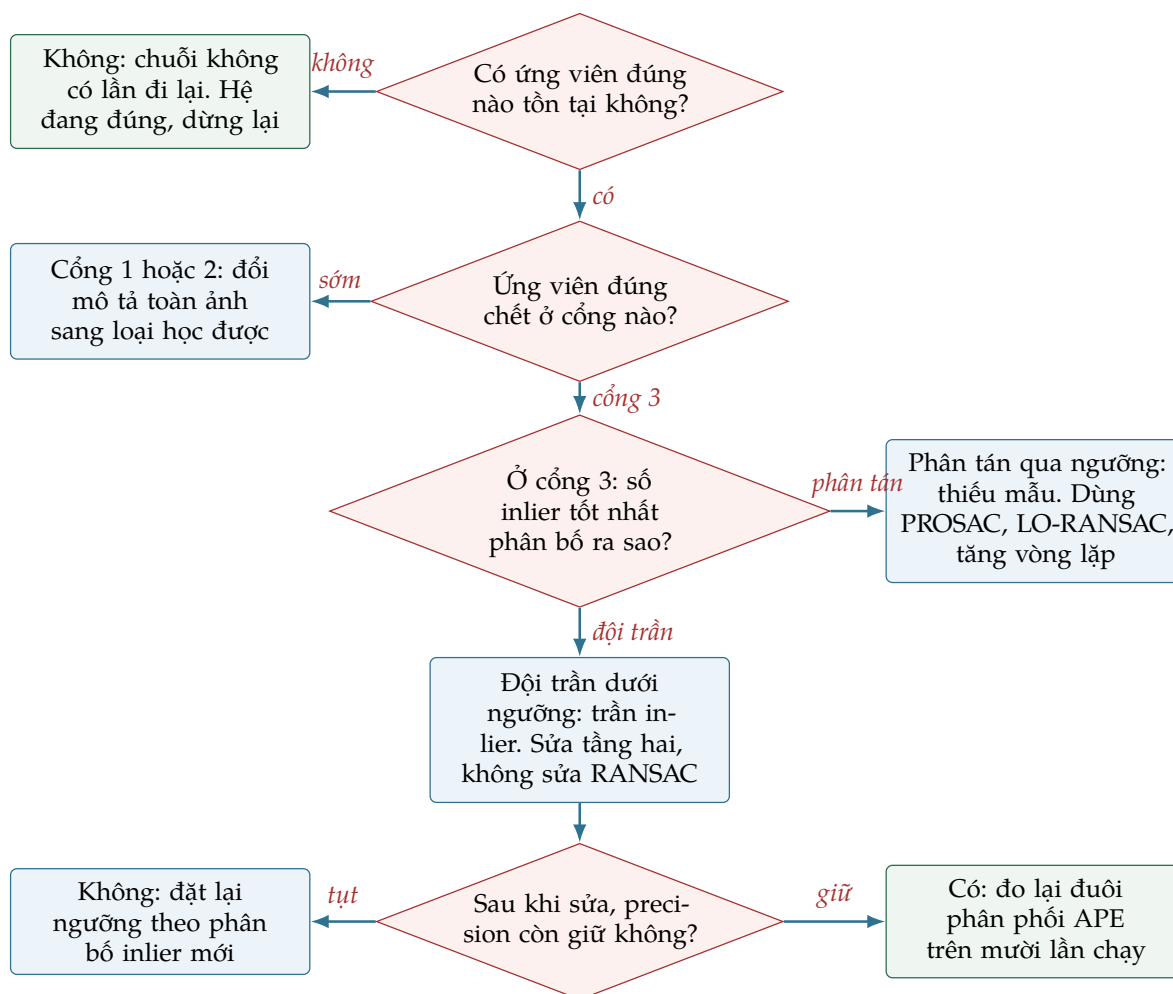
H. Hàng phòng thủ thứ hai: để bộ tối ưu tự chối vòng xấu

Vì precision không bao giờ đạt tuyệt đối, nên có một dòng nghiên cứu đặt lớp bảo vệ ở tầng bốn. Ý tưởng chung là để bộ tối ưu tự hạ trọng số hoặc tự tắt một ràng buộc trông không nhất quán với phần còn lại của đồ thị.

Ba cách kinh điển. Ràng buộc có công tắc gắn thêm một biến ẩn cho mỗi vòng, và biến đó được tối ưu cùng tư thế [73]. Co giãn hiệp phương sai động cho cùng hiệu ứng bằng một biểu thức đóng, rẻ hơn [74]. Mô hình hỗn hợp cực đại thay mỗi ràng buộc bằng một hỗn hợp Gauss, trong đó một thành phần rất rộng đóng vai trò “ràng buộc này có thể sai” [75]. Ngoài ra còn hướng kiểm tra tính nhất quán của cả nhóm vòng theo thời gian rồi loại cả nhóm nếu chúng mâu thuẫn nhau [76].

Điểm cần nói thẳng: các cơ chế này giảm thiệt hại, chúng không thay được một cổng kiểm chứng tốt. Chúng cũng có chi phí riêng, vì một ràng buộc đúng nằm ở vùng đuôi cũng có thể bị tắt nhầm. Với hệ đang chạy thời gian thực, hướng rẻ nhất vẫn là đặt ngưỡng chặt ở tầng ba và chấp nhận bỏ lỡ.

Hình 3.9 gói toàn bộ chương thành một cây quyết định chẩn đoán.



Hình 3.9. Cây chẩn đoán đóng vòng. Điểm quan trọng là thứ tự: hai câu hỏi đầu là phép đo, không phải lựa chọn kỹ thuật, nên bạn không được phép đi tiếp khi chưa có số. Giả thuyết làm việc — và chỉ là giả thuyết — là trên một hệ stereo-inertial chạy chuỗi ngắn cùng điều kiện ánh sáng, nhánh hay gặp nhất đi thẳng xuống ô “trần inlier”, tức phải sửa ở tầng ghép đặc trưng chứ không ở tầng RANSAC. Nếu hệ của bạn không đi vào nhánh đó, cái sai nằm ở giả thuyết chứ không ở cây.

3.7 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả chương phải nói thẳng ngay: chương này *không chứa số đo*. Nó không nói với bạn rằng cổng ba giết bao nhiêu phần trăm ứng viên trên hệ nào, vì một con số như thế chỉ có nghĩa kèm theo một hệ cụ thể, một nhóm tập dữ liệu cụ thể và một chính sách chọn khung hình khoá cụ thể — và khi thiếu ba thứ đó, nó gây hại nhiều hơn giúp. Thứ chương này cho bạn là bộ khung: bốn tầng, các cổng, phép đo đặt ở đâu, và cách đọc kết quả. Mọi phát biểu định tính trong chương — “cổng ba thường là chỗ lọc mạnh nhất”, “trần inlier hay gặp hơn thiếu mẫu trên hệ stereo-inertial chuỗi ngắn” — là giả thuyết làm việc, hợp lý về cơ chế nhưng chưa được chương này chứng minh. Hãy coi chúng là chỗ để bắt đầu tìm, không phải kết luận để trích dẫn.

Kèm theo đó là một giới hạn về phạm vi. Toàn bộ lập luận được viết cho một hệ stereo-inertial chạy chuỗi ngắn, trong nhà hoặc bán trong nhà, cùng điều kiện ánh sáng. Ba đặc điểm ấy quyết định kết luận.

Nếu đổi sang đơn ảnh, tỉ lệ trôi tỉ lệ sẽ lớn hơn, Sim3 thật sự cần bảy bậc, và tỉ lệ inlier thấp hơn. Nếu đổi sang ngoài trời qua nhiều buổi trong ngày, tầng một sẽ trở thành nút thắt thật, và kết luận “đừng thay DBow2” đảo ngược. Nếu đổi sang bản đồ hàng trăm nghìn khung hình khoá,

tf-idf bão hoà và truy hồi học được thắng rõ. Nói cách khác, kết luận trung tâm của chương là một kết luận *có điều kiện*, và điều kiện phải đi kèm mỗi lần nó được nhắc lại.

Chế độ hỏng thứ hai là chính phép đo. Bảng sống sót đo được các ứng viên *đã được đề xuất*. Nó mù hoàn toàn với các lần đi lại mà tăng một không đề xuất gì. Muốn thấy phần mù đó, phải dựng danh sách ứng viên bằng chân trị chứ không bằng bộ truy hồi, tức là chạy một thí nghiệm oracle. Không có bước đó, bạn có thể kết luận rằng tăng một tốt trong khi thật ra nó đang im lặng bỏ sót.

Chế độ hỏng thứ ba nằm ở chân trị. Ngưỡng khoảng cách để gọi hai khung hình là cùng nơi là một lựa chọn, không phải sự thật. Hai khung hình cách nhau ba mét nhưng quay lưng vào nhau thì không hề nhìn chung một cảnh. Một nhãn chỉ dựa trên khoảng cách sẽ gọi chúng là ứng viên đúng, và tăng ba loại chúng là hoàn toàn hợp lý. Nhãn tốt hơn phải tính cả phần chồng lấn trường nhìn, và việc bỏ qua điều này làm recall trông thấp hơn thực tế.

Chế độ hỏng thứ tư mang tính hệ thống. Khi vòng được chấp nhận, việc hiệu chỉnh bản đồ chặn luồng dựng bản đồ cục bộ trong một khoảng dài hơn hẳn một chu kỳ khung hình, và trên phần cứng nhúng thì khoảng đó còn dài hơn. Nghĩa là mỗi vòng đóng thành công cũng là một cửa sổ rủi ro cho việc mất bám vết và cho tranh chấp luồng. Một hệ đóng nhiều vòng hơn chưa chắc là một hệ tốt hơn, và đó là lý do chỉ số cuối cùng phải là sai số quỹ đạo chứ không phải số vòng.

Cuối cùng là một giới hạn về niềm tin. Toàn bộ chương giả định rằng bạn tin vào phép đo của chính mình. Cái bẫy biến đếm ở mục trước cho thấy niềm tin đó có thể sai suốt nhiều tuần mà không có triệu chứng nào. Cách duy nhất để bảo vệ là kiểm chéo mỗi bảng sống sót bằng một đường độc lập, chẳng hạn đếm lại từ nhật ký thô thay vì tin biến đếm.

3.8 Thực hành

Repo và tài nguyên. [cvgl/Hierarchical-Localization](#) là quy trình định vị nên đọc trước tiên, vì nó chính là cái phễu bốn tầng ở dạng mã nguồn. [dorian3d/DBow2](#) để đọc lại phần bạn đang dùng. [QVPR/Patch-NetVLAD](#) và [AnyLoc/AnyLoc](#) cho hai đầu của phổ mô tả toàn ảnh. [gmberton/CosPlace](#) và [amaralibey/MixVPR](#) nếu bạn muốn thử huấn luyện lại. [cvgl/LightGlue](#) cho ghép cặp có chi phí thích nghi, và [zju3dv/LoFTR](#) cho ghép cặp dày. [nianticlabs/ace](#) nếu tình huống triển khai của bạn là toà nhà cố định. [colmap/colmap](#) và [MichaelGrupp/evo](#) là hai công cụ hạ tầng bền nhất trong danh sách. Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; các repo hạ tầng như COLMAP và evo thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Bạn đặt biến đếm ở đâu để biết tầng nào đang giết vòng, và vì sao đặt trước câu lệnh `if` lại làm sai nhãn cột? Nhìn một biểu đồ số inlier tốt nhất, bạn phân biệt thiếu mẫu với trần inlier bằng dấu hiệu nào? Vì sao hồi quy tư thế trực tiếp lại nguy hiểm hơn hồi quy toạ độ cảnh trong một hệ SLAM? Nếu một vòng sai đất gấp một trăm lần một vòng bỏ lỡ, ngưỡng tối ưu nằm ở đâu trên đường precision–recall?

Mini project. Xem khối dự án ở cuối chương: dựng bảng sống sót bốn tầng cho chính hệ của bạn, kèm nhãn đúng–sai từ chân trị.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn tầng của đóng vòng	Lấy ứng viên, ghép đặc trưng, kiểm chứng hình học, tối ưu tư thế. Mỗi tầng là một bộ lọc riêng, nên câu hỏi đúng luôn là “tầng nào đang hẹp” chứ không phải “hệ có đóng vòng không”.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Bảng sống sót từng cổng	Số ứng viên vượt qua mỗi cổng, tính theo phần trăm của số ứng viên tới cổng đó. Nó là phép đo rõ nhất cho biết tầng nào giết vòng, và dụng nó chỉ tồn vài chục dòng mã.
●●● Biến đếm đặt trước câu lệnh <code>if</code> — hậu quả?	Cột mang tên cổng đó thật ra đếm số ca đã vượt cổng trước nó, nên mọi nhân cột dịch một bậc và cổng bị đổ tội luôn là cổng đứng <i>sau</i> thủ phạm thật. Bảng vẫn nhất quán với chính nó nên không có triệu chứng nào. Cách phòng: tăng biến đếm bên trong khối <code>if</code> .
●●● Trần inlier	Số cặp đúng thật sự tồn tại đã ít hơn ngưỡng, nên không cách rút mẫu nào cứu được. Dấu hiệu nhận biết: số inlier tốt nhất của các ứng viên đúng đội trần ngay dưới ngưỡng.
●● Thiếu mẫu	Inlier có đủ nhưng số vòng lặp không đủ để rút trúng một tập mẫu toàn inlier. Dấu hiệu: số inlier tốt nhất phân tán qua ngưỡng. Chữa bằng PROSAC, LO-RANSAC, hoặc tăng số vòng lặp.
●● Công thức hội tụ của RANSAC	$p = 1 - (1 - w^m)^N$, với w là tỉ lệ inlier, m là cỡ tập mẫu tối thiểu, N là số vòng lặp. Thay thử với $m = 3$, $N = 300$: $w = 0,2$ cho $p \approx 0,91$, còn $w = 0,1$ chỉ cho $0,26$ — chia đôi w đủ để biến một cổng gần như luôn mở thành một cổng gần như luôn đóng.
●●● Ba giả định ẩn khi rút tập mẫu từ ba xuống hai	Một: inlier có đủ, chỉ thiếu lượt rút trúng — sai nếu biểu đồ mang Dạng B. Hai: trọng lực ghim hai bậc xoay <i>đúng</i> — sai nếu phần nghiêng còn dư đáng kể. Ba: thêm vòng thì bản đồ chỉ tốt lên — sai vì mỗi lần hiệu chỉnh là một cửa sổ rủi ro. Phép kiểm rõ nhất: tăng N lên mười lần trước đã.
●● Nhãn đúng–sai từ chân trị	Gắn nhãn cho từng ứng viên bằng khoảng cách tư thế thật, với một ngưỡng do bạn chọn — vài mét trong nhà, rộng hơn nhiều ngoài trời. Nó biến bảng sống sót thành hai bảng, và chỉ khi đó mới biết tầng hẹp đang loại đúng hay loại sai.
●●● Túi từ DBoW2	Biểu đồ tần suất các từ thị giác nhị phân, dùng lại chính mô tả ORB đã trích cho bám vết nên gần như miễn phí. Gây khi đổi ánh sáng mạnh, lệch góc nhìn lớn, cấu trúc lặp, hoặc nền ít vân.
●●● NetVLAD	Gộp phần dư quanh K tâm cụm, với phép gán làm mềm bằng softmax nên khả vi. Nhờ đó cả tâm cụm lẫn mạng trích đặc trưng đều học được từ ảnh cùng nơi khác thời điểm.
●● Patch-NetVLAD	Xếp hạng lại danh sách ngắn bằng các mảnh đặc trưng có tính tới vị trí không gian. Nó kéo một phần việc kiểm chứng lên tầng truy hồi, và chi phí tăng theo số ứng viên.
●● HF-Net	Một <i>backbone</i> chung, ba đầu ra: mô tả toàn ảnh, điểm đặc trưng, mô tả cục bộ. Bài học thiết kế là trả một lần cho một bức ảnh thay vì trả hai lần cho hai mạng.
●● CosPlace	Đổi huấn luyện nhận dạng địa điểm từ xếp hạng bộ ba sang phân lớp trên các ô địa lý. Nó cho thấy nút thắt khi ấy nằm ở quy trình dữ liệu chứ không ở kiến trúc.
●● SALAD	Xem việc gán đặc trưng vào cụm như bài toán vận tải tối ưu, kèm một cụm thùng rác cho đặc trưng vô ích, đặt trên nền DINOv2.
●● AnyLoc	Lấy đặc trưng trung gian của mô hình nền rồi gộp bằng VLAD, không huấn luyện lại. Mạnh nhất khi ra ngoài phân phối; đắt hơn về tính toán và mô tả dài hơn.
●●● Phản bác của Sattler 2019	Hồi quy tư thế trực tiếp hành xử gần với truy hồi ảnh có nội suy hơn là với định vị dựa trên cấu trúc. Nó nội suy tốt trong phân phối tư thế huấn luyện và ngoại suy rất tệ ngoài đó.
●● Hồi quy toạ độ cảnh	Mạng dự đoán toạ độ ba chiều cho từng điểm ảnh, rồi PnP trong RANSAC giải tư thế. Khác hồi quy tư thế ở chỗ nó giữ lại bộ giải hình học, nên vẫn có inlier để làm cổng.
●● ACE	Tách <i>backbone</i> không phụ thuộc cảnh khỏi một đầu MLP nhỏ riêng cảnh; bản đồ của một cảnh thu về đúng trọng số của đầu MLP đó. Không hợp vòng SLAM trực tuyến vì bản đồ ở đó lớn dần liên tục.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● hloc	Truy hồi, trích đặc trưng học được, ghép cặp học được, rồi PnP và RANSAC trên mô hình ba chiều. Chính là cái phễu bốn tầng với ba tầng đầu học được và tầng cuối giữ nguyên hình học.
●●● Vì sao điểm vận hành lệch về precision?	Vì một vòng sai bẽ cả bản đồ và không tự khỏi, còn một vòng bỏ lỡ chỉ để lại phần trôi có giới hạn. Khi thiệt hại lệch một bậc, cực tiểu rủi ro kỳ vọng dịch hẳn về ngưỡng chặt.
●● APE trung vị gần như không đổi nhưng phân vị 95 nhảy lên nhiều lần — nghĩa là gì?	Dấu hiệu điển hình của vòng sai: nó hiếm nên không dời trung vị, nhưng mỗi lần xảy ra thì phá một lần chạy. Luôn báo cáo cả trung vị lẫn đuôi trên ít nhất mười lần chạy.
●● Ràng buộc có công tắc	Gắn thêm một biến ẩn cho mỗi vòng để bộ tối ưu tự tắt ràng buộc không nhất quán. Nó giảm thiệt hại nhưng không thay được một công kiểm chứng tốt, và có thể tắt nhầm ràng buộc đúng.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [1]	Chuỗi bốn tầng ở Hình 3.1 là mô tả trực tiếp mô-đun đóng vòng và gộp bản đồ của nó	Hạ tầng tham chiếu; mã nguồn mở
hloc [49]	Thay ba tầng đầu bằng mô-đun học được, giữ nguyên bộ giải hình học ở tầng cuối	Quy trình chuẩn cho định vị trực quan
LightGlue [69]	Ghép cặp học được với chi phí thích nghi theo độ khó cặp ảnh — đúng chỗ trần inlier	Dùng rộng rãi, có bản ONNX
AnyLoc [55]	Đặc trưng mô hình nền cộng gộp VLAD, không huấn luyện lại	Nhắm bài toán ngoài phân phối
ACE [61]	Tách <i>backbone</i> không phụ thuộc cảnh khỏi đầu MLP riêng cảnh, nên dựng bản đồ là huấn luyện một đầu nhỏ	Hợp với triển khai một toà nhà cố định
Đo trên nhánh của bạn	Bảng sống sót từng cổng, nhãn chân trị, phép thử tăng N trước khi hạ m	Chương không đưa số; nó đưa chỗ đặt biến đếm

MINI PROJECT SLAM · Bảng sống sót bốn tầng cho bộ đóng vòng của bạn · 1–2 ngày

Mục tiêu	biến câu “vòng không nổ” thành bốn con số, rồi tách bốn con số ấy theo nhãn đúng–sai để biết tầng hẹp đang loại đúng hay loại sai.
Dữ liệu	bốn chuỗi có lần đi lại thật và một chuỗi đi thẳng không quay lại, làm đối chứng âm. Cấu hình mặc định, không đổi tham số nào.
Các bước	thêm một biến đếm <i>bên trong</i> mỗi khối if của chuỗi cổng, không đặt trước nó; ghi mỗi ứng viên thành một dòng nhật ký gồm mã khung hình khoá, số khớp túi từ, số inlier tốt nhất của RANSAC, và kết cục; chạy mười lần mỗi chuỗi; tra chân trị cho từng cặp khung hình khoá rồi gắn nhãn đúng nếu khoảng cách dưới bốn mét; lập bảng sống sót hai lần, một lần chung và một lần tách theo nhãn; cuối cùng vẽ biểu đồ số inlier tốt nhất cho hai nhóm nhãn.
Xong khi	bạn nói được câu “trên hệ của tôi, $X\%$ số ca chết nằm ở cổng Y , và trong đó $Z\%$ là loại đúng” kèm bảng số. Kiểm tra chéo bắt buộc: chuỗi đi thẳng phải cho gần như không có ứng viên đúng nào; nếu nó vẫn cho nhiều, nhãn chân trị của bạn sai. Kiểm tra thứ hai: đếm lại một cổng từ nhật ký thô và đối chiếu với biến đếm.
Nối với	E/24 cho thiết kế thí nghiệm và số lần chạy; F/28 là nơi bạn sẽ sửa nếu kết quả chỉ vào trần inlier; B/07 cho cách định nghĩa recall và precision trước khi báo cáo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đặt một biến đếm *bên trong* mỗi khối *if* của chuỗi cổng, cộng một dòng nhật ký cho mỗi ứng viên. Đặt trước câu lệnh *if* thì cột đếm “đã tới cổng” chứ không phải “đã qua cổng”, nên mọi nhân dịch một bậc và thủ phạm bị đổ nhầm.
- A2. Bằng nhãn chân trị. Tách bảng sống sót làm hai theo nhãn, rồi nhìn phân bố số inlier tốt nhất của riêng nhóm SAI: nếu nó tắt hẳn từ lâu trước ngưỡng thì mọi ca chết trong nhóm ấy là loại đúng, và một tỉ lệ sống sót thấp là bằng chứng cổng khỏe mạnh chứ không phải cổng hỏng.
- A3. Nếu nguyên nhân thật là trần inlier chứ không phải thiếu mẫu thì không cách rút mẫu nào chạm tới ngưỡng, vì số cặp nhất quán hình học vốn đã ít hơn ngưỡng. Dấu hiệu là Dạng B ở Hình 3.2, và phép kiểm rẻ nhất là tăng số vòng lặp lên mười lần: nếu tăng N không đổi gì thì hạ m cũng sẽ không đổi gì.
- A4. Vì nếu bảng sống sót cho thấy hai cổng đầu để lọt gần hết ứng viên thì tăng lấy ứng viên không phải nút thắt. Một bộ truy hồi tốt hơn khi ấy chỉ đưa thêm ứng viên vào một cái phiếu đang nghẽn ở tầng ba, nên số vòng đóng được không đổi dù recall theo số ứng viên tăng rõ rệt.
- A5. Vì thiết hại lệch. Một vòng sai bẻ toàn bộ đồ thị và không tự khỏi; một vòng bỏ lỡ chỉ để lại phần trôi có giới hạn. Với $L_{FP} = 100 L_{FN}$, ngưỡng chặt cho rủi ro 0,7 còn ngưỡng lỏng cho 2,2.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn đi bộ trong một thành phố lạ và muốn biết mình đã đi qua chỗ này chưa. Bạn làm bốn việc, theo thứ tự. Đầu tiên bạn liếc nhanh và thấy vài góc phố quen quen. Sau đó bạn nhìn kỹ hơn, tìm những chi tiết cụ thể: cái biển hiệu, vết nứt trên tường, chậu cây ở góc. Tiếp theo bạn thử xem những chi tiết ấy có nằm đúng vị trí tương đối với nhau như trong trí nhớ không. Nếu có, bạn kết luận đã đi qua đây, và bạn sửa lại toàn bộ bản đồ trong đầu.

Bây giờ giả sử bạn không bao giờ nhận ra chỗ cũ. Phản xạ thông thường là đổ lỗi cho cái liếc đầu tiên, rồi cố nhìn kỹ hơn ngay từ bước một. Nhưng nếu bạn ghi lại xem mình dừng ở bước nào, bạn sẽ thấy điều khác. Bạn nhận ra góc phố rất tốt. Bạn dừng ở bước thứ ba, vì bạn chỉ nhớ được ba chi tiết, mà ba chi tiết thì không đủ để chắc chắn. Vấn đề nằm ở bước thứ hai: bạn ghi nhớ quá ít chi tiết.

Cái giá của việc dễ tính hơn thì rất nặng. Nếu bạn nhận nhầm một hành lang giống hệt hành lang khác, bạn không chỉ sai ở chỗ đó. Bạn sẽ kéo lệch toàn bộ bản đồ trong đầu, kể cả những đoạn bạn nhớ hoàn toàn chính xác. Một lần nhận nhầm phá nhiều hơn mười lần không nhận ra. Vì vậy quy tắc đúng là: thà đi qua chỗ cũ mà không biết, còn hơn tin nhầm một chỗ mới là chỗ cũ.

Chỗ không rút gọn được: lý do vì sao “ba chi tiết là không đủ” lại là một con số cụ thể chứ không phải cảm tính. Nó đến từ một phép tính xác suất về việc rút ngẫu nhiên bao nhiêu lần thì trúng một bộ chi tiết đều đúng. Mọi cách nói đơn giản hơn đều làm mất chỗ quan trọng nhất: bạn phải phân biệt “rút chưa đủ nhiều lần” với “vốn dĩ không có đủ chi tiết đúng để mà rút”.

Một câu: Đóng vòng là bốn cái sàng nối tiếp, và việc đầu tiên phải làm không phải là đổi cái sàng nào, mà là đo xem cái nào đang giữ lại thứ bạn cần.

3.9 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là tiền đề trực tiếp: khung bốn cổng ở đó chính là thứ được áp dụng nguyên vẹn trong mục quyết định có thay DBow2 hay không. [F/28-dac-trung-va-ghep-cap](#) là chương anh em và là chỗ mà giả thuyết trung tâm của chương này chỉ tới: nếu biểu đồ inlier của bạn mang Dạng B thì việc phải sửa nằm ở đó. [B/07-chia-du-lieu-va-danh-gia](#) cho định nghĩa recall và precision cùng cách chia dữ liệu để những con số ấy có nghĩa. [E/24-thi-nghiem-va-ablation](#) cho cách quyết định bao nhiêu lần chạy là đủ, và cho khái niệm thí nghiệm oracle mà mục đo dùng lại nguyên vẹn. [E/25-do-tin-cay-va-van-hanh](#) là nơi lập luận rủi ro kỳ vọng được trình bày tổng quát, còn ở đây nó chỉ là một trường hợp riêng với thiết hại rất lệch. [F/34-imu-bat-dinh-va-trien-khai](#) nối tiếp về hai chuyện: phương trọng lực từ

IMU đã được dùng ở đây để rút bậc tự do, và ngân sách độ trễ trên Orin quyết định mọi lựa chọn trong bảng so sánh.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một tỉ lệ sống sót rất thấp ở tầng kiểm chứng không tự nó chứng minh rằng tầng đó hỏng, và bạn cần thêm phép đo nào?
2. Nếu biểu đồ số inlier tốt nhất của các ứng viên đúng đội trần ngay dưới ngưỡng, ba hướng chữa nào bị loại ngay lập tức, và vì sao?
3. Lý thuyết RANSAC hứa hẹn một bước nhảy lớn về xác suất hội tụ khi rút tập mẫu từ ba xuống hai. Ba giả định ẩn của lời hứa đó là gì, và với mỗi giả định, phép đo rẻ nhất để biết nó có đúng trên hệ của bạn là gì?
4. Bạn thay DBoW2 bằng một mô tả học được và recall theo số ứng viên tăng rõ rệt, nhưng số vòng đóng được không đổi. Chuyện gì đang xảy ra, và bạn kiểm bằng cách nào?
5. Vì sao hồi quy tư thế trực tiếp lại nguy hiểm hơn hồi quy tọa độ cảnh khi đưa vào một hệ SLAM đang chạy, ngoài chuyện sai số lớn hơn?
6. Đưa một bộ ghép cặp học được vào tầng hai làm tăng số cặp. Vì sao việc đó buộc bạn phải đo lại ngưỡng ở tầng ba, thay vì giữ nguyên?
7. Trên một chuỗi mà chân trị chỉ phủ một phần đường đi, bạn dựa vào bằng chứng nào để nói một vòng vừa đóng là đúng hay sai?
8. Vì sao một hệ đóng nhiều vòng hơn chưa chắc là một hệ tốt hơn, kể cả khi mọi vòng đều đúng?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., "Image Matching Across Wide Baselines: From Paper to Practice," *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., "RoMa: Robust Dense Feature Matching," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., "Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., "HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, "Closed-form solution of absolute orientation using unit quaternions," *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., "Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, "Rethinking Visual Geo-localization for Large-Scale Applications," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, "MixVPR: Feature Mixing for Visual Place Recognition," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., "EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, "Optimal Transport Aggregation for Visual Place Recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features Without Supervision," *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., "AnyLoc: Towards Universal Visual Place Recognition," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Độ sâu, dòng quang và mô hình nền hình học (Depth, Flow and Geometric Foundation Models)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định thay hay không thay); F/28-dac-trung-va-ghep-cap (ghép cặp thừa hồng ở đâu); C/16/02-bieu-dien-3d-va-depth (tính không xác định được của tỉ lệ, bốn nguồn độ sâu); C/15/03-flow-tracking-va-rang-buoc (bài toán khẩu độ, Lucas-Kanade, kim tự tháp).

TÓM TẮT

Chương này hỏi một câu duy nhất: ba họ mô hình đang rất nóng — độ sâu đơn ảnh, dòng quang dày, và mô hình nền hình học — thêm được *thông tin* gì cho một hệ stereo-inertial vốn đã có tỉ lệ mét thật. Đọc xong bạn biết chính xác chỗ nào chúng thêm thông tin thật, chỗ nào chúng chỉ thêm phỏng đoán, và bạn tính được bằng số khi nào một tiên nghiệm dày sẽ đè bẹp phép đo thừa của bạn trong bài toán bình phương tối thiểu. Bạn cũng đọc được ba thiết kế đáng học nhất trong nhóm này: vòng lặp tra cứu của RAFT, phép chuẩn hoá theo tiêu cự của Metric3D, và cách DUST3R đổi biểu diễn đầu ra để xoá luôn bước ước lượng tư thế. Nếu chỉ nhớ một điều: những mô hình này không cho bạn phép đo mới, chúng cho bạn phỏng đoán dày — và chúng không bao giờ nói “tôi không biết”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tỉ lệ (scale)** — ba nghĩa khác nhau trong chương này — (a) *tỉ lệ mét* của một bản đồ SLAM: hệ số nhân biến toạ độ bản đồ thành mét, thứ mà stereo và gia tốc kế cố định được; (b) *tỉ lệ* trong “bất biến tỉ lệ và dịch”: một hệ số nhân tự do trên đầu ra của mạng độ sâu, khác nhau ở mỗi khung hình; (c) *đa tỉ lệ* của kim tự tháp ảnh: mức thu nhỏ. Ba nghĩa không liên quan gì nhau, và trộn chúng là nguồn hiểu nhầm lớn nhất của cả chủ đề này.
- **độ chênh lệch (disparity)** — với một cặp ảnh stereo đã nắn thẳng hàng, đó là hiệu toạ độ ngang của cùng một điểm giữa ảnh trái và ảnh phải, tính bằng pixel. Nó tỉ lệ nghịch với khoảng cách: $d = fb/Z$, trong đó f là tiêu cự tính bằng pixel, b là **đường cơ sở** (stereo baseline) tính bằng mét.
- **độ sâu nghịch đảo (inverse depth)** — đại lượng $1/Z$. Mạng độ sâu hiện đại hầu hết dự đoán đại lượng này chứ không dự đoán Z , vì nó bị chặn (điểm ở vô cực cho giá trị 0) và vì sai số của nó gần với sai số pixel hơn.
- **bản đồ điểm (pointmap)** — một tensor kích thước $H \times W \times 3$: mỗi pixel của ảnh được gán thẳng một điểm ba chiều (X, Y, Z) trong một hệ toạ độ nào đó. Khác với bản đồ độ sâu ở chỗ nó đã mang sẵn hệ toạ độ, nên không cần biết tiêu cự mới đọc ra được hình học.
- **khối chi phí (cost volume)** — bảng ghi mức giống nhau giữa mỗi vị trí ở ảnh này và mỗi vị trí ứng viên ở ảnh kia. Nó là dạng dữ liệu chung của mọi bài toán tương ứng: stereo, dòng quang, ghép cặp.
- **làm cong (warping)** — lấy ảnh hoặc bản đồ đặc trưng thứ hai rồi dịch từng pixel theo một trường chuyển động ước lượng, để nó nằm chồng lên ảnh thứ nhất. Sau khi làm cong, phần sai lệch còn lại chính là phần chưa ước lượng đúng.
- **ma trận thông tin** — nghịch đảo của ma trận hiệp phương sai. Trong một bài toán bình phương tối thiểu có trọng số, nó chính là trọng số: một phép đo có ma trận thông tin lớn kéo nghiệm về phía nó mạnh hơn. Cộng thêm phép đo là cộng thêm thông tin, và tổng thông tin quyết định nghiệm.

- **sai lệch có hệ thống (bias) so với nhiễu ngẫu nhiên** — nhiễu ngẫu nhiên triệt tiêu khi cộng nhiều phép đo; sai lệch có hệ thống thì không, vì mọi phép đo lệch cùng một hướng. Trong chương này, sai số của mạng độ sâu gần với loại thứ hai hơn — đó là lý do nó nguy hiểm.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn là stereo-inertial, đã có tỉ lệ mét thật từ đường cơ sở và gia tốc kế. Vậy một mạng độ sâu đơn ảnh — vốn không có đơn vị nào cả — còn thêm được *thông tin* gì mà bạn chưa có?
- Q2. RAFT bỏ kim tự tháp ảnh nhưng vẫn dùng một kim tự tháp. Nó dựng kim tự tháp trên cái gì, và vì sao chính chỗ đặt kim tự tháp quyết định một vật nhỏ bay nhanh có sống sót hay không?
- Q3. DUST3R nhận hai ảnh chưa hiệu chỉnh và trả về hình học ba chiều mà không hề chạy bước ước lượng tư thế. Tư thế biến đi đâu, hay chính xác hơn, nó nằm ở chỗ nào trong đầu ra?
- Q4. Bạn nhét 5000 pixel độ sâu học được vào cùng bài toán tối ưu với 20 điểm stereo đo thật. Vì sao mẹo “đặt trọng số nhỏ cho độ sâu học được” không cứu được bạn, và phép tính nào cho thấy điều đó trong ba mươi giây?
- Q5. Vì sao một mô hình độ sâu dựa trên khuếch tán lại phá hỏng đúng cái harness chạy lặp mười lần mà bạn đã mất nhiều công để làm cho tắt định?

4.1 Đặt vấn đề

Đọc nhanh. Nếu chỉ có mười phút: đọc Hình 4.1 và mục *Cổng đánh giá* ở cuối. Kết luận của cả chương là ba họ mô hình này chưa nên thay khối nào trong *frontend* của bạn, nhưng hai ý tưởng thiết kế trong đó thì đáng mang đi ngay: tách phần tính một lần khỏi phần lặp, và mô hình hoá sai số tương quan bằng một biến phụ thay vì hạ trọng số.

Chương 28 thay bộ phát hiện và bộ ghép cặp trong *frontend*. Chương này đi xa hơn một bước: nó hỏi xem có nên thay chính *phép đo* bằng một phỏng đoán học được hay không. Ba họ mô hình đang đứng xếp hàng ngoài cửa hệ thống của bạn.

- **Độ sâu từ một ảnh** — đưa vào một khung hình, trả về một giá trị độ sâu cho mọi pixel. Không cần cặp stereo, không cần thị sai, không cần chuyển động.
- **Dòng quang dày** — đưa vào hai khung hình, trả về một vector dịch chuyển cho mọi pixel. Không cần bộ phát hiện điểm đặc trưng.
- **Mô hình nền hình học** — đưa vào một nhóm ảnh chưa hiệu chỉnh, trả về thẳng hình học ba chiều và tư thế camera. Không cần cả pipeline hình học cổ điển.

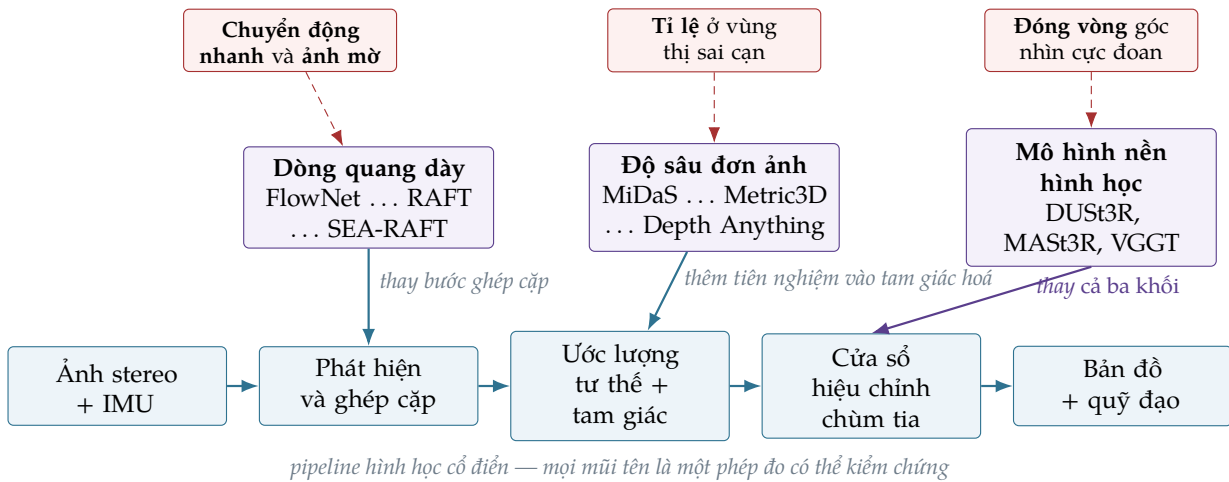
Ba lời chào hàng đó nghe rất khác nhau nhưng có chung một cấu trúc: chúng *đổi thưa lấy dày*. Hệ của bạn hiện sống bằng vài trăm phép đo mỗi khung hình, mỗi phép đo kèm một bài kiểm tra để loại điểm sai. Ba họ trên cho bạn hàng trăm nghìn giá trị mỗi khung hình, không kèm bài kiểm tra nào.

Và đây là chỗ phải nói thẳng ngay từ đầu, vì nó quyết định cách đọc cả chương. Bạn đang chạy một hệ *stereo-inertial*. Đường cơ sở stereo cho bạn tỉ lệ mét từ hình học; gia tốc kế cho bạn tỉ lệ mét từ vật lý. Phần lớn giá trị mà các bài báo độ sâu đơn ảnh quảng cáo — “lấy được hình học ba chiều từ một ảnh” — bạn đã có rồi, và có bằng phép đo chứ không bằng phỏng đoán. Vì vậy câu hỏi đúng không phải “mô hình nào chính xác nhất” mà là câu hỏi hẹp hơn nhiều: *ở đúng những pixel nào phép đo của tôi cạn kiệt, và ở đó một phỏng đoán học được có tốt hơn không có gì hay không*.

Học xong chương này bạn làm được gì mà trước đó không làm được? Bạn khoanh được bằng số vùng ảnh mà stereo của bạn đã hết thông tin. Bạn biết chi phí của từng họ mô hình *tỉ lệ với cái gì*, nên bạn loại được phương án sai cả bậc trước khi cài đặt, và bạn biết phép đo nào biến quan

hệ tỉ lệ ấy thành một con số dùng được cho phần cứng của mình. Và bạn tính được ngưỡng mà một tiên nghiệm dày bắt đầu lấn át phép đo thừa trong bộ tối ưu.

Hình 4.1 đặt ba họ mô hình cạnh nhau theo đúng chỗ chúng cắm vào một hệ SLAM, và theo nỗi đau mà mỗi họ hứa chữa.



Hình 4.1. Ba họ mô hình của chương này, đặt theo đúng khối mà chúng thay thế trong một hệ SLAM cổ điển, và theo nỗi đau mà mỗi họ hứa chữa (mũi tên đứt màu đỏ). Điểm cần chú ý là mức độ xâm lấn tăng dần từ trái sang phải: dòng quang chỉ thay một bước, độ sâu đơn ảnh chèn thêm một tiên nghiệm vào bước tam giác hoá, còn mô hình nền hình học nuốt luôn cả ba khối và trả về hình học đã dựng sẵn. Mức xâm lấn càng cao thì càng khó chẩn đoán khi hỏng, vì càng ít ranh giới để đặt điểm dừng.

TRỰC GIÁC

Hãy tìm một người bạn trong sân vận động đông người. Cách kim tự tháp ảnh: bạn lùi ra thật xa cho cả sân lọt vào tầm mắt, thấy một đám đông ở khán đài B, rồi tiến lại gần dần. Nếu bạn ấy đứng lẻ ở góc sân thì ở tầm nhìn xa bạn ấy chỉ là một chấm mờ, bạn đã đi nhầm về khán đài B, và mọi bước tiến lại gần chỉ tinh chỉnh quanh chỗ sai. Cách của RAFT khác hẳn: trước tiên lập một bảng ghi mức giống nhau giữa *mọi* cặp vị trí, rồi mỗi vòng lặp chỉ đọc vài ô quanh phỏng đoán hiện tại. Khi phỏng đoán còn xa đích, bạn đọc ở mức thô của *bảng* — nhưng cái bị làm thô là bảng, không phải sân, nên người đứng lẻ ở góc sân không hề biến mất. Toàn bộ chương này quay quanh đúng phân biệt đó: làm thô cái gì, và cái gì thì tuyệt đối không được làm thô.

4.2 Độ sâu từ một ảnh và vấn đề tỉ lệ (Monocular Depth and the Scale Problem)

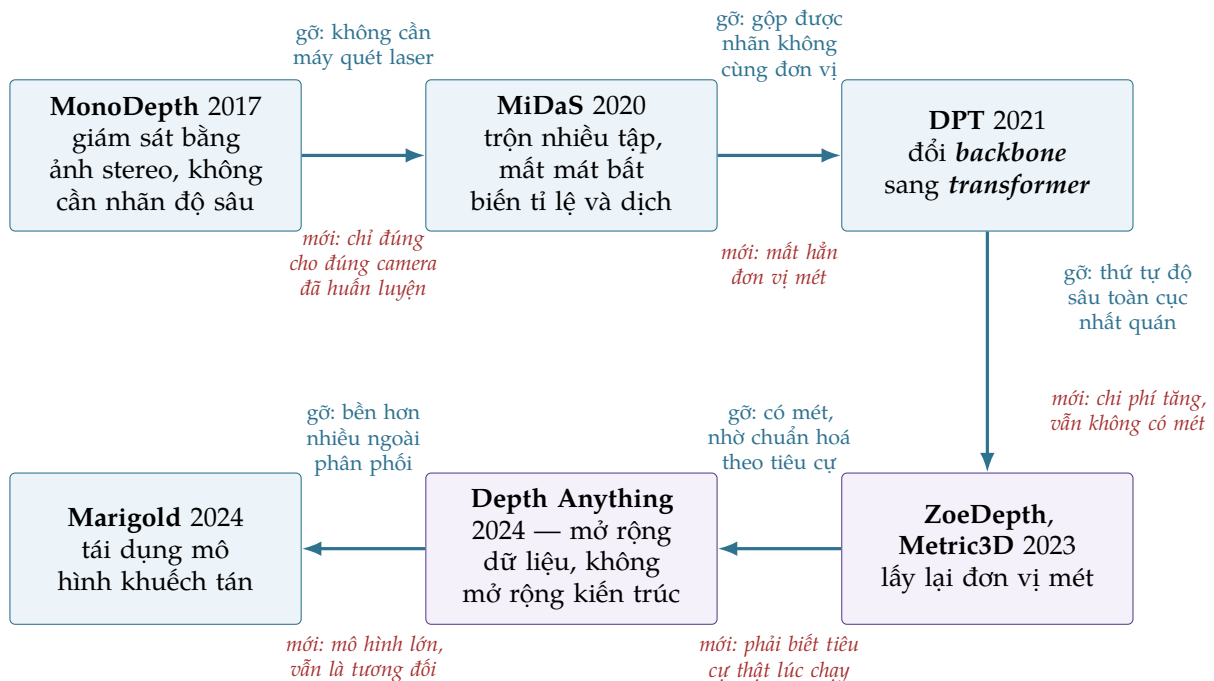
A. Bài toán gốc, và vì sao nó không phải bài toán khó mà là bài toán không giải được

Chương 16 đã chứng minh điều này bằng một hình vẽ, nên ở đây chỉ cần nhắc lại kết luận. Nhân toàn bộ cảnh với hệ số k và nhân luôn quãng đường camera với k thì mọi ảnh thu được giống hệt. Vì thế không lượng dữ liệu nào chứa được, chỉ **tiên nghiệm** (*prior*) mới chữa được.

↔ **tiền đề** C/16/02 — tính không xác định được của tỉ lệ đã được dựng đầy đủ ở đó; chương này chỉ dùng lại kết luận và đi tiếp vào hệ quả kỹ thuật.

Điều đó đặt ra ngay ranh giới đọc cả mục này: mọi con số độ sâu mà một mạng đơn ảnh trả về đều là *suy đoán có học*, không phải phép đo. Câu hỏi kỹ thuật thật sự là suy đoán ấy dựa vào tiên nghiệm gì, và tiên nghiệm ấy vỡ ở đâu.

Sáu thể hệ trong dòng này có thể đọc như sáu lần trả lời cho cùng một câu: *lấy giám sát ở đâu ra*. Hình 4.2 xếp chúng trên một trục, mỗi mũi tên ghi nút thắt được gỡ và vấn đề mới sinh ra.



Hình 4.2. Sáu bước của dòng độ sâu đơn ảnh, đọc theo câu hỏi “lấy giám sát ở đâu”. Đọc hàng trên từ trái sang phải rồi vòng xuống hàng dưới đi ngược lại, theo chiều mũi tên. Nhãn xanh là nút thắt được gỡ, nhãn đỏ nghiêng là vấn đề mới sinh ra. Chú ý hai ô tô tím: chúng là hai cách trả lời khác nhau cho cùng câu hỏi đơn vị mét — ZoeDepth gắn thêm một đầu ra riêng cho từng miền, còn Metric3D chuẩn hoá đầu vào để mọi ảnh trông như chụp bằng một camera duy nhất.

B. Từ MonoDepth tới Depth Anything, đọc theo nguồn giám sát

MonoDepth [77] là bước đầu tiên đáng nhớ vì nó xoá bỏ nhu cầu có nhãn độ sâu. Mạng nhận ảnh trái và dự đoán một trường độ chênh lệch; ta dùng trường đó để làm cong ảnh phải về vị trí ảnh trái; sai lệch quang trắc giữa hai ảnh chính là **hàm mất mát** (*loss function*). Thêm một số hạng buộc độ chênh lệch dự đoán từ ảnh trái và từ ảnh phải phải nhất quán với nhau. Vì lúc huấn luyện đường cơ sở đã biết, đầu ra thực chất có đơn vị mét — nhưng chỉ đúng cho đúng cấu hình camera đó. Monodepth2 sau này sửa hai chỗ hỏng cụ thể: dùng cực tiểu thay vì trung bình của sai lệch quang trắc qua các khung nguồn để chịu được vùng bị che, và tự động che các pixel không đối giữa hai khung để loại vật thể chuyển động cùng tốc độ với camera [78].

Chỗ hỏng thứ hai đáng dừng lại một nhịp, vì nó là ví dụ đẹp về việc một hàm mất mát tự tạo ra một nghiệm sai. Xét một chiếc xe chạy phía trước, cùng vận tốc với camera. Giữa hai khung hình liên tiếp, mọi pixel của chiếc xe đó *không dịch chuyển*. Bây giờ hỏi: giá trị độ sâu nào làm cho phép làm cong không dịch chuyển pixel? Đáp án là độ sâu vô cực. Vậy mạng có thể đạt sai lệch quang trắc bằng đúng không trên cả chiếc xe, chỉ bằng cách khai báo nó ở vô cùng xa. Triệu chứng đặc trưng là những cái lỗ đen ngòm hình chiếc xe trong bản đồ độ sâu. Không có gì sai trong quá trình tối ưu; hàm mất mát đúng là được cực tiểu hoá. Cái sai nằm ở giả định cảnh tĩnh, và cách chữa của Monodepth2 — che các pixel không đối giữa hai khung — là ép giả định đó thành một điều kiện lọc chứ không phải một hy vọng.

MiDaS [79] giải một bài toán khác hẳn: làm sao huấn luyện trên mười tập dữ liệu mà nhãn của chúng không cùng đơn vị. Có tập là quét laser ra mét, có tập là ảnh 3D từ phim với đường cơ sở không ai biết, có tập là kết quả dựng lại từ ảnh nên chỉ đúng sai khác một hệ số. Lời giải là bỏ luôn tham vọng về đơn vị. Trước khi tính sai lệch, ta căn dự đoán về nhãn bằng một phép biến

đối bậc nhất tối ưu: tìm s và t làm cực tiểu $\sum_i (s d_i + t - d_i)^2$, rồi mới đo phần dư. Mạng vì thế chỉ phải học *hình dạng* của trường độ sâu nghịch đảo, không phải giá trị tuyệt đối của nó. Kết quả là khả năng khái quát vượt trội, và một cái giá rất cụ thể mà mục sau sẽ khai thác kỹ.

DPT [80] giữ nguyên công thức dữ liệu và hàm mất mát của MiDaS, chỉ đổi *backbone* sang *transformer* với một bộ giải mã dày. Lý do nó ăn thua nằm ở **trường tiếp nhận** (*receptive field*): trong một mạng tích chập, tầng có tầm nhìn toàn ảnh cũng là tầng có độ phân giải thấp nhất, nên thông tin toàn cục và chi tiết biên không bao giờ gặp nhau ở cùng một tầng. **Attention** có tầm nhìn toàn ảnh ngay từ tầng đầu, nên mô hình giữ được thứ tự độ sâu nhất quán trên toàn khung hình mà không phải hy sinh biên. Với người làm SLAM, đây là điểm đáng chú ý riêng: *thứ tự độ sâu toàn cục* chính là thứ mà mọi bộ ước lượng cục bộ đều thiếu.

ZoeDepth [81] và **Metric3D** [82] đi lấy lại đơn vị mét bằng hai đường khác nhau. ZoeDepth giữ mạng độ sâu tương đối làm phần thân, rồi gắn thêm một đầu ra dự đoán các khoảng độ sâu có đơn vị, tinh chỉnh riêng cho từng miền, cộng một bộ định tuyến chọn đầu ra theo ảnh. Nó dùng được khi ảnh thử nghiệm rơi vào đúng một trong các miền đã huấn luyện. Metric3D làm điều thú vị hơn nhiều, và mục sau dành riêng cho nó.

Depth Anything [83] là một bài học về dữ liệu chứ không phải về kiến trúc. Một mạng thầy được huấn luyện trên phần dữ liệu có nhãn, rồi gán nhãn giả cho hàng chục triệu ảnh không nhãn; mạng trò học trên nhãn giả đó nhưng dưới nhiều loạn mạnh, cộng một số hạng buộc đặc trưng của nó bám theo một mô hình nền thị giác đã đóng băng. Ý tưởng cốt lõi: chỉ chép lại nhãn giả thì trò không giỏi hơn thầy, nên phải làm bài toán *khó hơn* cho trò. Bản v2 [84] đổi một chi tiết có sức nặng bất ngờ: thầy được huấn luyện hoàn toàn trên ảnh tổng hợp. Lý do rất rõ ràng khi nói ra — nhãn độ sâu từ cảm biến thật luôn có lỗi, có nhiễu, và sai bét ở vật trong suốt hoặc phản chiếu, còn nhãn tổng hợp thì hoàn hảo tuyệt đối. Khoảng cách miền giữa ảnh tổng hợp và ảnh thật sau đó được lấp bằng chính bước gán nhãn giả trên ảnh thật. Có một bản v3 xuất hiện trong giai đoạn 2025–2026, theo mô tả công khai thì nó mở rộng cùng công thức sang dự đoán hình học từ số lượng khung hình bất kỳ; tôi chưa tự kiểm chứng chi tiết, nên chỉ nhắc tên và ý tưởng.

Marigold [85] tái dụng một mô hình khuếch tán sinh ảnh đã huấn luyện sẵn: giữ nguyên bộ mã hoá tiềm ẩn, tinh chỉnh phần khử nhiễu để nó sinh ra *bản đồ độ sâu* với điều kiện là ảnh đầu vào. Điều đáng kinh ngạc là nó chỉ cần một lượng ảnh tổng hợp rất nhỏ và một ngân sách huấn luyện rất khiêm tốn so với việc huấn luyện từ đầu, vì tiên nghiệm về cấu trúc cảnh đã nằm sẵn trong mô hình sinh. Biên rất sắc, khái quát rất tốt. Nhưng với hệ của bạn thì hai tính chất của nó là ẩn tử: suy luận cần nhiều bước khử nhiễu, và *nó ngẫu nhiên* — cùng một ảnh, hai lần chạy cho hai bản đồ độ sâu khác nhau, nên người ta phải chạy nhiều lần rồi lấy trung bình. Ta sẽ quay lại chỗ này ở mục nói về chẩn đoán.

Cuối cùng, **Align3R** là một hướng đáng theo dõi vì nó tấn công đúng điểm yếu chí mạng của cả họ này: sự không nhất quán giữa các khung hình. Ý tưởng là kết hợp một mô hình độ sâu đơn ảnh với một mô hình bản đồ điểm hai ảnh, rồi tối ưu chung để tìm tỉ lệ và độ dịch của từng khung sao cho các khung khớp nhau trong không gian ba chiều. Đây là công trình rất mới, tôi không nắm chắc chi tiết nên chỉ nêu ý tưởng và không mô tả kết quả.

Bảng 4.1. Sáu họ độ sâu đơn ảnh. Cột giá trị nhất là cột cuối: mỗi họ mượn tiền nghiệm từ một nguồn khác nhau, nên nó hông đúng khi nguồn đó cạn. Cột đơn vị cho biết đầu ra có đọc thẳng ra mét được hay không.

Họ	Đơn vị	Cơ chế và giả định	Hông khi nào
MonoDepth	mét, riêng cho một camera	Giám sát bằng sai lệch quang trắc giữa hai ảnh stereo; giả định cảnh tĩnh và độ sáng không đổi	Đổi camera hoặc đổi đường cơ sở; vật chuyển động cùng tốc độ camera; vùng bị che ở biên
MiDaS, DPT	không có	Mất mát bất biến tỉ lệ và dịch trên độ sâu nghịch đảo, trộn nhiều tập dữ liệu	Mọi chỗ cần đơn vị; và tỉ lệ với độ dịch <i>khác nhau ở mỗi khung hình</i>
ZoeDepth	mét, theo miền	Thân tương đối cộng một đầu ra có đơn vị, tinh chỉnh riêng từng miền, có bộ định tuyến	Ảnh rơi ra ngoài mọi miền đã huấn luyện; bộ định tuyến chọn nhầm miền
Metric3D	mét	Chuẩn hoá ảnh về một camera chuẩn theo tiêu cự, dự đoán, rồi chuyển ngược lại	Tiêu cự khai báo sai; ống kính méo mạnh hoặc mất cá, nơi mô hình lỗ kim không còn đúng
Depth Anything	tương đối (có bản có đơn vị)	Mở rộng dữ liệu bằng nhãn giả dưới nhiễu loạn mạnh; v2 dùng thầy huấn luyện trên ảnh tổng hợp	Cảnh xa mọi thứ trong tập ảnh không nhãn; mô hình lớn, chi phí cao
Marigold	không có	Tái dụng mô hình khuếch tán sinh ảnh làm bộ dự đoán độ sâu	Cần nhiều bước khử nhiễu; đầu ra <i>ngẫu nhiên</i> giữa các lần chạy

C. Bất biến tỉ lệ và dịch: hai ẩn tự do, và cái bẫy nằm ở ẩn thứ hai

Đây là chỗ phải viết đủ chứ không được rút gọn, vì rút gọn là chỗ mọi người vấp. Một mạng thuộc họ MiDaS trả về một đại lượng $\hat{d}$ liên hệ với độ sâu thật theo

$$\hat{d} \approx \frac{s}{Z} + t,$$

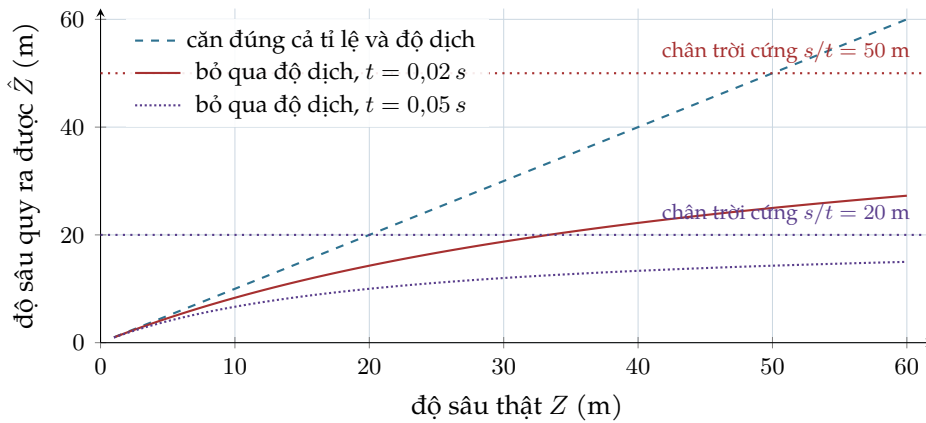
tức nó bất biến với một phép biến đổi bậc nhất trên *độ sâu nghịch đảo*. Nói bằng lời: mạng biết chắc điểm nào gần hơn điểm nào và gần hơn bao nhiêu lần, nhưng nó không biết gốc và không biết đơn vị.

Cách dùng ngây thơ nhất là chia một hằng số: lấy trung vị của $\hat{d}$ rồi co giãn cho khớp với trung vị độ sâu bạn đã biết. Làm thế là ngầm đặt $t = 0$. Hậu quả tính được ngay:

$$\hat{Z} = \frac{s}{\hat{d}} = \frac{s}{s/Z + t} = \frac{Z}{1 + tZ/s}.$$

Khi $Z \rightarrow \infty$, giá trị này không tiến ra vô cực mà bão hoà ở s/t . Mô hình của bạn vừa mọc ra một *chân trời cứng*: mọi thứ xa hơn s/t mét đều được báo về đúng khoảng đó.

Ví dụ nhỏ tính tay được. Giả sử sau khi căn tỉ lệ, độ dịch còn sót lại tương đương $t = 0,02 s$ trên thang độ sâu nghịch đảo, tức chân trời $s/t = 50$ m. Điểm ở 10 m được báo về $10/(1 + 0,2) = 8,3$ m, sai 17%. Điểm ở 20 m được báo về $20/(1,4) = 14,3$ m, sai 29%. Điểm ở 40 m được báo về 22 m, sai 45%. Sai số không ngẫu nhiên mà tăng đơn điệu theo khoảng cách — đúng dạng sai lệch có hệ thống, tức dạng nguy hiểm nhất cho một bộ tối ưu. Hình 4.3 vẽ chính hiện tượng đó.



Hình 4.3. Cái giá của việc bỏ quên ẩn thứ hai. Đầu ra của một mạng bất biến tỉ lệ và dịch cần *hai* tham số để quy về mét; cần bằng một hằng số nhân duy nhất tương đương đặt độ dịch bằng không, và điều đó tạo ra một chân trời cứng ở s/t mét. Đường liền đỏ và đường chấm tím là hai mức độ dịch còn sót khác nhau. Đây là hình minh hoạ công thức $\hat{Z} = Z/(1 + tZ/s)$, không phải số đo từ một mô hình cụ thể. Điểm cần nhớ: sai số tăng đơn điệu theo khoảng cách, nên nó là sai lệch có hệ thống chứ không phải nhiễu.

CẠM BẦY

Hai lỗi hay đi cùng nhau khi cắm một mô hình họ MiDaS vào pipeline. Thứ nhất là quên rằng đầu ra nằm trên thang *độ sâu nghịch đảo*, nên đem trung vị độ sâu ra co giãn thẳng là căn sai không gian, và sai số sinh ra sẽ tập trung hết vào vùng xa. Thứ hai là bỏ quên ẩn độ dịch, tạo ra chân trời cứng vừa nói ở trên. Nhưng lỗi thứ ba mới là lỗi giết hệ thống: s và t là của riêng từng khung hình. Bạn căn đúng cho khung k không có nghĩa là khung $k + 1$ dùng lại được cặp số đó. Một bức tường đứng yên sẽ “thở” phập phồng qua các khung, và bộ tối ưu của bạn — vốn tin rằng cảnh cứng — sẽ hấp thụ nhịp thở đó vào tư thế và vào vận tốc.

D. Metric3D: xoá một tính không xác định được bằng cách chuẩn hoá đầu vào

Mục này đáng đọc kỹ ngay cả khi bạn không định dùng Metric3D, vì thiết kế của nó là một bài học dùng lại được.

Bắt đầu từ nguồn gốc của tính không xác định được, lần này ở dạng khác với tính không xác định được của tỉ lệ. Với một camera lỗ kim, tọa độ ảnh của một điểm là $u = fX/Z$. Nhân đồng thời f và Z với cùng hệ số k thì u không đổi. Nghĩa là: cùng một bức ảnh có thể là một phòng nhỏ chụp bằng ống kính góc rộng, hoặc một nhà kho lớn chụp bằng ống kính tele. Một mạng chỉ nhìn pixel không có cách nào phân biệt hai trường hợp đó. Đây không phải khiếm khuyết của mạng; đó là thiếu thông tin thật.

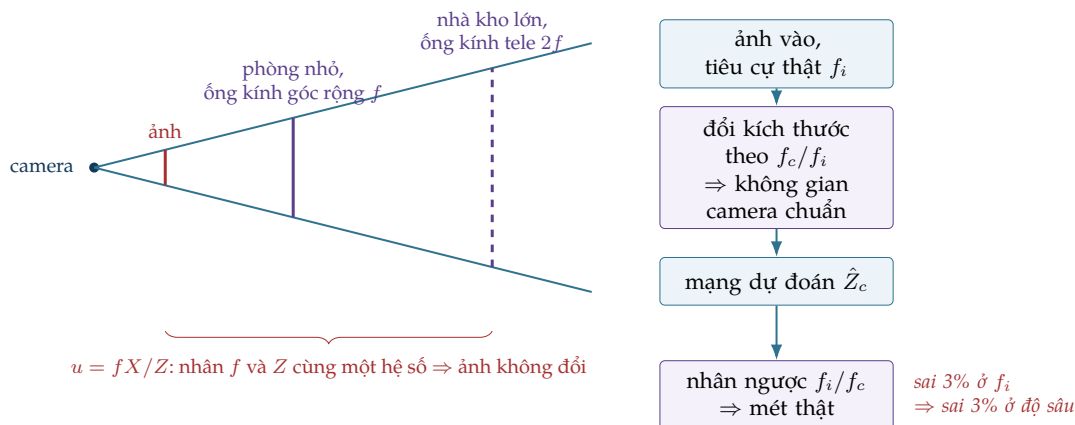
Điều đó giải thích một hiện tượng vốn khó hiểu khi huấn luyện: gộp nhiều tập dữ liệu chụp bằng nhiều camera khác nhau, có nhãn mét đầy đủ, mà mô hình vẫn học ra một thứ nhoè nhoẹt. Lý do là cùng một hình ảnh trong hai tập lại đi kèm hai nhãn mét khác nhau, chỉ vì hai camera có tiêu cự khác nhau. Mạng bị ép học một hàm không phải hàm.

Lời giải của Metric3D là gỡ ẩn số đó ra khỏi bài toán học, đưa nó về bước tiền xử lý. Chọn một tiêu cự chuẩn f_c . Với mỗi ảnh huấn luyện có tiêu cự thật f_i , hoặc thay đổi kích thước ảnh theo tỉ số f_c/f_i để tiêu cự hiệu dụng thành f_c , hoặc giữ ảnh và nhân nhãn độ sâu với f_c/f_i . Sau bước này, toàn bộ dữ liệu huấn luyện hành xử như thể chụp bằng một camera duy nhất, và ánh xạ từ ảnh sang mét trở lại là một hàm. Lúc chạy thật, ta đưa ảnh vào không gian chuẩn, dự đoán, rồi nhân ngược lại theo tiêu cự thật của camera mình.

Ý nghĩa bằng lời: đây chính là cố định gauge. Bạn đã làm đúng việc này mỗi lần cố định khung khoá đầu tiên trong một bài toán hiệu chỉnh chùm tia. Có một bậc tự do mà dữ liệu không nói gì về nó. Thay vì hy vọng bộ tối ưu tự xử lý, ta ghim nó lại bằng tay.

→ cùng nguyên lý với F/34 — đặt trọng số IMU cũng là bài toán “một đại lượng dữ liệu không nói gì tới”, và cách chữa cũng là ghim nó bằng một quy ước rõ ràng thay vì để bộ tối ưu đoán.

Cái giá thì rất thẳng thắn và rất dễ kiểm tra: đầu ra tuyến tính theo tiêu cự bạn khai báo. Nếu f_x hiệu chỉnh của bạn lệch 3%, độ sâu lệch đúng 3%, đều khắp, không có triệu chứng nào khác. Và trên ống kính mắt cá của một số cấu hình thực địa, “tiêu cự lỗ kim” thậm chí không phải một đại lượng xác định. Hình 4.4 vẽ lại toàn bộ lập luận.



Hình 4.4. Trái: vì sao độ sâu mét từ một ảnh là bài toán không giải được nếu không biết tiêu cự — hai cảnh khác nhau chụp bằng hai ống kính khác nhau cho đúng một ảnh. Phải: cách Metric3D gỡ nút. Ấn số bị đẩy ra khỏi bước học và bị ghim bằng một phép chuẩn hoá ở hai đầu, đúng kiểu cố định gauge. Hệ quả trực tiếp là độ chính xác của độ sâu bị chặn trên bởi độ chính xác của phép hiệu chỉnh tiêu cự.

E. Với một hệ stereo-inertial thì độ sâu học được thêm được gì

Bây giờ mới tới câu hỏi thật, và cách trả lời nó là bằng số chứ không bằng ý kiến.

Stereo cho độ sâu qua $Z = fb/d$, nên độ bất định của nó là $\Delta Z = Z^2 \Delta d / (fb)$. Đó là quan hệ bậc hai: sai số tăng theo bình phương khoảng cách. Lấy đúng con số của EuRoC làm ví dụ, với $f \approx 458$ pixel và $b \approx 0,11$ m, ta có $fb \approx 50,5$ đơn vị mét-pixel. Giả sử bộ ghép cặp của bạn đạt độ chính xác độ chênh lệch cỡ 0,2 pixel.

- Ở $Z = 5$ m: $\Delta Z = 25 \times 0,2/50,5 \approx 0,10$ m, tức 2%.
- Ở $Z = 15$ m: $\Delta Z = 225 \times 0,2/50,5 \approx 0,89$ m, tức 6%.
- Ở $Z = 30$ m: $\Delta Z = 900 \times 0,2/50,5 \approx 3,6$ m, tức 12%.

Ở khoảng 50 m, độ chênh lệch tụt xuống dưới một pixel và stereo của bạn về bản chất đã thành đơn mắt.

Điểm cần thấy không phải một con số mà là *hai hình dạng đường cong khác nhau*. Sai số stereo tăng theo Z^2 ; sai số tương đối của một mô hình độ sâu đơn ảnh, chạy trong miền nó đã học, về cơ bản *không* tăng theo khoảng cách, vì nó không dựa vào thị sai chút nào. Một đường cong tăng bậc hai và một đường cong gần như phẳng thì bắt buộc cắt nhau ở đâu đó. Xa hơn điểm cắt ấy, phỏng đoán học được đã tốt hơn phép đo của bạn; gần hơn, phép đo thắng.

Chương này cố ý *không* nói điểm cắt đó nằm ở bao nhiêu mét, vì một con số như thế đòi hai thứ mà không tài liệu nào cho bạn được: độ chính xác độ chênh lệch thật của bộ ghép cặp của bạn, và sai số thật của mô hình trên đúng loại cảnh của bạn. Cả hai đều đo được, và đo chúng chính là

nội dung mini project của chương. Nếu điểm cắt rơi ra ngoài dải khoảng cách mà cảnh của bạn có, thì cả hướng này đóng lại và bạn tiết kiệm được nhiều tuần.

Ba chỗ còn lại mà phép đo của bạn thật sự cạn:

- **Vùng ít vân** — tường trắng, hành lang, mặt sàn nhẵn. Không có vân thì không có ghép cặp, nên không có độ chênh lệch, nên không có độ sâu ở bất kỳ khoảng cách nào.
- **Khởi tạo khi thị sai chưa đủ** — lúc đứng yên hoặc lúc chỉ xoay tại chỗ. Với hệ stereo thì đường cơ sở cứu bạn, nên chỗ này ít quan trọng hơn nhiều so với hệ đơn mắt. Đây là lý do phải nói thẳng: rất nhiều lợi ích được quảng cáo trong các bài báo độ sâu học được là lợi ích của hệ đơn mắt, và chúng biến mất khi bạn có stereo.
- **Lắp lỗ cho bản đồ dày** — nếu bạn cần một bản đồ chiếm dụng để điều hướng chứ không phải để định vị. Đây là cách dùng an toàn nhất, vì nó nằm *hạ nguồn* của đồ thị tư thế và không phản hồi ngược vào nó.

F. Ba cách lấy tỉ lệ mét, và cách nào hợp với bạn

Nếu quyết định dùng một mô hình tương đối, bạn phải chọn một trong ba cách đưa nó về mét. Chúng không tương đương, và cách thứ ba là cách duy nhất bạn nên cân nhắc.

- **Căn theo chính phép đo stereo của bạn.** Ở mỗi khung khoá, lấy các điểm bản đồ đã có độ sâu đo được, khớp một phép biến đổi bậc nhất giữa dự đoán và phép đo trên tập điểm đó, rồi áp phép biến đổi ấy cho toàn ảnh. Ưu điểm: dùng đúng dữ liệu bạn tin. Nhược điểm nặng: cặp tham số được ước lượng từ vùng *gần*, nơi bạn có phép đo, rồi được ngoại suy sang vùng *xa*, nơi bạn cần nó. Ngoại suy một phép biến đổi bậc nhất ra ngoài miền khớp là đúng công thức để sai có hệ thống.
- **Căn theo IMU.** Dùng tích phân trước của gia tốc kế để cố định tỉ lệ trên một đoạn quỹ đạo, rồi buộc độ sâu theo đó. Vấn đề: gia tốc kế cố định tỉ lệ của *chuyển động*, không phải của cấu trúc, và nó chỉ làm được điều đó khi có gia tốc đủ kích thích. Trong một đoạn đi thẳng đều, nó không nói gì cả.
- **Dùng mô hình vốn đã ra mét.** Metric3D và các đầu ra có đơn vị của ZoeDepth hoặc Depth Anything không cần bước căn nào, nên không có chỗ nào để ngoại suy sai. Đổi lại bạn nhận toàn bộ rủi ro lệch miền và rủi ro sai tiêu cự.

Nhận xét thực dụng: hai cách đầu *tạo ra* một tham số mới ước lượng ở mỗi khung, tức tạo ra đúng cái nhíp thử mà mục sau nói là nguy hiểm. Cách thứ ba không tạo thêm tham số nào, nên nếu bạn buộc phải dùng độ sâu học được trong bộ tối ưu, hãy dùng mô hình ra mét và kiểm tra kỹ tiêu cự. Nếu chỉ dùng ngoài bộ tối ưu, cách nào cũng được.

G. Đo độ sâu học được cho đúng mục đích của bạn

Bảng xếp hạng của các **benchmark** độ sâu dùng một bộ thước quen thuộc: sai số tương đối tuyệt đối trung bình trên mọi pixel, và tỉ lệ pixel có sai số dưới một ngưỡng. Cả hai đều là trung bình trên toàn ảnh, và cả hai đều *giấu đi* đúng hai thứ quyết định với bạn.

Thứ nhất, chúng trộn lẫn mọi khoảng cách. Nhưng bạn không cần độ sâu ở mọi khoảng cách; bạn chỉ cần nó ở dải mà stereo đã cạn. Một mô hình đứng đầu bảng nhờ chính xác ở ba mét đầu có thể hoàn toàn vô dụng ở ba mươi mét. Phép đo đúng là vẽ sai số tương đối *theo khoảng cách*, rồi đọc riêng phần đuôi xa.

Thứ hai, và nặng hơn nhiều, chúng không nói gì về *cấu trúc* của sai số. Sai số trung bình 6% có thể là 6% nhiễu độc lập từng pixel, hoặc là 6% lệch chung cho cả mảng tường. Với bộ tối ưu của bạn, hai trường hợp đó cách nhau ba bậc độ lớn về mức nguy hiểm, và mục sau sẽ tính ra con số ấy. Không bảng xếp hạng nào phân biệt chúng.

Cách đo đúng thì đơn giản và bạn tự làm được. Với mỗi khung hình có chân trị độ sâu, tách sai số làm hai phần:

- **Phần chung của khung** — một cặp tỉ lệ và độ dịch tối ưu cho riêng khung đó, tìm bằng bình phương tối thiểu. Đây là phần mà một biến phụ trong bộ tối ưu có thể hấp thụ, nên nó *không* nguy hiểm nếu bạn mô hình hoá nó.
- **Phần dư sau khi trừ đi phần chung** — đây mới là thông tin thật mà mô hình đóng góp, và nó là thứ duy nhất đáng đem so giữa hai mô hình.

Rồi làm thêm một việc nữa: vẽ *phần chung* theo thời gian trên cả chuỗi. Nếu nó dao động ở tần số khung hình thì bạn đã nhìn thấy trước, bằng biểu đồ, đúng cái nhịp thở sẽ làm hỏng ước lượng vận tốc của mình — và bạn thấy nó trước khi viết một dòng mã tích hợp nào.

→ **cùng nguyên lý với E/24** — đây là phân rã lỗi theo loại, áp cho một mô-đun mới: tách phần sửa được bằng một tham số khỏi phần thật sự là sai số.

4.3 Dòng quang dày: từ FlowNet tới SEA-RAFT (Dense Optical Flow)

A. Vì sao ghép cặp thưa chết mà dòng quang sống

Chương 15 đã dựng bài toán khâu độ và Lucas–Kanade, nên ở đây ta đi thẳng vào chỗ liên quan tới nỗi đau của bạn: hai khung hình liên tiếp trong một pha xoay nhanh, ảnh bị mờ, và bộ bám đặc trưng mất dấu.

Cơ chế của cái chết đó đáng viết ra rõ ràng, vì nó giải thích luôn vì sao dòng quang lại sống sót. **Ảnh mờ do chuyển động** (*motion blur*) là một phép tích chập với một nhân dạng đoạn thẳng, dài

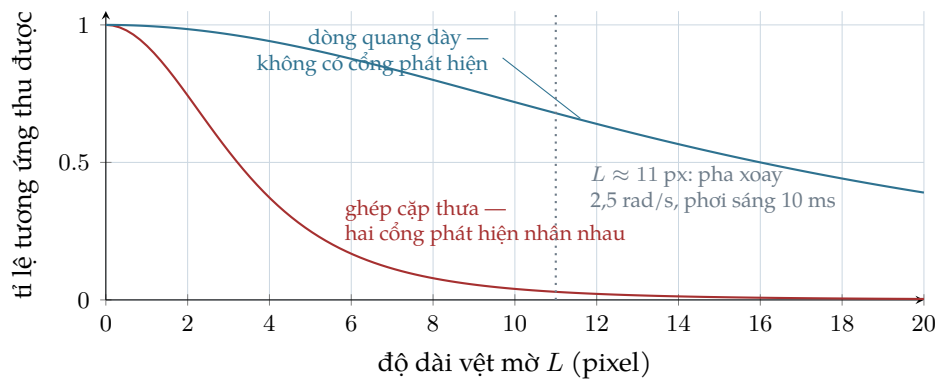
$$L \approx f \omega t_{\text{phơi sáng}} \quad \text{pixel},$$

với ω là tốc độ góc. Lấy số của EuRoC: $f \approx 458$ pixel, một pha xoay nhanh cỡ $\omega = 2,5$ rad/s, thời gian phơi sáng 10 ms trong nhà thiếu sáng. Ra $L \approx 11$ pixel. Đó là một vệt dài hơn cả bán kính của một mảnh mô tả ORB.

Bây giờ tới phần quan trọng. Bộ phát hiện góc đo hai giá trị riêng của ma trận mô men bậc hai của gradient trong một cửa sổ. Làm mờ theo một hướng là lọc thông thấp theo hướng đó: đỉnh gradient tụt đi khoảng L lần và trải ra trên L pixel, nên tổng bình phương gradient theo hướng ấy giảm cỡ L lần. Với $L \approx 11$, giá trị riêng nhỏ nhất mất khoảng một bậc độ lớn. Ngưỡng phát hiện là ngưỡng cứng, nên điểm đơn giản là *không được phát hiện nữa*. Đây là lập luận bậc độ lớn từ cơ chế, không phải số đo.

Và ghép cặp thưa là một chuỗi cổng nhân với nhau. Điểm phải được phát hiện ở khung này *và* ở khung kia, mô tả phải nằm trong ngưỡng, phải qua kiểm tra tỉ số, rồi phải sống sót qua RANSAC. Nếu tính lặp lại của bộ phát hiện tụt từ 0,8 xuống 0,3 thì xác suất một điểm sống qua cả hai khung tụt từ 0,64 xuống 0,09 — giảm bảy lần chỉ vì một cổng. Đó là lý do số điểm khóp sụp đổ chứ không giảm từ từ.

Dòng quang dày không có cổng nào cả. Nó không phát hiện gì hết; nó hỏi “pixel này đi đâu” cho mọi pixel, và nó trả lời bằng cách tổng hợp bằng chứng trên một vùng lớn. Làm mờ phá tần số cao nhưng giữ nguyên cấu trúc tần số thấp, mà tương quan ở mức thô lại đúng là thứ đọc cấu trúc tần số thấp. Thêm nữa, tính trơn học được của mạng lan nghiệm từ vùng có vân sang vùng mờ bên cạnh. Hình 4.5 vẽ hai đường cong tương ứng với hai cơ chế đó.



Hình 4.5. Vì sao ghép cặp thưa sụp đổ còn dòng quang chỉ xuống dốc. Hai đường này là *hình dạng của cơ chế*, không phải số đo: đường đỏ là tích của hai xác suất phát hiện nên nó bình phương mọi suy giảm, còn đường xanh không có cổng phát hiện nào để nhân. Kết luận dùng được: khi vết mờ vượt cỡ bán kính mảnh mô tả, khoảng cách giữa hai đường mở ra nhanh, và đó chính là dài mà một bộ dòng quang mới có lại.

B. Bốn thể hệ dòng quang học được

I Mạch 1 — dòng quang học được, từ FlowNet tới SEA-RAFT

- 1 Vấn đề** Phương pháp biến phân cổ điển chính xác nhưng chậm, và mỗi cảnh phải chỉnh lại tham số bằng tay.

Giải pháp FlowNet đưa cả bài toán vào một mạng tích chập huấn luyện đầu-cuối [86]. Có hai biến thể đáng nhớ vì chúng thể hiện hai triết lý: bản đơn giản xếp chồng hai ảnh thành sáu kênh rồi để mạng tự xoay xở, còn bản có tương quan cài sẵn một tầng tương quan giữa hai luồng đặc trưng.

Giả định cảnh trong dữ liệu tổng hợp dùng để huấn luyện đủ giống cảnh thật.

Vấn đề mới bản có tương quan thẳng rõ, cho thấy để mạng “tự học hình học” là lãng phí; nhưng độ chính xác vẫn thua các phương pháp cổ điển tốt.
- 2 Vấn đề** Một mạng đơn không đủ chính xác, và chuyển động nhỏ bị bỏ sót.

Giải pháp FlowNet 2.0 xếp chồng nhiều FlowNet nối tiếp, mỗi tầng *làm cong* ảnh thứ hai theo nghiệm của tầng trước rồi chỉ ước lượng phần dư; thêm một nhánh nhỏ chuyên cho chuyển động nhỏ và một khối hợp nhất ở cuối [87].

Hệ quả bài học lớn nhất của công trình này không phải kiến trúc mà là *liệu trình dữ liệu*: huấn luyện trên tập đơn giản trước rồi mới sang tập phức tạp cho kết quả khác hẳn so với trộn lẫn.

Giá phải trả mô hình rất lớn, và cấu trúc xếp chồng khiến mỗi tầng chỉ tinh chỉnh quanh nghiệm của tầng trước.
- 3 Vấn đề** Mạng quá to, và ba nguyên lý cổ điển đang bị học lại một cách lãng phí.

Giải pháp PWC-Net viết thẳng ba nguyên lý đó thành ba khối kiến trúc [88]: kim tự tháp đặc trưng học được, làm cong đặc trưng của ảnh hai theo nghiệm của mức thô hơn, và khối chi phí cục bộ với bán kính tìm kiếm nhỏ ở mỗi mức.

Hệ quả nhỏ hơn FlowNet 2.0 hơn một bậc mà chính xác hơn. Đây là ví dụ sạch nhất trong cả chương về giá trị của thiên lệch quy nạp đúng.

Giá phải trả thừa hưởng nguyên vẹn chế độ hỏng của chiến lược thô tới mịn: vật nhỏ bay nhanh biến mất ở mức thô, và khối chi phí cục bộ không đủ tầm để sửa lại.

- 4 Vấn đề** Sai ở mức thô là sai vĩnh viễn, vì mọi mức mịn hơn chỉ tinh chỉnh chứ không tìm lại.
- Giải pháp** RAFT tách hẳn hai việc [17]: tính *một lần* khối tương quan giữa mọi cặp vị trí, rồi lặp lại một phép tra cứu cục bộ quanh phỏng đoán hiện tại, với phép cập nhật do một mạng hồi tiếp đảm nhiệm.
- Giả định** khối tương quan toàn cặp vừa bộ nhớ.
- Hệ quả** chuyển động lớn không còn cần thu nhỏ ảnh, nên vật nhỏ bay nhanh không biến mất. Số vòng lặp trở thành một nút vận độ trễ chỉnh được lúc chạy.
- Giá phải trả** bộ nhớ của khối tương quan, và một mô hình mà bạn không ablate được từng phần.
- 5 Vấn đề** Phép tra cứu của RAFT chỉ nhìn cục bộ, nên vùng bị che và vùng không vân phải chờ thông tin lan qua nhiều vòng lặp.
- Giải pháp** FlowFormer mã hoá chính khối chi phí bằng một *transformer*, để thông tin đi khắp khối trong một lượt thay vì bò dần qua các vòng lặp [89].
- Giá phải trả** bộ nhớ và độ trễ tăng mạnh; đây là hướng cho máy để bàn, không cho hệ nhúng.
- 6 Vấn đề** RAFT cần nhiều vòng lặp vì nó khởi động từ dòng bằng không, và nó không nói được nó tin vào pixel nào.
- Giải pháp** SEA-RAFT đổi ba thứ [90]: hồi quy thẳng một nghiệm khởi đầu thay vì bắt đầu từ không, đổi hàm mất mát sang hỗn hợp hai phân phối Laplace, và đổi dữ liệu tiền huấn luyện.
- Hệ quả** ít vòng lặp hơn nhiều nên nhanh hơn hẳn; và hàm mất mát hỗn hợp cho ra một *độ bất định theo từng pixel* dùng được làm trọng số.
- Giả định** hai thành phần Laplace đủ để mô tả tính đa phương thức của dòng ở biên vật thể.

RAFT-3D là một nhánh đáng biết vì nó đổi đầu ra chứ không đổi thuật toán [91]. Thay vì một vector dịch chuyển hai chiều cho mỗi pixel, nó dự đoán một phép biến đổi cứng ba chiều cho mỗi pixel, rồi ép các pixel có biểu diễn giống nhau phải chia sẻ cùng một chuyển động cứng. Hệ quả trực tiếp và rất hữu ích: các mảng cứng của cảnh tự tách ra thành cụm. Nói cách khác, bài toán phân tách vật thể động rơi ra như một sản phẩm phụ của biểu diễn, không cần một bộ phân vùng ngữ nghĩa riêng.

↔ **đối lập với F/33** — đó là hai cách khác nhau để loại vật động: RAFT-3D dùng tính không nhất quán của chuyển động, còn mặt nạ ngữ nghĩa dùng tên lớp của vật.

C. Vòng lặp tra cứu của RAFT, giải thích kỹ

Đây là thiết kế đáng học nhất trong cả mục, và nó đáng học ngay cả khi bạn không bao giờ chạy RAFT.

Ba thành phần, theo đúng thứ tự dữ liệu chảy qua:

- Mã hoá đặc trưng.** Cả hai ảnh được đưa qua cùng một bộ mã hoá xuống độ phân giải $1/8$. Một bộ mã hoá thứ hai, chỉ chạy trên ảnh một, cho ra đặc trưng ngữ cảnh nuôi vào phép cập nhật.
- Khối tương quan toàn cặp.** Với mọi vị trí i trên lưới ảnh một và mọi vị trí j trên lưới ảnh hai, tính tích vô hướng của hai vector đặc trưng. Với ảnh EuRoC 752×480 , lưới $1/8$ là $94 \times 60 = 5640$ vị trí, nên khối có 5640^2 phần tử — *bình phương số ô lưới*, và đó là con số đáng nhớ vì nó nói rằng độ phân giải lưới là nút vận chi phí mạnh nhất. Rồi *gộp trung bình theo hai chiều cuối* với các hệ số 1, 2, 4, 8 để được bốn mức. Đây là câu quan trọng nhất của cả mục: cái bị làm thô là

hai chiều của ảnh hai trong khối tương quan, không phải lưới của ảnh một. Lưới ảnh một giữ nguyên độ phân giải ở mọi mức, nên một vật nhỏ ở ảnh một không bao giờ biến mất.

3. **Vòng lặp tra cứu và cập nhật.** Trường dòng khởi đầu bằng không. Ở mỗi vòng, với mỗi vị trí x của ảnh một, ta tính vị trí ứng viên $x' = x + f_k(x)$ rồi đọc các giá trị tương quan trong một lân cận bán kính 4 quanh x' , ở cả bốn mức. Mỗi vòng vì thế lấy ra $4 \times (2 \cdot 4 + 1)^2 = 324$ con số cho mỗi pixel. Ba thứ — các con số tra cứu được, dòng hiện tại, và đặc trưng ngữ cảnh — được đưa vào một khối hồi tiếp tích chập, khối này trả về Δf , và ta cộng vào. Lặp lại, dùng chung trọng số ở mọi vòng.

Ý nghĩa của thiết kế này bằng lời: nó tách bạch “chỗ nào giống chỗ nào” — tính một lần, đắt, nhưng không phụ thuộc nghiệm — khỏi “tôi đang đoán ở đâu” — rẻ, lặp lại, phụ thuộc nghiệm. Phép tra cứu ở mức thô thứ tư với bán kính 4 phủ được $\pm 4 \times 8 = \pm 32$ ô của lưới 1/8, tức ± 256 pixel trong ảnh gốc. Chuyển động lớn được xử lý mà không hề thu nhỏ ảnh một lần nào.

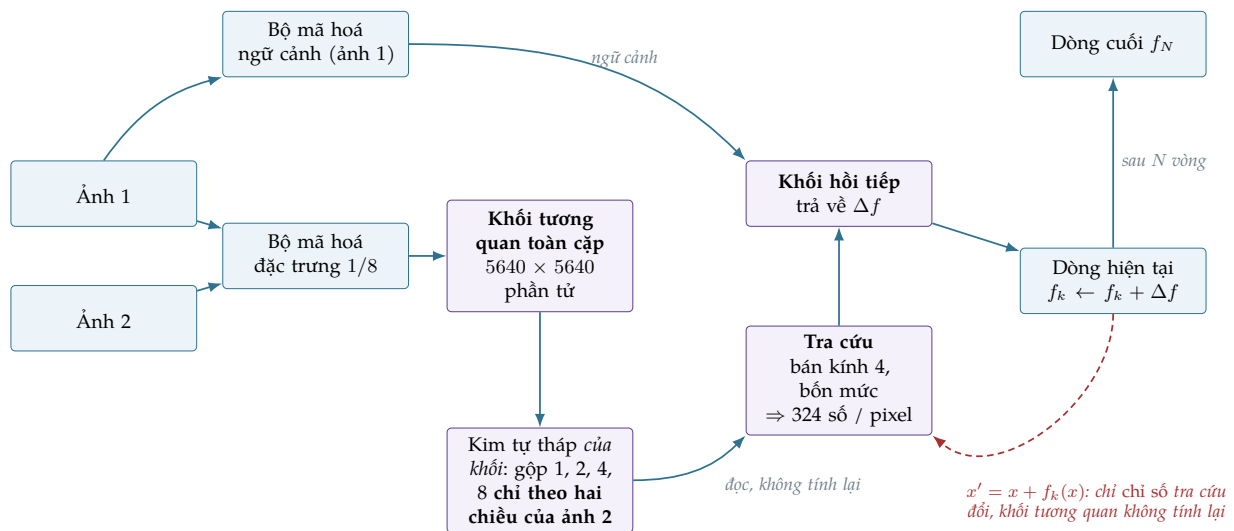
Bảng 4.2. Độ phủ của một phép tra cứu bán kính 4 ở bốn mức của kim tự tháp khối tương quan, quy về pixel của ảnh gốc. Cột cuối cho thấy vì sao không cần thu nhỏ ảnh: mức thô nhất đã phủ quá đủ cho mọi chuyển động thực tế giữa hai khung hình liên tiếp, trong khi lưới của ảnh một vẫn giữ nguyên độ phân giải ở cả bốn mức.

Mức	Hệ số gộp	Ô lưới 1/8	Pixel ảnh gốc	Vai trò
0	1	± 4	± 32	Tinh chỉnh dưới pixel ở các vòng cuối
1	2	± 8	± 64	Chuyển động vừa
2	4	± 16	± 128	Chuyển động nhanh
3	8	± 32	± 256	Xoay mạnh, mất khung, tìm lại từ xa

Với người đã viết bộ tối ưu phi tuyến, khối hồi tiếp kia chính là một *bước tối ưu học được*: nó nhận phần dư quanh nghiệm hiện tại và trả về một bước đi, đúng vai trò của một bước Gauss–Newton, chỉ khác là hướng đi được học chứ không được suy ra từ Jacobi. Và như mọi bộ giải lặp, số vòng lặp là ngân sách bạn tự đặt lúc chạy — huấn luyện với 12 vòng, chạy với 32 vòng nếu rảnh, hoặc 6 vòng nếu gấp.

↔ **trường hợp riêng của** C/15/03 — cùng một trực giác về bộ giải lặp có ngân sách, chỉ khác là ở đó số vòng lặp do bạn đặt còn ở đây nó là siêu tham số lúc suy luận.

Hình 4.6 là sơ đồ của đúng vòng lặp đó.



Hình 4.6. Vòng lặp tra cứu của RAFT. Phần bên trái chạy đúng một lần và không phụ thuộc nghiệm; phần bên phải chạy lặp và rẻ, chỉ đổi *chỉ số* tra cứu chứ không tính lại tương quan. Chi tiết quyết định nằm ở ô kim tự tháp: việc gộp trung bình chỉ áp lên hai chiều của ảnh hai, nên lưới của ảnh một giữ nguyên độ phân giải ở mọi mức. Đó là lý do RAFT xử lý được chuyển động lớn mà không mắc bẫy “vật nhỏ biến mất ở mức thô” của chiến lược thô tới mịn. Số vòng lặp là siêu tham số lúc suy luận, nên nó là núm vặn độ trễ chỉnh được ngay khi chạy.

E. Ba cách cắm dòng quang vào hệ của bạn, xếp theo mức xâm lấn

Trước khi tính giá, cần thấy rằng “dùng dòng quang” không phải một lựa chọn mà là ba lựa chọn rất khác nhau. Xếp theo mức xâm lấn tăng dần:

1. **Dùng làm bộ dự đoán, giữ nguyên ghép cặp thừa.** Chạy dòng quang, lấy vector dịch chuyển tại vị trí các điểm bản đồ đang bám, rồi dùng nó để đặt tâm cửa sổ tìm kiếm cho bộ ghép cặp thừa vốn có. Mọi bài kiểm tra cũ vẫn chạy: kiểm tra tỉ số, kiểm tra sai số chiếu lại, RANSAC. Bạn chỉ đổi *chỗ nhìn*, không đổi *cái tin*.
2. **Dùng làm nguồn tương ứng, bỏ bộ phát hiện.** Lấy thẳng trường dòng làm tập tương ứng, lấy mẫu thừa ra vài trăm điểm theo một tiêu chí nào đó, rồi đưa vào bộ giải. Bạn mất bài kiểm tra tỉ số vì không còn hai ứng viên để so, và bạn phải tự dựng lại một cơ chế loại điểm sai.
3. **Dùng làm phép đo dày, đưa thẳng vào bộ tối ưu.** Đây là con đường của các hệ đầu-cuối ở chương 31. Bạn được nhiều ràng buộc nhất và mất nhiều khả năng chẩn đoán nhất.

Với một hệ đã tinh chỉnh kỹ, lựa chọn thứ nhất gần như luôn là lựa chọn đúng để thử trước, vì lý do rất thực dụng: nó *có thể tắt được*. Nếu dòng quang cho dự đoán tồi, cửa sổ tìm kiếm đặt sai chỗ, số điểm khớp giảm, và bạn thấy ngay bằng đúng các bộ đếm đang có. Không có gì âm thầm cả. Lựa chọn thứ hai và thứ ba đều xoá mất tín hiệu “không còn điểm nội biên nào” — tín hiệu duy nhất mà logic phục hồi của bạn đang dựa vào.

Có một chi tiết dễ bỏ sót ở lựa chọn thứ nhất và nó quyết định lời lỗ. Hệ của bạn đã có một bộ dự đoán rồi: mô hình chuyển động hằng vận tốc, hoặc tốt hơn nữa là tích phân trước của IMU. Trong pha xoay nhanh, IMU chính là thứ dự đoán tốt nhất, vì con quay hồi chuyển đo trực tiếp cái gây ra dịch chuyển ảnh. Vậy dòng quang chỉ có lãi ở đúng những khung hình mà dự đoán từ IMU cũng hỏng: khi có tịnh tiến nhanh ở cự ly gần, khi mất đồng bộ thời gian, hoặc khi độ lệch con quay đang trôi. Đó là một tập khung hình hẹp hơn nhiều so với “mọi khung bị mờ”, và đo được nó chính là nội dung mini project thứ hai của chương.

↔ **đối lập với F/34** — IMU và dòng quang là hai bộ dự đoán cạnh tranh cho cùng một cửa sổ tìm kiếm; chọn cái nào là câu hỏi về việc cái nào hỏng trước.

F. Cái giá, nói bằng số

Bốn khoản, xếp theo mức độ dễ bị bỏ quên.

Bộ nhớ và độ trễ. Điều phải nắm là *cấu trúc* của chi phí, vì nó khác hẳn cấu trúc chi phí của một mạng thông thường. Khối tương quan toàn cặp có số phần tử bằng bình phương số ô lưới, và nó phải nằm nguyên trong bộ nhớ suốt các vòng lặp tra cứu. Hai hệ quả đi kèm. Thứ nhất, chi phí bộ nhớ tăng theo *bình phương* độ phân giải lưới, nên hạ độ phân giải là núm vặn mạnh nhất bạn có. Thứ hai, sau khi khối đã dựng xong thì mỗi vòng lặp chỉ là đọc bộ nhớ, nên bài toán chuyển từ bị chặn bởi phép tính sang bị chặn bởi băng thông. Con số mili-giây và megabyte cụ thể thì phải đo trên chính phần cứng đích, sau khi đã biên dịch bằng trình suy luận của nó — và mini project thứ hai của chương bắt bạn đo đúng con số ấy thay vì ước lượng nó.

↪ **ràng buộc bởi B/11** — bài toán ở đây bị chặn bởi bộ nhớ và băng thông chứ không phải bởi số phép tính, đúng như phân tích roofline ở chương giới hạn tính toán.

Không có tập điểm nội biên. Đây là khoản đắt nhất mà ít ai nói. Pipeline thừa của bạn trả về một tập điểm khớp *kèm nhãn đúng hay sai*, và khi mọi thứ hỏng thì nó trả về con số không — một tín hiệu rõ ràng để kích hoạt logic phục hồi. Dòng quang luôn trả lời. Ở vùng bị che, ở vùng ra khỏi khung, ở mặt gương, nó vẫn cho ra một vector trông rất thuyết phục. Hệ cổ điển hỏng âm ỉ; hệ học được hỏng lạng lẽ.

Nó chỉ thay bước ghép cặp. Dòng quang cho tương ứng hai chiều. Bạn vẫn cần độ sâu và vẫn cần một bộ giải để ra tư thế. Nó không rút ngắn pipeline, nó chỉ đổi một khối.

Chẩn đoán và tính lặp lại. Bạn vừa đổi một khối có thể ablate từng phần lấy một khối chỉ ablate được nguyên cục. Bù lại, đây đúng là chỗ SEA-RAFT có giá trị đặc biệt với bạn: hàm mất mát hỗn hợp Laplace cho ra một độ bất định theo từng pixel, và độ bất định đó chuyển thẳng thành trọng số trong bài toán bình phương tối thiểu. Bài học thiết kế rút ra được, dùng lại được ở chỗ khác: *muốn có trọng số dùng được thì phải huấn luyện cho nó*. Một hàm mất mát đơn phương thức như L_1 không có cách nào diễn đạt “pixel này nằm trên biên che khuất nên câu trả lời có hai đáp án”.

↪ **tiền đề F/34** — đó chính là cầu nối sang bài toán hiệp phương sai học được: trọng số trong đồ thị nhân tử phải đến từ một mô hình có khả năng biểu diễn tính đa phương thức.

4.4 Mô hình nền hình học: DUST3R, MAST3R và VGGT (Geometric Foundation Models)

A. Ý tưởng: đối biểu diễn đầu ra để xoá cả một chuỗi bước

Đây là hướng mới nhất và đáng chú ý nhất trong cả chương, và lý do đáng chú ý không nằm ở độ chính xác mà ở chỗ nó *xoá bỏ* cái gì.

Pipeline hình học cổ điển là một chuỗi dài: phát hiện điểm, mô tả, ghép cặp, ước lượng ma trận cơ bản, phân rã ra tư thế, tam giác hoá, rồi hiệu chỉnh chùm tia. Mỗi bước có giả định riêng, và mỗi bước hỏng theo kiểu riêng. Ước lượng tư thế đặc biệt mong manh: nó cần đủ điểm khớp đúng, nó suy biến khi các điểm gần đồng phẳng hoặc khi chuyển động thuần xoay, và nó luôn cần biết tiêu cự.

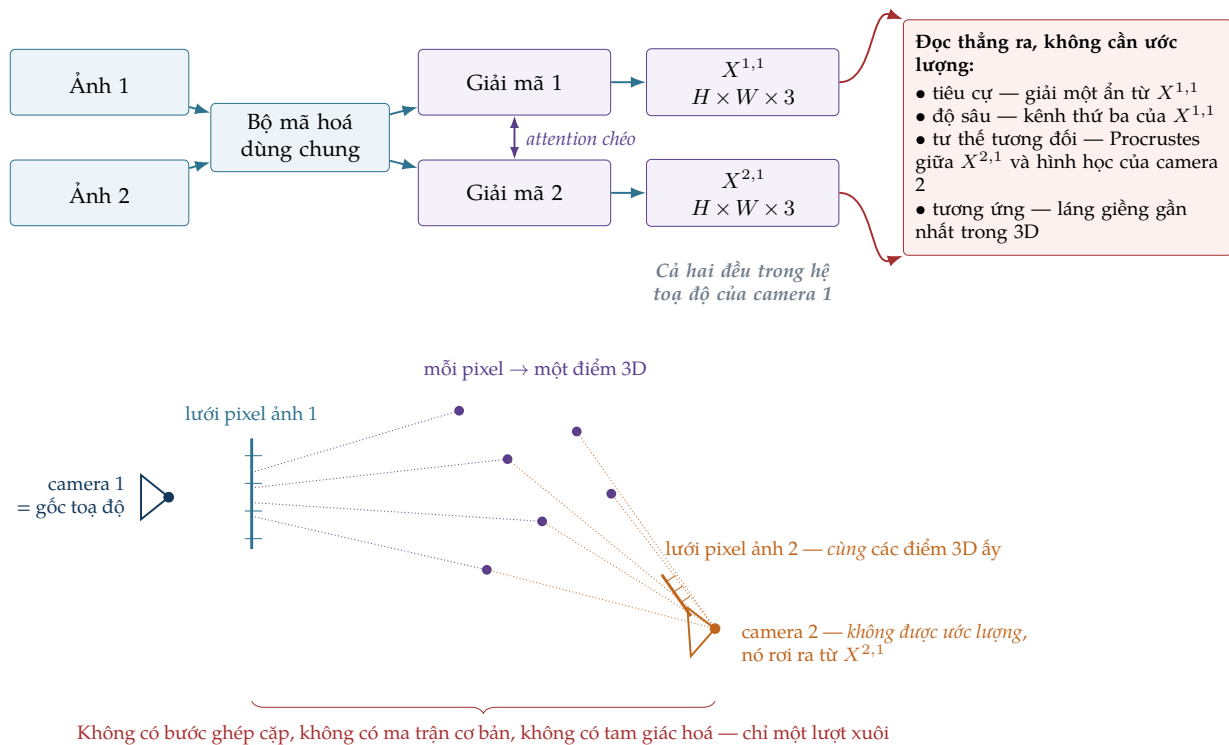
DUST3R [12] đặt câu hỏi ngược: nếu đầu ra của mạng không phải độ sâu mà là *bản đồ điểm* thì cần bao nhiêu bước trong chuỗi trên. Cụ thể, đưa vào hai ảnh chưa hiệu chỉnh, mạng trả về hai tensor $H \times W \times 3$: mỗi pixel của ảnh một được gán một điểm ba chiều, và mỗi pixel của ảnh hai cũng được gán một điểm ba chiều — nhưng *cả hai đều nằm trong hệ toạ độ của camera một*.

Câu “cả hai đều nằm trong hệ toạ độ của camera một” chính là toàn bộ mẹo. Tư thế tương đối không biến mất; nó bị *gói vào* đầu ra. Giả sử tôi biết mỗi pixel của ảnh hai ứng với điểm ba chiều nào trong hệ của camera một. Khi đó tư thế của camera hai chỉ là phép biến đổi cứng khớp bản

đồ điểm ấy với hình học của chính nó. Đó là một bài toán Procrustes hoặc PnP, có nghiệm dạng đóng, không cần RANSAC trên ma trận cơ bản. Tương tự, tiêu cự đọc ra được bằng cách giải một ẩn từ bản đồ điểm của ảnh một; độ sâu là kênh thứ ba; và tương ứng giữa hai ảnh là láng giềng gần nhất trong không gian ba chiều.

Kiến trúc thì đơn giản đến mức đáng ngạc nhiên: một bộ mã hoá *transformer* dùng chung cho hai ảnh, rồi hai bộ giải mã *liên tục nhìn sang nhau* bằng *attention chéo*, rồi hai đầu ra. Hàm mất mát là hồi quy trong không gian ba chiều, sau khi chuẩn hoá cả dự đoán lẫn nhãn về cùng một thang, và có trọng số theo một độ tin cậy do chính mạng dự đoán. Chuẩn hoá về cùng thang là chỗ tỉ lệ tuyệt đối bị bỏ đi, nên đầu ra gốc của DUS3R không có đơn vị mét.

Hình 4.7 vẽ cả cơ chế lẫn ý nghĩa hình học.



Hình 4.7. Bản đồ điểm của DUS3R: cơ chế ở trên, ý nghĩa hình học ở dưới. Mạng gán thẳng cho mỗi pixel của *cả hai* ảnh một điểm ba chiều trong *cùng* hệ toạ độ của camera một. Vì đã ở chung một hệ, mọi đại lượng mà pipeline cổ điển phải ước lượng qua nhiều bước đều trở thành phép đọc: tiêu cự, độ sâu, tư thế tương đối và tương ứng. Bài học thiết kế: chọn đúng biểu diễn đầu ra xoá được nhiều bước hơn là chọn đúng kiến trúc. Cái không xoá được là tỉ lệ tuyệt đối — hàm mất mát chuẩn hoá thang trước khi so, nên đầu ra không có đơn vị mét.

B. MAST3R, MAST3R-SLAM, VGGT, VGGT-SLAM

I MẠCH 2 — từ hai ảnh tới một hệ SLAM

1 Vấn đề

DUS3R cho hình học tốt nhưng tương ứng của nó chỉ là láng giềng gần nhất trong không gian ba chiều, nên không đủ chính xác để làm bộ ghép cặp.

Giải pháp

MAST3R gắn thêm một đầu ra mô tả đặc trưng dày lên chính kiến trúc đó, huấn luyện bằng một hàm mất mát ghép cặp, cộng một thủ tục ghép tương hỗ nhanh để tránh so sánh mọi cặp pixel [92].

Hệ quả

nó trở thành một bộ ghép cặp *được neo trong ba chiều*, nên nó chịu được thay đổi góc nhìn cực đoan — đúng trường hợp mà mọi bộ mô tả hai chiều đều chết.

- Giá phải trả** vẫn là một mô hình lớn chạy trên *từng cặp* ảnh, tức chậm hơn một bộ ghép cặp thừa nhiều bậc, và chi phí nhân lên theo số cặp.
- 2 Vấn đề** Có một bộ ghép cặp mạnh vẫn chưa phải một hệ SLAM: thiếu quản lý khung khoá, thiếu tối ưu toàn cục, thiếu đóng vòng.
- Giải pháp** MAST3R-SLAM dựng một hệ SLAM dày, chạy trực tuyến, đặt thẳng trên tiên nghiệm hai ảnh đó, và chạy được cả khi camera chưa hiệu chỉnh [13].
- Hệ quả** điểm thiết kế đáng học: nó tối ưu *trong không gian bản đồ điểm* chứ không trong không gian sai số chiếu lại, vì bản đồ điểm mới là thứ mạng cho ra.
- Giả định** phần cứng có GPU rời đủ mạnh. Đây là công trình rất mới; tôi nêu cơ chế mà không nêu con số hiệu năng.
- 3 Vấn đề** Suy luận theo từng cặp có chi phí bậc hai theo số ảnh, và phải có một bước căn chỉnh toàn cục riêng để ghép các cặp lại.
- Giải pháp** VGGT bỏ hẳn cấu trúc theo cặp: một *transformer* lớn nhận cả nhóm ảnh cùng lúc và trả về trong một lượt xuôi cả nội tham số, ngoại tham số, bản đồ độ sâu, bản đồ điểm và vết điểm ba chiều [93].
- Giải pháp** cơ chế cốt lõi là *xen kẽ hai loại attention*: một lượt attention trong phạm vi từng ảnh, rồi một lượt attention trên toàn bộ token của mọi ảnh, lặp lại nhiều lần. Lượt trong ảnh giữ chi tiết, lượt toàn cục làm cho các ảnh biết về nhau.
- Hệ quả** không còn bước căn chỉnh toàn cục nào cả; hình học ra thẳng, trong hệ toạ độ của camera đầu tiên.
- Giá phải trả** attention toàn cục trên mọi token của mọi khung là chỗ bộ nhớ nổ, và nó đặt một trần cứng lên số khung hình — mục sau tính con số đó.
- 4 Vấn đề** Nếu ghép các mảnh bản đồ do VGGT sinh ra bằng một phép biến đổi tương tự thì kết quả sai, chứ không chỉ là chưa tối ưu.
- Giải pháp** VGGT-SLAM nhận ra rằng với camera chưa hiệu chỉnh và cảnh gần phẳng, hai mảnh bản đồ liên hệ với nhau bằng một phép biến đổi xạ ảnh tổng quát chứ không phải phép tương tự; nên nó tối ưu trên nhóm $SL(4)$ mười lăm bậc tự do [94].
- Hệ quả** đây là bài học thiết kế đắt giá nhất của cả mục: *lớp bất định của đầu ra frontend quyết định đa tạp mà backend phải tối ưu trên đó*. Chọn nhầm đa tạp là sai mô hình, không phải chậm hội tụ.
- Giả định** cảnh và cấu hình camera thật sự rơi vào lớp bất định xạ ảnh đó. Công trình rất mới, tôi nêu ý tưởng chứ không nêu kết quả.

C. Hai kiểu trần số khung, và vì sao cả hai đều là trần

Đây là phần quyết định các phương pháp này cắm vào đâu được trong một hệ nhúng. Điểm mấu chốt không phải một con số megabyte — con số ấy đổi theo bản cài đặt và theo phần cứng, nên nêu nó ra ở đây là gây hiểu nhầm — mà là *dạng* của quan hệ giữa chi phí và số khung hình. Hai nhánh có hai dạng khác nhau, và chúng dẫn tới hai kiểu trần khác nhau.

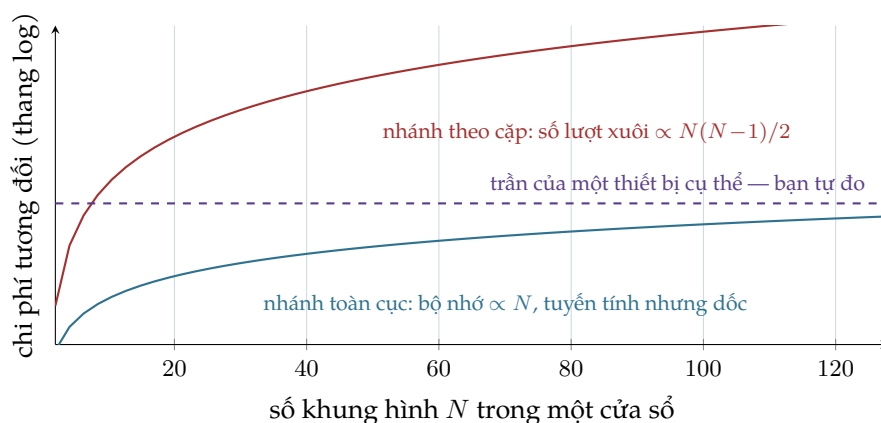
Nhánh theo cặp (DUST3R, MAST3R). Với N ảnh, đồ thị liên kết đầy đủ có $N(N - 1)/2$ cặp, và *mỗi cặp là một lượt xuôi riêng*. Đây là quan hệ bậc hai theo số khung, và nó là thứ chặn bạn chứ không phải bộ nhớ: một cửa sổ vài chục khung đã kéo theo hàng trăm lượt xuôi cho một lần cập nhật. Bộ nhớ thì tăng tuyến tính và hiền hơn nhiều, vì mỗi khung chỉ giữ một bản đồ điểm cỡ ba kênh trên mỗi pixel.

Nhánh toàn cục (VGGT). Ở đây chi phí đi theo số *token*, và số token là đại lượng đáng đếm vì nó suy ra được từ kiến trúc: một ảnh 518×518 chia thành mảnh 14×14 cho $37 \times 37 = 1369$ token

mỗi khung, nên N khung cho $1369N$ token. Attention *toàn cục* chạy trên toàn bộ chỗ token ấy, và mọi token phải giữ khoá cùng giá trị ở mọi tầng của thân *transformer*. Quan hệ vì thế tuyến tính theo N — nhưng với một hệ số nhân rất lớn, vì hệ số ấy là số tầng nhân chiều ẩn. Tuyến tính với hệ số lớn vẫn chậm trần rất sớm.

Cách đọc đúng hai đoạn trên: đừng nhớ megabyte, hãy nhớ *số mũ*. Nhánh theo cặp là bậc hai theo số khung; nhánh toàn cục là bậc một nhưng dốc. Muốn biết trần cụ thể của mình nằm ở đâu thì chạy một lượt xuôi với số khung tăng dần trên đúng phần cứng đích rồi đọc bộ nhớ đỉnh — một buổi chiều, và kết quả đúng cho máy của bạn thay vì đúng cho máy của người viết bài báo.

Hình 4.8 vẽ lại hai dạng ấy. Kết luận kiến trúc rút ra được ngay và không phụ thuộc con số: cả hai nhánh đều có một trần cứng theo số khung, nên mọi hệ SLAM dựng trên nhóm này đều buộc phải chia bản đồ thành các mảnh nhỏ rồi ghép lại — và bước ghép ấy chính là chỗ VGGT-SLAM phát hiện ra rằng phép biến đổi tương tự là sai mô hình.



Hình 4.8. Hai kiểu trần số khung, vẽ trên thang log. Trục đứng cố ý không có vạch số: hình này nói về *số mũ*, không về megabyte, vì con số tuyệt đối đổi theo bản cài đặt và theo phần cứng còn số mũ thì không. Nhánh theo cặp bị chặn bởi số lượt xuôi tăng bậc hai — qua vài chục khung thì số cặp đã lên hàng nghìn, và đó là thứ chặn bạn trước cả bộ nhớ. Nhánh attention toàn cục tăng tuyến tính, nhưng hệ số nhân là số tầng nhân chiều ẩn, nên nó vẫn chạm trần sớm. Đường ngang là trần của một thiết bị cụ thể; vị trí thật của nó là thứ bạn đo bằng cách tăng dần số khung trên đúng phần cứng đích rồi đọc bộ nhớ đỉnh. Hệ quả kiến trúc không phụ thuộc con số: mọi hệ SLAM dựng trên nhóm này đều buộc phải chia bản đồ thành mảnh nhỏ rồi ghép lại.

Bảng 4.3. Năm công trình trong nhánh mô hình nền hình học. Cột cuối là cột quyết định với một hệ nhúng thời gian thực, và nó ghi *loại* ràng buộc chứ không ghi con số: mọi nhận định ở đây lấy từ mô tả phương pháp, không từ số đo mà tôi tự chạy, nên chúng nói được cái gì chặn bạn nhưng không nói được chặn ở mức nào.

Công trình	Cơ chế cốt lõi	Thêm được gì	Hỏng khi nào / cái giá
DUST3R	Hồi quy bản đồ điểm cho cặp ảnh, cả hai trong hệ của camera một	Bỏ hẳn ghép cặp, ma trận cơ bản và tam giác hoá	Không có đơn vị mét; nhiều ảnh thì chi phí theo cặp tăng bậc hai
MASt3R	Thêm đầu ra mô tả dày và ghép tương hỗ nhanh lên DUST3R	Ghép cặp neo trong ba chiều, chịu được góc nhìn cực đoan	Một lượt xuôi mô hình lớn cho <i>mỗi cặp</i> ; chi phí nhân theo số cặp
MASt3R-SLAM	Hệ SLAM dày trực tuyến, tối ưu trong không gian bản đồ điểm	Chạy được cả khi camera chưa hiệu chỉnh	Cần GPU rời mạnh; rất mới, ít bằng chứng ngoài phòng thí nghiệm
VGGT	Một <i>transformer</i> nhận nhiều ảnh, xen kẽ attention trong ảnh và toàn cục	Bỏ luôn bước căn chỉnh toàn cục; ra thẳng tư thế, độ sâu, bản đồ điểm	Bộ nhớ attention toàn cục đặt trần cứng lên số khung
VGGT-SLAM	Ghép các mảnh bản đồ của VGGT trên nhóm xạ ảnh SL(4)	Chỉ ra rằng phép biến đổi tương tự là <i>sai mô hình</i> khi camera chưa hiệu chỉnh	Chỉ đúng khi lớp bất định thật sự là xạ ảnh; rất mới

D. Chỗ nhánh này chạm vào nỗi đau đóng vòng

Có một chỗ mà nhánh này đáng để bạn theo dõi nghiêm túc, và nó không phải chỗ được quảng cáo nhiều nhất.

Chương 29 tách đóng vòng thành bốn tầng và chỉ ra rằng tầng kiểm chứng hình học mới là chỗ hay giết vòng nhất, đặc biệt khi hai lần đi qua cùng một chỗ theo hai hướng ngược nhau. Đó chính xác là trường hợp mà một bộ mô tả hai chiều không cứu được: cùng một góc tường nhìn từ hai phía cho hai mảnh ảnh không có gì chung. MASt3R ghép cặp trong không gian ba chiều, nên nó không phụ thuộc vào việc hai mảnh ảnh trông giống nhau; nó phụ thuộc vào việc hai bản đồ điểm mô tả cùng một hình học.

→ **hệ quả của F/29** — đây là một lối thoát khá dễ cho đúng cái nút thắt mà chương 29 đo được ở tầng kiểm chứng hình học.

Cái giá thì rõ, và nó là lập luận về *tần suất* chứ không về mili-giây: đóng vòng chạy không thường xuyên và không nằm trên đường tới hạn thời gian thực, nên một mô hình đắt gấp nhiều lần ngân sách một khung hình vẫn dùng được ở đó, trong khi cùng mô hình ấy đặt vào *frontend* thì không. Nói cách khác, nếu có một chỗ trong hệ của bạn mà nhánh mô hình nền hình học đáng thử trước tiên, thì đó là tầng kiểm chứng ứng viên vòng, không phải *frontend*.

E. Tối ưu trong không gian bản đồ điểm: được gì và mất gì

MASt3R-SLAM tối ưu trong không gian bản đồ điểm chứ không trong không gian sai số chiều lại. Chi tiết ấy nghe như một lựa chọn cài đặt, nhưng nó đổi bản chất bài toán, và người đã viết bộ hiệu chỉnh chùm tia sẽ thấy ngay vì sao.

Sai số chiều lại có một tính chất mà ta hay quên vì nó quá quen: nó là sai số *trong không gian đo*. Nhiều của bộ phát hiện điểm gần với nhiều đẳng hướng vài phần mười pixel, nên bình phương tối thiểu trên sai số chiều lại xấp xỉ đúng ước lượng hợp lý cực đại. Chuyển sang đo sai lệch trong không gian ba chiều phá vỡ điều đó. Một sai lệch nửa pixel ở một điểm cách ba mét tương ứng

vài milimet trong không gian; cùng nửa pixel ấy ở ba mươi mét tương ứng vài chục centimet. Đo bằng mét là ngầm gán cho điểm xa một trọng số lớn hơn hàng trăm lần so với điều mà phép đo thật sự đáng được hưởng.

Vậy tại sao vẫn làm? Vì mạng không cho bạn phép đo trong không gian ảnh; nó cho bạn bản đồ điểm. Ép nó quay về không gian ảnh đòi hỏi biết nội tham số và đòi hỏi tin vào đó, mà một trong những điểm bán hàng của cả nhánh này lại là không cần hiệu chỉnh. Đây là một đánh đổi thật, không phải một sơ suất — nhưng nó giải thích một hiện tượng bạn sẽ thấy khi đọc kết quả của nhánh này: chất lượng dựng lại nhìn rất đẹp trong khi độ chính xác quỹ đạo thì chưa bằng một hệ cổ điển đã tinh chỉnh kỹ.

F. Lỗi hổng lớn nhất của nhánh này: không có chỗ cho IMU

Có một điều mà không bảng kết quả nào nói ra, và với bạn nó có thể là điều quan trọng nhất trong cả mục: *cả nhánh này thuần thị giác*. DUST3R, MAST3R và VGGT đều nhận vào ảnh và chỉ ảnh.

Hệ quả không phải là “thiếu một tính năng”. Nó là một mâu thuẫn ở gốc. Nhánh này thừa hưởng trọn vẹn tính không xác định được của tỉ lệ — đầu ra của chúng không có đơn vị mét, và mục trước đã chỉ ra rằng không lượng dữ liệu nào chứa được điều đó. Đồng thời chúng không có chỗ nào để nhận cảm biến duy nhất chứa được nó. Một hệ SLAM cổ điển chỉ cần thêm một gia tốc kế là mất hẳn bậc tự do tỉ lệ; một mô hình nền hình học không có cơ chế tương đương.

Điều đó dẫn tới ba việc phải làm nếu bạn muốn ghép nhánh này vào hệ của mình.

- **Tỉ lệ phải được cố định bên ngoài mạng.** Cách sạch nhất là để mạng lo hình dạng, còn tỉ lệ do phần cổ điển của bạn quyết, giống như phân công giữa tiên nghiệm và phép đo ở mục trước.
- **Không có bộ dự đoán chuyển động.** Mạng xử lý một nhóm ảnh như một tập, không như một chuỗi có động học. Nó không biết rằng khung sau phải gần khung trước, trong khi tích phân trước của IMU biết điều đó rất chắc.
- **Không có mốc thời gian.** Không có chỗ khai báo độ trễ giữa ảnh và IMU, và không có chỗ khai báo màn trập cuộn. Với chuyển động nhanh, cả hai thứ đó đều là nguồn sai số bậc nhất.

Nói cách khác, ba lỗi hổng ấy đúng là ba thứ mà một hệ thị giác-quán tính đã giải xong từ lâu. Đây là lý do mạnh nhất để đọc nhánh này như một *nguồn ý tưởng* và như một công cụ cho các tác vụ ngoài đường tới hạn, chứ không như một ứng viên thay thế.

↪ **tiền đề F/34** — mọi thứ chương 34 nói về trọng số IMU đều giả định có một chỗ trong bài toán để đặt phép đo IMU vào; nhánh này chưa có chỗ đó.

G. Đọc bằng chứng của nhánh này cho đúng

Ba điều mà các bảng kết quả trong nhánh này thường không nói ra, và cả ba đều quan trọng với bạn.

- **Chất lượng ảnh dựng lại không phải độ chính xác quỹ đạo.** Hai đại lượng này bị trình bày cạnh nhau đến mức người đọc tự động gộp chúng. Một bản đồ dày đẹp mắt vẫn có thể đi kèm một quỹ đạo trôi, vì phần lớn sai số quỹ đạo là sai số tần số thấp mà mắt không thấy trên ảnh dựng lại.
- **Phép căn chỉnh trước khi đo che mất đúng thứ bạn quan tâm.** Với hệ đơn mắt, quỹ đạo phải được căn chỉnh cả tỉ lệ trước khi tính sai số, và phép căn chỉnh đó hấp thụ trọn vẹn phần trôi tỉ lệ. Hệ stereo-inertial của bạn không được hưởng ưu ái ấy, nên hai con số không so trực tiếp được với nhau.

- **Điều kiện chạy khác hẳn điều kiện của bạn.** Phần lớn kết quả đến từ chuỗi ngắn, cảnh trong nhà, và máy để bàn có GPU rời. Ba khác biệt đó cộng lại đủ để đảo ngược một kết luận, và không cái nào trong ba xuất hiện dưới dạng một cột trong bảng.

Cách đọc thực dụng: khi gặp một hệ trong nhánh này, hãy tìm ba con số trước khi tìm bảng chính — độ dài chuỗi tính bằng khung hình, phần cứng, và quỹ đạo được căn chỉnh theo phép biến đổi nào. Nếu ba con số ấy không có trong bài, phần còn lại chưa dùng để quyết định được.

→ cùng nguyên lý với E/23 — đây là cùng danh sách câu hỏi bạn dùng để đọc mọi bảng kết quả, chỉ đổi ba mục cho đúng đặc thù của nhánh này.

4.5 Cổng đánh giá: khi nào nó cứu, khi nào nó phá (The Evaluation Gate)

A. Bốn chỗ nó thật sự cứu

Gộp lại từ ba mục trên, danh sách các chỗ mà một tiên nghiệm học được thêm *thông tin* chứ không chỉ thêm số:

- **Vùng thị sai đã cạn.** Xa hơn khoảng cách mà độ chênh lệch tụt xuống dưới sàn nhiều, stereo của bạn không còn nói gì. Ở đó một phỏng đoán có sai số vừa phải vẫn tốt hơn một phép đo mà sai số đã lớn hơn nhiều lần — và điểm cắt giữa hai đường cong ấy là thứ bạn tự đo, không phải thứ chép từ bảng của ai.
- **Vùng không có vân.** Không có ghép cặp thì không có phép đo ở bất kỳ khoảng cách nào. Đây là chỗ duy nhất mà tiên nghiệm không cạnh tranh với cái gì cả.
- **Chuyển động nhanh và ảnh mờ.** Dòng quang dày cho tương ứng ở đúng dải mà bộ phát hiện góc đã tắt. Đây là chỗ có bằng chứng cơ chế mạnh nhất trong cả chương.
- **Lắp lỗ cho bản đồ dày, ở hạ nguồn.** Nếu sản phẩm cuối là bản đồ chiếm dụng để điều hướng chứ không phải quỹ đạo, thì tiên nghiệm dày là đúng công cụ — miễn là nó không quay ngược vào đồ thị tư thế.

Có một quan sát về chính tập dữ liệu của bạn đáng ghi ra, vì nó thu hẹp cổng thứ nhất đi nhiều. Các chuỗi phòng Vicon của EuRoC diễn ra trong một không gian cỡ tám mét, nên gần như không có điểm nào rơi vào dải thị sai cạn. Các chuỗi nhà máy thì rộng cỡ vài chục mét, nên ở đó cổng mới thật sự hé mở. Nghĩa là: nếu bạn đo lợi ích của độ sâu học được trên các chuỗi phòng Vicon rồi kết luận “không có tác dụng”, kết luận ấy đúng nhưng nó chỉ đúng cho những chuỗi đó. Ngược lại, một kết quả tốt trên chuỗi nhà máy cũng chưa chuyển được sang cảnh ngoài trời rộng hơn nữa. Đây là lý do mini project của chương yêu cầu hai chuỗi có quy mô không gian khác nhau, chứ không phải hai chuỗi có độ khó khác nhau.

Còn khởi tạo đơn ảnh, thứ được nhắc nhiều nhất trong tài liệu, gần như không liên quan tới bạn. Hệ stereo có đường cơ sở nên nó khởi tạo được ngay cả khi đứng yên. Phải nói thẳng điều này vì nó là nguồn lãng phí thời gian phổ biến: rất nhiều lợi ích được quảng cáo là lợi ích *của hệ đơn mắt*, và chúng biến mất khi bạn có hai camera.

B. Bốn chỗ nó phá — và phép tính chứng minh chỗ thứ nhất

Thứ nhất: nó đưa sai lệch có hệ thống vào bộ tối ưu, và đặt trọng số nhỏ không cứu được.

Đây là chỗ đáng tính bằng số nhất trong cả chương, vì kết quả đi ngược trực giác. Xét việc ước lượng một đại lượng vô hướng, chẳng hạn khoảng cách tới một mảng tường. Bạn có hai nguồn.

- 20 phép đo stereo thật, mỗi phép đo độ lệch chuẩn $\sigma_m = 0,10$ m. Thông tin đóng góp: $20/0,10^2 = 2000$.
- 5000 pixel độ sâu học được, bạn khai báo độ lệch chuẩn $\sigma_p = 0,30$ m — đã cẩn thận đặt gấp ba lần phép đo thật. Thông tin đóng góp: $5000/0,30^2 \approx 55600$.

Tiên nghiệm áp đảo phép đo gấp gần ba mươi lần. Giả sử mạng lệch đều 0,25 m — cả mảng tường bị đẩy ra xa cùng một khoảng. Nghiệm bình phương tối thiểu có trọng số dịch đi

$$\frac{55600}{55600 + 2000} \times 0,25 \approx 0,241 \text{ m.}$$

Bạn vừa dịch nghiệm 24 cm bằng một nguồn mà bạn tưởng đã hạ trọng số.

Bây giờ thử hạ trọng số thật mạnh, khai báo $\sigma_p = 3,0$ m, tức gấp mười lần. Thông tin còn $5000/9 \approx 556$, và độ dịch còn $556/2556 \times 0,25 \approx 5,4$ cm. Muốn tiên nghiệm thật sự không đáng kể, bạn cần $5000/\sigma_p^2 \ll 2000$, tức $\sigma_p \gg 1,6$ m — một mức bất định mà ở đó tiên nghiệm chẳng còn nói gì nữa.

Vì sao lại thế? Vì bạn đang đếm sai số mẫu độc lập. Nếu 5000 pixel đó có sai số *độc lập* thì trung bình của chúng có sai số chuẩn $0,30/\sqrt{5000} \approx 0,004$ m, tức chính xác hơn phép đo stereo cả bậc độ lớn — và điều đó rõ ràng vô lý. Sự thật là sai số của chúng *tương quan gần như hoàn toàn*: cả mảng tường lệch cùng một hướng vì mạng hiểu sai cả mảng tường. Số mẫu độc lập hiệu dụng gần với 1 hơn là 5000. Một ma trận thông tin chéo giả định độc lập, nên nó thổi phồng lượng thông tin lên cỡ ba bậc độ lớn.

Suy ra cách chữa đúng, và nó không phải hạ trọng số: hãy **mô hình hoá phần tương quan bằng một biến phụ**. Thêm vào bài toán một cặp tham số tỉ lệ và độ dịch cho mỗi khung hình, để thành phần sai số chung bị hấp thụ vào tham số đó thay vì đẩy vào tư thế. Đây đúng là cách bạn đã xử lý độ lệch của con quay hồi chuyển — không hạ trọng số phép đo con quay, mà thêm một biến độ lệch vào trạng thái.

→ cùng nguyên lý với F/34 — thêm biến phụ để hấp thụ thành phần sai số chung chính là cách xử lý độ lệch cảm biến trong đồ thị nhân tử; ở đây đối tượng đối từ con quay sang mạng độ sâu.

Và đó cũng chính là bài toán mà Align3R đang tấn công ở phía mô hình.

Thứ hai: nó phá tính nhất quán đa khung. Với họ MiDaS, cặp tỉ lệ và độ dịch là của riêng từng khung hình. Một bức tường đứng yên vì thế thở phập phồng qua các khung. Bộ tối ưu của bạn tin rằng cảnh cứng, nên nó không có chỗ nào để nhét nhịp thở đó, ngoài tư thế và vận tốc. Chuỗi hậu quả rất cụ thể: tỉ lệ dao động ở tần số khung hình, vận tốc thị giác nhiễu theo, và bộ ước lượng độ lệch gia tốc kế — vốn là thứ hấp thụ mọi thứ không giải thích được giữa thị giác và IMU — bắt đầu trôi. Nói cách khác, một tiên nghiệm độ sâu đặt sai chỗ hiện ra dưới dạng một bệnh của IMU. Đó là kiểu triệu chứng tốn nhiều ngày nhất để lần ra.

Thứ ba: nó làm hỏng chẩn đoán. Ba dạng, tăng dần độ khó chịu.

- **Mất ranh giới ablation.** Một mạng là một khối. Bạn không tắt được nửa nó để xem nửa kia đóng góp gì.
- **Hỏng lặng lẽ.** Không có tín hiệu nào tương đương với “không còn điểm nội biên nào”. Mạng luôn trả về một bản đồ độ sâu trông hợp lý, kể cả trên mặt gương, kể cả khi ống kính bị bẩn.
- **Mất tính lặp lại.** Một mô hình dựa trên khuếch tán như Marigold cho hai kết quả khác nhau ở hai lần chạy trên cùng một ảnh, vì nó bắt đầu từ nhiễu ngẫu nhiên. Giả sử bạn đã bỏ công làm cho hệ của mình tắt định: ghim thứ tự duyệt container, xoá tranh chấp luồng. Cắm một khối như thế vào là tự tay dựng lại đúng cái sàn nhiều bạn vừa đẹp. Và lần này không có bug nào để sửa.

Thứ tư: nó lệch miền một cách âm thầm. Metric3D cho độ sâu tuyến tính theo tiêu cự bạn khai báo, nên một sai số hiệu chỉnh 3% thành sai số độ sâu 3% đều khắp. Các mô hình huấn luyện

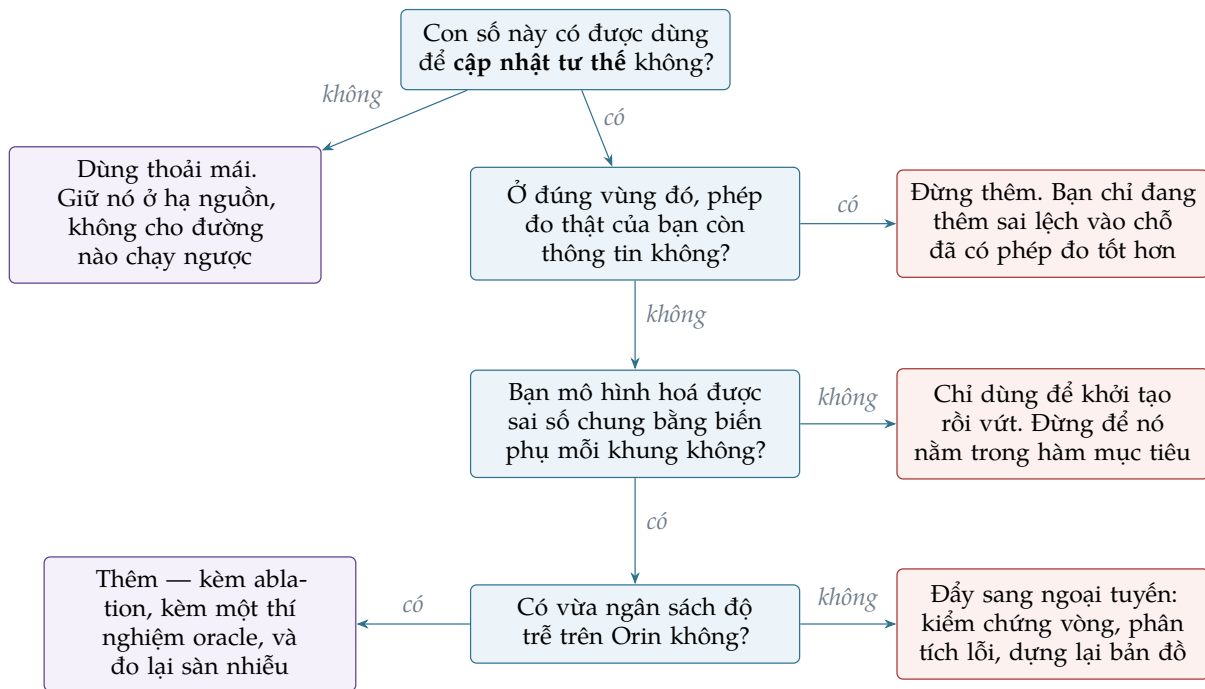
chủ yếu trên cảnh đường phố và cảnh trong nhà thông thường sẽ gặp gì ở một hành lang tối của TUM-VI, ở một nhà máy có kết cấu kim loại lặp lại, hay qua một ống kính mắt cá? Không ai biết trước, và triệu chứng duy nhất là quỹ đạo hơi tệ đi.

Bảng 4.4. Bốn chỗ cứu và bốn chỗ phá, đặt cạnh nhau. Cột cuối là điều bạn phải làm được *trước* khi tích hợp; nếu chưa làm được thì cổng chưa mở.

	Tình huống	Cơ chế	Phép kiểm tra trước khi tích hợp
Cứu	Thị sai đã cạn	ΔZ tăng theo Z^2 còn sai số tương đối của mạng thì không	Vẽ biểu đồ tỉ lệ điểm bản đồ theo khoảng cách; đo phần nằm ngoài ngưỡng một pixel độ chênh lệch
Cứu	Vùng không có vân	Không có ghép cặp thì không có phép đo để cạnh tranh	Đếm số khung hình có số điểm khớp dưới ngưỡng phục hồi
Cứu	Chuyển động nhanh, ảnh mờ	Ghép cặp thừa là chuỗi cổng nhân nhau, dòng quang không có cổng nào	Vẽ tỉ lệ điểm tìm lại được theo tốc độ góc lấy từ IMU
Cứu	Bản đồ dày ở hạ nguồn	Không phản hồi vào đồ thị tư thế nên không đưa sai lệch vào	Kiểm tra bằng mã rằng không có đường dữ liệu nào chạy ngược
Phá	Sai lệch có hệ thống	Sai số tương quan bị ma trận thông tin chéo đếm thành hàng nghìn phiếu độc lập	Tính tỉ số thông tin giữa hai nguồn trước khi viết dòng code nào
Phá	Mất nhất quán đa khung	Tỉ lệ và độ dịch riêng từng khung bị hấp thụ vào tư thế và vận tốc	Vẽ tỉ lệ ước lượng theo thời gian; nếu nó dao động ở tần số khung hình thì hỏng
Phá	Mất chẩn đoán	Không ablate được từng phần; hỏng không phát tín hiệu; đầu ra có thể ngẫu nhiên	Chạy cùng cấu hình mười lần và so sánh nhiều trước và sau khi thêm
Phá	Lệch miền	Đầu ra tuyến tính theo tiêu cự khai báo; miền huấn luyện không phủ cảnh của bạn	Chạy trên chuỗi khó nhất của bạn và so với chân trị độ sâu, không chỉ so quỹ đạo

C. Cây quyết định

Gộp tất cả lại thành một thứ dùng được trong ba mươi giây. Hình 4.9 là cây quyết định cho câu hỏi “có nên cấm tiên nghiệm học được này vào không”.



Hình 4.9. Cây quyết định cho việc cấm một tiên nghiệm dày học được vào hệ SLAM. Câu hỏi đầu tiên không phải “mô hình nào tốt nhất” mà “con số này có chạy ngược vào tư thế không”: mọi rắc rối trong chương này chỉ xuất hiện khi câu trả lời là có. Hai nhánh giữa lần lượt lọc theo thông tin (đừng cạnh tranh với phép đo tốt hơn) và theo khả năng mô hình hoá sai số tương quan. Nhánh cuối là ngân sách, và lời đáp “không” ở đó không phải lời từ chối — nó chỉ đổi chỗ dùng sang các tác vụ ngoại tuyến.

D. Bốn bài học thiết kế mang đi được, kể cả khi bạn không dùng mô hình nào

Phần này trả lời câu hỏi cuối trong bốn câu mà mọi phương pháp trong Phần F phải trả lời: nếu không dùng nguyên phương pháp thì học được gì từ thiết kế của nó. Bốn bài học dưới đây dùng lại được ngay trong mã C++ của bạn.

- **Tách phần tính một lần khỏi phần lặp** — từ *RAFT*. Bất cứ khi nào một vòng lặp tối ưu tính lại cùng một đại lượng ở mọi vòng, hãy hỏi xem đại lượng ấy có phụ thuộc nghiệm hiện tại không. Nếu không, đưa nó ra ngoài và biến vòng lặp thành phép tra cứu. Hệ quả kèm theo cũng quý: số vòng lặp trở thành núm vận độ trễ chính được lúc chạy.
- **Làm thô đúng trục** — cũng từ *RAFT*. Mọi sơ đồ đa tỉ lệ đều phải chọn làm thô cái gì. Làm thô sai trục thì bạn mất đúng thứ mình đang tìm, và mọi mức mịn hơn chỉ tinh chỉnh quanh chỗ sai. Câu hỏi cần đặt mỗi lần thiết kế một kim tự tháp: *trục nào ở đây không được phép mất độ phân giải*.
- **Ẩn số mà dữ liệu không nói tới thì phải ghim, không phải học** — từ *Metric3D*. Đưa nó ra khỏi phần học, chuẩn hoá ở đầu vào, khôi phục ở đầu ra. Đây là cổ định gauge, và bạn đã dùng nó mỗi lần cố định khung khoá đầu.
- **Muốn có trọng số dùng được thì phải huấn luyện cho nó** — từ *SEA-RAFT*. Một hàm mất mát đơn phương thức không thể diễn đạt “câu trả lời ở đây có hai đáp án”. Nếu bạn cần một hiệp phương sai để đưa vào đồ thị nhân tử thì phải chọn hàm mất mát cho phép biểu diễn nó, ngay từ lúc huấn luyện.
- **Lớp bất định của đầu ra quyết định đa tạp của bộ tối ưu** — từ *VGGT-SLAM*. Trước khi viết bộ ghép mảnh bản đồ, hãy hỏi hai mảnh ấy thật sự liên hệ với nhau bằng nhóm biến đổi nào. Chọn nhầm nhóm là sai mô hình, và sai mô hình không hiện ra dưới dạng hội tụ chậm — nó hiện ra dưới dạng một nghiệm sai mà bộ tối ưu rất tự tin.

Bài học thứ sáu không thuộc riêng công trình nào mà thuộc cả chương, và nó là bài học đắt nhất: *đổi biểu diễn đầu ra xoá được nhiều bước hơn là đổi kiến trúc*. DUS3R không giỏi hơn pipeline cổ điển ở bất kỳ bước riêng lẻ nào; nó chỉ chọn một đầu ra làm cho phần lớn các bước trở nên không cần thiết.

Bảng 4.5. Ba họ, đọc theo *cấu trúc* chi phí thay vì theo con số. Bảng này cố ý không có cột mili-giây hay cột megabyte: một con số như thế chỉ đúng cho một bản cài đặt, một trình biên dịch suy luận và một mức tải cụ thể, nên chép nó vào tài liệu học là dạy sai. Cột giữa nói chi phí *tỉ lệ với cái gì* — đó mới là thứ không đổi — và cột phải nói phép đo bạn phải chạy để biến nó thành một con số dùng được.

Họ	Chi phí tỉ lệ với cái gì	Chỗ dùng được	Phép đo phải chạy
Độ sâu đơn ảnh, bản nhỏ	Một lượt xuôi mỗi lần gọi; gọi được ở tần số thấp hơn tần số khung hình	Có thể vừa ngân sách nếu bạn chấp nhận cập nhật thưa hơn ảnh	Biên dịch bằng trình suy luận của thiết bị, đo độ trễ khi hệ đang chạy đủ tải
Độ sâu khuếch tán	Số bước khử nhiễu nhân một lượt xuôi, và phải chạy nhiều lần rồi lấy trung bình	Chỉ ngoại tuyến; thêm nữa nó không tắt định	Chạy cùng ảnh nhiều lần và đo độ tàn mạn của đầu ra
Dòng quang dày	Bình phương độ phân giải lưới cho khối tương quan, cộng số vòng lặp	Ngang cả ngân sách <i>frontend</i> hiện tại, nên là quyết định thay thế chứ không phải thêm vào	Đo bộ nhớ đỉnh và độ trễ ở đúng độ phân giải bạn định dùng
Mô hình nền hình học	Bậc hai theo số khung (nhánh theo cặp) hoặc tuyến tính dốc theo số token (nhánh toàn cục)	Chỉ dùng ngoài đường tới hạn: kiểm chứng vòng, dựng lại bản đồ, phân tích lỗi	Tăng dần số khung cho tới khi hết bộ nhớ; đó là trần thật của bạn

CẦN NHỚ

Ba họ mô hình trong chương này không cho bạn phép đo mới; chúng cho bạn phỏng đoán dày. Với một hệ đã có tỉ lệ mét thật, giá trị của chúng chỉ nằm ở đúng những chỗ phép đo thật đã cạn — thị sai dưới sàn nhiều, vùng không có vân, khung bị mờ. Ở mọi chỗ khác, chúng chỉ thêm sai lệch có hệ thống. Và cái giá cố định, không thương lượng được, là: *chúng không bao giờ nói “tôi không biết”*. Hệ cổ điển của bạn hồng âm ỉ bằng cách trả về không điểm nội biên; một mạng hồng lặng lẽ bằng cách trả về một câu trả lời trông rất hợp lý.

4.6 Giới hạn và trường hợp hồng

Mục này nói về giới hạn của chính chương, không chỉ của các phương pháp trong chương.

Chương này cố ý không đưa con số ngân sách nào. Không có mili-giây trên Orin, không có megabyte, không có FLOP. Lý do không phải là lười: một ước lượng tự tính từ đếm token hay đếm phép nhân cộng có sai số cả bậc độ lớn so với thực tế, vì nó bỏ qua hiệu suất sử dụng, trình biên dịch suy luận, và các thủ thuật tiết kiệm trong bản cài đặt thật — nhưng khi đã in ra giấy thì nó *trông* như một phép đo và bị dùng như một phép đo. Thứ chương này đưa thay vào đó là *dạng* của quan hệ: chi phí tỉ lệ với cái gì, và tăng theo số mũ nào. Dạng ấy không đổi theo phần cứng, nên nó dùng được lâu; con số thì bạn phải tự đo, và Bảng 4.5 nói rõ mỗi họ đo bằng cách nào.

Lập luận về ảnh mờ là lập luận cơ chế, không phải đường cong đo được. Chuyện giá trị riêng của ma trận mô men bậc hai giảm cỡ L lần là đúng về bậc, nhưng bộ phát hiện thật có ngưỡng phi tuyến và có bước triệt phi cực đại, nên đường cong thật có thể dốc hơn hoặc thoải hơn nhiều. Đó là lý do mini project thứ hai của chương đi đo chính đường cong này.

Phép tính về sai lệch có hệ thống đã đơn giản hoá bài toán xuống một chiều. Trong bài toán tư thế đầy đủ, hướng mà sai lệch đẩy nghiệm phụ thuộc vào hình học của cảnh, và một phần của nó có thể bị chiếu triệt tiêu. Hãy coi con số 24 cm là chặn trên trên một trục, không phải dự báo cho ATE.

Ba giả định chung của cả ba họ, và chúng vỡ ở đâu.

- **Màn trập toàn cục.** Mọi mô hình trong chương này giả định một khung hình ứng với một thời điểm. Camera màn trập cuộn dưới chuyển động nhanh vì phạm vi điều đó ngay ở mức pixel, và một bản đồ điểm sẽ pha trộn hình học của nhiều thời điểm mà không có chỗ nào để khai báo điều đó.
- **Cảnh cứng.** DUST3R, VGGT và phần lớn dữ liệu huấn luyện của chúng là cảnh tĩnh. Người đi qua trước ống kính không chỉ làm sai vùng đó; nó làm hỏng cả phép căn chỉnh vì bản đồ điểm được hồi quy chung.
- **Gần lỗ kim.** Metric3D chuẩn hoá theo một tiêu cự, và DUST3R suy nội tham số từ bản đồ điểm với giả định điểm chính nằm giữa ảnh. Ống kính mắt cá hoặc méo mạnh phá cả hai, và triệu chứng là sai số biến thiên theo bán kính ảnh chứ không đều.

Ngân sách GPU là ràng buộc cứng nằm ngoài chương. Nếu GPU của Orin đã bận cho một khối khác — một bộ phát hiện vật thể, một bộ khử nhiễu — thì mọi lựa chọn trong chương này đóng lại, bất kể chúng chính xác tới đâu. Cổng ngân sách phải mở trước cổng độ chính xác.

Các công trình 2025–2026 được mô tả theo phương pháp, không theo kết quả tôi tự kiểm chứng. MAST3R-SLAM, VGGT, VGGT-SLAM và bản thứ ba của Depth Anything đều rất mới. Cơ chế mà tôi trình bày lấy từ mô tả phương pháp và tôi tin là đúng; các nhận định về hiệu năng thì tôi cố ý không đưa ra. Trần bộ nhớ và trần số khung cũng là thứ sẽ dịch chuyển nhanh nhất khi có bản cài đặt tốt hơn.

4.7 Thực hành

Repo và tài nguyên. Độ sâu đơn ảnh: [isl-org/MiDaS](https://github.com/isl-org/MiDaS) là bản tham chiếu để đọc hàm mất mát bất biến tỉ lệ và dịch — hãy mở thẳng phần cài đặt phép căn bậc nhất, nó ngắn và nói rõ hơn cả bài báo. [DepthAnything/Depth-Anything-V2](https://github.com/DepthAnything/Depth-Anything-V2) cho bản dùng được nhất hiện nay, [YvanYin/Metric3D](https://github.com/YvanYin/Metric3D) cho phần chuẩn hoá theo tiêu cự, [prs-eth/Marigold](https://github.com/prs-eth/Marigold) nếu muốn thấy cách một mô hình khuếch tán được tái dụng. Dòng quang: [princeton-vl/RAFT](https://github.com/princeton-vl/RAFT) rồi [princeton-vl/SEA-RAFT](https://github.com/princeton-vl/SEA-RAFT); đọc RAFT trước vì bản gốc gọn hơn, và hàm tra cứu là chừng bốn mươi dòng đáng đọc từng dòng. Mô hình nền hình học: [naver/dust3r](https://github.com/naver/dust3r), [naver/mast3r](https://github.com/naver/mast3r), [facebookresearch/vgggt](https://github.com/facebookresearch/vgggt), và [rmurai0610/MASt3R-SLAM](https://github.com/rmurai0610/MASt3R-SLAM) nếu muốn xem một hệ SLAM hoàn chỉnh dựng trên chúng. Đối chiếu với hệ cổ điển thì vẫn là [COLMAP](https://github.com/colmap/colmap) và [evo](https://github.com/ethz-asl/evo). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; hạ tầng như COLMAP, evo và trình biên dịch suy luận thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Với cấu hình stereo của bạn, ở khoảng cách nào thì độ bất định độ sâu vượt 10% — và bạn tính ra nó bằng công thức nào? Nếu một mô hình họ MiDaS được căn bằng đúng một hằng số nhân, chân trời cứng của nó nằm ở đâu và vì sao nó tồn tại? Vì sao việc gộp trung bình khối tương quan chỉ theo hai chiều của ảnh thứ hai lại là chi tiết quyết định của RAFT? Nếu DUST3R không ước lượng tư thế thì tư thế nằm ở chỗ nào trong đầu ra của nó, và bạn đọc nó ra bằng phép toán gì?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●● MonoDepth	Mạng đoán độ chênh lệch từ một ảnh, huấn luyện bằng sai lệch quang trắc sau khi làm cong ảnh stereo còn lại. Vì đường cơ sở lúc huấn luyện đã biết nên đầu ra có đơn vị mét — nhưng chỉ đúng cho đúng cấu hình camera đó.
●●● Độ sâu bất biến tỉ lệ và dịch	Đầu ra chỉ xác định sai khác một phép biến đổi bậc nhất trên độ sâu nghịch đảo: $d \approx s/Z + t$. Mạng biết thứ tự và tỉ lệ tương đối, không biết đơn vị và không biết gốc. Hai ẩn đó khác nhau ở mỗi khung hình.
●●● MiDaS	Trộn nhiều tập dữ liệu có nhãn không cùng đơn vị, bằng cách căn dự đoán về nhãn qua một phép biến đổi bậc nhất tối ưu trước khi tính sai lệch. Đối lấy khả năng khái quát rất tốt, cái giá là mất hẳn đơn vị mét.
●●● Metric3D	Lấy lại mét bằng cách chuẩn hoá mọi ảnh về một camera chuẩn theo tiêu cự, dự đoán, rồi nhân ngược lại. Đây là cố định gauge: một ẩn số dữ liệu không nói gì tới thì bị ghim ở bước tiền xử lý. Hệ quả: sai 3% ở tiêu cự thành sai 3% ở độ sâu.
●● Depth Anything V2	Thầy huấn luyện hoàn toàn trên ảnh tổng hợp để nhận hoàn hảo, rồi gán nhãn giả cho hàng chục triệu ảnh thật để lấp khoảng cách miền. Bài học: mở rộng dữ liệu, không mở rộng kiến trúc.
●●● Độ sâu quy ra mét bão hoà ở một khoảng cách cố định — nguyên nhân?	Bạn căn bằng một hằng số nhân duy nhất, tức ngầm đặt độ dịch bằng không. Khi đó $\hat{Z} = Z/(1 + tZ/s)$ và nó bão hoà ở s/t . Phân biệt với lệch miền: chân trời cứng cho sai số tăng đơn điệu theo khoảng cách, lệch miền thì không có dạng đó.
●●● Chân trời stereo	Khoảng cách mà độ chênh lệch tụt xuống dưới sàn nhiễu. Từ $Z = fb/d$ suy ra $\Delta Z = Z^2 \Delta d/(fb)$: sai số tăng theo bình phương khoảng cách. Với $fb \approx 50$ của EuRoC, độ chênh lệch một pixel đã ứng với 50 m.
●●● Khối tương quan toàn cặp	Tích vô hướng đặc trưng giữa mọi vị trí của ảnh một và mọi vị trí của ảnh hai, tính một lần. Với lưới 1/8 của ảnh EuRoC là 5640^2 phần tử — chi phí bộ nhớ tăng theo bình phương độ phân giải lưới, nên hạ độ phân giải là núm vặn mạnh nhất.
●●● RAFT	Tách “chỗ nào giống chỗ nào” — tính một lần — khỏi “tôi đang đoán ở đâu” — tra cứu lặp quanh phỏng đoán hiện tại, cập nhật bằng một khối hồi tiếp. Số vòng lặp là núm vặn độ trễ chỉnh được lúc chạy.
●● Vì sao RAFT không cần thu nhỏ ảnh?	Vì kim tự tháp của nó dựng trên khối tương quan, và chỉ gộp hai chiều thuộc ảnh thứ hai. Lưới của ảnh một giữ nguyên độ phân giải ở mọi mức, nên vật nhỏ bay nhanh không biến mất — đúng chỗ chiến lược thô tới mịn thất bại.
●● SEA-RAFT	Bản RAFT gọn hơn: hồi quy thẳng nghiệm khởi đầu nên cần ít vòng lặp hơn, và đổi hàm mất mát sang hỗn hợp hai phân phối Laplace. Nhờ hàm mất mát đó nó xuất ra được độ bất định theo từng pixel, dùng thẳng làm trọng số.
●●● Vì sao ghép cặp thưa sụp đổ dưới ảnh mờ còn dòng quang chỉ xuống dốc?	Vì ghép cặp thưa là chuỗi cổng nhân nhau: phải phát hiện được ở cả hai khung, rồi mới tới mô tả và kiểm tra tỉ số. Tính lặp lại tụt từ 0,8 xuống 0,3 thì xác suất sống qua cả hai khung tụt từ 0,64 xuống 0,09. Dòng quang không có cổng phát hiện nào.
●●● Bản đồ điểm (pointmap)	Tensor $H \times W \times 3$ gán thẳng cho mỗi pixel một điểm ba chiều trong một hệ toạ độ cho trước. Khác bản đồ độ sâu ở chỗ nó đã mang sẵn hệ toạ độ, nên đọc ra hình học mà không cần biết tiêu cự trước.
●●● DUST3R	Nhận hai ảnh chưa hiệu chỉnh, trả về hai bản đồ điểm cùng nằm trong hệ toạ độ của camera một. Tư thế không biến mất mà bị gói vào đầu ra, nên nó đọc ra bằng Procrustes chứ không cần ma trận cơ bản. Đầu ra không có đơn vị mét.
●● MAST3R	DUST3R cộng một đầu ra mô tả đặc trưng dày và một thủ tục ghép tương hỗ nhanh. Kết quả là bộ ghép cặp neo trong ba chiều, nên nó chịu được thay đổi góc nhìn cực đoan — chính chỗ mà mọi bộ mô tả hai chiều chết.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• VGGT	Một <i>transformer</i> nhận nhiều ảnh cùng lúc, xen kẽ attention trong từng ảnh và attention trên toàn bộ token của mọi ảnh, rồi trả thẳng ra tư thế, độ sâu và bản đồ điểm. Không còn bước căn chỉnh toàn cục; đổi lại bộ nhớ attention toàn cục đặt trần cứng lên số khung.
• VGGT-SLAM	Ghép các mảnh bản đồ của VGGT trên nhóm xạ ảnh SL(4) thay vì phép biến đổi tương tự, vì với camera chưa hiệu chỉnh thì lớp bất định là xạ ảnh. Bài học: lớp bất định của frontend quyết định đa tạp mà backend phải tối ưu trên đó.
••• Vì sao hạ trọng số không cấu được một tiên nghiệm dày?	Vì sai số của nó tương quan gần như hoàn toàn, còn ma trận thông tin chéo lại đếm 5000 pixel thành 5000 phiếu độc lập. Với 20 phép đo $\sigma = 0,1$ và 5000 tiên nghiệm $\sigma = 0,3$, tiên nghiệm áp đảo gần ba mươi lần. Cách chữa đúng là thêm biến phụ mỗi khung để hấp thụ sai số chung.
•• Bản đồ dày thở phập phồng và độ lệch gia tốc kể trôi — nghĩa là gì?	Bạn đang nhét độ sâu bất biến tỉ lệ và dịch vào một bộ tối ưu tin rằng cảnh cứng. Tỉ lệ riêng từng khung bị hấp thụ vào tư thế và vận tốc, và độ lệch gia tốc kể — vốn hấp thụ mọi thứ không giải thích được — lãnh phần còn lại.
•• Cùng một ảnh, hai lần chạy cho hai bản đồ độ sâu khác nhau — nguyên nhân?	Mô hình dựa trên khuếch tán như Marigold khởi đầu từ nhiễu ngẫu nhiên, nên nó vốn ngẫu nhiên; người ta phải chạy nhiều lần rồi lấy trung bình. Phân biệt với phi tắt định của thư viện: cố định hạt giống mà vẫn khác thì là thư viện, cố định hạt giống mà giống nhau thì là bản chất mô hình.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DROID-SLAM [9]	Dùng nguyên vòng lặp tra cứu trên khối tương quan của RAFT làm <i>frontend</i> , rồi nối vào một tầng hiệu chỉnh chùm tia khả vi	Chi tiết ở F/31; đây là bằng chứng rõ nhất rằng ý tưởng của RAFT sống ngoài bài toán dòng quang
MASt3R-SLAM [13]	Đặt thẳng một hệ SLAM dày lên tiên nghiệm bản đồ điểm hai ảnh, và tối ưu trong không gian bản đồ điểm	Công trình 2025; nêu cơ chế, không nêu số
Depth-Anything-V2 [84]	Công thức thầy tổng hợp cộng nhân giả trên ảnh thật — mở rộng dữ liệu thay vì mở rộng kiến trúc	Được dùng rộng rãi làm nguồn độ sâu trong các pipeline dựng lại và điều hướng
Marigold [85], <i>diffusers</i>	Tái dụng mô hình khuếch tán sinh ảnh làm bộ ước lượng độ sâu, huấn luyện trên rất ít dữ liệu	Hạ tầng <i>diffusers</i> bền hơn nhiều so với tên mô hình
SEA-RAFT [90]	Hàm mất mát hỗn hợp Laplace để mô hình được tính đa phương thức và xuất ra độ bất định theo pixel	Đây là cầu nối thẳng sang bài toán hiệp phương sai học được ở F/34
VGGT [93]	Xen kẽ attention trong ảnh và attention toàn cục để bỏ hẳn bước căn chỉnh toàn cục	Công trình 2025; trần số khung là hệ quả cấu trúc của attention toàn cục, mức cụ thể phải tự do

MINI PROJECT SLAM • Đo trần đóng góp của độ sâu học được bằng một thí nghiệm oracle • 1–2 ngày

Mục tiêu trả lời câu “độ sâu học được có thêm được gì cho hệ stereo-inertial của tôi không” *trước* khi cài bất kỳ mạng nào. Nếu một mô hình hoàn hảo cũng chỉ nâng APE dưới sàn nhiễu, cả hướng này đóng lại và bạn tiết kiệm được nhiều tuần.

Dữ liệu	hai chuỗi EuRoC có độ khó khác nhau, chẳng hạn MH_01 và V2_03. Chân trị tư thế đi kèm; nếu bản tải của bạn có bản quét laser phòng Vicon thì dùng nó làm chân trị hình học, nếu không thì dùng bản đồ dày do chính hệ dựng sau khi tối ưu toàn cục làm chân trị xấp xỉ và ghi rõ hạn chế đó.
Các bước	(1) chạy hệ hiện tại, ghi độ sâu của mọi điểm bản đồ tại thời điểm tam giác hoá, vẽ biểu đồ phân bố và đếm tỉ lệ điểm có độ chênh lệch dưới 1 pixel và dưới 2 pixel — đó là phần “thị sai đã cặn”; (2) dựng bản đồ độ sâu oracle cho từng khung khoá từ chân trị; (3) thay độ sâu của riêng nhóm thị sai cặn bằng giá trị oracle, giữ nguyên mọi thứ khác, chạy năm lần mỗi cấu hình; (4) so chênh lệch APE với sần nhiễu bạn đã đo ở chương 24.
Xong khi	bạn phát biểu được một câu có số chống lưng: “trên chuỗi này, một mô hình độ sâu hoàn hảo nâng APE nhiều nhất X cm, và X lớn hơn hay nhỏ hơn sần nhiễu”. Kiểm tra chéo: tỉ lệ điểm thị sai cặn phải lớn hơn hẳn ở chuỗi ngoài trời hoặc chuỗi có cảnh xa; nếu hai chuỗi cho cùng tỉ lệ thì khả năng cao bạn đang đếm sai.
Nối với	E/24 cho thiết kế thí nghiệm oracle và cách đọc sần nhiễu; F/27 cho khung bốn cổng; kết quả của dự án này quyết định có nên đọc kỹ F/32 hay không.

MINI PROJECT SLAM · Dòng quang dày có cứu được các khung mất bám không · 1–2 ngày

Mục tiêu	biến câu “dòng quang chịu được ảnh mờ tốt hơn” thành một ngưỡng tốc độ góc cụ thể cho hệ của bạn, kèm một con số độ trễ đo trên Orin chứ không ước lượng.
Dữ liệu	các đoạn mất bám hoặc số điểm khớp thấp trong log chạy của bạn trên V2_03 và MH_05; chân trị tư thế của EuRoC; chuỗi IMU để lấy tốc độ góc.
Các bước	(1) trích danh sách các cặp khung hình có số điểm khớp dưới ngưỡng phục hồi; (2) với mỗi cặp, chạy princeton-vl/SEA-RAFT ngoại tuyến, lấy trường dòng, chiếu các điểm bản đồ của khung trước theo trường đó và đếm bao nhiêu điểm rơi đúng chỗ theo chân trị tư thế; (3) gán cho mỗi cặp một tốc độ góc trung bình từ IMU và một chỉ số độ mờ, chẳng hạn năng lượng gradient đã chuẩn hoá; (4) vẽ tỉ lệ điểm tìm lại được theo tốc độ góc cho cả hai nhánh, thưa và dày; (5) biên dịch mạng bằng trình suy luận của Jetson ở đúng độ phân giải EuRoC và đo độ trễ.
Xong khi	bạn có hai đường cong trên cùng một trục, một ngưỡng tốc độ góc mà trên đó nhánh dày mới có lãi, và một con số độ trễ đo được. Từ ba thứ đó bạn phát biểu được: “dòng quang dày đáng bật ở $X\%$ số khung hình của chuỗi này, và nó tốn Y ms mỗi lần bật”.
Nối với	F/28 cho phía đặc trưng học được của cùng bài toán; F/34 cho việc biến độ bất định theo pixel của SEA-RAFT thành trọng số thật; dự án trước cung cấp sần nhiễu để đọc kết quả.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chỉ ở đúng những chỗ phép đo của bạn đã cặn. Với $fb \approx 50$ của EuRoC và giả định độ chính xác độ chênh lệch 0,2 pixel, sai số stereo tăng theo Z^2 và vượt mười phần trăm ở vài chục mét, trong khi sai số tương đối của mạng không tăng theo khoảng cách — nên hai đường cong bắt buộc cắt nhau, và điểm cắt là thứ bạn phải tự đo. Cộng thêm vùng không có vân, nơi bạn không có phép đo nào để cạnh tranh. Ngoài hai chỗ đó, nó chỉ thêm sai lệch có hệ thống.
- A2. Nó dựng kim tự tháp trên *khối tương quan*, và chỉ gộp trung bình hai chiều thuộc ảnh thứ hai. Lưới của ảnh một giữ nguyên độ phân giải ở mọi mức, nên vật nhỏ không bao giờ bị thu nhỏ tới mức biến mất. Chiến lược thô tới mịn cổ điển thu nhỏ chính bức ảnh, nên nó đánh mất vật nhỏ trước khi kịp ước lượng.
- A3. Tư thế nằm trong chính đầu ra. Cả hai bản đồ điểm đều được dự đoán trong hệ toạ độ của camera một. Vì thế phép biến đổi cứng khớp bản đồ điểm của ảnh hai với hình học của camera hai chính là tư thế tương đối, và nó có nghiệm dạng đóng. Bước ước lượng bị đối thành bước đọc.
- A4. Vì bạn đang chống lại một tỉ số đếm: 5000 so với 20 là lợi thế 250 lần, nên muốn triệt tiêu nó phải nhân phương sai lên 250 lần, tức nhân độ lệch chuẩn lên gần 16 lần. Với $\sigma_m = 0,1$ thì bạn

phải khai báo $\sigma_p \gg 1,6$ m, mức mà tiên nghiệm không còn nói gì. Phép tính ba mươi giây là so hai đại lượng N/σ^2 .

- A5. Vì nó khởi đầu từ nhiễu ngẫu nhiên, nên cùng một ảnh cho hai đầu ra khác nhau. Bạn vừa dựng lại một sàn nhiễu ngay giữa pipeline, và lần này không có bug nào để sửa — khác hẳn với thứ tự duyệt container hay tranh chấp luồng. Một mô hình hồi quy thì tất định theo trọng số, nên nó không mang thêm nguồn phi tất định nào.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn cần biết mọi vật trong phòng cách bạn bao xa. Bạn có hai con mắt cách nhau mười một phân, nên với thứ ở gần bạn đo được khá chuẩn: hai mắt nhìn lệch nhau, độ lệch càng lớn thì vật càng gần. Nhưng với thứ ở cuối hành lang, độ lệch giữa hai mắt gần như bằng không, và bạn hết cách. Bây giờ có một người bạn chưa từng cầm thước bao giờ, nhưng đã nhìn hàng triệu căn phòng trong đời. Hỏi anh ta ghế kia xa bao nhiêu, anh ta nói ngay, và anh ta nói cho cả nghìn điểm trong phòng cùng lúc. Anh ta luôn đúng về chuyện cái nào gần hơn cái nào. Anh ta gần như luôn sai về chuyện bao nhiêu mét, và cái sai ấy đổi mỗi khi bạn đưa cho anh ta một bức ảnh khác.

Chuyện gì xảy ra nếu bạn ghi lời anh ta vào cùng cuốn sổ với các phép đo của mình rồi lấy trung bình? Bạn đo hai mươi lần, anh ta đoán năm nghìn lần, nên trung bình sẽ nghiêng hẳn về phía anh ta. Tệ hơn: khi anh ta sai, anh ta sai cả năm nghìn lần theo cùng một hướng, nên việc có nhiều số không giúp gì cả. Và điều nguy hiểm nhất là anh ta không bao giờ nói “chỗ này tôi không nhìn rõ”. Anh ta luôn có câu trả lời, kể cả khi nhìn vào một tấm gương.

Cách dùng anh ta cho đúng vì thế rất hẹp: chỉ hỏi ở cuối hành lang, chỗ thước của bạn đã hết tầm; và ghi lời anh ta vào một trang riêng, đừng trộn vào trang có số đo.

Chỗ không rút gọn được: chuyện “cái sai đổi mỗi khi đưa một bức ảnh khác”. Mọi cách nói gọn hơn đều làm sai lệch, vì đây không phải chuyện anh ta đoán lúc trúng lúc trượt. Cái đổi là *đơn vị* anh ta đang dùng, và nó đổi âm thầm giữa hai bức ảnh chụp cách nhau một phần ba mươi giây. Một bức tường đứng yên vì thế lúc gần lúc xa, và hệ thống của bạn — vốn tin rằng tường không di chuyển — buộc phải kết luận rằng chính bạn mới là người đang nhúc nhích.

Một câu: Chúng cho bạn hàng nghìn phỏng đoán tự tin ở chỗ bạn chỉ có vài chục phép đo thật, nên hãy chỉ hỏi chúng ở đúng chỗ phép đo của bạn đã hết.

4.8 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là bộ lọc mà chương này áp dụng: bốn cổng — bài toán có thật, đo được, vừa ngân sách, hông có kiểm soát — xuất hiện lại ở đây dưới dạng cây quyết định. [F/28-dac-trung-va-ghep-cap](#) là mặt đối lập trực tiếp: nó giữ cấu trúc thừa và thay chất lượng của từng điểm, còn chương này bỏ luôn cấu trúc thừa. Đọc hai chương cạnh nhau là cách nhanh nhất để thấy đánh đổi giữa “ít phép đo nhưng kiểm chứng được” và “nhiều phỏng đoán nhưng không kiểm chứng được”. [F/29-dinh-vi-lai-va-dong-vong](#) nhận từ chương này một lối thoát cụ thể cho tầng kiểm chứng hình học: ghép cặp neo trong ba chiều không cần hai mảnh ảnh trông giống nhau. [F/31-backend-kha-vi-va-dau-cuoi](#) là bước tiếp theo tự nhiên, vì DROID-SLAM chính là vòng lặp tra cứu của RAFT nối vào một tầng hiệu chỉnh chùm tia khả vi. [F/32-ban-do-neural](#) thụ đầu ra của chương này: một bản đồ dày cần một nguồn độ sâu, và chất lượng nguồn đó quyết định chất lượng bản đồ. [F/34-imu-bat-dinh-va-trien-khai](#) là nơi kết luận quan trọng nhất của chương được xử lý đàng hoàng: muốn dùng một tiên nghiệm dày thì phải có trọng số đúng, và trọng số đúng phải được huấn luyện chứ không được đoán. Về phía nền: [C/16/02-bieu-dien-3d-va-depth](#) là tiền đề hình học, [C/15/03-flow-tracking-va-rang-buoc](#) là tiền đề về dòng quang, [B/11-gioi-han-tinh-toan](#) cho khung phân tích bộ nhớ và băng thông, và [E/24-thi-nghiem-va-ablation](#) cho thí nghiệm oracle mà cả hai mini project của chương đều dựa vào.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao việc nhân toàn bộ cảnh và toàn bộ quãng đường camera với cùng một hệ số lại khiến độ sâu đơn ảnh trở thành bài toán *không giải được* chứ không phải bài toán khó, và điều đó ràng buộc gì lên mọi mô hình trong mục 2?
2. Metric3D ghim ẩn số tiêu cự ở bước tiền xử lý thay vì để mạng học. Chỗ nào trong công việc SLAM của bạn dùng đúng thủ thuật đó, và cái giá tương ứng ở cả hai chỗ là gì?
3. Nếu bạn chỉ được giữ *một* trong hai thành phần của RAFT — khối tương quan toàn cặp, hoặc vòng lặp tra cứu bằng khối hồi tiếp — thì bỏ cái nào ít hại hơn cho bài toán chuyển động nhanh, và vì sao?
4. Dòng quang dày luôn trả lời, kể cả ở vùng bị che. Vì sao tính chất đó lại nguy hiểm hơn cho một hệ SLAM so với việc bộ ghép cặp thừa trả về không điểm nội biên?
5. DUS3R xoá bước ước lượng tư thế nhưng không xoá được tính không xác định được của tỉ lệ. Vì sao một cái xoá được còn cái kia thì không?
6. Bạn hạ trọng số của một tiên nghiệm độ sâu xuống mười lần mà nghiệm vẫn dịch mấy centimet. Điều đó nói gì về cấu trúc sai số của tiên nghiệm, và cách chữa đúng là gì thay vì hạ tiếp?
7. Trần số khung của VGGT đến từ bộ nhớ attention toàn cục, còn trần của DUS3R đến từ số cặp. Hai trần đó dẫn tới hai chiến lược chia mảnh bản đồ khác nhau như thế nào?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., "Image Matching Across Wide Baselines: From Paper to Practice," *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., "RoMa: Robust Dense Feature Matching," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., "Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., "HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, "Closed-form solution of absolute orientation using unit quaternions," *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., "Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, "Rethinking Visual Geo-localization for Large-Scale Applications," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, "MixVPR: Feature Mixing for Visual Place Recognition," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., "EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, "Optimal Transport Aggregation for Visual Place Recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features Without Supervision," *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., "AnyLoc: Towards Universal Visual Place Recognition," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Backend khả vi và hệ VO/SLAM đầu-cuối (Differentiable Backends and End-to-End Systems)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định có nên thay một khối); A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (Gauss-Newton, Levenberg-Marquardt, quy tắc chuỗi); C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc (dòng quang và toán tử cập nhật hồi quy); F/28-dac-trung-va-ghep-cap là nửa frontend của cùng câu chuyện.

TÓM TẮT

Bạn đã thạo Gauss-Newton, Levenberg-Marquardt và đồ thị nhân tử. Chương này không dạy lại chúng. Nó nói về đúng một thứ mà chữ “khả vi” thêm vào: cho *gradient* chạy *xuyên qua* lời giải của bài toán tối ưu, để những con số bạn đang đặt bằng tay được huấn luyện theo lỗi quỹ đạo cuối cùng. Đọc xong bạn viết được đạo hàm của một nghiệm arg min bằng định lý hàm ẩn, viết ra được công thức chi phí bộ nhớ của lượt ngược qua một lớp tối ưu, và đọc DROID-SLAM hay CodeSLAM mà biết ngay chỗ nào là hình học cũ, chỗ nào là phần học được. Nếu chỉ nhớ một điều: khả vi *không* làm bộ giải tốt lên, nó chỉ biến các nút vặn thủ công thành tham số học được — và cái giá là bộ nhớ lượt ngược, độ ổn định số ở đúng những khung hình khó, cùng gần hết khả năng chẩn đoán khi hệ hỏng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán bình phương tối thiểu phi tuyến** — tìm x làm nhỏ nhất tổng bình phương của các phần dư $r_i(x)$. Trong SLAM, x là tư thế và điểm bản đồ, còn r_i là sai lệch chiếu lại của một quan sát.
- **ma trận thông tin (information matrix)** — nghịch đảo hiệp phương sai của một phép đo, và cũng là trọng số của cạnh đó trong bài toán bình phương tối thiểu. Đặt lớn nghĩa là “tôi tin phép đo này”.
- **Hessian** H — ma trận hệ phương trình chuẩn của Gauss-Newton, $H = J^T \Sigma^{-1} J$. Nghịch đảo của nó xấp xỉ hiệp phương sai của nghiệm, tức thứ bạn dùng khi biên hoá một khung hình.
- **tự do đo (gauge freedom)** — các hướng mà bài toán không xác định được. Dịch cả quỹ đạo đi một khoảng thì lỗi không đổi. Theo những hướng đó H suy biến, nên phải neo gốc toạ độ hoặc thêm damping.
- **damping** λ — số cộng vào đường chéo của H trong Levenberg-Marquardt. Nó vừa làm hệ khả nghịch, vừa rút ngắn bước đi khi mô hình tuyến tính hoá không đáng tin.
- **đạo hàm tự động (autodiff)** — kỹ thuật để máy tính đạo hàm của một chương trình bằng cách áp quy tắc chuỗi lên từng phép toán. **lan truyền ngược (backpropagation)** là autodiff chạy theo chiều ngược.
- **lượt xuôi và lượt ngược** — lượt xuôi là một lần chạy mạng để ra kết quả; lượt ngược là một lần đi ngược để tính đạo hàm. Lượt ngược cần các giá trị trung gian của lượt xuôi, nên phải giữ chúng trong bộ nhớ.
- **tự giám sát (self-supervised)** — huấn luyện không cần nhãn do người tạo. Tín hiệu dạy đến từ chính dữ liệu, ví dụ “dừng lại ảnh này từ ảnh kia thì phải khớp”.

CÂU HỎI MỞ ĐẦU

- Q1. Backend của bạn đã tối ưu tốt rồi. Vậy chữ “khả vi” còn thêm được gì, và nếu nó không làm Gauss-Newton hội tụ nhanh hơn thì nó bán cái gì?
- Q2. Đạo hàm của một nghiệm arg min theo tham số đầu vào tính được mà không cần lưu bước lặp nào. Vì sao lại có chuyện đó, và điều kiện nào phải đúng để nó không sai?
- Q3. Quyết định inlier hay outlier là một phép so sánh với ngưỡng, nên đạo hàm của nó bằng 0 gần như ở mọi nơi. Vậy một hệ đầu-cuối học cách loại điểm sai bằng cách nào?
- Q4. DROID-SLAM chính xác hơn hẳn nhiều hệ cổ điển trên *benchmark* công khai. Vì sao nó vẫn không phải lựa chọn cho một robot chạy Jetson Orin, và cái gì chặn nó?
- Q5. Nén cả bản đồ độ sâu vào 32 con số rồi tối ưu chung với tư thế nghe rất gọn. Ý tưởng đó ra đời năm 2018 và tới nay vẫn chưa thắng. Nó hỏng ở chỗ nào?

5.1 Đặt vấn đề

Backend của bạn không hỏng. Nó tuyến tính hoá, nó dùng phần bù Schur, nó chạy nhân Huber, nó biên hoá khung hình cũ. Đưa cho nó các phần dư đúng và các trọng số đúng thì nó trả về nghiệm tốt nhất theo nghĩa bình phương tối thiểu. Vấn đề nằm ở chỗ khác: **ai quyết định các trọng số đó.**

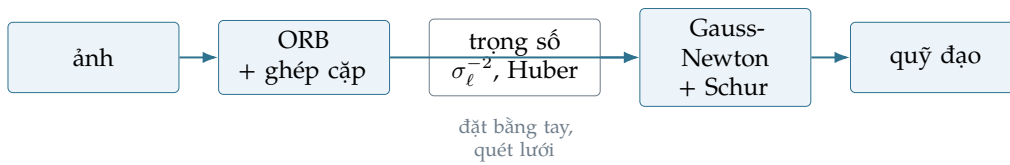
Hãy liệt kê thật cụ thể, trong một hệ kiểu ORB-SLAM3 [1]:

- **Trọng số cạnh chiếu lại** — ma trận thông tin là $\sigma_\ell^{-2}I$, với σ_ℓ suy từ hệ số tỉ lệ của kim tự tháp ORB. Nó chỉ phụ thuộc vào *tầng* mà điểm được phát hiện. Nó không biết mảnh ảnh đó có bị mờ không, có nằm trên vệt loá không, có nằm trên một hoa văn lặp không.
- **Bề rộng nhân robust** — ngưỡng χ^2 5,991 cho cạnh hai bậc tự do và 7,815 cho cạnh ba bậc. Đây là phân vị 95% của một giả định phân phối chuẩn, và giả định ấy hiếm khi được kiểm lại trên dữ liệu thật.
- **Nhiều IMU** — lấy từ tờ thông số cảm biến, cộng thêm một bước ngẫu nhiên cho độ trôi. Cùng một con số cho lúc đứng yên và lúc quay đầu 200 độ mỗi giây.
- **Các ngưỡng rời rạc** — tỉ lệ khoảng cách mô tả, ngưỡng inlier của RANSAC, chính sách chèn khung then chốt.

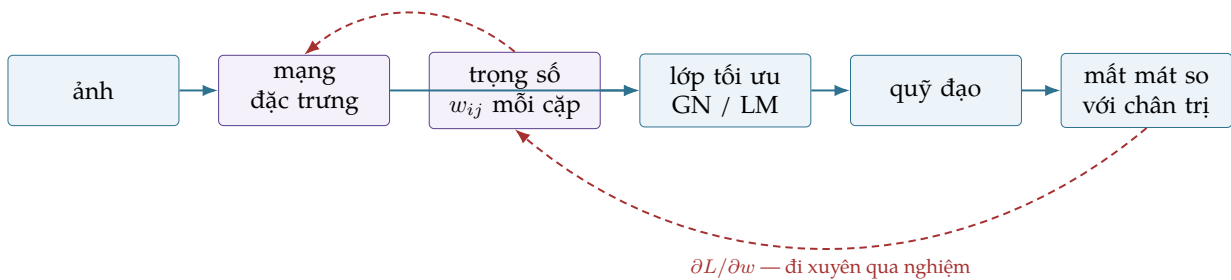
Mọi con số trên đều trả lời cùng một câu hỏi: *tôi nên tin phép đo này bao nhiêu*. Và mọi con số trên đều được đặt một lần, toàn cục, từ một tờ thông số hoặc một bài báo. Trong khi đó bạn có một đại lượng thật sự quan tâm ở cuối đường ống, là sai số quỹ đạo tuyệt đối. Giữa các nút vận và đại lượng đó **không có đạo hàm nào**. Nên bạn đóng khoảng trống ấy bằng cách duy nhất còn lại: quét lưới, chạy lặp nhiều lần, đọc bảng. Đó chính là lý do tồn tại của mọi *benchmark* harness.

Chữ **khả vi** (*differentiable*) trong chương này là lời hứa rằng đạo hàm ấy tồn tại được. Nếu toàn bộ đường đi từ điểm ảnh tới quỹ đạo là một đồ thị phép tính khả vi, thì đạo hàm của APE theo trọng số cạnh là một đại lượng tính được. Khi đó các trọng số trở thành tham số học được. Hình 5.1 vẽ đúng sự khác biệt: mũi tên xuôi giống nhau ở hai phía, mũi tên ngược chỉ tồn tại ở phía dưới.

Hệ cổ điển



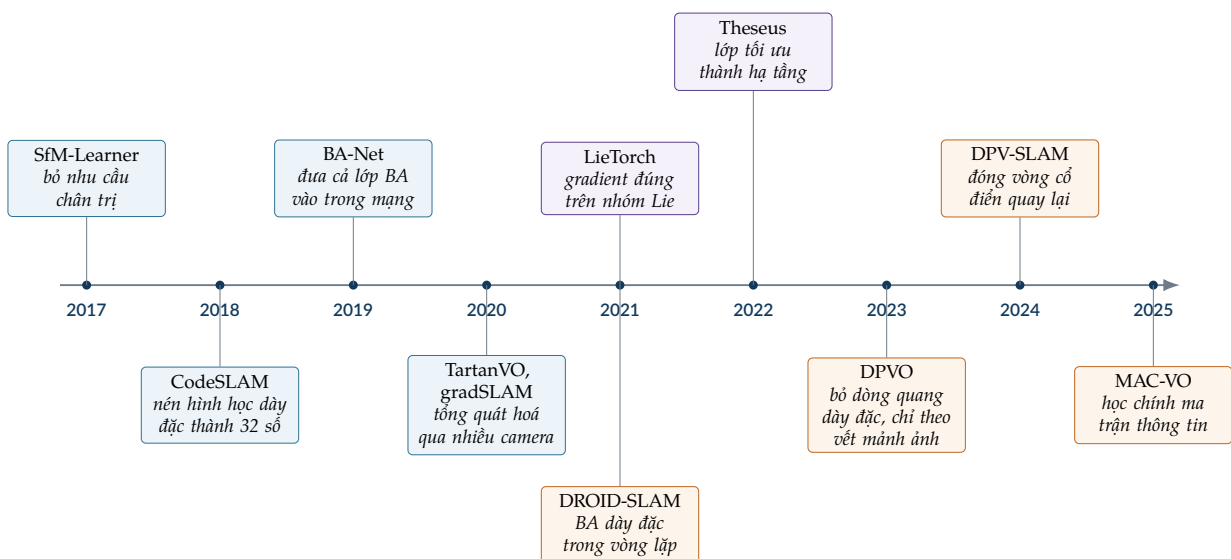
Hệ khả vi



Hình 5.1. Cái mà chữ “khả vi” thêm vào. Chiều xuôi, vẽ bằng mũi tên liền, giống hệt nhau ở hai hệ: đặc trưng, trọng số, bộ giải, quỹ đạo. Khác biệt duy nhất là mũi tên đứt màu đỏ. Nó đi ngược *xuyên qua* lời giải của bài toán tối ưu, nhờ đó các trọng số w_{ij} và mạng sinh ra chúng được huấn luyện theo lỗi quỹ đạo cuối cùng, thay vì được đặt bằng tay từ tờ thông số cảm biến.

Nói gọn giá trị của chương: học xong bạn phân biệt được ba nhóm hay bị gộp làm một. Nhóm thứ nhất là *lớp tối ưu khả vi*, một mảnh hạ tầng có thể lấy riêng. Nhóm thứ hai là *hệ đầu-cuối* kiểu DROID-SLAM, một kiến trúc trọn gói và đắt. Nhóm thứ ba là *VO tự giám sát*, vốn là một cách huấn luyện chứ không phải một hệ SLAM. Ba thứ đó có chi phí và rủi ro rất khác nhau. Trộn chúng lại là cách nhanh nhất để đánh giá sai cả ba.

Hình 5.2 xếp các công trình của chương lên một trục thời gian, và ghi cạnh mỗi mốc *nút thắt mà nó gỡ*. Đọc hình đó trước sẽ nhanh hơn đọc tuần tự, vì cả chương chỉ là một chuỗi lời đáp cho những nút thắt liên tiếp nhau.



Hình 5.2. Dòng thời gian của chương, mỗi mốc ghi kèm nút thắt mà nó gỡ. Hộp tím là hạ tầng, tức thứ dùng lại được ở mọi hệ. Hộp cam là các hệ đầu-cuối, và ba mốc cuối cho thấy hướng đi chung của chúng: bớt dày đặc, mượn lại đóng vòng cổ điển, rồi chuyển mục tiêu học từ đặc trưng sang ma trận thông tin — tức là càng ngày càng gần với một backend cổ điển được cấp trọng số tốt hơn.

5.2 Đạo hàm xuyên qua một nghiệm tối ưu (Differentiating Through an Argmin)

A. Bài toán phát biểu chính xác

Gọi θ là mọi thứ mạng sinh ra: trọng số cạnh, mục tiêu dòng quang, hệ số damping. Gọi

$$x^*(\theta) = \arg \min_x E(x, \theta)$$

là nghiệm mà bộ giải trả về, với x là tư thế và hình học. Ở cuối có một hàm mất mát $L(x^*(\theta))$, thường là sai lệch so với quỹ đạo chân trị. Ta cần $\partial L / \partial \theta$.

Chỗ khó là x^* không có công thức đóng. Nó là kết quả của một vòng lặp. Có đúng hai cách lấy đạo hàm qua nó, và hai cách đánh đổi ngược nhau.

B. Cách thứ nhất: trải phẳng vòng lặp

Cách thô là coi K bước lặp như một mạng K tầng rồi để autodiff làm việc:

$$x_{k+1} = x_k - (H_k + \lambda I)^{-1} g_k.$$

Mỗi bước là một biểu thức khả vi, nên cả vòng lặp cũng vậy. Cách này gọi là **trải phẳng** (*unrolling*).

Ưu điểm của nó là tính trung thực. Đạo hàm thu được là đạo hàm của *thuật toán thật sự đã chạy*, kể cả khi thuật toán đó mới chạy năm bước và chưa hội tụ. Nhược điểm là bộ nhớ. Lướt ngược cần mọi giá trị trung gian của lượt xuôi. Bạn phải giữ cả K bộ Jacobian, K bản phân tích Cholesky, và mọi bản đồ đặc trưng đã sinh ra chúng.

Chi phí ấy nên nhớ dưới dạng công thức, vì công thức mới là thứ dùng lại được. Gọi K là số bước lặp, N là số khung trong cửa sổ, C là số kênh của bản đồ đặc trưng, và HW là số vị trí ở độ phân giải đang chạy. Bộ nhớ lướt ngược tỉ lệ với tích $K N C H W$ — tuyến tính theo *cả bốn* thừa số cùng lúc, và một bước lặp hồi quy còn giữ vài tensor cùng cỡ nên hằng số nhân phía trước cũng không nhỏ. Đó là lý do các hệ trải phẳng thật đều huấn luyện với cửa sổ ngắn và độ phân giải thấp: N và HW là hai thừa số duy nhất người dùng cắt được mà không phải đổi kiến trúc. Nếu bạn muốn một con số gigabyte cho hệ của mình thì phải tự đo; công thức chỉ nói hình dạng, không nói độ lớn.

C. Cách thứ hai: định lý hàm ẩn

Cách thứ hai không lưu bước lặp nào. Nó dùng đúng một sự kiện: tại nghiệm, **gradient** (*gradient*) theo x bằng không.

$$g(x^*, \theta) = \left. \frac{\partial E}{\partial x} \right|_{x^*} = 0.$$

Đây là một hệ phương trình ràng buộc x^* với θ . Lấy vi phân toàn phần hai vế theo θ :

$$\underbrace{\frac{\partial^2 E}{\partial x^2}}_H \frac{\partial x^*}{\partial \theta} + \underbrace{\frac{\partial^2 E}{\partial x \partial \theta}}_B = 0 \quad \implies \quad \frac{\partial x^*}{\partial \theta} = -H^{-1} B.$$

Đây là **định lý hàm ẩn** (*implicit function theorem*) áp cho điều kiện tối ưu. Nó nói rằng nghiệm trượt đi theo hướng nào khi ta đẩy tham số, và câu trả lời chỉ phụ thuộc hai thứ: độ cong H tại nghiệm, và mức độ tham số đẩy vào gradient qua B .

Với hàm mất mát cuối, ta không cần cả ma trận $\partial x^* / \partial \theta$. Chỉ cần một tích vectơ:

$$\frac{\partial L}{\partial \theta} = - \left(\frac{\partial L}{\partial x} \right)^\top H^{-1} B = -v^\top B, \quad \text{với } Hv = \frac{\partial L}{\partial x}.$$

Đọc dòng cuối cho kỹ, vì đó là điểm mấu chốt của cả chương. **Lượt ngược chỉ tốn thêm đúng một lần giải hệ, với chính ma trận H mà lượt xuôi đã phân tích rồi.** Với cửa sổ 8 khung, hệ camera rút gọn chỉ là 48×48 , tức nhỏ hơn nhiều bậc so với chính lượt xuôi vừa chạy, nên lượt ngược không thêm khoản chi phí nào đáng kể. Bộ nhớ cho vòng lặp bằng không, vì không bước lặp nào được lưu.

Còn một cách đọc nữa mà kỹ sư SLAM sẽ thấy quen ngay. H^{-1} chính là hiệp phương sai của nghiệm, đúng đại lượng bạn đã tính khi biên hoá một khung hình. Cổ máy cần cho lượt ngược đã nằm sẵn trong mã nguồn của bạn. Ý tưởng lấy đạo hàm qua một bài toán tối ưu không sinh ra từ thị giác; nó đến từ dòng lớp tối ưu khả vi trong học máy [95], [96], và cùng một thủ thuật cũng là thứ làm cho mô hình cân bằng sâu chạy được [97].

TRỰC GIÁC

Hãy hình dung nghiệm là một hòn bi nằm dưới đáy một cái bát. Bạn không quan tâm nó lăn xuống đáy bằng đường nào. Bạn chỉ quan tâm một việc: nghiêng cái bát đi một chút thì hòn bi dịch bao nhiêu. Trả lời câu đó chỉ cần biết *độ cong của cái bát tại đáy* và *lực nghiêng đáy vào đâu*. Đó đúng là H^{-1} và B . Cách trải phẳng thì ngược lại: nó quay phim toàn bộ đường lăn rồi tua ngược. Tốn băng — nhưng nếu hòn bi chưa lăn tới đáy thì đoạn phim vẫn dùng được, còn cái bát thì không.

D. Ví dụ tính tay được

Lấy bài toán nhỏ nhất còn giữ nguyên bản chất: một ẩn, hai phép đo, một trọng số.

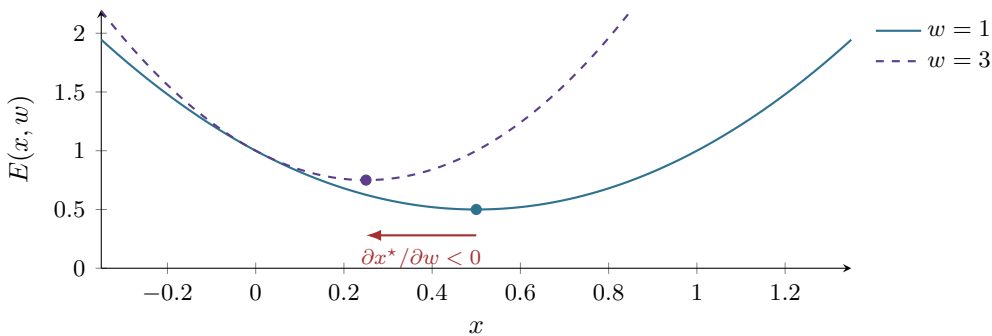
$$E(x, w) = w(x - a)^2 + (x - b)^2.$$

Nghiệm có công thức đóng, $x^* = (wa + b)/(w + 1)$. Bây giờ tính lại bằng định lý hàm ẩn. Ta có $H = 2(w + 1)$ và $B = \partial^2 E / \partial x \partial w = 2(x - a)$, nên

$$\frac{\partial x^*}{\partial w} = -\frac{2(x^* - a)}{2(w + 1)} = -\frac{x^* - a}{w + 1}.$$

Thay $a = 0, b = 1, w = 1$. Khi đó $x^* = 0,5$ và đạo hàm bằng $-0,25$. Kiểm tra trực tiếp: $x^* = 1/(w + 1)$, đạo hàm là $-1/(w + 1)^2 = -1/4$. Khớp.

Con số $-0,25$ nói điều gì bằng lời thường? Tăng trọng số của phép đo a lên 0,1 thì nghiệm dịch về phía a khoảng 0,025. Nếu hàm mất mát cuối muốn nghiệm nhỏ đi, **gradient** sẽ bảo mạng “hãy tin phép đo a hơn”. Đó chính xác là thứ một hệ đầu-cuối học, chỉ khác là với hàng chục nghìn phép đo cùng lúc. Hình 5.3 vẽ lại cùng ý đó bằng hình học.



Hình 5.3. Định lý hàm ẩn nhìn bằng mắt, cho $E = wx^2 + (x - 1)^2$. Tăng trọng số w của phép đo tại $x = 0$ làm đường cong dốc hơn về phía trái, nên đáy trượt từ $x^* = 0,5$ sang $x^* = 0,25$. Đạo hàm $-H^{-1}B$ chính là độ dốc của đường trượt ấy. Nó chỉ cần độ cong tại đáy, chứ không cần biết bộ giải đã đi đường nào tới đó.

E. Ba điều kiện, và cả ba đều bị vi phạm trong SLAM

Định lý hàm ẩn đẹp nhưng có điều kiện. Nó cần x^* là điểm dừng thật, cần H khả nghịch, và cần E đủ trơn quanh nghiệm. Trong SLAM cả ba đều lung lay.

- **H suy biến theo cấu tạo.** Bài toán đơn ảnh có bảy hướng tự do đo. Ngay cả stereo-inertial vẫn còn vài hướng nếu chưa neo gốc. Theo những hướng đó H không khả nghịch. Cách chữa duy nhất là neo gốc hoặc cộng damping, và *giá trị damping đổi thì đạo hàm cũng đổi*. Bạn đang huấn luyện theo một **gradient** phụ thuộc vào một lựa chọn số học, không phải vào vật lý.
- **Bộ giải bị cắt ngắn.** Năm bước LM từ một khởi tạo tồi không cho một điểm dừng. Khi đó định lý hàm ẩn trả về đạo hàm của một bài toán mà bạn *chưa* giải. Sai lệch này hiếm khi được đo, và nó lớn nhất đúng ở những khung khó.
- **Có những quyết định không khả vi.** Đây là điều sâu nhất trong chương. So sánh χ^2 với ngưỡng, chọn inlier trong RANSAC, quyết định chèn khung then chốt — tất cả đều là hàm bậc thang. Đạo hàm của chúng bằng 0 hầu khắp nơi và không tồn tại tại chỗ nhảy. **Gradient** không nhìn thấy chúng.

Gạch đầu dòng cuối giải thích được toàn bộ thiết kế của các hệ trong chương này. Vì **không thể học một quyết định cứng**, mọi hệ đầu-cuối đều thay quyết định cứng bằng một *trọng số mềm liên tục*. DROID-SLAM không xuất ra mặt nạ inlier; nó xuất ra một con số tin cậy cho từng điểm ảnh. MAC-VO không xuất ra nhãn “điểm này tốt”; nó xuất ra một hiệp phương sai. Đó không phải một lựa chọn thẩm mỹ mà là hệ quả bắt buộc của việc muốn có đạo hàm.

→ cùng nguyên lý với B/09-thiet-ke-ham-mat-mat — lại là quy tắc “muốn học được thì phải làm cho trơn”, cùng lý do focal loss thay cho việc chọn cứng mẫu khó

F. Vì sao gradient nổ đúng ở khung khó — một phép tính EuRoC

Điều kiện “ H khả nghịch” nghe như một chi tiết kỹ thuật. Thực ra nó là chế độ hỏng nặng nhất trong huấn luyện, và nó tính ra được bằng số liệu bạn đã quen.

Xét một điểm bản đồ quan sát bằng stereo. Bỏ qua các hướng khác, giữ hai bậc tự do: dịch ngang và độ sâu. Với tiêu cự f và đường cơ sở b , độ bất định của hai hướng là

$$\sigma_x = \frac{Z}{f} \sigma_u, \quad \sigma_Z = \frac{Z^2}{fb} \sigma_d,$$

với σ_u là sai số định vị điểm ảnh và σ_d là sai số disparity. Lấy $\sigma_d = \sigma_u$ cho gọn, tỉ số rút lại còn một biểu thức rất sạch:

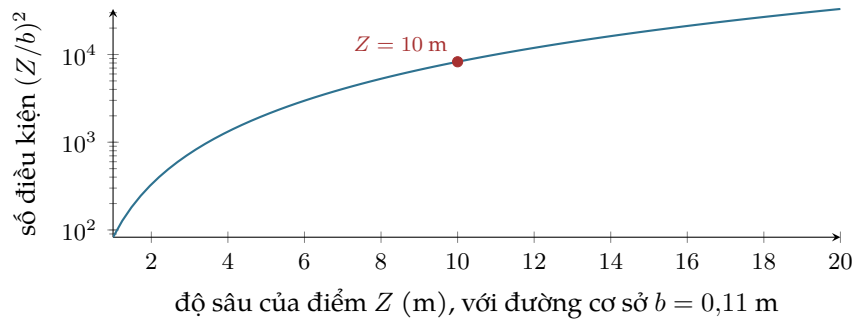
$$\frac{\sigma_Z}{\sigma_x} = \frac{Z}{b}.$$

Bây giờ thay số vào công thức — đây là phép thay số, không phải phép đo trên máy nào. Lấy thông số công khai của EuRoC, $b = 0,11$ m và $f \approx 460$ px, rồi *giả sử* $\sigma_u = 0,5$ px và một điểm ở $Z = 10$ m. Khi đó $\sigma_Z/\sigma_x = Z/b$ cho tỉ số cỡ chín chục lần: bất định theo hướng độ sâu lớn hơn bất định theo hướng ngang gần hai bậc độ lớn.

Ma trận thông tin là nghịch đảo bình phương của các độ lệch đó, nên khối 2×2 của H có số điều kiện khoảng $(Z/b)^2 \approx 8300$. Đây là con số quyết định. Trong công thức $v = H^{-1} \partial L / \partial x$, thành phần **gradient** theo hướng độ sâu bị nhân lên khoảng tám nghìn lần so với thành phần theo hướng ngang. Nói bằng lời: **ước lượng càng ít ràng buộc một hướng, thì đạo hàm theo hướng đó càng lớn**, vì một thay đổi nhỏ ở trọng số làm nghiệm trượt rất xa theo hướng yếu.

Hệ quả cho huấn luyện thì cụ thể và khó chịu. Các khung thị sai thấp, chuyển động gần như quay thuần túy, và ảnh mờ đều làm Z/b hiệu dụng lớn lên. Đó lại đúng là những khung bạn muốn mạng học từ chúng. Nên **gradient** từ mẫu quý nhất cũng là **gradient** nhiều nhất và dễ nổ

nhất. Mọi cách chữa thông dụng — cắt chuẩn *gradient*, tăng damping, bỏ qua mẫu có H xấu — đều làm giảm ảnh hưởng của đúng nhóm mẫu ấy. Hình 5.4 cho thấy số điều kiện tăng theo bình phương độ sâu.



Hình 5.4. Số điều kiện của khối hai bậc tự do (ngang, sâu) cho một điểm stereo trên cấu hình EuRoC. Đây là đường của công thức $(Z/b)^2$, không phải một tập điểm đo. Nó chính là hệ số khuếch đại mà H^{-1} áp lên *gradient* theo hướng độ sâu trong lượt ngược. Điều đáng nhớ là hình dạng: hệ số tăng theo bình phương độ sâu, nên tăng độ sâu gấp đôi thì nó tăng gấp bốn. Đây là lý do định lượng khiến huấn luyện qua một lớp tối ưu mất ổn định trên đúng những khung ít thị sai.

G. Khối nào trong hệ của bạn học được, khối nào không

Gộp hai kết luận vừa rồi — *gradient* chỉ chảy qua thứ liên tục, và nó nổ ở chỗ H xấu — ta có một bản kiểm kê rất cụ thể cho một hệ kiểu ORB-SLAM3. Bảng 5.1 là bảng đáng dán lên tường trước khi bắt đầu bất cứ thí nghiệm nào trong chương này.

Bảng 5.1. Kiểm kê các khối của một hệ stereo-inertial cổ điển theo tiêu chí “*gradient* có chảy qua được không”. Hai dòng in đậm là hai chỗ rẻ nhất để bắt đầu, vì chúng đã liên tục sẵn và không đòi thay kiến trúc.

Khối	Đầu ra	Liên tục?	Học được thể nào
Trích đặc trưng ORB	vị trí + mô tả nhị phân	không	Phải thay bằng bản đồ nhiệt và mô tả số thực; đó là chương 28
Ghép cặp + kiểm tra tỉ lệ	tập cặp khớp	không	Thay bằng ma trận gán mềm rồi mới lấy <i>gradient</i> qua nó
RANSAC và cổng χ^2	mặt nạ inlier	không	Thay hẳn bằng trọng số liên tục; không có cách nào khác
Ma trận thông tin cạnh	số thực dương	có	Học trực tiếp; đây là điểm vào rẻ nhất của cả chương
Bề rộng nhân robust	số thực dương	có	Học được, nhưng <i>gradient</i> yếu vì nhân robust bão hoà ở phần đuôi
Hiệu chỉnh chùm tia	nghiệm	có	Đi xuyên qua bằng định lý hàm ẩn, với điều kiện đã neo gốc
Chính sách chèn khung then chốt	quyết định đúng/sai	không	Không; phải giữ luật viết tay hoặc học bằng cách khác hẳn
Lấy ứng viên đóng vòng	xếp hạng + ngưỡng	một nửa	Học được phần <i>embedding</i> , không học được phần ngưỡng; chương 29
Kiểm chứng Sim3 + RANSAC	quyết định đúng/sai	không	Không
Tối ưu đồ thị tư thế	nghiệm	có	Đi xuyên qua được, nhưng đồ thị lớn nên lượt ngược đắt
Khởi tạo IMU	nghiệm + cổng chấp nhận	một nửa	Chỉ phần nghiệm; cổng chấp nhận vẫn rời rạc

Bảng này trả lời luôn câu hỏi “đầu-cuối tới mức nào”. Sáu trên mười một khối không cho *gradient* đi qua, và trong sáu khối đó có cả đóng vòng lẫn chèn khung then chốt — hai thứ chi phối sai số dài hạn của bạn. Nên không có hệ nào trong chương thật sự đầu-cuối theo nghĩa đen. Chúng chỉ đầu-cuối trên phần vốn đã trơn.

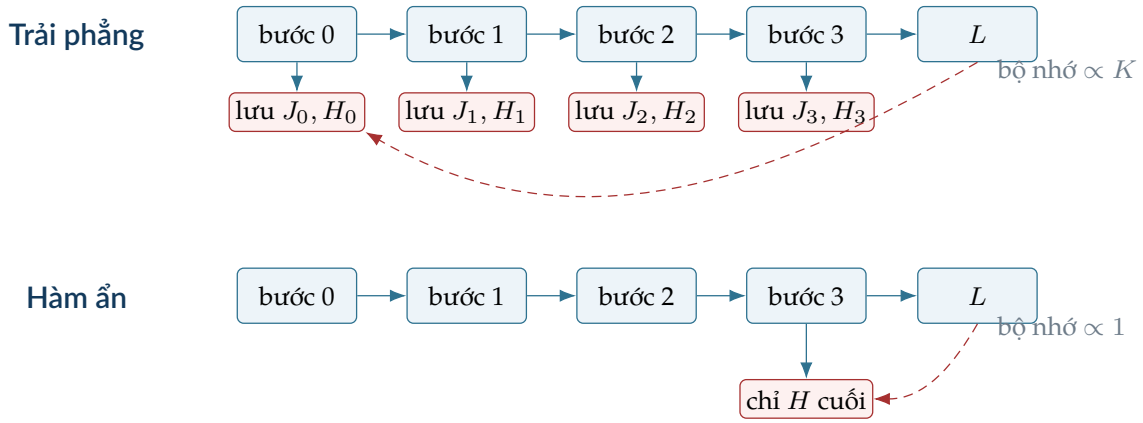
H. Ba cái giá, nói thẳng

Bảng 5.2 tổng kết cái được và cái mất. Cột cuối là cột đáng đọc nhất.

Bảng 5.2. Khả vi mua được gì và trả bằng gì. Cột “hỏng khi nào” là cột quyết định: nó cho biết trong hoàn cảnh nào một hệ khả vi trở nên tệ hơn hệ cổ điển, chứ không chỉ đắt hơn.

Chiều	Được gì	Trả bằng gì	Hỏng khi nào
Trọng số đo lường	Trọng số theo từng cặp khớp, biết mảnh ảnh này mờ hay sắc	Phải huấn luyện; cần chân trị quỹ đạo	Ra ngoài phân phối huấn luyện: trọng số vẫn tự tin, chỉ là sai
Bộ nhớ lượt ngược	—	Trái phằng: tỉ lệ với tích của số bước, số khung, số kênh và số vị trí. Hàm ẩn: không phụ thuộc số bước	Trái phằng buộc cửa sổ huấn luyện ngắn, nên không học được hành vi dài hạn
Ổn định số	—	Phải damping, cắt <i>gradient</i> , giới hạn số bước	H gần suy biến: thị sai thấp, quay thuần túy, ảnh mờ — đúng các khung cần học nhất
Chẩn đoán	—	Mất phần lớn	Quỹ đạo lệch mà không có χ^2 , không có số inlier, không có cạnh nào để đổ lỗi
Tính lặp lại	—	Phi tắt định của thư viện lọt vào trong bộ ước lượng	Hai lần chạy cho hai quỹ đạo; hết phân biệt được lỗi thuật toán với nhiễu

Chiều thứ tư đáng nói rõ hơn, vì với một người bảo trì hệ thống nó thường là chiều đắt nhất. Trong backend cổ điển, khi quỹ đạo lệch bạn có một chuỗi câu hỏi rất cụ thể để hỏi. Chi bình phương của cạnh nào lớn bất thường? Bao nhiêu cạnh bị nhân robust ép xuống? Hiệp phương sai biên của khung nào nở ra? Mỗi câu đều có một con số trả lời, và con số đó trở vào một dòng mã. Khi trọng số do một mạng sinh ra, câu hỏi “vì sao nó ra tư thế này” không còn câu trả lời ở mức đồ thị. Bạn vẫn in được phân dư. Bạn không truy được trọng số về nguyên nhân. Và điều tệ nhất là: với bộ giải, một trọng số sai trông y hệt một trọng số đúng.



Hình 5.5. Hai lối đi ngược qua một bộ giải lặp. Trái phẳng giữ mọi đại lượng trung gian, nên bộ nhớ tăng tuyến tính theo số bước lặp K ; đổi lại đạo hàm đúng với thuật toán đã chạy, kể cả khi nó chưa hội tụ. Định lý hàm ẩn chỉ cần ma trận H ở bước cuối, nên bộ nhớ không phụ thuộc K ; đổi lại đạo hàm chỉ đúng nếu bước cuối thật sự là một điểm dừng.

5.3 Hạ tầng khả vi: LieTorch, Theseus và BA-Net (Differentiable Infrastructure)

A. LieTorch: gradient sống trong không gian tiếp tuyến

Có một trục trặc rất cụ thể khi ghép autodiff với hình học. PyTorch chỉ biết tensor. Một phép biến đổi $SE(3)$ lưu dưới dạng ma trận 4×4 là 16 số; lưu dưới dạng quaternion cộng tịnh tiến là 7 số. Nhưng nhóm chỉ có 6 bậc tự do. Nếu để autodiff chạy trên biểu diễn nhúng, *gradient* có 16 hoặc 7 thành phần, và phần lớn trong đó trở ra ngoài đa tạp. Người ta chữa bằng cách chuẩn hoá lại quaternion sau mỗi bước, hoặc phạt độ lệch trực giao. Cả hai đều là vá vúi, và cả hai đều làm *gradient* sai một lượng không kiểm soát được.

LieTorch [98] sửa từ gốc. Nó lưu và lan truyền *gradient* trong *không gian tiếp tuyến*, tức đại số Lie. Một phép biến đổi vẫn được lưu bằng 7 số, nhưng đạo hàm của nó là một vectơ 6 chiều, và quy tắc chuỗi được viết lại bằng các Jacobian trái và phải của nhóm. Đó đúng là những Jacobian bạn đã gặp ở Barfoot [99]. Hệ quả thực dụng: không còn chuẩn hoá giữa đồ thị, số chiều *gradient* đúng bằng số bậc tự do, và bước cập nhật là một phép exp chứ không phải một phép cộng.

Bài học thiết kế lấy được ngay cả khi không dùng thư viện: **ghép một mạng vào một khối hình học thì hãy kiểm tra ngay số chiều của gradient**. Nếu nó lớn hơn số bậc tự do thì có một phần *gradient* đang chảy vào các hướng vô nghĩa. Phần đó sẽ hiện ra dưới dạng huấn luyện chậm hoặc không ổn định.

B. Theseus: bắt bạn phải chọn kiểu lượt ngược

Theseus [100] đóng gói cả một bộ giải phi tuyến thành một tầng PyTorch. Bạn khai báo biến và hàm chi phí, nó chạy Gauss-Newton hoặc Levenberg-Marquardt với bộ giải thừa, và nó khả vi. Điều đáng học không nằm ở sự tiện lợi mà ở chỗ nó *bắt người dùng khai báo kiểu lượt ngược*: trái phẳng đầy đủ, trái phẳng cắt ngắn, hay hàm ẩn. Thư viện không chọn hộ, vì không lựa chọn nào đúng cho mọi bài toán.

Đó là một tín hiệu trung thực đáng quý. Nó nói thẳng rằng đạo hàm qua một bộ giải là một *xấp xỉ có thiên lệch*, và bạn phải chọn nhận thiên lệch nào. Trước Theseus, gradSLAM [101] đã viết lại cả một đường ống SLAM dày đặc dưới dạng khả vi, chủ yếu để chứng minh rằng việc đó làm được.

Khác biệt giữa hai kiểu lượt ngược gọn tới mức viết ra được trong mười lăm dòng giả mã.

```
# Kiểu 1: trái phằng. Autodiff tu lo, nhưng giu moi buoc trong bo nho.
x = x0
for k in range(K):
    H, g = build_normal_equations(x, theta) # do thi phep tinh lon dan
    x = x - solve(H + lam*I, g)
L = loss(x); L.backward() # bo nho ~ K

# Kiểu 2: ham an. Chay bo giai KHONG ghi do thi, roi noi lai bang mot lop tu viet.
with torch.no_grad():
    x = newton_solve(x0, theta, K) # bo nho ~ 1
H, B = hessian_and_cross(x, theta) # chi tinh tai nghiem cuoi
v = solve(H, grad_of_loss_wrt_x(x)) # dung lai phan tich cua luot xuoi
grad_theta = -(B.t() @ v) # dinh ly ham an
```

Dòng `torch.no_grad()` chính là chỗ đánh đổi. Nó xoá đồ thị phép tính của vòng lặp, nên bộ nhớ tụt về hằng số. Đổi lại, mọi thứ đã xảy ra bên trong vòng lặp trở nên vô hình với **gradient**: bao nhiêu bước đã chạy, damping đã đổi thế nào, có bước nào bị từ chối không. Bảng 5.3 nói rõ ba lựa chọn và chỗ hỏng của từng cái.

Bảng 5.3. Ba kiểu lướt ngược qua một bộ giải lặp. Không kiểu nào đúng cho mọi bài toán, nên thư viện bắt bạn chọn — và lựa chọn đó phải được ghi lại cùng kết quả, giống như số seed.

Kiểu	Bộ nhớ	Đạo hàm là của cái gì	Hỏng khi nào
Trái phằng đầy đủ	Tỉ lệ với số bước K	Đúng thuật toán đã chạy	K lớn hoặc bài toán lớn: hết bộ nhớ, và gradient nhân dồn qua K tầng nên tắt hoặc nổ
Trái phằng cắt ngắn	Tỉ lệ với vài bước cuối	Xấp xỉ, thiên về hành vi cuối vòng lặp	Khởi tạo tồi: phân quan trọng của quỹ đạo tồi ưu nằm ở các bước đầu, và bạn vừa cắt bỏ nó
Hàm ẩn	Hằng số	Nghiem của bài toán <i>giả sử đã hội tụ</i>	Bộ giải bị cắt ngắn, hoặc H suy biến vì chưa neo gốc toạ độ

Một quy tắc thực dụng rút ra từ bảng trên. Nếu bộ giải của bạn hội tụ tốt và bạn neo gốc tử tế thì dùng hàm ẩn, vì nó rẻ hơn hẳn. Nếu bộ giải bị ngân sách thời gian ép dừng sớm — trường hợp thường gặp trong hệ thời gian thực — thì trái phằng cắt ngắn trung thực hơn, dù đắt. Và dù chọn cái nào, hãy ghi lựa chọn ấy vào cấu hình thí nghiệm. Nó ảnh hưởng tới kết quả ngang một siêu tham số.

C. BA-Net: học hệ số damping thay vì tự chỉnh

BA-Net [102] là công trình đầu tiên đặt nguyên một lớp **hiệu chỉnh chùm tia** (*bundle adjustment*) vào trong mạng và huấn luyện xuyên qua nó. Nó có ba ý tưởng, và giá trị của ba ý rất khác nhau.

- **Sai lệch trên đặc trưng thay vì trên độ sáng.** Phương pháp trực tiếp cổ điển tối ưu sai khác độ sáng, nên nó thừa hưởng giả định độ sáng không đổi và có lòng chảo hội tụ rất hẹp. BA-Net tối ưu trên các bản đồ đặc trưng học được. Bề mặt lỗi trơn hơn, lòng chảo rộng hơn, và kết quả bất biến với đổi sáng. Đây là ý tưởng bền nhất trong ba ý.
- **Damping học được.** Levenberg-Marquardt cổ điển chỉnh λ bằng luật kinh nghiệm: bước tốt thì chia mười, bước xấu thì nhân mười. BA-Net thay luật đó bằng một mạng nhỏ, đọc phần dư hiện tại rồi xuất ra một vectơ λ cho từng tham số. Vì λ vào bước cập nhật một cách trơn, cả bước LM vẫn khả vi, nên chính sách damping được huấn luyện theo lỗi tư thế cuối.
- **Độ sâu theo cơ sở.** Thay vì coi mỗi điểm ảnh là một ẩn, một mạng sinh ra khoảng một trăm bản đồ độ sâu cơ sở, và ẩn thật sự là các hệ số tổ hợp. Ý này là tổ tiên trực tiếp của CodeSLAM ở mục 6. Số phận của nó cũng báo trước số phận của CodeSLAM: cơ sở độ sâu là một tiên

nghiệm học được về hình dạng cảnh, nên nó tốt đúng trong phân phối huấn luyện và bịa ra bề mặt ở ngoài đó.

Có đáng không? Ý thứ nhất đáng, và quan trọng hơn là nó tách rời được: bạn có thể dùng đặc trưng học được làm mục tiêu căn chỉnh cho một phương pháp trực tiếp mà không cần bất cứ thứ gì khác của BA-Net. Dòng GN-Net đi đúng hướng này, huấn luyện đặc trưng sao cho bề mặt Gauss-Newton quanh nghiệm đúng là một lòng chảo sạch.

Ý thứ hai thì phải nghi ngờ. Một chính sách damping học được vẫn là *chính tay*, chỉ khác ở chỗ hằng số bây giờ được học từ một tập dữ liệu thay vì được gõ vào file cấu hình. Câu hỏi kiểm chứng đúng đắn không phải “nó có tốt hơn trên tập này không”. Câu hỏi đúng là “nó có còn tốt hơn trên một chuỗi có động lực khác hẳn không”. Mình chưa thấy bằng chứng mạnh cho về sau.

CẠM BẤY

Đừng nhầm *bộ giải khả vi* với *bộ giải tốt hơn*. Một lớp Gauss-Newton khả vi hội tụ đúng bằng tốc độ của Gauss-Newton thường, vì ở lượt xuôi nó *chính là* Gauss-Newton thường. Toàn bộ giá trị nằm ở lượt ngược, và lượt ngược chỉ có ý nghĩa nếu bạn thật sự huấn luyện một thứ gì đó bằng nó. Cấm một lớp tối ưu khả vi vào hệ thống rồi không huấn luyện gì cả thì bạn vừa đổi một bộ giải C++ đã tối ưu lấy một bộ giải Python chậm hơn, mà không nhận lại gì.

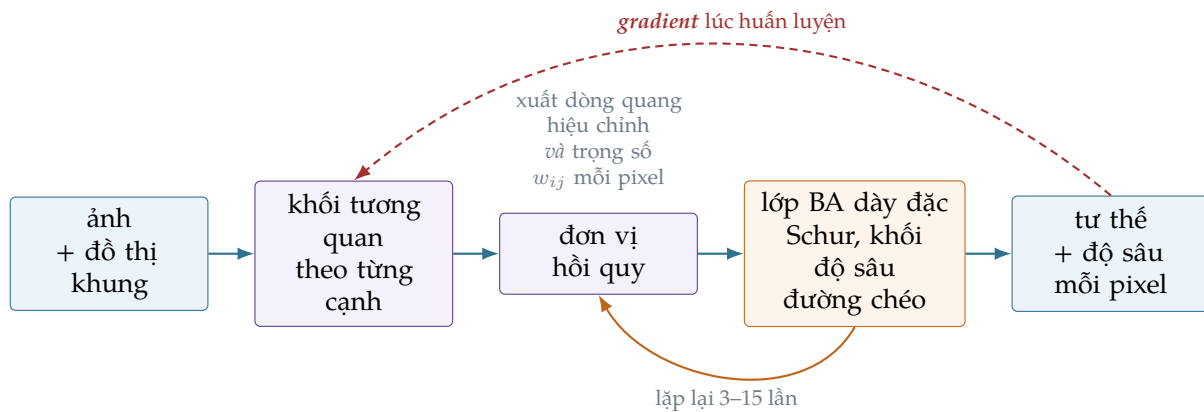
5.4 DROID-SLAM và cuộc thừa hoá sau nó (DROID-SLAM and the Sparsification Line)

A. Cơ chế của DROID-SLAM

DROID-SLAM [9] là hệ đầu-cuối thuyết phục nhất tính tới nay, và cấu tạo của nó đáng vẽ ra. Nó ghép hai thứ đã có: toán tử cập nhật hồi quy của RAFT [17] và một lớp hiệu chỉnh chùm tia dày đặc.

Vòng lặp chạy như sau. Hệ giữ một *đồ thị khung*, với nút là khung then chốt và cạnh là các cặp có chồng lẫn thị giác. Với mỗi cạnh, một khối tương quan được dựng từ hai bản đồ đặc trưng. Một đơn vị hồi quy đọc khối tương quan cùng dòng quang hiện tại rồi xuất ra hai thứ: một mục tiêu dòng quang đã hiệu chỉnh cho từng điểm ảnh, và một trọng số tin cậy cho từng điểm ảnh. Lớp hiệu chỉnh chùm tia dày đặc nhận hai thứ đó làm phép đo, rồi giải bài toán bình phương tối thiểu theo tư thế và theo nghịch đảo độ sâu từng điểm ảnh. Vì khối độ sâu là ma trận đường chéo, phần bù Schur cho ra một hệ camera rút gọn rất nhỏ. Nghiệm mới được đưa ngược lại cho đơn vị hồi quy, và vòng lặp tiếp tục.

Huấn luyện chạy trên TartanAir [103] với chân trị tư thế. *Gradient* đi xuyên qua lớp hiệu chỉnh chùm tia, về tới tận mạng đặc trưng.



Hình 5.6. Vòng lặp của DROID-SLAM. Điểm cốt lõi không phải mạng hồi quy, cũng không phải bộ giải, mà là *vòng phản hồi màu cam* giữa hai cái đó: mạng nhìn thấy nghiệm mà bộ giải vừa trả về rồi sửa phép đo cho lần sau. Mùi tên đứt là đường *gradient* lúc huấn luyện. Nhờ nó, mạng học cách xuất ra dòng quang và trọng số mà bộ giải dùng được, chứ không phải dòng quang trông đẹp mắt.

B. Vì sao một hệ 144 nghìn ẩn vẫn giải được

Với cửa sổ 30 khung then chốt ở $1/8$ độ phân giải, lớp hiệu chỉnh chùm tia này có khoảng 144 nghìn ẩn hình học. Con số đó nghe như một hệ không giải nổi trong thời gian thực. Lý do nó vẫn giải được là một tính chất cấu trúc bạn đã quen, chỉ khác quy mô.

Trong hiệu chỉnh chùm tia cổ điển, mỗi điểm bản đồ đóng góp một khối 3×3 trên đường chéo của khối cấu trúc, nên khử chúng bằng phân bù Schur là rẻ. Ở đây còn dễ hơn. Mỗi ẩn là *một* nghịch đảo độ sâu của *một* điểm ảnh, và nó không nối trực tiếp với điểm ảnh nào khác. Khối cấu trúc vì thế là một ma trận đường chéo thật sự, không phải khối đường chéo. Nghịch đảo nó là 144 nghìn phép chia.

Sau khi khử, hệ còn lại chỉ là hệ camera rút gọn cỡ $6N \times 6N$. Với $N = 30$ khung then chốt thì đó là ma trận 180×180 . Phân tích Cholesky một ma trận cỡ đó là chuyện vặt so với phần còn lại của vòng lặp. Nói cách khác, **phần đắt của DROID-SLAM không nằm ở bộ giải**. Nó nằm ở việc dựng khối tương quan rồi chạy mạng hồi quy để sinh ra 144 nghìn phép đo cùng 144 nghìn trọng số, lặp lại tới mười lăm lần cho mỗi khung.

Kết luận thực dụng rút ra khá rõ: nếu định thử một ý tưởng theo hướng này thì đừng tối ưu bộ giải. Ba chỗ cắt được là độ phân giải, số bước lặp, và số cạnh trong đồ thị khung. Cả ba đều là tham số cấu hình chứ không phải kiến trúc, nên thử rất nhanh.

C. Vì sao nó chính xác

Bốn cơ chế cộng lại, và chỉ một trong bốn liên quan tới chữ “khả vi”.

- **Dày đặc là dư thừa.** Mỗi điểm ảnh bỏ một lá phiếu, nên vài nghìn phiếu sai không lay chuyển được nghiệm. Một hệ thừa chỉ có vài trăm cặp khớp thì có.
- **Trọng số theo từng điểm ảnh.** Mạng hạ trọng số vùng trời, vật thể chuyển động, bề mặt phản chiếu. Kiểm tra tỉ lệ khoảng cách mô tả không làm được việc đó, vì nó không nhìn ngữ cảnh.
- **Sửa lặp lại.** Đơn vị hồi quy thấy nghiệm hiện tại rồi sửa phép đo, nên hệ tự gỡ được khởi tạo tồi. Một đường ống một chiều không có cơ chế này.
- **Hình học vẫn được ép.** Đầu ra đi qua một bộ giải thật, nên nó nhất quán đa khung. Nó không trôi theo kiểu một mạng hồi quy tư thế.

Cơ chế thứ ba mới là chỗ tính khả vi trả tiền. Không có *gradient* xuyên qua bộ giải thì mạng sẽ được huấn luyện để dự đoán dòng quang khớp với chân trị dòng quang. Có *gradient*, nó được huấn luyện để dự đoán dòng quang làm cho *tư thế* đúng. Hai mục tiêu đó khác nhau, và khác biệt nằm đúng ở các điểm ảnh có hình học xấu.

D. Vì sao nó đắt

Đếm ẩn số trước. Ảnh 640×480 ở $1/8$ độ phân giải cho 4 800 ẩn nghịch đảo độ sâu cho mỗi khung then chốt. Với 30 khung trong cửa sổ là 144 000 ẩn hình học, so với 6 ẩn tư thế mỗi khung. Bài toán vẫn giải được vì khối độ sâu là đường chéo, nhưng mọi tensor đều mang kích thước đó.

Đếm bộ nhớ sau, và đếm bằng công thức chứ không quy ra một con số. Một khối tương quan toàn phần giữa hai khung giữ một mục cho *mỗi cặp* vị trí, tức bình phương số vị trí ở độ phân giải đang chạy — và đó mới là một cạnh, một mức. Đồ thị khung có nhiều cạnh cho mỗi nút, nên tổng bộ nhớ tỉ lệ với số cạnh nhân bình phương số vị trí. Chữ “bình phương” đó giải thích vì sao bộ nhớ tăng rất nhanh theo độ phân giải, vì sao nó tăng theo độ dài chuỗi, và vì sao các công trình sau phải thêm cơ chế quản lý bộ nhớ.

Còn phần cứng. Các tác giả công bố hệ trên GPU để bàn cao cấp, không phải trên máy nhúng. Một Jetson Orin có ngân sách tính toán và băng thông bộ nhớ thấp hơn hẳn một GPU để bàn cùng thế hệ, và khoảng cách ấy thuộc loại bậc độ lớn chứ không phải vài chục phần trăm. Đây là hướng, không phải số đo: mình không có phép đo nào cho phát biểu này, và muốn một con số thì phải tự chạy. Thêm nữa, DROID-SLAM không dùng IMU, tức là bỏ đi tài sản mạnh nhất của một hệ stereo-inertial.

E. Một bảng tính ngân sách Orin, làm trong mười phút

Trước khi tải bất cứ mã nguồn nào về, có một việc rẻ hơn nhiều: đếm phép tính trên giấy. Nó không cho bạn con số đúng, nhưng nó cho bạn biết bạn đang hụt *hai lần* hay *hai mươi lần*. Hai kết luận đó dẫn tới hai quyết định hoàn toàn khác nhau.

Giả sử mục tiêu là vài chục khung mỗi giây, và frontend cổ điển của bạn đã ăn một phần đáng kể ngân sách mỗi khung. Bảng 5.4 liệt kê các khoản mục, cách tính từng khoản, và mức nặng nhẹ tương đối giữa chúng.

Bảng 5.4. Các khoản mục ngân sách của một hệ đầu-cuối kiểu DROID trên phần cứng nhúng. Cột giữa là công thức đếm; cột cuối là *xếp hạng định tính* giữa các khoản, kèm lý do của thứ hạng đó. Bảng này cố tình không có con số: nó dùng để biết nên cắt ở đâu, còn độ lớn thật thì chỉ phép đo trên chính thiết bị của bạn mới nói được.

Khoản mục	Cách tính	Thứ hạng, và vì sao
Mạng trích đặc trưng	Đếm phép nhân cộng của backbone ở độ phân giải đầu vào thật, không phải ở 224×224	Nặng. Chi phí tỉ lệ thẳng với số điểm ảnh, nên đo ở độ phân giải trong bài rồi suy ra độ phân giải thật là cách hệt nhiều lần
Khối tương quan	$(H/8)(W/8)$ nhân $(H/8)(W/8)$ cho mỗi cạnh, nhân số byte	Nặng nhất về bộ nhớ. Đây là khoản duy nhất tăng theo <i>bình phương</i> số vị trí, lại còn nhân với số cạnh của đồ thị khung
Vòng lặp hồi quy	Nhân chi phí một bước với số bước, thường 3 tới 15	Khoản đội chi phí lớn nhất, vì nó nhân thẳng mọi khoản phía trên lên vài lần tới hơn chục lần
Giải hệ BA	Hệ camera rút gọn $6N \times 6N$, với N là số khung trong cửa sổ	Nhẹ nhất, và hay bị đổ oan. Nó nhỏ hơn phần sinh phép đo nhiều bậc, nên tối ưu nó gần như không đổi được gì
Chuyển dữ liệu	Tổng byte đi qua bộ nhớ chia cho băng thông thật của thiết bị	Thường mới là nút cổ chai thật, và chỉ lộ ra khi bạn đếm byte thay vì đếm phép tính

Cách dùng bảng quan trọng ngang bản thân bảng. Nhân các khoản theo đúng công thức ở cột giữa, chia cho thông lượng *đo được* của thiết bị — không phải con số đỉnh trên tờ thông số, vốn luôn cao hơn nhiều — rồi so với ngân sách mỗi khung. Thứ bạn cần lấy ra từ phép tính đó không phải một con số mili-giây, mà đúng một bit thông tin: bạn đang hệt *một chút* hay hệt *một bậc độ lớn*. Hệt một chút thì đi đo thật, vì phép tính trên giấy không đủ chính xác để kết luận. Hệt một bậc thì đừng đo vội, vì kết luận đã rõ rồi: phải cắt hẳn một thừa số trước đã. Và đúng ba chỗ cắt được là: hạ độ phân giải, giảm số bước lặp, hoặc giảm mật độ. Mục kế tiếp là câu chuyện của lựa chọn thứ ba.

↔ **cùng nguyên lý với** B/11-gioi-han-tinh-toan — đây chính là bài toán roofline của chương 11 áp cho một hệ SLAM: đếm phép tính và đếm byte trước, đo sau

F. Thừa hoá: DPVO, DPV-SLAM và MAC-VO

Nếu chỗ đất là mật độ, thì câu hỏi tự nhiên là mật độ có cần thiết không.

I Mạch — từ dày đặc về thưa

1 Vấn đề

DROID-SLAM chính xác nhưng cần phần cứng GPU để bàn cao cấp và một lượng bộ nhớ vượt xa máy nhúng, nên không đưa lên robot được.

Giải pháp

DPVO [104] bỏ trường dòng quang dày đặc. Thay vào đó nó theo vết một tập nhỏ các *mảnh ảnh* vuông, mỗi khung khoảng một trăm mảnh, mỗi mảnh mang đúng một ẩn nghịch đảo độ sâu. Vẫn cùng toán tử hồi quy và cùng lớp hiệu chỉnh chùm tia, chỉ khác là chạy trên đồ thị mảnh.

Giả định

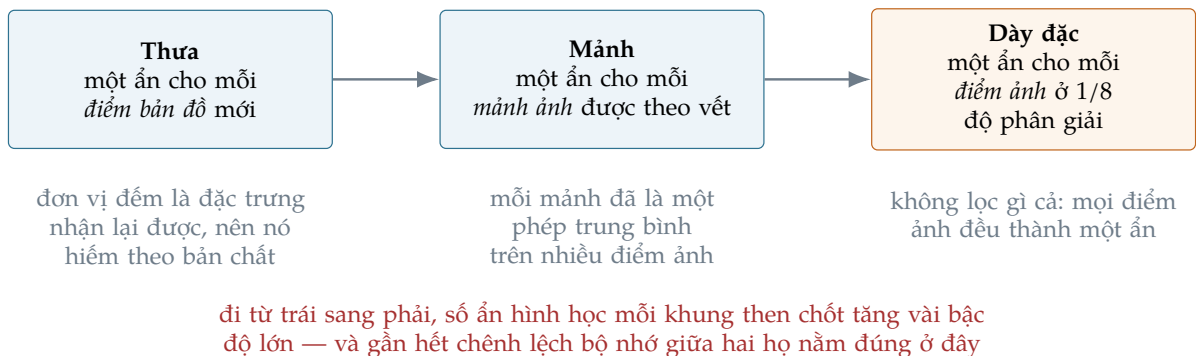
Ích lợi thống kê của việc dày đặc bão hoà rất sớm, vì bản thân một mảnh đã là một phép trung bình trên nhiều điểm ảnh.

Hệ quả

Số ẩn hình học mỗi khung giảm vài bậc độ lớn, và theo báo cáo của chính các tác giả thì hệ nhẹ đi đủ nhiều để chạy trên một GPU để bàn duy nhất thay vì nhiều card. Mình không kiểm chứng được các con số tốc độ và bộ nhớ họ đưa ra, nên ở đây chỉ giữ lại hướng của thay đổi.

- Giá phải trả** Mất khả năng tự nhiên cho ra bản đồ dày đặc, và mất một phần sức chống chịu khi cảnh có rất ít vân.
- 2 Vấn đề** DPVO là odometry: không có bản đồ toàn cục, nên nó trôi và không bao giờ đóng vòng được.
- Giải pháp** DPV-SLAM [105] thêm hai cơ chế đóng vòng. Một cơ chế dựa trên chính đồ thị mảnh cho các vòng gần, và một cơ chế truy hồi ảnh kiểu cổ điển cho các vòng xa. Sau đó chạy tối ưu đồ thị tư thế toàn cục.
- Hệ quả** Thành một hệ SLAM đầy đủ, vẫn gần thời gian thực trên một GPU.
- Giả định** Bài học còn quan trọng hơn kết quả: *chưa ai học được cách thay thế đóng vòng và tối ưu đồ thị tư thế*. Ngay cả một hệ đầu-cuối cũng phải mượn lại đúng hai khối cổ điển đó.
- 3 Vấn đề** Trọng số vô hướng cho mỗi cặp khớp vẫn quá thô. Bất định của một cặp khớp stereo có hình dạng rất dẹt: dài theo tia nhìn và ngắn theo phương ngang.
- Giải pháp** MAC-VO [106] dự đoán cả một hiệp phương sai cho từng cặp khớp. Quan trọng là hiệp phương sai ấy *có đơn vị mét*, tức nó tính tới việc sai số độ sâu nở ra khi disparity nhỏ. Con số đó được dùng hai lần: để chọn điểm nào đáng giữ, và để làm ma trận thông tin trong bài toán tư thế.
- Hệ quả** Đây là cầu nối trực tiếp sang *F/34-imu-bat-dinh-va-trien-khai*. Thứ được học không còn là đặc trưng hay dòng quang, mà chính là *ma trận thông tin* — đúng đại lượng bạn đang đặt bằng tay.
- Giá phải trả** Vẫn cần huấn luyện có chân trị, và bất định học được kế thừa mọi giới hạn phân phối ở mục 5. Chi tiết số của công trình này mình chưa tự kiểm chứng, nên chỉ nêu ý tưởng.

Hình 5.7 đặt ba mức mật độ cạnh nhau để thấy khoảng cách thật sự lớn tới đâu.



Hình 5.7. Ba mức mật độ, đọc theo *đơn vị đếm* chứ không theo con số. Điều đáng nhớ không phải mỗi họ giữ bao nhiêu ẩn, mà là mỗi họ lấy gì làm đơn vị: một hệ thưa đếm theo điểm bản đồ nhận lại được, DPVO đếm theo mảnh ảnh, DROID đếm theo điểm ảnh. Vì mỗi bậc trong ba bậc đó cách nhau vài bậc độ lớn về số lượng, thứ tự này quyết định gần như toàn bộ chênh lệch bộ nhớ giữa hai họ — nhiều hơn hẳn mọi khác biệt về cài đặt.

G. Đánh giá theo bốn câu hỏi

Nó giải quyết việc gì. Đóng góp thật của dòng này là trọng số theo từng cặp khớp và vòng sửa lặp lại. Hai thứ đó chạm đúng vào chuyển động nhanh và ảnh mờ. Đó là các trường hợp mà kiểm tra tỉ lệ khoảng cách mô tả cho ra hoặc quá ít cặp, hoặc quá nhiều cặp sai.

Có đáng làm không. Với một hệ stereo-inertial đã tinh chỉnh chạy trên Orin thì chưa. Ba lý do, theo thứ tự nặng dần. Ngân sách tính toán chênh một bậc độ lớn. Toàn bộ dòng này bỏ IMU, nên

bạn sẽ đánh đổi thứ đang cứu bạn ở những khung mờ để lấy thứ hứa cứu bạn ở những khung mờ. Và tính lặp lại sứt — một hệ bạn không chẩn đoán được là một hệ bạn không bảo trì được.

Học gì từ thiết kế. Ba bài học tách rời được, xếp theo mức dễ áp dụng. Dùng trọng số liên tục thay cho quyết định inlier nhị phân. Đưa nghiệm hiện tại quay lại cho tầng sinh phép đo, thay vì chạy một chiều. Và giữ đồ thị khung như một cấu trúc dữ liệu tường minh, thay vì để nó ẩn trong danh sách kề của bản đồ.

→ [tiền đề F/30-do-sau-dong-quang-va-mo-hinh-nen](#) — chất lượng của cả họ này bị chặn trên bởi chất lượng dòng quang, tức bởi đúng dòng RAFT mà chương 30 bàn

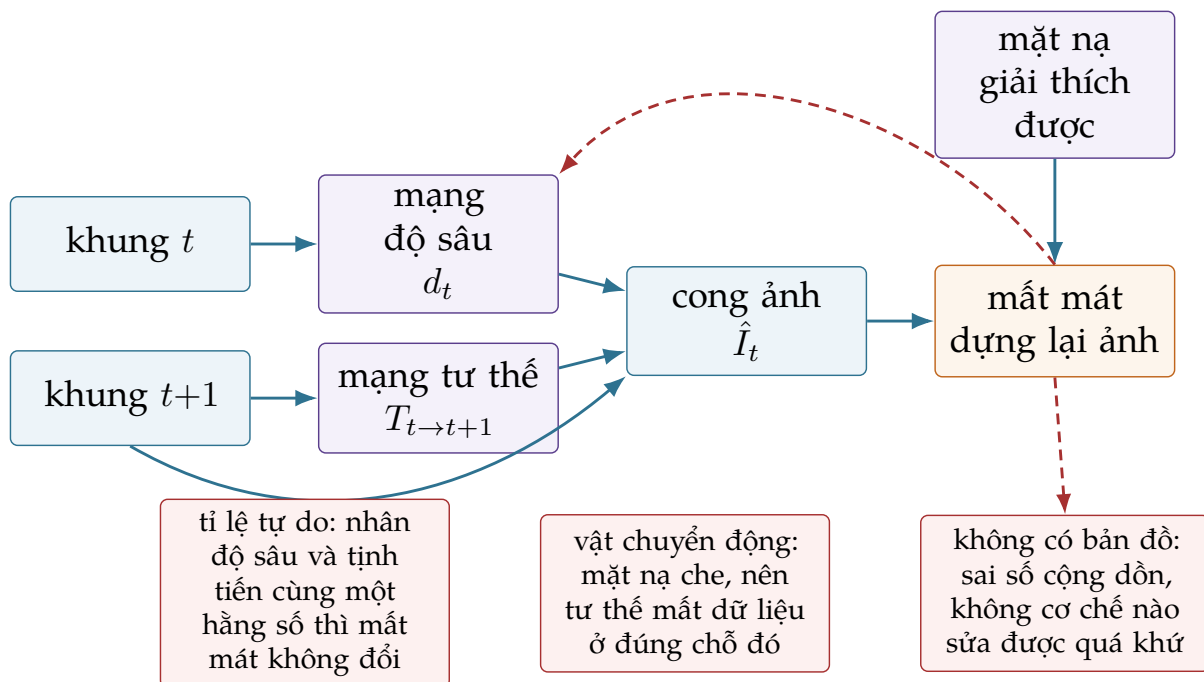
5.5 VO tự giám sát và giới hạn thật của nó (Self-Supervised VO)

A. Bài toán và cơ chế của SfM-Learner

Chân trị tư thế đắt. Cần một hệ đo động ngoài, hoặc một buổi chạy COLMAP dài. Câu hỏi của SfM-Learner [107] là: có thể huấn luyện chỉ bằng video thô không?

Cơ chế gọn và đẹp. Hai mạng cùng học. Mạng độ sâu nhận một khung và xuất bản đồ độ sâu. Mạng tư thế nhận một cặp khung và xuất phép biến đổi giữa chúng. Có độ sâu, có tư thế, có nội tham số camera thì *dựng lại* được khung đích từ khung nguồn bằng phép cong ảnh. **Hàm mất mát** (*loss function*) là chênh lệch giữa ảnh dựng lại và ảnh thật. Không cần nhãn nào.

Chi tiết cấu cả ý tưởng là *mặt nạ giải thích được*. Đó là một đầu ra thứ ba, một con số mỗi điểm ảnh, cho phép mạng nói “điểm này tôi không giải thích được, đừng phạt tôi”. Nó xử lý vật thể chuyển động, vùng bị che, vùng ra ngoài khung. Nhưng nó có một nghiệm suy biến hiển nhiên: đặt mặt nạ bằng 0 ở mọi nơi thì mất mát bằng 0. Nên phải cộng một số hạng chính quy kéo mặt nạ về 1. Điểm này đáng ghi nhớ, vì nó là mẫu chung của mọi thiết kế tự giám sát: **cho mô hình quyền bỏ qua dữ liệu thì phải bắt nó trả tiền cho quyền đó**.



Hình 5.8. Vòng huấn luyện tự giám sát kiểu SfM-Learner, và ba chỗ hỏng gắn liền với nó. Tín hiệu dạy duy nhất là “dựng lại khung t từ khung $t+1$ thì phải khớp”. Ba hộp đỏ không phải lỗi cài đặt mà là hệ quả trực tiếp của chính hàm mất mát đó: mặt nạ mua sự bền vững bằng cách vớt dữ liệu, tỉ lệ không quan sát được từ ảnh đơn, và không có trạng thái nào để một lần quan sát lại có thể sửa quá khứ.

B. Bốn giới hạn, nói thẳng

Đây là mục một kỹ sư SLAM cần đọc kỹ nhất, vì các bảng kết quả trong dòng này rất dễ đọc nhầm.

- **Mơ hồ tỉ lệ.** Với ảnh đơn, nhân độ sâu với s và nhân tịnh tiến với s cho ra đúng cùng một ảnh dựng lại. Mắt mắt không phân biệt được hai trường hợp. Nên đầu ra chỉ đúng tới một hằng số chưa biết, và hằng số đó còn trôi giữa các cảnh. Nhiều bảng KITTI trong dòng này căn tỉ lệ với chân trị *theo từng chuỗi* trước khi tính lỗi. Một robot không làm được việc đó, nên con số ấy không phải con số odometry dùng được. UnDeepVO [108] chữa bằng cách huấn luyện trên cặp stereo — đường cơ sở đã biết bơm tỉ lệ mét vào — rồi chạy đơn ảnh lúc suy luận.
- **Không có bản đồ, nên không có đóng vòng.** Mạng tư thế là phép ước lượng chết theo từng cặp khung. Sai số cộng dồn và không gì kéo nó về được. Toàn bộ độ chính xác của hệ thống của bạn đến từ việc quan sát lại cùng một điểm bản đồ qua nhiều khung then chốt. Ở đây cơ chế đó không tồn tại.
- **Không có bảo đảm, và hỏng êm.** Không hiệp phương sai, không phần dư để kiểm, không cổng χ^2 . Khi sai, nó sai một cách mượt mà và tự tin. So sánh với hệ của bạn: khi sắp mất bám, số inlier tụt trước, và bạn thấy được.
- **Phụ thuộc phân phối huấn luyện.** DeepVO [109] hồi quy tư thế trực tiếp và rất tốt trên KITTI. Một phần lý do là nó học luôn *tiên nghiệm chuyển động của một chiếc ô tô*: đi thẳng, tốc độ trong một dải hẹp, góc lệch nhỏ. Đó là một cơ chế thật chứ không phải lời che. Đưa mô hình sang camera cầm tay hoặc drone thì tiên nghiệm ấy sai, và nó sai theo hướng có hệ thống.

C. Mơ hồ tỉ lệ, nhìn bằng một phép tính

Giới hạn thứ nhất đáng được làm cho cụ thể, vì nó là chỗ nhiều người đọc bảng kết quả bị lừa.

Phép cong ảnh dùng đúng một biểu thức. Điểm ảnh u ở khung nguồn được đưa về khung đích qua độ sâu $d(u)$ và phép biến đổi (R, t) . Bây giờ thay $d \rightarrow s d$ và $t \rightarrow s t$ với cùng một hằng số $s > 0$. Toạ độ chiếu ra không đổi, vì s rút gọn giữa tử và mẫu. Ảnh dựng lại giống hệt, nên mắt mắt giống hệt. Kết luận: hàm mắt mắt *không chứa thông tin* nào về s .

Sự kiện nhỏ này dẫn tới một hệ quả lớn khi đọc bảng. Giả sử — đây là một phép tính giả định, không trích từ bài nào — một hệ báo lỗi tịnh tiến vài phần trăm chiều dài quỹ đạo, sau khi đã căn tỉ lệ theo từng chuỗi. Nếu bạn không được căn tỉ lệ, và tỉ lệ ước lượng lệch mười phần trăm, thì riêng sai lệch tỉ lệ đã sinh ra lỗi bằng mười phần trăm chiều dài quỹ đạo — lớn hơn hẳn con số được báo cáo, và lớn dần theo quãng đường đi. Con số trong bảng khi đó nói về *hình dạng* quỹ đạo, không nói về *vị trí*. Với một robot cần biết mình đang ở đâu, hai đại lượng đó không thay thế nhau được.

Cách kiểm nhanh khi đọc một bài báo trong dòng này chỉ có hai câu hỏi. Tỉ lệ được lấy từ đâu: từ stereo, từ IMU, hay từ chân trị? Và nếu từ chân trị thì căn một lần cho cả chuỗi hay căn lại theo từng đoạn? Căn theo từng đoạn là dấu hiệu tỉ lệ đang trôi, và độ trôi đó chính là thứ bị giấu đi.

D. TartanVO và cách chữa đúng chỗ

TartanVO [110] là câu trả lời có kỷ luật cho gạch đầu dòng cuối. Hai thủ thuật của nó đáng học kể cả khi bạn không dùng mô hình.

Thứ nhất, **bỏ tỉ lệ ra khỏi hàm mất mát**. Ép mạng học tỉ lệ tuyệt đối từ ảnh đơn thì nó buộc phải học tỉ lệ đặc thù của tập dữ liệu. Chuẩn hoá vectơ tịnh tiến trong hàm mất mát thì mạng chỉ còn phải học hướng, và hướng thì tổng quát hoá được.

Thứ hai, **đưa mô hình camera vào đầu vào**. Nối thêm vài kênh mã hoá nội tham số đã chuẩn hoá vào tensor đầu vào. Nhờ đó một mô hình duy nhất phục vụ nhiều camera có tiêu cự và độ méo khác nhau, thay vì phải huấn luyện lại cho từng camera.

Kết quả là một mô hình chạy qua nhiều tập dữ liệu mà không tinh chỉnh lại. Độ chính xác khiêm tốn, nhưng tính tổng quát là thật. Bài học chung: muốn tổng quát hoá thì lấy đại lượng đặc thù tập dữ liệu ra khỏi hàm mất mát, và đưa thông tin thiết bị vào đầu vào.

E. Nhánh giữ lại bộ giải cổ điển

Bốn công trình còn lại trong mục này đáng nhắc vì chúng chọn một kiến trúc khác hẳn. Với người đọc này, đó mới là kiến trúc đáng dùng.

- **DeepTAM** [111] tách tracking và mapping đúng như một hệ cổ điển rồi học từng phần, thay vì trộn cả hai vào một mạng.
- **DeepV2D** [112] luân phiên một khối độ sâu và một khối chuyển động, trong đó khối chuyển động là một bước Gauss-Newton khả vi. Nó là cầu nối giữa mục này và mục 4.
- **MonoRec** [113] học độ sâu đa khung có xử lý vật chuyển động, nhưng *lấy tư thế từ một hệ SLAM cổ điển*. Nó không cố thay bộ ước lượng tư thế.
- **TANDEM** [114] đi xa nhất theo hướng đó. Tracking bằng phương pháp trực tiếp cổ điển, còn phần học được chỉ lo dựng bản đồ dày đặc, rồi hợp nhất vào một trường TSDF.

Bốn công trình trên vẽ ra một mẫu thiết kế rõ ràng: **giữ bộ ước lượng tư thế cổ điển, học phần dày đặc**. Với một hệ stereo-inertial đã chạy tốt, đây là mẫu duy nhất trong cả mục 5 mà mình cho là đáng thử, vì nó không đặt cược thứ đang hoạt động.

I 5.6 Biểu diễn ẩn: nén hình học vào một mã ngắn (Latent Scene Representations)

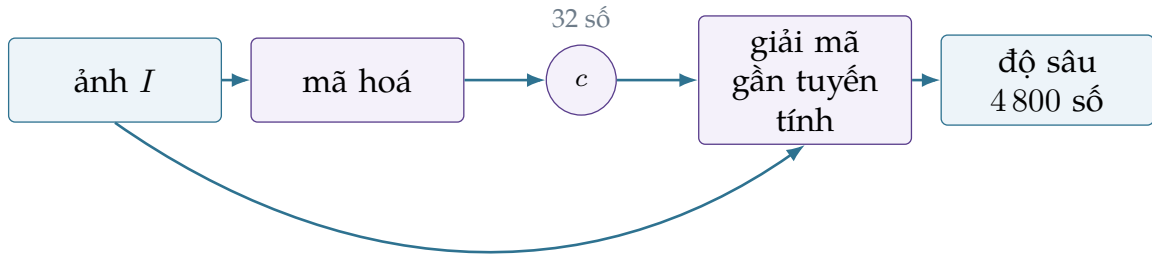
A. Vấn đề mà mã ngắn định giải

Bản đồ độ sâu dày đặc của một khung có cỡ 10^5 ẩn. Đưa từng ấy ẩn vào một bài toán bình phương tối thiểu thì hoặc phải chính quy hoá bằng luật kinh nghiệm, hoặc phải có GPU. Nhưng hình học của một căn phòng thật không có 10^5 bậc tự do. Tường phẳng, sàn phẳng, vật thể có hình dạng quen thuộc. Hình học ấy nằm gần một đa tạp số chiều thấp.

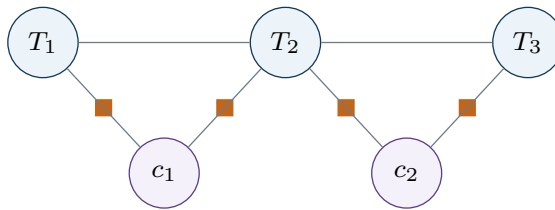
CodeSLAM [115] rút ra kết luận trực tiếp. Học một bộ mã hoá đưa ảnh về một *mã ngắn* c cỡ vài chục chiều, cùng một bộ giải mã $D(c, I)$ trả về bản đồ độ sâu. Rồi đặt c vào đồ thị nhân tử *ngay cạnh các nút tư thế*, và tối ưu chung.

Chi tiết kỹ thuật làm ý tưởng chạy được ít khi được nhắc, nhưng nó là mấu chốt. Bộ giải mã được thiết kế để *gần tuyến tính theo c* khi đã cố định ảnh. Nhờ đó $\partial D / \partial c$ tính một lần cho mỗi khung then chốt rồi dùng lại cho mọi bước Gauss-Newton. Không có tính chất đó thì mỗi lần tuyến tính hoá tốn một lượt ngược qua mạng, và bài toán hết khả thi thời gian thực.

Bộ mã hoá và bộ giải mã



Đồ thị nhân tử



mỗi nút mã thay cho 4800 ẩn độ sâu, nhưng nó nối tới mọi tư thế cùng quan sát khung đó

Hình 5.9. Ý tưởng của CodeSLAM. Bộ giải mã được thiết kế gần tuyến tính theo mã c , nên $\partial D/\partial c$ là một ma trận cố định, tính một lần cho mỗi khung then chốt. Nhờ đó mã trở thành một biến bình thường trong đồ thị nhân tử, đứng ngang hàng với tư thế, và hình học dày đặc được tối ưu chung với quỹ đạo bằng đúng bộ giải bạn đã có.

B. Đếm ẩn và đếm chi phí tuyến tính hoá

Ba con số cho thấy vì sao mã ngăn hấp dẫn tới vậy, và cũng cho thấy chỗ nó âm thầm đắt.

Số ẩn. Một bản đồ độ sâu ở 1/8 của ảnh 640×480 có 4800 ẩn. Một mã 32 chiều thay thế cả từng ấy. Tỉ số nén là 150 lần. Khối cấu trúc trong hệ phương trình chuẩn co từ 4800×4800 đường chéo xuống 32×32 đặc, và biên hoá một khối 32×32 là chuyện vặt.

Chi phí Jacobian. Đây mới là chỗ ẩn chi phí. Ma trận $\partial D/\partial c$ có kích thước 4800×32 , tức khoảng 154 nghìn số. Nếu bộ giải mã không gần tuyến tính theo c thì ma trận đó phải tính lại ở mỗi lần tuyến tính hoá, và mỗi lần tính là 32 lượt ngược qua bộ giải mã. Với năm bước Gauss-Newton là 160 lượt ngược cho mỗi khung then chốt. Không hệ thời gian thực nào chịu nổi.

Vì sao gần tuyến tính lại cứu được. Nếu $D(c, I) \approx D_0(I) + J(I)c$ thì J chỉ phụ thuộc ảnh, nên tính một lần khi khung then chốt được tạo rồi dùng lại mãi. Chi phí mỗi bước Gauss-Newton trở về đúng một phép nhân ma trận, y như một cạnh chiếu lại thông thường. Toàn bộ tính khả thi của CodeSLAM nằm ở dòng vừa rồi. Và giá của nó là điều kiện gần tuyến tính chỉ đúng trong một lân cận nhỏ, tức lý do thứ hai ở mục C bên dưới.

C. Dòng công trình

SceneCode [116] mở rộng mã sang cả nhãn ngữ nghĩa, để hình học và ngữ nghĩa cùng được tinh chỉnh. DeepFactors [117] đưa mã vào một đồ thị nhân tử thật với ba loại nhân tử — quang trắc, chiếu lại, và hình học thưa — rồi chạy được thời gian thực. NodeSLAM [118] đổi phạm vi: mỗi vật thể có một mã hình dạng riêng, và tối ưu chạy trên tư thế cộng hình dạng của từng vật. CodeMapping [119] thực dụng nhất: giữ nguyên ORB-SLAM3 làm xương sống thưa, rồi chạy một tầng bản đồ dày đặc bằng mã ở bên cạnh, có điều kiện trên chính các điểm thưa đó.

D. Vì sao hướng này chưa thẳng

Câu hỏi này giá trị hơn danh sách trên. Ý tưởng đẹp, cài đặt chạy được, người làm giỏi. Vậy tại sao tám năm sau nó vẫn không nằm trong hệ sản xuất nào? Bốn lý do, xếp theo mức nền tảng.

Thứ nhất, sai số của tiên nghiệm có tương quan chứ không thưa. Toàn bộ phòng thủ của ước lượng robust dựa trên một giả định: điểm ngoại lai thì ít và độc lập với nhau. Một bộ giải mã huấn luyện trên ảnh RGB-D trong nhà không sinh ra sai số như vậy. Nó sinh ra những bề mặt trơn, hợp lý, sai một cách có hệ thống, và *nhất quán giữa các khung* vì cùng một bộ giải mã tạo ra chúng. Với một cổng χ^2 , thứ đó trông y hệt một bức tường thật. Đây là lý do sâu nhất, và nó áp cho mọi tiên nghiệm học được chứ không riêng mã hình học.

Để thấy điều đó, hãy so hai kiểu lỗi trên cùng một cổng χ^2 , bằng một ví dụ *giả định* — không phải số đo trên hệ nào. Một cặp khớp sai lệch cỡ vài chục điểm ảnh cho χ^2 vọt lên hàng trăm, nên cổng 5,991 loại nó ngay ở khung đầu tiên. Bây giờ giả sử bộ giải mã dựng một bề mặt lệch ΔZ vài chục centimet ở khoảng cách vài mét. Đưa độ lệch ấy qua công thức chiếu $\Delta u = f b \Delta Z / Z^2$, với f và b của một cấu hình stereo cỡ EuRoC, phần dư thu được nhỏ hơn một điểm ảnh — vì Z^2 ở mẫu số nuốt gần hết độ lệch. Nó lọt qua mọi cổng. Tệ hơn, nó lọt qua ở *mọi* khung cùng một kiểu, nên các khung củng cố lẫn nhau và bộ ước lượng càng lúc càng tự tin vào một bức tường không có thật.

Thứ hai, bán kính hiệu lực của phép tuyến tính hoá rất nhỏ. Bộ giải mã chỉ gần tuyến tính *quanh mã mà bộ mã hoá vừa xuất ra*. Một hiệu chỉnh lớn sẽ đi ra ngoài vùng đó, và $\partial D / \partial c$ không còn đúng. Nói cách khác, tối ưu chỉ sửa được những sai lệch nhỏ — đúng trường hợp mà dự đoán ban đầu vốn đã tốt.

Thứ ba, độ phủ. Ba mươi hai chiều đủ cho phân phối huấn luyện và không đủ cho thứ khác. Ngoài trời, ban đêm, một sàn nhà máy đầy kim loại phản chiếu — bộ giải mã không có cơ sở nào để dựng bề mặt ở đó, nhưng nó vẫn dựng ra một bề mặt.

Thứ tư, phần thưởng đã dời chỗ. Năm 2018, đây gần như là con đường duy nhất để có bản đồ dày đặc tối ưu được. Tới 2023, 3D Gaussian Splatting [15] cho một bản đồ dày đặc, tinh chỉnh tăng dần được, chất lượng ảnh cao hơn hẳn, mà không cần bất cứ tiên nghiệm học được nào về hình dạng. Đó là nội dung của [F/32-ban-do-neural](#). Thêm nữa, xét riêng độ chính xác quỹ đạo, dòng mã ngắn chưa bao giờ vượt được SLAM đặc trưng thưa. Bản đồ mới là sản phẩm, còn tư thế chỉ là sản phẩm phụ.

E. Cái còn lại

Trước khi rời mục này, đáng chốt lại một bài kiểm tra hai câu, dùng được cho *mọi* tiên nghiệm học được mà bạn định cắm vào backend — mã hình học, độ sâu đơn ảnh, hay pháp tuyến bề mặt.

Câu thứ nhất: sai số của nó có độc lập giữa các khung không? Nếu cùng một mạng sinh ra dự đoán cho mọi khung thì câu trả lời gần như luôn là không. Khi đó nhân robust và cổng χ^2 mất tác dụng, và bạn phải hạ trọng số của cả nhóm phép đo ấy bằng tay, chứ không thể trông vào bộ ước lượng tự loại.

Câu thứ hai: sai số của nó có biểu hiện thành phần dư lớn không? Đây là câu hỏi định lượng, và mục trước đã cho cách tính: đổi độ lệch hình học ra phần dư điểm ảnh bằng công thức chiếu. Nếu một sai lệch nguy hiểm chỉ tạo ra phần dư dưới một điểm ảnh thì không cổng nào bắt được nó, và tiên nghiệm ấy phải được cấp một hiệp phương sai lớn ngay từ đầu.

Hai câu đó không cần thí nghiệm nào. Chúng trả lời được trên giấy trong mười phút, và chúng loại đi phần lớn ý tưởng nghe hay.

Dù vậy có một mảnh đáng giữ, và nó không phải cái mã. Đó là ý tưởng đặt một biến ẩn số chiều thấp vào đồ thị nhân tử rời ước lượng luôn hiệp phương sai của nó. Dạng khoẻ mạnh nhất của ý tưởng ấy là NodeSLAM. Một mã hình dạng cho *một loại vật thể cụ thể* là tiên nghiệm được xác

định rõ hơn nhiều so với một mã cho “một căn phòng”. Tiên nghiệm càng hẹp thì càng ít cơ hội bịa ra bề mặt.

CẦN NHỚ

Trong một bộ ước lượng bình phương tối thiểu, thứ nguy hiểm không phải là tiên nghiệm học được *sai*, mà là nó sai *một cách trơn tru và nhất quán giữa các khung*. Nhân robust, cổng χ^2 và RANSAC đều được thiết kế cho ngoại lai thừa và độc lập. Không cái nào bắt được một bề mặt bịa ra mà mọi khung đều đồng ý. Trước khi cắm bất cứ đầu ra học được nào vào backend, hãy hỏi đúng một câu: *sai số của nó có độc lập giữa các khung không*. [T1]

5.7 Giới hạn và trường hợp hỏng

A. Giới hạn của chính ý tưởng khả vi

Chế độ hỏng nền tảng nhất đã nêu ở mục 2 và đáng nhắc lại thành một câu: ***gradient chỉ chảy qua những thứ vốn đã liên tục***. Trong một hệ SLAM đầy đủ, các quyết định đắt giá nhất lại rời rạc. Chèn hay không chèn khung then chốt. Chấp nhận hay từ chối một ứng viên đóng vòng. Gộp hay không gộp hai bản đồ. Không cái nào có đạo hàm. Nên cụm từ “đầu-cuối” luôn hẹp hơn nghĩa đen của nó. Phần được học chính là phần vốn đã trơn, còn xương sống rời rạc vẫn do luật viết tay quyết định — và đó thường là phần gây ra các lần hỏng nặng nhất.

Chế độ hỏng thứ hai nằm ở ổn định số. H^{-1} nở ra khi H gần suy biến, tức khi thị sai thấp, khi camera quay gần như thuần tuý, khi ảnh mờ. Đó chính xác là những khung bạn muốn mạng học từ chúng. Nên ***gradient*** lớn nhất và nhiều nhất lại đến từ các mẫu quý nhất. Mọi cách chữa — tăng damping, cắt ***gradient***, giảm số bước lặp — đều làm thiên lệch đạo hàm theo hướng làm nhẹ đúng các mẫu đó.

Chế độ hỏng thứ ba là hàm mất mát. ***Gradient*** chỉ tốt bằng đại lượng bạn đặt ở cuối. Huấn luyện trên các đoạn video ngắn với lỗi tư thế theo cặp khung thì chỉ tối ưu tính nhất quán ngắn hạn. Nó không tối ưu độ trôi sau ba trăm mét, cũng không tối ưu xác suất đóng vòng đúng. Hai thứ đó lại chiếm phần lớn sai số thật của bạn. Đây là biến thể của bài học đo lường ở E/24-thi-nghiem-va-ablation: tối ưu cái bạn đo được thay vì cái bạn quan tâm là một cái bẫy, không phải một chiến lược.

B. Giới hạn của các hệ cụ thể

Ba giới hạn dùng chung cho cả bốn họ trong chương. *Dữ liệu huấn luyện* gần như luôn là TartanAir, KITTI hoặc ScanNet, nên phân phối chuyển động và phân phối cảnh đều hẹp hơn hiện trường. *Vấn đề IMU*: dòng đầu-cuối hầu như không dùng IMU, nên nó vứt đi thứ duy nhất còn tin được khi ảnh mờ. *Ngân sách nhúng*: các con số ẩn tượng đều đo trên GPU để bàn, và khoảng cách tới Orin là bậc độ lớn.

Còn một giới hạn ít được nói mà bạn sẽ gặp ngay ngày đầu, đó là ***tính lặp lại***. Bạn đã tốn công truy các nguồn phi tắt định trong hệ hiện tại — thứ tự duyệt container, tranh chấp luồng — để mỗi cấu hình chạy ba lần cho ra số dùng được. Đưa một mạng vào giữa bộ ước lượng là đưa thêm phi tắt định của thư viện GPU vào đúng chỗ đó, và lần này không có bug nào để sửa. Sàn nhiều của harness sẽ dâng lên, và mọi kết luận cũ phải đo lại.

Bảng 5.5. Bốn họ trong chương, đọc theo cùng bốn câu hỏi. Cột cuối là cột quyết định với một hệ stereo-inertial đã tinh chỉnh kỹ, chạy trên phần cứng nhúng.

Họ	Cơ chế cốt lõi	Hỏng khi nào	Lấy riêng được phần nào
Lớp tối ưu khả vi	Đạo hàm qua nghiệm arg min, bằng hàm ẩn hoặc trải phẳng	H suy biến; bộ giải bị cắt ngắn; hoặc không có gì để huấn luyện	Gần như toàn bộ: nó là hạ tầng, không phải một hệ SLAM
Đầu-cuối dày đặc	Dòng quang dày, trọng số học được, BA trong vòng lặp	Ngân sách nhúng; không dùng IMU; bộ nhớ tăng theo độ dài chuỗi	Trọng số liên tục thay quyết định inlier; vòng phản hồi nghiệm về phép đo
Đầu-cuối thưa	Theo vết mảnh ảnh; hiệp phương sai theo từng cặp khớp	Cảnh rất ít vân; vẫn cần GPU; vẫn phụ thuộc phân phối	Hiệp phương sai học được có đơn vị mét — mảnh đáng lấy nhất chương
Tự giám sát và mã ẩn	Mất mát dựng lại ảnh; nén hình học vào mã ngắn	Mơ hồ tỉ lệ; không đóng vòng; tiên nghiệm bìa bề mặt nhất quán	Mẫu “giữ bộ giải cổ điển, học phân dày đặc” kiểu TANDEM

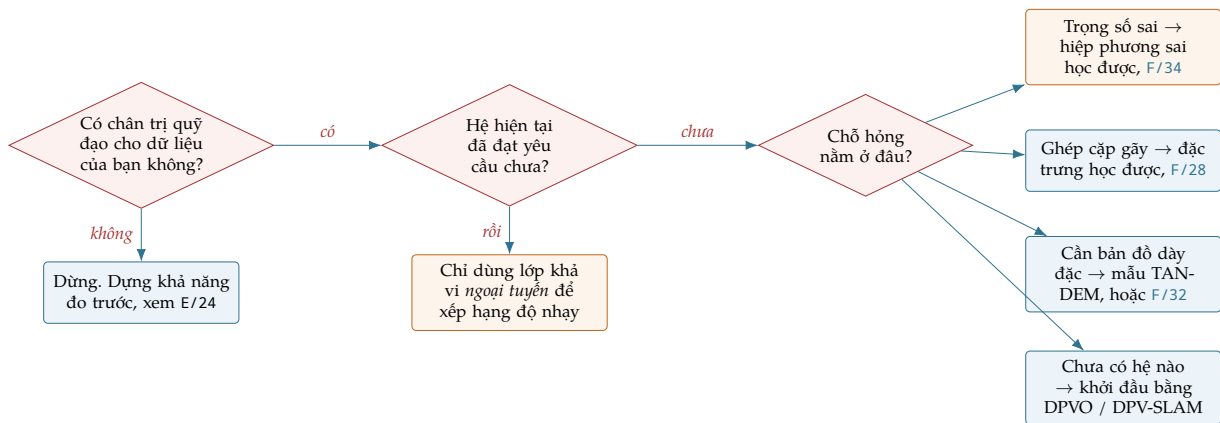
C. Kết luận thẳng thắn

Với một hệ stereo-inertial đã tinh chỉnh kỹ, chạy thời gian thực trên Orin, có harness đo được và có kỷ luật ablation: **hệ đầu-cuối hiện chưa phải lựa chọn thay thế**. Không phải vì ý tưởng sai, mà vì bảng cân đối không có lợi. Bạn sẽ đổi tính chẩn đoán, tính lặp lại và một bậc độ lớn ngân sách tính toán, để lấy một cải thiện chỉ hiện ra trên một phần các cảnh, và mất luôn IMU trên đường đi.

Nhưng kết luận đó không kéo theo “cả chương này vô dụng”. Có hai mảnh tách rời được, và cả hai đều nhỏ hơn nhiều so với việc thay cả hệ.

- **Lớp bất định học được.** Thay ma trận thông tin hằng số $\sigma_\ell^{-2}I$ bằng một hiệp phương sai dự đoán theo từng cặp khớp. Nó cắm vào đúng chỗ backend hiện có đang đọc trọng số, không đụng tới bộ giải, và tắt được bằng một cờ. Đây là mảnh đáng thử trước tiên, và [F/34-imu-bat-dinh-va-trien-khai](#) là chỗ bàn nó cho cả camera lẫn IMU.
- **Lớp tối ưu khả vi như một công cụ chẩn đoán ngoại tuyến.** Bạn không cần chạy nó lúc suy luận. Bạn có thể dựng lại bài toán backend của mình trong một lớp khả vi, chạy ngoại tuyến trên dữ liệu đã có chân trị, rồi lấy *gradient* của APE theo các nút vận để biết nút nào thật sự quan trọng. Đó là một công cụ *phân tích độ nhạy*, rẻ hơn quét lưới rất nhiều, và nó không thêm rủi ro nào vào hệ đang chạy.

Hình 5.10 gói cả kết luận đó thành một cây quyết định bốn câu hỏi. Nó cố tình bắt đầu bằng câu hỏi về dữ liệu chứ không phải về phương pháp, vì thiếu chân trị thì mọi nhánh còn lại đều vô nghĩa.



Hình 5.10. Cây quyết định cho cả chương. Câu hỏi đầu tiên cố tình không phải về phương pháp mà về dữ liệu: không có chân trị thì không có *gradient* nào để lấy, nên mọi nhánh phía sau đều vô nghĩa. Hai ô màu cam là hai kết luận “đáng làm” duy nhất cho một hệ stereo-inertial đã tinh chỉnh kỹ; ba ô còn lại chỉ dùng cho các tình huống khác hẳn.

5.8 Thực hành

Repo và tài nguyên. [princeton-vl/lietorch](#) cho phép toán nhóm Lie có *gradient*. [facebookresearch/theseus](#) cho lớp tối ưu phi tuyến khả vi; nên đọc phần tài liệu về các kiểu lượt ngược trước khi đọc mã. [princeton-vl/DRUID-SLAM](#) và [princeton-vl/DPVO](#) để so hai đầu của trục mật độ trên cùng một chuỗi. [nianticlabs/monodepth2](#) là cài đặt sạch nhất của dòng tự giám sát, và là chỗ tốt nhất để tự tay thấy hiện tượng mơ hồ tỉ lệ. [castacks/tartanvo](#) cho ý tưởng tầng nội tham số. *evo* để mọi con số bạn đo có cùng định nghĩa với phần còn lại của dự án. Danh sách này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; riêng LieTorch, Theseus và *evo* thuộc loại hạ tầng nên bền hơn hẳn các tên hệ thống đang dẫn đầu bảng.

Câu hỏi kiểm tra hiểu. Viết ra công thức $\partial L / \partial \theta$ qua một lớp arg min mà không nhìn lại, rồi nói rõ mỗi ký hiệu ứng với đại lượng nào trong backend của bạn. Nếu bộ giải chỉ chạy ba bước LM thì đạo hàm theo định lý hàm ẩn sai theo hướng nào, và bạn kiểm tra điều đó bằng thí nghiệm gì? Trong một hệ đầu-cuối, vì sao không có bước nào tương đương với “loại cạnh có χ^2 vượt ngưỡng”?

Bài tập phụ. Lấy một chuỗi EuRoC, chạy DPVO ở chế độ mặc định, rồi vẽ quỹ đạo của nó cạnh quỹ đạo hệ của bạn sau khi căn tỉ lệ. Mục đích không phải so hơn thua. Mục đích là thấy hình dạng lỗi của hai họ khác nhau ra sao: một bên lệch dần và trơn, một bên nhảy tại các khung khó.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Lớp tối ưu khả vi	Một bộ giải Gauss-Newton hoặc LM mà <i>gradient</i> đi ngược xuyên qua được. Lượt xuôi giống hệt bộ giải thường; toàn bộ giá trị nằm ở lượt ngược, và nó chỉ có nghĩa nếu bạn thật sự huấn luyện thứ gì đó bằng nó.
••• Định lý hàm ẩn cho arg min	Tại nghiệm, gradient theo x bằng không; vì phân điều kiện đó cho $\partial x^* / \partial \theta = -H^{-1} B$. Nhờ nó, lượt ngược chỉ tốn thêm một lần giải hệ với chính H đã phân tích ở lượt xuôi.
••• Trái phải (unrolling)	Coi K bước lặp như một mạng K tầng rồi để autodiff chạy. Đạo hàm đúng với thuật toán đã chạy, kể cả khi chưa hội tụ, nhưng bộ nhớ tăng tuyến tính theo K .

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Khi nào định lý hàm ẩn cho đạo hàm sai?	Khi bước cuối không phải điểm dừng thật, tức bộ giải bị cắt ngắn. Hoặc khi H suy biến vì tự do đo chưa được neo. Cả hai đều thường xuyên xảy ra trong SLAM.
••• Vì sao hệ đầu-cuối không xuất mặt nạ inlier?	Vì so sánh với ngưỡng là hàm bậc thang, đạo hàm bằng 0 hầu khắp nơi. Muốn học được thì phải thay quyết định cứng bằng một trọng số liên tục.
•• LieTorch	Thư viện lan truyền <i>gradient</i> trong không gian tiếp tuyến của nhóm Lie. Nó làm số chiều <i>gradient</i> bằng đúng số bậc tự do, sáu cho $SE(3)$, nên bỏ được việc chuẩn hoá quaternion giữa đồ thị.
•• Theseus	Lớp tối ưu phi tuyến khả vi trong PyTorch, có bộ giải thừa. Điểm đáng học là nó bắt người dùng chọn kiểu lượt ngược, tức thừa nhận thẳng rằng đạo hàm qua bộ giải là một xấp xỉ có thiên lệch.
•• BA-Net	Đặt cả một lớp hiệu chỉnh chùm tia vào mạng. Ba ý: sai lệch trên đặc trưng thay vì trên độ sáng, damping học được, và độ sâu theo cơ sở. Ý đầu bền nhất vì căn chỉnh trên đặc trưng học được có lòng chảo hội tụ rộng hơn và bất biến với đổi sáng.
••• DROID-SLAM	Toán tử hồi quy kiểu RAFT sinh dòng quang dày và trọng số theo từng điểm ảnh; một lớp BA dày đặc giải tư thế và độ sâu; nghiệm quay lại làm đầu vào cho vòng sau, nhờ đó hệ tự gỡ được khởi tạo tồi. Chính xác, nhưng cần GPU để bàn và không dùng IMU.
••• DPVO	Thư hoá DROID bằng cách theo vết khoảng một trăm mảnh ảnh mỗi khung, mỗi mảnh một ẩn độ sâu. Số ẩn hình học giảm vài bậc độ lớn, đủ để hệ chạy trên một GPU để bàn duy nhất thay vì nhiều card.
•• DPV-SLAM	DPVO cộng thêm đóng vòng và tối ưu đồ thị tư thế. Bài học lớn hơn kết quả: đóng vòng và tối ưu đồ thị tư thế vẫn là hai khối cổ điển mà chưa ai học thay được.
••• MAC-VO	Dự đoán hiệp phương sai có đơn vị mét cho từng cặp khớp stereo, dùng để chọn điểm và để làm ma trận thông tin. Đây là mảnh gần nhất với thứ một hệ cổ điển mượn được ngay.
••• SfM-Learner	Huấn luyện mạng độ sâu và mạng tư thế chỉ bằng video thô, tín hiệu dạy là mất mát dựng lại ảnh. Cần một mặt nạ giải thích được, và mặt nạ đó phải bị chính quy hoá vì nghiệm suy biến là tất hết.
••• Mơ hồ tỉ lệ với ảnh đơn	Nhân độ sâu và tịnh tiến với cùng một hằng số cho ra đúng cùng ảnh dựng lại, nên mất mát không phân biệt được. Hệ quả: nhiều bảng kết quả căn tỉ lệ theo từng chuỗi, việc mà robot không làm được.
•• DeepVO tốt trên KITTI mà kém chỗ khác — nguyên nhân?	Nó học luôn tiên nghiệm chuyển động của ô tô: đi thẳng, tốc độ trong dải hẹp, góc lệch nhỏ. Trên camera cầm tay hoặc drone, tiên nghiệm ấy sai theo hướng có hệ thống.
•• TartanVO	Chuẩn hoá tịnh tiến để bỏ tỉ lệ ra khỏi hàm mất mát, và nổi nội tham số camera vào đầu vào. Nhờ hai thủ thuật đó, một mô hình chạy được qua nhiều tập dữ liệu mà không tinh chỉnh lại.
•• TANDEM	Giữ tracking trực tiếp cổ điển, chỉ học phần dựng bản đồ dày đặc rồi hợp nhất TSDF. Đây là mẫu thiết kế đáng dùng nhất trong dòng tự giám sát cho một hệ đã chạy tốt.
••• CodeSLAM	Nén bản đồ độ sâu vào một mã cỡ vài chục chiều, với bộ giải mã gần tuyến tính theo mã, để đưa mã vào đồ thị nhân tử cạnh tư thế. Tính gần tuyến tính là điều kiện sống còn, nếu không thì mỗi lần tuyến tính hoá tốn một lượt qua mạng.
••• Vì sao dòng mã ngắn chưa thẳng?	Sai số của tiên nghiệm trơn và nhất quán giữa các khung nên cổng χ^2 không bắt được. Thêm nữa bán kính hiệu lực của phép tuyến tính hoá nhỏ, độ phủ hẹp, và 3DGS đã lấy mất phần thường “bản đồ dày đặc”.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
● ● ● Ngoại lai thừa và độc lập	Giả định nền của mọi ước lượng robust. Tiên nghiệm học được vi phạm nó, vì cùng một bộ giải mã sinh ra sai số ở mọi khung nên chúng đồng ý với nhau và trông như hình học thật.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
LieTorch [98]	Lan truyền <i>gradient</i> trong không gian tiếp tuyến của nhóm Lie — hạ tầng cho gần như mọi hệ trong chương	Hạ tầng; bền hơn tên hệ thống
Theseus [100]	Lớp Gauss-Newton và LM khả vi, bắt người dùng khai báo kiểu lượt ngược: trải phẳng, cắt ngắn, hay hàm ẩn	Hạ tầng; dùng được ngoại tuyến để phân tích độ nhạy
DROID-SLAM [9] và các nhánh dẫn xuất	BA dày đặc trong vòng lặp, cộng trọng số học được theo từng điểm ảnh	Vẫn là mốc chính xác tham chiếu cho nhiều bảng SLAM học sâu
DPVO và DPV-SLAM [104], [105]	Thừa hoá bằng mảnh ảnh; và việc phải mượn lại đóng vòng cổ điển	Ví dụ rõ nhất rằng đóng vòng chưa được học thay
MAC-VO [106]	Hiệp phương sai học được theo từng cặp khớp, dùng làm ma trận thông tin	Sau 2024; mình nêu ý tưởng, chưa tự kiểm chứng số liệu
Dòng monodepth2 và các hệ độ sâu tự giám sát	Mất mát dừng lại ảnh và mặt nạ giải thích được của SfM-Learner [107]	Ý tưởng còn sống mạnh ở phía độ sâu hơn là ở phía tư thế

MINI PROJECT SLAM · Đo độ nhạy của APE theo trọng số cạnh, bằng đạo hàm thay vì quét lưới · 1–2 ngày

Mục tiêu	biến câu “trọng số cạnh đặt bằng tay” thành một bảng số nói rõ nút vặn nào thật sự lay chuyển quỹ đạo. Đây là phiên bản rẻ nhất và không rủi ro của toàn bộ ý tưởng khả vi: bạn chỉ dùng <i>gradient</i> để <i>chẩn đoán</i> , không đưa mạng nào vào hệ đang chạy.
Dữ liệu	một chuỗi EuRoC có chân trị. Lấy đúng một cửa sổ 8–10 khung then chốt từ log của hệ hiện tại, kèm toàn bộ quan sát và khởi tạo của cửa sổ đó.
Các bước	dựng lại đúng bài toán bình phương tối thiểu của cửa sổ ấy trong một lớp tối ưu khả vi; kiểm tra nghiệm khớp với nghiệm của backend C++ tới cỡ 10^{-4} trước khi làm bất cứ việc gì khác; đặt L là sai lệch tư thế so với chân trị; lấy $\partial L / \partial \theta$ cho ba nhóm tham số là trọng số cạnh chiếu lại theo tầng kim tự tháp, bề rộng nhân robust, và trọng số cạnh IMU; cuối cùng đối chiếu với đạo hàm số bằng sai phân hữu hạn trên vài thành phần.
Xong khi	bạn có một bảng xếp hạng độ nhạy, và kiểm chứng được nó bằng một thí nghiệm độc lập. Đổi tay đúng nút vặn đúng đầu bảng một lượng nhỏ, rồi xác nhận APE đổi theo đúng dấu mà <i>gradient</i> dự đoán, trên ít nhất ba lần chạy để vượt sần nhiễu. Nếu dấu không khớp thì gần như chắc chắn cửa sổ của bạn chưa hội tụ tới điểm dừng. Đó cũng là một kết quả, và nó nói rằng định lý hàm ẩn chưa dùng được ở đây.
Nối với	E/24-thi-nghiem-va-ablation cho sần nhiễu và cỡ mẫu; F/34-imu-bat-dinh-va-trien-khai sẽ dùng lại chính bảng độ nhạy này để quyết định có đáng học hiệp phương sai hay không; A/01-toan-toi-thieu/02-vi-phan-va-toi-uu cho phần kiểm tra đạo hàm bằng sai phân hữu hạn.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Nó không làm Gauss-Newton hội tụ nhanh hơn, vì lượt xuôi y hết. Nó bán một thứ khác: đạo hàm của lỗi quỹ đạo theo các nút vận mà bạn đang đặt bằng tay, tức là biến việc quét lưới thành việc huấn luyện.
- A2. Vì tại nghiệm, gradient theo x bằng không, và điều kiện đó ràng buộc nghiệm với tham số. Vì phân nó ra $-H^{-1}B$, chỉ cần H ở bước cuối. Điều kiện là H phải khả nghịch và bước cuối phải là điểm dừng thật; trong SLAM cả hai đều hay bị vi phạm, vì tự do đo và vì bộ giải bị cắt ngắn.
- A3. Không học được, nên các hệ đầu-cuối không cố học. Chúng thay quyết định cứng bằng một trọng số liên tục cho từng cặp khớp: mạng hạ trọng số điểm xấu thay vì loại nó. Đây là hệ quả bắt buộc của việc muốn có *gradient*, không phải một lựa chọn thẩm mỹ.
- A4. Thứ chặn nó là ngân sách tính toán và bộ nhớ. Nó giữ một ẩn độ sâu cho *mỗi* điểm ảnh ở $1/8$ độ phân giải, cộng các khối tương quan có kích thước bằng bình phương số vị trí cho mỗi cạnh của đồ thị khung. Hệ được công bố trên GPU để bàn cao cấp, còn máy nhúng thấp hơn hẳn về cả tính toán lẫn băng thông — khoảng cách thuộc loại bậc độ lớn chứ không phải vài chục phần trăm. Thêm nữa nó không dùng IMU.
- A5. Hồng ở chỗ sai số của bộ giải mã trơn và nhất quán giữa các khung, nên nó vượt qua mọi cổng robust vốn giả định ngoại lai thừa và độc lập. Cộng thêm ba điều: phép tuyến tính hoá chỉ đúng quanh mã ban đầu, độ phủ hẹp ngoài phân phối huấn luyện, và 3DGS đã cho bản đồ dày đặc tốt hơn mà không cần tiên nghiệm hình dạng.

NÓI LẠI THẬT ĐƠN GIẢN

Hình dung bạn có một hội đồng gồm hai trăm người cùng đoán một con số. Bạn không lấy trung bình tròn. Bạn lấy trung bình có trọng số: ai bạn tin hơn thì tiếng nói nặng hơn. Từ trước tới nay bạn phát trọng số theo một quy tắc cứng, kiểu “ai ngồi gần bảng thì tin hơn”. Quy tắc đó không quan tâm hôm nay ai quên kính, ai ngồi ngược sáng, ai vừa ngủ dậy.

Bây giờ giả sử cuối buổi bạn biết đáp án đúng. Câu hỏi tự nhiên là: nếu cho người thứ 47 nói to hơn một chút, kết quả chung sẽ dịch về phía đáp án hay ra xa nó? Trả lời được câu đó cho cả hai trăm người thì bạn có một cách phát trọng số tốt hơn mọi quy tắc cứng. Điều bất ngờ là bạn không cần diễn lại cả buổi họp để trả lời. Chỉ cần biết kết quả chung nhảy thể nào quanh điểm cân bằng của nó, rồi làm một phép tính.

Cái giá thì có ba khoản. Muốn học kiểu đó, bạn phải có nhiều buổi họp đã biết đáp án. Phép tính trên trở nên rất nhạy đúng vào những buổi mà hội đồng chia rẽ nhất, mà đó lại là những buổi bạn cần học nhất. Và khi trọng số do một cỗ máy phát ra, hôm nào kết quả sai bạn sẽ không còn chỉ được vào ai đã nói sai. Bạn chỉ biết rằng nó sai.

Chỗ không rút gọn được: chỗ vấp thật nằm ở các quyết định bật-tắt. Nhiều thứ trong một hệ SLAM không phải chuyện tin nhiều hay tin ít, mà là chuyện có mời người đó vào phòng họp hay không: chèn khung then chốt hay không, nhận một vòng lặp hay không. Với những quyết định đó, câu hỏi “nói to hơn một chút thì sao” không có nghĩa, vì không có “một chút” nào cả. Mọi cách nói đơn giản mà mình biết đều làm mờ mất ranh giới này, trong khi chính nó mới là ranh giới thật giữa phần học được và phần không học được.

Một câu: *Khả vi không làm bộ giải khôn hơn; nó chỉ cho phép hỏi “tin phép đo này thêm một chút thì quỹ đạo tốt lên hay xấu đi”, và bạn trả tiền cho câu hỏi đó bằng bộ nhớ, độ ổn định và khả năng chắn đoán.*

5.9 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là bộ lọc mà chương này vừa được đưa qua: bốn cổng của nó cho ra kết luận “chưa đáng” với cả hệ, và “đáng thử” với hai mảnh nhỏ. [F/28-dac-trung-va-ghep-cap](#) là nửa còn lại của cùng một câu chuyện — ở đó phần học được nằm trước bộ giải, ở đây nó nằm bên trong. [F/30-do-sau-dong-quang-va-mo-hinh-nen](#) là tiền đề vật liệu: dòng quang và độ sâu học được chính là đầu vào mà mọi hệ trong chương này tiêu thụ. [F/32-ban-do-neural](#) là nơi cuộc thi về biểu diễn bản đồ dày đặc chuyển sang, và là lý do dòng mã ngăn mất động lực.

[F/34-imu-bat-dinh-va-trien-khai](#) là chỗ mảnh đáng lấy nhất của chương, tức hiệp phương sai học được, được bàn cho đủ, cùng với ngân sách độ trễ trên Orin.

Ngoài phần F thì có bốn mối nối. A/01/02 là nền của định lý hàm ẩn và của cả hai lối đi ngược. C/15/03 giải thích toán tử cập nhật hồi quy mà DROID-SLAM mượn lại. B/11-gioi-han-tinh-toan cho khung để biến các con số ẩn số và tensor ở mục 4 thành một ngân sách bộ nhớ thật. E/24 là kỷ luật bắt buộc nếu bạn định thử một trong hai mảnh tách rời: đo sần nhiều mới trước, nói về cải tiến sau.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao đạo hàm theo định lý hàm ẩn chỉ tốn thêm một lần thế ngược, trong khi trải phẳng tốn bộ nhớ tỉ lệ với số bước lặp?
2. Bộ giải của bạn dừng sau ba bước LM vì ngân sách thời gian. Đạo hàm thu được khi đó là đạo hàm của cái gì, và bạn thiết kế thí nghiệm nào để phát hiện sai lệch ấy?
3. Nếu quyết định inlier có đạo hàm bằng 0 hầu khắp nơi, thì cơ chế nào trong DROID-SLAM đang đóng vai trò loại điểm sai, và nó khác gì về hành vi khi gặp một cảnh ngoài phân phối huấn luyện?
4. Vì sao *gradient* lại lớn và nhiều nhất đúng ở các khung thị sai thấp hoặc quay thuần túy, và vì sao mọi cách chữa đều làm thiên lệch việc học ở chính các khung đó?
5. Một hệ VO tự giám sát báo lỗi quỹ đạo rất nhỏ trên KITTI. Bạn phải kiểm đúng một điều trước khi tin con số đó — điều gì, và vì sao nó quyết định con số ấy có dùng được cho robot hay không?
6. Vì sao một bề mặt do bộ giải mã bịa ra lại nguy hiểm hơn một cặp khớp sai, xét theo giả định mà nhân robust dựa vào?
7. Nếu chỉ được lấy đúng một mảnh từ chương này về hệ hiện tại, bạn lấy mảnh nào, và bạn đo cái gì để biết nó có tác dụng vượt sần nhiều?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fàcil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., "Image Matching Across Wide Baselines: From Paper to Practice," *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., "RoMa: Robust Dense Feature Matching," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., "Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., "HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, "Closed-form solution of absolute orientation using unit quaternions," *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., "Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, "Rethinking Visual Geo-localization for Large-Scale Applications," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, "MixVPR: Feature Mixing for Visual Place Recognition," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., "EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, "Optimal Transport Aggregation for Visual Place Recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features Without Supervision," *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., "AnyLoc: Towards Universal Visual Place Recognition," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Bản đồ neural: NeRF và 3D Gaussian Splatting (Neural Map Representations)

PHỤ THUỘC C/16/03-nerf-3dgs-va-hop-nhat (cơ chế dựng ảnh thể tích, rasterize Gaussian, và vì sao PSNR không chứng minh hình học — chương này giả định bạn đã nắm phần đó); F/27 (khung bốn cổng để quyết định thay một khối cổ điển); E/24 (thiết kế ablation, oracle experiment); F/31 nếu bạn muốn hiểu bộ tám vết mà nhiều hệ trong chương này mượn lại.

TÓM TẮT

Chương này hỏi một câu duy nhất: có nên thay bản đồ điểm thưa của hệ SLAM hiện tại bằng một bản đồ neural. Bạn sẽ đọc được hai dòng công trình — NeRF và 3D Gaussian Splatting — theo đúng nút thắt mà mỗi mốc gỡ, chứ không theo thứ tự thời gian. Phần nặng nhất là phần đánh giá, vì đây là chủ đề mà chất lượng ảnh và độ chính xác quỹ đạo bị trộn lẫn nhiều nhất. Đọc xong bạn tách được hai trục đó, chỉ ra được điều kiện thí nghiệm ẩn sau một bảng kết quả đẹp, và lập được ngân sách bộ nhớ cùng băng thông cho Jetson Orin dưới dạng công thức, trước khi thử. Nếu chỉ nhớ một điều: một bản đồ đẹp ảnh không phải một bản đồ đúng hình học, và cho tới nay chưa có bằng chứng nào nói rằng bản đồ neural giúp *quỹ đạo* của một hệ stereo-inertial đã tinh chỉnh kỹ tốt lên.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bản đồ điểm thưa** — bản đồ của ORB-SLAM3 và họ hàng: vài chục nghìn tới vài trăm nghìn điểm 3D, mỗi điểm kèm một bộ mô tả để nhận lại. Nó đủ để trả lời “tôi đang ở đâu” và không trả lời được “chỗ kia có trống không”.
- **bản đồ dày, TSDF, lưới chiếm dụng** — ba cách nói về cùng một ý: bản đồ gán một giá trị cho *mọi* vị trí trong không gian, không chỉ ở vài điểm đặc trưng. TSDF lưu ở mỗi ô một khoảng cách có dấu tới bề mặt gần nhất — âm ở trong vật, dương ở ngoài, bằng không đúng trên mặt — nên bề mặt có định nghĩa sạch. Lưới chiếm dụng lưu xác suất mỗi ô bị chiếm, và đó là thứ bộ lập kế hoạch đường đi thật sự cần.
- **dựng ảnh ở góc nhìn mới** — cho một số ảnh đã chụp, tạo ra ảnh của cùng cảnh đó từ một vị trí camera chưa từng chụp. Đây là bài toán mà NeRF và 3DGS sinh ra để giải.
- **PSNR, SSIM, LPIPS** — ba thước đo “ảnh dựng ra giống ảnh thật tới đâu”. PSNR đo sai lệch bình phương theo từng điểm ảnh, SSIM đo tương đồng cấu trúc cục bộ, LPIPS đo khoảng cách trong không gian đặc trưng của một mạng đã huấn luyện. Cả ba đều nói về *ảnh*, không nói gì về hình học.
- **ATE và RPE** — hai thước đo quỹ đạo. ATE là sai số tuyệt đối của cả quỹ đạo sau khi căn chỉnh với chân trị; RPE là sai số của các chuyển vị tương đối trong một cửa sổ thời gian. Đây là hai con số quyết định một hệ SLAM có dùng được không.
- **bám vết theo mô hình** — cách ước lượng tư thế bằng cách dựng ảnh từ bản đồ hiện có rồi tối ưu tư thế cho ảnh dựng khớp ảnh thật, thay vì ghép đặc trưng giữa hai khung hình. Nó chính xác khi bản đồ đúng và trôi theo bản đồ khi bản đồ sai.
- **quên thăm khốc** — hiện tượng một mạng học dữ liệu mới rồi mất khả năng tái tạo dữ liệu cũ, vì cùng bộ trọng số phải gánh cả hai. Trong bản đồ neural, nó hiện ra thành “đi sang phòng bên rồi quay lại thì phòng cũ mờ đi”.

- **Replica, TUM RGB-D, ScanNet** — ba tập dữ liệu mà gần như mọi bài trong chương này báo cáo trên đó. Replica là cảnh tổng hợp có độ sâu hoàn hảo; TUM RGB-D và ScanNet là cảnh trong nhà quay bằng cảm biến thật, chuỗi ngắn, tốc độ đi bộ.

CÂU HỎI MỞ ĐẦU

- Q1. Bản đồ của ORB-SLAM3 chỉ có vài trăm nghìn điểm rời rạc, mà hệ vẫn định vị chính xác tới centimét. Vậy chính xác thì một bản đồ dày cho bạn thêm *điều gì* mà bạn đang thiếu?
- Q2. Nếu bạn thật sự cần bản đồ dày, vì sao lại là bản đồ neural chứ không phải TSDF — thứ đã chạy thời gian thực từ 2011 và cho bề mặt có định nghĩa sạch?
- Q3. Một hệ dựng ảnh đẹp tới mức khó phân biệt với ảnh thật. Vì sao điều đó vẫn không nói gì về việc quỹ đạo của nó tốt hay tệ hơn hệ bạn đang có?
- Q4. 3DGS nhanh hơn NeRF nhiều bậc độ lớn. Khoản chênh đó đến từ một thay đổi thuật toán duy nhất — nó là gì, và vì sao nó lại đúng lúc mở ra khả năng sửa bản đồ sau khi đóng vòng?
- Q5. Một triệu Gaussian nghe không nhiều. Nhưng khi đưa lên Orin và chạy vài chục vòng tối ưu mỗi khung, cái gì cản trước — dung lượng bộ nhớ hay băng thông bộ nhớ — và vì sao câu trả lời đó làm mọi mẹo tăng tốc suy luận trở nên vô dụng?

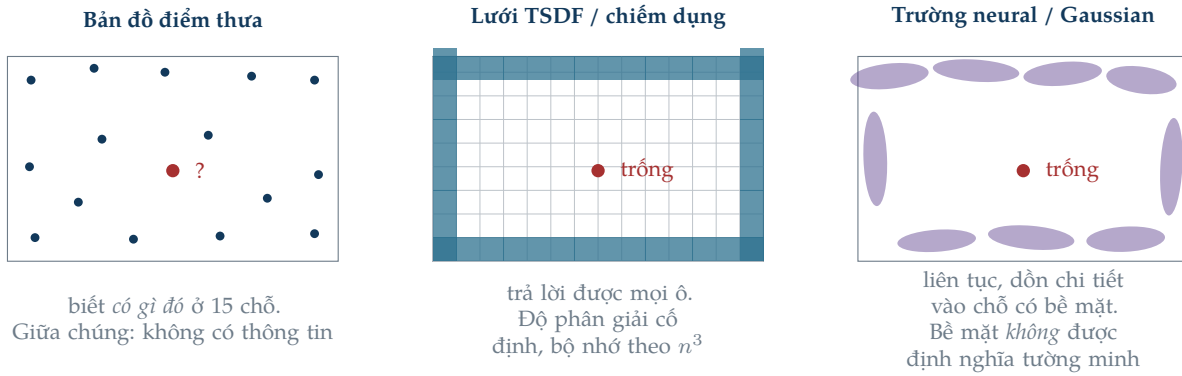
6.1 Đặt vấn đề: bản đồ thưa đủ cho việc gì

Hệ SLAM bạn đang bảo trì dựng một **bản đồ điểm thưa** (*sparse point map*). Mỗi điểm là một vị trí 3D cộng một bộ mô tả nhị phân. Với một tầng văn phòng, con số thường rơi vào bậc 10^5 điểm. Bản đồ đó làm rất tốt đúng một việc: trả lời câu “tôi đang ở đâu”. Nó làm tốt vì nó chỉ cần các điểm *nhận lại được*, và những điểm nhận lại được thì hiếm — góc, biên, đốm vân.

Nhưng có ba câu hỏi hạ nguồn mà chính bản đồ ấy không trả lời nổi. Thứ nhất: khoảng không giữa hai điểm là tường hay là lối đi? Bản đồ thưa không nói. Thứ hai: bề mặt này hình dạng ra sao, để robot đặt tay hoặc để người kiểm tra vết nứt? Bản đồ thưa không nói. Thứ ba: cảnh nhìn từ chỗ camera chưa từng đi qua thì trông thế nào, để người điều khiển từ xa xem trước, hoặc để sinh dữ liệu huấn luyện? Bản đồ thưa cũng không nói.

Trước khi đi tiếp, một điều chỉnh cần thiết cho đúng hệ của bạn. Một cặp stereo đã cho độ sâu dày ở *mỗi* khung hình. Vậy thứ bạn thiếu không phải độ sâu, mà là một bản đồ dày *bền vững*: một chỗ để hợp nhất hàng nghìn quan sát độ sâu nhiều thành một mô hình duy nhất, nhất quán, và tra cứu được ở mọi vị trí. Phân biệt này quan trọng vì nó thu hẹp câu hỏi rất nhiều. Nếu bộ lập kế hoạch của bạn chỉ cần nhìn một mét phía trước, độ sâu stereo tức thời đã đủ và bạn không cần chương này. Nhu cầu bản đồ dày chỉ thành thật khi robot phải nhớ chỗ nó đã đi qua.

Hình 6.1 đặt ba biểu diễn cạnh nhau và hỏi chúng cùng một câu. Đây là hình đáng nhìn kỹ nhất của phần đặt vấn đề, vì nó cho thấy khác biệt không nằm ở “chi tiết hơn” mà nằm ở *loại câu hỏi trả lời được*.



Hình 6.1. Cùng một hành lang, ba biểu diễn, và cùng một câu hỏi hỏi vào điểm đỏ ở giữa: chỗ này trống hay đặc? Bản đồ thưa không có câu trả lời — không phải vì thiếu chi tiết mà vì nó chưa bao giờ mô hình hoá khoảng không. Lưới trả lời được mọi chỗ nhưng trả bằng bộ nhớ tăng theo lũy thừa ba của độ phân giải. Biểu diễn neural trả lời được mọi chỗ và đồn tham số vào nơi có bề mặt, đối lại bề mặt chỉ tồn tại ngầm dưới dạng một vùng mật độ cao.

TRỰC GIÁC

Bản đồ thưa giống một mạng lưới mốc trắc địa đóng trên nền đất. Đứng giữa công trường, bạn nắm ba cái mốc là biết chính xác mình ở đâu, tới từng centimét. Nhưng không ai đưa bản vẽ mốc trắc địa cho người lái xe cầu, vì nó không nói cái cột nào đang chắn đường. Bản đồ dày là bản vẽ mặt bằng: kém chính xác hơn ở từng điểm, nhưng nói được chỗ nào đi lọt. Hai thứ trả lời hai câu hỏi khác nhau, và một hệ SLAM tốt cho việc định vị hoàn toàn có thể vô dụng cho việc điều hướng.

A. Đối thủ thật của bản đồ neural không phải bản đồ thưa

Đây là chỗ phần lớn bài báo trong chương này lạng lẽ bỏ qua, và cũng là cổng đầu tiên bạn nên dựng. Nhu cầu **bản đồ dày** (*dense map*) có từ lâu và đã có lời giải cổ điển. KinectFusion hợp nhất độ sâu vào một lưới **trường khoảng cách có dấu** (*truncated signed distance field*), viết tắt TSDF, theo thời gian thực từ 2011 [120]. OctoMap cho **lưới chiếm dụng** (*occupancy grid*) dạng cây tám nhánh, tiết kiệm bộ nhớ, có mô hình xác suất rõ ràng, và đã nằm trong hệ điều hướng của rất nhiều robot [121]. Cả hai đều chạy được trên phần cứng nhúng, đều cho bề mặt hoặc thể tích trống có định nghĩa sạch, và đều đã được kiểm chứng hàng chục năm.

Vậy câu hỏi đúng không phải “thưa hay neural”. Nó là: bản đồ neural cho thêm gì so với TSDF, và đổi lại mất gì. Câu trả lời trung thực gồm ba khoản được và ba khoản mất, và cả sáu khoản đều nên nằm sẵn trong đầu bạn trước khi đọc bất kỳ bảng kết quả nào.

- **Được — nội suy chỗ chưa quan sát.** Lưới TSDF để trống ô nào chưa có tia đi qua. Trường neural, do là một hàm liên tục có thiên lệch về hàm trơn, tự lấp các khe nhỏ. Với cảnh quét thưa, khác biệt này rất dễ thấy.
- **Được — dựng được ảnh.** TSDF cho hình học nhưng không cho màu phụ thuộc góc nhìn. Muốn ảnh dựng ra giống ảnh thật, bạn cần một mô hình bức xạ, và đó chính là thứ neural rendering mang lại.
- **Được — tham số đồn theo nội dung.** Lưới đều tiêu bộ nhớ cho cả không khí. Trường neural và tập Gaussian tiêu tham số ở nơi có bề mặt.
- **Mất — bề mặt không tường minh.** Trong TSDF, bề mặt là tập mức 0. Trong trường mật độ hoặc tập Gaussian, không có định nghĩa nào tương đương, nên mọi thứ hạ nguồn cần bề mặt đều phải thêm một bước dựng lại.

- **Mất — không có mô hình xác suất chiếm dụng.** OctoMap nói “ô này bị chiếm với xác suất 0,8”. Mật độ của NeRF không phải xác suất chiếm dụng, và độ đục của một Gaussian càng không. Bộ lập kế hoạch nào cần biên an toàn sẽ không có thứ để dựa vào.
- **Mất — chi phí là vòng lặp tối ưu, không phải một lượt tích hợp.** TSDF cập nhật bằng một phép trộn có trọng số, chi phí cố định mỗi khung. Bản đồ neural cập nhật bằng vài chục bước hạ gradient, và chi phí đó không giảm được bằng lượng tử hoá hay TensorRT.

Khoản mất cuối cùng là khoản đắt nhất trên phần cứng nhúng, và mục 6.5 sẽ quy nó thành con số. Trước đó, hai mục tiếp theo đọc hai dòng công trình theo đúng mạch nút thắt.

6.2 Dòng NeRF: một MLP làm bản đồ, rồi mọi thứ sinh ra từ chỗ nó hỏng

Cơ chế dựng ảnh thể tích đã nằm ở C/16/03-nerf-3dgs-va-hop-nhat, nên mục này không nhắc lại. Điều đáng học ở đây là chuyện khác: khi mang cơ chế đó vào một hệ SLAM chạy trực tuyến, những giả định nào vỡ, và mỗi công trình tiếp theo vá đúng chỗ nào.

A. iMAP — ý tưởng cực đoan nhất, và nó giải quyết việc gì

iMAP làm đúng một việc táo bạo: dùng *một* perceptron nhiều lớp làm toàn bộ bản đồ của một hệ SLAM RGB-D chạy trực tuyến [122]. Không có lưới, không có point cloud, không có TSDF. Cảnh là trọng số của một mạng nhỏ. **Bám vết theo mô hình** (*frame-to-model tracking*) là tối ưu tư thế khung hình mới sao cho ảnh và độ sâu dựng ra từ mạng khớp với quan sát. Dựng bản đồ là tiếp tục hạ gradient trên trọng số mạng, dùng các tia lấy mẫu từ một tập **khung hình chốt** (*keyframe*).

Nó giải quyết việc gì cụ thể? Một hệ TSDF phải chọn trước độ phân giải ô lưới và phải trả bộ nhớ cho toàn bộ thể tích. iMAP không phải chọn gì cả: sức chứa nằm ở số trọng số, và mạng tự phân bổ chi tiết. Với một căn phòng, tác giả báo cáo mô hình chỉ vài trăm nghìn tham số — nhỏ hơn nhiều bậc so với lưới tương đương. Thêm một điểm thiết kế đáng học: iMAP chọn khung hình chốt theo *lượng thông tin mới*, đo bằng sai số dựng ảnh của khung hình đó dưới bản đồ hiện tại. Đây là một tiêu chí có cơ sở, khác hẳn các luật chốt khung hình bằng ngưỡng số điểm khớp mà hệ cổ điển vẫn dùng.

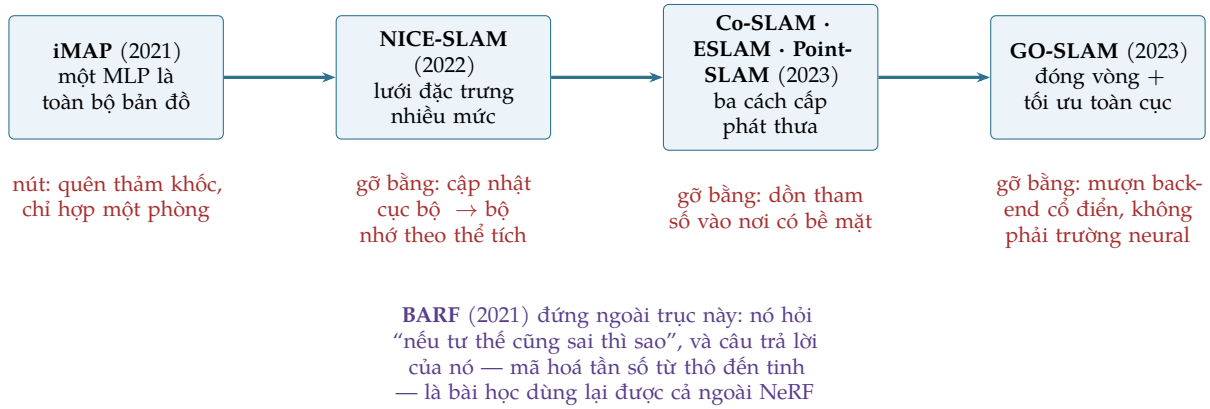
Cụm “vài trăm nghìn tham số” đáng dừng lại một nhịp, vì nó là lý lẽ mạnh nhất của cả dòng này. Điều đáng nhớ không phải một tỉ số cụ thể mà là *hai đại lượng đó lớn lên theo hai kiểu khác nhau*. Sức chứa của một MLP bốn tầng ẩn rộng 256 là một hằng số: nó không đổi khi căn phòng to ra. Bộ nhớ của một lưới thể tích thì tăng theo lũy thừa ba của độ phân giải và tỉ lệ với thể tích, lại còn trả tiền cho cả không khí lẫn tường. Nên ở độ phân giải vài centimet cho một căn phòng, biểu diễn ẩn gọn hơn hẳn — và càng gọn hơn khi bạn đòi độ phân giải mịn hơn. Tỉ số gọn hơn bao nhiêu lần thì phụ thuộc độ phân giải và kích thước phòng, nên nó không phải một hằng số trích dẫn được.

Cái giá thì lộ ra ngay. Vì toàn bộ cảnh nằm trong một bộ trọng số dùng chung, mỗi bước cập nhật ở phòng B đều làm đổi hàm ở phòng A. Đó là **quên thảm khốc** (*catastrophic forgetting*), và cách chữa duy nhất trong khuôn khổ ấy là liên tục phát lại các tia từ khung hình cũ — tức là trả chi phí tăng theo kích thước cảnh cho một biểu diễn vốn được quảng cáo là gọn. Các tác giả công bố hệ trên một GPU để bàn đời cao, với khâu dựng bản đồ chạy chậm hơn hẳn khâu bám vết. Chính chênh lệch đó, chứ không phải con số tốc độ, mới là điều đáng nhớ: khâu bản đồ là khâu bị ngân sách ép, và mọi công trình sau đều tấn công vào đúng nó. Phạm vi thí nghiệm là một căn phòng.

Mạch 1 — dòng NeRF làm bản đồ SLAM

- 1 Vấn đề** Bản đồ dày cổ điển phải chọn trước độ phân giải và trả bộ nhớ cho cả không khí.
- Giải pháp** iMAP đặt toàn bộ cảnh vào trọng số một MLP, tối ưu bằng chính sai số ảnh và độ sâu [122].
- Giả định** cảnh cỡ một căn phòng, có độ sâu RGB-D, camera đi chậm, và một GPU để bàn luôn sẵn.
- Giá phải trả** mỗi khung hình cần vài chục bước hạ gradient thay vì một lượt tích hợp.
- Vấn đề mới** trọng số dùng chung cho cả cảnh, nên học chỗ mới làm hỏng chỗ cũ.
- 2 Vấn đề** Quên thảm họa, và chi phí phát lại tăng theo kích thước cảnh.
- Giải pháp** NICE-SLAM thay MLP đơn bằng lưới đặc trưng nhiều mức, mỗi bước cập nhật chỉ chạm các ô lưới quanh tia [123].
- Giả định** cảnh có biên giới hạn trước để cấp phát lưới; vẫn cần độ sâu.
- Giá phải trả** bộ nhớ quay lại phụ thuộc thể tích cảnh, đúng thứ iMAP muốn tránh.
- Vấn đề mới** lưới đều lại tiêu tham số cho không khí, và tra nhiều mức thì chậm.
- 3 Vấn đề** Lưới đều tốn bộ nhớ; MLP đơn quên; cần một chỗ đứng giữa.
- Giải pháp** ba lời giải song song — Co-SLAM ghép mã hoá toạ độ với bảng băm thưa [124]; ESLAM thay lưới ba chiều bằng ba mặt phẳng đặc trưng và dùng TSDF thay mật độ [125]; Point-SLAM neo đặc trưng lên chính các điểm đã quan sát, dày thưa theo độ chi tiết ảnh [126].
- Giả định** bề mặt chiếm phần rất nhỏ thể tích, nên cấu trúc thưa là đúng bản chất bài toán.
- Giá phải trả** thêm siêu tham số cấp phát, và mỗi lời giải mạnh trên một loại cảnh khác nhau.
- Vấn đề mới** cả ba vẫn không có tính nhất quán toàn cục: không đóng vòng, không hiệu chỉnh chùm tia toàn phần.
- 4 Vấn đề** Đi một vòng lớn rồi quay lại thì bản đồ lệch, và không có cơ chế nào kéo nó về.
- Giải pháp** GO-SLAM ghép một bộ bám vết dày kiểu DROID với bản đồ neural, thêm đóng vòng và tối ưu toàn cục, rồi biến dạng bản đồ theo tư thế đã sửa [127].
- Giả định** bản đồ neo được vào các khung hình chốt, nên sửa tư thế thì kéo được bản đồ theo.
- Giá phải trả** chi phí tăng thêm một bộ bám vết nặng, và toàn hệ cần bộ nhớ GPU lớn.
- Hệ quả** đây là mốc đầu tiên trong dòng này trả lời được nỗi đau đóng vòng — và nó trả lời bằng cách mượn lại kiến trúc của SLAM cổ điển, không phải bằng bản thân trường neural.

Hình 6.2 vẽ lại chính mạch đó trên một trục, để bạn nhìn thấy mỗi mốc gỡ nút nào.



Hình 6.2. Dòng NeRF làm bản đồ SLAM, đọc theo nút thất chứ không theo năm. Mỗi mũi tên là một câu trả lời cho đúng một chỗ hỏng của mốc trước, và mốc cuối đáng chú ý nhất vì nó gỡ nút bằng cách gọi lại kiến trúc SLAM cổ điển. Nếu bài toán của bạn không có nút thất tương ứng, mốc đó không mang lại gì cho bạn.

B. NICE-SLAM và ý nghĩa của tính cục bộ

NICE-SLAM chữa quên thăm khốc bằng một thay đổi nghe rất kỹ thuật nhưng thực ra là thay đổi khái niệm [123]. Thay vì để cảnh nằm trong trọng số, nó để cảnh nằm trong **lưới đặc trưng nhiều mức** (*multi-level feature grid*): mỗi ô lưới giữ một vectơ đặc trưng học được, còn các bộ giải mã nhỏ thì cố định. Muốn biết mật độ tại một điểm, ta nội suy đặc trưng từ các ô lân cận rồi cho qua bộ giải mã.

Hệ quả quan trọng nằm ở *giá đỡ của gradient*. Với MLP đơn, một tia ở phòng B sinh gradient chảy vào mọi trọng số, nên nó đổi hàm ở khắp nơi. Với lưới đặc trưng, tia đó chỉ chạm vài ô quanh nó. Cập nhật trở thành cục bộ, và quên thăm khốc biến mất gần hết. Hình 6.3 vẽ đúng khác biệt này.



Hình 6.3. Vì sao đổi từ trọng số sang lưới lại chữa được quên thăm khốc. Câu trả lời nằm ở giá đỡ của gradient: bên trái, một quan sát mới chạm tới toàn bộ tham số, nên nó bắt buộc phải đổi cả những vùng cảnh không liên quan; bên phải, quan sát chỉ chạm các ô lân cận. Đây là cùng một nguyên lý với việc chia bản đồ thành nhiều bản đồ con trong SLAM cổ điển: giới hạn phạm vi ảnh hưởng của một quan sát chính là cách mua lấy tính ổn định.

Cái giá thì đúng bằng thứ iMAP muốn tránh: bộ nhớ quay lại phụ thuộc thể tích cảnh, và bạn phải biết trước biên của cảnh để cấp phát. Ba công trình năm 2023 đi tìm chỗ đứng giữa hai cực đó, mỗi cái theo một hướng khác. Co-SLAM ghép một mã hoá tọa độ trơn với một bảng băm thưa, để có cả tính lấp khe của hàm liên tục lẫn tốc độ của bảng tra [124]. ESLAM bỏ lưới ba chiều, thay bằng ba mặt phẳng đặc trưng vuông góc nhau, nên bộ nhớ tăng theo bình phương chứ không theo lũy thừa ba của độ phân giải; nó cũng đổi mật độ lấy TSDF, tức là kéo bề mặt về lại chỗ có định nghĩa sạch [125]. Point-SLAM đi xa nhất theo hướng “chỉ cấp phát nơi có vật”: đặc trưng neo trên chính đám mây điểm đã quan sát, và mật độ điểm được điều chỉnh theo độ biến thiên của ảnh, tức là dồn sức chứa vào chỗ nhiều chi tiết [126].

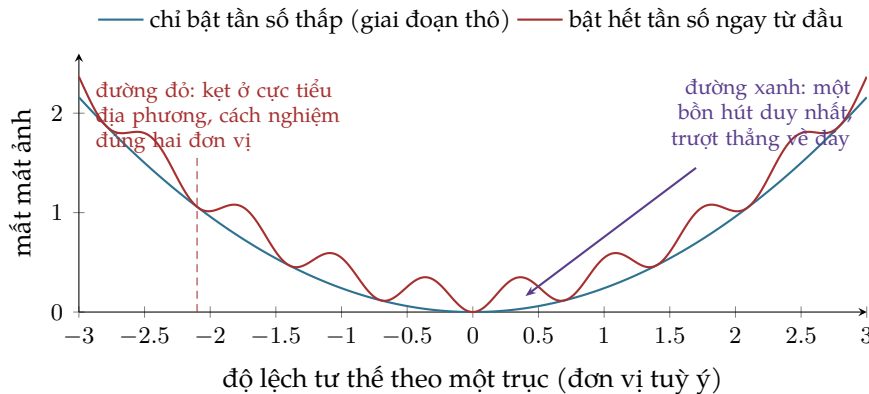
Ba lời giải này đáng nhớ không phải vì tên của chúng mà vì chúng là ba câu trả lời cho cùng một câu hỏi: *đặt tham số ở đâu*. Đó cũng đúng là câu hỏi bạn sẽ phải trả lời nếu tự dựng một bản đồ dày cho hệ của mình.

C. BARF: khi tư thế cũng là ẩn số, và bài học từ thô đến tinh

Cả nhóm trên giả định tư thế camera đã có, hoặc ít nhất có một ước lượng ban đầu tốt. BARF hỏi câu ngược lại: nếu tối ưu tư thế *cùng lúc* với trường thì sao [128]. Về mặt hình thức, đây chính là **hiệu chỉnh chùm tia** (*bundle adjustment*), chỉ khác là điểm 3D được thay bằng một trường liên tục và phần dư được đo trên chính điểm ảnh.

Phát hiện quan trọng của BARF không phải kết quả mà là *nguyên nhân của thất bại*. **Mã hoá vị trí** (*positional encoding*) bằng chuỗi sin và cos nhiều tần số là thứ cho NeRF độ sắc nét. Nhưng chính nó phá hỏng gradient theo tư thế. Lý do là đạo hàm của một thành phần tần số k có biên độ tỉ lệ với k . Các thành phần tần số cao vì thế tạo ra một mặt phẳng mất mát lờm lờm chớm với vô số cực tiểu địa phương cách nhau rất gần. Hạ gradient trên tư thế rơi vào cái hố gần nhất và đứng lại đó.

Lời giải của BARF là che dần: bật các tần số từ thấp lên cao theo tiến trình tối ưu. Giai đoạn đầu, mặt phẳng mất mát trơn và có một bồn hút rộng, nên tư thế đi đúng hướng. Giai đoạn sau, tần số cao mở ra và chi tiết trở lại. Hình 6.4 vẽ minh hoạ mặt cắt một chiều của hai mặt phẳng mất mát đó.



Hình 6.4. Mặt cắt một chiều của mặt phẳng mất mát theo tư thế, hình *minh hoạ* chứ không phải số đo. Đường đỏ là điều xảy ra khi mọi tần số của mã hoá vị trí đều bật: biên độ đạo hàm của thành phần tần số k tỉ lệ với k , nên các gợn tần số cao sinh ra hàng loạt cực tiểu địa phương gần nhau. Đường xanh là cùng bài toán khi chỉ bật tần số thấp: một bồn hút duy nhất, đủ rộng để hạ gradient tìm về đáy. BARF bật dần từ đường xanh sang đường đỏ trong quá trình tối ưu.

Bài học ở đây quan trọng hơn bản thân BARF, và nó nói bằng đúng ngôn ngữ bạn đã quen. Mọi phương pháp căn chỉnh theo cường độ ảnh đều cần một lịch làm trơn. Các hệ trực tiếp cổ điển gọi nó là kim tự tháp ảnh: căn chỉnh ở mức thô trước, rồi mới xuống mức tinh. BARF phát hiện lại đúng nguyên lý đó trong không gian tần số của mã hoá vị trí. Nếu một ngày bạn ghép một hàm mất mát quang trắc vào hệ của mình — dù là để tinh chỉnh tư thế, để căn chỉnh với bản đồ, hay để hiệu chỉnh trực tuyến — thì câu hỏi đầu tiên phải hỏi là: lịch làm trơn của tôi ở đâu.

D. Bốn nhánh còn lại, mỗi nhánh một câu hỏi khác

Bốn công trình còn lại trong danh sách không nằm trên cùng một trục với nhóm trên, vì mỗi cái hỏi một câu riêng.

- **NeRF-SLAM — bỏ độ sâu cảm biến đi thì sao?** Nó ghép một bộ bám vết dày đơn ảnh với một bản đồ kiểu Instant-NGP, và điểm thiết kế đáng học là dùng *độ bất định* của độ sâu ước lượng làm trọng số cho hàm mất mát dựng bản đồ [129]. Đây chính là tư duy ma trận thông tin của đồ thị nhân tử, mang sang phía bản đồ. Nếu bạn chỉ đọc một ý từ cả nhóm này, hãy đọc ý đó.
- **NICER-SLAM — chỉ có RGB thì lấy hình học ở đâu?** Nó thay độ sâu cảm biến bằng các tín hiệu học được: độ sâu đơn ảnh, pháp tuyến bề mặt, và ràng buộc căn chỉnh giữa các khung hình [130]. Nói cách khác, khi mất một ràng buộc vật lý thì phải mượn một tiên nghiệm học được để thay — và mọi giới hạn của tiên nghiệm đó, đặc biệt là vấn đề tỉ lệ, đi theo vào hệ. Chỗ này nổi thẳng sang F/30.
- **vMAP — vì sao cả cảnh phải nằm chung một trường?** Nó cấp cho mỗi vật thể một mạng nhỏ riêng, rồi hợp thành cảnh lúc dựng ảnh [131]. Cái được rất đáng chú ý: vật thể quan sát được một phần vẫn ra hình dạng kín, vì mỗi mạng chỉ phải học một vật chứ không phải cả căn phòng. Đây là bước bắc cầu sang bản đồ mức đối tượng ở F/33.
- **GO-SLAM — ai kéo bản đồ về khi đóng vòng?** Nó thêm phát hiện vòng lặp và tối ưu toàn cục, rồi biến dạng bản đồ neural theo tư thế đã sửa [127]. Đáng chú ý là cách nó làm được: bản đồ được neo vào các khung hình chốt, nên sửa tư thế thì kéo được bản đồ theo. Bài học chung sẽ trở lại ở mục sau: *biểu diễn nào neo được vào tư thế thì sửa được sau khi đóng vòng*.

E. Giải phẫu hàm mất mát, để đọc bài nhanh hơn

Mọi hệ trong dòng này tối ưu một tổng của vài số hạng, và bốn số hạng dưới đây gần như luôn có mặt. Nhận ra chúng giúp bạn đọc một bài mới trong mười phút thay vì một giờ, vì phần khác biệt giữa các bài thường chỉ là trọng số và một số hạng phụ.

- **Số hạng màu** — sai lệch giữa ảnh dựng và ảnh thật, trên các tia lấy mẫu. Đây là số hạng duy nhất bắt buộc, và cũng là số hạng *không* ràng buộc hình học, đúng như phép tính ở mục sau cho thấy.
- **Số hạng độ sâu** — sai lệch giữa độ sâu dựng ra và độ sâu cảm biến. Đây mới là số hạng thực sự dựng hình học, và nó biến mất trong các hệ chỉ có RGB. Bỏ nó đi thì phải mượn tiên nghiệm học được để thay.
- **Số hạng không gian trống** — ép mật độ về không dọc phần tia trước điểm chạm. Nghe phụ nhưng rất quan trọng: nếu thiếu, mô hình thoải mái để lại vật chất giả lơ lửng, vì không có gì phạt nó.
- **Số hạng chính quy hoá** — ép bề mặt trơn, ép Gaussian dẹt và bám mặt, ép pháp tuyến nhất quán. Đây là chỗ người ta *trả lại* phần ràng buộc hình học đã cố tình bỏ đi lúc đầu.

Một quy tắc đọc rút ra: xem tỉ lệ giữa số hạng độ sâu và số hạng màu. Tỉ lệ nghiêng về màu thì bài đó đang tối ưu cho ảnh đẹp; nghiêng về độ sâu thì đang tối ưu cho hình học đúng. Rất ít bài nói thẳng điều này, nhưng bảng siêu tham số luôn nói.

Bảng 6.1. Dòng NeRF làm bản đồ SLAM. Cột cuối là cột đáng đọc nhất: nó cho biết bạn sẽ gặp gì khi mang hệ đó ra khỏi điều kiện mà nó được công bố.

Hệ	Cơ chế cốt lõi	Giả định	Hỏng khi nào
iMAP	Một MLP là toàn bộ bản đồ; bám vết và dựng bản đồ đều bằng hạ gradient	RGB-D, một phòng, GPU để bàn	Cảnh lớn dần: quên thăm khốc, chi phí phát lại tăng theo diện tích
NICE-SLAM	Lưới đặc trưng nhiều mức, bộ giải mã cố định, cập nhật cục bộ	Biên cảnh biết trước để cập phát lưới	Cảnh lớn hoặc mở: bộ nhớ theo thể tích; không có đóng vòng
Co-SLAM	Mã hoá toạ độ trơn ghép bảng băm thưa	Bề mặt trơn từng mảnh	Va chạm bám ở vùng chi tiết dày
ESLAM	Ba mặt phẳng đặc trưng thay lưới 3D; TSDF thay mật độ	Cảnh không quá nhiều lớp chồng theo mỗi trục	Cảnh phức tạp làm ba mặt phẳng chồng lẫn thông tin
Point-SLAM	Đặc trưng neo trên điểm đã quan sát, dày theo chi tiết ảnh	Độ sâu đủ tốt để đặt điểm đúng chỗ	Độ sâu nhiễu đặt điểm sai; vùng chưa quan sát để trống
NeRF-SLAM	Bám vết dày đơn ảnh + bản đồ băm, trọng số theo bất định độ sâu	Bộ bám vết đủ tốt và chạy được thời gian thực	Bộ bám vết trôi thì bản đồ nhận sai; nặng GPU
NICER-SLAM	Chỉ RGB, mượn độ sâu và pháp tuyến học được làm ràng buộc	Tiên nghiệm đơn ảnh đúng cho loại cảnh này	Ra ngoài phân phối huấn luyện; tỉ lệ trôi
vMAP	Mỗi vật một mạng nhỏ, hợp lúc dựng ảnh	Có bộ tách vật thể đủ ổn định	Tách nhầm vật; cảnh không phân được thành vật
GO-SLAM	Bám vết dày + đóng vòng + tối ưu toàn cục, bản đồ biến dạng theo	Bản đồ neo được vào khung hình chốt	Ngân sách GPU lớn; độ trễ khi biến dạng bản đồ lớn

6.3 Dòng 3D Gaussian Splatting: vì sao rasteri hoá đổi cuộc chơi

Từ 2023 trở đi gần như cả lĩnh vực chuyển sang 3D Gaussian splatting. Chuyện này thường được kể như “nhanh hơn nên tốt hơn”, và cách kể đó bỏ mất phần đáng học. Khoản chênh tốc độ đến từ một thay đổi thuật toán duy nhất, và chính thay đổi ấy kéo theo ba hệ quả rất hợp với SLAM.

A. Khác biệt cơ chế, nói gọn trong một đoạn

NeRF phải *đi tìm* vật chất: với mỗi điểm ảnh, nó lấy hàng chục mẫu dọc tia và chạy mạng ở từng mẫu, vì nó không biết trước mặt nằm ở đâu. 3DGS đã lưu sẵn vật chất thành các nguyên thể tường minh, nên nó chỉ cần *rasterize*: chiếu chúng xuống mặt ảnh, sắp theo độ sâu rồi trộn — đúng việc mà phần cứng đồ hoạ sinh ra để làm [15]. Chi phí mỗi điểm ảnh vì thế đổi từ “số mẫu dọc tia” thành “số nguyên thể phủ lên điểm ảnh đó”, và con số thứ hai nhỏ hơn hẳn. Cơ chế đầy đủ nằm ở C/16/03-nerf-3dgs-va-hop-nhat; ở đây ta chỉ cần ba hệ quả của nó.

Chênh lệch nằm ở đâu thì viết ra được mà không cần một con số nào. Chi phí một khung hình của NeRF là (số điểm ảnh) $\times$ (số mẫu dọc mỗi tia) $\times$ (chi phí *một lần chạy mạng*). Của 3DGS là (số điểm ảnh) $\times$ (số nguyên thể phủ lên điểm ảnh đó) $\times$ (chi phí *một phép trộn*). Hai công thức có chung thừa số đầu và khác hẳn ở hai thừa số còn lại: số nguyên thể phủ lên một điểm ảnh nhỏ hơn số mẫu dọc một tia, và một phép trộn số học rẻ hơn một lần chạy mạng rất nhiều. Hai thừa số cùng nhỏ đi, nhân với nhau, là lý do khoảng cách lên tới nhiều bậc độ lớn — và đó là chênh lệch về *bản chất thuật toán*, không phải về cài đặt khéo hay vụng.

- **Cập nhật bản đồ trở thành sửa chữa cục bộ.** Thêm quan sát mới nghĩa là thêm vài nghìn Gaussian ở vùng vừa thấy và tinh chỉnh các Gaussian lân cận. Không có tham số dùng chung toàn cục nào bị chạm. Quên thăm hoa, nút thắt lớn nhất của iMAP, biến mất theo kiến trúc chứ không nhờ mẹo huấn luyện.
- **Bản đồ dịch chuyển được sau khi đóng vòng.** Mỗi Gaussian có một vị trí tường minh. Gắn nó vào khung hình chốt đã sinh ra nó, rồi khi tối ưu toàn cục sửa tư thế khung hình đó, ta dịch chuyển cứng cả cụm Gaussian theo. Với một MLP đơn, phép tương đương không tồn tại: không có cách nào “dịch một phần của trọng số”.
- **Đạo hàm theo tư thế có dạng giải tích.** Phép chiếu Gaussian là một chuỗi phép biến đổi trơn, nên gradient của sai số ảnh theo sáu bậc tự do của camera viết ra được, không cần đi qua mạng. Đây là điều làm bám vết theo mô hình chạy được ở tốc độ dùng được [132].

Hệ quả thứ hai đáng dừng lại một nhịp, vì nó chạm đúng một trong bốn nỗi đau của bạn. Trong hệ hiện tại, đóng vòng sửa tư thế rồi kéo các điểm bản đồ theo, và toàn bộ việc đó rẻ vì bản đồ là một tập điểm rời rạc gắn với khung hình chốt. Một bản đồ Gaussian giữ nguyên tính chất đó. Một bản đồ MLP thì không. Nếu bạn phải chọn một biểu diễn dày cho một hệ có đóng vòng, tiêu chí “sửa được sau khi đóng vòng” quan trọng hơn hẳn tiêu chí PSNR.

I Mạch 2 — dòng Gaussian làm bản đồ SLAM

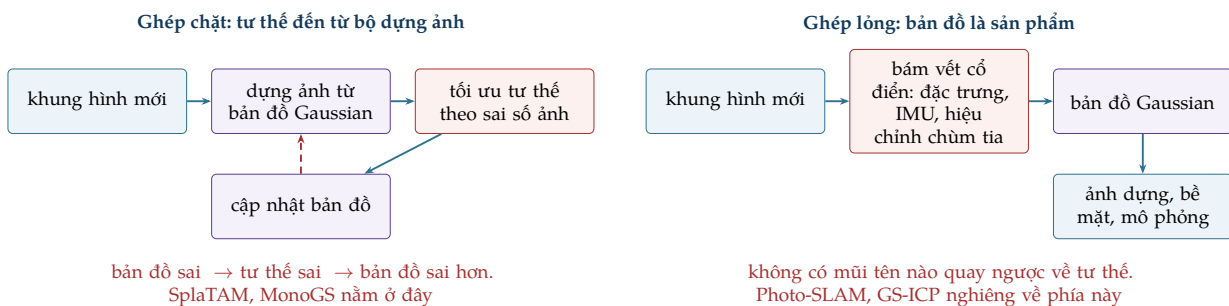
- | | | |
|---|---------------------|---|
| 1 | Vấn đề | Dòng NeRF chậm ngay cả trên GPU để bản — chậm tới mức khâu dựng bản đồ không theo kịp khâu bám vết — và lại quên chỗ cũ khi đi sang chỗ mới. |
| | Giải pháp | thay trường ảnh bằng tập Gaussian tường minh, dựng ảnh bằng raster hoá thay vì lấy mẫu dọc tia [15]. |
| | Giả định | bề mặt xấp xỉ được bằng các hạt mờ dị hướng, và số hạt phủ lên mỗi điểm ảnh không quá lớn. |
| | Giá phải trả | dung lượng bản đồ lớn hơn hẳn một MLP, và từ đây bộ nhớ chứ không phải tốc độ mới là ràng buộc quyết định. |
| | Hệ quả | cập nhật thành cục bộ, nên quên thăm khốc biến mất theo kiến trúc. |
| 2 | Vấn đề | Bám vết cần đạo hàm của sai số ảnh theo tư thế, và đi qua một mạng thì vừa chậm vừa ồn. |
| | Giải pháp | MonoGS viết thẳng Jacobian giải tích của phép chiếu Gaussian theo tư thế trên nhóm Lie [132]. |
| | Giả định | chuyển động giữa hai khung hình đủ nhỏ để nghiệm nằm trong bồn hút quang trắc. |
| | Giá phải trả | vẫn là bám vết theo mô hình, nên lỗi bản đồ chảy thẳng vào quỹ đạo. |
| | Vấn đề mới | chuyển động nhanh và ảnh mờ đẩy nghiệm ra ngoài bồn hút, và không có gì kéo nó về. |
| 3 | Vấn đề | Vòng lặp dựng ảnh nằm trên đường ước lượng tư thế nên vừa chậm vừa rủi ro. |
| | Giải pháp | GS-ICP tách hẳn hai việc: tư thế do ICP tổng quát trên tâm Gaussian, dựng ảnh chỉ là sản phẩm [133]. Photo-SLAM đi xa hơn, giữ nguyên đặc trưng và hiệu chỉnh chùm tia cổ điển [134]. |
| | Giả định | đã có một nguồn tư thế đủ tốt — và với bạn thì điều này đúng. |
| | Giá phải trả | bản đồ không còn đóng góp gì cho quỹ đạo; nó chỉ là bên tiêu thụ. |
| | Hệ quả | đây chính là mẫu ghép lỏng, và là cách vào rẻ nhất cho một hệ đang chạy sản phẩm. |
| 4 | Vấn đề | Một triệu Gaussian không vừa ngân sách bộ nhớ và băng thông của máy nhúng. |

Giải pháp	RTG-SLAM ép mỗi Gaussian hoặc đục hẳn hoặc trong hẳn, để một bề mặt cần ít hạt hơn nhiều [135].
Giả định	bề mặt trong cảnh phần lớn là đục, không bán trong suốt.
Giả phải trả	mất khả năng biểu diễn khói, kính, và vật phản chiếu.
Vấn đề mới	số hạt vẫn tăng theo diện tích đã đi qua, nên vấn đề chỉ được đẩy lùi chứ chưa được giải.

B. Sáu hệ, và điểm thiết kế đáng học của mỗi hệ

- **SplaTAM** — hệ RGB-D thuần Gaussian: bám vết bằng cách dựng ảnh từ bản đồ rồi tối ưu tư thế, dựng bản đồ bằng cách thêm Gaussian ở vùng chưa được phủ [136]. Điểm thiết kế đáng học là **mặt nạ hiển thị** (*visibility mask*): hệ chỉ tính mất mát ở các điểm ảnh mà bản đồ hiện có thật sự phủ tới. Nói theo ngôn ngữ của bạn, đó là một cổng loại bỏ phần dư không có cơ sở — cùng vai trò với việc bỏ các điểm không quan sát được trước khi tính phần dư trong hiệu chỉnh chùm tia.
- **MonoGS** — đưa 3DGS về đơn ảnh, và viết ra đạo hàm giải tích của sai số ảnh theo tư thế camera trong nhóm Lie [132]. Đây là bài trong nhóm gần với gu của một người làm tối ưu trên đa tạp nhất; nếu bạn định đọc mã nguồn một hệ 3DGS SLAM, hãy đọc hệ này.
- **GS-ICP SLAM** — tách hẳn bám vết khỏi dựng ảnh: tư thế do một biến thể ICP tổng quát tính trên chính tâm của các Gaussian, còn dựng ảnh chỉ là sản phẩm [133]. Đây là bằng chứng trực tiếp cho luận điểm trung tâm của chương: khi bỏ vòng lặp dựng ảnh ra khỏi đường ước lượng tư thế, hệ nhanh hơn hẳn, và chất lượng quỹ đạo không xấu đi.
- **Photo-SLAM** — giữ nguyên xương sống đặc trưng và hiệu chỉnh chùm tia cổ điển cho tư thế, gắn thêm một bản đồ Gaussian chỉ để dựng ảnh [134]. Đây là mẫu kiến trúc mà tôi cho là đáng bắt chước nhất cho một hệ đang chạy sản phẩm, vì nó không đặt tư thế vào tay bộ dựng ảnh.
- **RTG-SLAM** — tấn công thẳng vào số lượng Gaussian: ép mỗi Gaussian hoặc đục hẳn hoặc trong hẳn, để một bề mặt cần ít hạt hơn nhiều [135]. Nó cho thấy số Gaussian là một biến thiết kế chứ không phải hằng số của bài toán — điều rất quan trọng khi bạn có ngân sách bộ nhớ cố định.
- **OpenGS-SLAM** — dòng ghép ngữ nghĩa mở vào bản đồ Gaussian, để truy vấn bản đồ bằng ngôn ngữ. Tôi *không* chắc về chi tiết và kết quả của công trình này, nên chỉ nhắc tên và ý tưởng; phần ngữ nghĩa mở được bàn kỹ ở F/33.

Sáu hệ trên chia thành hai kiểu ghép nối, và phân biệt được hai kiểu đó quan trọng hơn nhớ tên từng hệ. Hình 6.5 vẽ cả hai.



Hình 6.5. Hai kiểu ghép nối, và đây là phân biệt quan trọng nhất của cả chương. Bên trái, tư thế được ước lượng bằng cách so ảnh dựng với ảnh thật, nên mọi lỗi bản đồ chây thẳng vào quỹ đạo; mũi tên đứt là vòng phản hồi đó. Bên phải, tư thế vẫn do đường cổ điển tính, còn bản đồ neural chỉ tiêu thụ tư thế; nó không thể làm quỹ đạo xấu đi, và cũng không thể làm quỹ đạo tốt lên. Chọn kiểu nào là quyết định kiến trúc, không phải chi tiết cài đặt.

Bảng 6.2. Dòng 3D Gaussian splatting làm bản đồ SLAM. Cột “tư thế đến từ đâu” quyết định hệ đó có thể làm hỏng quỹ đạo của bạn hay không.

Hệ	Tư thế đến từ đâu	Điểm thiết kế đáng học	Hỏng khi nào
SplaTAM	Dựng ảnh khác biệt được, RGB-D	Mặt nạ hiển thị: chỉ tính mất mát ở điểm ảnh bản đồ thật sự phủ	Độ sâu thiếu hoặc nhiều; chuyển động lớn giữa hai khung hình
MonoGS	Dựng ảnh, đơn ảnh, đạo hàm giải tích trên nhóm Lie	Viết thẳng Jacobian theo tư thế thay vì vi phân tự động	Tỉ lệ đơn ảnh; cảnh ít vân; chuyển động nhanh
GS-ICP SLAM	ICP tổng quát trên tâm Gaussian	Tách bám vết khỏi dựng ảnh, và hệ nhanh lên rõ rệt	Hình học nghèo nàn làm ICP trượt; vẫn cần độ sâu
Photo-SLAM	Đặc trưng + hiệu chỉnh chùm tia cổ điển	Bản đồ neural là bên tiêu thụ tư thế, không phải bên tạo ra nó	Bản đồ ảnh xấu ở nơi bám vết thừa khung hình chốt
RTG-SLAM	Bám vết dày, RGB-D	Ép Gaussian đục hoặc trong để giảm mạnh số hạt	Bề mặt bán trong suốt và vật phản chiếu
OpenGS-SLAM	<i>chưa kiểm chứng</i>	Ghép ngữ nghĩa mở vào bản đồ Gaussian	<i>chưa kiểm chứng</i>

C. Chỗ nổi rẻ nhất vào hệ bạn đang có

Nếu bạn muốn thử dòng này mà không đung vào đường ước lượng tư thế, có một chỗ nổi rất tự nhiên, và ít bài báo nói ra vì với họ nó hiển nhiên.

Bản 3DGS gốc khởi tạo các Gaussian từ đám mây điểm thừa của một pipeline dựng cảnh cổ điển. Hệ của bạn *đã* sinh ra đúng loại đám mây đó, mỗi khung hình chốt, kèm tư thế đã được hiệu chỉnh chùm tia. Nghĩa là bạn có sẵn đầu vào tốt nhất mà một hệ 3DGS có thể mơ tới: điểm 3D đã lọc ngoại lai, tư thế đã tối ưu toàn cục, và quan hệ đồng quan sát giữa các khung hình chốt.

Từ đó ra một kiến trúc ghép lỏng rất gọn, và nó không thêm rủi ro nào cho quỹ đạo:

- **Gieo hạt.** Mỗi điểm bản đồ thành một Gaussian, tỉ lệ khởi tạo theo khoảng cách tới các điểm lân cận.
- **Neo vào khung hình chốt.** Mỗi Gaussian ghi nhớ khung hình chốt đã sinh ra nó. Khi đóng vòng sửa tư thế khung hình đó, cả cụm Gaussian dịch theo bằng một phép biến đổi cứng.
- **Chia bản đồ con.** Dùng chính đồ thị đồng quan sát để cắt cảnh thành các bản đồ con, rồi chỉ tối ưu bản đồ con đang hoạt động. Đây là cách duy nhất giữ ngân sách bộ nhớ hữu hạn khi robot đi mãi.
- **Chạy lệch nhịp.** Khâu Gaussian chạy ở tần số thấp, trên một luồng riêng, và được phép rút khung hình. Nó là bên tiêu thụ, nên rút khung chỉ làm ảnh xấu đi chứ không làm hỏng quỹ đạo.

Đây không phải một phương pháp mới; nó là mẫu Photo-SLAM viết lại bằng các khối bạn đã có. Giá trị của nó nằm ở chỗ rủi ro gần bằng không: nếu bỏ hắt khâu Gaussian đi, hệ quay về đúng trạng thái hiện tại, không mất gì.

D. Ba chỗ dòng này vẫn chưa chạm tới

Đọc xong bảng trên dễ có cảm giác vấn đề đã xong. Chưa. Ba chỗ vẫn để ngỏ, và cả ba đều nằm đúng chỗ hệ của bạn đau.

- **Chuyển động nhanh.** Bám vết bằng dựng ảnh là một bài toán tối ưu quang trắc, nên nó có bồn hút hữu hạn. Chuyển động lớn giữa hai khung hình đẩy nghiệm ra ngoài bồn, và cách

chứa duy nhất trong khuôn khổ đó là một dự đoán ban đầu tốt — tức là quay lại cần IMU hoặc cần ghép đặc trưng. Đây chính là bài học của BARF nói lại lần nữa.

- **Ảnh mờ do chuyển động.** Một khung hình mờ không phải quan sát nhiễu của cảnh tĩnh. Nó là tích phân của cảnh dọc một quãng chuyển động. Hàm mất mát quang trắc không có chỗ nào biểu diễn điều đó, nên tối ưu sẽ giải thích vết mờ bằng cách nhét thêm hình học sai. Các dòng khử mờ có ý thức về chuyển động, như BAD-NeRF và BAD-Gaussians, đưa mô hình phơi sáng vào bộ dựng ảnh; chúng được bàn ở F/34.
- **Phơi sáng và cân bằng trắng tự động.** Cùng một cơ chế: mô hình buộc phải giải thích thay đổi độ sáng bằng hình học. Các hệ trong chương này thường chạy trên chuỗi có phơi sáng cố định, và điều đó ít khi được ghi rõ.

E. Điều cả hai dòng đều thiếu: một bài toán tối ưu chung

Có một khoảng trống chung cho toàn bộ chương, và nó là khoảng trống mà một người quen đồ thị nhân tử sẽ thấy ngay.

Hiệu chỉnh chùm tia tối ưu tư thế và cấu trúc *cùng lúc*, trong một bài toán bình phương tối thiểu duy nhất. Nó cho ba thứ đi kèm: nghiệm hội tụ bậc hai gần cực tiểu, một **ma trận thông tin** (*information matrix*) để biết mình chắc chắn tới đâu, và một chỗ tự nhiên để cắm thêm nhân tử — tiền tích phân IMU, ràng buộc đóng vòng, tiên nghiệm hiệu chỉnh.

Gần như mọi hệ trong chương này làm khác hẳn. Chúng *luân phiên*: cố định bản đồ rồi tối ưu tư thế vài bước, cố định tư thế rồi tối ưu bản đồ vài bước. Đây là **hạ theo khối toạ độ** (*block coordinate descent*), và nó kém hơn ở đúng ba chỗ vừa kể.

- **Hội tụ chậm và phụ thuộc lịch trình.** Số bước mỗi phía trở thành siêu tham số, và nó tương tác với ràng buộc thời gian thực. Chạy trên máy chậm hơn nghĩa là chạy ít bước hơn, tức là một hệ *khác*. Đây là lý do sâu xa vì sao kết quả trên Orin không suy ra được từ kết quả trên máy để bàn.
- **Không có ma trận thông tin.** Không có hiệp phương sai của tư thế, nên không có gì để cắm vào một bộ lọc, không có gì để so bằng kiểm định chi bình phương, và không có cách nào phát hiện hệ đang không chắc chắn.
- **Không có chỗ cho IMU.** Đây là chỗ thiếu nghiêm trọng nhất với hệ của bạn. Trong khuôn khổ luân phiên, không có chỗ tự nhiên nào để đặt một nhân tử tiền tích phân IMU với trọng số đúng. Vài công trình gần đây có ghép IMU vào, thường bằng cách dùng nó làm dự đoán ban đầu; tôi *không* nắm chắc kết quả của chúng và không mô tả số liệu.

Nói ngắn gọn: dòng này mạnh về biểu diễn và yếu về ước lượng. Hệ của bạn thì ngược lại. Đó chính là lý do mẫu ghép lỏng không phải một thoả hiệp tạm bợ, mà là cách ghép đúng thể mạnh của hai bên.

I 6.4 Hai trục đánh giá, và vì sao chúng bị trộn lẫn

Đây là mục quan trọng nhất của chương. Nếu bạn chỉ đọc một mục, đọc mục này.

A. Hai câu hỏi hoàn toàn khác nhau

Một hệ trong chương này báo cáo hai nhóm con số. Nhóm thứ nhất là chất lượng ảnh dựng lại: PSNR, SSIM, LPIPS [137], [138]. Nhóm thứ hai là độ chính xác quỹ đạo: **sai số quỹ đạo tuyệt đối** (*absolute trajectory error*), viết tắt ATE, đôi khi kèm RPE. Hai nhóm này trả lời hai câu hỏi khác nhau và có thể tách rời hoàn toàn.

Lý do tách rời đã được nêu ở C/16/03-nerf-3dgs-va-hop-nhat và đáng nhắc lại một câu: hàm mất mát chỉ ràng buộc ảnh dựng ra, không ràng buộc hình học. Màu phụ thuộc góc nhìn cho mô hình thêm một đường thoát: nó có thể “vẽ” thứ đáng lẽ phải “dựng”. Vì thế PSNR cao chứng minh mô hình khớp được các ảnh đã thấy, không chứng minh bản đồ đúng, và càng không chứng minh tư thế đúng.

Chiều ngược lại cũng đúng và ít được nói tới. Một hệ có ATE rất tốt vẫn có thể cho ảnh dựng xấu, đơn giản vì nó dành ít tài nguyên cho bản đồ. Photo-SLAM là ví dụ có chủ ý theo hướng ngược lại: quỹ đạo do đường cổ điển bảo đảm, còn ảnh đẹp là phần thêm.

Hình 6.6 bày cả hai trục lên một mặt phẳng. Đây là hình bạn nên vẽ lại cho chính hệ của mình.



Hình 6.6. Hai trục đánh giá trên một mặt phẳng; các nhãn là góc phân tử khái niệm, không phải vị trí đo được của hệ cụ thể nào. Bảng kết quả chỉ có cột PSNR không phân biệt được góc trên bên phải với góc dưới bên phải, và đó chính là chỗ người đọc bị dẫn sai. Quy tắc thực dụng: trước khi tin một hệ, hãy hỏi nó nằm ở đâu theo trục dọc, đo bằng đúng giao thức mà bạn dùng cho hệ hiện tại.

Bảng 6.3. Ba nhóm thước đo cho cùng một hệ. Cột cuối là cột hay bị bỏ qua nhất, và nó là lý do một hệ có thể đứng đầu bảng ở cột đầu mà vẫn không dùng được cho robot.

Nhóm	Đo cái gì	Trả lời được câu hỏi nào	Không chứng minh được gì
PSNR, SSIM, LPIPS	Ảnh dựng gần ảnh thật tới đâu	Bản đồ có dùng được để xem, để mô phỏng cảm biến không	Không nói gì về hình học, và không nói gì về tư thế
ATE, RPE	Quỹ đạo lệch chân trị bao nhiêu	Hệ có định vị được không — câu hỏi quyết định của SLAM	Không nói bản đồ có dày, có kín, có dùng để tránh vật cản được không
Sai số hình học: độ sâu L1, độ chính xác và độ phủ, F-score với lưới chân trị	Bề mặt dựng ra lệch bề mặt thật bao nhiêu	Bản đồ có dùng để lập kế hoạch, để đo đạc được không	Không nói ảnh có đẹp không, và cần chân trị hình học nên hiếm khi có ngoài đời

B. Ví dụ tính tay: vì sao một sai lệch hình học lớn chỉ tốn một điểm ảnh

Câu “PSNR không chứng minh hình học” nghe như một lời cảnh báo chung chung. Nó không phải vậy. Nó là một bất đẳng thức số học, và tính ra được trong ba dòng.

Trước khi tính, nói rõ một điều: **ví dụ dưới đây là một bài toán giả định hoàn toàn**. Mọi con số trong đó đều do mình đặt ra để thay vào công thức, không con số nào đến từ một phép đo trên thiết bị, cũng không con số nào trích từ một bài báo. Thứ đáng mang đi là *hình dạng* của quan hệ — Δu tỉ lệ thuận với khoảng dịch camera và tỉ lệ nghịch với bình phương độ sâu — chứ không phải con số ở vế phải.

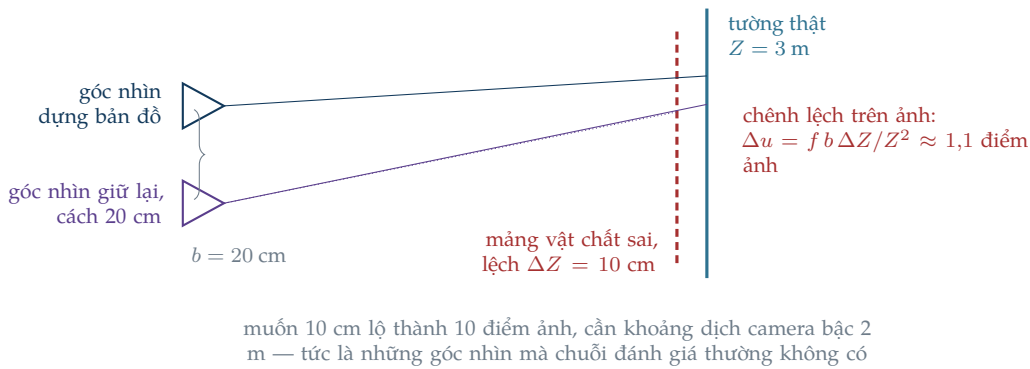
Xét một bức tường phẳng cách camera $Z = 3$ m. Giả sử bản đồ đặt sai một mảng vật chất, lệch $\Delta Z = 10$ cm về phía trước tường. Với robot, sai số này rất nghiêm trọng: nó là chênh lệch giữa dừng đúng và va vào tường. Bây giờ hỏi xem nó lộ ra bao nhiêu trong ảnh.

Một điểm ở độ sâu Z , nhìn từ một camera dịch ngang một đoạn b , có vị trí ảnh dịch đi một lượng tỉ lệ với fb/Z . Sai một lượng ΔZ làm vị trí ảnh lệch đi

$$\Delta u \approx \frac{fb\Delta Z}{Z^2} = \frac{500 \times 0,2 \times 0,1}{3^2} \approx 1,1 \text{ điểm ảnh},$$

với tiêu cự $f = 500$ điểm ảnh và khoảng dịch camera $b = 20$ cm. *Một điểm ảnh*. Trên một ảnh mà mô hình còn được phép chỉnh màu theo góc nhìn, một điểm ảnh chìm hoàn toàn trong nhiễu cảm biến và trong sai số nội suy. PSNR gần như không đổi. Bản đồ thì sai 10 cm.

Hình 6.7 vẽ đúng hình học của phép tính đó.



Hình 6.7. Vì sao sai số hình học trốn được khỏi PSNR; các con số trên hình là *giả định* của ví dụ, không phải số đo. Với những giá trị giả định đó, sai 10 cm ở tầm 3 m chỉ dời điểm ảnh khoảng một điểm ảnh khi góc nhìn kiểm tra chỉ cách góc nhìn dựng bản đồ 20 cm. Độ nhạy tỉ lệ thuận với khoảng dịch camera và tỉ lệ nghịch với bình phương độ sâu. Hệ quả thực hành: chỉ khi tập góc nhìn giữ lại *cách xa* quỹ đạo chụp thì chất lượng ảnh mới bắt đầu nói được điều gì về hình học — và đó đúng là loại tập đánh giá mà các bài trong chương này hiếm khi dựng.

Ba hệ quả rút ra ngay, và cả ba đều dùng được khi bạn đọc bảng của người khác.

- **Độ nhạy tỉ lệ nghịch với bình phương độ sâu.** Vật xa hoàn toàn không bị ràng buộc bởi ảnh. Đây là lý do các bản đồ neural dựng ngoài trời hay có “sương mù” vật chất giả ở nền xa.
- **Độ nhạy tỉ lệ thuận với khoảng dịch camera.** Một tập góc nhìn giữ lại nằm sát quỹ đạo chụp gần như không kiểm tra được hình học. Muốn kiểm tra thật, phải giữ lại những góc nhìn *lệch hẳn* khỏi quỹ đạo.
- **Muốn ràng buộc hình học thì phải đo hình học.** Không có cách nào lách: nếu đầu ra sẽ được dùng để tránh vật cản, thước đo phải là sai số bề mặt so với một quét chân trị, không phải PSNR.

C. Bảy câu hỏi phải hỏi trước khi tin một bảng kết quả

Phần lớn kết quả trong chương này được công bố trong một điều kiện rất hẹp, và điều kiện đó thường nằm rải rác trong phần thí nghiệm chứ không nằm ở bảng. Bảy câu hỏi dưới đây lọc được gần hết.

1. **Ảnh được đo trên góc nhìn nào?** Rất nhiều bảng báo cáo PSNR trên chính các khung hình đầu vào hoặc các khung hình chốt. Con số đó đo *độ khớp*, không đo khả năng tổng quát sang góc nhìn mới. Hai đại lượng này khác nhau rất xa với một mô hình có hàng triệu tham số cho một cảnh.
2. **Tư thế lúc dựng bản đồ là tư thế ước lượng hay chân trị?** Một số thí nghiệm chạy ở chế độ “chỉ dựng bản đồ” với tư thế chân trị. Kết quả đó hợp lệ, nhưng nó nói về bản đồ chứ không nói về hệ SLAM.
3. **Độ sâu đến từ đâu?** Replica cho độ sâu tổng hợp hoàn hảo [139]. Cảm biến thật cho độ sâu có nhiễu, có lỗ, và có giới hạn tầm. Một hệ dựa nặng vào độ sâu sẽ tụt hạng rất nhiều giữa hai điều kiện này.
4. **Chuỗi dài bao nhiêu và có vòng không?** Phần lớn chuỗi trong Replica và TUM RGB-D chỉ dài vài chục giây tới vài phút, đi bộ trong một phòng [140], [141]. Trôi tích lũy và đóng vòng gần như không được kiểm tra.
5. **ATE được căn chỉnh thế nào?** Với hệ đơn ảnh, ATE thường tính sau khi căn chỉnh có cả tỉ lệ. Phép căn chỉnh đó *hấp thụ* sai lệch tỉ lệ, tức là giấu đúng chế độ hỏng nguy hiểm nhất của hệ đơn ảnh. Nếu bạn so với hệ stereo-inertial của mình, hai con số không cùng đơn vị nghĩa.
6. **Phần cứng nào và tốc độ nào?** Con số fps trong bài gần như luôn là GPU để bàn. Và “thời gian thực” trong nhiều bài nghĩa là xử lý kịp tốc độ phát lại đã giảm, không phải kịp tốc độ cảm biến.
7. **Baseline cổ điển có được tinh chỉnh tử tế không?** Đây là câu hỏi của E/24 áp vào đây. Một bảng so bản đồ neural với ORB-SLAM3 chạy cấu hình mặc định trên chuỗi mà cấu hình đó không hợp là một so sánh không có nghĩa.

CẠM BẮY

Cái bẫy hay gặp nhất không phải số liệu sai mà là *ghép hai kết luận đúng thành một kết luận sai*. Bài báo nói “PSNR của chúng tôi cao nhất bảng” — đúng. Bài báo nói “ATE của chúng tôi tốt hơn NICE-SLAM” — cũng đúng. Người đọc ghép lại thành “hệ này tốt hơn hệ tôi đang chạy” — sai, vì cả hai hệ được so đều thuộc dòng neural, và cả hai đều có thể tệ hơn một hệ đặc trưng cổ điển đã tinh chỉnh trên cùng chuỗi. Cách kiểm tra rẻ nhất là tìm trong bài xem có dòng nào so với một hệ cổ điển *trên cùng giao thức căn chỉnh* hay không. Nếu không có, bạn chưa biết gì về vị trí của nó theo trục dọc trong Hình 6.6.

Bảng 6.4. Điều kiện ẩn sau bốn tập dữ liệu hay gặp. Bảng này giải thích vì sao thứ hạng trên hai dòng đầu gần như không dự đoán được hành vi trên dòng cuối — tập dữ liệu của chính bạn.

Tập	Độ sâu	Độ dài, vòng	Điều kiện để bị bỏ qua
Replica	Tổng hợp, hoàn hảo, không lỗ	Ngắn, quỹ đạo dựng sẵn	Không nhiều ảnh, không mờ, phơi sáng cố định; đây là điều kiện dễ nhất có thể
TUM RGB-D	Cảm biến kết cấu ánh sáng, có nhiều và lỗ	Vài chục giây, ít vòng lớn	Trong nhà, đi bộ, một phòng; chuyển động vẫn khá êm
ScanNet	Cảm biến, quét tay	Dài hơn, có quay lại chỗ cũ	Ảnh mờ và phơi sáng tự động xuất hiện thật; chân trị tư thế cũng từ một hệ dựng lại
EuRoC, chuỗi của bạn	Stereo, không có độ sâu trực tiếp	Vài phút, chuyển động mạnh	Bay trong không gian rộng, chuyển động nhanh, ảnh mờ, thay đổi sáng — gần như mọi giả định của chương đều bị vi phạm

D. Ba thí nghiệm bạn phải tự chạy

Đọc bảng của người khác chỉ để loại bớt ứng viên. Muốn biết một hệ có giúp được mình không, có ba thí nghiệm, và chúng đi theo đúng thứ tự kinh tế: rẻ trước, đắt sau.

- **Thí nghiệm oracle tư thế.** Cấp tư thế chân trị cho hệ và chỉ để nó dựng bản đồ. Con số thu được là *trần* chất lượng bản đồ khi khâu tư thế hoàn hảo. Nếu trần đó đã không đủ cho việc hạ nguồn của bạn, mọi công sức về sau đều bị chặn trên. Đây đúng là công cụ ở E/24, áp vào đây.
- **Thí nghiệm tách ghép nối.** Chạy cùng hệ ở hai chế độ: tư thế do bộ dựng ảnh tính, và tư thế do đường cổ điển của bạn tính. Hiệu số ATE giữa hai chế độ trả lời thẳng câu hỏi “bản đồ neural có làm quỹ đạo tốt lên không”. Kinh nghiệm đọc các bảng đã công bố cho thấy câu trả lời thường là không; nhưng đó là kết luận rút từ bảng của người khác, không phải phép đo của tôi.
- **Thí nghiệm ngoài phân phối.** Chạy trên chính chuỗi khó của bạn: chuyển động nhanh, ảnh mờ, phơi sáng tự động, ngoài trời. Đây là chỗ khoảng cách giữa bảng công bố và thực tế lộ ra nhiều nhất, và cũng là chỗ rẻ nhất để phát hiện sớm.

Và một nhắc nhở về kỷ luật đo, vì nó đã cản bạn ở chỗ khác: mọi con số trên đây phải chạy nhiều lần. Các hệ này có thêm hai nguồn tản mát mà hệ cổ điển không có — khởi tạo ngẫu nhiên của bản đồ và số bước tối ưu thực sự chạy được dưới ràng buộc thời gian thực. Sàn nhiều vì thế thường rộng hơn bạn quen.

E. Dựng tập góc nhìn giữ lại cho đúng

Hai mục trên gộp lại thành một yêu cầu rất cụ thể, và nó đáng viết riêng vì hầu như không ai làm đúng.

Cách làm mặc định là giữ lại mỗi khung hình thứ tám của chính chuỗi đầu vào. Cách này rẻ và gần như vô dụng. Lý do đã tính ở mục B: khung hình thứ tám chỉ cách khung hình lân cận vài centimét, mà $\Delta u \approx fb\Delta Z/Z^2$ tỉ lệ thuận với khoảng dịch b . Thu b từ hàng chục centimét xuống vài centimét thì độ nhạy tụt theo đúng tỉ lệ ấy, và với các giá trị giả định của mục B thì sai lệch hình học lộ ra ở mức nhỏ hơn một phần mười điểm ảnh — tức là chìm hẳn dưới nhiễu. Bạn đang đo khả năng nội suy giữa hai ảnh liền kề, không đo bản đồ.

Ba cách dựng tập giữ lại có nghĩa, xếp theo công sức tăng dần:

- **Giữ lại theo đoạn, không theo khung hình.** Cắt bỏ nguyên một đoạn quỹ đạo dài vài giây khỏi dữ liệu dựng bản đồ, rồi đánh giá trên đoạn đó. Khoảng dịch camera khi ấy tính bằng mét chứ không bằng centimét.
- **Quay hai lượt.** Đi qua cùng một chỗ hai lần theo hai đường khác nhau, dựng bản đồ bằng lượt một, đánh giá bằng lượt hai. Đây là cách rẻ nhất cho dữ liệu tự thu, và nó cũng cho bạn luôn một phép thử đóng vòng.
- **Đo thẳng hình học.** Nếu có một quét chân trị — máy quét laser, hoặc một mô hình đã dựng kỹ ngoại tuyến — thì bỏ hẳn ảnh đi và đo sai số bề mặt. Đây là câu trả lời đúng khi đầu ra sẽ được dùng để tránh vật cản.

Nếu bạn chỉ đọc bảng của người khác, câu hỏi rút gọn thành một dòng: tập giữ lại của họ cách quỹ đạo chụp bao xa? Bài nào không trả lời được câu đó thì cột PSNR của nó chỉ nói về nội suy.

6.5 Ngân sách trên Jetson Orin, và câu hỏi có đáng cho robot chưa

Mọi thứ ở trên đều giả định có một GPU để bàn. Mục này hỏi chi phí ấy có *hình dạng* gì trên phần cứng bạn thật sự triển khai. Mục này cố tình **không đưa ra con số nào**: mình không có phép đo trên Orin cho bất kỳ hệ nào trong chương, và một ước lượng tự tính trên giấy rất dễ bị đọc lại thành một số đo. Cái đáng học ở đây không phải độ lớn, mà là *thừa số nào nhân với thừa số nào* — vì chính cấu trúc đó nói cho bạn biết nên cắt ở chỗ nào, và nó đúng bất kể con số cụ thể là bao nhiêu.

A. Bộ nhớ: đếm bằng công thức, không quy ra con số

Một Gaussian lưu 3 số vị trí, 3 số tỉ lệ, 4 số quaternion, 1 số độ đục và 48 hệ số màu bậc ba — gần sáu chục số thực cho *một* hạt. Đã là bội số lớn so với một điểm bản đồ thưa, vốn chỉ cần ba số tọa độ cộng một bộ mô tả ngắn. Và một cảnh cỡ căn phòng dựng cho tử tế cần bậc một triệu hạt như thế.

Nhưng bộ nhớ tham số chưa phải khoản phải trả. Trong SLAM, bản đồ *đang được tối ưu* chứ không nằm yên. Mỗi tham số cần thêm một ô gradient và hai ô trạng thái của bộ tối ưu kiểu Adam, nên bộ nhớ thực tế là bốn lần bộ nhớ tham số:

$$\text{bộ nhớ bản đồ} \propto \underbrace{4}_{\text{tham số, gradient, hai ô Adam}} \times \underbrace{59}_{\text{số thực mỗi hạt}} \times \underbrace{N}_{\text{số Gaussian}}.$$

Thừa số duy nhất bạn điều khiển được là N ; hai thừa số kia là hằng số của biểu diễn. Đó là lý do mọi công trình muốn đưa 3DGS lên máy nhúng đều tấn công vào N chứ không vào chỗ nào khác.

Hướng của kết luận thì rõ, và nó không cần một con số nào: **bản đồ Gaussian tốn bộ nhớ hơn đám mây điểm thưa vài bậc độ lớn cho cùng một căn phòng, và bộ nhớ mới là ràng buộc quyết định của cả dòng này chứ không phải tốc độ dựng ảnh**. Muốn biết bao nhiêu megabyte trên cảnh của bạn thì phải đo trên cảnh của bạn: số hạt mà một cảnh cần phụ thuộc vào chính cảnh đó, nên không có hằng số nào chuyển thẳng được từ bài báo sang.

Dung lượng nghe thì vẫn có vẻ vừa một Orin. Nhưng bộ nhớ trên Orin là bộ nhớ *dùng chung* giữa CPU và GPU, và nó đang phải chứa cả hệ SLAM, cả các mạng nhận thức khác, cả ROS. Quan trọng hơn, dung lượng không phải ràng buộc cản trước.

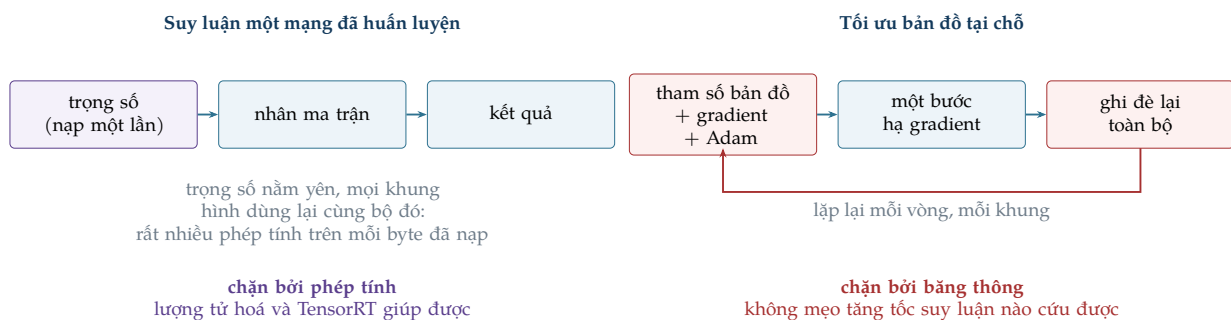
B. Bảng thông mới là ràng buộc cần trước

Mỗi vòng tối ưu phải đọc *toàn bộ* tham số cùng gradient và trạng thái bộ tối ưu, rồi ghi lại chúng. Viết lưu lượng ấy thành công thức là đủ thấy vấn đề:

$$\text{lưu lượng mỗi giây} \propto \underbrace{N}_{\text{số hạt}} \times \underbrace{4}_{\text{tham số+gradient+Adam}} \times \underbrace{2}_{\text{đọc rồi ghi}} \times \underbrace{n_{\text{vòng}}}_{\text{vòng tối ưu mỗi khung}} \times \underbrace{f}_{\text{khung mỗi giây}}.$$

Năm thừa số, và bốn trong năm là hằng số của kiến trúc chứ không phải của cài đặt. Kết luận định tính rút ra: với một bản đồ cỡ căn phòng, riêng việc *di chuyển* tham số qua lại đã ăn phần lớn băng thông bộ nhớ của một máy nhúng, trước khi thực hiện một phép nhân nào. Đây là lý do cơ học — không phải lý do kỹ thuật cài đặt — khiến một hệ 3DGS SLAM đầy đủ chưa chạy được ở tốc độ cảm biến trên phần cứng nhúng. Mình không có phép đo để nói nó hút bao nhiêu lần; công thức chỉ nói nó hút theo hướng nào, và cắt thừa số nào thì đỡ. Ba hướng thoát đều tồn tại và đều phải trả giá: giảm số Gaussian như RTG-SLAM, giảm số vòng tối ưu mỗi khung, hoặc chạy khâu bản đồ ở tần số thấp hơn hẳn khâu bám vết.

Hình 6.8 nói vì sao khoản này khác hẳn mọi khoản học sâu khác bạn từng đưa lên Orin.



Hình 6.8. Vì sao khoản chi phí ở chương này không giống bất kỳ khoản học sâu nào khác trong Phần F. Bên trái là thứ bạn đã quen triển khai: trọng số nạp một lần rồi nằm yên, nên mỗi byte đã nạp được dùng cho rất nhiều phép tính và nút thắt nằm ở phép tính. Bên phải là thứ đang chạy trong một hệ SLAM neural: chính tham số bản đồ là biến đang bị sửa, nên mỗi vòng phải đọc rồi ghi lại toàn bộ, và số phép tính trên mỗi byte rất thấp. Mũi tên đỏ quay ngược là toàn bộ khác biệt — nó biến bài toán từ “chặn bởi phép tính” thành “chặn bởi băng thông”, và đó là lý do lượng tử hoá cùng TensorRT gần như không giúp gì.

Còn một ràng buộc nữa mà bảng thông số không nói: nhiệt và điện. Orin chạy ở nhiều mức công suất, và mức thấp cắt cả xung nhịp GPU lẫn băng thông. Một khối chiếm gần trọn băng thông trong nhiều phút sẽ kéo cả hệ vào vùng giảm xung, nên các khối khác — phát hiện vật thể, lập kế hoạch — chậm theo. Đây là dạng ghép nối không xuất hiện trong bất kỳ phép đo đơn lẻ nào, và chỉ lộ ra khi chạy cả hệ trong thời gian dài.

Với dòng NeRF, chỗ nghẽn dời từ băng thông sang phép tính, và công thức đổi theo. Chi phí một vòng tỉ lệ với (số tia lấy mẫu) $\times$ (số mẫu đọc mỗi tia) $\times$ (chi phí một lần chạy MLP), rồi vẫn phải nhân tiếp với số vòng mỗi khung và số khung mỗi giây. Ba thừa số đầu đều là hằng số kiến trúc, và tích của chúng lớn vì mỗi tia phải dò chứ không *chiếu* — đúng cơ chế đã nói ở mục 3. Hướng kết luận giống hệt dòng Gaussian: một khối duy nhất ăn phần lớn ngân sách của máy, trong khi máy còn phải chạy cả phần nhận thức lẫn phần điều khiển. Lần nữa, đây là hình dạng của bài toán chứ không phải một phép đo; con số thật chỉ có nếu bạn chạy trên chính thiết bị của mình.

Một phản xạ không dùng được ở đây. Khi thấy một mô hình quá chậm trên Orin, phản xạ quen thuộc là lượng tử hoá và biên dịch bằng TensorRT. Ở đây phản xạ đó gần như không giúp gì, và lý do đáng nhớ: cái đang chạy không phải *suy luận* mà là một *vòng lặp tối ưu* trên chính tham số bản đồ. Lượng tử hoá làm hỏng gradient; đồ thị tính toán thay đổi liên tục vì Gaussian được thêm và bị cắt; và phần lớn thời gian nằm ở lưu lượng bộ nhớ chứ không ở phép nhân. Đây là chỗ

khác biệt căn bản giữa chương này và mọi chương khác của Phần F, nơi ta triển khai một mạng đã huấn luyện xong.

C. Vì sao gần như mọi kết quả đều là trong nhà và chuỗi ngắn

Ba ràng buộc trên giải thích luôn một điều mà người đọc thường coi là ngẫu nhiên: vì sao gần như toàn bộ kết quả trong chương này đến từ cảnh trong nhà, chuỗi vài chục giây tới vài phút, có độ sâu RGB-D, chạy trên GPU để bàn.

- **Trong nhà** vì cảnh có biên, nên lưới hoặc tập Gaussian cấp phát được. Ngoài trời không có biên, và bộ nhớ tăng theo diện tích đã đi qua.
- **Chuỗi ngắn** vì bản đồ chỉ lớn thêm chứ không bao giờ nhỏ đi, và vì chuỗi ngắn thì trôi tích lũy chưa kịp thành vấn đề.
- **RGB-D** vì độ sâu cho hình học gần như miễn phí. Bỏ độ sâu đi thì phải mượn tiên nghiệm học được, và kéo theo mọi giới hạn của tiên nghiệm đó.
- **GPU để bàn** vì ở đó cả dung lượng lẫn băng thông bộ nhớ đều dư ra rất nhiều so với một máy nhúng, và lại không phải chia cho phần còn lại của robot.
- **Camera đi chậm và đều** vì bám vết quang trắc có bồn hút hữu hạn, và vì ảnh mờ phá thẳng vào hàm mất mát.

Không điều nào ở trên là lỗi của các tác giả. Đó là những điều kiện cần để bài toán giải được ở trạng thái hiện tại của lĩnh vực. Nhưng chúng *là* khoảng cách giữa bảng kết quả và chiếc robot của bạn, và đó là khoảng cách bạn phải tự trả.

D. Điền số vào bốn cổng

Khung bốn cổng ở F/27 có ích nhất khi nó được điền số thật thay vì cảm giác. Bảng 6.5 điền cho chương này, với ba nhu cầu hạ nguồn khác nhau. Chú ý là cùng một công nghệ cho ba kết luận khác nhau, và khác biệt nằm ở nhu cầu chứ không ở công nghệ.

Bảng 6.5. Bốn cổng của F/27 điền cho bản đồ neural, tách theo nhu cầu hạ nguồn. Cùng một công nghệ, ba kết luận khác nhau — và đó là lý do câu hỏi “bản đồ neural có đáng không” không có câu trả lời chung.

Cổng	Để định vị	Để tránh vật cản	Để dựng lại và xem
Bài toán có thật không	Không: hệ hiện tại đã định vị tốt	Có, nhưng TSDF và lưới chiếm dụng đã giải	Có, và không có lời giải cổ điển nào thay được
Đo được không	Có: ATE, cùng giao thức căn chỉnh	Có, nhưng cần chân trị hình học, thứ khó có ngoài đời	Có: PSNR, SSIM, LPIPS trên góc nhìn giữ lại thật sự
Vừa ngân sách Orin không	Không: riêng lưu lượng đọc-ghi tham số mỗi vòng đã ăn phần lớn băng thông	Không ở tốc độ cảm biến; có nếu chạy khâu bản đồ ở tần số thấp	Có, nếu chạy ngoại tuyến trên máy trạm
Hồng ra sao	Im lặng, qua vòng phản hồi dương, không có chỉ báo	Vật chất giả lơ lửng bị coi là vật cản, hoặc ngược lại	Ảnh xấu đi, thấy ngay bằng mắt — chế độ hồng lạnh nhất
Kết luận	Chưa đáng	Dùng TSDF trước	Đáng, và nên làm ngoại tuyến

Hàng cuối là chỗ đáng nhớ. Cùng một đồng mã nguồn, cùng một bài báo, mà kết luận đảo hẳn theo việc bạn định dùng đầu ra vào việc gì. Ai hỏi “bản đồ neural có đáng không” mà không nói rõ dùng để làm gì thì đang hỏi một câu không có đáp án.

E. Kết luận thẳng thắn: có đáng chưa

Trả lời tách theo mục đích, vì gộp chung là chỗ mọi tranh luận đi vào ngõ cụt.

- **Thay bản đồ định vị bằng bản đồ neural: chưa đáng.** Không có bằng chứng nào cho thấy nó cải thiện ATE của một hệ stereo-inertial đã tinh chỉnh kỹ, trên chuỗi thật, ở ngân sách của Orin. Đổi lại bạn mất tính chắc chắn đoán được, mất tính lặp lại, và tiêu gần hết ngân sách tính toán. Đây là kết luận đọc từ điều kiện thí nghiệm của các bài đã công bố, không phải từ một phép đo của tôi — nhưng nó là kết luận mà chính các điều kiện đó bắt buộc.
- **Bản đồ neural như một sản phẩm phụ, chạy ở tần số thấp: đáng thử, nếu có nhu cầu thật.** Nếu bài toán hạ nguồn cần ảnh — xem lại từ xa, mô phỏng cảm biến, sinh dữ liệu huấn luyện, kiểm tra công trình — thì mẫu ghép lỏng của Photo-SLAM là cách vào rẻ và không rủi ro. Tư thế vẫn do đường cổ điển bảo đảm, nên xấu nhất là bạn mất một luồng GPU.
- **Bản đồ dày để tránh vật cản: chọn TSDF hoặc lưới chiếm dụng trước.** Chúng rẻ hơn nhiều bậc, có mô hình xác suất, có bề mặt định nghĩa sạch, và đã được kiểm chứng lâu năm. Chỉ chuyển sang biểu diễn neural khi bạn chỉ ra được một việc cụ thể mà chúng không làm nổi.
- **Dựng lại chất lượng cao ngoại tuyến: đáng, và đây là chỗ dòng này thật sự thắng.** Bỏ ràng buộc thời gian thực đi thì mọi phép tính ở trên đối hân. Ghi bag trên robot, chạy dựng bản đồ trên máy trạm, đó là cách dùng đúng sức của công nghệ này hôm nay.

F. Bốn bài học thiết kế đáng giữ kể cả khi không dùng

Đây là phần trả lời câu hỏi thứ tư của khung đọc: nếu không lấy nguyên phương pháp thì lấy được gì.

- **Chọn biểu diễn theo tiêu chí “sửa được sau khi đóng vòng”.** Đây là tiêu chí phân loại mạnh nhất trong cả chương, và nó tách dòng Gaussian khỏi dòng MLP đơn rõ hơn mọi bảng PSNR. Nó cũng đúng cho mọi thứ khác bạn gắn vào bản đồ: bản đồ ngữ nghĩa, bản đồ chiếm dụng, bộ nhớ đóng vòng.
- **Mọi hàm mất mát quang trắc cần một lịch làm trơn.** Bài học của BARF, và nó là bản sao của kim tự tháp ảnh trong các hệ trực tiếp cổ điển. Nếu bạn thêm một số hạng quang trắc vào hệ mình mà không có lịch làm trơn, nó sẽ mắc kẹt và bạn sẽ đổ lỗi nhầm cho khởi tạo.
- **Chốt khung hình theo lượng thông tin mới, đo được.** iMAP đo bằng sai số dựng ảnh dưới bản đồ hiện tại. Ý tưởng đó chuyển được sang hệ cổ điển: thay ngưỡng “số điểm khớp dưới 90%” bằng một đại lượng có nghĩa thống kê. Bạn đã biết chính sách chốt khung hình là chỗ nhạy cảm bậc nhất của hệ mình.
- **Trọng số của phần dư phải đến từ bất định, không đến từ hằng số.** NeRF-SLAM cân hàm mất mát dựng bản đồ bằng bất định của độ sâu. Đây đúng là tư duy ma trận thông tin, và nó là sợi dây nối chương này với F/34.

CẦN NHỚ

Hai câu hỏi phải luôn tách rời: *bản đồ này trông có đẹp không* và *quỹ đạo này có đúng không*. Chúng được tối ưu bởi hai hàm mất mát khác nhau, đo bằng hai bộ thước đo khác nhau, và một hệ hoàn toàn có thể thắng ở câu thứ nhất trong khi thua ở câu thứ hai. Quy tắc kiến trúc rút ra: đừng để bộ dựng ảnh là nguồn của tư thế, trừ khi bạn đã tự đo và thấy nó tốt hơn đường cổ điển của mình trên chính chuỗi khó của mình. [T3]

I 6.6 Giới hạn và trường hợp hòng

Giới hạn nặng nhất đã nói ở trên và đáng nhắc lại theo cách khác: các hệ này tối ưu *riêng cho từng cảnh*. Chi phí vì thế là chi phí lặp lại, không phải chi phí một lần như huấn luyện một mạng rồi triển khai. Mọi kỹ thuật tăng tốc suy luận mà bạn quen đều không áp được. Các dòng mô hình tiến thẳng, dự đoán thẳng tập Gaussian từ vài ảnh, đang xoá dần ranh giới này; tại thời điểm viết, chất lượng của chúng vẫn dưới bản tối ưu riêng từng cảnh, và đây là quan sát xu hướng chứ chưa phải kết quả đã ổn định.

Chế độ hỏng thứ hai là hỏng *im lặng* và nó đáng sợ nhất. Khi tư thế sai vài phần mười độ, tối ưu không báo lỗi. Nó bù bằng cách sinh ra vật chất giả lơ lửng, và ảnh dựng ra vẫn đẹp. Với một hệ ghép chặt, vật chất giả đó lại quay lại làm sai tư thế của khung hình sau. Bạn có một vòng phản hồi dương mà không có chỉ báo nào bật lên. Ở hệ cổ điển, cùng tình huống ấy làm số điểm khớp tụt và làm phần dư tăng — tức là bạn *nhìn thấy* nó. Mất tính chẩn đoán được là khoản mất ít được nhắc nhất và đắt nhất khi triển khai.

Chế độ hỏng thứ ba nằm ở giả định về quá trình tạo ảnh. Cảnh phải tĩnh, phơi sáng phải nhất quán, ảnh không được mờ. Ba giả định này bị vi phạm liên tục trên một robot thật, và chúng bị vi phạm *cùng lúc* đúng vào những đoạn khó: robot quay nhanh trong hành lang thiếu sáng, camera tự tăng thời gian phơi, ảnh mờ đi, người đi qua khung hình. Mỗi vi phạm đều được mô hình giải thích bằng cách thêm hình học sai, vì đó là biến duy nhất nó có.

Cuối cùng, một giới hạn về kiến thức chứ không về công nghệ: tôi không có phép đo riêng nào cho các hệ trong chương này trên phần cứng Orin, và cũng không kiểm chứng được các con số hiệu năng mà các bài báo đưa ra. Vì thế mục 6.5 cổ tình chỉ nói về *cấu trúc* của chi phí — thừa số nào nhân với thừa số nào — chứ không đưa ra con số nào. Cấu trúc đó đủ để nói “khoản này bị chặn bởi băng thông chứ không bởi phép tính” và “thừa số duy nhất cắt được là số hạt”. Nó không đủ để nói hệ nào nhanh hơn hệ nào, cũng không đủ để nói hệt bao nhiêu lần. Nếu bạn cần một con số, chỉ có một cách lấy nó, là chạy trên chính thiết bị của mình.

6.7 Thực hành

Repo và tài nguyên. Mã nguồn của các hệ trong chương đều mở: [cvg/nice-slam](#), [HengyiWang/Co-SLAM](#), [idiap/ESLAM](#), [eriksandstroem/Point-SLAM](#), [youmi-zym/GO-SLAM](#) cho dòng NeRF; [spla-tam/SplaTAM](#), [muskie82/MonoGS](#), [Lab-of-AI-and-Robotics/GS_ICP_SLAM](#), [HuajianUP/Photo-SLAM](#) cho dòng Gaussian. Bản 3DGS gốc là [graphdeco-inria/gaussian-splatting](#). Để đo, dùng lại chính *evo* mà bạn đã quen cho ATE, và một script nhỏ tính PSNR cùng LPIPS trên tập góc nhìn giữ lại. Tập dữ liệu vào cửa nhanh nhất là TUM RGB-D vì nó nhỏ và mọi hệ đều báo cáo trên đó. Danh sách tên hệ ở đây chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; các hạ tầng bền hơn nhiều là bộ rasterize gốc, *evo*, và *opencv*.

Câu hỏi kiểm tra hiểu. Vì sao lưới đặc trưng chứa được quên thăm khốc trong khi tăng số tham số của một MLP đơn thì không? Một bài báo cáo PSNR đứng đầu bảng trên Replica và ATE nhỏ hơn mọi hệ neural khác được so: hai kết quả đó nói gì và không nói gì về việc hệ chạy được trên chuỗi EuRoC của bạn? Nếu chỉ được chạy một thí nghiệm để quyết định có theo đuổi hướng này không, bạn chạy thí nghiệm nào, và nó chặn trên cái gì?

Mini project. Xem khối *Mini project SLAM* bên dưới; nó chính là thí nghiệm rẻ nhất trả lời được câu hỏi trung tâm của chương.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bản đồ điểm thưa	Vài chục tới vài trăm nghìn điểm 3D kèm bộ mô tả. Đủ để trả lời “tôi ở đâu”, không trả lời được “chỗ kia có trống không” vì nó chưa bao giờ mô hình hoá khoảng không giữa các điểm.
••• TSDF	Lưới lưu khoảng cách có dấu tới bề mặt gần nhất. Bề mặt là tập mức 0 nên có định nghĩa sạch, và đây là đối thủ thật sự của bản đồ neural chứ không phải bản đồ thưa.
•• iMAP	Hệ SLAM RGB-D dùng một MLP làm toàn bộ bản đồ. Gọn tới mức một phòng chỉ cần vài trăm nghìn tham số, nhưng trọng số dùng chung nên học chỗ mới làm mờ chỗ cũ.
••• Quên thảm hoạ	Học dữ liệu mới làm mất khả năng tái tạo dữ liệu cũ, vì cùng bộ tham số gánh cả hai. Triệu chứng trong bản đồ neural: sang phòng bên rồi quay lại thì phòng cũ mờ đi.
••• Vì sao lưới đặc trưng chưa được quên thảm khốc?	Vì nó thu hẹp giá đỡ của gradient. Một tia chỉ chạm vài ô lưới quanh nó thay vì chạm mọi trọng số, nên cập nhật trở thành cục bộ. Cái giá là bộ nhớ quay lại phụ thuộc thể tích cảnh.
•• Ba cách cấp phát thưa	Co-SLAM ghép mã hoá toạ độ với bảng băm; ESLAM dùng ba mặt phẳng đặc trưng nên bộ nhớ theo bình phương chứ không theo lũy thừa ba; Point-SLAM neo đặc trưng lên chính điểm đã quan sát.
••• BARF	Tối ưu tư thế cùng lúc với trường bức xạ. Phát hiện chính: mã hoá vị trí tần số cao làm mặt phẳng mất mát lờ mờ, nên phải bật tần số từ thô tới tinh.
••• Vì sao tần số cao phá gradient theo tư thế?	Vì đạo hàm của thành phần tần số k có biên độ tỉ lệ với k . Các gọn tần số cao sinh ra vô số cực tiểu địa phương gần nhau, và hạ gradient rơi vào cái hố gần nhất.
•• GO-SLAM	Hệ đầu tiên trong dòng NeRF có đóng vòng và tối ưu toàn cục thật sự. Nó làm được vì bản đồ neo vào khung hình chốt, nên sửa tư thế thì kéo bản đồ theo.
• vMAP	Mỗi vật thể một mạng nhỏ riêng, hợp lại lúc dựng ảnh. Vật quan sát được một phần vẫn ra hình dạng kín, vì mỗi mạng chỉ phải học một vật.
••• Vì sao 3DGS nhanh hơn NeRF về bản chất?	NeRF phải đi tìm vật chất bằng cách lấy hàng chục mẫu dọc mỗi tia. 3DGS đã lưu vật chất thành nguyên thể tường minh nên chỉ chiếu, sắp theo độ sâu rồi trộn. Chi phí mỗi điểm ảnh đổi từ số mẫu sang số nguyên thể phủ lên nó.
••• Ba hệ quả của việc chuyển sang nguyên thể tường minh	Cập nhật bản đồ thành sửa chữa cục bộ; bản đồ dịch chuyển được sau khi đóng vòng; đạo hàm theo tư thế có dạng giải tích.
••• Ghép chặt và ghép lỏng	Ghép chặt lấy tư thế từ chính bộ dựng ảnh, nên lỗi bản đồ chảy vào quỹ đạo. Ghép lỏng để tư thế cho đường cổ điển và coi bản đồ là sản phẩm; nó không thể làm quỹ đạo xấu đi.
•• Photo-SLAM	Giữ xương sống đặc trưng và hiệu chỉnh chùm tia cho tư thế, gắn thêm bản đồ Gaussian chỉ để dựng ảnh. Đây là mẫu ghép lỏng điển hình và là cách vào rẻ nhất cho một hệ đang chạy sản phẩm.
••• PSNR cao nhưng ATE tệ — giải thích thế nào?	Hàm mất mát chỉ ràng buộc ảnh dựng, không ràng buộc hình học, và màu phụ thuộc góc nhìn cho mô hình chỗ để “vẽ” thay vì “dựng”. Hai đại lượng đó tách rời được hoàn toàn.
••• PSNR đo trên khung hình đầu vào	Dấu hiệu cảnh báo quan trọng nhất khi đọc bảng: con số đó đo độ khớp chứ không đo khả năng tổng quát sang góc nhìn mới. Phải tìm xem bài có tập góc nhìn giữ lại không.
••• ATE có căn chỉnh tỉ lệ	Với hệ đơn ảnh, ATE thường tính sau khi căn chỉnh cả tỉ lệ. Phép căn chỉnh đó hấp thụ sai lệch tỉ lệ, tức là giấu đúng chế độ hỏng nguy hiểm nhất, nên không so trực tiếp được với hệ stereo-inertial.
•• Thí nghiệm oracle tư thế	Cấp tư thế chân trị và chỉ để hệ dựng bản đồ, để lấy trần chất lượng bản đồ khi khâu tư thế hoàn hảo. Trần đó không đủ thì mọi công sức về sau bị chặn trên.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Ngân sách của một bản đồ Gaussian	Gần sáu chục số thực cho mỗi hạt, nhân bốn khi đang tối ưu vì còn gradient và hai ô Adam. Lưu lượng mỗi giây tỉ lệ với số hạt nhân số vòng tối ưu mỗi khung nhân số khung mỗi giây — nên bằng thông cần trước dung lượng, và thừa số duy nhất bạn điều khiển được là số hạt.
●● Vì sao TensorRT không cứu được?	Vì cái chạy trên Orin không phải suy luận mà là vòng lặp tối ưu trên chính tham số bản đồ. Lượng tử hoá phá gradient, đồ thị đổi liên tục, và nút thắt nằm ở băng thông chứ không ở phép nhân.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
gaussian-splatting	Bộ rasterize Gaussian khác biệt được — hạ tầng mà toàn bộ dòng SLAM Gaussian trong chương dựng lên trên đó	Hạ tầng; bền hơn tên từng hệ SLAM
Photo-SLAM [134]	Mẫu ghép lồng: tư thế do đặc trưng và hiệu chỉnh chùm tia cổ điển, bản đồ Gaussian chỉ tiêu thụ tư thế	Kiến trúc đáng bất chước nhất cho hệ đang chạy sản phẩm
GO-SLAM [127]	Neo bản đồ neural vào khung hình chốt để biến dạng được sau khi đóng vòng	Bằng chứng cho tiêu chí “sửa được sau khi đóng vòng”
NeRF-SLAM [129]	Dùng bất định của độ sâu làm trọng số cho hàm mất mát dựng bản đồ	Tư duy ma trận thông tin mang sang phía bản đồ
Công cụ số hoá công trình và quét nội thất	Dựng lại chất lượng cao <i>ngoại tuyến</i> từ video quay tay — đúng chỗ dòng này thắng rõ	Được cho là đã dùng trong sản phẩm thương mại; tôi không kiểm chứng chi tiết

MINI PROJECT SLAM · Đặt một hệ bản đồ neural lên đúng hai trục đánh giá · 1–2 ngày

Mục tiêu	trả lời bằng số cho câu hỏi trung tâm của chương: một hệ SLAM Gaussian đứng ở góc phần tư nào của Hình 6.6 khi đo bằng <i>giao thức của bạn</i> , chứ không phải giao thức của bài báo.
Dữ liệu	một chuỗi TUM RGB-D để dễ hệ chạy được, cộng đúng một chuỗi EuRoC khó mà bạn đã có sẵn nhiều từ dự án chương 24 — ví dụ V2_03. Baseline là hệ ORB-SLAM3 của chính bạn ở cấu hình mặc định đã tinh chỉnh.
Các bước	cài một hệ ghép chặt và chạy nó ba lần trên mỗi chuỗi; ghi ATE bằng <i>evo</i> với <i>cùng một</i> phép căn chỉnh cho cả hai hệ, và ghi rõ có căn chỉnh tỉ lệ hay không; ghi PSNR trên các khung hình giữ lại, không phải trên khung hình đầu vào; chạy thêm chế độ oracle với tư thế chân trị để lấy trần chất lượng bản đồ; đo đỉnh bộ nhớ GPU và thời gian mỗi khung hình, trên máy để bàn trước, rồi trên Orin nếu nó chạy nổi.
Xong khi	bạn vẽ được một điểm cho mỗi hệ trên mặt phẳng (chất lượng ảnh, ATE) kèm thanh sai số ba lần chạy, và phát biểu được một trong hai câu: “hệ neural nằm cùng góc với ORB-SLAM3 theo trục dọc” hoặc “nó tệ hơn X cm, tức là ngoài sàn nhiều đã đo”. Kiểm tra chéo bắt buộc: nếu bỏ căn chỉnh tỉ lệ mà ATE của hệ đơn ảnh nhảy vọt, bạn vừa nhìn thấy chỗ mà bảng công bố đã giấu.
Nối với	E/24 cho sàn nhiều và cho thí nghiệm oracle; F/27 cho khung bốn cổng để kết luận đi tiếp hay dừng; F/34 nếu chuỗi khó của bạn hỏng vì ảnh mờ chứ không vì bản đồ.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bản đồ dày cho bạn ba thứ mà bản đồ thưa không có: biết chỗ nào trống để tránh vật cản, biết hình dạng bề mặt để thao tác hoặc kiểm tra, và dựng được ảnh ở góc nhìn mới. Cả ba đều là câu hỏi về *khoảng không giữa các điểm*, thứ mà bản đồ thưa chưa bao giờ mô hình hoá.
- A2. Không có lý do mặc định nào cả — và đó chính là cổng đầu tiên. Bản đồ neural chỉ hơn TSDF ở ba chỗ: nội suy vùng chưa quan sát, dựng được ảnh có màu phụ thuộc góc nhìn, và dồn tham số theo nội dung. Nếu nhu cầu của bạn không nằm trong ba chỗ đó thì TSDF hoặc lưới chiếm dụng vẫn là câu trả lời đúng.
- A3. Vì hàm mất mát chỉ ràng buộc ảnh dựng ra. Hai cấu hình hình học rất khác nhau có thể sinh ra cùng một bộ ảnh, nhất là khi màu được phép phụ thuộc hướng nhìn. Muốn biết quỹ đạo, phải đo ATE bằng đúngng giao thức căn chỉnh mà bạn dùng cho hệ hiện tại.
- A4. Thay đổi duy nhất là bỏ việc đi dò dọc tia, thay bằng chiếu các nguyên thể tường minh rồi trộn. Nó mở ra đóng vòng vì nguyên thể có vị trí tường minh: gắn chúng vào khung hình chốt thì sửa tư thế sẽ kéo được cả cụm theo, còn với một MLP đơn thì không có phép nào tương đương.
- A5. Bảng thông cần trước dung lượng, và lý do nằm ở công thức chứ không ở một con số: mỗi vòng tối ưu đọc rồi ghi lại *toàn bộ* tham số cùng gradient và hai ô trạng thái Adam, và lưu lượng đó còn phải nhân với số vòng mỗi khung rồi nhân với số khung mỗi giây. Với một bản đồ cỡ căn phòng, riêng việc di chuyển tham số đã ăn phần lớn băng thông của một máy nhúng trước khi tính một phép nhân nào. Muốn biết hệt bao nhiêu lần thì phải đo trên chính thiết bị của mình.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về hai cách ghi lại một căn phòng. Cách thứ nhất là đóng vài chục cái đinh lên tường ở những chỗ dễ nhận ra, rồi ghi lại khoảng cách giữa chúng. Cách thứ hai là thuê một họa sĩ vẽ căn phòng đó thật giống, giống tới mức nhìn bức tranh cứ tưởng đang nhìn phòng thật. Cái đinh giúp bạn biết mình đang đứng ở đâu, chính xác tới từng centimét. Nhưng nhìn bản đồ đinh thì không biết cái ghế có chắn lối đi hay không. Bức tranh thì ngược lại: nhìn rất rõ căn phòng, mà đo tường từ bức tranh lại sai, vì họa sĩ chỉ cần vẽ cho *trông* giống chứ không cần vẽ cho *đo* đúng. Anh ta được phép vẽ một cái bóng thay cho một cái hốc tường, và người xem không phát hiện ra. Toàn bộ chương này là chuyện có nên đổi đinh lấy tranh hay không. Câu trả lời hôm nay là: đừng đổi, hãy giữ đinh để định vị và treo thêm bức tranh nếu bạn thật sự cần nhìn. Vì bức tranh rất đắt — vẽ một căn phòng ngốn gần hết sức của cái máy tính nhỏ gắn trên robot — và vì độ giống của bức tranh không hề chứng minh nó đo đúng.

Chỗ không rút gọn được: chỗ bức tranh *có thể* đo đúng. Khi được vẽ từ rất nhiều góc nhìn phủ kín, họa sĩ hết chỗ để gian lận, và bức tranh dần trở thành phép đo thật. Không có ranh giới gọn nào nói bao nhiêu góc nhìn là đủ, và mọi cách nói đơn giản về ranh giới đó đều bỏ sót một phần ví dụ.

Một câu: *Đinh để biết mình ở đâu, tranh để biết chỗ đó trông thế nào — và tranh đẹp không có nghĩa là tường được đo đúng.*

6.8 Kết nối

C/16/03-nerf-3dgs-va-hop-nhat là tiền đề trực tiếp: cơ chế dựng ảnh thể tích, phép rasterize Gaussian, và lập luận vì sao PSNR không chứng minh hình học đều nằm ở đó, còn chương này chỉ hỏi chuyện dùng chúng làm bản đồ. F/27 là bộ lọc: khung bốn cổng ở đó chính là thứ dẫn tới kết luận “chưa đáng” của mục 6.5, và bạn nên chạy lại khung đó với số liệu của riêng mình. F/31 giải thích các bộ bám vết dày mà NeRF-SLAM và GO-SLAM mượn lại, nên đọc nó trước nếu bạn muốn hiểu vì sao hai hệ đó chính xác hơn phần còn lại của dòng. F/30 là nguồn của mọi tiên nghiệm độ sâu và pháp tuyến mà NICER-SLAM dùng để thay cảm biến, kèm theo vấn đề tỉ lệ đi cùng. F/33 tiếp nhận ý tưởng bản đồ mức đối tượng của vMAP và đẩy nó lên đồ thị cảnh. F/34 nhận lại hai chỗ chương này bỏ ngỏ: ảnh mờ do chuyển động, và việc lấy trọng số phần dư

từ bất định thay vì từ hằng số. Còn E/24 là bộ công cụ để chạy ba thí nghiệm ở mục 6.4 một cách có kỷ luật — đặc biệt là thí nghiệm oracle, thứ trả lời được nhiều nhất với chi phí ít nhất.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao khoản chênh tốc độ giữa NeRF và 3DGS lại kéo theo khả năng sửa bản đồ sau khi đóng vòng, trong khi hai chuyện đó nghe không liên quan gì tới nhau?
2. Một hệ báo PSNR cao nhất bảng và ATE tốt hơn mọi hệ neural khác. Vì sao điều đó vẫn chưa nói gì về việc nó có thay được hệ của bạn không, và bạn tìm dòng nào trong bài để biết?
3. Giả sử bạn buộc phải dùng bản đồ neural cho tránh vật cản. Đại lượng nào của TSDF hoặc lưới chiếm dụng bị mất, và bạn dựng lại nó bằng cách nào?
4. Bài học từ thô đến tinh của BARF tương đương với kỹ thuật cổ điển nào, và điều gì xảy ra nếu bạn thêm một số hạng quang trắc vào hệ mình mà quên nó?
5. Vì sao lượng tử hoá và TensorRT gần như vô dụng ở chương này, trong khi chúng là công cụ chính ở mọi chương khác của Phần F?
6. Một hệ ghép chặt bị lệch tư thế nhẹ ở khung hình thứ 200. Chuỗi sự kiện nào dẫn tới hỏng, và vì sao không có chỉ báo nào bật lên như ở hệ cổ điển?
7. Trong bốn bài học thiết kế ở cuối chương, bài nào áp được vào hệ của bạn ngay tuần này mà không cần thêm một dòng học sâu nào?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., “Image Matching Across Wide Baselines: From Paper to Practice,” *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., “RoMa: Robust Dense Feature Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., “Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., “HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, “Closed-form solution of absolute orientation using unit quaternions,” *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., “Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, “Rethinking Visual Geo-localization for Large-Scale Applications,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, “MixVPR: Feature Mixing for Visual Place Recognition,” in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., “EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, “Optimal Transport Aggregation for Visual Place Recognition,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., “AnyLoc: Towards Universal Visual Place Recognition,” *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Ngữ nghĩa, vật thể động và đồ thị cảnh (Semantics, Dynamics and Scene Graphs)

PHỤ THUỘC C/14-bai-toan-cv-loi (detection và segmentation là gì — chương này chỉ nói về việc *dùng* chúng); F/27-dat-hoc-sau-vao-dau (khung bốn cổng); F/30-do-sau-dong-quang-va-mo-hinh-nen (dòng quang và độ sâu, hai nguồn bằng chứng hình học cho “đang động”); B/08-dich-chuyen-phan-phoi (mặt nạ hồng đúng lúc bạn cần nó nhất).

TÓM TẮT

SLAM cổ điển giả định thế giới đứng yên. Giả định đó không nằm trong dòng code nào, nhưng mọi hàm giá đều dựa vào nó. Chương này chỉ ra chỗ giả định gây, rồi hỏi ngữ nghĩa mua lại được bao nhiêu và với giá nào. Đọc xong bạn tách được ba việc mà tài liệu hay trộn làm một: vật *có thể* động, vật *đang* động, và vật *đã* động rồi lại đứng yên. Bạn cũng có một bảng xếp hạng định tính về sức nặng của các bộ nhận thức, kèm phép đo mà bạn phải tự chạy trên máy mình, để biết mô hình nào có cửa trong vòng lặp và mô hình nào chỉ dùng được ngoại tuyến. Nếu chỉ nhớ một điều: mặt nạ ngữ nghĩa là một điều chỉnh *trọng số*, không phải một lệnh *xoá* — xoá quá tay làm hỏng điều kiện của bài toán tối ưu, và hỏng điều kiện là kiểu hỏng im lặng, không có cờ ngoại lai nào báo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mặt nạ và phân đoạn thực thể** — mặt nạ là một ảnh nhị phân cùng kích thước ảnh gốc, đánh dấu từng điểm ảnh thuộc hay không thuộc một vật; ở đây nó nói “đừng tin các phép đo rơi vào vùng này”. Phân đoạn thực thể là việc tách *từng* vật riêng lẻ thành một mặt nạ riêng kèm nhãn lớp, khác với phân đoạn ngữ nghĩa vốn gộp hai người đứng cạnh nhau làm một.
- **vật thể động** — trong chương này luôn phải nói rõ nghĩa nào trong ba nghĩa: thuộc lớp *có khả năng* tự di chuyển, *đang* thật sự di chuyển ở khung hình này, hay *đã* đổi chỗ giữa hai lần robot đi qua.
- **điểm bản đồ và phần dư tái chiếu** — điểm bản đồ là một điểm 3D mà hệ thống tin là cố định. Phần dư tái chiếu là khoảng cách pixel giữa chỗ ta chiếu nó xuống ảnh và chỗ ta thật sự quan sát nó. Nếu điểm không hề cố định, phần dư đó *không* còn chỉ nói về sai số tư thế camera.
- **ma trận thông tin** — nghịch đảo của hiệp phương sai, tức là trọng số của một ràng buộc trong bài toán tối ưu. Đặt trọng số bằng không chính là *xoá* ràng buộc; hạ trọng số là giữ nó lại nhưng bớt tin.
- **điều kiện của bài toán** — mức độ “đủ mọi hướng” của tập ràng buộc. Nếu mọi điểm còn lại nằm dồn về một phía ảnh, tư thế camera vẫn giải được nhưng có một hướng gần như không bị ràng buộc. Sai số theo hướng đó lớn mà *không ai báo*.
- **tập nhãn đóng và mở** — tập đóng nghĩa là mô hình chỉ biết đúng những lớp có trong tập huấn luyện, thường là 80 lớp của COCO. Tập mở nghĩa là mô hình nhận một chuỗi chữ bất kỳ làm tên lớp lúc chạy, qua một *prompt* — một gợi ý dạng điểm bấm, khung, hoặc câu chữ.
- **bản đồ đặc** — cách biểu diễn bề mặt bằng lưới ô lập phương hoặc lưới tam giác, thay vì bằng một đám mây điểm rời rạc. Nó tốn bộ nhớ hơn *nhiều bậc* so với bản đồ điểm thưa mà ORB-SLAM3 dựng.
- **độ trễ đuôi** — phân vị cao của phân bố độ trễ, ví dụ p99. Với vòng lặp thời gian thực, chính *đuôi* quyết định có rớt khung hay không, chứ không phải trung bình.

CÂU HỎI MỞ ĐẦU

- Q1. Một người ngồi yên trên ghế chiếm một mảng lớn liên khối của khung hình. Mặt nạ “person” của bạn xoá đúng vùng đó, và quỹ đạo tệ đi. Vì sao xoá đúng vật lại làm hỏng kết quả?
- Q2. RANSAC đã là một bộ loại ngoại lai rất tốt và bạn đã dùng nó nhiều năm. Vậy chính xác thì mặt nạ ngữ nghĩa bắt được loại ngoại lai nào mà RANSAC không bắt được?
- Q3. Vì sao mạng phân đoạn của bạn hỏng nặng nhất đúng ở những khung hình mà bạn cần nó nhất, và điều đó có phải trùng hợp không?
- Q4. Ba hệ thống cùng báo “xử lý được cảnh động”: một cái nhìn nhãn lớp, một cái nhìn phần dư hình học, một cái nhìn lịch sử bản đồ. Chúng giải ba bài toán khác nhau ở chỗ nào?
- Q5. Nếu đầu ra của hệ thống bạn chỉ là một quỹ đạo và một con số APE, thì tầng ngôn ngữ mở kiểu “tìm cho tôi cái cốc” giúp được gì — và câu trả lời trung thực nói lên điều gì về việc nên đầu tư vào đâu?

7.1 Đặt vấn đề

ORB-SLAM3 không có khái niệm “vật thể”. Mô hình thế giới của nó là một đám mây điểm 3D, mỗi điểm được giả định nằm cố định trong một hệ quy chiếu cố định [1]. Giả định đó không được viết ra ở đâu cả. Nhưng mọi hàm giá trong hệ đều dựa vào nó.

Hãy nhìn kỹ một phần dư tái chiếu. Nó là hiệu giữa chỗ ta chiếu một điểm bản đồ xuống ảnh và chỗ ta thật sự đo được điểm đó. Ta đọc phần dư ấy như sai số tư thế camera. Cách đọc đó chỉ đúng khi điểm không tự di chuyển. Nếu điểm có di chuyển, phần dư trộn hai ẩn số vào một con số. Bộ tối ưu không biết điều đó, nên nó gỡ rối theo cách duy nhất nó biết: dịch camera. Một vật động vì thế không tạo ra ngoại lai ngẫu nhiên. Nó tạo ra một sai lệch *có hướng*, và sai lệch có hướng là thứ nguy hiểm nhất với một bộ ước lượng bình phương tối thiểu.

A. Vì sao OpenLORIS khó hơn EuRoC

Bạn đã có bằng chứng cho điều này trong chính bảng kết quả của mình. EuRoC quay trong một sảnh máy bay và một phòng thí nghiệm, gần như không có người trong khung hình. OpenLORIS quay ở văn phòng, nhà ở, siêu thị và hành lang, từ độ cao của một robot dịch vụ, lặp lại nhiều lần trong nhiều tuần [142]. Chênh lệch độ khó giữa hai tập không nằm ở chuyển động của camera. Nó nằm ở nội dung cảnh, và nội dung đó phá vỡ ba giả định cùng lúc.

- **Có người đi lại** — giả định thế giới tĩnh gãy trong lúc đang chạy, ở quy mô vài giây.
- **Cảnh đổi giữa các lần đi qua** — ghế bị đẩy, hộp bị chuyển, cửa mở rồi đóng. Bản đồ cũ vẫn còn điểm ở chỗ không còn gì cả.
- **Bề mặt trắng và sàn bóng** — số đặc trưng còn lại vốn đã ít, nên bất kỳ việc xoá bớt nào cũng đắt hơn hẳn so với trên EuRoC.

Điểm thứ ba quan trọng hơn về ngoài của nó, và ta sẽ quay lại. Trên một cảnh giàu văn, xoá một phần đáng kể số điểm gần như không mất gì. Trên một hành lang trắng, đó có thể là toàn bộ khoảng cách giữa bám được và mất bám.

B. Ngữ nghĩa là kênh đưa tri thức “cái gì có thể di chuyển” vào hệ

Cần phát biểu cho hẹp lại, vì cụm “hiểu cảnh” quá rộng để làm việc được. Với một hệ đo quỹ đạo, ngữ nghĩa làm đúng một việc: nó đưa vào hệ một mẫu tri thức tiên nghiệm mà hình học không thể tự suy ra được từ hai khung hình liên tiếp — *loại vật nào trên đời này có khả năng tự di*

chuyển. Hình học chỉ nhìn thấy điểm ảnh và phần dư. Nó không biết một mảng điểm ảnh là cái ghế hay là bức tường. Ngữ nghĩa biết, vì nó đã xem hàng triệu ảnh có nhãn.

Đó là tất cả những gì tầng ngữ nghĩa cho không. Mọi thứ khác — vật này có đang động không, nó động lúc nào, nó sẽ còn ở đó lần sau không — vẫn phải do hình học hoặc do bộ nhớ của bản đồ trả lời.

Còn một việc thứ hai, lớn hơn nhiều nhưng phục vụ người khác. Ngữ nghĩa cho bản đồ một *từ vựng*. Một bản đồ có từ vựng dùng được bởi một bộ lập kế hoạch hoặc một con người, chứ không chỉ bởi một script tính APE. Hai việc này thường bị viết chung trong cùng một paper, và trộn chúng là nguồn nhầm lẫn lớn nhất khi đọc mảng này. Nửa sau của chương tách chúng ra.

Học chương này thì làm được gì mà trước đó không làm được? Bạn đọc một paper “semantic dynamic SLAM” và nói ngay được nó đang giải bài nào trong ba bài, nó nặng cỡ nào so với ngân sách bạn còn trống, và bảng kết quả của nó có chuyển sang robot của bạn được không. Trong phần lớn trường hợp câu trả lời là không, và biết lý do sẽ tiết kiệm cho bạn vài tháng.

TRỰC GIÁC

Một mặt nạ ngữ nghĩa là danh sách nhãn chứng mà bạn quyết định không tin, dựa trên nghề nghiệp của họ chứ không dựa trên lời khai. Loại nhãn chứng theo nghề rất rẻ và thường đúng, vì có những nghề hay nói dối thật. Nhưng giả sử tất cả người làm nghề đó tình cờ đứng ở nửa trái căn phòng. Sau khi loại họ, mọi lời khai còn lại đều đáng tin tuyệt đối — và tất cả đều nói về nửa phải. Bạn không còn cách nào dựng lại chuyện xảy ra ở nửa trái. Đó chính xác là chuyện xảy ra với điều kiện của bài toán tối ưu khi mặt nạ xoá một mảng lớn liên khối của ảnh: phần dư còn lại nhỏ và đẹp, còn tư thế thì có một hướng không ai ràng buộc.

C. Chương này chạm nỗi đau nào

Trung thực về phạm vi trước khi đi tiếp. Trong bốn nỗi đau của Phần F, chương này chạm thẳng vào **cảnh động**, một nỗi đau mà EuRoC không hề đo và vì thế dễ bị bỏ quên. Nó chạm *gián tiếp* vào **đóng vòng**, theo hai đường: một vòng sai có thể sinh ra vì hai khung hình giống nhau ở chỗ có cùng một chiếc xe đẩy chứ không phải cùng một hành lang; và đồ thị cảnh phân tầng đề xuất một cách tìm ứng viên vòng khác hẳn cách dùng bộ mô tả ảnh toàn cục.

Nó gần như không chạm vào **chuyển động nhanh**, **ảnh mờ** và **trọng số IMU**. Tệ hơn thế: chuyển động nhanh và ảnh mờ làm cho chính tầng ngữ nghĩa hỏng, nên chương này *thừa hưởng* hai nỗi đau đó chứ không chữa chúng. Đó là một trong những kết luận đáng nhớ nhất ở đây, và mục cuối sẽ nói kỹ.

7.2 Công cụ nhận thức chạy được trong vòng lặp (Perception Front-Ends in the Loop)

Chương 14 đã dạy các mô hình này là gì: một giai đoạn khác hai giai đoạn ở đâu, vì sao **triệt phi cực đại** (*non-maximum suppression*) là heuristic cuối cùng, DETR bỏ nó bằng **ghép cặp Hungary** (*Hungarian matching*) thế nào, và phân đoạn hợp nhất được ba bài toán ra sao. Mục này không nhắc lại cơ chế. Nó hỏi một câu khác hẳn, câu mà một kỹ sư hệ thống thời gian thực phải hỏi trước tiên: *cái nào bỏ vừa vào ngân sách của tôi, và bỏ vào chỗ nào.*

A. Ngân sách thật trên Orin, trước khi bàn tới mô hình nào

Bắt đầu từ ràng buộc chứ đừng bắt đầu từ bảng mAP. Ngân sách của một vòng theo vết là chu kỳ khung hình, không hơn. Trong ngân sách đó, một hệ stereo-inertial đã tiêu phần lớn cho trích xuất đặc trưng, ghép cặp và tối ưu tư thế. Phần còn trống cho một mạng nhận thức chỉ là phần thừa ra sau ba việc ấy, và nó hẹp hơn nhiều so với trực giác. Bao nhiêu thì *chỉ máy của bạn trả lời được*, và phép đo thì rẻ: chạy hệ SLAM một mình trên phần cứng thật, ghi phân vị cao của chu kỳ luồng theo vết, rồi lấy chu kỳ khung trừ đi con số đó. Phần dư ra mới là ngân sách thật, và nó thường nhỏ tới mức bất ngờ.

Ba sự thật về phần cứng quyết định thiết kế, và cả ba đều hay bị bỏ qua khi đọc bảng FPS trong paper.

- **GPU là tài nguyên dùng chung.** Trên Jetson Orin không có GPU rời thứ hai. Nếu bạn đã chuyển trích xuất ORB lên GPU, mạng nhận thức và bộ trích xuất tranh nhau cùng các nhân tính toán và cùng băng thông bộ nhớ. “Chạy song song” trên giấy trở thành chạy xen kẽ trên máy, và độ trễ của cả hai đều tăng.
- **Bộ tăng tốc học sâu chuyên dụng là đòn bẩy song song thật.** Dòng AGX Orin có hai khối tăng tốc học sâu tách rời khỏi GPU. Đưa mạng nhận thức xuống đó là cách duy nhất để nó thật sự chạy song song với luồng theo vết. Cái giá là tập phép toán được hỗ trợ hẹp hơn nhiều, nên phần lớn mạng phải sửa mới chạy nổi, và một số tầng vẫn rơi ngược về GPU.
- **Đuôi mới giết bạn, không phải trung bình.** Một mạng có trung bình nằm gọn trong ngân sách nhưng có phân vị 99 vượt hẳn ngân sách sẽ làm rớt khung đúng lúc cảnh phức tạp. Với các detector còn dùng triết phi cực đại, thời gian hậu xử lý tỉ lệ với số khung sống sót, nên đuôi độ trễ dày lên đúng ở cảnh đông người — tức là đúng lúc bạn cần mặt nạ.

Từ ba điều đó suy ra một kết luận kiến trúc, và nó quan trọng hơn việc chọn mô hình nào. **Tầng nhận thức không nên nằm trong đường tới hạn của luồng theo vết.** Cấu hình thực dụng là chạy mạng trên một luồng riêng, ở tần số thấp hơn, chỉ trên khung hình chính, rồi truyền mặt nạ sang các khung xen giữa bằng một cơ chế rẻ. Hình 7.1 vẽ cả hai cách nổi, và cho thấy vì sao cách đồng bộ hỏng.



Hình 7.1. Hai cách nối tầng nhận thức vào vòng lặp. Cách đồng bộ cộng thẳng độ trễ mạng vào ngân sách mỗi khung, và đuôi độ trễ dày lên đúng ở cảnh đông người. Cách bất đồng bộ giữ nhịp cho luồng theo vết nhưng đổi lấy một mặt nạ luôn lạc hậu và khung — sai lệch đó tỉ lệ với tốc độ của vật, nên nó lớn nhất đúng ở trường hợp mặt nạ đáng giá nhất.

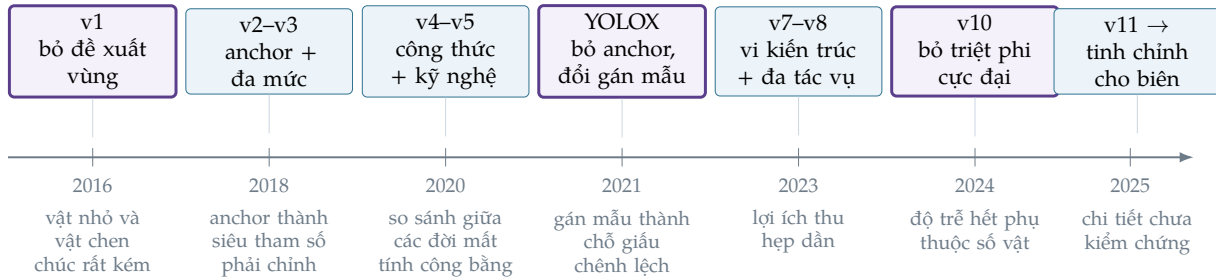
B. Dòng YOLO: cái gì thật sự đổi qua các đời

Mười một phiên bản trong tám năm tạo ấn tượng về một chuỗi đột phá liên tiếp. Đọc kỹ thì không phải vậy. Phần lớn chênh lệch giữa các đời đến từ ba nguồn ít được viết trong tiêu đề: quy tắc gán mẫu, công thức huấn luyện, và kỹ nghệ triển khai. Chỉ có vài bước thật sự đổi cấu trúc bài toán. Với một kỹ sư SLAM, chỉ hai bước trong số đó đáng quan tâm.

- **v1 (2016)** — bỏ hẳn giai đoạn đề xuất vùng, hồi quy thẳng trên một lưới [143]. Đây là lần đổi phát biểu thật sự đầu tiên, và nó mua lấy tốc độ bằng cách hy sinh vật nhỏ và vật chen chúc.
- **v2, v3 (2017–2018)** — *anchor* lấy từ phân cụm kích thước hộp, rồi dự đoán trên ba mức độ phân giải. Bước thứ hai chứa đúng điểm yếu vật nhỏ của v1.
- **v4 (2020)** — một tập hợp thủ thuật huấn luyện và một cái *neck* tốt hơn, gói lại thành “bag of freebies” [144]. Đóng góp thật nằm ở khảo sát có hệ thống xem thủ thuật nào cộng dồn được, chứ không ở một ý tưởng mới.
- **v5 (2020)** — không có bài báo. Cái đổi là kỹ nghệ: đóng gói, xuất mô hình, huấn luyện lại dễ. Với người triển khai, đây là phiên bản đổi nhiều nhất trong đời thực, và đó là một nhận xét về ngành chứ không phải về thuật toán.
- **YOLOX (2021) và v8 (2023)** — bỏ *anchor*, tách đầu phân loại khỏi đầu hồi quy, và đổi quy tắc gán mẫu [145]. Đây là bước đổi cấu trúc thứ hai, và Chương 14 đã cho thấy vì sao gán mẫu mới là chỗ giấu phần lớn chênh lệch.
- **v6, v7, v9, v11 (2022–2024)** — vi kiến trúc *backbone* và *neck*: khôi tái tham số hoá cho thân thiện với lượng tử hoá, luồng thông tin phụ lúc huấn luyện, khối chú ý nhẹ [146]. Có lợi, nhưng lợi ích thu hẹp dần qua từng đời và không đổi cách bạn nối nó vào hệ.
- **v10 (2024)** — huấn luyện với hai phép gán nhất quán để bỏ hẳn triết phi cực đại lúc suy luận [147]. Đây là bước đáng quan tâm nhất cho hệ thời gian thực, vì nó cắt đúng cái phần hậu xử lý có thời gian chạy phụ thuộc nội dung cảnh.

Dòng này vẫn tiếp tục sau v11; tới 2025–2026 đã có các bản đi tiếp theo hướng bỏ triệt phi cực đại và tối ưu cho biên. Tôi không kiểm chứng được chi tiết của chúng nên không mô tả số liệu ở đây. Hình 7.2 xếp cả dòng trên một trục và ghi rõ mỗi bước gỡ nút thắt nào.

Khung viền tím: ba bước thật sự đổi cấu trúc bài toán. Các bước còn lại đổi công thức, vi kiến trúc hoặc kỹ nghệ.



Hình 7.2. Dòng YOLO đọc theo nút thắt chứ không theo số phiên bản. Dòng dưới mỗi mốc ghi vấn đề còn lại sau bước đó. Chỉ ba mốc đổi cấu trúc: bỏ đề xuất vùng, bỏ anchor kèm đổi gán mẫu, và bỏ triệt phi cực đại. Với hệ thời gian thực, mốc cuối đáng giá nhất vì nó cắt phần hậu xử lý có thời gian chạy phụ thuộc nội dung cảnh.

C. Họ DETR: vì sao “không triệt phi cực đại” là một tính chất hệ thống

DETR đổi phát biểu thành dự đoán một tập hợp, ghép với chân trị bằng ghép cặp Hungary một–một, nên mỗi vật chỉ có đúng một dự đoán được thưởng và mô hình tự học cách không trùng lặp [148]. Chương 14 đã giải thích cơ chế đó. Điều Chương 14 chưa nói là vì sao một kỹ sư thời gian thực nên quan tâm.

Lý do không phải độ chính xác. Lý do là *hình dạng phân bố độ trễ*. Một detector có triệt phi cực đại tiêu thời gian tỉ lệ với số khung sống sót sau ngưỡng điểm. Số đó nhảy vọt ở cảnh đông người. Nói cách khác, đuôi độ trễ của bạn dày lên đúng ở cảnh mà bạn bật tầng ngữ nghĩa lên để xử lý. Một mô hình không triệt phi cực đại có thời gian chạy gần như hằng số, và với một bộ lập lịch thời gian thực thì hằng số quý hơn nhanh.

↪ **tiền đề** C/14/02 – cơ chế ghép cặp một–một nằm ở Chương 14; ở đây ta chỉ dùng hệ quả của nó về phân bố độ trễ

RT-DETR đẩy ý tưởng đó vào vùng thời gian thực [149]. Cơ chế là một bộ mã hoá lai: nó tách việc trộn thông tin trong cùng một mức độ phân giải khỏi việc trộn giữa các mức. Cách chọn truy vấn khởi đầu cũng tốt hơn bản gốc. Nó còn một tính chất hiếm và rất hợp với robot: có thể đổi tốc độ bằng cách bớt tầng giải mã lúc chạy, *không cần huấn luyện lại*. Với một hệ phải hạ chất lượng khi nóng máy hoặc khi tải cao, đó là một nút vặn có thật. Dòng RF-DETR năm 2025 đi tiếp bằng cách gắn một *backbone* tự giám sát mạnh vào khung DETR; các con số họ công bố tôi chưa kiểm chứng độc lập nên không dẫn lại ở đây.

D. SAM và SAM 2: phân đoạn bất kỳ, và chi phí thật của nó

SAM đổi câu hỏi từ “vật này là lớp gì” sang “vùng nào là một vật”, và trả lời theo prompt: một điểm bấm, một khung, hoặc một mặt nạ thô [150]. SAM 2 mở rộng sang video bằng một bộ nhớ chạy dọc thời gian, nên mặt nạ theo được vật qua các khung thay vì phải phân đoạn lại từ đầu [151].

Cấu trúc chi phí của SAM là bài học thiết kế đáng lấy đi kể cả khi bạn không dùng SAM. Bộ mã hoá ảnh rất nặng và chạy *một lần mỗi ảnh*. Bộ giải mã mặt nạ rất nhẹ và chạy *một lần mỗi prompt*. Vì thế hỏi thêm mười vùng nữa trên cùng một ảnh gần như miễn phí, trong khi chuyển sang ảnh mới thì trả lại toàn bộ giá. Đây đúng là mẫu “đặt phần đắt ở chỗ được khấu hao” mà bạn đã dùng khi tách trích xuất đặc trưng khỏi ghép cặp.

Nhưng có hai giới hạn phải nói thẳng, và cả hai đều chí mạng cho việc lọc vật động.

- **SAM không cho bạn nhãn.** Nó trả về các vùng, không trả về chữ “person”. Muốn biết vùng nào là vật có thể động, bạn vẫn cần một bộ phân loại hoặc một mô hình ngôn ngữ-thị giác chạy thêm. Chi phí thật vì thế là chi phí SAM cộng chi phí gán nhãn.
- **Chi phí bộ mã hoá không vừa ngân sách khung hình.** Bản gốc dùng một *transformer* thị giác cỡ lớn chạy toàn độ phân giải, tức dạng phép tính nặng nhất trong tất cả các công cụ mục này bàn tới. Các biến thể nhẹ của SAM 2 nhẹ hơn hẳn, nhưng vẫn không cùng hạng với một detector nhỏ. Tôi không nêu con số vì tôi không tự đo được nó trên phần cứng của bạn; phép đo đúng nằm ở dự án thứ hai cuối chương. Kết luận thực dụng thì không phụ thuộc con số: đây là công cụ ngoại tuyến, hoặc công cụ chạy *thừa* trên khung hình chính.

Tới 2026 dòng này đã có bản nhận prompt ở mức khái niệm và chạy trên video; tôi không kiểm chứng được chi tiết nên chỉ ghi nhận sự tồn tại của nó.

E. Grounding DINO và Open-YOLO 3D: khi tập nhãn phải mở

Grounding DINO nhận một câu chữ làm điều kiện và phát hiện vật theo mô tả, nhờ trộn đặc trưng ảnh với đặc trưng chữ ở nhiều tầng [152]. Nó gỡ đúng nút thắt tập nhãn đóng: nhà kho của bạn có xe đẩy hàng, pallet và tấm chắn, và COCO không có lớp nào cho chúng.

Cái giá là kích thước. Một bộ mã hoá chữ cộng một *backbone* nặng đặt nó vào cùng hạng nặng với SAM. Vai trò đúng của nó vì thế không phải trong vòng lặp. Nó là **bộ gán nhãn ngoại tuyến**: cho nó chạy chậm trên dữ liệu thu được, lấy nhãn ra, rồi chùng cất xuống một detector nhỏ chạy được thời gian thực. Mẫu “mô hình lớn dạy mô hình nhỏ” này là cách duy nhất tôi biết để có tập nhãn mở mà vẫn giữ được ngân sách.

Open-YOLO 3D là một ví dụ tốt của cùng tinh thần tiết kiệm, ở một tầng khác [153]. Bài toán là phân đoạn thực thể 3D với tập nhãn mở. Cách làm thông thường là chạy một mô hình thị giác-ngôn ngữ nặng trên từng khung rồi nâng kết quả lên 3D, và nó rất chậm. Open-YOLO 3D thay mặt nạ 2D bằng *hộp* 2D từ một detector mở nhẹ hơn, rồi dùng các hộp đó để bỏ phiếu nhãn cho các đề xuất 3D. Bài học thiết kế nằm ở chỗ này, và nó áp được sang chỗ khác: hỏi xem bạn thật sự cần độ chi tiết nào, rồi mua đúng độ chi tiết đó. Nếu việc chỉ cần biết “đám điểm này thuộc vật gì”, một cái hộp là đủ và mặt nạ từng điểm ảnh là lãng phí.

Bảng 7.1. Sáu công cụ nhận thức, đọc theo con mắt ngân sách. Cột sức nặng là một *thứ hạng định tính*, xếp theo hai thứ nhìn thấy được mà không cần chạy: kích thước mô hình, và dạng phép tính (một lượt trên ảnh nhỏ là nhẹ; một *transformer* thị giác chạy toàn độ phân giải là nặng). Thứ hạng này *không thay được phép đo*: sức nặng thật chỉ hiện ra khi bạn chạy mô hình cùng lúc với hệ SLAM trên chính máy của mình, theo quy trình ở dự án thứ hai cuối chương. Cột cuối là cột quyết định.

Công cụ	Cho bạn cái gì	Sức nặng (thứ hạng, không phải số đo)	Song song với theo vết?	Hóng khi nào
YOLO nhỏ (n/s, 640)	Hộp + nhãn, tập đóng	Nhẹ — mạng nhỏ, một lượt, ảnh nhỏ	Được, kể cả trên bộ tăng tốc chuyên dụng	Vật nhỏ, vật bị che, lớp ngoài COCO; đuôi độ trễ nếu còn triết phi cực đại
YOLO trung bình, RT-DETR nhẹ	Hộp + nhãn, độ trễ ổn định hơn	Trung bình — cùng dạng phép tính, mô hình lớn hơn	Chỉ ở tần số thấp hơn nhịp khung	Tranh chấp GPU với trích xuất đặc trưng; vẫn tập đóng
Phân đoạn thực thể nhẹ	Mặt nạ + nhãn	Trung bình — thêm một đầu ra dày đặc theo từng điểm ảnh	Khung hình chính thời	Biên mặt nạ lệch vài điểm ảnh — đúng chỗ góc ORB hay nằm
SAM	Vùng, <i>không</i> nhãn	Nặng — <i>transformer</i> thị giác cỡ lớn cho mỗi ảnh	Không	Không nói được vùng nào là vật động; phải cộng thêm bộ gán nhãn
SAM 2 (bản nhẹ)	Vùng + theo vết qua khung	Trung bình tới nặng, tùy biến thể	Chỉ trên khung hình chính, chạy thưa	Trôi mặt nạ khi vật bị che lâu; vẫn không có nhãn
Grounding DINO	Hộp theo mô tả chữ, tập mở	Nặng — cộng thêm cả một bộ mã hoá chữ	Không	Quá chậm cho vòng lặp; dùng làm bộ gán nhãn ngoại tuyến

CẠM BẤY

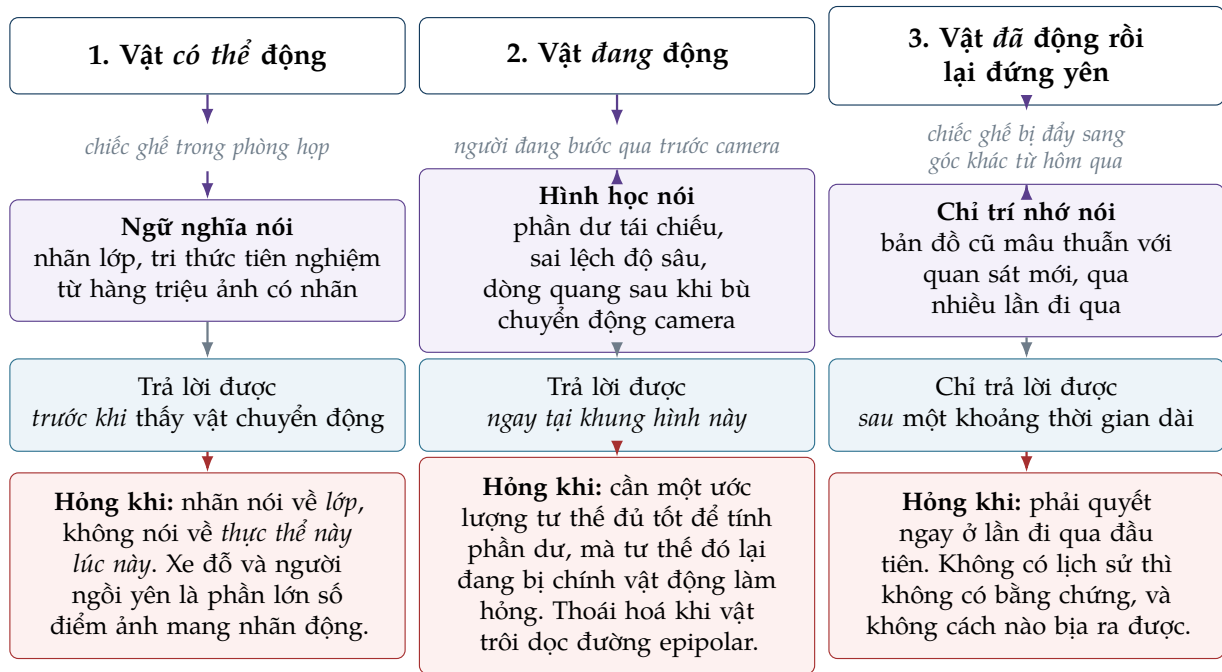
Bảng FPS trong paper gần như luôn đo trên máy để bàn có GPU rời, ảnh đơn lẻ, kích thước *batch* lớn, và *không* có tiến trình nào khác chạy cùng. Ba điều kiện đó đều sai với bạn. Chuyển sang Orin, *batch* bằng một, và có một hệ SLAM đang chiếm GPU, con số tụt đi rất xa — và không bảng nào cho bạn biết trước tụt bao nhiêu. Cách kiểm nhanh và trung thực: chạy mạng *cùng lúc* với hệ SLAM thật, đo p99 của cả hai, rồi mới nói tới trung bình.

7.3 Ba việc khác nhau khi loại vật thể động (Three Different Jobs)

Đây là mục cốt lõi của chương. Tài liệu về mảng này gộp ba việc rất khác nhau vào cùng một cụm “xử lý cảnh động”, và chính việc gộp đó khiến người đọc không thấy được vì sao một phương pháp hoạt động trên tập này mà chết trên tập kia. Tách ra thì mọi thứ sáng ra ngay.

A. Ba câu hỏi, ba nguồn bằng chứng

Ba việc là ba câu hỏi khác nhau về cùng một mảng điểm ảnh, và mỗi câu chỉ có đúng một nguồn bằng chứng trả lời được. Hình 7.3 là bản đồ của cả mục, và cũng là hình đáng nhớ nhất của chương.



Ba cột độc lập: không cột nào suy ra được cột nào. Một hệ chỉ có cột 1 xoá nhầm xe đỗ. Một hệ chỉ có cột 2 không khởi động được khi vật động chiếm phần lớn khung hình. Một hệ chỉ có cột 3 vô dụng ở lần chạy đầu tiên.

Hình 7.3. Ba việc khác nhau bị gộp dưới cùng một cái tên “xử lý cảnh động”. Mỗi cột có một nguồn bằng chứng riêng, một thời điểm mà nó trả lời được, và một kiểu hông riêng. Đây là bản đồ để đọc mọi paper trong mảng này: hãy hỏi paper đó đứng ở cột nào, và nó giả định hai cột kia đã được ai lo.

B. Cột 1 – vật có thể động, và ngữ nghĩa là nguồn duy nhất

Cột này là chỗ ngữ nghĩa thật sự không thể thay thế. Không có phép đo hình học nào trên hai khung hình liên tiếp nói được rằng cái ghế *có khả năng* bị đẩy, còn bức tường thì không. Tri thức đó đến từ nơi khác: từ việc mô hình đã xem hàng triệu ảnh trong đó ghế xuất hiện ở đủ chỗ còn tường thì không.

Giá trị của tri thức tiên nghiệm ấy là nó có *trước* khi có bằng chứng. Ở khung hình đầu tiên của chuỗi, bạn chưa có tư thế nào để tính phần dư, chưa có lịch sử nào để so. Nhưng bạn đã biết ngay rằng nửa dưới khung hình là một người. Điều đó cho phép hạ trọng số ngay lập tức, và giai đoạn khởi tạo là giai đoạn mong manh nhất của cả hệ.

Giới hạn cũng nằm ngay trong định nghĩa. Nhãn là một mệnh đề về *lớp*, không phải về *thực thể này ở khung hình này*. Trong phần lớn cảnh thật, đa số điểm ảnh mang nhãn “có thể động” lại đang đứng yên hoàn toàn: xe đỗ ven đường, người ngồi làm việc, ghế kê quanh bàn. Chúng là cấu trúc tốt, ổn định, giàu văn, và thường nằm ở khoảng cách vừa đẹp cho tam giác hoá. Xoá chúng là xoá dữ liệu tốt.

C. Cột 2 – vật đang động, và cái vòng luẩn quẩn của nó

Cột này thuộc về hình học, và bằng chứng thì có nhiều dạng. Bốn dạng hay dùng nhất, xếp theo độ mạnh của tín hiệu.

1. **Sai lệch độ sâu.** Chiếu một điểm bản đồ vào khung hiện tại, so độ sâu dự đoán với độ sâu đo được từ stereo hoặc RGB-D. Lênh quá ngưỡng nghĩa là vật đã dịch dọc trục nhìn. Đây là kiểm tra mạnh nhất khi có độ sâu tin cậy, và là kiểm tra mà DynaSLAM dùng [10].

2. **Khoảng cách tới đường epipolar.** Với hai khung và một tư thế tương đối, một điểm tĩnh phải nằm trên đường epipolar của nó. Lệch khỏi đường đó là bằng chứng chuyển động. Rẻ, không cần độ sâu, nhưng mù với chuyển động *dọc theo* đường epipolar.
3. **Dòng quang sau khi bù chuyển động camera.** Dự đoán trường dòng quang mà một cảnh tĩnh sẽ tạo ra dưới chuyển động camera đã ước lượng, rồi trừ khỏi dòng quang đo được. Phần dư khác không là chuyển động thật của vật. Đây là tín hiệu dày đặc nhất, và là chỗ [F/30-do-sau-dong-quang-va-mo-hinh-nen](#) nổi vào mục này.
4. **Tính nhất quán của một cụm điểm.** Nếu một nhóm điểm cùng vi phạm ràng buộc theo cùng một hướng, đó là một vật, không phải nhiễu. Đây là tín hiệu phân biệt vật động thật với ngoại lai ngẫu nhiên, và cũng là tín hiệu đắt nhất để tính.

Cả bốn đều mắc chung một vòng luẩn quẩn, và nói thẳng ra nó là điều trung thực nhất có thể nói về cột này. Muốn tính phần dư, bạn cần tư thế camera. Nhưng tư thế camera đang được ước lượng từ chính tập điểm chứa vật động. Khi vật động chiếm phần nhỏ khung hình, RANSAC gỡ được vòng này: nghiệm đồng thuận bám theo nền tĩnh, phần dư của vật động vọt lên, và bạn phát hiện được chúng. Khi vật động chiếm phần lớn khung hình, nghiệm đồng thuận bám theo *vật động*, phần dư của nền tĩnh vọt lên, và bạn phát hiện ngược hoàn toàn.

CẦN NHỚ

Đây là câu trả lời cho câu hỏi “RANSAC đã tốt rồi, mặt nạ thêm được gì”. RANSAC hỏng khi ngoại lai *đồng và nhất quán*, vì khi đó chúng thắng phiếu đồng thuận. Một đám đồng đi cùng chiều, một xe tải chạy song song, một bức tường người trong thang máy — đó chính xác là các trường hợp đó. Ngữ nghĩa hữu ích ở đúng chỗ này, và chỉ ở đây: nó phá thế bế tắc bằng một lá phiếu đến từ ngoài hình học. Ngoài vùng đó, nó chủ yếu chỉ lấy đi dữ liệu tốt. [T3]

Thoái hoá thứ hai đáng nhớ vì nó im lặng: vật di chuyển *dọc theo* tia nhìn hoặc dọc theo đường epipolar không tạo ra vi phạm nào cả. Một người đi thẳng ra xa camera trông y hệt một người đứng yên ở xa hơn một chút. Không kiểm tra hai khung nào bắt được trường hợp này. Chỉ có độ sâu đo trực tiếp, hoặc nhãn ngữ nghĩa, mới nói được.

D. Cột 3 – vật đã động rồi lại đứng yên, và chỉ trí nhớ nói được

Cột này bị bỏ quên nhiều nhất, dù với một robot dịch vụ nó lại là cột tồn kém nhất. Chiếc ghế hôm qua ở giữa phòng, hôm nay ở góc phòng. Ở cả hai lần đi qua, nó đứng yên tuyệt đối. Không phép kiểm tra hình học nào trong một lần chạy phát hiện được gì, vì trong lần chạy đó nó đứng là tĩnh. Nhãn ngữ nghĩa thì báo động ở cả hai lần, kể cả khi ghế không hề bị đụng tới.

Bằng chứng duy nhất là mâu thuẫn giữa bản đồ cũ và quan sát mới. Điểm bản đồ ở vị trí cũ liên tục không được quan sát lại dù nằm trong tầm nhìn và không bị che. Đó là tín hiệu “chỗ này không còn gì”. Nó đòi hỏi ba thứ mà một hệ SLAM một-lần-chạy không có: bản đồ sống qua nhiều phiên, cơ chế đếm số lần quan sát được và số lần đáng lẽ phải quan sát được, và một chính sách xoá điểm dựa trên tỉ số đó.

Đây chính là chỗ khái niệm “bản đồ theo thời gian” bước vào, và mục về đồ thị cảnh sẽ quay lại với Khronos. Điều đáng ghi nhớ ngay bây giờ: đây là bài toán *bảo trì bản đồ*, không phải bài toán nhận thức. Ném một mạng phân đoạn vào nó không giải quyết được gì.

E. DynaSLAM và họ hàng: các thiết kế đã có

Với ba cột trong tay, đọc các hệ thống có sẵn trở nên nhanh. Mạch dưới đây đi theo thứ tự lịch sử, và mỗi bước gỡ đúng một nút thắt của bước trước.

I Mạch — từ mặt nạ tới bản đồ bốn chiều

- 1 Vấn đề** ORB-SLAM2 chạy trên chuỗi có người đi ngang thì quỹ đạo lệch hẳn, dù mọi thứ khác đều đúng. Nguyên nhân là các điểm trên người vẫn được đưa vào tối ưu như điểm tĩnh.

Giải pháp DynaSLAM đặt một mạng phân đoạn thực thể trước bộ theo vết, xoá mọi đặc trưng rơi vào mặt nạ của các lớp có thể động, rồi chạy thêm một kiểm tra hình học đa khung cho phần còn lại [10].

Giả định cột 1 và cột 2 bổ sung cho nhau, và cái nào bắt được thì bắt.

Giá phải trả một lượt mạng nặng cho mỗi khung, và mất toàn bộ đặc trưng trên các vật đứng yên thuộc lớp động.
- 2 Vấn đề** kiểm tra hình học của DynaSLAM cần độ sâu, nên bản đơn ảnh chỉ còn lại mặt nạ.

Giải pháp DS-SLAM ghép một mạng phân đoạn nhẹ với một kiểm tra nhất quán chuyển động dựa trên dòng quang và ràng buộc epipolar, chạy trên hai luồng song song [154].

Hệ quả đây là bản thiết kế đầu tiên coi tầng ngữ nghĩa là một luồng riêng thay vì một bước trong đường tới hạn.

Giá phải trả mặt nạ trễ một khung, và kiểm tra epipolar mù với chuyển động dọc đường epipolar.
- 3 Vấn đề** xoá vật động là vứt bỏ thông tin; vật động cũng là một vật có tư thế đáng ước lượng.

Giải pháp nhóm hợp nhất theo vật — MaskFusion và họ hàng — dựng một mô hình 3D riêng cho từng vật và theo vết tư thế của nó song song với tư thế camera [155]. DynaSLAM II đưa hẳn tư thế vật vào bài toán hiệu chỉnh chùm tia, nên vật động trở thành ràng buộc chứ không còn là nhiễu [156].

Giả định mỗi vật là một khối cứng.

Giá phải trả giả định khối cứng sai với con người, tức là với loại vật động phổ biến nhất; chi phí tính toán tăng theo số vật.
- 4 Vấn đề** mọi thứ ở trên chỉ sống trong một lần chạy, nên cột 3 vẫn bỏ ngỏ.

Giải pháp Khronos phát biểu lại bài toán theo trục thời gian: thay vì phân loại tĩnh hay động, hệ ước lượng *khoảng thời gian tồn tại* của từng vật, nên xử lý được cả chuyển động ngắn hạn lẫn thay đổi dài hạn trong cùng một khung [157].

Hệ quả bản đồ trả lời được câu hỏi “thế giới trông thế nào lúc t ” thay vì chỉ “thế giới trông thế nào”.

Giá phải trả cần một stack bản đồ đặc và cần lưu lịch sử, hai thứ mà một hệ điểm thưa không có.

F. Vì sao mặt nạ ngữ nghĩa một mình là không đủ

Gom lại thành sáu lý do độc lập. Mỗi lý do là một chế độ hỏng riêng, và một hệ thật thường gặp vài lý do cùng lúc.

- **Nhân là mệnh đề về lớp, không phải về thực thể.** Đây là lý do gốc, và năm lý do sau chỉ là hệ quả ở các tầng khác nhau. Xe đỗ, người ngồi, ghế kê sát tường đều mang nhãn động mà không hề động.

- **Vật động ngoài tập nhãn.** Xe đẩy hàng, cánh cửa, rèm cửa, thùng hàng được bê đi, con chó. Tám mươi lớp của COCO không phủ được một nhà kho hay một bệnh viện. Tập nhãn mở giải được vấn đề này về nguyên tắc, nhưng với chi phí đã nêu ở mục trước.
- **Chuyển động gián tiếp.** Một vật tĩnh bị vật động làm đổi vẻ ngoài: bóng đổ chạy trên tường, ánh phản chiếu trên sàn bóng và trên kính, một cái ghế bị người đẩy. Điểm ảnh nằm trên bề mặt tĩnh, nên không mặt nạ nào phủ chúng, nhưng phép đo thì sai.
- **Biên mặt nạ lệch đúng chỗ đau nhất.** Mạng phân đoạn sai vài điểm ảnh ở biên. Mà góc ORB thì thích nằm đúng ở biên, vì đó là chỗ gradient mạnh nhất. Bạn buộc phải chọn: nở mặt nạ ra và xoá thêm cấu trúc tĩnh, hay giữ nguyên và để lọt đúng những điểm tệ nhất — các điểm nằm trên đường viền vật động.
- **Mặt nạ luôn cũ.** Nó được tính trên khung k và đến nơi ở khung $k + 2$ hoặc muộn hơn. Sai lệch vị trí tỉ lệ với tốc độ vật, nên nó lớn nhất đúng ở trường hợp mặt nạ đáng giá nhất. Truyền tiếp bằng dòng quang hoặc theo vết hợp chỉ giảm chứ không xoá được sai lệch này.
- **Mạng hồng đúng lúc bạn cần nó.** Ảnh mờ do chuyển động, thiếu sáng, phơi sáng nhảy — đó vừa là những khung hình mà hình học yếu nhất, vừa là những khung hình nằm ngoài phân phối huấn luyện của mạng. Hai kiểu hồng *tương quan dương* với nhau. Một hệ dự phòng chỉ có giá trị khi nó hồng độc lập với hệ chính, và ở đây thì không.

Lý do cuối đáng được nhấn mạnh vì nó là kết luận thiết kế, không phải một chi tiết kỹ thuật. Bạn không thể coi tầng ngữ nghĩa là lưới an toàn cho các khung hình khó. Nó là một tầng hỗ trợ chỉ hoạt động tốt ở đúng những khung hình mà hình học vốn đã ổn.

↪ cùng nguyên lý với B/08 — đây là dịch chuyển phân phối ở dạng độc nhất: phân phối trượt đi đúng theo hướng làm cả hai thành phần của hệ cùng yếu đi

G. Hai phép tính nhỏ cho thấy vùng an toàn hẹp tới đâu

Trước khi bàn tới cái giá của việc xoá, hãy làm hai phép tính bằng tay. Cả hai chỉ dùng số liệu bạn đã có, và cả hai đều cho ra kết quả hẹp hơn trực giác.

Phép tính thứ nhất: một người đứng gần chiếm bao nhiêu phần khung hình. Đây là một phép tính *giả định*, làm để thấy hình dạng của quan hệ chứ không phải để lấy con số. Giả sử một camera kiểu EuRoC, tiêu cự khoảng 460 điểm ảnh, ảnh 752×480 , tức 360 960 điểm ảnh. Một người rộng chừng 0,5 m và cao 1,7 m. Ở khoảng cách 3 m, bề rộng chiếu xuống ảnh là $460 \times 0,5/3 \approx 77$ điểm ảnh và bề cao là $460 \times 1,7/3 \approx 261$. Diện tích chừng 20 000 điểm ảnh, tức *khoảng 6%* khung hình. Vô hại.

Bây giờ cũng người đó ở khoảng cách 1 m. Bề rộng thành $460 \times 0,5 \approx 230$ điểm ảnh, bề cao thành 782 nên bị cắt còn 480. Diện tích là $230 \times 480 = 110\,400$, tức *khoảng 31%* khung hình. Cùng một người, chỉ đổi khoảng cách ba lần, mà tỉ lệ nhảy năm lần. Điều đáng mang đi không phải hai con số mà là quan hệ: diện tích đi theo nghịch đảo bình phương khoảng cách, nên phân bố diện tích vật động có đuôi rất dày. Hệ quả thực dụng là bạn phải đo ở một phân vị cao chứ không ở trung bình — trung bình của cả chuỗi có thể rất nhỏ trong khi đúng vài chục khung hình quyết định lại rất lớn, và chính vài chục khung đó làm hồng quỹ đạo.

Phép tính thứ hai: RANSAC chịu được tới đâu. Số vòng lặp cần để lấy được một mẫu sạch với xác suất p là

$$N = \frac{\log(1 - p)}{\log(1 - (1 - \varepsilon)^s)},$$

với ε là tỉ lệ ngoại lai và s là số điểm tối thiểu mỗi mẫu. Lấy $p = 0,99$ và $s = 4$, rồi thay vài giá trị:

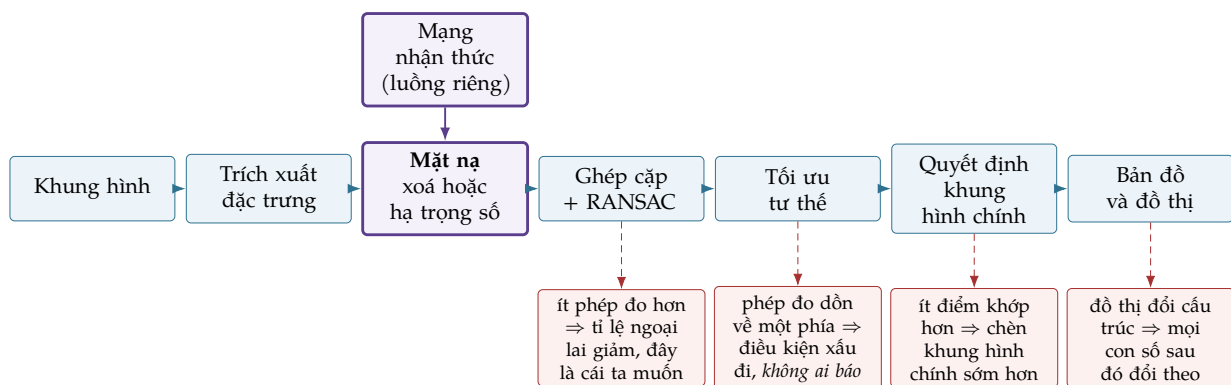
- $\varepsilon = 0,20 \rightarrow (0,8)^4 = 0,41 \rightarrow N \approx 9$ vòng. Rẻ như không.
- $\varepsilon = 0,50 \rightarrow (0,5)^4 = 0,0625 \rightarrow N \approx 72$ vòng. Vẫn chạy được.

- $\varepsilon = 0,70 \rightarrow (0,3)^4 = 0,0081 \rightarrow N \approx 566$ vòng. Bắt đầu đau.
- $\varepsilon = 0,85 \rightarrow (0,15)^4 = 5,1 \times 10^{-4} \rightarrow N \approx 9\,000$ vòng. Hết cửa.

Chi phí bùng nổ theo hàm mũ của s , nhưng đó chưa phải phần đáng sợ nhất. Công thức trên giả định ngoại lai là *ngẫu nhiên*, nên chúng không bao giờ đồng thuận với nhau. Vật động thì không ngẫu nhiên: cả đám điểm trên một người cùng vi phạm theo cùng một hướng. Khi tỉ lệ đó vượt một nửa, tập điểm động trở thành *nhóm đa số nhất quán*, và RANSAC chọn đúng chúng làm nghiệm. Lúc đó nó không chỉ chậm; nó cho ra câu trả lời sai một cách tự tin.

Ghép hai phép tính lại thì ra vùng làm việc thật của tầng ngữ nghĩa, và nó hẹp. Khi vật động chỉ chiếm một phần nhỏ khung hình, RANSAC gánh được và mặt nạ gần như không thêm gì. Khi vật động chiếm quá nửa số phép đo và cùng vi phạm theo một hướng, chúng thành nhóm đa số nhất quán, RANSAC lật ngược, và mặt nạ là thứ duy nhất cứu được — ranh giới “quá nửa” này suy từ chính cơ chế bỏ phiếu, không phải từ một ngưỡng chính tay. Giữa hai đầu là một vùng xám, và chỗ nó bắt đầu với chỗ nó kết thúc thì phụ thuộc cảnh, phụ thuộc mật độ đặc trưng và phụ thuộc chính hệ của bạn. Đó là thứ phải *đo*, không phải thứ đọc ra từ một con số in sẵn, và thí nghiệm oracle ở mục I là cách đo nó.

Hình 7.4 cho thấy mặt nạ cắm vào chỗ nào trong đường ống, và quan trọng hơn, nó đổi những gì ở phía sau. Chú ý rằng nó không chỉ đổi tập phép đo: nó đổi luôn quyết định chèn khung hình chính, và quyết định đó lan ra toàn bộ đồ thị.



Mặt nạ can thiệp ở một chỗ, nhưng hậu quả lan ra bốn chỗ. Chỉ hậu quả đầu tiên là thứ bạn định làm. Ba hậu quả sau là tác dụng phụ, và chúng là lý do một so sánh “có mặt nạ với không mặt nạ” hiếm khi cô lập được đúng biến bạn muốn đo.

Hình 7.4. Mặt nạ cắm vào đâu và nó đổi những gì. Điểm can thiệp chỉ có một, ngay sau trích xuất đặc trưng, nhưng hậu quả lan qua bốn khối phía sau. Hậu quả thứ hai là kiểu hồng im lặng, và hậu quả thứ ba giải thích vì sao mọi so sánh trong mảng này đều cần khoá hoặc báo cáo chính sách khung hình chính.

H. Vì sao xóa quá tay còn tệ hơn không xóa

Đây là điểm mà tôi cho là quan trọng nhất của cả chương, và cũng là điểm ít được viết ra nhất trong các paper về mảng này. Trực giác thông thường nói rằng xóa thừa thì cùng lắm là lãng phí. Trực giác đó sai, vì nó coi các phép đo là những mẫu bằng chứng độc lập và có thể thay thế cho nhau. Chúng không phải vậy.

Bộ tối ưu không cần *nhiều* ràng buộc. Nó cần ràng buộc *trải đủ mọi hướng*. Một mặt nạ xóa đi một mảng lớn liền khối của ảnh — và mặt nạ ngữ nghĩa luôn liền khối, vì vật thể thì liền khối. Các điểm còn lại vì thế dồn về phần còn lại của khung hình. Ma trận thông tin của bài toán tư thế mất đi một hướng, và tư thế có một chiều gần như không bị ràng buộc.

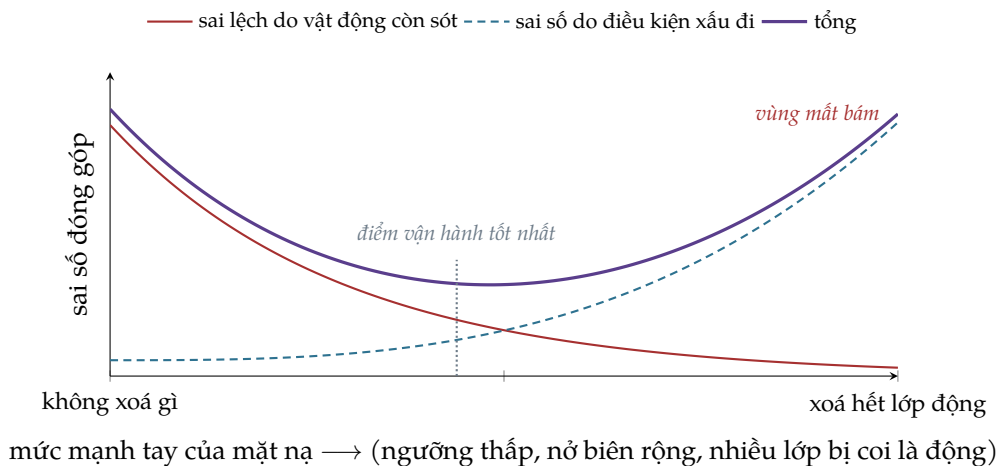
Kiểu hồng này im lặng, và đó là chỗ nguy hiểm. Không có cớ ngoại lai nào bật lên. Phần dư còn lại nhỏ và đẹp, vì mọi phép đo còn lại đều là phép đo tốt. Hiệp phương sai mà hệ báo ra thì lạc

quan, vì nó tính từ chính tập điểm đã bị lọc. Sai số chỉ hiện ra sau đó vài giây, dưới dạng trôi, và lúc đó bạn sẽ đi tìm nguyên nhân ở chỗ khác.

Ba cái giá cụ thể, xếp theo mức độ hay gặp:

- **Mất bám hần.** Hệ tuyên bố mất bám khi số điểm bản đồ theo được tụt dưới ngưỡng. Trên một hành lang trắng, xoá một người đứng gần camera có thể lấy đi phần lớn số đặc trưng còn lại. Cái giá không phải vài xăng-ti-mét sai số; nó là một lần định vị lại, và định vị lại có thể chọn nhầm chỗ.
- **Đổi chính sách khung hình chính.** Ít điểm khớp hơn làm điều kiện chèn khung hình chính bật sớm hơn. Cấu trúc đồ thị đổi, mật độ khung hình chính đổi, và mọi con số sau đó đổi theo. Khi bạn so hai cấu hình mà một bên có mặt nạ, bạn thường đang so hai đồ thị khác nhau chứ không phải hai bộ phép đo khác nhau.
- **Điều kiện xấu đi mà không ai báo.** Trường hợp đã mô tả ở trên. Nó là kiểu hỏng khó chẩn đoán nhất vì mọi chỉ số nội bộ đều trông khoẻ.

Hình 7.5 vẽ hai cái giá đối nghịch nhau theo mức độ mạnh tay của mặt nạ. Đây là hình minh hoạ dạng hàm, không phải số đo: hình dạng của hai đường thì tôi tin, còn vị trí cực tiểu thì phụ thuộc cảnh và phải tự đo trên dữ liệu của bạn.



Hình 7.5. Hai cái giá đối nghịch. Xoá ít thì vật động còn sót lại và tạo sai lệch có hướng. Xoá nhiều thì các điểm còn lại dồn về một phía và điều kiện của bài toán tư thế xấu đi; đường này dốc lên nhanh vì nó phi tuyến, và ở cuối nó chuyển thành mất bám hần. Đường tổng có cực tiểu ở giữa, và cực tiểu đó *phụ thuộc cảnh*: hành lang trắng đẩy nó sang trái, quảng trường đông người đẩy nó sang phải. Hình minh hoạ dạng hàm, không phải số đo.

Từ hình đó rút ra hai kết luận thực dụng. Thứ nhất, **hãy hạ trọng số thay vì xoá**. Một mặt nạ nhị phân là một quyết định cứng đặt ở chỗ tồi nhất có thể: đặt ngay đầu đường ống, trước khi bất kỳ bằng chứng nào được thu thập. Nhân độ tin cậy của mặt nạ vào ma trận thông tin của phép đo thì mềm hơn nhiều. Điểm nghi ngờ vẫn đóng góp vào điều kiện của bài toán, nhưng đóng góp ít. Nếu nó thật sự là ngoại lai, hàm mất mát bền vững sẽ đập nó ở vòng lặp sau, khi đã có bằng chứng thật.

↪ **cùng nguyên lý với F/34** — đây đúng là luận điểm của chương 34: mọi quyết định “tin hay không tin” trong đồ thị nhân tử cuối cùng đều là một con số trong ma trận thông tin, và biến nó thành nhị phân là vứt đi thông tin

Kết luận thứ hai ngắn hơn. **Điểm vận hành phải phụ thuộc vào số điểm còn lại**, chứ không phải là một hằng số chỉnh một lần. Quy tắc mà tôi thấy hợp lý — và nói rõ đây là suy luận thiết kế chứ chưa phải kết quả đo — là để mặt nạ mạnh tay khi số đặc trưng còn lại vẫn dư dả, rồi tự động

nơi lỏng khi số đó tụt về gần ngưỡng mất bám. Hệ nên chấp nhận vài phép đo bản hơn là chấp nhận mất bám, vì mất bám đắt hơn nhiều bậc.

I. Đo cho đúng, nếu không bạn sẽ tin nhầm

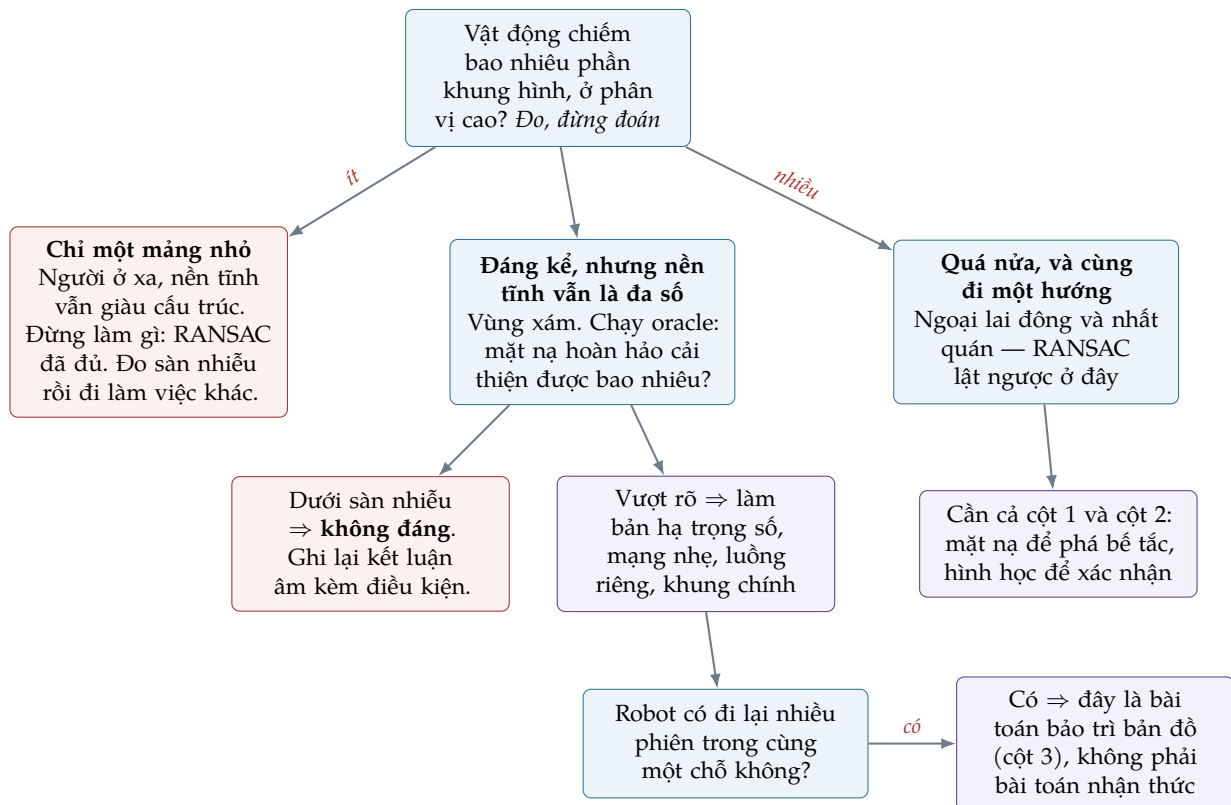
Mảng này có một vấn đề đánh giá nghiêm trọng, và biết trước nó sẽ giúp bạn đọc mọi bảng kết quả trong mảng với đúng liều hoài nghi.

Các con số “cải thiện rất lớn” trong mảng này phần lớn được báo trên vài chuỗi RGB-D trong nhà, nơi có một hoặc hai người chiếm phần rất lớn khung hình, và đối chứng là một hệ hoàn toàn không xử lý cảnh động [140]. Kể cả khi các con số đó thật, chúng gần như vô nghĩa với robot của bạn, vì chúng đo một chế độ hồng thảm khốc trong một cấu hình cực đoan. Robot dịch vụ đi hành lang gặp người chiếm một phần nhỏ khung hình, và ở chế độ đó thì RANSAC vốn đã xử lý ổn.

Bốn kỷ luật đo, và cả bốn đều là thứ bạn đã dùng ở phía SLAM, chỉ cần bê nguyên sang.

1. **Báo tỉ lệ hoàn thành cùng với APE, luôn luôn.** Một hệ mất bám giữa chừng có thể có APE rất đẹp trên đoạn nó còn sống. Với mặt nạ ngữ nghĩa, đây không phải rủi ro lý thuyết mà là chế độ hồng chính. Xem B/07-chia-du-lieu-va-danh-gia cho cách đọc cặp số này.
2. **Đo sàn nhiều trước.** Chạy cùng một cấu hình nhiều lần và lấy độ lệch chuẩn, rồi mới nói về cải tiến [158]. Bạn đã biết hệ của mình là phi tất định; thêm một mạng vào không làm nó tất định hơn.
3. **Chạy một thí nghiệm oracle trước khi làm gì cả.** Gán mặt nạ bằng tay, hoặc bằng một mô hình rất nặng chạy ngoại tuyến, trên vài trăm khung hình. Đó là mặt nạ hoàn hảo. Mặt nạ hoàn hảo cho bạn cái *trần*: mọi công sức làm mặt nạ chạy thời gian thực đều bị chặn trên bởi đúng con số đó. Nếu trần ấy không vượt nổi sàn nhiều bạn đã đo, hãy dừng và đi làm việc khác. Đây là công cụ có giá trị cao nhất trong cả mục.
4. **Cô lập yếu tố gây nhiễu về khung hình chính.** So sánh có mặt nạ với không mặt nạ thường đồng thời đổi mật độ khung hình chính. Muốn biết mặt nạ đóng góp gì, hãy khoá chính sách khung hình chính lại, hoặc báo cáo số khung hình chính kèm theo.

Hình 7.6 gói cả mục thành một cây quyết định, và câu trả lời phổ biến nhất trên cây đó là “chưa cần”.



Hình 7.6. Cây quyết định cho tầng ngữ nghĩa. Các nhánh được mô tả bằng *tình huống* chứ không bằng ngưỡng số, vì ngưỡng đúng phụ thuộc mật độ đặc trưng của cảnh và phải tự do. Nhánh trái cùng là nhánh hay đúng nhất và ít được chọn nhất. Chú ý rằng nhánh phải cùng dẫn tới một bài toán khác hẳn — bảo trì bản đồ qua nhiều phiên — mà không mạng nhận thức nào giải được.

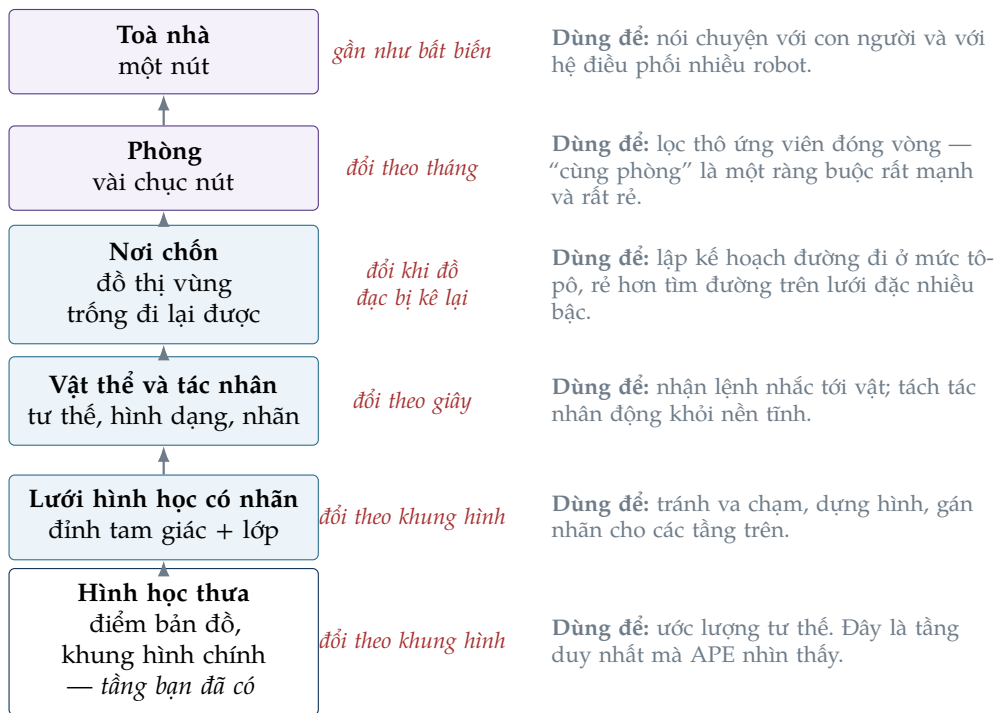
7.4 Đồ thị cảnh 3D (3D Scene Graphs)

Tới đây ta đổi câu hỏi. Nửa đầu chương hỏi ngữ nghĩa giúp *ước lượng tư thế* được gì. Nửa sau hỏi ngữ nghĩa cho *bản đồ* thêm được gì, và người tiêu thụ bản đồ ở đây không còn là script tính APE.

Điểm xuất phát là một quan sát về biểu diễn. Bản đồ của ORB-SLAM3 là một danh sách phẳng gồm điểm 3D và khung hình chính. Nó hoàn hảo cho việc nó sinh ra để làm: cung cấp ràng buộc cho bộ tối ưu. Nhưng nó không trả lời được bất kỳ câu hỏi nào khác. “Tôi đang ở phòng nào” không có câu trả lời, vì không có khái niệm phòng. “Cái ghế nào gần tôi nhất” cũng vậy. Đồ thị cảnh 3D là câu trả lời cho câu hỏi: nếu ta xếp thêm vài tầng trừu tượng lên trên đám mây điểm đó thì được gì.

A. Cấu trúc tầng, và mỗi tầng mua được gì

Hình 7.7 vẽ cấu trúc mà dòng Kimera và Hydra dùng, cùng tốc độ thay đổi và người tiêu thụ của từng tầng [159], [160]. Cột bên phải là cột đáng đọc nhất: nó cho thấy các tầng trên không phải trang trí, chúng có người dùng thật.



Khi một vòng đóng lại, toàn bộ các tầng phải biến dạng theo, không chỉ đồ thị tư thể. Đó là bài toán kỹ thuật khó nhất của Hydra, và là lý do một đồ thị cảnh không phải chỉ là “bản đồ cộng nhân”.

Hình 7.7. Sáu tầng của một đồ thị cảnh 3D, kèm tốc độ thay đổi và người tiêu thụ. Tốc độ thay đổi giảm dần khi lên cao, và đó chính là giá trị của cấu trúc tầng: các tầng trên ổn định nên dùng được làm mỏ neo, trong khi tầng dưới thì đổi liên tục. Tầng dưới cùng là thứ hệ SLAM của bạn đã có; năm tầng trên là thứ phải trả thêm để có.

B. Kimera: thư viện, và bài học về ranh giới mô-đun

Ý tưởng gắn nhãn ngữ nghĩa lên một bản đồ đặc không mới; SemanticFusion đã làm điều đó từ 2017 bằng cách hợp nhất dự đoán từng khung vào một bản đồ bề mặt [161]. Cái Kimera thêm vào là tính mô-đun và tính thời gian thực. Kimera là một thư viện mã nguồn mở dựng bản đồ đo đặc kèm ngữ nghĩa theo thời gian thực, gồm bốn mô-đun tách rời: một bộ đo quán tính–thị giác, một bộ tối ưu đồ thị tư thể bền vững, một bộ dựng lưới cục bộ nhanh, và một bộ gắn nhãn ngữ nghĩa lên lưới toàn cục [11].

Với bạn, giá trị lớn nhất của Kimera không nằm ở thuật toán nào cả. Nó nằm ở chỗ bốn mô-đun ấy có giao diện rõ ràng và thay được từng cái. Bạn có thể giữ nguyên bộ đo quán tính–thị giác của mình và chỉ mượn phần dựng lưới có nhãn. Rất ít hệ thống trong Phần F cho phép mượn từng phần như vậy; phần lớn là một khối liền không tháo rời được. Đó là một bài học thiết kế đáng lấy đi kể cả khi bạn không dùng dòng code nào của Kimera.

Bộ tối ưu đồ thị tư thể bền vững cũng đáng chú ý riêng. Nó loại vòng sai bằng một cơ chế bền vững tăng dần thay vì bằng một ngưỡng cứng, và đó là một câu trả lời trực tiếp cho câu hỏi “một vòng sai đắt bao nhiêu” của F/29-dinh-vi-lai-va-dong-vong.

C. Hydra: dựng đồ thị theo thời gian thực, và biến dạng khi vòng đóng

Hydra là bước từ “đồ thị cảnh như một sản phẩm hậu xử lý” sang “đồ thị cảnh dựng tăng dần trong lúc chạy” [160]. Ba đóng góp, và cái thứ hai là cái khó nhất.

- **Dựng tăng dần.** Các tầng được cập nhật liên tục từ dữ liệu cảm biến, chứ không dựng lại từ đầu. Phân đoạn phòng chạy trên đồ thị nơi chốn theo lối hình học chứ không cần một mạng riêng.
- **Biến dạng toàn đồ thị khi đóng vòng.** Đây là chỗ khó. Một đồ thị tư thể chỉ cần chỉnh lại vài chục tư thể. Một đồ thị cảnh phải kéo theo lưới hàng trăm nghìn đỉnh, các vật thể, các nơi chốn và ranh giới phòng, và phải làm trong thời gian thực. Hydra dùng một đồ thị biến dạng nhúng để lan hiệu chỉnh xuống các tầng dưới. Nếu bạn từng viết phần cập nhật bản đồ sau một lần tối ưu toàn cục, bạn biết đây là loại việc dễ sinh bết tắc và dễ sinh trạng thái không nhất quán.
- **Phát hiện vòng từ chính cấu trúc phân tầng.** Thay vì so một bộ mô tả ảnh toàn cục, Hydra so từ trên xuống: cấu hình phòng, rồi chòm sao vật thể, rồi mới tới vẻ ngoài. Đây là ý đáng suy nghĩ nhất của cả mục đối với nỗi đau đóng vòng, vì nó có *chế độ hồng khác* với bộ mô tả vẻ ngoài. Bộ mô tả vẻ ngoài chết vì đối ánh sáng và vì hành lang giống nhau; chòm sao vật thể thì không quan tâm tới ánh sáng, nhưng lại chết ở nơi trống trải không có vật nào.

Hydra-Multi mở rộng sang nhiều robot cùng dựng chung một đồ thị, với một nút tập trung lo việc gộp và lo các vòng liên robot [162]. Vấn đề kỹ thuật thật nằm ở gộp: hai robot gọi cùng một cái phòng bằng hai nút khác nhau, và phải quyết định khi nào chúng là một.

D. Clio: tác vụ quyết định mức chi tiết

Clio đặt một câu hỏi mà tôi cho là câu hỏi hay nhất trong cả mảng đồ thị cảnh: *cái gì được tính là một vật thể?* [163]

Câu trả lời thông thường lấy từ tập nhãn: một vật là một thứ có tên trong danh sách lớp. Clio nói câu trả lời đó sai về nguyên tắc, vì “vật thể” không phải thuộc tính của cảnh, nó là thuộc tính của *tác vụ*. Một chồng sách là một vật nếu nhiệm vụ là “dọn chồng sách này đi”. Nó là mười vật nếu nhiệm vụ là “tìm quyển bìa đỏ”. Không có mức chi tiết nào đúng khi chưa biết nhiệm vụ.

Cơ chế Clio dùng để hình thức hoá điều đó là nút thất thông tin: cho trước một danh sách nhiệm vụ viết bằng ngôn ngữ tự nhiên, gom các mảnh nguyên thuỷ thành các cụm sao cho cụm giữ được nhiều thông tin nhất về nhiệm vụ trong khi vẫn đơn giản nhất có thể. Bản đồ vì thế *co giãn theo nhiệm vụ*: cùng một cảnh cho ra hai đồ thị khác nhau với hai danh sách nhiệm vụ khác nhau.

Bài học thiết kế ở đây rộng hơn đồ thị cảnh rất nhiều, và nó nổi thẳng vào thói quen của bạn. Đó chính là nguyên tắc “chọn metric khớp chi phí thật” đặt vào tầng biểu diễn: mức chi tiết của bản đồ phải suy ra từ việc bản đồ sẽ được dùng để làm gì, chứ không suy ra từ việc cảm biến cho được bao nhiêu.

→ cùng nguyên lý với B/05 — đây là bước “phát biểu bài toán từ chi phí thật” của chương thiết kế bài toán, lần này áp cho biểu diễn bản đồ chứ không cho hàm mất mát

E. Khronos: thời gian là một trục, không phải một cái cờ

Khronos quay lại đúng cột 3 của mục trước, và nó là câu trả lời nghiêm túc nhất tôi biết cho cột đó [157]. Ý tưởng cốt lõi là bỏ hẳn phép phân loại nhị phân tĩnh hay động. Thay vào đó, mỗi vật được gán một *khoảng thời gian tồn tại*: nó xuất hiện ở đâu, từ lúc nào tới lúc nào.

Phát biểu đó gộp được hai hiện tượng vốn bị coi là khác loại. Một người đi ngang là một vật có khoảng tồn tại rất ngắn ở mỗi vị trí. Một chiếc ghế bị kê lại là một vật có khoảng tồn tại dài ở vị trí cũ, rồi một khoảng khác ở vị trí mới. Cùng một mô hình, cùng một cơ chế suy luận. Bản đồ kết quả trả lời được câu hỏi “thế giới trông thế nào lúc t ”, và với robot đi lại lâu dài thì đó mới là câu hỏi đúng.

Cái giá thì thẳng thắn. Bạn cần bản đồ đặc, cần lưu lịch sử, và cần đủ nhiều lần đi qua để có bằng chứng. Với một hệ điểm thưa chạy một lần trên một chuỗi, không có gì để lấy về ở đây cả.

F. ConceptGraphs và EFM3D: tập nhãn mở, và một cái thước

ConceptGraphs dựng đồ thị cảnh với tập nhãn mở bằng cách ghép ba mảnh có sẵn: một bộ phân đoạn không phân biệt lớp cho ra vùng, một mô hình thị giác-ngôn ngữ gán đặc trưng cho từng vùng, và một mô hình ngôn ngữ suy ra quan hệ giữa các vật để làm cảnh [164]. Không huấn luyện gì thêm.

Đây là kiến trúc rất hợp thời và cũng rất đắt. Mỗi khung hình phải qua ít nhất hai mô hình nền, và chất lượng đồ thị bị chặn trên bởi chất lượng của bước liên kết vùng qua các khung — đúng cái bước mà không mô hình nền nào lo hộ bạn. Với ngân sách Orin, đây là công cụ ngoại tuyến.

EFM3D không phải một hệ thống mà là một cái thước: một benchmark cho các mô hình nền 3D nhìn từ góc nhìn người đeo thiết bị, với hai tác vụ là phát hiện vật thể 3D và hồi quy bề mặt [165]. Tôi nhắc nó ở đây vì một lý do đã lặp lại nhiều lần trong Phần F: trong một mảng chưa chín, một cái thước tốt thường đẩy tiến bộ đi xa hơn một hệ thống mới. Mảng đồ thị cảnh vẫn đang thiếu thước đo chung, và đó là một phần lý do khó so sánh các công trình trong mảng.

G. Cái giá thật của việc nhận một đồ thị cảnh vào hệ của bạn

Cần nói rất rõ chỗ này, vì nó dễ bị đánh giá thấp. Nhận một đồ thị cảnh không phải là “thêm một mạng”. Nó là đổi loại bản đồ.

Mọi hệ trong mục này đều dựng trên một bản đồ *đặc*: lưới tam giác hoặc lưới voxel, cập nhật liên tục từ độ sâu. ORB-SLAM3 không có thứ đó và không có chỗ để cắm vào. Nhận đồ thị cảnh nghĩa là phải thêm một luồng dựng bản đồ đặc, và luồng đó tiêu bộ nhớ cùng băng thông ở bậc hoàn toàn khác với một đám mây điểm thưa. Trên Orin, đây thường là ràng buộc chặn chứ không phải chi phí mạng nhận thức.

Kèm theo đó là một khoản nợ kỹ thuật ít ai nhắc: mỗi lần tối ưu toàn cục, tất cả các tầng phải được kéo theo cho nhất quán. Bạn đã từng gặp bế tắc giữa luồng lập bản đồ cục bộ và luồng hiệu chỉnh toàn cục. Số tầng phải giữ đồng bộ giờ tăng từ một lên sáu.

7.5 Ngữ nghĩa mở và ngôn ngữ (Open-Vocabulary Semantics and Language)

Nhóm cuối cùng là nhóm ồn ào nhất, và với riêng bạn thì là nhóm ít liên quan nhất. Vì vậy mục này ngắn, và nó tập trung vào một câu hỏi duy nhất: *cái này giải bài toán của ai?*

A. Bốn công trình, một ý tưởng

Ý tưởng chung của cả nhóm là gán một véc-tơ đặc trưng vào từng phần của bản đồ 3D, sao cho véc-tơ đó nằm trong cùng không gian với đặc trưng của chữ. Khi đó truy vấn bằng chữ trở thành phép so cosin, và bạn không cần biết trước danh sách lớp.

- **ConceptFusion** hợp nhất đặc trưng đa phương thức mở vào bản đồ 3D, không cần huấn luyện thêm, và truy vấn được bằng chữ, bằng ảnh hoặc bằng một cú bấm [166]. Đóng góp kỹ thuật đáng chú ý là cách trộn đặc trưng cục bộ của từng vùng với đặc trưng toàn cục của cả ảnh, vì đặc trưng cục bộ đơn thuần mất ngữ cảnh.
- **LERF** nhúng đặc trưng ngôn ngữ vào một trường bức xạ ở nhiều mức tỉ lệ, rồi dựng ảnh chúng như dựng ảnh màu [167]. Kết quả là một trường “mức liên quan” trong không gian

3D. Giới hạn nằm ngay trong cách làm: nó cho vùng chứ không cho ranh giới thực thể, nó cần tối ưu riêng cho từng cảnh, và nó chậm.

- **OpenScene** chứng cất đặc trưng mở từ ảnh 2D xuống một mạng 3D, nên mỗi điểm 3D mang sẵn một đặc trưng đồng không gian với chữ [168]. Ưu điểm so với LERF là không phải tối ưu lại cho từng cảnh.
- **SpatialLM** (2025) đi theo hướng khác: nhận đám mây điểm và sinh ra mô tả cảnh có cấu trúc — tường, cửa, cửa sổ, hộp bao vật thể — ở dạng máy đọc được. Tôi chưa kiểm chứng chi tiết công trình này, nên chỉ ghi nhận ý tưởng: dùng một mô hình ngôn ngữ làm bộ giải mã *bổ cục*, không phải làm bộ trả lời câu hỏi.

B. Cái này giải bài toán của ai

Trả lời thẳng: không phải của bạn, nếu đầu ra của hệ thống bạn là một quỹ đạo.

Không công trình nào trong nhóm này làm APE tốt lên. Chúng không chạm vào bốn nỗi đau của Phần F. Chúng cộng thêm một mô hình nền vào ngân sách vốn đã chật, và cộng thêm một chế độ hỏng mới — một truy vấn chữ trả về sai vật, không có cách nào chẩn đoán và không có ngưỡng nào để chỉnh. Với một hệ đo quỹ đạo, đây là chi phí thuần.

Nhưng câu trả lời đối hẩn khi người tiêu thụ bản đồ đổi. Nếu robot của bạn phải nhận lệnh “ra chỗ máy in lấy tập tài liệu”, thì không có tầng nào trong hệ hiện tại của bạn hiểu được chữ “máy in”. Đám mây điểm không có từ vựng. Đây đúng là tầng còn thiếu, và không có cách nào vòng qua nó bằng hình học. Với robot có nhiệm vụ, nhóm này chuyển từ “chi phí thuần” thành “điều kiện cần”.

Có đúng một đường mà nhóm này có thể quay lại giúp phần hình học, và tôi nêu ra như một giả thuyết chưa được kiểm chứng chứ không phải một kết luận. Đó là kiểm chứng vòng: một chòm sao vật thể có nhãn là bất biến với ánh sáng và với góc nhìn theo cách mà bộ mô tả vẽ ngoài không có. Hydra đã đi được một đoạn theo hướng này bằng cấu trúc phân tầng [160]. Liệu ngữ nghĩa mở có làm tốt hơn ngữ nghĩa tập đóng ở đây không thì tôi chưa thấy bằng chứng nào, và đó là một ablation đáng làm.

Bảng 7.2. Bốn nhóm trong nửa sau chương, đọc theo người tiêu thụ. Cột “ảnh hưởng tới APE” là cột phân loại nhanh nhất: nếu nó bằng không thì công trình đó thuộc về một cuộc trò chuyện khác, dù nó xuất hiện trong cùng một hội nghị và cùng một phiên.

Nhóm	Người tiêu thụ thật	Ảnh hưởng tới APE	Điều kiện để nó đáng làm
Lọc vật động (cột 1 + 2)	Bộ tối ưu tư thế	Có, cả hai chiều	Vật động vượt vùng RANSAC xử lý được; oracle cho thấy trần cải thiện đủ lớn
Bản đồ theo thời gian (cột 3)	Bản đồ sống qua nhiều phiên	Gián tiếp, qua chất lượng định vị lại	Robot đi lại nhiều lần cùng một chỗ, và cảnh thật sự đổi giữa các lần
Đồ thị cảnh phân tầng	Bộ lập kế hoạch; và tầng lấy ứng viên vòng	Gián tiếp, chỉ qua đóng vòng	Đã có sẵn bản đồ đặc; nếu chưa có thì đây là dự án nhiều tháng
Ngữ nghĩa mở, ngôn ngữ	Con người ra lệnh	Bằng không	Nhiệm vụ được phát biểu bằng chữ; không có nhiệm vụ như vậy thì đây là chi phí thuần

I 7.6 Giới hạn và trường hợp hỏng

Giới hạn nặng nhất đã nêu ở giữa chương, nhưng nó đáng được nhắc lại như một kết luận độc lập: **tầng ngữ nghĩa hỏng tương quan với tầng hình học, chứ không độc lập**. Ảnh mờ, thiếu sáng, chuyển động nhanh, phơi sáng nhảy — đó vừa là những khung hình mà ghép cặp đặc

trung yếu nhất, vừa là những khung hình nằm xa nhất khỏi phân phối huấn luyện của mạng. Một tầng dự phòng chỉ đáng tiền khi nó hỏng độc lập với tầng chính. Ở đây thì không, nên đừng thiết kế hệ thống dựa trên giả định rằng ngữ nghĩa sẽ đỡ cho bạn ở các khung hình khó.

Chế độ hỏng thứ hai là giả định khối cứng. Các hệ theo vết vật thể đều mô hình mỗi vật như một khối cứng có một tư thế. Con người thì không phải khối cứng, và con người lại chính là vật động phổ biến nhất trong mọi tập dữ liệu trong nhà. Hệ quả là các phương pháp đẹp nhất về mặt lý thuyết trong dòng này lại yếu nhất đúng ở trường hợp thường gặp nhất.

Chế độ hỏng thứ ba nằm ở tập nhãn. Một mô hình huấn luyện trên COCO biết tám mươi lớp, và nhà kho của bạn có xe nâng, pallet, tấm chắn, xe đẩy. Tập nhãn mở giải được về nguyên tắc, nhưng chi phí của nó đặt nó ra ngoài vòng lặp, nên trong thực tế bạn quay về đường chưng cất: mô hình lớn gán nhãn ngoại tuyến, mô hình nhỏ chạy trực tuyến. Đường đó có một điểm gãy riêng — mô hình nhỏ kế thừa luôn mọi sai lệch của mô hình lớn, và bạn không còn chân trị nào để phát hiện.

Chế độ hỏng thứ tư là chế độ hỏng của *đánh giá*, và nó âm thầm nhất. Các bảng kết quả trong mảng này phần lớn đo trên chuỗi trong nhà, RGB-D, người chiếm phần rất lớn khung hình, đối chứng là hệ không xử lý gì. Con số cải thiện lớn ở đó không chuyển sang chế độ vận hành của bạn. Muốn biết nó có chuyển hay không, chỉ có một cách: chạy thí nghiệm oracle trên dữ liệu của chính bạn, trước khi viết dòng code nào.

Riêng với đồ thị cảnh, ba giới hạn thêm. Khái niệm “phòng” không tồn tại ngoài trời và rất mơ hồ trong không gian mở, nên tầng phòng mất giá trị đúng ở môi trường mà bạn hay chạy nhất. Chi phí bản đồ đặc là ràng buộc chặn thật trên Orin, không phải chi phí mạng. Và việc giữ sáu tầng nhất quán sau mỗi lần tối ưu toàn cục là một nguồn bế tắc và trạng thái hỏng mà bạn đã nếm ở quy mô nhỏ hơn nhiều.

Cuối cùng, một giới hạn về chính chương này. Phần lớn công trình tôi nhắc sau năm 2023 tôi chỉ đọc chứ chưa chạy. Vì thế chương này cố ý *không* nêu một con số hiệu năng nào: không mili-giây, không FPS, không tỉ lệ cải thiện. Vài phép tính bằng tay còn lại đều có dẫn “giả sử” và chỉ nói về hình dạng của một quan hệ hình học, không nói về hiệu năng của hệ nào. Chỗ nào trước đây cần một con số thì bây giờ là một thứ hạng định tính kèm phép đo bạn phải tự chạy. Thứ hạng dùng để loại phương án; chỉ con số bạn tự đo trên máy mình mới dùng được để lập kế hoạch.

7.7 Thực hành

Repo và tài nguyên. Bắt đầu từ phía nhận thức: [ultralytics/ultralytics](#) cho dòng YOLO và cho đường xuất sang TensorRT, [open-mmlab/mmdetection](#) khi cần so nhiều họ detector trong cùng một khung, [facebookresearch/segment-anything](#) và [facebookresearch/sam2](#) để thử phân đoạn theo prompt, [IDEA-Research/GroundingDINO](#) khi cần một bộ gán nhãn tập mở chạy ngoại tuyến. Phía SLAM động: [BertaBescos/DynaSLAM](#) là bản tham chiếu ngắn gọn nhất để đọc, vì nó cắm thẳng vào ORB-SLAM2 nên bạn thấy rõ đúng chỗ mặt nạ can thiệp. Phía đồ thị cảnh: [MIT-SPARK/Kimera-VIO](#), [MIT-SPARK/Hydra](#) và [MIT-SPARK/Khronos](#) là ba bậc thang của cùng một dòng, đọc theo thứ tự đó thì thấy rõ mỗi bậc thêm gì. Phía tập nhãn mở: [concept-graphs/concept-graphs](#). Cuối cùng vẫn là [evo](#) để mọi con số bạn báo ra so được với nhau. Danh sách này chốt tại tháng 9/2026; tên mô hình dẫn đầu sẽ cũ trong sáu tới mười hai tháng, còn hạ tầng như TensorRT, [evo](#) và các thư viện dựng lưới thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Trên chuỗi khó nhất của bạn, vật động chiếm bao nhiêu phần khung hình ở phân vị 95, và bạn đo con số đó bằng cách nào? Nếu mặt nạ của bạn xóa một phần lớn số đặc trưng, chính sách chèn khung hình chính có đổi không, và bạn kiểm điều đó ra sao? Một vật đi thẳng ra xa camera vi phạm ràng buộc hình học nào — và nếu câu trả lời là “không

cái nào”, thì cái gì phát hiện được nó? Đồ thị cảnh của Hydra cần thứ gì mà bản đồ điểm thưa của bạn không có, và thêm thứ đó tốn bao nhiêu bộ nhớ trên Orin?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Mặt nạ ngữ nghĩa	Ảnh nhị phân đánh dấu các điểm ảnh thuộc lớp có thể di chuyển. Nó là tri thức tiên nghiệm về <i>lớp</i> , không phải phép đo về thực thể trước mặt, nên một mình nó không kết luận được vật đang động.
●●● Ba việc khác nhau khi loại vật động	Vật <i>có thể</i> động (ngữ nghĩa nói), vật <i>đang</i> động (hình học nói), vật <i>đã</i> động rồi đứng yên (chỉ trí nhớ của bản đồ nói). Không việc nào suy ra được việc nào, và mỗi việc có một kiểu hỏng riêng.
●● Vòng luẩn quẩn tư thế–vật động	Muốn tính phần dư để phát hiện vật động thì cần tư thế camera, mà tư thế lại đang ước lượng từ chính tập điểm chứa vật động. Khi vật động chiếm phần lớn khung hình, kết luận đảo ngược hoàn toàn.
●●● Điều kiện của bài toán	Mức độ trải đủ hướng của tập ràng buộc còn lại. Xoá một mảng liên khối làm điểm dồn về một phía và tư thế có một hướng gần như không bị ràng buộc; không cờ ngoại lai nào bật lên, nên đây là kiểu hỏng im lặng.
●●● Hạ trọng số thay vì xoá	Nhân độ tin cậy của mặt nạ vào ma trận thông tin của phép đo, thay cho một quyết định nhị phân đặt ở đầu đường ống. Điểm nghi ngờ vẫn góp vào điều kiện của bài toán, và hàm mất mát bền vững vẫn đập được nó khi có bằng chứng thật.
●●● Thí nghiệm oracle cho mặt nạ	Dựng mặt nạ hoàn hảo bằng tay hoặc bằng mô hình nặng chạy ngoại tuyến, rồi đo cải thiện. Nó chặn trên mọi công sức làm mặt nạ thời gian thực; nếu trần đó nằm dưới sàn nhiều thì dừng ngay.
●●● DynaSLAM	Ghép mặt nạ phân đoạn thực thể cho các lớp có thể động với một kiểm tra sai lệch độ sâu đa khung, rồi vá lại nền. Bản tham chiếu gọn nhất của việc dùng cả hai nguồn bằng chứng; giá phải trả là một lượt mạng nặng mỗi khung. Họ hàng gần: DS-SLAM chạy mạng trên luồng riêng, DynaSLAM II đưa tư thế vật vào hiệu chỉnh chùm tia.
●●● SAM	Phân đoạn theo prompt: điểm, khung hoặc mặt nạ thô. Bộ mã hoá ảnh nặng chạy một lần mỗi ảnh, bộ giải mã nhẹ chạy một lần mỗi prompt. Nó <i>không</i> trả về nhãn lớp, nên một mình nó không lọc được vật động.
●● Grounding DINO	Phát hiện vật theo mô tả bằng chữ, nên gỡ được ràng buộc tập nhãn đồng. Quá nặng cho vòng lặp trên Orin; vai trò đúng là bộ gán nhãn ngoại tuyến để chưng cất xuống một detector nhỏ. Open-YOLO 3D làm cùng tinh thần tiết kiệm ở tầng 3D bằng cách thay mặt nạ nặng bằng hộp.
●●● Đồ thị cảnh 3D	Bản đồ nhiều tầng: hình học, lưới có nhãn, vật thể và tác nhân, nơi chốn, phòng, toà nhà. Tốc độ thay đổi giảm dần khi lên cao, nên tầng trên dùng được làm mỏ neo. Kimera là thư viện mở dựng tầng lưới có nhãn, với các mô-đun thay được từng cái.
●● Hydra	Dựng đồ thị cảnh tăng dần theo thời gian thực. Việc khó nhất không phải nhận dạng mà là biến dạng cả sáu tầng cho nhất quán mỗi khi một vòng đóng lại. Nó cũng phát hiện vòng bằng cấu trúc phân tầng thay vì bằng vẽ ngoài, nên có chế độ hỏng khác.
●● Clio	Mức chi tiết của bản đồ được quyết định bởi <i>danh sách nhiệm vụ</i> , qua nút thắt thông tin. Một chồng sách là một vật hay mười vật tùy nhiệm vụ; “vật thể” không phải thuộc tính của cảnh.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Khronos	Bỏ phân loại tĩnh–động, thay bằng ước lượng khoảng thời gian tồn tại của từng vật. Nhờ đó chuyển động ngắn hạn và thay đổi dài hạn dùng chung một cơ chế, và bản đồ trả lời được “lúc t thế giới trông thế nào”.
●● Ngữ nghĩa mở	ConceptFusion, LERF và OpenScene gắn vào bản đồ 3D các đặc trưng nằm cùng không gian với chữ, nên truy vấn được bằng ngôn ngữ; ConceptGraphs ghép chúng thành đồ thị. Không cái nào làm APE tốt lên: chúng phục vụ người ra lệnh, không phục vụ bộ ước lượng tư thế.
●●● Vì sao RANSAC không đủ?	RANSAC hổng khi ngoại lai đông và nhất quán, vì khi đó chúng thắng phiếu đồng thuận. Vật động tạo sai lệch <i>có hướng</i> chứ không phải nhiễu ngẫu nhiên, nên cả cụm cùng bỏ phiếu cho một nghiệm sai. Đó là vùng duy nhất mặt nạ thật sự đáng tiền.
●●● Vì sao mặt nạ ngữ nghĩa một mình không đủ?	Sáu lý do: nhân nói về lớp chứ không về thực thể; có vật động ngoài tập nhân; có chuyển động gián tiếp như bóng đổ và phản chiếu; biên mặt nạ lệch đúng chỗ góc ORB nằm; mặt nạ luôn cũ vài khung; và mạng hổng đúng ở khung hình khó.
●● Vì sao bỏ triết phi cực đại lại quan trọng với hệ thời gian thực?	Vì thời gian hậu xử lý tỉ lệ với số khung sống sót, nên đuôi độ trễ dày lên đúng ở cảnh đông người. YOLOv10 bỏ nó bằng hai phép gán nhất quán, RT-DETR bỏ nó bằng dự đoán tập hợp; cái được là độ trễ hằng số, không phải mAP.
●●● Vì sao mạng hổng đúng lúc cần nó nhất?	Ảnh mờ, thiếu sáng và chuyển động nhanh vừa làm hình học yếu, vừa nằm ngoài phân phối huấn luyện của mạng. Hai kiểu hổng tương quan dương, nên tầng ngữ nghĩa không dùng được làm lưới an toàn.
●● APE đẹp lên nhưng chuỗi hay mất bám — nghĩa là gì?	Gần như chắc chắn là xoá quá tay: APE chỉ tính trên đoạn còn sống nên hệ mất bám sớm lại có APE đẹp. Luôn báo tỉ lệ hoàn thành cùng APE, và so cả hai với sản phẩm đo từ nhiều lần chạy.
●● Bật mặt nạ thì số khung hình chính giảm hẳn — nghĩa là gì?	Mặt nạ đã đổi luôn chính sách chèn khung hình chính, nên bạn đang so hai đồ thị khác nhau chứ không phải hai bộ phép đo. Cách phân biệt: khoá chính sách khung hình chính lại rồi chạy lại, hoặc báo cáo số khung hình chính kèm APE.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
BertaBescos/DynaSLAM [10]	Ghép cột 1 với cột 2: mặt nạ ngữ nghĩa cho “có thể động”, kiểm tra độ sâu đa khung cho “đang động”	Bản tham chiếu ngắn nhất để đọc; cắm thẳng vào ORB-SLAM2
MIT-SPARK/Hydra [160]	Dựng đồ thị cảnh phân tầng theo thời gian thực và biến dạng cả sáu tầng khi vòng đóng	Đang được bảo trì; là nền cho Clio và Khronos
Khronos [157]	Thay phân loại tĩnh–động bằng khoảng thời gian tồn tại của từng vật — lời giải cho cột 3	Cần bản đồ đặc và cần nhiều lần đi qua
Clio [163]	Mức chi tiết của bản đồ suy ra từ danh sách nhiệm vụ, qua nút thất thông tin	Bài học thiết kế rộng hơn phạm vi đồ thị cảnh
ultralytics	Dòng detector nhẹ chạy được trong vòng lặp; đường xuất sang TensorRT là phần dùng nhiều nhất	Hạ tầng; bền hơn tên phiên bản mô hình
Dòng SAM [150], [151]	Cấu trúc chi phí “mã hoá một lần, giải mã theo prompt” — mẫu đặt phần đất ở chỗ được khẩu hao	Chủ yếu dùng ngoại tuyến để gán nhãn dữ liệu

MINI PROJECT SLAM · Đo trần của việc lọc vật động trên dữ liệu của bạn · 1–2 ngày

Mục tiêu	trả lời câu hỏi “có đáng làm không” bằng một con số, <i>trước</i> khi bỏ công làm mặt nạ chạy thời gian thực. Đây là thí nghiệm oracle của chương 24, áp cho tầng ngữ nghĩa.
Dữ liệu	một chuỗi có người đi lại thật — OpenLORIS, hoặc một chuỗi bạn tự quay ở hành lang có người qua lại. Kèm một chuỗi EuRoC làm đối chứng âm, vì EuRoC gần như không có vật động và kết quả ở đó phải là “không đối gì”.
Các bước	trước hết đo phân bố diện tích vật động: chạy một mô hình phân đoạn nặng ngoại tuyến, ghi tỉ lệ điểm ảnh mang nhãn động cho từng khung, lấy phân vị 95. Sau đó dựng <i>mặt nạ oracle</i> bằng chính mô hình nặng đó chạy ngoại tuyến, nở biên vài điểm ảnh, lưu ra đĩa. Chạy ba cấu hình, mỗi cấu hình mười lần: không mặt nạ, mặt nạ oracle dạng xoá cứng, mặt nạ oracle dạng hạ trọng số. Với mỗi lần chạy ghi ba số chứ không phải một: APE, tỉ lệ hoàn thành, và số khung hình chính.
Xong khi	bạn có một bảng ba cấu hình nhân ba số, kèm độ lệch chuẩn qua mười lần chạy, và bạn phát biểu được một trong hai câu: “mặt nạ hoàn hảo cải thiện APE thêm X cm, vượt sàn nhiều, nên bản thời gian thực đáng làm”, hoặc “mặt nạ hoàn hảo không vượt sàn nhiều trên chuỗi này, nên dừng”. Kiểm tra chéo bắt buộc: trên EuRoC cả ba cấu hình phải trùng nhau trong sàn nhiều; nếu không, đường ống mặt nạ của bạn đang đổi một thứ khác ngoài mặt nạ.
Nối với	E/24 cho thí nghiệm oracle và cỡ mẫu; B/07 cho cách đọc cặp APE và tỉ lệ hoàn thành; kết quả của dự án này là điều kiện vào của dự án thứ hai bên dưới.

MINI PROJECT SLAM · Ngân sách thật của mạng nhận thức trên Orin · 1 ngày

Mục tiêu	biến con số FPS trong paper thành con số dùng được, bằng cách đo trong đúng điều kiện mà nó sẽ chạy: cùng máy, cùng lúc, cùng tranh chấp tài nguyên.
Dữ liệu	một chuỗi bất kỳ phát lại ở tốc độ thật trên Jetson Orin, cộng một detector nhẹ đã xuất sang TensorRT ở nửa độ chính xác.
Các bước	đo ba lần. Lần một: chỉ chạy mạng, ghi trung bình và p99 của độ trễ. Lần hai: chỉ chạy SLAM, ghi trung bình và p99 của thời gian mỗi khung, cùng tỉ lệ khung bị bỏ. Lần ba: chạy cả hai cùng lúc, ghi lại đúng bốn con số đó. Sau đó lặp lần ba với mạng đặt ở tần số một phần ba nhịp khung, chỉ trên khung hình chính.
Xong khi	bạn có một bảng bốn dòng cho thấy p99 của cả hai bên tăng bao nhiêu khi chạy chung, và bạn nói được tần số cao nhất mà mạng chạy được mà không đẩy p99 của luồng theo vết vượt ngân sách khung. Nếu p99 chạy chung gần bằng tổng hai p99 riêng, bạn vừa xác nhận rằng “song song” ở đây là chạy xen kẽ.
Nối với	D/21-inference-va-trien-khai cho phần xuất mô hình và lượng tử hoá; B/11 cho cách đọc con số theo mô hình roofigline; F/34 cho phần đo lại trung thực trên phần cứng thật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bộ tối ưu không cần nhiều ràng buộc mà cần ràng buộc trái đủ hướng. Mặt nạ luôn liên khối, nên xoá nó làm các điểm còn lại dồn về một phía và tư thế có một hướng gần như không bị ràng buộc. Kiểu hỏng này im lặng: phần dư còn lại nhỏ và đẹp, hiệp phương sai báo ra thì lạc quan, và sai số chỉ hiện ra vài giây sau dưới dạng trôi.
- A2. Đúng một loại: ngoại lai *đồng và nhất quán*. RANSAC chọn nghiệm được nhiều phiếu nhất, nên khi vật động chiếm phần lớn khung hình và cùng chuyển động theo một hướng, chính chúng thắng phiếu và nền tĩnh bị coi là ngoại lai. Ngữ nghĩa phá thế bế tắc bằng một lá phiếu đến từ ngoài hình học. Ngoài vùng đó, nó chủ yếu lấy đi dữ liệu tốt.
- A3. Không trùng hợp, và đó là kết luận thiết kế quan trọng nhất của chương. Ảnh mờ, thiếu sáng và chuyển động nhanh vừa là các khung hình mà ghép cặp yếu nhất, vừa là các khung hình xa nhất khỏi phân phối huấn luyện của mạng. Một tầng dự phòng chỉ đáng tiền khi nó hỏng độc lập với tầng chính; ở đây hai kiểu hỏng tương quan dương.

- A4. Cái thứ nhất trả lời “lớp này có khả năng tự di chuyển không” và trả lời được ngay từ khung hình đầu, nhưng nó nói về lớp chứ không về thực thể. Cái thứ hai trả lời “vật này đang động ở khung hình này không” nhưng cần một tư thế đủ tốt, tức là mắc vòng luẩn quẩn. Cái thứ ba trả lời “chỗ này đã đổi so với lần trước chưa” và chỉ trả lời được sau nhiều lần đi qua; nó là bài toán bảo trì bản đồ, không phải bài toán nhận thức.
- A5. Nó không giúp gì cả, và câu trả lời trung thực đó chính là công cụ phân loại. Nếu người tiêu thụ bản đồ là một script tính APE, tầng ngôn ngữ là chi phí thuần cộng thêm một chế độ hồng không chẩn đoán được. Nếu người tiêu thụ là một bộ lập kế hoạch nhận lệnh bằng chữ, thì đám mây điểm không có từ vựng và đây đúng là tầng còn thiếu. Hãy hỏi “ai đọc bản đồ này” trước khi hỏi “phương pháp nào tốt hơn”.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn vẽ lại sơ đồ một siêu thị bằng cách đi vòng quanh và ghi nhớ đồ vật ở đâu. Cách này chạy tốt, cho tới khi bạn ghi nhớ nhầm một hộp ngũ cốc nằm trong xe đẩy của người khác. Hôm sau bạn quay lại, hộp ngũ cốc đã đi mất. Nhưng bạn thì tin nó vẫn ở chỗ cũ, nên bạn kết luận rằng *chính bạn* mới là người đang đứng sai chỗ. Toàn bộ sơ đồ trượt theo.

Mẹo là dạy cho máy biết loại đồ nào hay nằm trong xe đẩy, rồi bảo nó bớt tin những thứ đó. Mẹo này ăn tiền vì máy học được từ hàng triệu bức ảnh, còn bản thân phép đo thì không thể tự biết điều đó.

Cái giá thì có ba phần, và cả ba đều thật. Thứ nhất, máy chỉ biết *loại* đồ, không biết cái hộp trước mặt nó lúc này đang nằm trên kệ hay trong xe. Thứ hai, nếu nó gạch bỏ mọi thứ có thể nằm trong xe đẩy, nó có thể gạch nguyên một dãy kệ, và rồi cả một phía của sơ đồ không còn mốc nào để dựa. Thứ ba, nhìn ra đồ vật tốn thời gian, mà mỗi bước chân bạn chỉ có một khoảnh khắc rất ngắn.

Chỗ không rút gọn được: vì sao gạch bỏ dữ liệu *đúng* lại làm sơ đồ tệ đi. Mọi cách nói gọn đều thành “cần mốc rải đều”, nghe như một sự tiện lợi. Sự thật là một phát biểu về hình dạng của một ma trận, và tôi chưa tìm được cách nói ngắn nào vừa đúng vừa không cần tới ma trận.

Một câu: *Máy chỉ biết loại đồ nào hay bị xô dịch, chứ không biết cái trước mặt nó lúc này có xô dịch hay không — nên đừng bảo nó xoá, hãy bảo nó bớt tin.*

7.8 Kết nối

- **C/14-bai-toan-cv-loi — tiền đề.** Chương 14 dạy detection và segmentation là gì: gán mẫu, triệt phi cực đại, dự đoán tập hợp, phân loại mặt nạ. Chương này chỉ dùng chúng, và chỗ nổi là câu hỏi “mô hình nào bỏ vừa vào ngân sách khung hình”.
- **F/28-dac-trung-va-ghep-cap — cùng nguyên lý với.** Mặt nạ ngữ nghĩa và bộ mô tả học được can thiệp vào đúng cùng một chỗ: quyết định của frontend về việc tin phép đo nào. Cái thứ nhất đổi *trọng số*, cái thứ hai đổi *chất lượng*. Hai đòn bẩy độc lập, và nên được đo tách rời nhau.
- **F/29-dinh-vi-lai-va-dong-vong — hệ quả của.** Vòng sai vì hai chỗ chia nhau cùng một chiếc xe đẩy là một chế độ hồng của tầng lấy ứng viên. Đồ thị cảnh phân tầng đề xuất một tầng lấy ứng viên khác, có chế độ hồng khác — và “chế độ hồng khác” mới là thứ đáng giá khi ghép hai bộ phát hiện lại.
- **F/30-do-sau-dong-quang-va-mo-hinh-nen — tiền đề.** Cột 2 của Hình 7.3 chạy bằng dòng quang và độ sâu. Chất lượng phát hiện “đang động” bị chặn trên bởi chất lượng của hai thứ đó, nên hai chương phải đọc cùng nhau.
- **F/32-ban-do-neural — đối lập với.** Cả hai chương đều thay bản đồ điểm thừa bằng thứ khác. Bản đồ neural đi về phía *đặc và liên tục*, tối ưu cho việc dựng lại ảnh. Đồ thị cảnh đi về phía *thừa và có ký hiệu*, tối ưu cho việc suy luận. Hai hướng ngược nhau, và chúng phục vụ hai người tiêu thụ khác nhau.

- **F/34-imu-bat-dinh-va-trien-khai** — cùng nguyên lý với. Kết luận “hạ trọng số thay vì xoá” là một trường hợp riêng của luận điểm chính chương 34: mọi quyết định tin cậy trong đồ thị nhân tử cuối cùng đều là một con số trong ma trận thông tin. Chương 34 cũng cho một nguồn bằng chứng mà chương này không dùng tới: IMU biết camera đã chuyển động thế nào, nên nó giúp tách chuyển động của vật khỏi chuyển động của người quan sát.
- **B/08-dich-chuyen-phan-phoi** — trường hợp riêng của. Việc mạng hồng đúng ở khung hình khó là dịch chuyển phân phối ở dạng độc nhất, vì hướng dịch chuyển trùng với hướng làm hình học yếu đi.
- **E/24-thi-nghiem-va-ablation** — tiền đề. Thí nghiệm oracle và quy tắc đo sản nhiễu trước đều lấy nguyên từ chương 24. Không có hai công cụ đó, mọi kết luận trong chương này đều không kiểm chứng được.
- **B/11-gioi-han-tinh-toan** — ràng buộc bởi. Bảng 7.1 chỉ là chương 11 áp cho một loại phần cứng cụ thể; điều đáng mang đi là thói quen bắt đầu từ ngân sách chứ không từ bảng độ chính xác.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một vật động tạo ra sai lệch *có hướng* chứ không phải nhiễu ngẫu nhiên, và điều đó thay đổi cách bạn chọn hàm mất mát bền vững ra sao?
2. Nếu mặt nạ của bạn nở thêm vài điểm ảnh ở biên, bạn được gì và mất gì — và bạn đo hai thứ đó bằng phép đo nào?
3. Một chuỗi cho APE tốt lên rõ rệt khi bật mặt nạ, nhưng số khung hình chính cũng giảm hẳn. Bạn kết luận được gì, và thí nghiệm nào tách được hai hiệu ứng đó?
4. Vì sao thí nghiệm oracle phải chạy *trước* khi bạn viết dòng code nào cho mặt nạ thời gian thực, và điều gì xảy ra nếu bạn đảo thứ tự?
5. Đồ thị cảnh phân tầng phát hiện vòng bằng cấu trúc thay vì bằng vẻ ngoài. Chế độ hồng của nó khác chế độ hồng của bộ mô tả ảnh toàn cục ở đâu, và vì sao sự khác biệt đó mới là lý do đáng ghép hai bộ lại?
6. Nếu bạn phải chọn giữa một detector nhanh hơn hai lần và một detector có độ trễ ổn định hơn hai lần, bạn chọn cái nào cho vòng lặp theo vết, và lý do nằm ở đại lượng nào?
7. Vì sao “bản đồ theo thời gian” không phải là một bài toán nhận thức, và việc ném thêm một mạng phân đoạn vào nó hồng ở bước nào?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., "Image Matching Across Wide Baselines: From Paper to Practice," *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., "RoMa: Robust Dense Feature Matching," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., "Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., "HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, "Closed-form solution of absolute orientation using unit quaternions," *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., "Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, "Rethinking Visual Geo-localization for Large-Scale Applications," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, "MixVPR: Feature Mixing for Visual Place Recognition," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., "EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, "Optimal Transport Aggregation for Visual Place Recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features Without Supervision," *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., "AnyLoc: Towards Universal Visual Place Recognition," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

IMU, bất định học được và đưa lên phần cứng (IMU, Learned Uncertainty and Deployment)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng — chương này giả định bạn đã có bộ lọc đó); F/28-dac-trung-va-ghep-cap (ghép cặp học được, nguồn của bất định theo từng điểm khớp); D/21-inference-va-trien-khai và D/20-toi-uu-va-nen-mo-hinh (TensorRT, lượng tử hoá — chương này không nhắc lại); E/24-thi-nghiem-va-ablation (sản nhiễu, cỡ mẫu).

TÓM TẮT

Chương này gom hai nỗi đau còn lại của một hệ stereo-inertial đã tinh chỉnh kỹ: đặt trọng số cho IMU, và ảnh mờ do chuyển động. Cả hai thật ra là một bệnh, nhìn từ hai phía. Hệ cổ điển ước lượng *trạng thái* rất tốt và ước lượng *độ tin cậy của chính nó* rất tệ. Đọc xong bạn kiểm được bằng số ma trận thông tin hiện tại của mình có trung thực không, biết mỗi công trình đóng góp cái gì thật, và biết vì sao khứ mờ trước khi trích đặc trưng thường không giúp. Nếu chỉ nhớ một điều: cái mà học sâu mang lại ở đây không phải một ước lượng tư thế tốt hơn, mà là một ước lượng *độ tin cậy* tốt hơn — và đó đúng là thứ hệ cổ điển đang thiếu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ma trận thông tin (information matrix)** — nghịch đảo của ma trận hiệp phương sai, ký hiệu $\Omega = \Sigma^{-1}$. Trong một đồ thị nhân tử, nó chính là *trọng số* của một ràng buộc: càng tin phép đo thì Ω càng lớn và ràng buộc kéo nghiệm càng mạnh.
- **độ lệch cảm biến (sensor bias)** — sai số cộng thêm, thay đổi chậm, của con quay hồi chuyển và gia tốc kế. Trong tài liệu này “độ lệch” luôn mang nghĩa đó. Nó khác hẳn hai nghĩa còn lại của *bias*: độ lệch hệ thống của một bộ ước lượng thống kê, và tham số cộng b trong một tầng tuyến tính.
- **tiền tích phân IMU (IMU preintegration)** — cách gộp hàng trăm mẫu IMU giữa hai khung hình thành một ràng buộc duy nhất về xoay, vận tốc và vị trí, sao cho khi tư thế đổi thì không phải tích phân lại từ đầu [8]. Nó đi kèm một ma trận hiệp phương sai, và ma trận đó chính là “trọng số IMU” mà cả chương bàn tới.
- **phương sai Allan (Allan variance)** — một cách đo đặc tính nhiễu của cảm biến quán tính: ghi dữ liệu lúc đứng yên trong nhiều giờ rồi vẽ độ lệch chuẩn của trung bình theo độ dài cửa sổ trung bình. Nó tách được nhiễu trắng, độ trôi của độ lệch, và bước đi ngẫu nhiên [169].
- **NEES** — viết tắt của *normalized estimation error squared*. Với phần dư r và hiệp phương sai Σ mà bạn khai báo, đại lượng $r^T \Sigma^{-1} r$ phải có trung bình đúng bằng số bậc tự do nếu Σ trung thực. Đây là phép thử chính của cả chương.
- **bất định thay đổi theo đầu vào** — tiếng Anh gọi là *heteroscedastic*. Nghĩa là mức nhiễu không phải một hằng số mà phụ thuộc chính đầu vào: một điểm ảnh trên vùng mờ có sai số lớn hơn hẳn một điểm trên vùng sắc nét, dù cùng một camera.
- **thời gian phơi sáng (exposure time)** — khoảng thời gian cảm biến ảnh gom ánh sáng cho một khung hình, ký hiệu τ . Ảnh ghi được là trung bình của cảnh trong suốt khoảng đó, nên mọi chuyển động trong τ đều biến thành vết mờ.

- **độ trễ đuôi (tail latency)** — phân vị cao của phân bố độ trễ, thường là p99. Nó khác trung bình ở chỗ nó mô tả những lần chậm nhất — và trong hệ thời gian thực thì chính những lần đó phá hạn chết.

CÂU HỎI MỞ ĐẦU

- Q1. Tin IMU quá mức và tin ảnh quá mức đều làm hỏng quỹ đạo. Vì sao hai kiểu hỏng đó nhìn hoàn toàn khác nhau trên đồ thị lỗi, và bạn tách chúng bằng phép đo nào?
- Q2. Con số nhiều con quay trên datasheet là một phép đo thật, do nhà sản xuất làm. Vậy vì sao đặt đúng con số đó vào đồ thị nhân tử lại gần như luôn sai?
- Q3. Nếu một mạng nơ-ron không cho tư thế tốt hơn hình học, nó còn cho được cái gì đáng để bạn trả một phần ngân sách khung hình trên Orin?
- Q4. Khử mờ bằng một mạng rồi mới trích ORB nghe hiển nhiên là đúng. Vì sao nó thường *không* làm tăng số điểm khớp đúng?
- Q5. Bạn đo riêng mạng mới và thấy nó chỉ chiếm một phần nhỏ chu kỳ khung hình. Vì sao phép đo đó vẫn chưa cho phép kết luận là nó vừa ngân sách?

8.1 Đặt vấn đề

Bảy chương trước của Phần F đi theo các khối của hệ thống: đặc trưng, đóng vòng, độ sâu, back-end, bản đồ, ngữ nghĩa. Chương này đi theo hai *triệu chứng* còn lại. Cả hai đều không nằm gọn trong một khối nào.

Triệu chứng thứ nhất: bạn không biết đặt trọng số IMU bằng bao nhiêu. Nói ra thì hệ rung khi ảnh xấu. Siết vào thì hệ trôi theo độ lệch con quay. Bạn đã thử vài giá trị, thấy mỗi giá trị thắng ở một chuỗi khác nhau, và không có lý do nguyên tắc nào để chọn. Triệu chứng thứ hai: những đoạn ảnh mờ làm mất bám, và mọi cách chữa nghĩ ra đều có mùi vá vú.

Hai triệu chứng này trông không liên quan. Chúng là một bệnh. Trong cả hai, hệ thống đang dùng một con số về độ tin cậy mà nó không hề đo. Trọng số IMU là hằng số lấy từ datasheet. Trọng số của một điểm ảnh là σ^2 nhân theo tăng kim tự tháp của điểm đó, tức một hàm của *độ phân giải*, không phải của *chất lượng*. Không con số nào trong hai con số ấy biết ảnh có mờ không, robot có rung không, hay camera vừa quay nhanh cỡ nào.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn đo được ma trận thông tin hiện tại của mình sai bao nhiêu lần. Phép thử có sẵn trong lý thuyết ước lượng và chạy được trong một buổi chiều. Từ con số đó bạn mới quyết định có cần một mạng hay không. Và phần lớn trường hợp, câu trả lời trung thực là chưa cần: cần sửa lại hằng số theo một mô hình. Đó là lý do chương này đặt phép đo trước phương pháp.

TRỰC GIÁC

Một đồ thị nhân tử là một mô hình lò xo. Mỗi ràng buộc là một lò xo nối hai nút, và ma trận thông tin Ω chính là *độ cứng* của lò xo đó. Nghiệm tối ưu là vị trí cân bằng của cả hệ lò xo.

Hình ảnh này giải thích ngay ba chuyện. Một, chỉ *tỉ lệ* độ cứng mới quan trọng: nhân đôi mọi lò xo thì cân bằng không đổi. Hai, một lò xo khai quá cứng sẽ kéo cả hệ về phía nó, kể cả khi nó gắn sai chỗ. Đó chính là một phép đo mờ được tin tưởng quá mức. Ba, độ cứng là một *ma trận* chứ không phải một số: một lò xo có thể rất cứng theo phương ngang và rất mềm theo phương dọc. Chúng ta sẽ thấy vết mờ tạo ra đúng loại lò xo dị hướng đó.

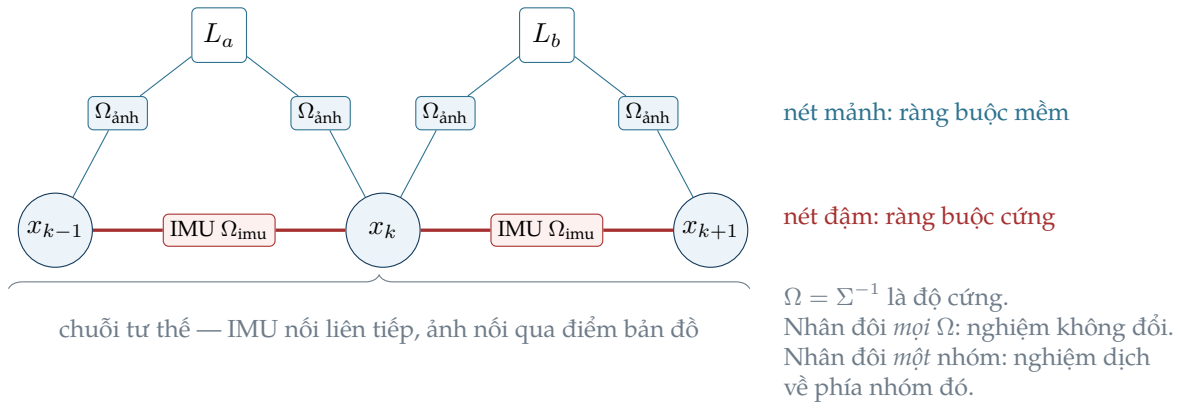
8.2 Trọng số chính là ma trận thông tin (Weights Are the Information Matrix)

A. Trọng số không phải siêu tham số, nó là đại lượng vật lý

Trong ước lượng bình phương tối thiểu phi tuyến, mỗi ràng buộc đóng góp một số hạng $r^T \Sigma^{-1} r$ vào hàm mục tiêu. Ma trận $\Omega = \Sigma^{-1}$ không phải một núm vặn tùy ý. Nó là hiệp phương sai nghịch đảo của phép đo, và nó có nghĩa xác suất: bạn đang khai báo phép đo này sai cỡ bao nhiêu [99].

Đây là chỗ phân biệt quan trọng nhất của cả chương. Nó cũng khớp với nguyên tắc bạn đã tự đặt ra là từ chối các phương pháp kiểu tinh chỉnh. Đối Σ bằng tay cho tới khi APE đẹp là tinh chỉnh. Suy Σ từ một mô hình sai số rồi kiểm lại bằng số là ước lượng. Hai việc có thể cho cùng một con số, nhưng chúng khác nhau hoàn toàn về chỗ sẽ hỏng. Con số tinh chỉnh chỉ đúng trên tập bạn đã tinh chỉnh. Con số suy từ mô hình đúng ở mọi chỗ mô hình còn đúng, và bạn biết mô hình đó giả định gì.

Hình 8.1 cho thấy chỗ mỗi ma trận thông tin nằm trong đồ thị, và vì sao chỉ tỉ lệ giữa chúng mới có tác dụng.



Hình 8.1. Ma trận thông tin nằm ở đâu trong một đồ thị nhân tử stereo-inertial. Mỗi cạnh IMU mang Ω_{imu} , mỗi phép chiếu mang $\Omega_{\text{ảnh}}$. Vì nghiệm là cân bằng của cả hệ, chỉ *tỉ số* giữa hai nhóm độ cứng mới đổi kết quả. Hệ quả thực dụng: “siết trọng số IMU” và “nới trọng số ảnh” là cùng một thao tác, nên không sửa được cái này mà không nghĩ về cái kia.

B. Ví dụ nhỏ nhất tính tay được

Lấy bài toán một chiều. Hai phép đo cùng một đại lượng x . IMU nói $x = 1,0$ với $\sigma_{\text{imu}} = 0,1$. Ảnh nói $x = 0,0$ với $\sigma_{\text{ảnh}} = 1,0$. Nghiệm hợp nhất là trung bình có trọng số theo nghịch đảo phương sai:

$$\hat{x} = \frac{\Omega_{\text{imu}} \cdot 1,0 + \Omega_{\text{ảnh}} \cdot 0,0}{\Omega_{\text{imu}} + \Omega_{\text{ảnh}}} = \frac{100 \cdot 1 + 1 \cdot 0}{100 + 1} = 0,990.$$

Ảnh chỉ kéo được nghiệm đi 1%. Bây giờ giả sử σ_{imu} thật là 1,0 chứ không phải 0,1; datasheet đã lạc quan mười lần. Nghiệm đúng khi đó là 0,5. Sai số do khai nhầm trọng số là 0,49, tức gần bằng toàn bộ khoảng cách giữa hai phép đo.

Đây là con số đáng nhớ nhất của mục này. Sai một bậc về σ không làm nghiệm lệch một chút. Nó làm nghiệm bỏ gần hết một trong hai nguồn thông tin. Vì Ω đi với bình phương của σ , sai mười lần về độ lệch chuẩn là sai một trăm lần về trọng số.

C. Hai kiểu hỏng đối xứng

Đặt sai trọng số hỏng theo đúng hai kiểu, và chúng đối xứng nhau.

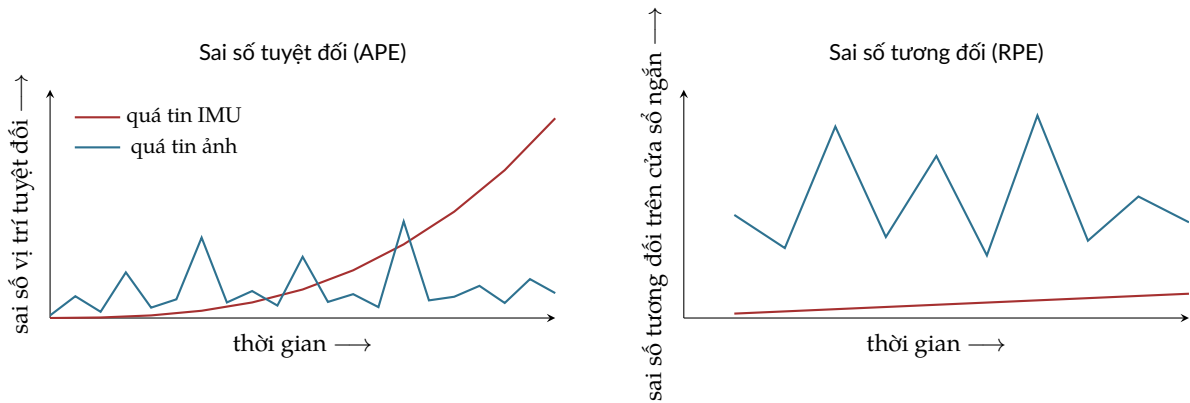
Quá tin IMU nghĩa là Σ_{imu} khai quá nhỏ. Bộ tối ưu không còn được phép sửa phép đo quán tính. Nó hấp thụ mọi mâu thuẫn vào chỗ khác: vào vị trí điểm bản đồ, vào tỉ lệ, vào độ lệch cảm biến. Cơ chế trôi thì thuần vật lý. Một độ lệch con quay ϵ tạo sai số góc ϵt . Sai số góc làm vectơ trọng lực bị nghiêng, nên một phần của g rò vào gia tốc ngang, cỡ $g\epsilon t$. Tích phân hai lần cho sai số vị trí

$$\Delta p(t) \approx \frac{1}{6} g \epsilon t^3.$$

Giả sử $\epsilon = 0,01$ rad/s và $t = 10$ s — một phép tính giả định để thấy *bậc* của công thức, không phải số đo của hệ nào: $\frac{1}{6} \cdot 9,81 \cdot 0,01 \cdot 1000 \approx 16$ m. Trong một hệ có ảnh, thị giác chặn phần lớn mức trôi đó. Nhưng *hình dạng* vẫn giữ nguyên: mượt, đơn điệu, tăng theo lũy thừa của thời gian.

Quá tin ảnh nghĩa là $\Sigma_{ảnh}$ khai quá nhỏ. Mỗi khung hình xấu có toàn quyền kéo nghiệm. Trên khung mờ, phần dư phép chiếu không phải nhiễu trắng mà là một *độ lệch có hệ thống*, vì đỉnh cực trị của điểm đặc trưng dịch dọc theo vệt mờ. Nghiệm giật theo. Rồi phần giật đó chảy ngược vào ước lượng độ lệch cảm biến qua nhân tử tiền tích phân, làm hỏng độ lệch, làm dự đoán khung sau tệ hơn. Đó là một vòng phản hồi dương. Nó là cơ chế đứng sau phần lớn các lần mất bám ở đoạn quay nhanh.

Hình 8.2 cho thấy hai chữ ký khác nhau. Quan trọng hơn, nó cho thấy phép đo tách được chúng.



Hình 8.2. Hai kiểu hỏng đối xứng khi đặt sai trọng số. Trục cố ý không có nhãn số: hình này nói về *hình dạng* của hai đường, không về độ lớn của sai số, và độ lớn thì bạn phải tự đo. Quá tin IMU cho một đường trôi mượt và đơn điệu: APE xấu dần trong khi RPE trên cửa sổ ngắn vẫn rất đẹp. Quá tin ảnh cho một đường răng cưa bị chặn: APE có thể trông chấp nhận được trong khi RPE ngắn hạn xấu ở mọi chỗ. Vì thế *tỉ số giữa RPE ngắn hạn và APE* tách được hai kiểu hỏng này, còn con số APE tổng thì không.

Kết luận chẩn đoán rút ra từ hình này đáng ghi lại. Báo APE một mình không phân biệt được hai bệnh ngược nhau. Báo thêm RPE ở vài độ dài cửa sổ thì phân biệt được ngay, và đó là lý do các hướng dẫn đánh giá quỹ đạo đòi cả hai [170].

D. Vì sao hằng số lấy từ datasheet gần như luôn sai

Datasheet không nói dối. Nó trả lời một câu hỏi khác với câu hỏi bạn đang hỏi. Có năm lý do, và chúng cộng dồn.

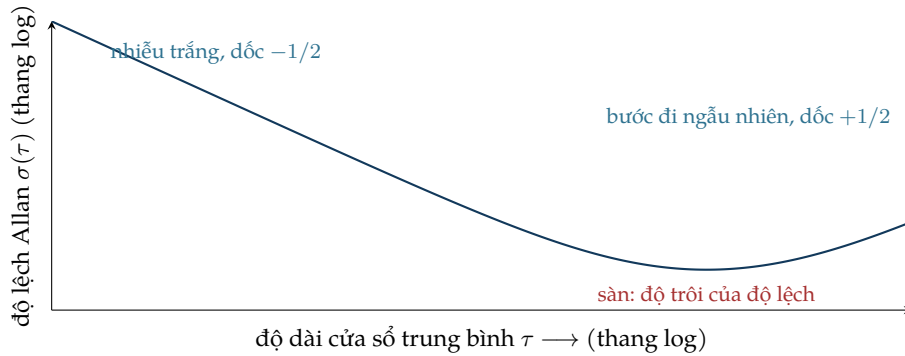
- **Con số datasheet là của con chip, không phải của thiết bị.** Nó đo trên đế thử nghiệm, nhiệt độ ổn định, không rung, và thường là giá trị điển hình chứ không phải chặn trên. Thiết bị của bạn có bo mạch, giá đỡ, động cơ và một vỏ máy.
- **Cái đồ thị nhân tử cần là một σ hiệu dụng.** Nó không phải nhiễu trắng của cảm biến. Nó phải nuốt hết mọi thứ mô hình không tả: sai lệch thời gian t_d giữa camera và IMU, sai số hiệu chỉnh ngoại, sai số hệ số tỉ lệ và lệch trục, sai số rời rạc hoá của chính phép tiền tích phân. Vì vậy giá trị đúng luôn *lớn hơn* datasheet, thường là vài lần. Đây là lý do các hệ thực dụng như VINS-Mono khuyến nghị nhân hệ số nhiễu lên so với giá trị đo được [171].

- **Nhiệt độ.** Độ lệch và hệ số tỉ lệ của con quay MEMS phụ thuộc nhiệt độ khá mạnh. Thiết bị nóng dần trong lúc chạy, nên độ lệch *trôi có hướng*. Mô hình bước đi ngẫu nhiên trong tiền tích phân giả định trôi không hướng, nên nó mô tả sai đúng cái đang xảy ra.
- **Rung.** Động cơ và cánh quạt bơm năng lượng vào dải tần cao. Mọi thành phần trên tần số Nyquist của bộ lấy mẫu bị gập xuống dải đo. Nó trông như nhiễu trắng nhưng không phải. Cùng một IMU trên bàn và trên drone đang bay có hai mức nhiễu hiệu dụng khác hẳn nhau.
- **Tốc độ chuyển động.** Ở tốc độ góc thấp, nhiễu trắng chi phối. Ở tốc độ góc cao, sai số hệ số tỉ lệ chi phối, và sai số đó tỉ lệ với ω chứ không phải hằng số. Nói cách khác, σ đúng cho lúc đứng yên và σ đúng cho lúc quay 200 độ mỗi giây không thể là cùng một số.

Điểm cuối giết chết ý tưởng “tìm một hằng số tốt”. Không tồn tại một hằng số đúng cho cả hai chế độ. Đó cũng là lý do vì sao khi bạn thử vài giá trị, mỗi giá trị lại thắng ở một chuỗi khác nhau: bạn đang tìm một hằng số cho một đại lượng không hằng.

CẠM BẦY

Phương sai Allan là công cụ đúng và bị dùng sai thường xuyên. Nó đo lúc cảm biến *đứng yên*, trong nhiều giờ, ở nhiệt độ ổn định. Đó chính xác là chế độ bạn không vận hành. Số Allan cho bạn một *sàn*: hệ thống không thể tốt hơn thế. Nó không cho bạn giá trị hiệu dụng lúc bay. Dùng thẳng số Allan làm Σ trong đồ thị nhân tử là khai quá tự tin một cách có hệ thống, và luôn theo cùng một hướng. Nên chạy thêm chuỗi không phát hiện ra nó.



Hình 8.3. Hình dạng của một đường Allan cho con quay MEMS. Trục không có nhãn số vì hình này nói về *ba đoạn dốc*, không về giá trị của một cảm biến cụ thể — giá trị thì bạn đo trên chính con quay của mình. Ba đoạn ứng với ba tham số mà tiền tích phân cần: mật độ nhiễu trắng ở dốc trái, độ trôi của độ lệch ở đáy, bước đi ngẫu nhiên của độ lệch ở dốc phải. Cả ba đo lúc *đứng yên*, nên chúng là sàn chứ không phải giá trị vận hành.

E. Phép thử biến câu hỏi thành con số

Trước khi nghĩ tới một mạng, hãy đo xem Σ hiện tại sai bao nhiêu. Phép thử là NEES, và nó rẻ.

Nếu Σ trung thực thì đại lượng $r^T \Sigma^{-1} r$ có kỳ vọng đúng bằng số bậc tự do d của phần dư. Với phần dư phép chiếu hai chiều, $d = 2$. Cách làm: ghi lại mọi phần dư ở cuối một lần tối ưu, tính trung bình của $r^T \Sigma^{-1} r$ theo từng loại nhân tử, rồi so với d .

Đọc kết quả thì gọn nhất là viết ra công thức chứ không thay số. Đặt

$$k = \frac{r^T \Sigma^{-1} r}{d},$$

thì k chính là hệ số mà Σ bạn khai đang sai, tính theo *phương sai*; sai theo *độ lệch chuẩn* là $\sqrt{k}$. Ba trường hợp và chỉ ba: $k \approx 1$ là trung thực; $k > 1$ nghĩa là bạn tự tin hơn mức được phép đúng $\sqrt{k}$

lần; $k < 1$ nghĩa là bạn quá dè dặt và đang vứt đi thông tin. Không cần một con số cụ thể nào để phát biểu điều đó, và đó chính là lý do phép thử này chuyển được từ hệ này sang hệ khác. Với nhân tử tiền tích phân, d là 9 cho phần xoay, vận tốc và vị trí, hoặc 15 nếu tính cả hai độ lệch.

Ba lý do khiến phép thử này quan trọng hơn về ngoài của nó. Thứ nhất, nó là *phép đo*, không phải tính chỉnh: bạn không dò tìm giá trị làm APE đẹp, bạn kiểm một mệnh đề thống kê. Thứ hai, nó tách được theo nhóm: NEES của nhân tử ảnh, của con quay, của gia tốc kể là ba con số riêng, nên bạn biết chỗ nào sai. Thứ ba, nó chạy được trên chính chuỗi bạn đang thất bại, và không cần chân trị.

Một tình tế phải nói rõ. Sau khi tối ưu, phần dư đã bị co lại vì chính nó tham gia định ra nghiệm. Nên NEES tính trên phần dư sau tối ưu thiên về phía *nhỏ hơn* d . Cách sạch hơn là tính trên phần dư *dự đoán*: đánh giá nhân tử của khung mới bằng trạng thái ước lượng từ các khung trước, trước khi khung mới tham gia tối ưu. Nếu bạn chỉ làm được cách thứ nhất, hãy nhớ rằng con số bạn thấy đã là bản lạc quan. Một NEES lớn hơn d khi đó là bằng chứng còn mạnh hơn.

Để thấy một bảng NEES đọc như thế nào, Bảng 8.1 viết ra bốn tình huống theo hệ số k . Giá trị của bảng nằm ở chỗ bốn dòng dẫn tới bốn kết luận khác nhau, và không dòng nào cần một con số cụ thể mới đọc được.

Bảng 8.1. Bốn cách một bảng NEES đọc ra, viết theo hệ số $k = \overline{r^T \Sigma^{-1} r} / d$ chứ không theo con số cụ thể. Bảng cố ý không có số: con số phải đến từ chính lần chạy của bạn, còn cái chuyển được từ hệ này sang hệ khác là bốn dòng dưới đây. Không dòng nào đọc được nếu thiếu bậc tự do d .

Điều bạn thấy	Nghĩa là	Làm gì tiếp
$k \approx 1$ cho một nhóm nhân tử	Σ của nhóm đó trung thực	Không đụng tới. Ghi lại làm mốc để so khi đổi cấu hình
$k > 1$, và lớn dần theo tầng kim tự tháp	Hệ số trọng số theo tầng đang quá lạc quan ở tầng cao	Sửa <i>hình dạng</i> của hàm phụ thuộc tầng, đừng nhân một hằng số chung cho mọi tầng
$k > 1$ trên nhân tử con quay	Hoặc σ khai quá nhỏ, hoặc phần dư có một thành phần <i>hệ thống</i>	Loại trừ sai lệch thời gian t_d và sai số hiệu chỉnh ngoại trước; chỉ khi hết cả hai mới được kết luận là nhiễu
$k < 1$ trên nhân tử gia tốc kể	Quá dè dặt: bạn đang vứt đi thông tin gia tốc	Siết lại theo đúng hệ số $\sqrt{k}$; hệ quả quan sát được là tỉ lệ đơn ảnh hội tụ nhanh hơn

Chú ý dòng cuối. Quá dè dặt ít khi làm hệ sập, nên nó gần như không bao giờ bị phát hiện bằng cách nhìn quỹ đạo. Nó chỉ hiện ra khi bạn tính k cho từng nhóm. Đó là lý do phép thử đáng chạy cả khi hệ đang chạy tốt.

I Năm nấc của việc đặt trọng số IMU

1 Vấn đề

Không có số nào cả.

Giải pháp

lấy hằng số từ datasheet.

Giả định

cảm biến hoạt động như trong phòng thí nghiệm.

Giá phải trả

mọi sai lệch chưa mô hình hoá bị dồn vào chỗ khác, và luôn theo hướng quá tự tin.

2 Vấn đề

Datasheet không khớp thiết bị thật.

Giải pháp

đo phương sai Allan trên chính thiết bị, nhiều giờ, đứng yên.

Hệ quả

bạn có ba tham số đúng cho một chế độ.

Vấn đề mới

chế độ đó là chế độ đứng yên, không phải chế độ bay.

3 Vấn đề

Độ lệch trôi theo nhiệt và theo thời gian.

- Giải pháp** đưa độ lệch vào vectơ trạng thái và ước lượng nó trực tuyến, đúng như ORB-SLAM3 và VINS-Mono làm [1], [171].
- Giả định** trôi là bước đi ngẫu nhiên không hướng.
- Giá phải trả** bạn phải chọn một tham số bước đi ngẫu nhiên, và nó lại là một hằng số nữa.
- 4 Vấn đề** Nhiều hiệu dụng đổi theo chuyển động và theo rung.
- Giải pháp** luật thích nghi: nói Σ khi phát hiện tốc độ góc cao hoặc rung.
- Giả định** bạn viết đúng được hàm phụ thuộc đó bằng tay.
- Giá phải trả** đây là tinh chỉnh trá hình nếu hàm được chọn theo kết quả; kinh nghiệm thực hành trên EuRoC cho thấy một hằng số chặt, hiệu chỉnh đúng, thường không thua các luật thích nghi viết tay.
- 5 Vấn đề** Không viết được hàm phụ thuộc đó bằng tay.
- Giải pháp** học nó: một mạng nhận cửa sổ IMU thô và trả về cả phép đo lẫn hiệp phương sai của phép đo.
- Giả định** phân phối chuyển động lúc chạy giống lúc huấn luyện.
- Giá phải trả** mất tính chẩn đoán, thêm một tạo tác phải bảo trì, và một chế độ hỏng mới là tự tin sai khi ra ngoài phân phối.

Bảng 8.2. Năm cách đặt trọng số IMU. Cột cuối là cột đáng giá nhất: nó nói khi nào con số bạn khai không còn nghĩa.

Cách	Cơ chế	Mạnh khi nào	Hỏng khi nào
Hằng số datasheet	Đọc mật độ nhiễu từ tài liệu chip	Bản dựng đầu tiên, cần một con số để chạy được	Gần như luôn quá tự tin; sai theo cùng một hướng nên lặp lại nhiều lần không lộ
Hiệu chỉnh Allan	Đo nhiều giờ lúc đứng yên, tách ba tham số	Thiết bị cố định, ít rung, nhiệt ổn định	Chế độ vận hành có rung hoặc quay nhanh; số Allan là sà n chứ không phải giá trị lúc chạy
Ước lượng độ lệch trực tuyến	Đưa độ lệch vào trạng thái, ràng buộc bằng bước đi ngẫu nhiên	Mọi hệ thị giác-quán tính hiện đại; bất được trôi chậm	Trôi có hướng theo nhiệt; giai đoạn khởi tạo khi chưa đủ quan sát để tách độ lệch khỏi trọng lực
Luật thích nghi viết tay	Nói Σ theo một chỉ báo như tốc độ góc hay mức rung	Khi bạn có mô hình vật lý thật cho phần phụ thuộc đó	Khi hàm được chọn theo kết quả: đó là tinh chỉnh và nó không chuyển sang tập khác
Hiệp phương sai học được	Mạng xuất cả phép đo lẫn Σ , huấn luyện bằng độ hợp lý	Có dữ liệu đúng nền tảng và đúng phân phối chuyển động	Ra ngoài phân phối: mạng vẫn khai Σ nhỏ, tức tự tin sai — chế độ hỏng tệ nhất cho một trọng số

↔ cùng nguyên lý với E/24 — NEES cũng là một phép “đo sà n trước, kết luận sau”: trước khi so hai cấu hình trọng số, hãy kiểm xem trọng số hiện tại có trung thực không

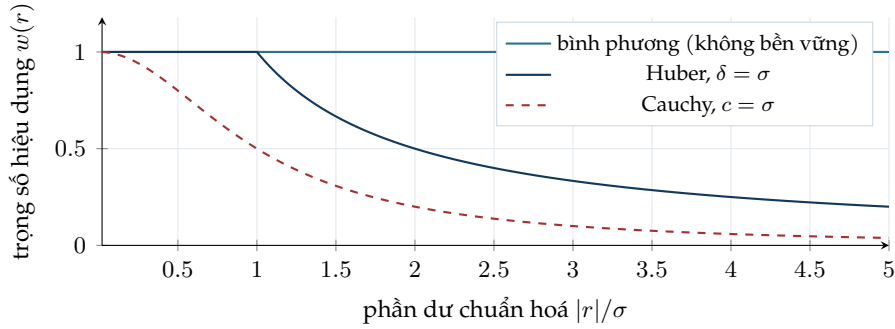
F. Hàm mất mát bền vững là một trọng số thích nghi trá hình

Còn một chỗ mà trọng số đã bị thay đổi trong hệ của bạn mà ít ai gọi tên như vậy: hàm mất mát bền vững. Huber và Cauchy được dùng ở khắp nơi trong SLAM để chống điểm ngoại lai, và cơ chế của chúng đúng là đổi trọng số.

Cách nhìn gọn nhất là bình phương tối thiểu có trọng số lặp lại. Một hàm mất mát bền vững $\rho(r)$ tương đương với bình phương tối thiểu thường, nhưng mỗi phần dư mang thêm một hệ số

$$w(r) = \frac{\rho'(r)}{r}.$$

Với chuẩn bình phương thì $w \equiv 1$. Với Huber ngưỡng δ , $w = 1$ khi $|r| \leq \delta$, và $w = \delta/|r|$ khi ra ngoài. Tính tay một trường hợp: phần dư bằng 3δ thì trọng số hiệu dụng chỉ còn một phần ba, tức ma trận thông tin của ràng buộc đó bị chia ba. Với Cauchy tham số c , $w = 1/(1 + (r/c)^2)$, nên cùng phần dư $3c$ chỉ còn một phần mười.



Hình 8.4. Hàm mất mát bền vững nhìn dưới dạng trọng số. Đường cong là $w(r) = \rho'(r)/r$, tức hệ số nhân vào ma trận thông tin của một ràng buộc khi phần dư của nó lớn. Đây là *công thức*, không phải số đo. Điểm cần thấy: một hàm bền vững cũng là cách đặt trọng số thích nghi, nhưng nó thích nghi theo *phần dư* chứ không theo *đầu vào*.

Hình 8.4 cho thấy điểm phân biệt quan trọng nhất của mục này. Hàm bền vững phản ứng *sau khi* đã thấy phần dư. Một hiệp phương sai học được, hay một Σ suy từ vệt mờ, dự đoán *trước khi* thấy phần dư. Hai thứ không thay thế nhau, và lý do rất cụ thể. Phần dư lớn có thể do phép đo tồi, mà cũng có thể do trạng thái hiện tại đang sai. Hàm bền vững không phân biệt được hai nguyên nhân đó; nó hạ trọng số trong cả hai trường hợp.

Hệ quả rơi thẳng ra là một cạm bẫy hay gặp. Đặt hàm bền vững lên cạnh IMU thường sai, và sai theo hướng nguy hiểm. Phần dư IMU lớn hiếm khi có nghĩa là phép đo quán tính bị ngoại lai. Nó thường có nghĩa là tư thế hoặc độ lệch đang sai, tức đúng lúc bạn cần ràng buộc IMU kéo mạnh nhất. Hạ trọng số ở đúng lúc đó là giấu triệu chứng đi. Với cạnh ảnh thì ngược lại: sai ghép là chuyện thường ngày, nên hàm bền vững ở đó là công cụ đúng.

Tóm lại, ba tầng nên tách bạch. Tầng một là Σ trung thực, suy từ mô hình và kiểm bằng NEES. Tầng hai là hàm bền vững, dành cho các loại ràng buộc thật sự có ngoại lai. Tầng ba là loại bỏ hẳn bằng kiểm chứng hình học. Trộn ba tầng — ví dụ nối Σ để bù cho việc thiếu bước loại bỏ — là cách phổ biến nhất để một hệ trở nên không chẩn đoán được.

8.3 Học nhiễu và bất định IMU (Learned Noise and Learned Uncertainty)

A. Ý trung tâm của cả chương

Đọc các công trình trong mục này rất dễ rút ra kết luận sai. Kết luận sai là: mạng nơ-ron ước lượng tư thế tốt hơn hình học. Nó không tốt hơn. Trên dữ liệu có ảnh tốt và hiệu chỉnh tốt, một bộ ước lượng hình học viết cẩn thận vẫn thắng, và thắng với chi phí thấp hơn nhiều.

Kết luận đúng là: mạng ước lượng *độ tin cậy* tốt hơn. Nó đọc được từ đầu vào những thứ mà một hằng số không đọc được. Ảnh này có mờ không, đoạn IMU này có rung không, điểm khớp này

nằm trên cạnh hay trên góc. Và ước lượng độ tin cậy chính là ô trống trong hệ cổ điển, vì hệ cổ điển lấy nó từ datasheet và từ tầng kim tự tháp.

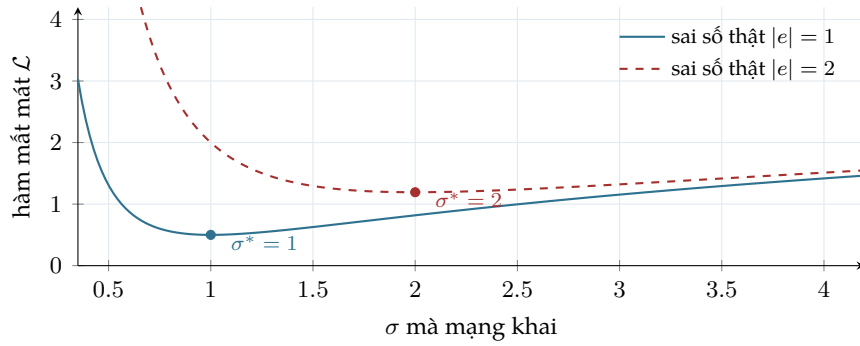
Cơ chế làm được điều đó gọn tới bất ngờ, và đáng suy dẫn ra vì nó nói rõ mạng thực sự học cái gì. Cho mạng xuất ra cả giá trị dự đoán μ lẫn phương sai σ^2 , rồi huấn luyện bằng độ hợp lý âm của phân phối chuẩn [172]:

$$\mathcal{L} = \frac{1}{2} \left(\frac{(y - \mu)^2}{\sigma^2} + \log \sigma^2 \right).$$

Số hạng thứ nhất phạt việc tự tin mà sai. Số hạng thứ hai phạt việc khai bất định lớn để né. Đặt $e = y - \mu$ rồi lấy đạo hàm theo σ^2 , cực tiểu nằm ở

$$\sigma^{2*} = e^2.$$

Nói bằng lời: đầu ra tối ưu của nhánh phương sai là *bình phương sai số kỳ vọng*, có điều kiện theo đầu vào. Một hiệp phương sai học được không phải phép màu. Nó là một bài hồi quy bình phương sai số lên chính đầu vào. Hình 8.5 cho thấy hình dạng của hàm mất mát đó và vì sao nó có đúng một cực tiểu.



Hình 8.5. Vì sao độ hợp lý âm ép mạng nói thật. Nhánh trái dốc đứng: khai σ quá nhỏ bị phạt bởi số hạng bình phương. Nhánh phải đi lên chậm: khai σ quá lớn bị phạt bởi $\log \sigma^2$. Cực tiểu nằm đúng tại $\sigma = |e|$, nên nhánh phương sai của mạng chỉ đang hồi quy sai số kỳ vọng theo đầu vào — không hơn, và cũng không kém.

Hai hệ quả rơi thẳng ra từ suy dẫn này, và cả hai đều là giới hạn.

- **Mạng chỉ biết loại sai số nó đã thấy.** Bất định học được theo cách này là bất định của dữ liệu, tức phần nhiễu vốn có trong đầu vào. Nó không phải bất định của mô hình. Gặp một đầu vào lạ, mạng vẫn nội suy và vẫn khai một σ nhỏ. Đó là tự tin sai, và với một trọng số thì đây là chế độ hổng tệ nhất có thể [173].
- **Muốn bất phần bất định của mô hình thì phải trả thêm.** Cách rẻ nhất là hợp nhiều mô hình rồi lấy độ tán mát giữa chúng làm bất định [174], hoặc lấy mẫu bằng dropout lúc suy luận [175]. Cả hai nhân chi phí suy luận lên vài lần, nên với một hệ thời gian thực trên Orin, chúng gần như luôn ngoài ngân sách.

B. Bốn công trình về phía IMU

Bốn công trình dưới đây được xếp theo mức độ can thiệp vào hệ của bạn, từ nhẹ tới nặng. Với mỗi cái, câu hỏi đúng không phải “nó chính xác bao nhiêu” mà là bốn câu của Phần F: nó chữa thất bại nào, cơ chế gì, có đáng không, và học được gì kể cả khi không dùng.

Khử nhiễu con quay bằng mạng tích chập giãn

Đây là can thiệp nhẹ nhất [176]. Thất bại được chữa: tư thế góc ước lượng vòng hở từ con quay MEMS trôi rất nhanh, vì độ lệch, sai số hệ số tỉ lệ và lệch trục cộng dồn.

Cơ chế: một mạng tích chập giãn nhận cửa sổ dữ liệu IMU thô và xuất ra một *lượng hiệu chỉnh* cộng vào phép đo con quay. Hàm mất mát không đặt trên từng mẫu. Nó đặt trên số gia xoay tích lũy qua nhiều độ dài cửa sổ, tính trên nhóm xoay.

Hai chi tiết thiết kế đáng lấy dù bạn không dùng phương pháp. Thứ nhất, mạng sửa *phép đo* chứ không sửa *trạng thái*, nên toàn bộ phần tiền tích phân phía sau không phải đưng tới. Thứ hai, hàm mất mát đặt trên đúng đại lượng mà đồ thị nhân tử tiêu thụ, chứ không phải trên đại lượng dễ đo nhất. Tích chập giãn cho trường tiếp nhận dài với ít phép tính, và điều đó quan trọng khi tần số mẫu là 200 Hz.

Chi phí: đây là mạng nhỏ nhất trong cả chương — một chồng tích chập một chiều trên một cửa sổ tín hiệu, không phải một mô hình ảnh — nên nó là ứng viên duy nhất trong mục này có cửa ngay trên luồng theo vết. Nhưng “có cửa” không phải “vừa”; đại lượng duy nhất đáng tin là mức thay đổi phân vị cao của chu kỳ luồng theo vết khi bạn bật nó lên trong hệ đầy đủ. Hồng khi nào: nó được hiệu chỉnh cho một đơn vị IMU cụ thể và một phân phối chuyển động cụ thể. Đổi thiết bị thì lượng hiệu chỉnh sai, và sai một cách im lặng.

TLIO

TLIO can thiệp ở mức vừa [20]. Thất bại được chữa: định vị chỉ bằng quán tính trên thiết bị đeo trôi trong vài giây, nên không dùng được khi mất ảnh.

Cơ chế: một mạng nhận cửa sổ IMU đã căn theo trọng lực và hồi quy một vectơ dịch chuyển ba chiều *cùng với hiệp phương sai của nó*. Cặp giá trị đó được đưa vào một bộ lọc như một phép đo giả.

Đây là bài học thiết kế quan trọng nhất của cả mục: *mạng không phải bộ ước lượng, bộ lọc mới là*. Mạng chỉ cung cấp một phép đo kèm mức tin cậy đã khai báo. Trạng thái — độ lệch, vận tốc, hướng trọng lực — vẫn do bộ lọc giữ. Vì thế nó cắm được vào một đồ thị nhân tử có sẵn dưới dạng một nhân tử mới, không đưng gì tới phần còn lại.

Có đáng không, với trường hợp của bạn: nó chữa một nỗi đau bạn không có, vì bạn luôn có stereo. Nhưng *khuôn* của nó là thứ đáng bắt chước nhất trong toàn chương.

RoNIN

RoNIN đi xa hơn về phía dữ liệu [177]. Nó hồi quy vận tốc trong một hệ quy chiếu không phụ thuộc hướng thiết bị, chỉ từ IMU, và đi kèm một bộ dữ liệu chuẩn để so sánh.

Điều đáng học không phải con số mà là nguồn gốc của độ chính xác. Nó đến từ một *tiên nghiệm chuyển động* rất mạnh: đáng đi của người có cấu trúc lặp lại, và mạng học chính cấu trúc đó. Đây vừa là lý do nó chạy được vừa là lý do nó không chuyển sang nền tảng khác. Một drone hay một xe không có tiên nghiệm tương đương. Nếu chuyển động của robot bạn không bị ràng buộc, đây là hướng sai.

IDOL

IDOL tách bài toán làm hai tầng [178]: học hướng trước, rồi học vị trí trong hệ quy chiếu đó. Bài học thiết kế khớp thẳng với phép tính $\frac{1}{6}get^3$ ở mục trước. Sai số xoay là thứ đầu độc sai số tịnh tiến, nên giải nó trước và giải riêng là hợp lý về mặt cấu trúc. Mức tin cậy của tôi về chi tiết hiện thực của công trình này chỉ ở mức vừa, nên tôi nêu ý tưởng và không mô tả kết quả số.

Ngoài bốn cái trên, có một mảnh bằng chứng đáng để ý cho người làm chuyển động mạnh. Hướng học đo quán tính cho đưa drone cho thấy khuôn “phép đo học được cộng hiệp phương sai, đưa vào bộ ước lượng cổ điển” vẫn hoạt động ở chế độ chuyển động rất hung hãn [179]. Điều kiện là phân phối chuyển động lúc huấn luyện khớp lúc chạy. Điều kiện đó không phải chi tiết nhỏ; nó là toàn bộ vấn đề.

C. MAC-VO: bất định theo từng điểm khớp

Phía ảnh có một ô trống lớn hơn phía IMU, và ít người nói ra. Trong ORB-SLAM3, trọng số của một phần dư phép chiếu là một hằng số nhân theo hệ số tỉ lệ của tầng kim tự tháp mà điểm đặc trưng được phát hiện [1]. Đó là một đại diện cho *độ phân giải*. Nó không biết gì về mờ, về vùng ít vân, về hoa văn lặp lại, về vật thể đang chuyển động, hay về việc điểm khớp thứ hai gần bằng điểm khớp thứ nhất tới mức nào.

MAC-VO lấp đúng ô đó [106]. Thất bại được chữa: mọi điểm khớp bị đối xử như nhau, nên một nhóm nhỏ điểm khớp tồi có thể kéo lệch cả tư thế.

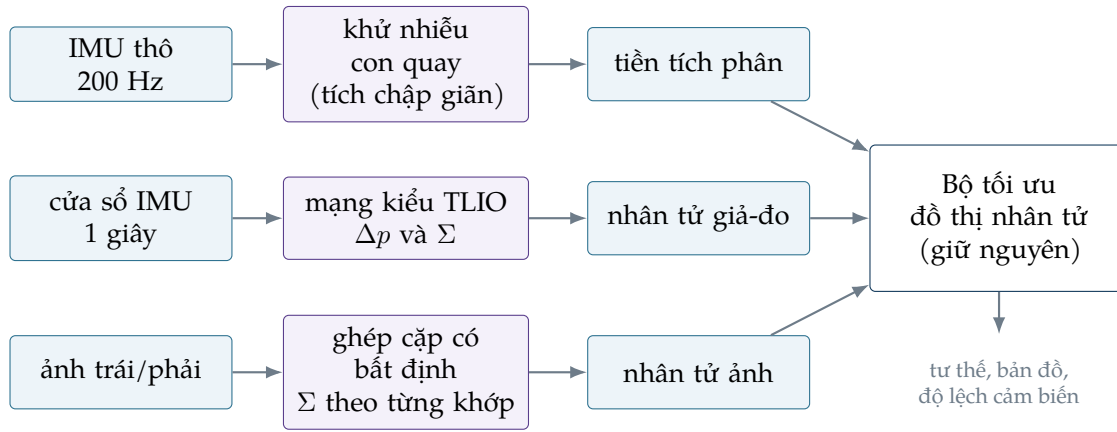
Cơ chế: bộ ghép cặp học được xuất ra bất định của chính nó cho từng điểm khớp. Bất định trong mặt phẳng ảnh được đưa qua hình học phép chiếu để thành một hiệp phương sai ba chiều đúng nghĩa, dẹt và dài dọc theo tia nhìn với điểm ở xa. Hiệp phương sai đó trực tiếp làm trọng số cho từng ràng buộc ảnh. Đây là ý “bất định phải mang đơn vị đo đúng”. Một sai số hai điểm ảnh tương ứng với sai số độ sâu rất khác nhau tùy điểm gần hay xa, và một trọng số vô hướng trong mặt phẳng ảnh không tài nào biểu diễn được điều đó.

Có đáng làm không, xét theo bốn cổng của F/27. Chi phí: đây là cả một frontend học được, không phải một mô-đun cắm thêm. Nó chiếm GPU, và GPU của bạn đang bận trích ORB và chạy hiệu chỉnh chùm tia cục bộ. Dữ liệu: nó cần dữ liệu có chân trị độ sâu hoặc mô phỏng. Hồng khi ra ngoài phân phối: giống mọi bất định học được, mạng vẫn khai bất định nhỏ trên cảnh lạ. Kết luận trung thực tính tới tháng 9/2026 là chưa nên thay frontend.

Nhưng bài học thiết kế thì rút được ngay và gần như miễn phí: **hãy làm cho ma trận thông tin của ảnh phụ thuộc vào một đại lượng đo được của từng điểm khớp**. Bạn không cần một mạng để làm điều đó. Ba đại diện rẻ, tính được ngay trong lúc trích đặc trưng, đều tốt hơn tầng kim tự tháp:

- **Khoảng cách tới điểm khớp tốt thứ hai.** Tỉ số giữa khoảng cách mô tả tốt nhất và tốt nhì đã là một đo lường mơ hồ, và nó có sẵn trong mọi bộ ghép cặp.
- **Năng lượng gradient cục bộ và tính dị hướng của nó.** Ma trận cấu trúc 2×2 quanh điểm đặc trưng cho ngay hai trị riêng. Tỉ số giữa chúng nói điểm đó bị ràng buộc theo một phương hay hai phương. Đây đúng là thứ dùng để dựng một Σ dị hướng.
- **Vệt mờ dự đoán từ con quay.** Mục sau tính con số này; nó cho cả hướng lẫn độ dài của trục dài trong ellipse bất định.

Và ba lựa chọn ấy kiểm chứng được bằng NEES, theo đúng cách ở mục trước. Chia điểm khớp thành các nhóm theo tỉ số khoảng cách mô tả rồi tính NEES riêng từng nhóm, bạn sẽ thấy ngay nhóm nào đang bị khai quá tự tin. Đó là bằng chứng, không phải tinh chỉnh.



■ mạng ■ khối cổ điển giữ nguyên

Cả ba chỗ cắm đều *cung cấp phép đo kèm Σ* , không thay bộ ước lượng. Đó là lý do chúng thử được từng cái một, và bỏ ra được khi không đáng.

Hình 8.6. Ba chỗ mà một mạng cắm được vào một hệ thị giác-quán tính cổ điển mà không đụng bộ tối ưu. Trên: sửa phép đo con quay trước khi tiền tích phân. Giữa: thêm một nhân tử giả-đo dịch chuyển kèm hiệp phương sai, theo khuôn TLIO. Dưới: thay trọng số vô hướng của ảnh bằng hiệp phương sai theo từng điểm khớp, theo khuôn MAC-VO. Điểm chung là bộ ước lượng vẫn là bộ ước lượng cổ điển, nên mỗi chỗ cắm là một ablation độc lập.

Bảng 8.3. Năm công trình, đọc theo bốn câu hỏi của Phần F. Cột cuối là phần giữ được giá trị kể cả khi bạn kết luận là chưa đáng dùng.

Công trình	Chứa thất bại nào	Có đáng với hệ của bạn không	Bài học thiết kế giữ lại
Khử nhiễu con quay	Tư thế góc vòng hờ trôi vì độ lệch và sai số tỉ lệ	Có thể: mạng rất nhỏ, không đụng backend; nhưng phải hiệu chỉnh lại cho từng đơn vị IMU	Sửa <i>phép đo</i> , không sửa trạng thái; đặt hàm mất mát trên đúng đại lượng backend tiêu thụ
TLIO	Định vị chỉ bằng quán tính trôi trong vài giây	Không trực tiếp: bạn luôn có stereo	Mạng cấp phép đo kèm Σ ; bộ lọc vẫn là bộ ước lượng và vẫn giữ mọi trạng thái
RoNIN	Không có tư thế khi thiếu ảnh, trên thiết bị đeo	Không: tiên nghiệm đáng đi không tồn tại cho drone hay xe	Độ chính xác đến từ tiên nghiệm chuyển động; đó cũng là chỗ nó không chuyển được
IDOL	Sai số hướng đầu đọc sai số vị trí	Không trực tiếp	Tách hướng khỏi vị trí; xử lý xoay trước vì lỗi xoay khuếch đại theo t^3
MAC-VO	Mọi điểm khớp bị đối xử như nhau dù chất lượng rất khác	Chưa, tính tới 9/2026: cả một frontend, tranh GPU với ORB và hiệu chỉnh chùm tia	Cho $\Omega_{\text{ảnh}}$ phụ thuộc đại lượng đo được của từng khớp, và cho nó dị hướng

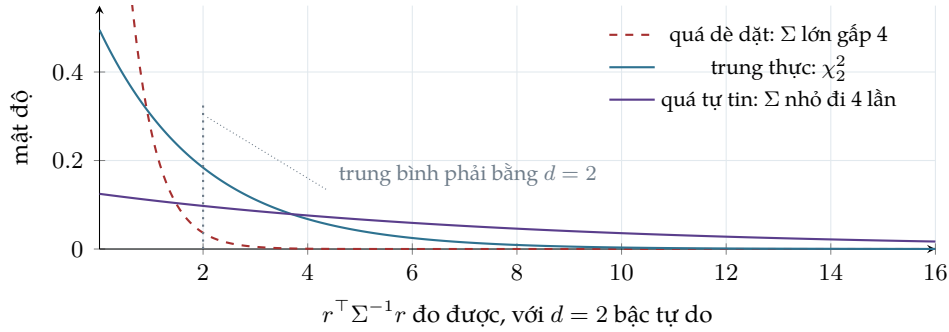
↔ **tiền đề F/28** — bất định theo từng điểm khớp chỉ có nghĩa khi bộ ghép cặp xuất được một điểm số hiệu chỉnh tốt; chương 28 là nơi bàn các bộ ghép cặp đó

D. Kiểm một bất định có trung thực không, kể cả khi nó do mạng sinh ra

Mục 2 đã giới thiệu NEES cho hằng số cổ điển. Phép thử đó áp nguyên vào một hiệp phương sai học được, và đây là điều kiện nghiệm thu bắt buộc trước khi cắm bất kỳ mạng nào vào bộ ước lượng.

Hình dạng của phép thử rất sạch về mặt lý thuyết. Nếu Σ khai đúng thì $r^T \Sigma^{-1} r$ theo phân phối χ^2 với d bậc tự do. Nếu Σ thật lớn hơn k lần thì đại lượng đo được theo phân phối $k \chi_d^2$. Nói cách

khác, cả phân bố dịch sang phải đúng k lần, và trung bình đi từ d lên $k d$. Vì vậy bạn không chỉ đọc được “sai hay đúng” mà đọc được luôn hệ số sai.



Hình 8.7. Đọc một biểu đồ NEES. Ba đường là mật độ *chính xác* của $k \chi_2^2$ với $k = 1/4, 1, 4$, không phải số minh họa. Trung bình đo được chia cho d chính là hệ số sai k về phương sai, tức $\sqrt{k}$ về độ lệch chuẩn. Đuôi phải dày bất thường mà trung bình vẫn gần d thì lại là chuyện khác: đó là dấu hiệu ngoại lai, và cách chữa là loại bỏ chứ không phải nói Σ .

Với một mạng, phép thử này còn cho thêm một thứ mà hằng số không có: bạn kiểm được *tính hiệu chỉnh theo từng mức*. Cách làm là chia các phép đo thành nhóm theo chính σ mà mạng khai, rồi tính NEES riêng cho từng nhóm. Một mạng trung thực cho NEES gần d ở *mọi* nhóm. Một mạng chỉ đúng trung bình thì sẽ lộ ra ngay: nó thường quá tự tin ở nhóm σ nhỏ và quá dè dặt ở nhóm σ lớn, tức nó đã học đúng thứ tự nhưng sai thang đo.

Phân biệt đó quan trọng vì hai bệnh có hai cách chữa khác nhau. Sai thang đo sửa được bằng một hệ số duy nhất tính từ dữ liệu kiểm định, và đó là hiệu chỉnh chứ không phải tinh chỉnh. Sai thứ tự thì không sửa được bằng bất kỳ hệ số nào; nó nghĩa là mạng không đọc được dấu hiệu cần đọc, và bạn phải quay lại phần dữ liệu hoặc đầu vào.

Cuối cùng, một phép thử rẻ mà nên chạy trước cả hai phép trên: cho mạng ăn dữ liệu ngoài phân phối một cách cố ý. Che một phần ảnh, tăng nhiễu, đảo tương phản, hoặc lấy một chuỗi từ thiết bị khác. Nếu σ mạng khai *không* tăng lên trong các trường hợp đó, bạn đã có bằng chứng trực tiếp cho chế độ hỏng tệ nhất, và bạn có nó trước khi đưa mạng vào hệ thống.

8.4 Ảnh mờ do chuyển động và chuyển động nhanh (Motion Blur and Fast Motion)

A. Trước khi bàn tới mạng: hãy tính vệt mờ ra pixel

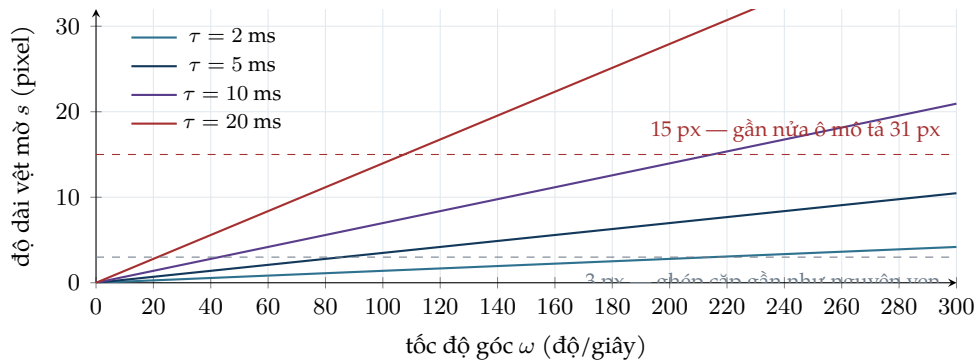
Ảnh mờ hay được nói tới như một điều kiện xấu mơ hồ. Nó là một đại lượng tính được, và tính nó ra trước là bước rẻ nhất trong cả mục.

Trong thời gian phơi sáng τ , camera quay một góc $\omega \tau$. Với tiêu cự f tính bằng pixel, độ dài vệt mờ do xoay xấp xỉ

$$s \approx f \omega \tau \quad (\text{pixel}).$$

Thay số vào để thấy bậc của quan hệ. Đây là phép tính *giả định* trên một camera giả định, không phải số đo: giả sử $f = 400$ pixel và $\tau = 10$ ms. Quay 30 độ mỗi giây cho $s \approx 2,1$ pixel, gần như vô hại. Quay 200 độ mỗi giây cho $s \approx 14$ pixel, trong khi ô mô tả của ORB rộng 31 pixel — một vệt cỡ đó phá gần hết cấu trúc bên trong ô. Điều đáng mang đi không phải hai con số mà là *quan hệ*: s tuyến tính theo cả ω lẫn τ , nên tốc độ quay tăng vài lần là đủ đưa vệt mờ từ vô hại sang phá huỷ. Với camera của bạn, thay f và τ thật vào rồi vẽ phân bố của s trên chuỗi thật — đó mới là con số dùng được.

Vệt do tịnh tiến thì phụ thuộc độ sâu: cùng một vận tốc, điểm gần trượt nhiều pixel hơn điểm xa. Chi tiết này sẽ quay lại ở mục sau, vì nó là lý do một nhân chấp duy nhất cho cả ảnh là mô hình sai.



Hình 8.8. Vệt mờ do xoay, tính bằng $s \approx f\omega\tau$ với một tiêu cự giả định $f = 400$ pixel. Đây là đồ thị của một công thức, không phải số đo: bốn đường chính xác theo mô hình, và chúng đối chỗ ngay khi bạn thay f của camera mình vào. Đọc hình này ngược lại cho đòn bẩy rõ nhất: s tuyến tính theo τ , nên hạ thời gian phơi sáng đi bốn lần thì chia độ dài vệt cho bốn, và cái giá chỉ là phải tăng độ khuếch đại.

Từ hình này rơi ra một kết luận cần nói trước mọi thứ khác. **Bộ điều khiển phơi sáng tự động là một phần của hệ SLAM của bạn, và hầu như không ai đối xử với nó như vậy.** Nó được chỉnh cho ảnh đẹp mắt, tức ưu tiên đủ sáng hơn là đủ ngắn. Đặt trần cho τ và bù bằng độ khuếch đại là một thay đổi vài dòng, không tốn thêm một phép tính nào, và nó tấn công thẳng nguyên nhân. Nhiều cảm biến tăng lên là cái giá. Nhưng nhiều trắng làm giảm chất lượng mô tả ít hơn nhiều so với một vệt mờ dài gần bằng nửa ô mô tả — và tỉ lệ đánh đổi đó thì đo được bằng số điểm khớp đúng, trên chính chuỗi của bạn.

B. Vì sao khử mờ rồi mới trích đặc trưng thường không giúp

Thứ tự “chạy mạng khử mờ, rồi trích ORB trên ảnh đã khử” nghe hợp lý tới mức ít ai kiểm. Nó thường không tăng số điểm khớp đúng, và có ba lý do độc lập.

Lý do thứ nhất, mục tiêu bị lệch. Mạng khử mờ được huấn luyện và đánh giá cho mắt người: PSNR, SSIM, LPIPS, và ngày càng nhiều hàm mất mát đối kháng hoặc tri giác. Có một đánh đổi đã được chứng minh giữa độ trung thực và mức “trông thật”. Vượt qua một ngưỡng, muốn ảnh trông sắc hơn thì buộc phải hy sinh độ trung thực với ảnh gốc [180]. Ảnh khử mờ nhìn sắc có thể chứa cạnh do mô hình bịa ra.

Điều đó giết tính lặp lại vì một lý do tinh tế. Ghép cặp đòi hỏi cùng một cấu trúc vật lý phải sinh ra cùng một cực trị cục bộ trong *cả hai* ảnh. Sai số của mạng khử mờ không độc lập giữa hai ảnh. Nó phụ thuộc nhân mờ của từng ảnh, mà hai ảnh có hai nhân mờ khác nhau vì camera chuyển động khác nhau. Vậy mạng sinh ra sai số *tương quan với chuyển động*, tức đúng loại sai số mà bộ ước lượng tư thế không chịu nổi. Nó không phải nhiều; nó là độ lệch.

Lý do thứ hai, mô hình mờ sai. Phần lớn mạng khử mờ giả định một nhân mờ chung cho cả ảnh, hoặc học từ dữ liệu tổng hợp mờ đều. Mờ do xoay thì gần như đều thật, nhưng mờ do tịnh tiến phụ thuộc độ sâu. Trong một cảnh có cả vật gần và vật xa, không tồn tại một nhân chung nào đúng.

Lý do thứ ba, độ trễ. Một mạng khử mờ là mô hình ảnh-sang-ảnh ở độ phân giải đầy đủ, tức hình dạng đất nhất trong các hình dạng mạng — và bạn vẫn phải trích đặc trưng *sau* nó. So với việc hạ thời gian phơi sáng, một thay đổi vài dòng không tốn phép tính nào, đây là một cuộc đổi chác rất xấu. Nếu vẫn muốn kiểm, đại lượng phải đo không phải độ trễ riêng của mạng mà là mức thay đổi phân vị cao của chu kỳ luồng theo vệt khi bật nó lên trong hệ đầy đủ.

Một ngoại lệ trung thực phải nêu. Với tầng *lấy ứng viên đóng vòng*, phép tính đối khác. Ở đó bạn dùng một mô tả toàn ảnh, chạy thừa thớt, không nằm trên đường tới hạn của luồng theo vết. Tiêu chí cũng không phải cực trị cục bộ mà là sự giống nhau toàn cục. Khử mờ trước khi tính mô tả toàn ảnh có thể có ích, và chi phí thì chịu được. Chi tiết về tầng đó nằm ở [F/29-dinh-vi-lai-va-dong-vong](#).

C. Hướng đúng thứ nhất: coi vết mờ là quỹ đạo trong thời gian phơi sáng

Đây là chỗ đảo ngược cách nghĩ, và là ý hay nhất trong mục này.

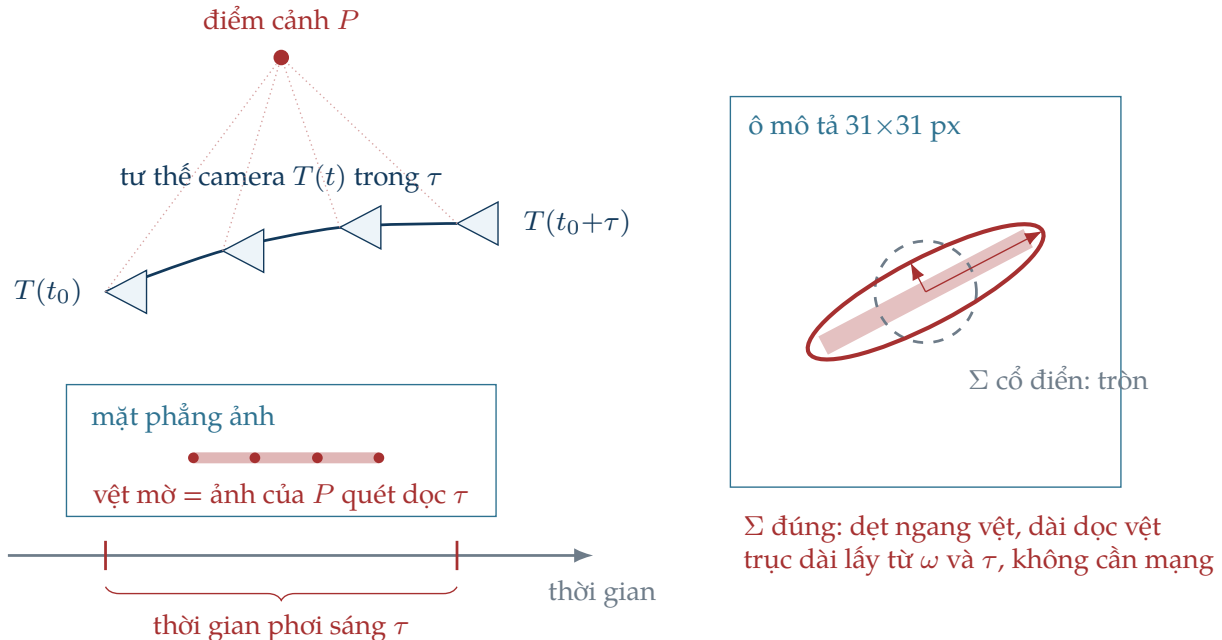
Ảnh mờ không phải ảnh sắc nét bị hỏng. Nó là *tích phân* của một chuỗi ảnh sắc nét, chụp từ các tư thế camera nằm dọc một đoạn quỹ đạo trong khoảng $[t_0, t_0 + \tau]$. Viết ra thì gọn: giá trị một pixel là trung bình theo thời gian của ảnh sắc nét được dựng từ tư thế $T(t)$.

Dòng BAD-NeRF và BAD-Gaussians dùng đúng phát biểu đó [181], [182], tiếp nối Deblur-NeRF [183]. Thay vì khử mờ ảnh, chúng mô hình hoá quá trình tạo ảnh. Chúng dựng vài ảnh ảo từ các tư thế nội suy giữa tư thế đầu và tư thế cuối của khoảng phơi sáng, lấy trung bình, rồi so với ảnh mờ đã ghi. Cảnh và cặp tư thế đầu-cuối được tối ưu đồng thời.

Hệ quả khái niệm là phân quan trọng nhất. **Vết mờ trở thành thông tin về chuyển động, thay vì một phiền toái.** Chiều dài và hướng của vết là một phép đo vận tốc. Một khung mờ không còn là khung phải bỏ; nó là khung mang một loại ràng buộc khác.

Bạn không cần chạy NeRF để lấy ý này. Phiên bản cổ điển của nó là: thay vì đánh giá phần dư phép chiếu tại một thời điểm duy nhất, hãy tích phân phép chiếu trên khoảng phơi sáng. Khi đó điểm đặc trưng mờ không còn là điểm ngoại lai; nó là một ràng buộc lên vận tốc. Cái giá là mô hình phép đo phức tạp hơn và Jacobian đắt hơn.

Có đáng làm không, tính tới tháng 9/2026: dòng BAD chạy *ngoại tuyến*, tối ưu riêng cho từng cảnh, mất từ vài phút tới vài giờ. Đó là NO-GO rõ ràng cho vòng thời gian thực. Giá trị của nó với bạn là mô hình phép đo, không phải phần mềm.



Hình 8.9. Trái: vết mờ là quỹ đạo camera chiếu xuống mặt phẳng ảnh trong thời gian phơi sáng, chứ không phải một ảnh sắc nét bị hỏng. Đây là mô hình mà dòng BAD-NeRF và BAD-Gaussians tối ưu trực tiếp. Phải: hệ quả tức thời cho một hệ cổ điển. Điểm đặc trưng mờ vẫn được định vị tốt *ngang* vết và rất tệ *dọc* vết, nên hiệp phương sai đúng là một ellipse dẹt xoay theo hướng vết — và hướng đó tính được từ con quay, miễn phí.

D. Hướng đúng thứ hai: dùng IMU để dự đoán vệt mờ

Bạn đã có tốc độ góc ở 200 Hz. Bạn cũng có thời gian phơi sáng, hoặc đọc được nó từ trình điều khiển camera. Vậy vệt mờ do xoay dự đoán được ở dạng đóng, trước khi nhìn vào một pixel nào. Ba cách dùng, xếp theo tỉ lệ lợi ích trên công sức.

1. **Đặt trọng số.** Với mỗi khung, tính $s = f\omega\tau$. Với mỗi điểm khớp, dựng Σ dị hướng: trục dài cỡ $s/2$ theo hướng vệt, trục ngắn giữ nguyên giá trị cũ. Đây không phải tinh chỉnh, vì cả hướng lẫn độ lớn đều suy từ mô hình chứ không dò theo kết quả. Và nó kiểm được bằng NEES: nhóm khung có s lớn phải hết bị khai quá tự tin sau khi sửa. Đây là thay đổi rẻ nhất trong cả chương có cơ hội thắng thật.
2. **Ghép cặp.** Làm mờ ô mô tả mẫu theo đúng vệt dự đoán trước khi so, hoặc trải rộng cửa sổ tìm kiếm dọc hướng vệt. Ý là đưa hai ảnh về cùng một mức mờ thay vì cố làm cả hai sắc nét.
3. **Điều khiển.** Đưa s dự đoán ngược về bộ điều khiển phơi sáng, đúng như mục A đã nói. Đây là vòng phản hồi mà rất ít hệ SLAM đóng lại.

Chỗ hỏng của cả ba: chúng chỉ tính phần mờ do xoay. Phần mờ do tịnh tiến cần độ sâu, mà độ sâu thì bạn chỉ có cho các điểm đã tam giác hoá. Với cảnh gần, phần bỏ sót này không nhỏ. Nói thẳng ra là mô hình chỉ đúng khi chuyển động chủ yếu là xoay. May thay, đó cũng chính là chế độ gây mờ nhiều nhất trên thiết bị cầm tay và trên drone.

E. Hướng đúng thứ ba: camera sự kiện, một lối thoát về phía cảm biến

Ba hướng trên đều có sống chung với thời gian phơi sáng. Camera sự kiện xoá bỏ nó.

Cơ chế: mỗi điểm ảnh báo cáo bất đồng bộ khi độ sáng log của nó đổi quá một ngưỡng. Không có khung hình, không có phơi sáng, nên không có mờ do chuyển động. Độ trễ thấp hơn nhiều bậc so với một cảm biến khung hình, và dải động rộng hơn hẳn [184]. Đây là một câu trả lời khác về chất cho cả chuyển động nhanh lẫn cảnh chênh sáng.

Bằng chứng gần nhất với bài toán của bạn là hướng kết hợp sự kiện với khung hình và IMU. Lợi ích tập trung đúng vào các tình huống dải động cao và tốc độ cao, chứ không trải đều [185]. Và có một kết quả đẹp về mặt vật lý: dòng sự kiện trong khoảng phơi sáng liên hệ với ảnh mờ qua một tích phân kép, nên từ một khung mờ cộng dòng sự kiện có thể dựng lại chuỗi ảnh sắc nét [186]. Đó là khứu mờ có căn cứ vật lý, không phải tiên nghiệm học được, và nó không bị ra cạnh.

Cổng đánh giá thì khắt khe. Đây là quyết định *phần cứng*, không phải quyết định thuật toán: thêm cảm biến, thêm hiệu chỉnh, thêm đồng bộ thời gian, và một chuỗi công cụ mỏng hơn hẳn. Có một lỗ hổng cần biết trước: camera sự kiện gần như không thấy gì khi cảnh và camera đều đứng yên. Với robot có lúc dừng, đó là một chế độ vận hành thật. Kết luận thực dụng: chỉ dựng thử nghiệm nếu bạn đã đo được rằng phần lớn thất bại của mình tập trung ở tốc độ cao hoặc chênh sáng. Mục A cho bạn cách đo đúng điều đó bằng phân bố của s .

Bảng 8.4. Bốn cách đối phó với ảnh mờ. Cột cuối cho biết cách nào hỏng ở đâu, và vì sao thứ tự thứ nên đi từ rẻ tới đắt.

Cách	Cơ chế	Chi phí và giả định	Hỏng khi nào
Hạ thời gian phơi sáng	Đặt trần τ , bù bằng độ khuếch đại	Gần như miễn phí; giả định cảnh đủ sáng để bù	Thiếu sáng nặng; nhiều cảm biến vượt mức mô tả chịu được
Khử mờ rồi trích đặc trưng	Mạng ảnh-sang-ảnh chạy trước frontend	Nặng nhất trong bốn cách: một mạng ảnh-sang-ảnh toàn độ phân giải đặt trước frontend; giả định một nhân mờ chung	Cạnh bị ra không lặp lại giữa hai ảnh; mờ do tịnh tiến phụ thuộc độ sâu
Mô hình hoá vệt như quỹ đạo	Tích phân phép chiếu trên τ ; dựng Σ dị hướng từ ω và τ	Rẻ nếu chỉ dùng phần trọng số; đắt nếu tích phân đầy đủ	Chuyển động chủ yếu là tịnh tiến với cảnh gần; τ không đọc được
Camera sự kiện	Báo cáo bất đồng bộ theo thay đổi độ sáng; không có phơi sáng	Thêm phần cứng, hiệu chỉnh, đồng bộ; chuỗi công cụ mỏng	Cảnh và camera cùng đứng yên: gần như không có dữ liệu

I Năm nấc đối phó với ảnh mờ, đúng thứ tự nên thử

1 Vấn đề

Giải pháp

Giả định

Giá phải trả

Ảnh mờ làm mất bám ở các đoạn quay nhanh.

tính $s = f\omega\tau$ rồi đặt trần cho thời gian phơi sáng.

cảnh đủ sáng để bù bằng độ khuếch đại.

nhiều cảm biến tăng, và trong bóng tối thì cách này hết đường.

2 Vấn đề

Giải pháp

Giả định

Hệ quả

Trần phơi sáng không đủ, vẫn còn khung mờ.

đừng bỏ khung mờ, hãy hạ trọng số của nó theo s .

bạn đọc được τ và có con quay đồng bộ.

đây là thay đổi rẻ nhất còn lại, và nó kiểm được bằng NEES.

3 Vấn đề

Giải pháp

Giả định

Vấn đề mới

Trọng số vô hướng không tả được kiểu sai số của điểm mờ.

dựng Σ dị hướng, trục dài dọc hướng vệt.

mờ chủ yếu do xoay.

phần mờ do tịnh tiến vẫn bị bỏ sót, và nó phụ thuộc độ sâu.

4 Vấn đề

Giải pháp

Giá phải trả

Ngay cả trọng số đúng cũng không lấy lại được thông tin đã mất.

mô hình hoá phép đo như tích phân trên khoảng phơi sáng, theo cách dòng BAD làm.

mô hình phép đo phức tạp hơn, Jacobian đắt hơn, và các hiện thực hiện có đều ngoại tuyến.

5 Vấn đề

Giải pháp

Giả định

Giá phải trả

Bản thân cảm biến khung hình là nguyên nhân gốc.

đổi sang hoặc thêm camera sự kiện.

bạn đã đo được rằng thất bại tập trung ở tốc độ cao và chênh sáng.

thêm phần cứng, thêm hiệu chỉnh, chuỗi công cụ mỏng, và một lỗ hỏng khi mọi thứ đứng yên.

F. Chuyển động nhanh và ảnh mờ là một bài toán nhìn từ hai phía

Khép mục này bằng một phát biểu gộp. Chuyển động nhanh phá *sự liên kết*: cửa sổ tìm kiếm quanh vị trí dự đoán đặt sai chỗ, và diện mạo của ô ảnh đã đổi. Ảnh mờ phá *phép đo*: vị trí điểm đặc trưng bị lệch có hệ thống, và bất định của nó trở thành dị hướng.

Phía liên kết cũng tính tay được, bằng đúng kiểu phép tính giả định vừa dùng. Với nhịp khung Δt và tốc độ góc ω , tâm của một ô ảnh dịch khoảng $f \omega \Delta t$ pixel giữa hai khung. Giữ nguyên giả định $f = 400$ pixel ở trên, một camera 20 Hz và một cú quay 200 độ mỗi giây cho quãng dịch cỡ 70 pixel — lớn hơn nhiều lần cửa sổ tìm kiếm mặc định của một bộ theo vết thừa. Nghĩa là ở tốc độ đó, việc mất bám không phải do mô tả kém mà do *tìm sai chỗ*, và cách chữa đúng là dùng con quay để dịch tâm cửa sổ tìm kiếm chứ không phải đổi bộ mô tả.

Hai đại lượng $f\omega\Delta t$ và $f\omega\tau$ đến từ cùng một tình huống nhưng chỉ vào hai chỗ sửa khác nhau, và cái thứ nhất lớn hơn cái thứ hai đúng bằng tỉ số $\Delta t/\tau$. Đó là lý do phải đo cả hai phân bố trên dữ liệu của mình trước khi chọn hướng đi.

Vì thế hai nỗi đau này có hai cách chữa khác nhau, và trộn chúng là lỗi hay gặp. Liên kết được chữa bằng dự đoán tốt hơn và mô tả bền hơn, tức địa hạt của [F/28-dac-trung-va-ghep-cap](#) và của dòng quang ở [F/30-do-sau-dong-quang-va-mo-hinh-nen](#). Phép đo được chữa bằng một ma trận thông tin trung thực hơn, tức địa hạt của chính chương này. Chẩn đoán đúng bệnh trước khi mua thuốc là toàn bộ giá trị của việc tách chúng ra.

CẦN NHỚ

Ma trận thông tin là chỗ mà mọi hiểu biết về chất lượng phép đo phải chảy vào. Nếu bạn biết một điều gì đó về một phép đo — ảnh mờ cỡ này, con quay đang quay nhanh, điểm khớp này mơ hồ — mà điều đó không xuất hiện trong Σ , thì bộ ước lượng không biết. Và trước khi thay Σ bằng một mạng, hãy đo NEES: rất nhiều hệ thống đang khai sai vài lần, và sửa hằng số theo phép đo rẻ hơn huấn luyện hai bậc độ lớn. [T1]

8.5 Đưa lên Jetson Orin: phần đặc thù SLAM (Deploying on Orin)

Mục này cố ý không nhắc lại TensorRT, các họ runtime, sai lệch số học sau export, lượng tử hoá, hay chuyện tụt xung vì nhiệt. Tất cả nằm ở [D/21-inference-va-trien-khai](#) và [D/20-toi-uu-va-nen-mo-hinh](#), và chúng đúng nguyên vẹn ở đây. Mục này chỉ nói bốn thứ mà một hệ SLAM có, còn một dịch vụ suy luận thì không.

A. Ngân sách không phải một con số, nó chia theo luồng

Một hệ kiểu ORB-SLAM3 có ba luồng với ba loại hạn chót rất khác nhau [1]. Luồng theo vết chạy mỗi khung hình và có hạn chót cứng bằng chu kỳ khung. Luồng lập bản đồ cục bộ chạy mỗi khung khoá, hạn chót mềm hơn nhiều bậc. Luồng đóng vòng chạy hiếm, nhưng khi nó chạy hiệu chỉnh chùm tia toàn cục thì nó khoá bản đồ.

Hệ quả thiết kế đến trước mọi phép đo. **Câu hỏi đầu tiên không phải mạng chạy hết bao nhiêu mili-giây, mà mạng sống ở luồng nào.** Cùng một mạng có thể là bất khả thi trên luồng theo vết và hoàn toàn thoải mái trên luồng đóng vòng. Khoảng cách giữa hai hạn chót ấy rộng hơn nhiều so với mọi mức tăng tốc bạn có thể mua bằng lượng tử hoá hay bằng một kiến trúc gọn hơn. Nên chọn luồng là biến thiết kế mạnh nhất bạn có, và nó phải được chọn *trước* khi đo bất cứ thứ gì.

Bảng 8.5. Ngân sách chia theo luồng cho một hệ stereo-inertial trên Orin. Bảng cố ý *không có một con số nào*: hạn chót của mỗi luồng suy ra từ nhịp của chính luồng đó, và phần còn trống thì chỉ máy của bạn trả lời được. Cái chuyển được từ hệ này sang hệ khác là thứ tự chặt-lỏng giữa ba dòng; cột cuối nói loại mạng nào có cửa ở mỗi dòng, và phép đo xác nhận nằm ở mục D.

Luồng	Nhịp	Hạn chót	Chỗ một mạng vừa được, và loại mạng nào
Theo vết	mỗi khung hình	Cứng: đúng bằng chu kỳ khung, không co giãn	Phần trống là chu kỳ khung trừ đi trích đặc trưng, ghép cặp và tối ưu tư thế — hãy đo phần đó trước tiên. Chỉ vừa cho mạng rất nhỏ: khử nhiễu con quay, hoặc một đầu ra bất định gắn vào bộ trích sẵn có
Lập bản đồ cục bộ	mỗi khung khoá, tức thừa hơn nhịp khung nhiều lần	Mềm: dài hơn chu kỳ khung nhiều bậc, và trễ một nhịp không làm rớt khung	Vừa cho mạng cỡ trung: hiệp phương sai theo từng điểm khớp cho hiệu chỉnh chùm tia cục bộ, hoặc ước lượng độ sâu cho khung khoá
Đóng vòng	hiếm, và không đều	Rất mềm: cả hệ chịu được một quãng dừng khi vòng đóng	Vừa cho mạng lớn: mô tả toàn ảnh, ghép cặp dày, kiểm chứng hình học học được

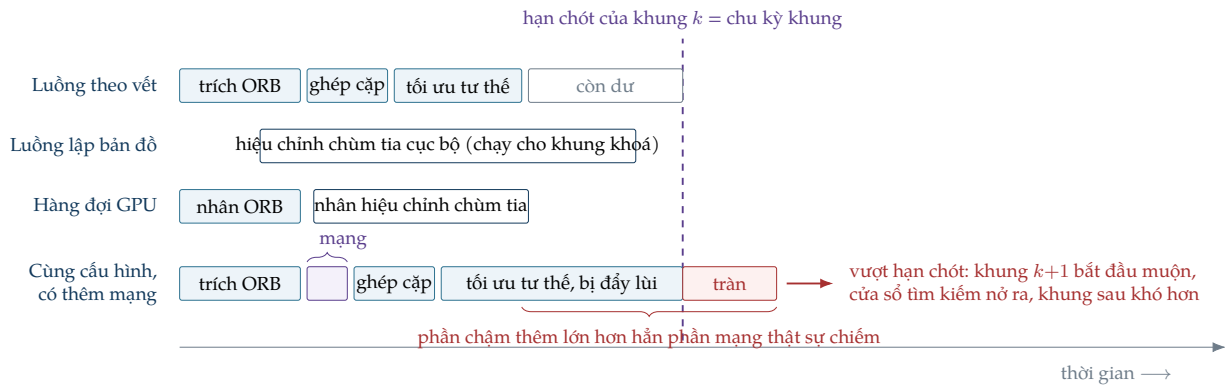
B. GPU không rảnh: mạng tranh chấp với chính bạn

Trong một dịch vụ suy luận, mạng là thứ duy nhất trên GPU. Trong hệ của bạn thì không. Nếu bạn đã đưa trích ORB và hiệu chỉnh chùm tia cục bộ lên GPU, thì một mạng mới không cộng vào một GPU rảnh. Nó xếp hàng sau các nhân của chính bạn. Ba hệ quả:

- **Độ trễ không cộng được.** Thêm một mạng có thể làm luồng theo vết chậm đi *nhiều hơn* đúng phần thời gian mà mạng chiếm khi chạy một mình, vì các nhân xếp hàng và vì cả hai tranh băng thông bộ nhớ. Trên Orin, CPU và GPU dùng chung bộ nhớ, nên tranh chấp xảy ra cả với phần chạy trên CPU.
- **Phép đo cô lập gần như vô dụng.** Độ trễ của mạng đo khi nó chạy một mình không nói gì. Đại lượng đúng là *mức thay đổi p99 của chu kỳ luồng theo vết* khi bật mạng lên, đo trong hệ đầy đủ. Chỉ có hiệu số đó mới trả lời được câu hỏi “có vừa không”.
- **Có ba đòn bẩy để giảm tranh chấp,** và cả ba là quyết định kiến trúc chứ không phải tối ưu: đặt mạng lên DLA để chừa GPU cho phần khác, kèm cảnh báo ở D/21 rằng toán tử không hỗ trợ sẽ rơi ngược về GPU; dùng dòng CUDA có mức ưu tiên để nhân của luồng theo vết chen lên trước; hoặc chạy mạng ở tần suất thấp hơn tốc độ khung.

Đòn bẩy thứ ba có một điều kiện hay bị bỏ qua. Chạy thừa là *hợp lệ với một hiệp phương sai* và *không hợp lệ với một tư thế*. Một ước lượng bất định thay đổi chậm và giữ giá trị cũ vài khung không sai nhiều. Một tư thế thì không.

Hình 8.10 vẽ ra cùng một cảnh trên trục thời gian, và nó cho thấy điều mà bảng không cho thấy: chỗ tranh chấp không nằm trong luồng nào cả, nó nằm ở hàng đợi GPU dùng chung.



Hình 8.10. Vì sao độ trễ không cộng được. Trục thời gian cố ý không có nhãn số: hình này nói về *thứ tự* và về chỗ tranh chấp, không về độ dài. Hàng thứ ba là hàng đợi GPU dùng chung: nhân của mạng chen vào giữa nhân trích ORB và nhân hiệu chỉnh chùm tia, nên phần tối ưu tư thế ở hàng dưới cùng bị đẩy lùi nhiều hơn hẳn phần thời gian mà mạng thật sự chiếm. Đó là lý do phải đo mức thay đổi p99 của cả chu kỳ, chứ không đo riêng mạng.

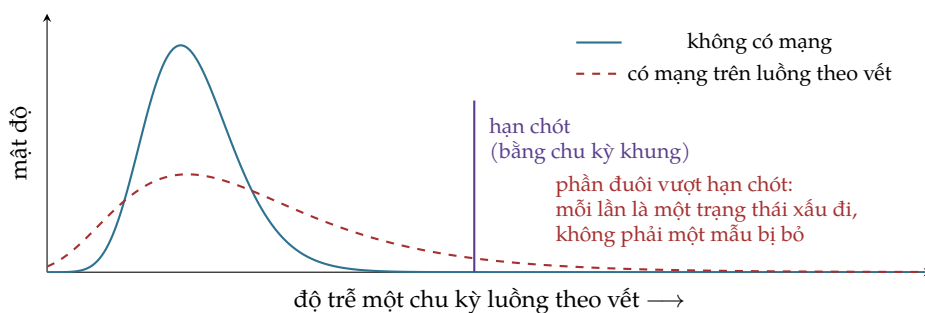
C. Độ trễ đuôi quan trọng hơn độ trễ trung bình

Chương 21 đã nói phải báo phân vị thay vì trung bình. Với SLAM, lý do mạnh hơn một bậc, và có ba tầng.

Một khung trễ không phải một khung mất. Nó đổi trạng thái của bộ ước lượng. Luồng theo vết thiếu một quan sát, cửa sổ tìm kiếm cho khung sau nở ra, nên khung sau khó hơn. Hàm chi phí của việc vượt hạn chót vì thế có trí nhớ và lỗi lên: hai lần trễ liên tiếp đắt hơn nhiều hai lần trễ rời rạc.

Đuôi lan qua khoá. Ba luồng dùng chung bản đồ sau một mutex. Một đỉnh trễ ở luồng lập bản đồ cục bộ, hoặc một lần hiệu chỉnh chùm tia toàn cục ở luồng đóng vòng, sẽ chặn luồng theo vết truy cập bản đồ. Nghĩa là một sự kiện đuôi ở luồng *mềm* biến thành một lần lỡ hạn chót ở luồng *cứng*. Đây là lớp lỗi mà đo từng luồng riêng không bao giờ thấy. Bất kỳ ai từng gỡ một lần kẹt giữa lập bản đồ và hiệu chỉnh toàn cục đều nhận ra nó.

Mạng làm đuôi béo thêm. Thời gian chạy của một mạng phụ thuộc dữ liệu nhiều hơn mã C++ thuần: số hộp trước bước hậu xử lý, hình dạng động, lần đầu gặp một kích thước mới làm bộ cấp phát chạy lại. Nên đuôi của hệ có mạng dày hơn đuôi của hệ không có mạng, kể cả khi trung bình bằng nhau.



Hình 8.11. Vì sao trung bình nói dối trong hệ thời gian thực. Trục không có nhãn số vì hình này nói về *hình dạng đuôi*, không về độ trễ của một hệ cụ thể — độ trễ thì bạn đo bằng harness của mình. Hai phân bố có trung vị gần nhau, nhưng phân bố có mạng dày đuôi hơn hẳn, nên phần diện tích vượt hạn chót lớn hơn nhiều lần. Trong SLAM, mỗi lần vượt hạn chót không độc lập với lần sau: nó làm khung kế tiếp khó hơn, nên chi phí tích lũy chứ không cộng.

D. Đo lại cho trung thực bằng chính harness lặp nhiều lần của bạn

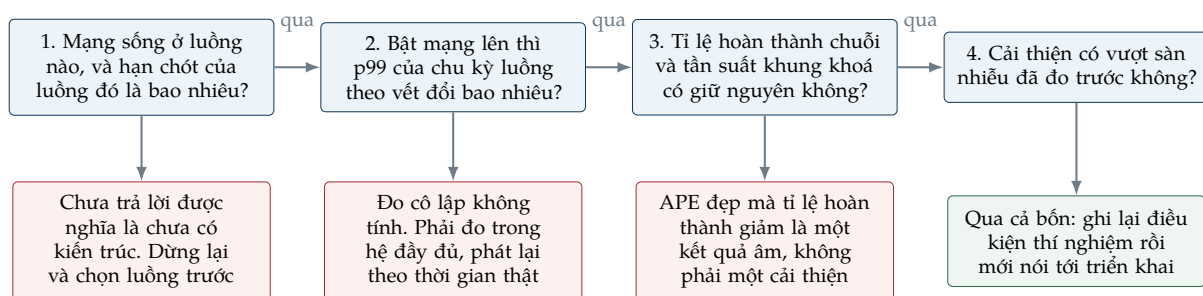
Bạn đã có công cụ đúng: một harness chạy lặp, và thói quen đo sần nhiều trước khi so sánh. Mục này chỉ nói năm điều chỉnh cần thêm khi vật được đo là một mạng trong một hệ thời gian thực.

1. **Coi “bật/tắt mạng” là đúng một yếu tố ablation, và đo sần nhiều trước.** Toàn bộ kỷ luật ở E/24-thí-nghiệm-va-ablation áp nguyên vào đây. Một lần chạy không nói gì, và sần nhiều của một hệ có mạng thường rộng hơn sần nhiều của hệ cũ.
2. **Đừng chạy lockstep khi đo độ trễ.** Chế độ nạp khung theo nhịp hệ thống tiêu thụ là công cụ đúng để cô lập tính phi tắt định, và là công cụ sai để đánh giá thời gian thực. Nó biến một lần lỡ hạn chót thành một lần chạy chậm, tức xoá đúng thứ bạn đang muốn đo. Phát lại bag theo thời gian thật, và đếm số khung bị bỏ.
3. **Ghi thời gian theo luồng, không chỉ ghi APE.** Tối thiểu: p50 và p99 của chu kỳ luồng theo vết, tần suất khung khoá, thời gian hiệu chỉnh chùm tia cục bộ, số lần mất bám. Một mạng có thể làm APE đẹp lên trong khi bóp tần suất khung khoá, và con số APE tổng che kín điều đó.
4. **Báo tỉ lệ hoàn thành chuỗi cùng với APE.** Một hệ mất bám giữa chừng rồi khởi tạo lại có thể cho APE rất đẹp trên phần nó còn bám được. Đây là cái bẫy đọc số đã nêu ở B/07-chia-du-lieu-va-danh-gia, và nó xuất hiện nhiều nhất đúng ở các cấu hình có mạng.
5. **Chạy đủ dài, ở đúng chế độ nguồn.** Quy trình mười phút liên tục của D/21 áp nguyên, với một điều chỉnh: tải của SLAM phụ thuộc dữ liệu, nên phải phát lại một bag thật lặp vòng, không phải một vòng lặp suy luận tổng hợp.

→ **tiền đề** D/21 — mọi kỹ thuật export, chọn runtime và đo dưới tải nhiệt nằm ở chương 21; mục này chỉ thêm phần mà kiến trúc đa luồng của SLAM sinh ra

E. Bốn câu hỏi trước khi bật một mạng trong vòng thời gian thực

Gộp bốn mục trên thành một cây quyết định chạy được trong đầu. Bốn câu hỏi dưới đây phải trả lời theo đúng thứ tự, vì một câu trả lời “không” ở câu trước làm ba câu sau vô nghĩa.



Hình 8.12. Bốn cổng cho một mạng trong vòng thời gian thực, theo đúng thứ tự phải hỏi. Ba cổng đầu là câu hỏi kỹ thuật hệ thống và không liên quan gì tới chất lượng của mạng; chỉ cổng thứ tư mới hỏi mạng có tốt không. Thứ tự này rẻ hơn thứ tự ngược lại, vì phần lớn ứng viên chết ở cổng một hoặc cổng hai, nơi chi phí kiểm tra gần bằng không.

Điểm đáng chú ý ở Hình 8.12 là ba cổng đầu không hỏi gì về chất lượng của mạng. Chúng là câu hỏi kỹ thuật hệ thống thuần túy. Trả lời chúng trước tiết kiệm được rất nhiều thời gian, vì phần lớn ứng viên chết ở cổng một hoặc hai, và hai cổng đó rẻ hơn cổng bốn hàng chục lần.

8.6 Lộ trình học cho riêng hướng này (A Learning Path for This Direction)

Chương E/26-lo-trình-hoc đã cho sáu lộ trình theo mục tiêu, và lộ trình D là “áp dụng vào SLAM production”. Mục này nối tiếp lộ trình đó ở chỗ nó dừng lại: bạn đã học xong nền, giờ đọc tám chương của Phần F theo thứ tự nào.

A. Tuần 0: đo trước khi đọc

Điều bất thường của lộ trình này là nó bắt đầu bằng ba phép đo chứ không bằng một chương. Lý do đơn giản: bảy chương còn lại của Phần F cùng hứa chứa những nỗi đau khác nhau, và bạn chỉ có ngân sách cho một. Ba phép đo dưới đây, tất cả gói trong hai tới ba ngày, quyết định bạn đọc gì.

- **NEES theo từng loại nhân tử**, trên một chuỗi dễ và một chuỗi khó. Nó nói trọng số hiện tại sai bao nhiêu lần và sai ở đâu.
- **Phân bố vết mờ dự đoán** $s = f\omega\tau$, và tương quan của nó với tỉ lệ điểm khớp đúng. Nó nói ảnh mờ có thật là nút thắt hay không.
- **Hồ sơ độ trễ theo luồng**, p50 và p99 cho từng luồng, trên phần cứng thật. Nó nói bạn còn bao nhiêu ngân sách và ở luồng nào.

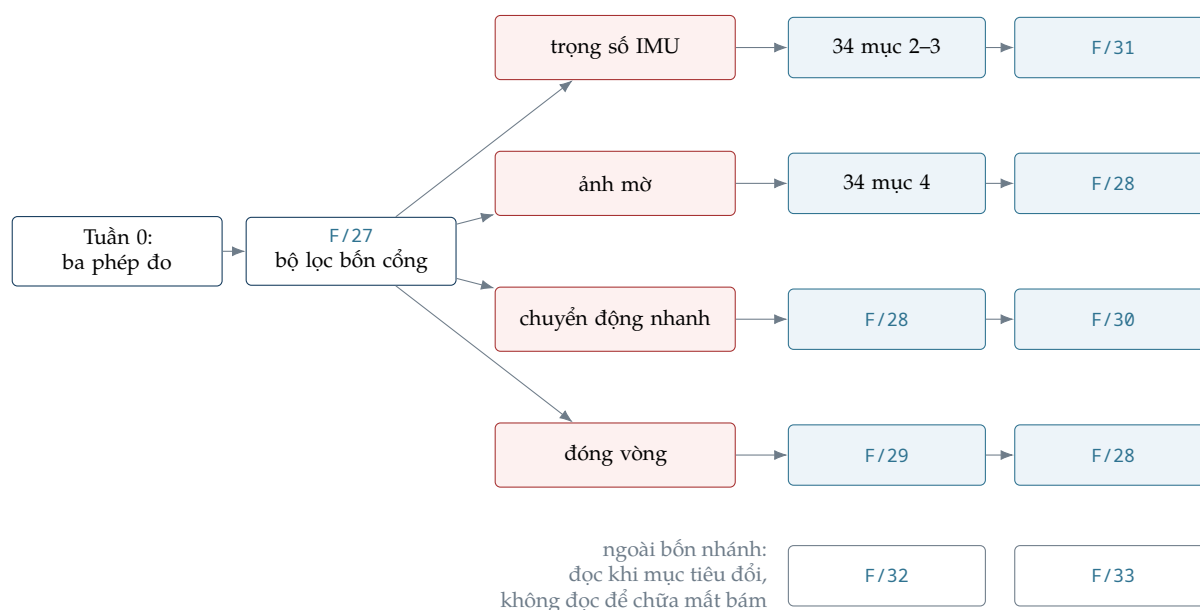
Nếu ba phép đo này nói trọng số của bạn đã trung thực, ảnh mờ hiếm, và không còn ngân sách, thì kết luận đúng là không đọc tiếp chương nào cả. Và đó cũng là một kết quả.

B. Thứ tự đọc tám chương theo bốn nỗi đau

Bảng 8.6. Đọc Phần F theo nỗi đau đang gặp, không đọc tuần tự. Chương 27 luôn đọc trước vì nó là bộ lọc. Cột cuối nêu chương *không* nên đọc cho nỗi đau đó, để tiết kiệm thời gian.

Nỗi đau	Thứ tự đọc	Bạn tìm gì trong đó	Bỏ được
Trọng số IMU	F/27 → 34 mục 2–3 → F/31	Phép thử NEES, khuôn “phép đo kèm Σ ”, và “khả vi” mua được gì ở backend	32, 33
Ảnh mờ	F/27 → 34 mục 4 → F/28	Vết mờ tính ra pixel, Σ dị hướng, rồi mới tới mô tả bền với mờ	31, 32, 33
Chuyển động nhanh	F/27 → F/28 → F/30 → 34 mục 4	Ghép cặp bên, dòng quang thay ghép cặp thừa, rồi phần bất định	32, 33
Đóng vòng	F/27 → F/29 → F/28	Bốn tầng của đóng vòng, và tầng nào thật sự giết vòng	30, 31, 32, 34

Hình 8.13 vẽ lại cùng bảng đó dưới dạng bản đồ, để thấy hai chương nào nằm ngoài mọi nhánh.



Hình 8.13. Bản đồ đọc Phần F theo nỗi đau. Ba phép đo của tuần 0 quyết định vào nhánh nào, và chương 27 lọc tiếp. Mỗi nhánh chỉ hai tới ba chương, nên không nhánh nào cần đọc quá một nửa Phần F. Hai chương ở dưới cùng nằm ngoài mọi nhánh: chúng phục vụ mục tiêu đổi biểu diễn bản đồ và thêm hiểu biết về cảnh, không phục vụ việc chống mất bám.

Hai chương [F/32-ban-do-neural](#) và [F/33-ngu-nghia-va-do-thi-can](#) không phục vụ bốn nỗi đau này. Chúng phục vụ một mục tiêu khác: đổi biểu diễn bản đồ, hoặc thêm hiểu biết về cảnh. Đọc chúng khi mục tiêu đổi, chứ không đọc để chữa mất bám.

C. Mốc thời gian thực tế, và kết cục nhiều khả năng nhất

Ước lượng dưới đây giả định mười giờ mỗi tuần và giả định bạn đã có hệ thống chạy được. Nó là phán đoán về thứ tự phụ thuộc, không phải số đo.

- **Tuần 0**, hai tới ba ngày: ba phép đo ở mục A.
- **Tuần 1–2**: [F/27](#) và mục 2–3 của chương này. Kết quả cụ thể phải có: một bảng NEES, và một quyết định về hằng số trọng số dựa trên bảng đó.
- **Tuần 3–5**: đúng *một* chương theo nỗi đau đã chọn, cộng một lần dựng lại một phương pháp trong đó cho chạy ngoại tuyến trên dữ liệu của bạn. Ngoại tuyến trước, để tách câu hỏi “nó có đúng không” khỏi câu hỏi “nó có kịp không”.
- **Tuần 6–8**: một thí nghiệm có cổng. Bật/tắt, nhiều lần chạy, sắn nhiều đo trước, ghi cả thời gian theo luồng lẫn tỉ lệ hoàn thành.
- **Sau đó**: chỉ đi tiếp nếu cổng đã qua.

Hai tháng để tới một quyết định. Kết cục nhiều khả năng nhất, và cần nói thẳng, là một NO-GO kèm lý do đo được. Đó không phải hai tháng bỏ đi. Một NO-GO ghi đủ điều kiện thí nghiệm là thứ ngăn bạn thử lại cùng ý tưởng vào năm sau, và là thứ cho bạn biết điều kiện nào đổi thì đáng thử lại. Đúng như [E/24-thi-nghiem-va-ablation](#) lập luận.

8.7 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả chương là giả định rằng nhiều *có thể* mô tả được bằng một hiệp phương sai Gauss. Nhiều chế độ hỏng thật thì không. Một điểm khớp sai ghép sang cấu trúc lặp lại không

phải một mẫu Gauss lệch xa; nó là một mẫu từ một phân phối khác hẳn. Với loại đó, cách đúng không phải nói Σ mà là loại bỏ, bằng kiểm chứng hình học hoặc bằng hàm mất mát bền vững. Nói Σ cho một điểm ngoại lai chỉ làm nó gây hại chậm hơn. Ranh giới thực dụng: nếu phần dư còn nằm trong vài lần σ , đó là chuyện của trọng số; nếu nó ở ngoài, đó là chuyện của loại bỏ.

Chế độ hỏng thứ hai thuộc về chính bất định học được. Suy dẫn ở mục 3 cho thấy nhánh phương sai chỉ hồi quy bình phương sai số kỳ vọng theo đầu vào. Ra ngoài phân phối huấn luyện, nó vẫn xuất ra một số nhỏ, và bộ ước lượng nhận số đó như một lời cam đoan. So với một hằng số dờ, một mạng tự tin sai còn tệ hơn. Hằng số dờ thì sai đều và bạn bù được; mạng thì sai đúng vào những cảnh lạ, tức đúng những cảnh bạn cần được cảnh báo nhất. Cách phòng rẻ duy nhất là giữ một chặn dưới cứng cho Σ , tức không bao giờ để mạng khai bất định nhỏ hơn giá trị vật lý của cảm biến.

Chế độ hỏng thứ ba nằm ở phép thử NEES chứ không ở phương pháp. NEES giả định mô hình phép đo *đúng dạng* và chỉ sai về độ lớn. Nếu bạn có một sai lệch thời gian chưa hiệu chỉnh, hoặc hiệu chỉnh ngoại sai, thì phần dư mang một thành phần có hệ thống. NEES lớn hơn bậc tự do sẽ dụ bạn nói Σ , và nói Σ khi đó là giấu một lỗi hiệu chỉnh vào trọng số. Cách phân biệt là nhìn phần dư có tương quan với chuyển động không: nếu phần dư ảnh tỉ lệ với tốc độ góc, bạn đang nhìn một t_d sai, không phải nhiễu.

Chế độ hỏng thứ tư là chỗ chương này nói ít nhất: mờ do tịnh tiến. Toàn bộ phần dự đoán vệt mờ ở mục 4 chỉ dùng con quay, nên nó bỏ sót phần mờ phụ thuộc độ sâu. Với cảnh xa thì bỏ sót đó không đáng kể. Với cảnh gần và chuyển động tịnh tiến nhanh, mô hình sẽ đánh giá thấp vệt mờ, và nó lại sai theo hướng quá tự tin. Tôi không biết một cách rẻ nào để sửa điều này mà không cần độ sâu tin cậy trước khi ghép cặp.

Cuối cùng, một giới hạn về bằng chứng. Nhiều công trình được nhắc trong chương này rất mới, và tôi mô tả cơ chế chứ không mô tả kết quả số. Với các công trình từ 2024 trở đi, hãy coi phần đánh giá “có đáng không” của tôi là một phán đoán dựa trên cấu trúc chi phí, không phải kết luận từ một lần dựng lại. Nếu bạn dựng lại và thấy khác, con số của bạn thắng.

8.8 Thực hành

Repo và tài nguyên. [evo](#) cho APE và RPE; nhớ báo RPE ở nhiều độ dài cửa sổ chứ không chỉ APE. [ethz-asl/kalibr](#) cho hiệu chỉnh camera-IMU và cho sai lệch thời gian t_d . Một t_d chưa hiệu chỉnh trông hết như trọng số IMU sai, nên hãy loại trừ nó trước. Công cụ tính phương sai Allan từ một bag ROS đứng yên nhiều giờ thì đơn giản tới mức tự viết cũng được. Về phía học sâu, mã nguồn của TLIO, RoNIN và MAC-V0 đều công khai; hãy đọc phần định nghĩa hàm mất mát trước phần kiến trúc, vì đó là chỗ chứa toàn bộ quyết định thiết kế. [kornia](#) có sẵn các phép biến đổi ảnh khả vi nếu bạn muốn thử mô hình mờ dạng tích phân. Danh sách này chốt tại tháng 9/2026 và tên mô hình trong đó sẽ cũ trong sáu tới mười hai tháng; các công cụ hạ tầng như [evo](#) và [ethz-asl/kalibr](#) thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. NEES trung bình của nhân tử ảnh trên chuỗi khó gấp k lần bậc tự do: σ của bạn đang sai bao nhiêu lần, và vì sao con số đó vẫn còn là bản lạc quan? Nếu tăng thời gian phơi sáng gấp đôi thì độ dài vệt mờ đổi bao nhiêu, và tỉ lệ tín hiệu trên nhiễu đổi bao nhiêu, hai đại lượng đó có cùng bậc không? Một mạng khử mờ làm PSNR tăng rõ rệt nhưng số điểm khớp đúng lại giảm: hai cơ chế nào giải thích được, và phép thử nào phân biệt chúng?

Mini project. Hai bài ở hai khối bên dưới. Làm bài thứ nhất trước, vì bài thứ hai dùng lại đúng bộ đo mà bài thứ nhất dựng ra.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ma trận thông tin	Nghịch đảo hiệp phương sai, $\Omega = \Sigma^{-1}$, tức trọng số của một ràng buộc trong đồ thị nhân tử. Chỉ tỉ số giữa các Ω mới đối nghiệm, nên siết IMU và nối ảnh là cùng một thao tác.
••• Vì sao sai một bậc về σ lại nghiêm trọng?	Vì trọng số đi với $1/\sigma^2$. Sai mười lần về độ lệch chuẩn là sai một trăm lần về trọng số, đủ để bộ ước lượng bỏ hẳn một nguồn thông tin.
••• Quá tin IMU	Σ_{imu} khai quá nhỏ. Độ lệch con quay không được sửa, sai số góc nghiêng trọng lực, và sai số vị trí lớn theo $\frac{1}{6} g e t^3$. Chữ ký: APE trôi mượt trong khi RPE ngắn hạn vẫn đẹp.
••• Quá tin ảnh	$\Sigma_{\text{ảnh}}$ khai quá nhỏ. Khung mờ có toàn quyền kéo nghiệm; phần giạt chảy ngược vào ước lượng độ lệch và làm dự đoán khung sau tệ hơn. Chữ ký: RPE ngắn hạn xấu đều.
•• Phương sai Allan	Cách tách nhiễu trắng, độ trôi của độ lệch và bước đi ngẫu nhiên từ dữ liệu ghi lúc đứng yên nhiều giờ. Nó cho một <i>sân</i> , không cho giá trị hiệu dụng lúc vận hành.
••• σ hiệu dụng	Giá trị mà đồ thị nhân tử thật sự cần. Nó phải nuốt cả sai lệch thời gian, sai số hiệu chỉnh ngoại, lệch trục và sai số rời rạc hoá, nên luôn lớn hơn datasheet.
••• NEES	Trung bình của $r^T \Sigma^{-1} r$; nếu Σ trung thực thì nó bằng số bậc tự do. Đây là phép đo biến câu hỏi “trọng số của tôi có đúng không” thành một con số, và nó không cần chân trị.
••• NEES trung bình gấp k lần bậc tự do — nghĩa là gì?	σ của bạn nhỏ hơn thực tế $\sqrt{k}$ lần, tức bạn đang tự tin hơn mức được phép đúng $\sqrt{k}$ lần. Phân biệt với lỗi hiệu chỉnh: nếu phần dư tỉ lệ với tốc độ góc thì đó là sai lệch thời gian t_d , không phải nhiễu.
••• Độ hợp lý âm cho hiệp phương sai	Hàm mất mát $\frac{1}{2}(e^2/\sigma^2 + \log \sigma^2)$. Số hạng đầu phạt tự tin mà sai, số hạng sau phạt né bằng cách khai bất định lớn. Cực tiểu nằm tại $\sigma^2 = e^2$.
••• Bất định học được thực chất là gì?	Một bài hồi quy bình phương sai số lên chính đầu vào. Vì thế nó chỉ đúng cho loại sai số đã thấy lúc huấn luyện, và ra ngoài phân phối nó vẫn khai bất định nhỏ.
•• TLIO	Mạng hồi quy dịch chuyển ba chiều kèm hiệp phương sai từ một cửa sổ IMU, rồi đưa cặp đó vào bộ lọc như một phép đo giả. Bài học: mạng cấp phép đo, bộ lọc vẫn là bộ ước lượng.
• RoNIN	Hồi quy vận tốc chỉ từ IMU trong hệ quy chiếu không phụ thuộc hướng, kèm một bộ dữ liệu chuẩn. Độ chính xác của nó đến từ tiên nghiệm đáng đi, nên nó không chuyển sang drone hay xe.
•• Khử nhiễu con quay bằng mạng	Mạng tích chập giãn xuất lượng hiệu chỉnh cộng vào phép đo con quay, huấn luyện trên số gia xoay tích lũy. Nó sửa phép đo chứ không sửa trạng thái, nên backend giữ nguyên.
••• MAC-VO	Bộ ghép cặp xuất bất định cho từng điểm khớp, đưa qua hình học phép chiếu thành hiệp phương sai ba chiều đúng đơn vị, rồi dùng nó làm trọng số cho từng ràng buộc ảnh.
••• Vệt mờ	Độ dài $s \approx f \omega \tau$ pixel, tuyến tính theo cả tốc độ góc lẫn thời gian phơi sáng. Với các giá trị f và τ thường gặp, một cú quay nhanh đủ đưa s lên cỡ nửa ô mô tả ORB rộng 31 px, và lúc đó cấu trúc trong ô gần như mất hết. Thay f và τ của camera mình vào rồi vẽ phân bố của s .
••• Vì sao khử mờ rồi trích đặc trưng thường không giúp?	Mạng khử mờ tối ưu cho mắt người nên bịa cạnh, và sai số của nó phụ thuộc nhân mờ nên khác nhau giữa hai ảnh. Kết quả là một độ lệch tương quan với chuyển động, chứ không phải nhiễu.
•• BAD-NeRF và BAD-Gaussians	Coi ảnh mờ là trung bình của các ảnh dựng từ tư thế dọc quỹ đạo trong thời gian phơi sáng, rồi tối ưu đồng thời cảnh và quỹ đạo đó. Ngoại tuyến, nhưng mô hình phép đo thì dùng lại được.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Σ dị hướng cho điểm mờ	Điểm đặc trưng mờ được định vị tốt ngang vệt và tẻ dọc vệt, nên hiệp phương sai đúng là ellipse dẹt xoay theo hướng vệt. Hướng và độ dài suy từ con quay, không cần mạng.
• Camera sự kiện	Cảm biến báo cáo bất đồng bộ khi độ sáng đổi; không có phơi sáng nên không có mờ, độ trễ thấp hơn nhiều bậc so với cảm biến khung hình, dải động rộng hơn hẳn. Nó không thấy gì khi cảnh và camera cùng đứng yên.
• • • Vì sao độ trễ đo riêng của một mạng chưa nói được nó có vừa ngân sách không?	Vì nó tranh GPU và băng thông bộ nhớ với trích đặc trưng và hiệu chỉnh chùm tia của chính bạn, nên độ trễ không cộng được. Đại lượng đúng là mức thay đổi p99 của chu kỳ luồng theo vệt khi bật mạng, đo trong hệ đầy đủ.
• • • Đuôi độ trễ trong SLAM	Một khung trễ đổi trạng thái bộ ước lượng chứ không chỉ bị bỏ, nên chi phí tích lũy. Và một đỉnh trễ ở luồng mềm chặn luồng cứng qua khoá bản đồ, điều mà đo từng luồng riêng không thấy.
• • Vì sao không đo độ trễ ở chế độ lockstep?	Vì nạp khung theo nhịp hệ thống tiêu thụ biến một lần lỡ hạn chót thành một lần chạy chậm. Lockstep đúng cho việc cô lập tính phi tất định và sai cho việc đánh giá thời gian thực.
• • • APE đẹp lên nhưng tỉ lệ hoàn thành giảm — nghĩa là gì?	Hệ mất bám giữa chừng và bạn đang đo APE trên phần dễ còn lại. Luôn báo tỉ lệ hoàn thành cùng APE, và báo RPE ở vài độ dài cửa sổ.

• • • phải thuộc • • nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [1]	Ước lượng độ lệch cảm biến trực tuyến, và trọng số ảnh theo tầng kim tự tháp — đúng hai chỗ mà mục 2 chỉ ra là ô trống	Hạ tầng; chính là hệ bạn đang bảo trì
VINS-Mono [171]	Khuyến nghị nhân hệ số nhiễu IMU lên so với giá trị đo, tức thừa nhận σ hiệu dụng lớn hơn datasheet	Thông lệ đã thành chuẩn trong cộng đồng VIO
Tiền tích phân trên đa tạp [8]	Sinh ra hiệp phương sai của nhân tử IMU; mọi bàn luận về trọng số trong chương đều xoay quanh ma trận đó	Nền của gần như mọi hệ VIO hiện đại
TLIO [20]	Mạng cấp phép đo kèm hiệp phương sai, bộ lọc vẫn giữ trạng thái — khuôn cấm mạng vào hệ cổ điển	Được dùng lại rộng rãi trong dòng đo quán tính học được
MAC-V0 [106]	Hiệp phương sai theo từng điểm khớp, mang đúng đơn vị đo, làm trọng số cho từng ràng buộc ảnh	Rất mới; mô tả cơ chế, chưa kiểm chứng số
BAD-Gaussians [182]	Mô hình hoá vệt mờ như quỹ đạo trong thời gian phơi sáng, tối ưu đồng thời cảnh và tư thế	Ngoại tuyến; giá trị với bạn là mô hình phép đo

MINI PROJECT SLAM • Kiểm tra ma trận thông tin của bạn có trung thực không • 1–2 ngày

Mục tiêu	biến câu “trọng số IMU của tôi sai” thành một con số cho từng loại nhân tử và từng chuỗi. Đây là bài phải làm trước mọi thứ khác trong chương.
Dữ liệu	hai chuỗi EuRoC có độ khó khác nhau, ví dụ một chuỗi dễ và V2_03 [2]; cấu hình mặc định; ba lần chạy mỗi chuỗi để thấy độ tản mát.

Các bước	ghi lại mọi phần dư ở cuối một lần hiệu chỉnh chùm tia cục bộ, kèm hiệp phương sai bạn khai cho nó; tính $r^T \Sigma^{-1} r$ cho từng loại nhân tử; lấy trung bình rồi chia cho số bậc tự do, tức 2 cho phép chiếu và 9 hoặc 15 cho tiền tích phân tùy bạn có tính hai độ lệch không; lặp lại nhưng chia điểm khớp thành nhóm theo tầng kim tự tháp và theo vết mờ dự đoán $s = f\omega\tau$.
Xong khi	bạn có một bảng NEES với một dòng cho mỗi loại nhân tử và mỗi chuỗi, và phát biểu được câu “trên chuỗi này, σ của nhân tử X đang nhỏ hơn thực tế k lần”. Kiểm tra chéo: tỉ số phải lớn hơn ở chuỗi khó. Nếu bằng nhau thì nhiều khả năng bạn đang đo phần dư sau tối ưu và cần chuyển sang phần dư dự đoán.
Nối với	E/24 cho sàn nhiễu và số lần chạy tối thiểu; B/07 cho cách báo con số kèm khoảng; bảng này là đầu vào của mini project thứ hai.

MINI PROJECT SLAM · Vết mờ dự đoán từ IMU làm trọng số cho ảnh · 2 ngày

Mục tiêu	kiểm xem một Σ ảnh phụ thuộc vết mờ có thắng được trọng số theo tầng kim tự tháp hay không, và làm điều đó mà không tinh chỉnh một hằng số nào.
Dữ liệu	cùng hai chuỗi của bài trước, cộng một chuỗi TUM-VI có đoạn quay nhanh nếu bạn có [3].
Các bước	với mỗi khung, tính $s = f\omega\tau$ từ con quay và thời gian phơi sáng; vẽ phân bố của s và vẽ tương quan giữa s với tỉ lệ điểm khớp đúng; dựng Σ dị hướng có trục dài $s/2$ theo hướng vết và trục ngắn giữ nguyên; chạy lại, so với đường <i>baseline</i> , mỗi cấu hình lặp theo số lần mà bài trước đã tính ra.
Xong khi	bạn có đồ thị phân tán s so với tỉ lệ điểm khớp đúng, bảng NEES <i>sau khi sửa</i> cho thấy nhóm khung có s lớn không còn bị khai quá tự tin, và một so sánh APE cộng RPE cộng tỉ lệ hoàn thành, đặt cạnh sàn nhiễu. Một kết luận NO-GO kèm ba con số đó cũng là hoàn thành bài.
Nối với	F/28 nếu bước tiếp theo là đổi bộ mô tả; D/21 nếu chỉ phí tính s và dựng Σ trở thành vấn đề độ trễ.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai kiểu hỏng có hai phổ tần khác nhau. Quá tin IMU cho sai số trôi mượt theo lũy thừa của thời gian, nên APE xấu trong khi RPE trên cửa sổ ngắn vẫn đẹp. Quá tin ảnh cho sai số giật bị chặn, nên RPE ngắn hạn xấu ở mọi chỗ. Phép đo tách chúng là báo RPE ở vài độ dài cửa sổ bên cạnh APE.
- A2. Vì datasheet trả lời câu hỏi “con chip này nhiễu bao nhiêu trong phòng thí nghiệm”, còn đồ thị nhân tử hỏi “phép đo này sai bao nhiêu trên thiết bị đang chạy”. Giá trị thứ hai phải nuốt thêm sai lệch thời gian, sai số hiệu chỉnh ngoại, lệch trục, rung gập tần và sai số rời rạc hoá. Nó cũng đổi theo nhiệt độ và theo tốc độ chuyển động, nên không có hằng số nào đúng cho mọi chế độ.
- A3. Nó cho một ước lượng *độ tin cậy* phụ thuộc đầu vào. Suy dẫn từ độ hợp lý âm cho thấy nhánh phương sai hội tụ về bình phương sai số kỳ vọng có điều kiện, tức đúng đại lượng mà một hằng số datasheet không thể biểu diễn. Đó là ô trống thật của hệ cổ điển, còn ước lượng tư thế thì không phải.
- A4. Vì mạng khử mờ tối ưu cho cảm nhận của mắt người, và có một đánh đổi đã chứng minh giữa “trông sắc” và “trung thực”. Cảnh do mạng bịa ra không lặp lại được, và sai số của mạng phụ thuộc nhân mờ của từng ảnh nên nó tương quan với chuyển động. Kết quả là một độ lệch có hệ thống, đúng loại sai số mà bộ ước lượng tư thế không chịu được.
- A5. Vì đó là con số đo cô lập, còn mạng chạy trong một hệ mà GPU và băng thông bộ nhớ đã bị trích đặc trưng và hiệu chỉnh chùm tia chiếm. Đại lượng đúng là mức thay đổi p99 của chu kỳ luồng theo vết khi bật mạng, đo ở tốc độ phát lại thật chứ không phải lockstep, kèm tỉ lệ hoàn thành chuỗi.

NÓI LẠI THẬT ĐƠN GIẢN

Hai người cùng chỉ đường cho bạn trong một chuyến lái xe dài. Người thứ nhất chỉ nhìn đồng hồ và cảm giác quán tính. Anh ta rất nhất quán trong từng phút, nhưng sau nửa giờ thì lệch hẳn mà không tự biết. Người thứ hai nhìn biển báo bên đường. Anh ta không lệch dân, nhưng thỉnh thoảng biển bị che hoặc nhòe và anh ta đọc bừa.

Câu hỏi đúng không phải ai nói đúng. Câu hỏi là bạn nên nghe mỗi người bao nhiêu phần. Và đây là chỗ mọi người làm sai. Họ lấy con số ghi trong sách hướng dẫn của cái đồng hồ, đo trong phòng lạnh, đứng yên, để quyết định mức tin cho một cái đồng hồ đang rung trên xe nóng.

Cái mà máy học thêm vào đây không phải một người chỉ đường thứ ba giỏi hơn. Nó là một người biết tự nói mình đang chắc bao nhiêu, thay vì luôn nói bằng một giọng đều. Nó nhìn vào chính dữ liệu và nói: đoạn này tôi chắc, đoạn này tôi đoán.

Cái giá thì rõ và không né được. Người đó chỉ biết mình chắc bao nhiêu trên những con đường đã từng đi. Ra một con đường lạ, anh ta vẫn nói “tôi chắc” bằng đúng giọng cũ, và lúc đó anh ta nguy hiểm hơn cả người luôn nói bằng giọng đều. Vì vậy việc đầu tiên phải làm không phải thuê thêm người, mà là kiểm xem hai người sẵn có đang nói quá hay nói dè. Điều đó đo được: đếm xem họ sai bao nhiêu so với mức họ tự nhận.

Chỗ không rút gọn được: chuyện mức tin cậy là một cái bảng chứ không phải một con số. Một điểm ảnh bị nhòe theo một chiều vẫn rất chính xác theo chiều vuông góc. Mọi cách nói gọn thành “điểm này đáng tin bao nhiêu phần trăm” đều đánh mất chính cái chiều đó, mà đó lại là phần dùng được nhiều nhất.

Một câu: *Đừng hỏi phép đo nào đúng; hãy hỏi mỗi phép đo đáng tin bao nhiêu, rồi kiểm câu trả lời đó bằng số.*

8.9 Kết nối

[F/27-dat-hoc-sau-vao-dau](#) là tiền đề trực tiếp: bốn công ở đó chính là bộ lọc mà mọi kết luận “có đáng không” trong chương này đi qua. [F/28-dac-trung-va-ghep-cap](#) là mặt bổ sung: chương 28 làm cho *phép đo* bền hơn, chương này làm cho *trọng số* của phép đo trung thực hơn, và hai việc đó cộng chứ không thay nhau. [F/31-backend-kha-vi-va-dau-cuoi](#) nối vào phần hiệp phương sai học được từ phía ngược lại: nó hỏi điều gì xảy ra khi chính bộ tối ưu trở nên khả vi, còn ở đây bộ tối ưu vẫn giữ nguyên.

Ba chương ngoài Phần F cũng bị chương này gọi tới liên tục. E/24-thi-nghiem-va-ablation là điều kiện để mọi con số trong chương có nghĩa: không đo sần nhiều trước thì bảng NEES cũng chỉ là một mẫu. D/21-inference-va-trien-khai chứa toàn bộ phần triển khai chung mà mục 5 cố ý không lặp lại. Và E/26-lo-trinh-hoc là chỗ mục 6 nối vào: lộ trình D dừng ở ranh giới Phần F, còn mục 6 nối tiếp đọc tám chương này theo thứ tự nào.

Cuối cùng, một liên kết ngược lên đầu cây. Ý trung tâm của chương — rằng cái thiếu không phải ước lượng tốt hơn mà là ước lượng độ tin cậy tốt hơn — chính là trục đánh đổi giữa giả định và dữ liệu của B/04 xuất hiện dưới một hình dạng cụ thể. Một hằng số datasheet là một giả định rất mạnh và rất rẻ. Một hiệp phương sai học được là dữ liệu thay cho giả định, đắt hơn nhiều, và hồng đúng ở chỗ mọi thứ dựa vào dữ liệu đều hồng; ngoài phân phối.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chỉ tỉ số giữa các ma trận thông tin mới đổi nghiệm, và điều đó nói gì về việc bạn có sửa được trọng số IMU mà không đụng tới trọng số ảnh không?
2. Sai số vị trí do độ lệch con quay tăng theo t^3 chứ không phải t^2 : chuỗi nhân quả nào tạo ra bậc ba đó?
3. Nếu số Allan là một phép đo đúng, vì sao dùng thẳng nó làm Σ trong đồ thị nhân tử lại sai một cách có hệ thống, và sai theo hướng nào?

4. NEES của nhân tử ảnh lớn hơn hẳn bậc tự do: hai cách giải thích khác nhau là gì, và phép thử nào phân biệt chúng?
5. Trong hàm mất mát độ hợp lý âm, chuyện gì xảy ra nếu bỏ số hạng $\log \sigma^2$, và vì sao mạng sẽ khai bất định vô hạn?
6. Vì sao một hiệp phương sai học được chạy được ở tần suất thấp hơn tốc độ khung, trong khi một tư thế thì không?
7. Hai kỹ sư báo cùng một con số APE trên cùng một chuỗi, nhưng một người chạy lockstep còn người kia phát lại theo thời gian thật: hai con số đó khác nhau ở chỗ nào, và bạn hỏi thêm gì để biết?
8. Vì sao nói Σ là cách sai để xử lý một điểm khớp sai ghép, và ranh giới nào tách chuyện trọng số khỏi chuyện loại bỏ?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., "Image Matching Across Wide Baselines: From Paper to Practice," *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., "RoMa: Robust Dense Feature Matching," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., "Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., "HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, "Closed-form solution of absolute orientation using unit quaternions," *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., "Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, "Rethinking Visual Geo-localization for Large-Scale Applications," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, "MixVPR: Feature Mixing for Visual Place Recognition," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., "EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, "Optimal Transport Aggregation for Visual Place Recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features Without Supervision," *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., "AnyLoc: Towards Universal Visual Place Recognition," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [3] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [4] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [5] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] C. Forster, L. Carlone, F. Dellaert, et al., “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [9] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [11] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [13] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [15] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [16] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.

- [18] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] W. Liu, D. Caruso, E. Ilg, et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [21] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [22] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [23] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [24] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [25] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [26] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [27] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [29] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [30] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [31] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [32] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [33] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [34] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [35] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [36] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [37] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.

- [38] Y. Jin, D. Mishkin, A. Mishchuk, et al., "Image Matching Across Wide Baselines: From Paper to Practice," *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [39] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [40] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., "RoMa: Robust Dense Feature Matching," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [41] Y. Wang, X. He, S. Peng, et al., "Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [42] V. Balntas, K. Lenc, A. Vedaldi, et al., "HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [43] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [44] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [45] B. K. P. Horn, "Closed-form solution of absolute orientation using unit quaternions," *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [46] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [47] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [48] S. Hausler, S. Garg, M. Xu, et al., "Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [49] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [50] G. Berton, C. Masone, and B. Caputo, "Rethinking Visual Geo-localization for Large-Scale Applications," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [51] A. Ali-bey, B. Chaib-draa, and P. Giguère, "MixVPR: Feature Mixing for Visual Place Recognition," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [52] G. Berton, G. Trivigno, B. Caputo, et al., "EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [53] S. Izquierdo and J. Civera, "Optimal Transport Aggregation for Visual Place Recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [54] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features Without Supervision," *TMLR*, 2024.
- [55] N. Keetha, A. Mishra, J. Karhade, et al., "AnyLoc: Towards Universal Visual Place Recognition," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.

- [56] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [57] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [58] E. Brachmann, A. Krull, S. Nowozin, et al., "DSAC — Differentiable RANSAC for Camera Localization," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [59] E. Brachmann and C. Rother, "Learning Less is More — 6D Camera Localization via 3D Surface Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [60] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [61] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [62] E. Brachmann, J. Wynn, S. Chen, et al., "Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer," in *European Conference on Computer Vision (ECCV)*, 2024.
- [63] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.
- [64] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, 2010.
- [65] O. Chum and J. Matas, "Matching with PROSAC — Progressive Sample Consensus," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [66] O. Chum, J. Matas, and J. Kittler, "Locally Optimized RANSAC," in *DAGM Symposium on Pattern Recognition*, 2003.
- [67] D. Barath, J. Noskova, M. Ivashechkin, et al., "MAGSAC++, a Fast, Reliable and Accurate Robust Estimator," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [68] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [69] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [70] J. Sun, Z. Shen, Y. Wang, et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [71] J. Edstedt, Q. Sun, G. Bökman, et al., "RoMa: Robust Dense Feature Matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [72] S. Lowry, N. Sünderhauf, P. Newman, et al., "Visual Place Recognition: A Survey," *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [73] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [74] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.

- [75] E. Olson and P. Agarwal, "Inference on Networks of Mixtures for Robust Robot Mapping," *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [76] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.
- [77] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [78] C. Godard, O. Mac Aodha, M. Firman, et al., "Digging Into Self-Supervised Monocular Depth Estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [79] R. Ranftl, K. Lasinger, D. Hafner, et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [80] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [81] S. F. Bhat, R. Birkel, D. Wofk, et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv preprint arXiv:2302.12288*, 2023.
- [82] W. Yin, C. Zhang, H. Chen, et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [83] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [84] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [85] B. Ke, A. Obukhov, S. Huang, et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [86] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," in *ICCV*, 2015.
- [87] E. Ilg, N. Mayer, T. Saikia, et al., "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [88] D. Sun, X. Yang, M.-Y. Liu, et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," in *CVPR*, 2018.
- [89] Z. Huang, X. Shi, C. Zhang, et al., "FlowFormer: A Transformer Architecture for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2022.
- [90] Y. Wang, L. Lipson, and J. Deng, "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2024.
- [91] Z. Teed and J. Deng, "RAFT-3D: Scene Flow using Rigid-Motion Embeddings," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [92] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," in *European Conference on Computer Vision (ECCV)*, 2024.
- [93] J. Wang, M. Chen, N. Karaev, et al., "VGGT: Visual Geometry Grounded Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [94] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold," *arXiv preprint arXiv:2505.12549*, 2025.

- [95] B. Amos and J. Z. Kolter, "OptNet: Differentiable Optimization as a Layer in Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.
- [96] A. Agrawal, B. Amos, S. Barratt, et al., "Differentiable Convex Optimization Layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [97] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [98] Z. Teed and J. Deng, "Tangent Space Backpropagation for 3D Transformation Groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [99] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [100] L. Pineda, T. Fan, M. Monge, et al., "Theseus: A Library for Differentiable Nonlinear Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [101] K. M. Jatavallabhula, G. Iyer, and L. Paull, "gradSLAM: Dense SLAM meets Automatic Differentiation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [102] C. Tang and P. Tan, "BA-Net: Dense Bundle Adjustment Networks," in *International Conference on Learning Representations (ICLR)*, 2019.
- [103] W. Wang, D. Zhu, X. Wang, et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [104] Z. Teed, L. Lipson, and J. Deng, "Deep Patch Visual Odometry," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [105] L. Lipson, Z. Teed, and J. Deng, "Deep Patch Visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [106] Y. Qiu, Y. Chen, Z. Zhang, et al., "MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [107] T. Zhou, M. Brown, N. Snavely, et al., "Unsupervised Learning of Depth and Ego-Motion from Video," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [108] R. Li, S. Wang, Z. Long, et al., "UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [109] S. Wang, R. Clark, H. Wen, et al., "DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [110] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [111] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [112] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [113] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [114] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [115] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [116] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [117] J. Czarowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [118] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [119] H. Matsuki, R. Scona, J. Czarowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [120] R. A. Newcombe, S. Izadi, O. Hilliges, et al., "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011.
- [121] A. Hornung, K. M. Wurm, M. Bennewitz, et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [122] E. Sucar, S. Liu, J. Ortiz, et al., "iMAP: Implicit Mapping and Positioning in Real-Time," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [123] Z. Zhu, S. Peng, V. Larsson, et al., "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [124] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [125] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [126] E. Sandström, Y. Li, L. Van Gool, et al., "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [127] Y. Zhang, F. Tosi, S. Mattoccia, et al., "GO-SLAM: Global Optimization for Consistent 3D Instant Reconstruction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [128] C.-H. Lin, W.-C. Ma, A. Torralba, et al., "BARF: Bundle-Adjusting Neural Radiance Fields," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [129] A. Rosinol, J. J. Leonard, and L. Carlone, "NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields," *arXiv preprint arXiv:2210.13641*, 2022.
- [130] Z. Zhu, S. Peng, V. Larsson, et al., "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.
- [131] X. Kong, S. Liu, M. Taher, et al., "vMAP: Vectorised Object Mapping for Neural Field SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [132] H. Matsuki, R. Murai, P. H. J. Kelly, et al., "Gaussian Splatting SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [133] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.
- [134] H. Huang, L. Li, H. Cheng, et al., "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular, Stereo, and RGB-D Cameras," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [135] Z. Peng, T. Shao, Y. Liu, et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting,” in *ACM SIGGRAPH Conference Papers*, 2024.
- [136] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [137] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [138] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [139] J. Straub, T. Whelan, L. Ma, et al., “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.
- [140] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [141] A. Dai, A. X. Chang, M. Savva, et al., “ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [142] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [143] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [144] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [145] Z. Ge, S. Liu, F. Wang, et al., “YOLOX: Exceeding YOLO Series in 2021,” *arXiv preprint arXiv:2107.08430*, 2021.
- [146] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [147] A. Wang, H. Chen, L. Liu, et al., “YOLOv10: Real-Time End-to-End Object Detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [148] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [149] Y. Zhao, W. Lv, S. Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [150] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [151] N. Ravi, V. Gabeur, Y.-T. Hu, et al., “SAM 2: Segment Anything in Images and Videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [152] S. Liu, Z. Zeng, T. Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [153] M. E. A. Boudjoghra, A. Dai, J. Lahoud, et al., “Open-YOLO 3D: Towards Fast and Accurate Open-Vocabulary 3D Instance Segmentation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [154] C. Yu, Z. Liu, X.-J. Liu, et al., “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

- [155] M. Rünz, M. Buffier, and L. Agapito, “MaskFusion: Real-Time Recognition, Tracking and Reconstruction of Multiple Moving Objects,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2018.
- [156] B. Bescos, C. Campos, J. D. Tardós, et al., “DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5191–5198, 2021.
- [157] L. Schmid, M. Abate, Y. Chang, et al., “Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [158] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [159] A. Rosinol, A. Gupta, M. Abate, et al., “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” in *Robotics: Science and Systems (RSS)*, 2020.
- [160] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization,” in *Robotics: Science and Systems (RSS)*, 2022.
- [161] J. McCormac, A. Handa, A. Davison, et al., “SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [162] Y. Chang, N. Hughes, A. Ray, et al., “Hydra-Multi: Collaborative Online Construction of 3D Scene Graphs with Multi-Robot Teams,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [163] D. Maggio, Y. Chang, N. Hughes, et al., “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” *IEEE Robotics and Automation Letters*, 2024.
- [164] Q. Gu, A. Kuwajerwala, S. Morin, et al., “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [165] J. Straub, D. DeTone, T. Shen, et al., “EFM3D: A Benchmark for Measuring Progress Towards 3D Egocentric Foundation Models,” *arXiv preprint arXiv:2406.10224*, 2024.
- [166] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, et al., “ConceptFusion: Open-set Multimodal 3D Mapping,” in *Robotics: Science and Systems (RSS)*, 2023.
- [167] J. Kerr, C. M. Kim, K. Goldberg, et al., “LERF: Language Embedded Radiance Fields,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [168] S. Peng, K. Genova, C. Jiang, et al., “OpenScene: 3D Scene Understanding with Open Vocabularies,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [169] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and Modeling of Inertial Sensors Using Allan Variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [170] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [171] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [172] D. A. Nix and A. S. Weigend, “Estimating the Mean and Variance of the Target Probability Distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, 1994.

- [173] A. Kendall and Y. Gal, “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [174] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [175] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.
- [176] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.
- [177] S. Herath, H. Yan, and Y. Furukawa, “RoNIN: Robust Neural Inertial Navigation in the Wild — Benchmark, Evaluations, and New Methods,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [178] S. Sun, D. Melamed, and K. Kitani, “IDOL: Inertial Deep Orientation-Estimation and Localization,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [179] G. Cioffi, L. Bauersfeld, and D. Scaramuzza, “Learned Inertial Odometry for Autonomous Drone Racing,” *IEEE Robotics and Automation Letters*, 2023.
- [180] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [181] P. Wang, L. Zhao, R. Ma, et al., “BAD-NeRF: Bundle Adjusted Deblur Neural Radiance Fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [182] L. Zhao, P. Wang, and P. Liu, “BAD-Gaussians: Bundle Adjusted Deblur Gaussian Splatting,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [183] L. Ma, X. Li, J. Liao, et al., “Deblur-NeRF: Neural Radiance Fields from Blurry Images,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [184] G. Gallego, T. Delbrück, G. Orchard, et al., “Event-based Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 154–180, 2022.
- [185] A. R. Vidal, H. Rebecq, T. Horstschaefer, et al., “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, 2018.
- [186] L. Pan, C. Scheerlinck, X. Yu, et al., “Bringing a Blurry Frame Alive at High Frame-Rate with an Event Camera,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.